

FILE: www.diracprogram.org/doc/release-26/_sources/development/adding_moving_removing_sources.md

orphan

How to add/move/remove sources

`git grep` for a source file “close to” the one you want to introduce. Then you will see where it is to be listed.

How to move/remove sources

`git grep` for the source file name. Then you will see where it is listed and you can move/remove it there.

FILE: www.diracprogram.org/doc/release-26/_sources/development/beta_testing.md

orphan

Checklist for beta-testers

Verify that the release installs correctly

Check out the release branch:

```
$ git checkout release-25 # adapt name of branch
```

Try to configure and build the code:

```
$ ./setup [--fc=ifx --cc=icx --cxx=icpx]
```

Please report problems. Pay attention whether math libraries are correctly detected.

Run the test set

Of course one should test the code regularly: to check for errors in your new implementations that may break functionality in other parts of the code, and to check that modifications that you pulled from the main repository work on your machine. The latter tests should typically not reveal problems, because the code is daily and nightly tested on different architectures, but one is of course never sure.

The easiest way to test is to type `ctest` inside the build directory. This will run a standard test set (without additional `ctest` parameters) and check for errors. You may also run a subset by specifying regular expressions: e.g. :

```
$ ctest -R dft
```

where the “-R” means regular expression, so this will run all tests that contain the substring “dft”.

Since tests are provided with descriptive tags (labels), you can run group of tests based on label. For example, this command selects only short tests from the whole suite :

```
$ ctest -L short
```

Good approach is to run the (selected) test suite with the full CDash report, so that other developers can see the actual status (before accepting code change): :

```
$ ctest -jN (-L short) -D ExperimentalUpdate -D ExperimentalConfigure -D ExperimentalBuild -D ExperimentalTest -D ExperimentalSubmit
```

You can also choose to run the test set, for instance, using the extracted tarball (see further up), and with simpler command :

```
$ ./setup [--flags]
$ cd build
$ ctest -jN -D Experimental
```

The test results also appears automatically on the [DIRAC Dashboard](#). And we can all see which tests fail and why.

Verify the author list

Verify the author list below the logo in the output. Check versions. Check typos.

Run tutorials from the website

Verify that they actually run.

Verify the manual and other documentation

Correct typos and mark sections that are wrong or not clear with a warning:

```
.. warning::
```

```
    Something wrong here! (replace this text)
```

This will create a red box with the warning inside.

FILE: www.diracprogram.org/doc/release-26/_sources/development/cdash.md

orphan

Nightly testing dashboard

Please contribute to the [DIRAC Dashboard](http://www.diracprogram.org/doc/release-26/_sources/development/cdash.md). All you need is a desktop which is idle and lonely over night or a cluster with some free CPU hours, a deploy key, and a script that you launch via crontab (desktop) or that you submit to the batch system (cluster). Below we show you how to do that.

Creating a deploy key

A deploy key is a single-purpose ssh-key without a passphrase which is not attached to your repository user account and which only can read (clone) and not write (push).

We create one with the following command:

```
$ ssh-keygen -t rsa -b 4096
```

We need to specify its name (for instance “/home/user/.ssh/nightly_rsa”):

```
Enter file in which to save the key (/home/user/.ssh/id_rsa): /home/user/.ssh/nightly_rsa
```

And we use an empty passphrase (we just hit enter twice):

```
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

And we are done with the key:

```
Your identification has been saved in /home/user/.ssh/nightly_rsa.
Your public key has been saved in /home/user/.ssh/nightly_rsa.pub.
```

Uploading the public key to GitLab

Now upload the public key to gitlab. If are not sure how to do this, please consult the documentation at <https://docs.gitlab.com/ee/ssh/>.

Writing a CDash script

A CDash script can in principle be as brief as this:

```
$ ./setup [--flags]
$ cd build
$ ctest -jN -D Nightly --track master
```

The result of appears automatically on the [DIRAC Dashboard](http://www.diracprogram.org/doc/release-26/_sources/development/cdash.md).

However, typically we need a little bit more because we want it to clone the code in an unsupervised fashion or because we want to also test the tarball creation. This is the script that Radovan uses: https://gitlab.com/dirac/dirac-private/blob/master/maintenance/cdash/cdash_crontab.sh This script takes care of using a specific deploy key and integrates nicely with the crontab example given below.

Launching your script automatically via crontab

Edit your crontab with:

```
$ crontab -e
```

Here is an example:

```
# m h dom mon dow command
00 04 * * * /home/user/nightly/cdash_crontab.sh -b 'master' -j 4 -c 'Intel' -n 'Intel-Mac' -t 'master' -k /home/user/.ssh/nightly_rs
55 04 * * * killall -9 dirac.x
```

You can verify your crontab file changes by :

```
$ crontab -l
```

You can also verify that a crontab file with your username exists by:

```
$ sudo ls -l /var/spool/cron/crontabs
```

CDash tracks

We currently have the following tracks installed: master, release-14, release-15, Nightly, and Experimental. It is no problem to install additional tracks - just ask Radovan. The tracks are there to organize the various builds. So instead of submitting a build to the track “Nightly” with the build name “master-foo” it is clearer and more concise to submit a build “foo” to the track “master”. We recommend to use “master” or “release-15” depending on the context and not to use “Nightly” or “Experimental”. But it is OK to use “Nightly” or “Experimental” to calibrate your scripts.

The semantic difference between “Nightly” and “Experimental” is that “Nightly” corresponds to a Git hash closest to a certain configurable time whereas “Experimental” corresponds to “now”. This means that for extremely actively developed projects it may be important to test all nightly tests using a well defined version and then one should prefer “Nightly” over “Experimental”.

FILE: www.diracprogram.org/doc/release-26/_sources/development/checkpoint.md

orphan

The CHECKPOINT.h5 file

What is this file ?

- It is an hdf5 file containing the essential data given as input to and generated by DIRAC.
- All data stored in this file is defined and documented in the DIRAC data schema, the source of which is found in `utils/DIRACschema.txt`.

What can I do with it ?

- One purpose is to make restarting and data curation after a run easier.
- Another purpose is to facilitate communication with other programs.
- With the hdf5 format and h5py it is trivial to import data into Python and further process and view it.

Can I also extend the schema as I have data that is not listed here ?

- The first question is whether this data is indeed essential. The CHECKPOINT.h5 file is not intended for large data sets as it gets saved automatically after a run. It is also not intended for highly specialized or intermediate data. If you want to use hdf5 for such data consider making a special hdf5 file using the interface provided in the `labeled_storage` module. This is even easier as you need not define everything as thoroughly as with the schema (see below).
- In case a new data type is indeed a generally useful addition, please start by documenting it (type and description) and ask for a peer review by one of the developers before proceeding to the next step.

How the schema is processed.

- The source text in `utils/DIRACschema.txt` is processed at run time by the python functions `read_schema` and `write_schema` that are found in `utils/process_schema.py` and which are called by the DIRAC run script `pam`. This produces a new text file called `schema_labels.txt` which is placed in the work directory. This is the file used by the actual DIRAC code and contains the set of labels also found on CHECKPOINT.h5. To familiarize yourself with this: copy `schema_labels.txt` from the work directory and compare it to `DIRACschema.txt`.
- Note that the hierarchical structure is defined by `/s`, much like you see in a Unix directory. This also means that one can not use `/s` in data labels as hdf5 would get confused.
- In the Fortran code the generated labels are used directly, an example is found in `gp/dircmo.F90`: `call checkpoint_write ('/result/wavefunctions/scf/energy', rdata=toterg)` which writes the total energy (a single real number) with the appropriate label.
- Note that data is classified as optional or required in the schema. This is used to define whether restart is possible, for this purpose all required data should be present on the checkpoint file.

How can the schema be extended ?

- For extending the schema: Do NOT edit the `schema_labels.txt` file. All edits should be made in `DIRACschema.txt`.
- Check first whether the data is optional or required. Be careful to define new required data as restart files will be invalid if this data is missing and this may hamper restarting from old checkpoint files.
- If the data consists of a simple standard type (real, integer or string) which fits in an existing subsection you can simply define it at the appropriate place and the scripts will automatically generate the label. After inspecting this you can then use this in calls in the Fortran code.
- If the data is of composite type, you need to define its elements below. This is done by creating new subsection in the file, an example is the `molecule` data type that is part of input and is defined in a separate section. Each section starts with a `*` and ends with `*end`.
- You may also nest sections, see for instance the data type `wavefunctions` that has the composite type `scf` as an element.

What happens at run time and on the Fortran side ?

- At the start of the run DIRAC checks whether CHECKPOINT.h5 is present and contains all required data. If it is, a restart will be attempted. Note that you can use the copy facilities of `pam` to place the file in the work directory.
- During the run the only calls needed on the Fortran side are `checkpoint_read`, `checkpoint_write` and possibly `checkpoint_query`. These subroutines are found in the module `checkpoint` and support writing of reals, integers and strings. The query routine can be used to determine whether a data set already exists and what its size is. It is intended to keep the hdf5 interface simple and easily maintainable, so more complicated types should be split up in these standard types. Note that we have a layered structure with the `labeled_storage` module being responsible for the actual I/O, the `checkpoint` module is merely an additional layer that checks the schema. If you feel inclined to change the `checkpoint` module, make sure to not directly call hdf5 routines to not break the fallback option (see below) that is provided in case we hit a system or user that has no hdf5 installed. There are two more public routines in this module (for opening and closing a checkpoint file), but these are already called at the appropriate places in DIRAC and should not be called at other places.

- If the checkpoint_read routine is called data is located on the file and given back to the caller. Some error handling is provided, like checking whether the array given to hold the data is large enough, but crashes are still possible, for instance in case you try to read a file that is not in hdf5 format or is otherwise corrupted.
- If the checkpoint_write routine is called data is stored on file after checking that the label is indeed known in the schema. Undefined data will not be written and a warning is issued. This guarantees that all data placed on the checkpoint file is properly documented.
- At the end of the run pam checks whether CHECKPOINT.h5 is present and contains all required data. If it is, it will be copied to the directory from which pam was called and renamed following the same convention as used for the output file, but giving a file extension .h5 instead of .out

What happens if hdf5 is not installed ?

- At configure time cmake will detect whether DIRAC can be linked with the required hdf5 routines (via the interface contained in the module mh5 that you find in src/gp/hdf5_util). If this is not the case, a simple Unix mimic of hdf5 is activated by creating a directory CHECKPOINT.noh5 in which the hierarchical data structure is implemented with subdirectories containing Fortran unformatted files. A hidden file .initialized is used to determine whether a data group has been defined, the data itself is stored in a two-record file with the first record containing the data type, data size and data length (the latter being only different from 1 in case of string data). All read, write and query functionalities work, but moving data to and from the work directory is now more complicated as we need to move directories. This is currently implemented by tarring and gzipping the CHECKPOINT.noh5 directory. In case hdf5 can not be linked but h5py is found, the fallback format is converted into proper hdf5 format by the Python routines nohdf5_load_data and write_hdf5 called by the pam script at the end of the run.
- Note that use of the fallback option is strongly discouraged as it is less efficient and offers only limited functionality.

FILE: www.diracprogram.org/doc/release-26/_sources/development/code_review.md

orphan

Code review

Master and release branches are protected

This means that nobody can push to them directly, all contributions have to be submitted using merge requests (see below).

How to submit a merge request

- Before you do anything, create a new branch from the branch that you wish to modify.
- Implement your change and commit to the new branch.
- Push your new branch.
- Go to <https://gitlab.com/dirac/dirac-private> and click on the blue button “Create merge request” (top right).
- Describe the change unless it’s evident from the commit message.
- Check the box next to “Remove source branch when merge request is accepted”.
- Make sure it is directed to the right branch.
- Click green button “Submit merge request”.
- If you commit does not modify code execution you may use the command “git push -o ci.skip” to avoid invoking runners.

How to ask for a code review

- Either assign to somebody.
- Or mention somebody via their @username in the discussion. Then they get notified.
- Or let the merge request sit and somebody will pick it up.

How to do code review

- Be polite.
- Check that code is submitted to the right branch.
- Check that commit messages are clear.
- Changelog entry needed?
- Documentation needed?
- Tests preserved and new functionality tested?
- Wait until the test runner reports before merging or click “merge when pipeline succeeds”.
- If test runner result is red, investigate what happened.

What if you committed locally to master or a release branch?

Then you cannot push directly but do not worry and follow these steps:

- Create a new branch from the branch that you changed and tried to push.
- Push the newly created branch to <https://gitlab.com/dirac/dirac-private>.
- Create a merge request and point it to the branch you originally meant to modify.
- Finally clean up your local branch by rewinding to either origin/master or origin/release-N :

```
$ git checkout master
#WARNING! next step will delete commits that came after origin/master
$ git reset --hard origin/master
```

If you do not clean up your local master or release branch, it can start to diverge next time you pull since your local history may be different from remote history.

How to get changes from a release branch to master?

Visit https://gitlab.com/dirac/dirac-private/-/merge_requests/new, select the release branch as “Source branch”, verify that “Target branch” is the master branch, then click the green button “Compare branches and continue”.

Scroll down and have a look at the commits whether the changes to be merged are what you expected. Abort if this looks unexpected. You can also browse the “Changes” to see what is part of this merge request.

Then fill out the rest (title, description) like any other merge request. Note that you cannot and should not mark the source branch to be deleted upon merge and that’s good, since we want to keep the release branches forever (these branches are protected against deletion).

Finally green button: “Submit merge request”.

How to create a new release branch?

If you create a new release branch (e.g. “release-25” in the year 2025) locally using the terminal, you will experience that the repository will refuse to accept your `git push origin -u release-25`. This is because we protect master and `release-*` against (accidental) direct pushes.

But in this case it will be in your way. You (repository/group admins) can temporarily unprotect the matching wild-card pattern at GitLab (Settings -> Repository -> Protected branches).

Further reading

- <https://docs.gitlab.com/ee/gitlab-basics/add-merge-request.html>
- <https://about.gitlab.com/2017/03/17/demo-mastering-code-review-with-gitlab/>

FILE: www.diracprogram.org/doc/release-26/_sources/development/commit_policy.md

orphan

Commit policy after the release is out

Since DIRAC12 we have two novelties:

- We have put a patching/updating mechanism in place for patches and bugfixes. - We have a versioned documentation.

Both the patching and the documentation **will** break down if we do not follow the guidelines outlined here.

1. Commit patches and bugfixes (large or small) that relate to DIRAC14 to the release-14 branch and not to master.
2. Commit changes to the DIRAC14 documentation to the release-14 branch and not to master. In the days and weeks following the release we will modify the DIRAC14 documentation a lot. For this simply commit to release-14. The documentation is generated from that branch. This branch continues to live until we release the next version and discontinue support of DIRAC14.

To summarize the above two points: all commits that go to master will **never** make it to this documentation: <http://www.diracprogram.org/doc/release-14/> and will **never** be part of patches to DIRAC14 if you don’t carefully plan your commits, the patches **will** get lost (from the user’s point of view).

All commits that go to release-14 can be integrated to master. This means that all changes to release-14 will get integrated into the main line development. **Never** merge from master to release-14.

If, by accident, you committed to the wrong branch, you can `git cherry-pick` the commit to the release-14 branch.

3. Do not change the file VERSION. This file is changed when we release a new patch. But we will not release a patch from each commit to release-14

To summarize, ask yourself the following questions:

- Is this code or documentation change useful for DIRAC14 users? If yes, commit to release-14. - Is this code or documentation change only useful for future versions of DIRAC? If yes, commit to master or some other branch.

FILE: www.diracprogram.org/doc/release-26/_sources/development/contributing-changes.md

orphan

Contributing changes

If you have access to <https://gitlab.com/dirac/dirac-private/>

Please have a look at `multiple-remotes`.

Small changes

For small changes like fixing a typo or fixing small mistakes or relatively small feature improvements you can fork <https://gitlab.com/dirac/dirac/>, create a new branch on your fork, and directly send a merge request towards the master branch of <https://gitlab.com/dirac/dirac/>.

Larger changes

For larger changes, anything that would take more than a day of work, we recommend to first open an issue at <https://gitlab.com/dirac/dirac/-/issues> and to describe your idea or your suggestion, before actually spending days or weeks or months programming it. It can be very useful to collect feedback first and let others know. What you have in mind might already be in the code or already in the works by somebody else, or there may be a better way to do it.

Then after you collect feedback, the issue can stay open and you can later reference it in the merge request weeks or months later and the person reviewing the merge request will have a context and it will make the review process easier.

To submit your changes, fork <https://gitlab.com/dirac/dirac/>, create a new branch, then later submit merge request towards the master branch of <https://gitlab.com/dirac/dirac/> and in your commit and/or merge request please reference the issue where the proposal was discussed.

Another medium to collect feedback before doing a lot of coding is to write to <https://groups.google.com/g/dirac-users>.

We are really looking forward to your code changes and improvements!

License terms

The DIRAC code distributed under the LGPL v2.1 and also all contributions to the code are understood to be provided under this license with a copyright shared among the DIRAC code authors. If this is not the case and the submitted code is provided under a different license and requires a different copyright notice, please point this out in the merge request.

FILE: www.diracprogram.org/doc/release-26/_sources/development/debugging.md

orphan

Localizing bugs in DIRAC run using debugger

The DIRAC program system is continuously evolving. From time to time, however, it gets - unintentionally - spoiled with various programming bugs. Here we would like to share knowledge on how to localize crashes/bugs in the code.

We believe that this short tutorial is useful not only for DIRAC developers, but also for ordinary users, who likewise could send us precise localization and specification of the bug.

Introduction

Let us assume that we have serially compiled DIRAC code. Make sure that DIRAC is compiled with “-g” flag for all Fortran, C and C++ compilers. Anyway, this flag is the the default part in all combination of compiler flags.

If you intend to run many debugging runs, please choose `-type=Debug` in the setup flags. This means omitting optimization flags by the CMake buildup system and speed up of (repeating) DIRAC compilation. Note that some bugs (crashes) might “disappear” when the optimization is completely turned off, because not all compilers have bug-free optimization procedure.

Ensure that proper debugger, installed on your machine, is called with the “pam” script. For example, for the GNU compilers set this is the debugger “gdb”. Some commercial compilers are providing their own debuggers: Intel OneAPI includes the gdb-oneapi.

Debugging with the “pam” script

Launching DIRAC with the “pam -debug ...” is suitable for one-shoot debugging. Pam script calls the debugger, which afterwards shows up its command line. Afterwards the user can insert breakpoints, stops into the code and repeatedly run ‘dirac.x’ inside the debugger.

Stack printing

There is the possibility to print out the stack content.

Any ‘QUIT’ crash of the code causes a SegFault termination you can investigate with the debugger - you can ‘visit’ individual procedures within the stack sequence and examine their variables.

For example, beeing the “gdb” debugger session:

```
> run
(Dirac runs and crashes somewhere)
> backtrace
(show the stack)
> up {n}
(go to the n-the previous function in the call stack; see help up)
> print <VARIABLE>
(whatever you want to inspect).
```

Oposite way can be achieved using ‘down’ command.

Debugging in the plain mode with the GNU/gdb debugger

We give a simple demonstrative session of an thorough and effective debugging. Most recent version of the widely used the GNU/gdb debugger is recommended.

Make a working directory with enough disk space. Provide a link of the \$DIRAC/dirac.x executable file there. This is because you will repeat modifications of DIRAC source files and subsequent compilation, so it is easier to have linked executable rather than copying it.

Make sure that the 'DIRAC.INP', 'MOLECULE.INP', *gdb_demo* and other necessary files (for example, the 'CHECKPOINT.h5' if you are restarting SCF iterations) are in the working directory as well.

Demonstration of GNU/gdb debugging commands

This *gdb_demo* file (create© from this web-page) contains some commands (and macros) of the gdb debugger:

```
### load the executable file with all symbols
file dirac.x

### This settings is necessary for complex breakpoint conditions in fortran...
#set language fortran
set language c
set language auto

### Write where I am...
echo In directory:
shell echo pwd
echo \n Files in dir:
shell ls -a

#####
### Delete DIRAC files in the scratch directory
### - needed for restarting and for further catching of bugs..
#####
shell /bin/rm -r DF* AOPROPER BSSMAT
echo \n After deletion remaining files in dir:
shell ls -a

### clean the desk: delete all previous breakpoints
d b

#####
# Example of macros
#####
define my_macro
printf "in GMOTRA begin NZ=%d N2BBASXQ=%d SPINFR=%d\n",nz,n2bbasxq,spinfr
end

define my_macro1
printf "in GMOTRA begin SPINFR2=%d\n",spinfr2
end

### demo of a conditional breakpoint
b gmotra_ if (nz==1 | (n2bbasxq==0 && spinfr==0))
command
my_macro
my_macro1
echo type continue or sipmly c...\n
# c
end

### example of the unconditional breakpoint
b dirone.F:1399
command
if ( (nfsym.eq.1 && n2bbasxq.gt.10) | nz.eq.1 | spinfr.eq.0)
printf "NFSYM=%d\n", NFSYM
if (nfsym.eq.2)
printf "NTM0(1)=%d NTM0(2)=%d\n",NTM0(1),NTM0(2)
else
printf "One symmetry...NTM0(1)=%d \n",NTM0(1)
end
else
printf "... else branch...."
end

##### while cycle demo #####
printf "Few values of the WORK(KTMAT) array, KTMAT=%d... \n",KTMAT
set $indx=0
while ($indx.lt.5)
printf "WORK(%d)=%lf\n",ktmat+$indx,(*WORK)(ktmat+$indx)
# p ktmat+$indx
set $indx=$indx+1
end
end
```

In the working space type *gdb dirac.x* to run the *dirac.x* executable in the debugging mode. When the debugger's command line appears, type *source gdb_demo* to load the debugger source file.

Each time when you modify and recompile DIRAC please retype again *file dirac.x* in the gdb mode to reload the fresh executable.

If you modify the *gdb_demo* debugger source file apply again the gdb command *source gdb_demo* to reload it.

More GNU/gdb know-how is given at [this web-page](#).

Other debuggers

To debug the *dirac.x* program compiled with the Intel suite we recommended to employ own Intel debugger, *idb*, which is works with the same set of commands as the GNU/gdb counterpart. The same holds for Portland compilers. Combining different compilers with different debuggers does not work well.

For parallel debugging we recommend special (commercial) software, which is able to follow individual threads, like the *totalview debugger*. Otherwise in a multithreaded *dirac.x* run each thread can be caught and debugged with any serial debugger.

FILE: www.diracprogram.org/doc/release-26/_sources/development/dfcoef.md

orphan

NOTE: *DFCOEF* file was replaced by *CHECKPOINT.h5* in DIRAC22, and the information below is only relevant for older versions of DIRAC.

DFCOEF

The binary *DFCOEF* file serves for storing molecular orbitals and related information.

In the Fortran language it can be written like

```
CHARACTER TEXT*74
DIMENSION CMO(NCMOTQ),EIG(NORBT),IBEIG(NORBT),IDIM(3,2)
DOUBLE PRECISION TOTERG

DO I = 1,NFSYM
  IDIM(1,I) = NPSH(I)
  IDIM(2,I) = NESH(I)
  IDIM(3,I) = NFBAS(I,0)
ENDDO
WRITE(IUNIT) TEXT,NFSYM,((IDIM(I,J),I = 1,3),J=1,NFSYM),TOTERG
```

Let *i* be the integer size (4 or 8) and *d* the double precision size (8). Size of the stored data, *Size*, given in first section above, is $74+i(1+3*NFSYM)+d*$.

i	NFSYM	Size
4	1	$74+4*4+8 = 98$
4	2	$74+4*7+8 = 110$
8	1	$74+8*4+8 = 114$
8	2	$= 138$

After calculating the *Size* we can store remaining data:

- molecular orbitals coefficients, CMO
- SCF eigenvalues, EIG
- specification of the irreducible representation per orbital, IBEIG

```
WRITE(IUNIT) CMO      Size d*NCMOTQ
WRITE(IUNIT) EIG      Size d*NORBT
WRITE(IUNIT) IBEIG     Size i*NORBT
```

FILE: www.diracprogram.org/doc/release-26/_sources/development/diag.md

orphan

Problems with the diagonalization

Diagonalizations are important part of (relativistic) quantum chemistry methods.

In the DIRAC program suite, there is the diagonalization of the Lowdin matrix, then of the *J_z* operator matrix in the case of linear symmetry and, finally, of the Fock MO matrix.

In DIRAC, there are few routines to accomplish these tasks:

- non-default lapack' routine DSYEVR (for real symmetric matrixes only)
- eispack's routine RS (for real symmetric matrixes only)
- own DIRAC routine JACOBI (for real symmetric matrixes only)
- own DIRAC routine QJACOBI (for quaternion Hermitian matrixes)
- and own DIRAC routine based on Householder diagonalization (for quaternion Hermitian matrixes)

For certain cases one has to carefully choose the diagonalization method, depending either on the quality (accuracy) of obtained results - eigenvalues and eigenvectors - or on the time performance.

Note that DSYEVR is not the default diagonalization routine in DIRAC, but can be activated via *Jsetup -D LAPACK_QDIAG=1 ...*

Problem case - DSYEVR

The lapack's routine for diagonalization, DSYEVR, which is fast enough, is giving - for certain input matrices - numerically unstable eigenvectors. We discovered it in the linear symmetry.

The \$O_2\$ test system

The real test system, where numerical discrepancies with active DSYEVR are observed, is the \$O_2\$ molecule of the *fsc_highspin* test: :


```
pam --noarch --inp=hf.inp --mol=O2.mol
```

(Input files for download are from the DIRAC test directory, namely hf.inp <../../test/fsc_highspin/hf.inp> and O2.mol <../../test/fsc_highspin/O2.mol>)

The proper Total SCF energy of this system is, :

```
-149.68666145184687
```

while the DSYEVR routine employed in the LINSYM procedure gives different value by about 10^{-7} , like (obtained by Miro) :

```
-149.68666198782267
```

together with some positron states intruding.

Molecular symmetries lower than linear symmetry (D2h, C2v ...) give proper and identical SCF energies.

The smallest test system

By tracing the error to the smallest system where error is clearly showing up we came to the artificial X2 one-electron linear system in the smallest 1s1p1d basis :

```
pam --noarch --inp=dc.1el.2fs.inp --mol=X2.1s1p1d.asd.mol --get "Jz_SS_matrix.fermirp2-2"
pam --noarch --inp=dc.1el.2fs.inp --mol=X2.1s1p1d.D2h.mol
```

(Input files for download are dc.1el.2fs.inp <../../test/fsc_highspin/dc.1el.2fs.inp>, X2.1s1p1d.asd.mol <../../test/fsc_highspin/X2.1s1p1d.asd.mol> and X2.1s1p1d.D2h.mol <../../test/fsc_highspin/X2.1s1p1d.D2h.mol>.)

The affected value is the “Inactive energy (Output from ONEERG)” in the output :

```
-15622.826231066265 (wrong value, linear symmetry)
-15622.826368346343 (correct value, linear symmetry)
-15622.826368346305 (correct value, D2h symmetry)
```

We see differences in energies by about 10^{-4} a.u. what is sufficient for detecting an error.

The DIRAC demonstration program

By tracking down the source of the problem we get to the 9x9 matrix of the X2.1s1p1d test system above, *Jz_SS_matrix.fermirp2-2*, for the diagonalization testing, which is saved in the LINSYM routine.

In the linear symmetry eigenvectors obtained with the lapack’s DSYEVR routine happen to be numerically inaccurate. This can be verified with the abovementioned matrix.

With the standalone DIRAC-dependent diagonalization testing program, *diag.x* (its source code is in the utils directory, together with the text input file DIAGONALIZE_TESTS.INP <../../test/diagonalization/DIAGONALIZE_TESTS.INP>), we found discrepancies in numerical accuracy of eigenvectors (U) obtained with the DSYEVR routine in comparison to other diagonalization methods. Eigenvalues (eps) are unaffected. A is the original matrix for diagonalization, I is the unit matrix. The norm is defined as sum of absolute values of elements divided by total number of elements. We split matrix elements into diagonal and off-diagonal.

Values around 10^{-6} - 10^{-5} clearly indicate deviation from the zero threshold for which one would expect numbers cca 10^{-15} - 10^{-14} .

Eigenvalues and eigenvectors

By using the testing matrix, *Jz_SS_matrix.fermirp2-2*, we get properly degenerate eigenvalues with DSYEVR and with other diagonalization methods:

```
1 -1.500000000000001
2 -1.500000000000000
3 -1.500000000000000
4 0.499999999999999
5 0.500000000000000
6 0.500000000000000
7 0.500000000000000
8 0.500000000000002
9 2.500000000000000
```

Now we can proceed to checking the numerical accuracy of obtained eigenvectors.

Intel+MKL-i8, DSYEVR:

```
U^{+}*A*U - eps = 0 > norm/diag:0.1684D-14 norm/offdiag:0.6623D-06
U^{+}*U - I = 0 > norm/diag:0.2097D-15 norm/offdiag:0.1325D-05
U*U^{+} - I = 0 > norm/diag:0.6293D-05 norm/offdiag:0.4174D-05
```

GNU+ownmath-i8, DSYEVR:

```
U^{+}*A*U - eps = 0 > norm/diag:0.1739D-14 norm/offdiag:0.5091D-06
U^{+}*U - I = 0 > norm/diag:0.2097D-15 norm/offdiag:0.1018D-05
U*U^{+} - I = 0 > norm/diag:0.5239D-05 norm/offdiag:0.3407D-05
```

Intel+MKL-i8, eispack RS:

```
U^{+}*A*U - eps = 0 > norm/diag:0.5058D-15 norm/offdiag:0.9620D-16
U^{+}*U - I = 0 > norm/diag:0.6168D-15 norm/offdiag:0.1400D-15
U*U^{+} - I = 0 > norm/diag:0.5674D-15 norm/offdiag:0.1586D-15
```

Independent demonstration program

For public purposes we are offering the standalone and DIRAC independent demonstration program *dsyerv_check.F90* which does the same checks as the above mentioned (DIRAC dependent) *diag.x* program. (Source codes for downloading are `dsyerv_check.F90 <../../../../utils/dsyerv_check.F90>`, `eispack.F <../../../../src/epack/eispack.F>`, and the testing matrix is `Jz_SS_matrix.fermirp2-2 <../../../../test/diagonalization/Jz_SS_matrix.fermirp2-2>`.)

Results are as follows:

GNU+/usr64/lib, DSYEVR:

```
U^{+}*A*U - eps ?= 0> norm/diag:0.1783D-14 norm/offdiag:0.1419D-06
U^{+}*U - I ?= 0> norm/diag:0.2591D-15 norm/offdiag:0.2838D-06
U*U^{+} - I ?= 0> norm/diag:0.1047D-05 norm/offdiag:0.7950D-06
```

Intel+MKL, DSYEVR:

```
U^{+}*A*U - eps ?= 0> norm/diag:0.1684D-14 norm/offdiag:0.2746D-15
U^{+}*U - I ?= 0> norm/diag:0.2097D-15 norm/offdiag:0.6923D-16
U*U^{+} - I ?= 0> norm/diag:0.2467D-16 norm/offdiag:0.8681D-16
```

It is interesting to observe that obtained Intel+MKL results are in fact correct and different that those from the *diag.x* DIRAC based standalone program. This can be attributed to more complicated compiling and linking flags of the *diag.x* code in the complex DIRAC's cmake buildup apparatus.

Intel+lapack 3.5.0, DSYEVR:

```
U^{+}*A*U - eps ?= 0> norm/diag:0.1665D-14 norm/offdiag:0.9524D-07
U^{+}*U - I ?= 0> norm/diag:0.2097D-15 norm/offdiag:0.1905D-06
U*U^{+} - I ?= 0> norm/diag:0.7649D-06 norm/offdiag:0.5035D-06
```

The recent lapack 3.6.0 from netlib still suffers with non-orthogonal eigenvectors (see this [bug report](#)).

Solution of the problem

The simplest solution is to keep non-DSYEVR diagonalization routine in the LINSYM part to avoid possibility of highly degenerate values with numerically unstable eigenvectors. The eispack's RS routine, own DIRAC's Householder or Jacobi methods do fit for this purpose because they can properly handle eigenvectors of degenerate eigenvalues. The drawback is the time efficiency, especially for large systems.

Addendum

The dsyevr standalone testing program was placed on github, <https://github.com/miroi/lapack-dsyevr-test>.

FILE: www.diracprogram.org/doc/release-26/_sources/development/documentation_howto.md

orphan

How this documentation works and how you can contribute

This documentation is generated using [sphinx](#) which translates RST (reStructured Text) markup to html and latex. Sphinx and RST are extremely widely used in the Python community. The nice thing about RST markup that it looks nice by itself in the terminal. RST and Sphinx are very simple and intuitive and for first steps consult the [reStructuredText Primer](#) and the [Sphinx Markup Constructs](#). Typically RST markups is auto-colored in the terminal so you can see when the markup is correct or wrong.

Sphinx installation

We recommend to install all the required packages using either [virtualenv](#) or [Conda](#) but not by using standard installation packages. Possibly the only packages that you may need to install system-wide are python-pip and python-virtualenv).

Now you can set up a virtual environment (the last argument is its name/location and you can change it):

```
$ python -m venv venv
```

On some systems, instead of the above, you need:

```
$ python3 -m venv venv
```

This will create a folder called venv in your current working directory. You can create it in another place and you can give it a different name. It is a good practice to have separate environments for separate projects. Once you have set it up, you need to activate it:

```
$ source venv/bin/activate
```

Once activated, you can install Python packages into the virtual environment:

```
$ python -m pip install sphinx sphinx_rtd_theme sphinxcontrib-bibtex
```

If this sphinx installation does not work, please use its lower version:

```
$ python -m pip install "sphinx<7.0.0" sphinx_rtd_theme sphinxcontrib-bibtex
```

This restriction on the sphinx version is a workaround for <https://github.com/readthedocs/readthedocs.org/issues/10279>.

Again, on some systems you need to type `python3 -m pip ...` instead.

Next time you log on, you do not need to install it again, you just activate (source) it like above, you do not need to install the dependencies again.

The advantage of using virtualenv is that it is cross-platform, you get the latest packages, you get an isolated and disposable environment, and also you do not require sudo permissions.

Generation of html pages

The central documentation web pages are generated automatically from the RST files every hour. But you can also generate the web pages by yourself. For this install the sphinx-packages first (see above), then:

```
$ mkdir build
$ cd build
$ cmake ..
$ make -k html > build_html.log 2>&1
```

Now point your browser to `build/html/index.html`. We recommend to check the `build_html.log` file for possible errors and warnings in generating the documentation.

How to add new sections and text

Modify existing RST files or add new ones and reference them in `index.rst`.

New theme

The [sphinx_rtd_theme](#) has been customized and integrated as the new theme for Dirac. Theme extension can be installed using pip:

```
pip install sphinx_rtd_theme
```

Here is example configuration with the fallback to the old theme:

```
try:
    import sphinx_rtd_theme
    html_theme = 'sphinx_rtd_theme'

    # set paths for loading local and also theme static resources
    html_theme_path = ['_themes', sphinx_rtd_theme.get_html_theme_path()]
    html_static_path = ['_static']

    # styles can be customized in this file
    html_context = {
        'css_files': ['_static/theme-custom.css'],
    }
except ImportError:
    # fallback to old theme
    html_theme = 'dirac'
    html_theme_path = ['_themes']
```

If you want to customize the HTML templates, just copy the original ones from the theme and place them inside the `_templates` folder inside the project. Sphinx will automatically prefer the ones in the `_templates` folder over the global ones from the theme itself.

```
# layouts can be customized in these paths
_templates/layout.html
_templates/footer.html
```

To override/customize the CSS styles, there is a css file placed under the `_static` folder called `theme-custom.css`.

New format of citations

Citations have been updated to use the BibTeX format. This format is more maintainable and standardized. Support for this format is provided by the [sphinxcontrib-bibtex](#) extension and it can be installed using pip:

```
pip install sphinxcontrib-bibtex
```

Citations can be added using the new `cite` directive:

```
# old format
[Dubillard2006]_

# new format
:cite:`Dubillard2006`
```

The new format also enables the option to download the citations in BibTeX. This code will generate a link to static BibTeX file with all citations:

```
:download:`BibTeX <zreferences.bib>`
```

The list of references can be added to the document using the `bibliography` directive:

```
.. bibliography:: references.bib
   :enumtype: upperroman
   :style: diracstyle
```

[!NOTE] Ensure that the bibliography directive is processed after all cites. The simplest way is to name the references page so that it starts with the “z” letter in order to be alphabetically last. The issue is being tracked [here](#).

When defining you own styles, please consider studying these [examples](#), as the documentation of this project is really poor. In order for citations to be sorted alphabetically, the plain or alpha formatting must be used.

Available labels:

- alpha - Display data from citation fields.
- numbers - Displays numbers.

Available formatting:

- alpha - Uses alpha label, citations are sorted by author_year_title.
- plain - Uses number label, citations are sorted by author_year_title.
- unsrt - Uses number label, citations are sorted by order of appearance in documentation.
- unsrtalpha - Uses alpha label, citations are sorted by order of appearance in documentation.

To customize citations style you can edit it in `conf.py`, for example here we are using the citation key as the label:

```
from pybtex.plugin import register_plugin
from pybtex.style.formatting.plain import Style
from pybtex.style.labels.alpha import LabelStyle

class DiracLabelStyle(LabelStyle):
    def format_label(self, entry):
        return entry.key

class DiracStyle(Style):
    default_label_style = 'dirac'

register_plugin('pybtex.style.labels', 'dirac', DiracLabelStyle)
register_plugin('pybtex.style.formatting', 'diracstyle', DiracStyle)
```

Download mathjax formulas in LaTeX format:

The mathjax extension has been extended and renamed to **mathjaxplus**. The new **mathjaxplus** extension provides a way to download formulas in LaTeX format. There are also configuration parameters exposed to customize the generation process:

```
# disable or enable
mathjax_generate_latex = True

# style of save as link
mathjax_latex_style = 'font-size:14px;text-align:right;'

# title of the save as link
mathjax_latex_title = 'Save as Latex'
```

Gitlog - display the GIT commit history for pages

A new extension called **gitlog** has been developed and integrated to the documentation. The motivation for this extension was to display information about the last git commit for each document in the documentation. There are some parameters exposed to configure the process:

```
# path to the git repository
gitlog_source_path = '../doc/'

# date format used to format git commit date in templates
gitlog_date_format = '%d %b %Y, %H:%M:%S'
```

The extension adds following data available to the context of given document:

```
git_author_name # name of the commit author
git_author_email # email address of the commit author
git_commit_sha # sha1 hash of the commit
git_commit_date # date of the commit
```

This code shows a simple usage in Jinja2 template:

```
{% if git_author_name %}
    Committer name: {{ git_author_name }}
    Committer email: {{ git_author_email }}
    Commit date: {{ git_commit_date }}
    Commit sha1 hash: {{ git_commit_sha }}
{%- endif %}
```

Continuous integration and deployments

The current trend is to test your code on each push to the versioning system. We use this to ensure the stability of the codebase and get valuable early feedback on our changes.

There are some free CI services available which suits our needs (search internet).

Most of the CI services also offer ability to deploy built code. We use this feature to deploy the documentation after successful builds. To deploy the code we use simple python script which wraps `rsync` command and adds some additional metadata to each deployed HTML. After build, most of the services leaves the build environment untouched and so we can just deploy the built HTML files. Some services do not, and we need to bootstrap the environment with system and python dependencies. To include build logs we need to rebuild documentation and capture the standard and error outputs.

Example documentation build and deploy:

```

BUILD_DIR=build_ci_name
cd $BUILD_DIR
SPHINX_LOG="sphinx-log.txt"
DOXYGEN_LOG="doxygen-log.txt"
SLIDES_LOG="slides-log.txt"
make html > $SPHINX_LOG 2> $SPHINX_LOG
make slides > $SLIDES_LOG 2> $SLIDES_LOG
make doxygen > $DOXYGEN_LOG 2> $DOXYGEN_LOG
cd ..
python maintenance/deploy_doc.py --root=/path/to/documentation/root --user=user123 --host=my.hostname.com --port=22 --post_script=/path/

```

When deploying logs it is expected that logs are named according to this mapping:

```

mapping = {
'sphinx': 'sphinx-log.txt',
'slides': 'slides-log.txt',
'doxygen': 'doxygen-log.txt',
}

```

Deploy script is located at maintenance/deploy_doc.py and code is documented so feel free to study it when implementing new deployer. For example this is implementation of Semaphore deployer:

```

class Semaphore(Deployment):
name = 'semaphoreci'

def get_root_path(self):
return os.environ['SEMAPHORE_PROJECT_DIR']

def get_branch(self):
return os.environ['BRANCH_NAME']

def get_revision(self):
return os.environ['REVISION']

```

As you can see we use environment variables to get information about the build. Each CI service provides its own set of environment variables so please refer to the respective CI documentation for more info. When implementing a new deployer you have to inherit from Deployment and implement some methods to provide CI specific data about build:

```

def get_root_path(self):
"""
Returns path to the root directory - git repository root.
"""
raise NotImplementedError

def get_revision(self):
"""
Returns the current git revision.
"""
raise NotImplementedError

def get_branch(self):
"""
Returns the current git branch.
"""
raise NotImplementedError

```

After implementing the custom deployer class, you have to add it to CI mapping in order to be able to use it. You can do so in the main function of the deployment script:

```

ci_mapping = {
'semaphoreci': Semaphore,
'codeship': Codeship,
'magnumci': Magnum,
}

```

To deliver the built HTML to the remote server we use rsync over SSH protocol. You can configure the script behaviour using keyword command line arguments:

```

--root
Remote root path, documentation will be synced to this path.

--user
Remote user.

--host
Remote hostname.

--port
Remote SSH port.

--post_script
Path to optional Python script on remote host.

```

After keyword arguments you have to provide two positional arguments.

- **ci** - CI name, as named in ci_mapping.
- **build_dir** - The name of the build directory, for example build, same as positional argument passed to the setup script.

Every operation on the remote server will be executed under the given user. If the post_script option is passed it will be executed after the rsync operation finishes. Example post deploy script can be found in maintenance/post_deploy.py.

FILE: www.diracprogram.org/doc/release-26/_sources/development/external_projects.md

orphan

Including external projects using Git submodules

See also this nice [tutorial](#) and these [tips and tricks](#).

We can use CMake's support for external projects together with the Git submodule functionality to fetch, compile, and link source code from external repositories.

When such external libraries/repositories are fetched, they are put in the directory `external`. Later when everything is set up, you can go in there, make changes, but you are not making changes to the parent code repository, but to the respective external repository. This way you can work on both codes at the same time, while making sure that changes to modules are communicated to the respective repositories.

This mechanism takes some time to get used to but turns out to be extremely powerful and convenient. Below are some typical scenarios.

Adding a new external project

First we need to set it up on the Git side. Let's assume the external project is hosted on `git@repo.ctcc.no:pcmsolver` and we want to put it in `external/pcmsolver`:

```
$ git submodule add git@repo.ctcc.no:pcmsolver external/pcmsolver
```

This will clone the external project and with `git status` we see:

```
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# modified:   .gitmodules
# new file:   external/pcmsolver
```

Now we commit this:

```
$ git commit
```

Great. Now the external project is fetched by Git and we need to tell CMake to build and link it. For this we open `CMakeLists.txt`, define some variables that we want to pass to the external project and we use the macro `add_external`:

```
set(ExternalProjectCMakeArgs
  -DCMAKE_INSTALL_PREFIX=${CMAKE_SOURCE_DIR}/external
  -DCMAKE_Fortran_COMPILER=${CMAKE_Fortran_COMPILER}
  -DCMAKE_C_COMPILER=${CMAKE_C_COMPILER}
  -DCMAKE_CXX_COMPILER=${CMAKE_CXX_COMPILER}
  -DCMAKE_BUILD_TYPE=${CMAKE_BUILD_TYPE}
  -DCBLAS_ROOT=${CBLAS_ROOT}
  -DEIGEN3_ROOT=${EIGEN3_ROOT}
)
```

```
add_external(pcmsolver "")
```

The `add_external` macro takes two arguments. The name of the external project and the command to be used for testing it. The name of the external project will be used as target name. If the testing command string is empty, as in the example above, the external project will not be tested after it had been built.

Finally we have to make sure that the library is linked.

If DIRAC f90 modules depend on f90 modules provided by the external library, we have to impose a compilation order:

```
add_dependencies(dirac pcmsolver)
```

You just want to build DIRAC

In this case you don't have to know anything about Git submodules:

```
$ ./setup
$ cd build
$ make
```

However, you need to have access to all repositories that the DIRAC will fetch from.

You want to check out the external sources without building DIRAC

Do this:

```
$ git submodule init
$ git submodule update
```

You will find the external sources in `external`.

You want to update external sources

For each external module DIRAC will reference a specific commit. This pointer (HEAD) will not move when external modules change until you tell DIRAC to reference some other commit. In other words, if I commit a bug to an external project repository and I don't tell DIRAC to use the new commit, I will not see this bug in DIRAC.

But sometimes you want to update the reference. For this go to the external repository (example: xcint):

```
$ cd dirac/external/xcint
```

Switch to the branch that you want to reference and update:

```
$ git checkout master
$ git pull origin master
```

Now register the new reference in DIRAC:

```
$ cd dirac
$ git add external/xcint
```

You want to commit and push modifications to external sources

The nice thing about Git submodules is that you can work on several projects or modules within one parent project. This is what we do very often. So I can change things on the DIRAC side, change things on the external project side, and commit changes to the respective repositories.

Before you modify external sources switch to the branch that you want to commit to. In the initial state external source repositories do not point to any branch (detached HEAD state).

Here is a typical work-flow:

```
$ cd dirac/external/xcint
$ git checkout master          # switch to the branch that you want to commit to
$ vi xcint-source.F90         # edit xcint source
$ git commit xcint-source.F90 # commit
$ git push origin master      # push to xcint

$ cd dirac
$ git commit external/xcint    # move the DIRAC reference to xcint
$ vi dirac-source.F90         # edit DIRAC source
$ git commit dirac-source.F90 # commit
$ git push origin master      # push to DIRAC
```

External project tracks different remotes on different DIRAC branches

This sounds involved but it might happen more often than you think. Let's again take PCMSolver as an example. The version released in DIRAC lives on GitHub and is publicly accessible, the remote URL being [git@github.com:PCMSolver/pcmsolver.git](https://github.com:PCMSolver/pcmsolver.git). On some other DIRAC branches though, we would maybe like to work with the development version of the module. This version is not publicly accessible and resides at another remote URL.

Each DIRAC branch has its own .gitmodules file, correctly configured to track the right remote. When switching DIRAC branches though, the same switch of remotes does not happen for submodules. To be more explicit, assume you have DIRAC_branch-public tracking GitHub and DIRAC_branch-private tracking repo.ctcc.no. Moreover, assume you are currently working on DIRAC_branch-public. As explained [here](#), the correct workflow to switch from one to the other is:

```
$ git checkout DIRAC_branch-private
$ git submodule sync
```

The git submodule sync command synchronizes submodules' remote URL configuration setting to the value specified in .gitmodules (taken from [this page](#))

FILE: www.diracprogram.org/doc/release-26/_sources/development/faq.md

orphan

F.A.Q.

How can I get the AO density matrix?

Use subroutine DENMAT dirac/dirden.F.

How can I get MO coefficients?

Use subroutine REACMO_NO_WORK in dirac/dirgp.F.

How are density/Fock/... matrices blocked?

They are blocked according to NBBAS and IBBAS.

Why do all routines end with “RETURN” before “END”?

Most of the routines end with:

```
subroutine foo()
...
return
end
```

The “return” is redundant. Simply write “end” without “return”. To make it better, we recommend programmers to stick to the Fortran90/95 coding standards. So the structure of the routine shall be as:

```
subroutine foo()
...
end subroutine
```

What are all the “Decks” before the subroutines good for?

Historical reasons. They have no meaning today.

Can I use zgemm or do I have to use the module matrix_defop?

There is nothing wrong with zgemm. You can use either one - whichever you prefer.

I want my new module to be completely modular. Can I then use DIRAC infrastructure routines like QUIT or READT?

If you want to be 100% independent of the DIRAC infrastructure then you cannot use such routines.

FILE: www.diracprogram.org/doc/release-26/_sources/development/further_development.md

orphan

Past and current works on DIRAC development

This web-page lists various projects for the development of the DIRAC software. It involved the collection of finished, on-going and proposed bachelor and master thesis projects for (mainly) IT-students, plus general wishlist.

IT-students at universities around the world have potential to contribute to the DIRAC infrastructure development. For them, one has specify suitable, “IT-only” projects around DIRAC, without the need of the deep knowledge of relativistic quantum chemical methods.

The DIRAC developers are kindly invited to share possible project proposals here. Each proposed theme should have short description (annotation), author (supervisor) and assigned student if any.

Thesis list

Comparison of CPU and GPU mathematical libraries

The aim of this thesis was to compare the speed of some numerical procedures from CUBLAS and CULA libraries with their Fortran analogues BLAS and LAPACK libraries. We wrote a short working program (hosted on github now) in Fortran and C which compares the time complexity for various input parameters (matrix, vector sizes etc). The usefulness of this student project is in possible application of CUBLAS / CULA library functions in the entire DIRAC program suite. (Finished in summer 2012, student D.Kuzma, supervised by Miro Ilias.)

CMake platform universal system for the software buildup

First, learn the CMake buildup system on simple testing programs written in C,C++,Fortran languages. Later, try to program more complex CMake buildup receipes, including macros. Assigned student I.Hrasko. Miro Ilias was supervising, Rado Bast was the referee. This bachelor project was finished in summer 2013.

Documentation generator Sphinx and its usage for DIRAC

The Sphinx documentation system, based upon Python language, has great potential for creating and easy maintaining the on-line documentation. This is clearly demonstrated on the DIRAC documentation. However, there is place for various improvements in the documentation framework. The assigned IT-student (Juraj Bubniak, supervised by Miro Ilias) polished the bibliography (for instance, better formatted table of references would fit), math equations (export in various formats), and introduces other goodies. Juraj succesfully defended his thesis in summer 2015.

Adpatation of DIRAC for Microsoft Windows and high-performace computing

We need to adapt DIRAC thoroughly for the MS Windows operating system and enable high-performace computing also on this (commertial) operating system. Assigned and currently working student is Ivan Hrasko, who gained rich experience with CMake in his bachelor project (Miro Ilias supervising). This master project was finished in summer 2015.

IT-aspects of grid-computations with DIRAC

Complex adaptation of DIRAC for computing in the grid environment. Create set of universal, low-level scripts (bash), handling the DIRAC suite calculations on unknown grid computing elements. Supervisor Miro Ilias is currently looking for an IT-student.

Testing various compilers and mathematical libraries for DIRAC

It is known that selection of compilers, their flags and mathematical libraries has influence on the code's time performance. For DIRAC, the aim is to employ various compilers (GNU, Intel, Portland), available libraries (MKL, ACML, ATLAS, netlib) for the time performance measurements. We hope to select set of compiler flags and suitable libraries giving the best execution timing. Supervisor Miro Ilias got an IT-student for bachelor thesis, hopefully he will start the work from 2016.

Git and web-based project development platforms in the service of DIRAC

The Git version control system is very popular for the code development. On top of it, various web-based project development platforms (like assembla, bitbucket, deveo, gitorious etc) are helpful for managing git repositories and controlling the program development. The aim is to describe (in the form of manual) the Git system and available project development web-servers. We investigate various features and show how all these goodies can be applied for the collaborative DIRAC development. This project is suitable for bachelor thesis.

Doxygen code documentation for DIRAC

The *Doxygen* is popular and free code documentation generator. It works for many programming languages, including C, C++ and Fortran. Our aim is to adapt DIRAC code description comments for the *Doxygen*, and, eventually, the *Doxygen* source code as well in order to get properly generated documentation of all Fortran, C and C++ DIRAC source files. This theme is suitable for one or more IT-students who are wanted.

Other ideas

New input reader with documenting of keywords

Let us write/use an keyword input reader which requires keywords to have some documentation. Otherwise the code won't even compile. Then the "make html" command would go through the source code and generate the RST files which would exactly reflect the state of the code (unless of course somebody changes the keyword without changing the text 2 lines below but we could introduce some punishment for it).

FILE: www.diracprogram.org/doc/release-26/_sources/development/good_fortran_90_practices.md

orphan

Good Fortran90 programming practices

We present some proper Fortran90 programming practices. By adopting them by programmers one could be working with readable and clean code.

Module names and file names

Use the same name for the module and the file. For instance, if your module is called foobar, call the file foobar.F90. This makes it easier to find the source code linked by "use foobar".

It is possible to group several modules into one file. Do it when your modules share some common characteristics. Modules grouping in files prevents flood of individual module source files slowing down the repository connections.

Example module

nice_module.F90

Include files and common blocks in the F90 era

Please do not introduce new common blocks. When you write new code and use F90, there is no need to introduce new common blocks (and include files) - use modules instead, they are much safer. Common blocks make the code harder to maintain.

Sometimes you have to use already present include files in F90. To make this work:

- only use "!" to start comments in include files (all F77 compilers understand this) - continuation use a & in column 73 of the first line and a & (nothing else!) in column 6 of the following line.

This will work with both free form and fixed form when the file is included)

- declare all variables explicitly, so that they can be used with the *IMPLICIT NONE* declaration.

There is a Python-script *reformat_includes.py* in the utils directory, that reformats include files according to these rules.

I/O control statements

To ease the localization of input/output-type bugs during the code run please provide main I/O statements (OPEN, READ) with the IOSTAT parameter evaluation or with other error-trapping programming scheme.

Examples :

```
!... control reading if gaunt=true
READ(LUCMD,*,IOSTAT=IOS) ILLINT,ISLINT,ISSINT,IGTINT
IF (IOS.NE.0) THEN
  WRITE(LUPRI,'(/,2X,A)') 'Error in SCF INTFLG reading !'
  & //'4 parameters needed for Gaunt term !'
  CALL FLSHFO(LUPRI)
  CALL QUIT( 'DHFIMP: Error in INTFLG reading for GAUNT=true!')
ENDIF
```

or (assuming that LU is correct unit number: :

```
WRITE(LU,desired_format,IOSTAT=IOS) SOMETHING
IF (IOS.NE.0) THEN
! Yell, that some variables in "SOMETHING" are out of desired format-bounds ;
! this could indicate run-time error (getting variable(s) hurt before)
! might activate program stop (CALL QUIT)....
ENDIF
```

F90 INCLUDE keyword

Do not use it. Use #include preprocessor statements only.

SIXLTR SBRTNE NAMING OBSOLT

Because it is so hard to read and understand. There is no need to to stick to 6 letter names anymore. The limit is 31 characters. Use meaningful names so that other people (and you in a couple of months) will understand your code.

Dynamical strings

The programmer may utilize short dynamical strings carrying descriptive labels, and attach this data type to his own data structures.

The module is :

use mystring

Methods at hand are:

1. creating the string

```
type(string) :: this_string
call make_string(this_string,'some description text')
```

2. printing the string on the LUPRI channel

```
call print_string(this_string)
```

3. copying one string into another

```
call copy_string(this_string, another_string)
```

4. canceling (deallocating) the string

```
call cancel_string(this_string)
```

For more details about the dynamical string functionalities check directly the `dynamic_string.F90` <../src/gp/dynamic_string.F90> source file.

Code debugging with LINEFILE

C-debugging symbols are defined by the preprocessor on each line. Therefore it is useful to insert printouts of file name and line number: :

```
print *, 'I am in file ',__FILE__, ' on line ', __LINE__
```

Now, of course we don't want to write that everywhere. What one can do is to use a macro: :

```
#define MYFUNCTION(x) myfunction(x,__FILE__,__LINE__)
```

This allows "myfunction" to print good error messages, because it knows the value of __FILE__ and __LINE__. But the macro has to be defined somewhere, so you need an "#include "mymem.h"" or something.

Here is an example that works, just for using __FILE__ and __LINE__ statements: :

```
subroutine mysub(someargument,file,line)
character(*) file
integer someargument, line
print *, 'mysub(',someargument,')'
print *, 'I was called in file "',file,'" at line ',line
end subroutine mysub
```

```
#define mysub_wrapped(arg) mysub(arg,__FILE__,__LINE__)
```

```
program test
integer x
x = 12
call mysub_wrapped(x)
end program test
```

For large scale employment one has to put the "define" in an include file, or better, in a Fortran90 module.

FILE: www.diracprogram.org/doc/release-26/_sources/development/input_reading.md

orphan

Modern input reading

What is presently there

Have a look in SUBROUTINE PAMINP this is the place where various input reading routines are called and input sections are read.

This is also the place where the new input reading routine `read_menu_input()` is called which reads the whole DIRAC.INP again.

The purpose of this documentation is to motivate the use of the new `read_menu_input()` instead of the many input section reading routines and to show how this can be done.

Problems of the old input reading

- Similar and very lengthy code is repeated over and over again. Sometimes buggy code is copy-pasted over and over again.
- Keyword matching is realized using a table/goto scheme which is cumbersome and buggy to program and difficult to read and to extend with new keywords. It is difficult to see what code is activated by a specific keyword.
- No type checking is done: wrong inputs can result in wrong parameter initialization without segfault, infinite loop situations, or abort without any error message.
- Section reading routines are called twice, first time to initialize common block data (just in case they are not actually called, meaning in case the `**SECTION` is not in the input) and possibly a second time to read input. The advantage of this is that options are initialized close to where they are read and set, but the price is a confusing top-level calling scheme.
- Case sensitive (caps lock).

Features of the new input reading

- Like the old scheme: will segfault and print list of available keywords if unknown keyword found.
- Case insensitive.
- Some basic type checking: it will catch situations (segfault and point to the place) where user has given no or not enough arguments or arguments of unexpected type (integer instead of real).
- More compact code.
- No table/goto scheme, keywords are defined in one place only and added easily, without extending table parameters or other structures.
- It is easy to define aliases to sections for backwards compatibility (example: `**WAVE FUNCTION` and `**METHOD` can do the same thing without much code duplication).

How does the new routine read the input

- The new input reading routine reads the whole input. The general structure (sectioning) is in one place. For sections and subsections individual routines are called.
- It ignores lines that start with `"!"` or `"#"`.
- Lines that start with `"."` are matched against available keywords. How does it know which section it belongs to? The routine remembers the last line with a section label (last line that starts with `**"` or `**"`).

This also means that this scheme favors:

```
*SECTION
.BLA
```

instead of the present:

```
.SECTION
*SECTION
.BLA
```

However, the more compact scheme is not enforced.

How to move a input section from old to new

- Have a look in the `src/input/input_reader_sections.F90`, add your section in subroutine `read_input_sections` and copy/paste/adapt one of the `read_input_something` subroutines.
- All keywords that belong to a section have to be between call `reset_available_kw_list()` and call `check_whether_kw_found(word, kw_section)`, otherwise the user will not get the correct full list of available keywords.
- Inside if (`kw_matches(word, 'FOOBAR')`) then and end if you can do whatever you like but you can take advantage of type checking by using the `kw_read` subroutine interface. If the interface does not cover your problem, please extend the interface.
- The way the old scheme is set up you cannot simply remove a section and move it to the new one. You have to advance to the next section in the old routine, otherwise you get into an infinite loop. Search for `move_to_next_star()`. This is a workaround and as soon as everything is replaced, these calls can be removed (the whole old structure can be removed).
- If you use common blocks to save parameters then you have the problem of how to initialize them (in modules you can initialize them directly). I recommend to move your parameters to a module but if you insist you can use common blocks but then find a good way of initializing them. Please don't do it by calling section subroutines twice, this is confusing.

FILE: www.diracprogram.org/doc/release-26/_sources/development/int8.md

orphan

Working with integer*8 variables

Introduction

What happens when a subroutine expecting an integer*4 argument is called with a integer*8 argument?

The answer depends on the hardware and software platform, and unfortunately errors can often be difficult to detect. To understand this, look at the memory layout of the number 12345 stored in integer*4 and integer*8 variables, on an regular Intel CPU (x86):

```
39 30 00 00
39 30 00 00 00 00 00 00
```

The first line shows the four bytes of the integer*4, in the order they are stored in memory on this particular machine. The second line shows the eight bytes of the integer*8 representing the same number. As you can see the start of the integer*8 is the same as the integer*4 (on this machine!).

Similarly, the number -1 is stored as

```
ff ff ff ff
ff ff ff ff ff ff ff ff
```

Again the first four bytes are the same.

Testing program

To understand what can go wrong take the FORTRAN77 example

```
subroutine f(x)
integer*4 x
print *, x
x = 12345
end
```

```
program p
integer*8 y
y = -1
call f(y)
print *,y
end
```

Since Fortran arguments are passed by reference, the subroutine f will get a pointer to the variable y. From the example above we know that the first four bytes of an integer*8 “-1” are the same as those of a integer*4 “-1”.

Therefore the number “-1” will be printed from f(), after which things go bad. This is what happens:

First we set y = -1:

```
y = ff ff ff ff  ff ff ff ff
```

Then f(y) is called. The variable x will reference “y”, but will only use the first four bytes:

```
x = ff ff ff ff (ff ff ff ff)
```

This x is printed as “-1”. The we set x = 12345, note that only the first four bytes are modified!

```
x = 39 30 00 00 (ff ff ff ff)
```

When f() returns y will have the value

```
y = 39 30 00 00  ff ff ff ff
```

Which will be printed as -4294954951.

This is just one example of how things can go wrong, and why things can often work by “chance”.

Note that this illustrated situation is very platform dependent; on a big-endian CPU the above example would not work at all. Always make sure to use the correct integer type in arguments, and link to correct versions of external libraries.

Notes

- The above example would work “correctly” if y was set to 0 on entry. If you find that you need to initialize intent(out) variables to 0 you may have an integer size mismatch.
- Passing integer arrays of the wrong size gives even more obvious bugs.
- Problems also appear if you pass integer*4 variables to a subroutine expecting integer*8.

Integer*4/8 type safe programs

FORTRAN90 compilers, which are checking also agreement of data types, should catch most of their incompatibilities in the code.

Though this is not the case of the Fortran77 example given above, one can rewrite this small piece of the code into a proper ‘Fortran90 checking form’ by utilizing module.

Program “test_f90.F90” (don’t forget big F in the suffix when applying preprocessor statements!) shall have this form:

```
module pass_integer
contains
subroutine f(x)
#if defined (F_INT4)
```

```

integer(kind=4), intent(inout):: x! F90 compiler gives error!
#else
integer(kind=8), intent(inout):: x! F90 compiler passed
#endif
print *, x
x = 12345
end subroutine f
end module pass_integer
program p
use pass_integer
integer (kind=8):: y
y = -1
call f(y)
print *,y
end program p

```

Typing

```
gfortran -DF_INT4 test_f90.F90
```

gives error of “Type/rank mismatch in argument ‘x’” while

```
gfortran test_f90.F90
```

compiles with the proper integer kind.

FILE: www.diracprogram.org/doc/release-26/_sources/development/multiple-remotes.md

orphan

Working with multiple remotes

There are two repositories

With the move to LGPL, the repository was split into two:

- Public repository: <https://gitlab.com/dirac/dirac/>
 - Contains master and release-* branches
 - Everything that is on master is meant to become the next release
- Private repository: <https://gitlab.com/dirac/dirac-private/>
 - Nothing is public, you decide when to share
 - No master branch, no release-* branches

How to start a new private development branch

Before starting a new private development branch on <https://gitlab.com/dirac/dirac-private/> you want to start from the most up-to-date public master.

We assume you have cloned from `git@gitlab.com:dirac/dirac-private.git` and your origin therefore refers to `git@gitlab.com:dirac/dirac-private.git`.

You can verify that this is the case:

```
$ git remote -v
```

```
origin  git@gitlab.com:dirac/dirac-private.git (fetch)
origin  git@gitlab.com:dirac/dirac-private.git (push)
```

To start from public master, you first want to add a new remote pointing to the public repository. You can call it `upstream`, or give it a different name (e.g. `public`):

```
$ git remote add upstream https://gitlab.com/dirac/dirac.git
$ git remote -v
$ git fetch upstream
```

The above fetches all commits from upstream which you did not have yet locally but will not merge anything.

Now you can create a new local branch `myownbranch` from master:

```
$ git branch myownbranch upstream/master
```

And then push the new branch to the private repository (origin):

```
$ git push origin -u myownbranch
```

Updating your development with upstream changes

This is something you will want to do relatively often, maybe one per month.

To merge the public master into your development branch `myownbranch`, you first want to add a new remote pointing to the public repository, like described further above:

```
$ git remote add upstream https://gitlab.com/dirac/dirac.git
$ git remote -v
$ git fetch upstream
```

The above fetches all commits from upstream which you did not have yet locally but will not merge anything yet.

We merge in a separate step:

```
$ git checkout myownbranch
$ git merge upstream/master
```

Your local branch myownbranch contains now commits from public master.

Contributing your changes upstream

This describes how to get your changes from your own development branch into public master. Typically this happens once the feature is implemented, and submitted for publication and you want it to become part of the next DIRAC release.

- Push the branch to <https://gitlab.com/dirac/dirac.git>
- Create merge request towards master
- This code will be in the next release (no more code removal in the release process)
- Then delete the branch on <https://gitlab.com/dirac/dirac-private/> (or keep it, up to you)

FILE: www.diracprogram.org/doc/release-26/_sources/development/museum.md

orphan

Dirac Museum

As the Dirac suite is developing, from time to time, it is abandoning various outdated parts of code or is switching to new programming schemes.

Each good software should keep and document its own history. Past involves many hours behind the computer screen, writing many lines of the code, repeated tests and mainly new ideas how to do things better.

Everything can be tracked in the deepness of the git repository. But some milestones in the code progress deserve to be highlighted and remembered here.

With this web page we would like to give tribute to some (old good) files and IT-solutions, which served us for a long time.

Repositories

The first repository one was under the CVS. Later it was replaced by the SVN (subversion) connected with own web-interface, <http://dirac.chem.sdu.dk/tracDirac>. Test run results were displayed on the web for the first time.

The last (and the best) option is the GIT versioning. The redmine project management web application serves for hosting the git repository on Norway's server.

Multiple git-repositories is the current trend.

Documentation

It started as versioned LaTeX files within in the Dirac repository. Later, it was replaced by non-versioned wiki web-pages, http://wiki.chem.vu.nl/dirac/index.php/Dirac_Program.

Finally the documentation returned to the Dirac repository under the universal Sphinx documentation generator.

Discussion groups

First we had own installed mailing system within <http://dirac.chem.sdu.dk>. This changed to two separate Google groups, one for developers, one for users.

Buildup and scripts

configure, config.guess

Shell (sh) scripts for configuring Dirac. Set the platform, compilers, flags, libraries. Completely replaced with the new CMake scheme.

pamadm

Used to be a shell (sh) script for selecting modules of Dirac. Later replaced with platform universal Python version. Later removed.

Makefile, Makefile.in

Each subdirectory with source files contained own Makefile.in file, which was copied into local Makefile. There was the central Makefile in the trunk directory. Now there are no longer user written makefiles, all are generated by the CMake scheme in buildup directories.

pam

Shell (sh) script for running Dirac. Later it was replaced by Miro's Python version. Ulf also provided an alternative 'wrapper.py' script.

testlast.sh

Shell (sh) script for running last test. Removed.

runtest

Also shell (sh) script for running test suite. Later substituted with Radovan's Python runtest.

However, it was found that CMake's own ctest program can do the whole job. Also *results/filter* and *menu* files in test directories were replaced with Radovan's new test Python file.

Programming practices**Fortran90 and modular programming**

The first programming language was Fortran77. In some time (maybe 2004?) one happens to have Dirac compiled also with Fortran90.

Instead of “*#include <common_block.h>*” we started to use “*use mymodule*”.

C++

The C++ programming language was introduced into Dirac by Ulf. It followed the C language which was in Dirac from its beginnings.

Memory management

The WORK/MEMGET/MEMREL scheme started to be replaced with classic Fortran dynamical allocations. Andre Gomes provided interface for that.

Long-term usage of *implicit.h* was no longer recommended, instead *implicit none* with explicit declarations are to be preferred.

8-bytes Integer

In the past we did not care about linking with integer*8 pre-compiled mathematical libraries when using 8-bytes integers. This changed by introducing integer-4/8 control tests against linked libraries.

FILE: www.diracprogram.org/doc/release-26/_sources/development/num_constants.md

orphan

How numerical constants are handled in DIRAC

In Dirac numerical constants are handled in a few different ways.

For example the code pieces ::

```
double precision :: x,y y = 2*x
```

with an integer “two”, ::

```
double precision :: x,y y = 2.0D0*x
```

with a double precision “two” and ::

```
double precision :: x,y double precision, parameter :: DTWO = 2 y = DTWO*x
```

all results in the same value being assigned to the variable y. The two first examples often result in exactly the same machine code, since the compiler know how to convert the integer or double precision number “two” to the most efficient form. In the third example the number is first stored in a (parameter) variable, and can then be used. Besides obvious uses for constants such as pi, this is perhaps most important when the constant is passed as an argument to a subroutine.

Since Fortran does not know what type of argument the subroutine expects it cannot convert the argument given by the programmer to the correct type. Therefore it is very important that the argument is passed with the correct type by the programmer. By making, for example, a double precision number DTWO the type is guaranteed to have this representation when passed to the subroutine. The same effect can be achieved by writing i.e. 2.0D0 (if a double is expected), or 2 (if an integer is expected). This does not mean that it is somehow “wrong” to write $y = 2*x$ in a line that is not a subroutine call, even if x and y are floating point numbers. In this case the compiler knows what to do, and you don't have to write $y = 2.0D0*x$. In the same way you can write $y = x/3$, and get correct results. On certain older cpus it was slightly faster to execute :

```
onethird = 1.0D0/3.0D0
y = onethird*x
```

than a simple divide by 3 (if repeated maybe times). On modern cpus the difference is most likely minimal. Using this style might cost one bit of precision, but in most cases it is best to write code in the most readable way possible and how to do this has to be decided on a case by case basis.

Finally most compilers are now smart enough to evaluate constants such as sqrt(2.0D0) at compile time, so if you only use this constant once there is no point in declaring this value as a parameter. However, here you have to be careful to pass the correct type to sqrt(). The call sqrt(2.0) will return a single precision approximation to the square root of two.

See [<http://www.ibiblio.org/pub/languages/fortran/ch2-2.html>] for more information.

FILE: www.diracprogram.org/doc/release-26/_sources/development/one-electron-property-operators.md

orphan

One-electron property operators

4-component one-electron operators are discussed in `one_electron_operators`. Here we look at the handling of matrix representation of such operators inside DIRAC. The general form of 4-component one-electron operators is

$$\{\backslash,\}^{4c}\backslash\Omega\text{mega} = \sum_{i=1}^{n_{\text{comp}}}\{\backslash,\}^{4c}M_i f_i P_i$$

where $\{\backslash,\}^{4c}M_i$ is a 4×4 matrix, f_i a real factor, P_i a real scalar operator and the sum running over the numbers of components of the operator (not to be confused with the number of components of a spinor).

The matrices $\{\backslash,\}^{4c}M$ are either *even*, sampling LL- and SS- blocks of the matrix of the scalar operator P , or *odd*, sampling the LS- and SL-blocks of the scalar operator.

The general input routine for 4-component one-electron operators is XPRINP (src/prp/paminp.F).

List of 4-component operators

Each unique operator gets an index and is added to the list of 4-component one-electron operators stored in the common block *dcbxpr.h*. For each 4-component operator the following information is stored:

PRPNAM

(user-defined) operator name

IPRTYP

operator type (see `one_electron_operators`), corresponding to a particular linear combination of 4×4 matrices. This automatically defines n_{comp} .

IPRPSYM

spatial symmetry (boson irrep)

IPRPTIM

time-reversal symmetry (-1,0,+1)

For each component:

FACPRP

real factor f_i

IPRPLBL

pointer to scalar operator P_i

List of scalar operators

A list of scalar operators is kept in common block *dcbprl.h*. For each scalar operator the following information is recorded:

PRPLBL

name of property label identifying the corresponding integrals on file AOPROPER

IPRLREP

spatial symmetry (boson irrep) of scalar operator

IPRLTYP

symmetry of scalar operator under matrix transpose (-1,0,1)

PDOINT

a CHARACTER*4 array where the four slots correspond to the LL, SL, LS and SS block of the scalar property matrix and have three possible values:

- '0': the block is not used (and therefore not generated)
- '+': the block is used
- '-': the block is used and scaled with minus one

Generating the matrix of a 4-component property operator

In the present setup DIRAC/HERMIT generates the integrals corresponding to the list of scalar operators in *dcbprl.h* and writes them to disk on the file AOPROPER. For property matrices that are symmetric or anti-symmetric only the lower triangle is stored.

DIRAC comes with a utility program *labread.x* which can be used to print the list of labels (and more) corresponding to the integrals stored in AOPROPER.

FILE: www.diracprogram.org/doc/release-26/_sources/development/patch_preparation.md

orphan

Patch preparation

Patches are *not* intended for new features, rather bug fixes and polish.

How to check out the release branch (example release-21 for DIRAC21)

Check out the release branch:

```
$ git checkout release-21
```

Now you have it next to master (verify this):

```
$ git branch
```

You can switch back to master:

```
$ git checkout master
```

And then switch back to the release branch:

```
$ git checkout release-21
```

How to prepare the patch

1. Bump the version (the version number is kept in file VERSION). Also `git grep -i` for “orphaned” version numbers in the source code), e.g.:

```
$ git grep "21\." src/
```

2. Verify that CHANGelog.rst is up to date (you may want to check history since previous version using `git log`).

At this point, you will have to create a branch in order to commit these changes, e.g.:

```
$ git checkout -b saue_patch21.1
$ git add CHANGelog.rst VERSION
$ git commit
$ git push
```

Next create a merge request to merge these changes into the current release branch.

Once the merge request has been accepted, we are now ready to download the tarball from GitLab and upload it to Zenodo (later we will automate this further).

As an example, for the 21.1 patch, visit <https://gitlab.com/dirac/dirac/-/tree/release-21> and click on the Download icon next to the Clone button. Rename the tarball to DIRAC-21.1-Source.tar.gz.

Before uploading the tarball, it is useful to compare to the existing tarball. We shall use the powers of git to do this. First unpack the tarball we just created, e.g.:

```
$ tar xvf DIRAC-21.1-Source.tar.gz
```

Next, go to the DIRAC homepage and click on the download button. This will take you to the proper zenodo page. From here you download the current release tarball. We unpack it and create a temporary git repository of it, e.g.:

```
$ tar xvf DIRAC-21.0-Source.tar.gz
$ cd DIRAC-21.0-Source/
$ git init
$ git add .
$ git commit
```

We have now staged and committed the entire content of the current release tarball. This information is stored in the `.git` subdirectory. The trick is now to copy this over to the directory containing the contents of the patch tarball, e.g.:

```
$ cp -R .git $HOME/Dirac/build/DIRAC-21.1-Source
$ cd $HOME/Dirac/build/DIRAC-21.1-Source
$ git status
$ git diff
```

This directory has now been activated as git repository, allowing us to check what files have been modified (`git status`) and in what manner (`git diff`). If things are okay, we are ready to upload the patch tarball to the zenodo page of the current release.

FILE: www.diracprogram.org/doc/release-26/_sources/development/profiling.md

orphan

Profiling how to

This is a quick and dirty guide to profiling. You are **strongly** recommended to read the profiling software documentation to get deeper insight in the commands suggested in the following.

Intel VTune Amplifier XE

This software is a nice tool for profiling and has a GUI working also on Linux. Moreover, you will not need to compile your program with profiling flags on. The only bad thing is that it is commercial software. Profiling takes two steps: 1. run your program under VTune; 2. analyze the collected data.

Assuming your program is called `foobar`, and further assuming that you only want to collect the *hotspots*, for step 1 you will launch:

```
amplxe-cl -collect hotspot -r /results/directory ./foobar
```

the results of the sampling will be put in `/results/directory`. If the results directory is not specified VTune will put everything in a subdirectory of the current directory. For step 2:

```
amplxe-cl -report hotspots -r /results/directory > report_name
```

this will produce a hotspots report called `report_name`.

gprof

`gprof` is free software. It works a bit differently from VTune since we need to explicitly compile our code with the `-pg` flag. This step is needed in order to link correctly the profiling library to the executable that will be produced. Once the program is correctly compiled you just need to execute it without specifying `gprof` as launcher as is the case for VTune:

```
./foobar [--flag1 --flag2 ...] [input1 input2 ...]
```

your program will automatically create a file, called `gmon.out` containing the profiling information collected. This file will by default sit in the current working directory. After data collection you need to generate a human-readable profile summary. We run then the `gprof` command:

```
gprof options [executable-file [profile-data-files ...]] [>outfile]
```

this will produce a so-called flat profile from the raw data collected. A more detailed introduction to `gprof` can be found here: [gprof](#).

Profiling DIRAC with Intel VTune Amplifier XE

The situation is a bit different if you want to profile DIRAC, because we usually launch the executable through a script. If you are using the `wrapper.py` script, you will specify as launcher:

```
--launcher="amplxe-cl -collect hotspot -r /results/directory"
```

the rest of the procedure remains the same. What if we want to profile during an MPI run? We modify the launcher as follows:

```
--launcher="mpirun -np 12 amplxe-cl -collect hotspots -follow-child -mrte-mode=auto -target-duration-type=medium -no-allow-multiple-runs -no-
```

Some more words of comment (shamelessly copied from `amplxe-cl -help collect`):

- `follow-child`, collects data on processes launched by the target process;
- `mrte-mode`, selects the profiling mode;
- `target-duration-type`, estimates the application duration time. This value affects the size of collected data;
- `no-allow-multiple-runs`, disables multiple runs to achieve more precise results for hardware event-based collections;
- `no-analyze-system`, disables analyzing all processes running on the system;
- `data-limit`, limits the amount of raw data to be collected;
- `slow-frames-threshold`, specifies a threshold to separate slow and good frames. It must be smaller than the threshold for fast frames;
- `fast-frames-threshold`, specifies the threshold to separate good and fast frames.

Profiling DIRAC with gprof

In contrast with VTune using `gprof` to profile DIRAC is rather straightforward. The only thing you will need to do is to link the profiling library. Thus if using the `setup` script you will type:

```
./setup --fc=... --cc=... --cxx=... --profiling --release
```

we recommend to build in release mode if you want to have your collected profiling data to be really significant. Once you managed to compile the sources correctly, you just run DIRAC as you're used to using `pam` or `wrapper.py`. The program itself will produce in your current working directory the `gmon.out` file and you will translate it to human-readable form with `gprof` as explained before.

One word of caution for MPI runs. To avoid all the processes trying to write to the same `gmon.out` file you should export the `GMON_OUT_PREFIX` environment variable:

```
export GMON_OUT_PREFIX=foobarmon
```

and pass it to `mpirun`:

```
mpirun -x GMON_OUT_PREFIX -np <np> ./foobar
```

In this way you will have a series of files named `GMON_OUT_PREFIX.pid`, post-collection analysis works exactly as for non-parallel runs. One warning from the `mpirun` manual: *“Users are advised to set variables in the environment, and then use `-x` to export (not define) them.”*

FILE: www.diracprogram.org/doc/release-26/_sources/development/release_preparation.md

orphan

Release preparation

Feature freeze

Two months before the release (example: DIRAC21), a release branch (example: release-21) is created from master. This is a feature freeze and from this moment on the release branch ideally only receives cosmetics and bugfixes, no major new features, no merges from master or other branches (except cherry-picks; more about it later). You as developer are responsible to commit or merge the to-be-released code to master, before the release branch is created.

Can I commit bleeding edge code to master after the feature freeze?

Yes you can! This is one of the reasons we have the release branch and exactly for this reason we will never merge from master to the release branch, only from the release branch to master.

How to check out the release branch (example release-21 for DIRAC21)

Check out the release branch:

```
$ git checkout release-21
```

Now you have it next to master (verify this):

```
$ git branch
```

You can switch back to master:

```
$ git checkout master
```

And then switch back to the release branch:

```
$ git checkout release-21
```

How to commit changes and bugfixes that are relevant for the released code

Commit all such changes to the release branch, not to master. This way no commit or bugfix will get lost. Please do *not* transfer commits from the release branch to master manually. This is not only unnecessary work but it is harmful (conflicts)! Again, do not commit to master, commit to the release branch:

```
$ git checkout release-21
$ git pull                # update the local release-21 branch
$ git commit              # commit your modifications
$ git push                # push your changes to release-21 branch
$ git checkout master     # switch to the master branch
$ git merge --no-ff release-21 # merge changes to master, fix conflicts if any
$ git push                # push your changes to master
```

Note that by merging your changes to master you might get conflicts. Git points them out clearly. In such cases open conflicting file(s) and fix discrepancies inside manually. They are marked with “<<<<<<” and “>>>>>>” strings.

What if you accidentally committed something to master which belongs to the release? In this case do *not* merge master to the release branch, but rather cherry-pick the individual commit(s) to the release branch:

```
$ git checkout release-21
$ git cherry-pick [hash]    # with [hash] that corresponds
                           # to the commit
```

How to exclude code from being released

Code that is part of master is meant to be part of the next release. There is no mechanism anymore to remove code from master with preprocessing or other scripts. This mechanism got removed when we changed the license to LGPL.

- Do not send merge requests towards master for code that you do not wish to be released.
- As a reviewer, please raise a question if a merge request towards master looks like something that should not be released quite yet.
- We will not release any of your private development branches on <https://gitlab.com/dirac/dirac-private>. Place your code there if you do not wish to release it quite yet.
- By sending a merge request towards master you signal that this code is meant to be released.
- Do not send merge requests towards master of code written by others without asking them.

Check copyright headers and version

1. Verify copyright headers.
2. Verify logo (list of authors).
3. Bump the version (the version number is kept in file VERSION; but also git grep -i for “orphaned” version numbers in the source code).

Copyright statements and markers

In the development source code you can find the following copyright statement markers:

```
!dirac_copyright_start
!dirac_copyright_end
!dalton_copyright_start
```

```
!dalton_copyright_end
\* dirac_copyright_start \*
\* dirac_copyright_end \*
\* dalton_copyright_start \*
\* dalton_copyright_end \*
```

They are there so that we do not have to update copyrights manually. You can update them with the `maintenance/update_copyright.py` script. This script will overwrite whatever is between these markers so it is a bad idea to modify text between the markers manually.

Create the final tarball

In principle we could just let people download the tarball directly from GitLab. However, since we include external code through Git submodules, this has the following disadvantages:

- Users would require Git
- Tarball is not self-contained and not fully reproducible since we have little control over the external repositories and whether they won't disappear in few years

For this reason we create a self-contained tarball using the `maintenance/create-release-tarball.sh` script and this is then the tarball that we upload to Zenodo.

We then direct users to download the tarball from Zenodo and not from GitLab for two reasons:

- More self-contained and reproducible
- We get download metrics from Zenodo which we would not get from GitLab

FILE: www.diracprogram.org/doc/release-26/_sources/development/rules.md

orphan

Programming rules

Here we provide some guidelines for DIRAC programmers.

Use of languages

DIRAC is for historical reasons largely written in Fortran 77 (F77), but in the newer parts of the code you may also see parts in Fortran 90 (F90), C and C++. Fortran 90 programming style with modules and other usefull features is required.

Do not introduce a new language before consulting the authors forum. New languages might require compilers that may be proprietary and consequently not be available to all authors and users.

Likewise do not introduce compiler-dependent extensions, though they are inviting and so convenient. Do not use them. You will regret it later when you have to modify the code for other compilers.

Fortran 90

We require a F90 compiler by default. Therefore all files with `.F`, `.F90`, `.f`, and `.f90` suffixes can contain F90 syntax. In `.F` and `.f` you can use F77 syntax, not in `.F90` and `.f90` files. In `.F` and `.f` files you can use F90 syntax however respecting the F77 line limits. Files with `.F` and `.F90` suffixes are preprocessed, files with `.f` and `.f90` are not preprocessed. If you introduce new source code file, always use `.F90` suffix.

F90 is case insensitive and allows both uppercase and lowercase. It is not necessary to program with the caps lock. Use uppercase and lowercase in a way that makes your code more readable.

“C” at the beginning of empty lines is unnecessary.

Do not read keywords with `table/goto` lookups

Perhaps 98% of the input reading is done using tables and gotos.

Please do not introduce new keyword sections like that for at least three reasons:

1. It is a nightmare to read and maintain.
2. There is no type checking so the user gets useless error messages or unexpected results.
3. New scheme minimizes conflicts when merging branches.

Rather have a look in `src/input`. This is the modern way to do it.

Never allocate memory with `MEMGET/MEMREL`

Large parts of the DIRAC code use `MEMGET/MEMREL`. Please do not allocate memory using `MEMGET/MEMREL`. We are trying to get rid of `MEMGET/MEMREL`.

Memory allocation should be done either using Andre's `alloc/dealloc` or using the native `allocate/deallocate` routines. `Alloc/dealloc` are frontends to the native `allocate/deallocate`. The advantage of `alloc/dealloc` is that the memory allocation gets tracked and shows up in the high water mark. Do not use other home-cooked memory allocation schemes unless you have absolutely no other choice.

Here is an example for `alloc/dealloc`:

```
use memory_allocator
```

```

real(8), allocatable :: one_dim_array(:)
integer, allocatable :: two_dim_array(:, :)

call alloc(one_dim_array, 1000)
call alloc(two_dim_array, 1000, 500)

...

call dealloc(one_dim_array)
call dealloc(two_dim_array)

```

Here is an example for allocate/deallocate:

```

real(8), allocatable :: one_dim_array(:)
integer, allocatable :: two_dim_array(:, :)

allocate(one_dim_array(1000))
allocate(two_dim_array(1000, 500))

...

dealloc(one_dim_array)
dealloc(two_dim_array)

```

When allocating memory please consider the following:

- The memory is not a bottomless pit. Try to be economic. - Do not allocate and deallocate inside of a loop that is traversed a billion times. Allocate/deallocate outside. - If you allocate arrays, clean up after yourself and deallocate data that is not needed anymore as soon as possible to keep the peak allocation as low as possible. Ideally at the end of the run we should have zero allocations. We are very far away from that.

Never use implicit.h

Plating the *implicit.h* into the code is bad programing practice and I strongly recommend to always use the *implicit none* statement.

Reasons: 1. With *implicit.h* it is easy to use variables that are undefined. Example :

```
call daxpy(size(array1), D1, array1, 1, array2, 2)
```

if one forgets to define the parameter D1, the code will compile and D1 can be any number so the result is undefined. With “implicit none” this will not compile.

Another example is a typo: instead of :

```
NDIM = 12 ! correct code
```

you could type :

```
NDIN = 12 ! typo
```

With *implicit none* such mistake won’t compile, with the *implicit.h* the code will compile and behave unexpectedly. In this case also the know compiler flag “-Wuninitialized” won’t help!

In the code we had (and unfortunately still have(many bugs like this and some of us have spent several long afternoons chasing them. The insertions of “implicit.h” is too dangerous and also experienced programmers make typos.

2. The insertion of *implicit none* is nice for other people reading your code. They can see which variables are local, which are global (from common blocks). Without implicit none you have no idea whether this is a local or a global variable if you don’t know the common blocks by heart.

3. The command *implicit none* makes it easier to identify include statements that are really used and includes that are just included but not used. With *implicit.h* you can comment out includes and the code may still compile. Contrary, the *implicit none* will warn you. If you remove an include file using *implicit none* and the code compiles, then this include was useless.

The time that you gain with *implicit.h*, you will spend it in debugging bugs that implicit none would have detected. *Implicit none* makes it easier for other people to reuse your routines. With *implicit none* they know exactly which variables are local to the routine and which are passed via header files or modules. With *implicit.h* they are completely in the blind.

Integer kinds

Please always use *KIND free* declarations in your codes:

```
integer :: variable
```

instead of *KIND restricted* integer variables :

```
integer(KIND=4) :: variable
integer(KIND=8) :: variable
```

This is because we need flexibility for integer variables: compilation of the DIRAC suite goes also with predefined integer*8 variables (through -i8 Fortranflag), for what all integer variables in the code must be declared without KIND restriction.

Radovan: I don’t agree. I think we should do the exact oposite. We would avoid a lot of trouble if we would use well defined kinds.

Interface Fortran and C using iso_c_binding

This is robust and portable. The iso_c_binding is supported by all modern compilers. Matching types at configure time with fortran_interface.h is painful and not robust.

If you replace a code remove the old code

Do not keep old code “just to be sure”. We have the well preserved Git history. Cut dead wood.

Don’t check in commented out lines of code. It will make it harder for other developers to modify or improve your code because they cannot know whether the commented code is important or not. If you really have to leave commented code you should document exactly why you feel that the old code has to stay as a comment inside the source file.

Never commit new functionality without tests

Functionality without tests will break sooner or later and nobody will notice for long time.

Also with tests you make it easier to understand what your code does and you make it easier for other people to improve your code. If there are no tests, then others will be afraid to modify untested code.

Make sure that new code compiles with and without MPI

Before you push new code to the main line development, test whether it compiles with and without MPI.

FILE: www.diracprogram.org/doc/release-26/_sources/development/screening.md

orphan

Notes on screening

We have some number of old density matrices D_{k-1}, D_{k-2} etc, and want to write the new density matrix as

..figure:: /dirac/images/math/6/c/8/6c86ef3df163c5e325edbc447abb31d9.png :align: center :alt: $D_k = D + \sum_{i=1}^N c_i D_{k-i}$,

$$D_k = D + \sum_{i=1}^N c_i D_{k-i},$$

where $\Delta^* D^*$ will be sent to TWOFCK and should give efficient screening. Currently we are minimizing the Frobenius norm (sum of squares of elements) of $\Delta^* D^*$, using one old density matrix. In the general case we have to solve $Sc = y$, where

$$S_{i,j} = (D_{k-1}^* \cdot D_{k-2}^*) \text{ and } y_i = (D_{k-1}^* \cdot D_{k-1}^*).$$

Here “ $(\cdot)$ ”(FIX) is the inner product between the D “vectors”. However, the use of the Frobenius norm gives the largest weight to the large elements of $\Delta^* D^*$. This is not really what we want, because we care more about producing as many small elements in $\Delta^* D^*$ as possible. Therefore we can imagine to introduce a metric matrix M , so that

..figure:: /dirac/images/math/8/7/7/877cd0b8aba2f7fcefddaa52ff414fdd.png :align: center :alt: $(A,B) = \sum_{ij} A_{ij} M_{ij} B_{ij}$,

$$(A,B) = \sum_{ij} A_{ij} M_{ij} B_{ij},$$

and set M to emphasize the elements that have a chance of being made small. In practice one could imagine

- Calculate $\Delta^* D^*$ using a unit ($M_{i,j} = 1$) metric.
- Find the largest elements (with some definition of largest) of $\Delta^* D^*$ and set the corresponding elements in M to zero.
- Recalculate S using the new metric.
- Solve for the new $\Delta^* D^*$

Doing this for the water molecule SCF in test 1 shrinks the elements in the final iteration $\Delta^* D^*$ by four orders of magnitude. In this case I used five old density matrices. Just using a large number of old densities without modifying the metric did very little. A simple test inside Dirac shows that it is not enough to use one historical density matrix with a modified metric. The reason is that at the end of the SCF the change in density is almost orthogonal to the last density matrix.

It would also be possible to give larger weight to certain elements of $\Delta^* D^*$, for example those that come from different integral blocks or centers.

FILE: www.diracprogram.org/doc/release-26/_sources/development/static_linking.md

orphan

Static linking

Theoretically, it’s easy to tell the *setup* script to prepare the *dirac.x* executable as statically linked (with the “-static” flag, thanks to the intertwined python scripts and the universal CMake buildup system).

However, neither all DIRAC buildups are ending up happily with static *dirac.x*, nor - in cases of successful compilation - resulting static executables are performing correctly.

Therefore we show some example configurations giving correct, partially correct, totally wrong and non-compilable executables. We hope that this info shall help developers to identify possible cracks in the generation of their own static executables.

Developers and users are invited to extend this collection of static buildup experiences.

Report about successful static DIRAC buildups and deployments is in reference Ilias2014.

Generating static OpenMPI

If you want to have static `dirac.x` running in parallel way, users have to prepare own static version of the OpenMPI wrappers. Download the installation package first, unpack it and follow installation instructions.

You get static OpenMPI suite by adding specific configuration flags. For example, with GNU compilers:

```
./configure --prefix=<path> --without-memory-manager CXX=g++ CC=gcc F77=gfortran FC=gfortran LDFLAGS=--static --disable-shared --enable-static
```

and with Intel compilers:

```
./configure --prefix=<space> --without-memory-manager CXX=icpc CC=icc F77=ifort FC=ifort LDFLAGS=--static --disable-shared --enable-static
```

finally type

```
make all install
```

You could to check your obtained OpenMPI wrappers for static linking:

```
ldd mpif90
not a dynamic executable
```

Correct static executables

- `miro.utcpd.sk:`

```
Linux-2.6.30-1-amd64, ownmath, i32lp64, GNU Fortran/gcc (Debian 4.4.5-10) 4.4.5; -static -fpic, nooptim
Linux-2.6.30-1-amd64, ownmath, i32lp64, GNU Fortran/gcc (Debian 4.6.2-9); -static -fpic, nooptim
```

- `frpd2.utcpd.sk:`
 - Linux-2.6.32-5-amd64; ownmath; i32lp64, GNU Fortran/gcc (Debian 4.6.1-4); -static -fpic, nooptim
 - Linux-2.6.32-5-amd64, ATLAS library; i32lp64, GNU Fortran/gcc (Debian 4.6.1-4); optimized gives few warnings at the linking step:

```
/usr/lib/gcc/x86_64-linux-gnu/4.6/../../../../x86_64-linux-gnu/libpthread.a(sem_open.o): In function `sem_open':
/home/aurel32/eglibc/eglibc-2.13/nptl/sem_open.c:333: warning: the use of `mktemp' is dangerous, better use `mkstemp' or `mkdtemp'
```

This is due to the " -Wl,-whole-archive -lpthread -Wl,-no-whole-archive " linking command containing the pthread-library, but this must be, because placing the sole "-lpthread" keyword elsewhere generates wrong executable giving this error:

```
Program received signal 11 (SIGSEGV): Segmentation fault.
Backtrace for this error:
+ function __restore_rt (0x15D0620)
  from file sigaction.c
```

- `grid3.ui.savba.sk:`

```
Linux-2.6.18-274.3.1.el5; ownmath; x86_64, ilp64, GNU Fortran (GCC) 4.4.4 20100726 (Red Hat 4.4.4-13)'; -g -static -fpic; opt
Note: this gfortran shows error in test program for NAMELIST reading, however, DIRAC runs OK...
NB: gfortran44 gives crashes for these OpenRSP tests: openrsp_pv,openrsp_jones,openrsp_cme.
For "pure" runs you have to have higher version of GNU compilers.
```

- `miro.ilias_desktop` (home PC Desktop, with most recent Ubuntu)
 - Linux-3.0.0-14-generic; x86_64; i32lp64; GNU Fortran/gcc (Ubuntu/Linaro 4.6.1-9ubuntu3) 4.6.1; /usr/lib/libblas.a+/usr/lib/liblapack.a; -g -static -fpic -O0 (NB: as superuser remove xerbla.o from /usr/lib/liblapack.a -mixing with /usr/lib/libblas.a)
 - Linux-3.0.0-14-generic;x86_64; i32lp64; GNU Fortran/gcc (Ubuntu/Linaro 4.6.1-9ubuntu3) 4.6.1; ATLAS-static; -g -static -fpic -O0 (fixed bug below)
- `opteron.tau.ac.il:`

```
./setup --type=debug -D VERBOSE_OUTPUT=ON --build=build_static --static --internal-math --int64
/opt/intel/fce/10.1.015/bin/ifort (ifort (IFORT) 10.1 20080312); /usr/bin/gcc - gcc (GCC) 4.1.2 20080704 (Red Hat 4.1.2-51)
```

- 194.160.34.207, Miro's old PC Desktop with updated Ubuntu
 - Linux-3.0.0-16-generic; GNU Fortran (Ubuntu/Linaro 4.6.1-9ubuntu3) 4.6.1 - both serial and parallel OpenMPI `dirac.x` are working

Based on these achievements we may say that GNU compilers produce proper static executable mostly not linked against GNU external static libraries (libblas-dev, liblapack-dev, libatlas). Likewise the Intel compiler - ifort - gives correct static executable.

Partially wrong executables

One can state that static executables are partially wrong when large portion of tests is crashing. For instance, failures are happening in input readings of DFTINPUT, CI/RELCCSD.

- `gbb4.grid.umb.sk:`

```
Linux-2.6.26-2-xen-amd64;ilp64,MKL ilp64 static math, ifort (IFORT) 12.0.2 20110112, gcc (Debian 4.3.2-1.1) 4.3.2
export MATH_ROOT='/home/chemia/ucitelia/milias/bin/intel_stuff/intel/mkl'
setup --static -D VERBOSE_OUTPUT=ON --int64
```

- `pd.uniza.sk:`

```
Linux-2.6.32-5-amd64, i32lp64, ownmath; GNU Fortran/gcc (Debian 4.4.5-8) 4.4.5
setup --static -D VERBOSE_OUTPUT=ON --internal-math (--debug)
```

- grafix.fpv.umb.sk:
- Linux-2.6.32-28-generic; x86_64; i32lp64; GNU Fortran/Gcc (Ubuntu 4.4.3-4ubuntu5) 4.4.3; ATLAS-static:

```
setup --static -D VERBOSE_OUTPUT=ON --debug
```

- Linux-2.6.32-28-generic; x86_64; ilp64; GNU Fortran/Gcc (Ubuntu 4.4.3-4ubuntu5) 4.4.3; ownmath

```
setup -D VERBOSE_OUTPUT=ON --debug --static --int64 --internal-math
```

Note that the Gfortran error in NAMELIST reading is described <http://comments.gmane.org/gmane.comp.gcc.bugs/289633> . This ill Gcc installation is probably causing mentioned tests failures.

Wrong executables

Here we describe cases of static dirac.x executables crashing already at the start of the run.

- miro.ilias.desktop:

Linux-3.0.0-14-generic;x86_64; i32lp64; GNU Fortran/gcc (Ubuntu/Linaro 4.6.1-9ubuntu3) 4.6.1; ATLAS-static; -g -static -fpic -O0, giving :

```
Program received signal 11 (SIGSEGV): Segmentation fault.
Backtrace for this error:
+ function __restore_rt (0x15C4220)
from file sigaction.c
Segmentation fault
(see also http://gcc.gnu.org/bugzilla/show_bug.cgi?format=multiple&id=42477)
```

- 194.160.135.47:

Linux-2.6.30-1-amd64, i32lp64, GNU Fortran/gcc (Debian 4.4.5-10) 4.4.5; -static -fpic, nooptim does not compile with static external /usr/lib/libblas.a+usr/lib/liblapack.a, giving :

```
/usr/lib/liblapack.a(xerbla.o):function xerbla_: error: undefined reference to '_gfortran_transfer_character_write'
/usr/lib/liblapack.a(xerbla.o):function xerbla_: error: undefined reference to '_gfortran_transfer_integer_write'
/usr/bin/ld: error: /usr/lib/libblas.a(xerbla.o): multiple definition of 'xerbla_'
/usr/bin/ld: /usr/lib/liblapack.a(xerbla.o): previous definition here
```

The upgrade is compilable, but does not work with static ATLAS (as frpd2): Linux-2.6.30-1-amd64, ATLAS BLAS+LAPACK; i32lp64, GNU Fortran/gcc (Debian 4.6.2-9); -static -fpic, nooptim

Executables not compilable

We give some cases when one can not obtain the static dirac.x executable due to error at linking stage.

Linking Fortran executable dirac.x gives for parallel, OpenMPI, for example :

```
/home/ilias/bin/openmpi_ilp64/lib/libopen-pal.a(dlopen.o): In function 'vm_open':
dlopen.c:(.text+0x148): warning: Using 'dlopen' in statically linked applications requires at runtime the shared libraries from the glibc ver
/home/ilias/bin/openmpi_ilp64/lib/libopen-rte.a(plm_rsh_module.o): In function 'setup_launch':
plm_rsh_module.c:(.text+0x821): warning: Using 'getpwuid' in statically linked applications requires at runtime the shared libraries from the
/home/ilias/bin/openmpi_ilp64/lib/libmpi.a(btl_tcp_component.o): In function 'mca_btl_tcp_component_create_listen':
btl_tcp_component.c:(.text+0x104): warning: Using 'getaddrinfo' in statically linked applications requires at runtime the shared libraries fr
[100%] Built target dirac.x
```

- opteron.tau.ac.il

export MATH_ROOT='/opt/intel/mkl/10.0.3.020/lib/em64t' , static buildup does not work with MKL libs, giving

```
/opt/intel/fce/10.1.015/bin/ifort -static -WL,-E -w -assume byterecl -DVRT -g -traceback -static-libgcc -static-intel -i8 -O0 CMakeFiles/dir
ld: cannot find libmkl_intel_lp64.a
```

- grid3.ui.savba.sk
- SL5 Linux, 2.6.18-348.1.1.el5, x86_64; i32lp64 using system static blas+lapack:

```
/usr/bin/gfortran44 -static -WL,-E -g -fcray-pointer -fbacktrace -DVAR_GFORTRAN -DVAR_MFDS -fno-range-check -static -O0
CMakeFiles/dirac.x.dir/src/main/main.F90.o -o dirac.x lib/libdirac.a lib/libxcfun.a -lstdc++ /usr/lib64/liblapack.a /usr/lib64/libblas.a
```

```
/usr/lib64/liblapack.a(ilaenv.o): In function 'ilaenv_':
(.text+0x25d): undefined reference to '_gfortran_copy_string'
/usr/lib64/liblapack.a(ilaenv.o): In function 'ilaenv_':
(.text+0x2ea): undefined reference to '_gfortran_copy_string'
/usr/lib64/liblapack.a(ilaenv.o): In function 'ilaenv_':
(.text+0x305): undefined reference to '_gfortran_copy_string'
/usr/lib64/liblapack.a(ilaenv.o): In function 'ilaenv_':
(.text+0x31b): undefined reference to '_gfortran_copy_string'
/usr/lib64/liblapack.a(zlargv.o): In function 'zlargv_':
(.text+0x7d2): undefined reference to '_gfortran_pow_r8_i4'
/usr/lib64/liblapack.a(zlartg.o): In function 'zlartg_':
(.text+0x4d9): undefined reference to '_gfortran_pow_r8_i4'
/usr/lib64/liblapack.a(dlamch.o): In function 'dlamch_':
(.text+0x44c): undefined reference to '_gfortran_pow_r8_i4'
/usr/lib64/liblapack.a(dlamch.o): In function 'dlamch_':
(.text+0x594): undefined reference to '_gfortran_pow_r8_i4'
/usr/lib64/liblapack.a(dlamch.o): In function 'dlamch_':
(.text+0xe15): undefined reference to '_gfortran_pow_r8_i4'
/usr/lib64/liblapack.a(dlamch.o):(.text+0xe39): more undefined references to '_gfortran_pow_r8_i4' follow
collect2: ld returned 1 exit status
```

This problem is fixed by abandoning /usr/lib64/liblapack.a completely, what is accomplished by using “-lapack=off” in the setup flags.

FILE: www.diracprogram.org/doc/release-26/_sources/development/test_basis_sets.md

orphan

Testing basis set files in DIRAC

The DIRAC program suite hosts more than different 300 basis set input files. These are either relativistic basis sets (all-electron, ECP) or nonrelativistic basis sets, collected from other sources (Dalton, EMSL etc). Therefore it is necessary to have an (automated) tool for testing all basis set files distributed with DIRAC.

Currently, there are few tests focused at checking basis set files:

- test/basis_input_scripted (only reads basis sets, but all of them)
- test/basis_input (reads few basis sets)
- test/basis_contraction (run few SCFs in contracted basis)
- test/basis_automatic_augmentation (run few SCFs in augmented basis)

Basis set files testing

The test **test/basis_input_scripted** is dedicated for automated checking of (almost all) basis set sets files sitting in the DIRAC directory.

How it works

The tests performs reading of multiple basis set files, and merely checks that the input parser does not choke on picked basis set. No reference outputs are involved.

The principal Python script test `<../../../../test/basis_input_scripted/test>` is employing the runtest library (<https://runtest.readthedocs.io>).

When the typical string sequence is found in the basis set file, the script extracts the proton number of a given element. Script then launches run(s) with the input `<../../../../test/basis_input_scripted/input_test_basis.inp>`, molecule `<../../../../test/basis_input_scripted/element.mol>` files and with pam parameters containing selected basis set name and element's proton number as strings for replacements by Python.

Default run

Test's default run checks all basis set files in given day once per month. This is achieved by running through basis set directories where each file with several basis sets inside is read. The total number of individual runs makes more than 5000, and it takes about 20 minutes.

On other days the test makes empty run.

Examples of usage

User can specify by environment variables (BSFILE, ZELEM, RANDOM_BSF_Z) on what basis files/elements to run.

Read all elements basis sets from selected files:

```
export BSFILE="basis_ecp/ECPDS28SDFSF basis/dyall*"
./test -b <BUILDIR>
```

Run only selected elements in selected basis set file(s):

```
export BSFILE="basis_ecp/ECPDS28SDFSF"
export ZELEM="30 35"
./test -b <BUILDIR>
```

Run all elements in two randomly selected basis set files:

```
export RANDOM_BSF_Z="2"
./test -b <BUILDIR>
```

Run three elements in three randomly selected basis set file:

```
export RANDOM_BSF_Z="3 3"
./test -b <BUILDIR>
```

Nonworking cases

Because of many basis set files (more than 300) one can not expect all of them working properly.

There are cases of wrong basis sets. For example, the proton number can not be extracted from "basis_dalton/aug-pc-0_emsl". For more info about that, see the [DIRAC issue](#).

FILE: www.diracprogram.org/doc/release-26/_sources/development/testing.md

orphan

Never commit functionality to the main development line without tests

If you commit functionality to the main development line without tests then this functionality will break sooner or later and we have no automatic mechanism to detect it. Committing new code without tests is bad karma.

When you add a new test always add it to CMake

Otherwise the test will never be run as part of the default test suite. `git grep add_test` to see where tests are defined. For details, see below.

Strive for portability

Avoid shell programming or symlinks in test scripts otherwise the tests are not portable to Windows. Therefore do not use `os.system()` or `os.symlink()`. Do not use explicit forward slashes for paths, instead use `os.path.join()`.

Never add inputs to the test directories which are never run

We want all inputs and outputs inside `test/` to be accessible by the default test suite. Otherwise we have no automatic way to detect that some inputs or outputs have degraded. And degraded inputs and outputs are useless and confusing.

How can I run a single test?

The test runner is CTest. You can run a single test with:

```
ctest -R mynewtest
```

where *mynewtest* is the name of test directory.

Alternatively you can run a test directly in the test directory. For this check out:

```
cd test/mynewtest
./test --help
```

Always test that the test really works

It is easy to make a mistake and create a test which is always “successful”. Test that your test catches mistakes. Verify whether it extracts the right numbers.

How to add labels to tests

Individual tests can have extra descriptors (labels, tags) helping to categorize them. Pick suitable label names characterizing given test.

Here is an example:

```
dirac_test('dft_ac', 'short;dft')
```

If you give several labels, separate them with “;”.

The advantage of labeling a test is that one can call group of tests according to their labels. For instance, launching all tests containing label “short”:

```
ctest -L short
```

How to add a new test

1. First thing to add a new test is to add a python script called “test” in your test directory.

In most cases you probably want to use the `runtest_dirac.py` test library since it gives you the framework to run DIRAC jobs and to filter numbers with numerical tolerance. But you don’t have to. You have full liberty to test whatever you want in whatever way you want. Important is that you return an exit code. If the exit code is 0, the test was successful, if non-0, then the test has failed.

Please provide also `README.rst` description file to the test directory so that other developers know what functionalities the test covers, what is the meaning of the test.

2. You have to add the new test also to CMake infrastructure, otherwise the test will never be run as part of the selected test suite. Read the following.

Tests categories in Dirac

Tests are grouped into several categories. Consider to choose suitable class for your test depending on test’s execution time, test’s intention etc.

2.1. Default tests. These are always run as they cover at least released Dirac functionalities. Be sure they do take not too much time !

Modify the `cmake/custom/test.cmake` file <../../cmake/custom/test.cmake>.

2.2. Tutorial tests. These are added to default tests upon **-D ENABLE_TUTORIALS=ON**. Tutorial tests are directly interconnected with tutorial documentation files, which refer also to corresponding input and output files of given tutorial test(s). Therefore these tests serve mainly to verify correctness of (ascii) files read and produced by Dirac. These tests are not restricted by time as are default tests, but please consider rather shorter than longer examples.

See and modify the `cmake/custom/tutorials.cmake` file `<../../../../cmake/custom/tutorials.cmake>`.

2.3 Benchmark tests. These tests serve to assess the Dirac execution performance. These tests are activated with `ctest -C benchmarks -L benchmark`.

See and modify the `cmake/custom/benchmarks.cmake` file `<../../../../cmake/custom/benchmarks.cmake>`.

2.4 Features restricted tests. In the DIRAC development version, preprocessor restricted features are accompanied with dedicated tests.

How to include them, see `src/CMakeLists.tx` file `<../../../../src/CMakeLists.txt>` for feature attached tests and the

FILE: www.diracprogram.org/doc/release-26/_sources/development/transferring_uncommitted_code.md

orphan

Transferring uncommitted code between machines

As regular tarball

For this make sure that you also take the `.git` directory otherwise compilation will complain with:

```
fatal: Not a git repository (or any of the parent directories): .git
CMake Error at cmake/ConfigGitRevision.cmake:9 (string):
  string sub-command STRIP requires two arguments.
```

Also make sure that you get all the external sources. In other words if you manually tar up a fresh DIRAC clone without `.git` that has never been built, externals will be missing.

Using scp

This should work. But even better is git clone over ssh (below).

Git clone

You can git clone from one machine to another. Note that you cannot push to a non-bare repository. In other words if you clone a regular clone, you cannot push to it.

FILE: www.diracprogram.org/doc/release-26/_sources/development/windows.md

orphan

DIRAC on Windows - developers section

After completing the explanation of DIRAC installation and first steps on the Microsoft (MS) Windows (shortly Windows) platform for users (see `windows_users`) we continue this manual for the developers.

Motivation

DIRAC is being developed and works mainly on machines with Linux/Unix/Mac operating systems. Since many desktop computers are sold with preinstalled Windows operating system, it is worth, we believe, to adapt the DIRAC software for the Windows environment. Also, the Windows operating system has great potential for the [high-performance computing](#). This is the motivation for investing efforts in the code's adaptation for the Windows platform.

Thanks to the modern, platform-universal *CMake* buildup system and related Windows supported software you can fully set up the DIRAC software under the Windows operating system.

We have tested the DIRAC downloading from its git-repositories, configuration, compilation and tests running, all in the framework of the Windows 7/8 operating systems.

Dependencies

In addition to the necessities listed in the section for users, DIRAC developers have to install:

5. The Git version control system (<http://git-scm.com>). When ask you can choose options "Use Git from Git Bash only" or "Use Git from the Windows Command Prompt" (recommended). If you choose the first option then you have to manually add into your `%PATH%` `"..."` (depends on your installation). It is not possible to successfully run *CMake* with optional Unix tools in `%PATH%`.
6. The Sphinx documentation generator and related packages, see below.

Documentation tools

To be able to generate the html documentation (*mingw32-make html*), the user has to download and install these Python packages:

Sphinx (<http://sphinx-doc.org/>). Python Wheel can be found at <https://pypi.python.org/pypi/Sphinx/> and then run (for example): :

```
pip install Sphinx-1.3b1-py2.py3-none-any.whl
```

You might need also the *python-dateutil* and *pyarsing* packages which you can obtain from <https://pypi.python.org/pypi/python-dateutil/> and <https://pypi.python.org/pypi/pyarsing/> respectively. You can install their Python Wheels using *pip install* command.

Next download Python Wheel file of *sphinxcontrib-bibtex* from <https://pypi.python.org/pypi/sphinxcontrib-bibtex> and Python Wheel file of *sphinx_rtd_theme* from https://pypi.python.org/pypi/sphinx_rtd_theme. Install both using *pip install* command.

numpy (<http://www.numpy.org/>), which provides own (self-)installation file. For 32-bit Python you can download installer from <http://sourceforge.net/projects/numpy/files/NumPy/> and for 64-bit Python there are only unofficial builds for example at <http://www.lfd.uci.edu/~gohlke/pythonlibs/>.

matplotlib (<http://matplotlib.org/>). Windows installer or Python Wheel can be found at <http://matplotlib.org/downloads.html>. Please download version suitable for your version (2.x or 3.x) and type (32/64 bit) of Python.

Install *doxygen* if you want to generate documentation using *mingw32-make doxygen*. You can find self-installer (.exe) at <http://www.stack.nl/~dimitri/doxygen/download.html>. We recommend to use self-installer because it adds doxygen binary directory to system path variable automatically.

To create html slides (*mingw32-make slides*). Install *hieroglyph* (<https://pypi.python.org/pypi/hieroglyph>) by command: :

```
pip install hieroglyph
```

Notes:

Check that the Windows *%Path%* environment variable points to both Python and its extensions, and has the the form of "...C.;C;...". Doxygen binary directory "...C;..." should also be present in *%Path%*. Do not use 32 and 64 bit packages together. You can obtain *pip* from <https://pip.pyba.io/en/latest/>.

Git & PuTTY

To enable server connectivity please install full PuTTY package for handling your SSH key pair. The best way is through the PuTTY self installer, see <http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html>.

Prepare your own pair of SSH keys - private and public. You can generate them with the PuTTYGen generator which writes private key in wanted ppk-format.

For downloading/uploading from/to the DIRAC repository you have to have the PuTTY "Pageant" authentication agent active with loaded private key, while the public key has to be placed in the DIRAC git repository user space. Pageant can be activated during the Windows startup, for example, launched at the start of user's login on Windows machine: :

```
"C:\Program Files\PuTTY\pageant.exe" C:\Users\<user_name>\.ssh\id_rsa_Windows7.ppk
```

The "pageant" agent, running in the background, also enables passwordless terminal connections (or PuTTY sessions) onto server which are provided with users's public key.

For communication with the DIRAC repository activate the PuTTY's "plink.exe" executable through the "GIT_SSH" environment variable (please do not add this variable into system environment variables because people who do not want to use "pageant" can have problems to normally use "git" - create user environment variable instead). Its typical value is :

```
GIT_SSH=C:\Program Files (x86)\PuTTY\plink.exe
```

assuming that the PuTTY package is placed in the default installation space: :

```
C:\Program Files (x86).
```

Also, before using "git" refresh your private key in your computer's cache, use :

```
"C:\Program Files (x86)\PuTTY\plink.exe" -agent git@gitlab.com:dirac/dirac-private.git
plink -agent git@bitbucket.org
```

For comfortable commits with the git version control system you should set up the editor. For the popular *Notepad++* editor setting for git [follow](#) : :

```
git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -noPlugin"
```

To update your local DIRAC source (must have the ssh connection enabled, see above): :

```
C:\Users\<username>\Documents\Work\Dirac_git\trunk>git pull
```

Do not configure DIRAC in the Linux-like "git-bash" window, which is part of common Git installation. Executing cmake configuration commands in this windows causes unhomogenous treatment of path-strings: mixing of Linux and Windows path-delimiters.

Some caveats

Windows operating system does not accept certain reserved words for file and folder names, like aux, con ... (google after forbidden file and folder names on Windows).

Likewise Windows does not like "*" characters in file names and due to this is not able to download such files from the repository. For that reason several traditional basis set files have been renamed accordingly, see the following table:

Windows new name	Previous file name
3-21++G-star	3-21++G*
3-21G-star	3-21G*
6-311+G-star	6-311+G*
6-311G-star	6-311G*
6-311++G-star-star	6-311++G**

Windows new name Previous file name

6-311G-star-star	6-311G**
6-31++G-star	6-31++G*
6-31+G-star	6-31+G*
6-31G-star	6-31G*
6-31++G-star-star	6-31++G**
6-31G-star-star	6-31G**

The solution is that the basis set reader automatically translates “*” basis set names to the “star” filenames.

Testing

Run specific tests: :

```
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>ctest -VV -R fsccl
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>ctest -V -D ExperimentalTest -R fsccl -D ExperimentalSubmit
```

Perform the complete CDash buildup in one step: :

```
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>mingw32-make Experimental
```

or :

```
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>ctest -D Experimental
```

You can perform the CDash buildup in separate steps: :

```
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>mingw32-make ExperimentalUpdate
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>mingw32-make ExperimentalConfigure
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>mingw32-make ExperimentalBuild
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>mingw32-make ExperimentalTest
C:\Users\<username>\Documents\Work\Dirac_git\trunk\build>mingw32-make ExperimentalSubmit
```

Automatic test runs

You have to set up the *Task Scheduler* accordingly for DIRAC automatic buildups.

Don’t forget that in order to perform the git-synchronization you must have your private key loaded in the *pageant* program (see above). This can be achieved in this way (bar Actions of the Task Scheduler): :

Action: Start a program

Program/script:
"C:\Program Files (x86)\PuTTY\pageant.exe"

Add arguments (optional):
"C:\Users\milias\Documents\milias_private_key.ppk" -c "C:\Users\milias\Documents\Dirac\software\devel_trunk\maintenance\cdash\cdash.Win8.0.F"

In the buildup script (here cdash.Win8.0.FPV-Miro.bat) - in order to kill the zombie pageant process - you may to choose either to kill the thread directly or to restart your Windows PC. In the latter case please check the sleeping mode is working not have the office PC running in the night. The buildup script wakes the PC up, which after the whole build switches back to sleeping.

Further development plans

So far we have tested DIRAC in sequential buildups (only) with the MinGW64 (free of charge) suite of compilers. To profit from multithreading under Windows, enthusiastic developers (where are you?) should investigate Open MPI parallelization, www.open-mpi.org. Also, one should try some commercial compilers, like Intel, Nvidia, ...

Concerning mathematical libraries, we have to resort to DIRAC own BLAS & LAPACK libraries. The ATLAS library preparation for Windows is only in the experimental phase (<http://math-atlas.sourceforge.net/errata.html#wjw64>). There is the Intel-MKL library at hand, but costs some money :). For sure we should investigate the Windows version of the ACML library. OpenBlas <http://www.openblas.net/>.

Another topic is the flavour of various CMake generators. On Linux systems we are using the default choice, “Unix Makefiles”. On the MS Windows operating system we are employing the “MinGW Makefiles” generator - part of the GNU MinGW suite. However, there are many more generators worth to try, like NMake, MSYS Makefile, Visual Studio, Codeblocks...

FILE: www.diracprogram.org/doc/release-26/_sources/development/xml.md

orphan

Documentation for Fortran XML libraries

These libraries can read an xml document into fortran or write fortran data to xml. For more documentation on xml see wikipedia.

To improve performance a special attribute has been defined for tags:
<tag type="realdata">. When this attribute is the data in this tag will be treated as a huge chunk of real data. The dimensions of this block have to be set by the attributes size=, which indicates the number of lines and width=, which indicates the number of columns.

Usage

The libraries can be accessed by including in the fortran code

```
USE xml_parser
```

A given XML document is read at once to memory, from where any tag can easily be accessed.

The next thing to be done is to declare a pointer to an XML document.

```
type(xml_tag):: tag
```

Then the following functions can be called:

- `xml_tag=>xml_open(treename)`

Starts an new xml document tree, with name `treename`. Returns a pointer to the xml tree

- `xml_tag=>xml_read(filename,treename)`

Reads a file named with `filename`, containing an xml document into the document tree with name `treename` (name is used if `treename` is missing). Returns a pointer to the `xmltree`.

- call `xml_write(filename,treename)`

Writes the document tree `treename` into an xml file, with `filename`.

- `xml_tag=>xml_close(treename)`

Removes the document tree `treename` from memory, returns the null-pointer.

- `logical=xml_tag_next(xml_tag,name,nameattribute)`

Sets the pointer `xml_tag` to the next tag with name and having optionally the attribute name to `nameattribute` (ie. `<name name="nameattribute" ... >`). If name is "" or missing then the pointer `xml_tag` is set to the next subtag of the last non empty search name (or the entire document). The function returns true or false depending on whether a next tag is found. So in order to search all the subtags of say the tag 'grid' it's sufficient to use:

```
type(xml_tag),pointer:: tag
!
tag=>open('xmlfile')
do while(xml_tag_next(tag,'grid'))
  do while(xml_tag_next(tag))
    ... code ...
  end do
end do
tag=>xml_close('xmlfile')
```

- `name=xml_name(tag) / call xml_name(tag,tag_name)`

returns the name of the tag or it sets the name of a tag.

- `value=xml_attribute(tag,attribute_name) / call xml_attribute(tag,attribute_name,value)`

returns the value of an tag attribute, or sets it. `xml_name` returns the name of the tag or it sets the name of a tag.

- `real_pointer(:,:) / char_pointer(:) => xml_get_data(xml_tag)`

Returns a pointer to an array containing the character of real data that is stored in the tag pointed to by `xml_tag`. So usage could be

```
character,pointer:: char_data(:)
real,pointer:: real_data(:,:)
!
... some code ...
char_data=>xml_get_data(tag)
real_data=>xml_get_data(tag)
... more code ...
```

The function returns `null()` if no data is stored.

- call `xml_add_data(xml_tag,real / char_data)`

Writes real or character data to the current tag stored in the tag pointed to by `xml_tag`. So usage could be

```
character(len=...),pointer:: char_data(:)
character(len=...): string
real:: real_data(:,:)
!
... some code ...
call xml_add_data(tag,char_data)
call xml_add_data(tag,string)
call xml_add_data(tag,real_data)
... more code ...
```

If real data is entered to this function a tag will automatically be treated as a chunk of realdata. Meaning that it will automatically get the attributes `type=*real_data*`, `size` and `width` (ie. `<tag type="realdata" size= width= >`).

- `xml_tag=>xml_tag_open(xml_tag,tagname)`

Adds to the current `xml_tag` a new tag with name `tagname` and returns a pointer to the new tag.

- `xml_tag=>xml_tag_close(xml_tag)`

Returns the a pointer to the tag that contains the current tag. to the new tag.

Example: Writing an xml document

Typical use to write an xml document: As an example we write an HTML table:

```
use xml_parser
type(xml_tag),pointer:: tag
!
tag=>xml_open('mydoc.xml')
tag=>xml_tag_open(tag,'table')
  tag=>xml_tag_open(tag,'tr')
    tag=>xml_tag_open(tag,'td')
      ... set up a string or data ...
      call xml_attribute(tag,'valign','20')
      call xml_attribute(tag,'textcolor','#AAAAAA')
      call xml_add_data(tag,string)
    tag=>xml_tag_close(tag)
  tag=>xml_tag_open(tag,'td')
    ... similar ...
  tag=>xml_tag_close(tag)
... etc ...
tag=>xml_tag_close(tag)
tag=>xml_tag_close(tag)
xml_write('mydoc.xml')
xml_close('mydoc.xml')
```

Example: Reading an xml document

Typical use to read an xml document: Let's get all the coordinates of a bunch of atoms:

```
use xml_parser
type(xml_tag),pointer:: tag
real,pointer:: coordinates(:, :)
character(len=...): atom_name
!
tag=>xml_open('mydoc.xml')
tag=>xml_read('mydoc.xml')
do while(xml_get_next(tag,'atom'))
  atom_name=xml_attribute(tag,'name')
  do while(xml_get_next(tag))
    if (xml_name(tag)=='coordinates') then
      coordinates=>xml_get_data(tag)
      write(*,*) 'Coordinates of ',atom_name,',:',coordinates
    end if
  end do
end do
xml_close('mydoc.xml')
```

xml_settings.h

This file is included in the fortran code and contain settings that fine-tune the performance and layout. The variables are described inside the file.

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/Corrientes2012/computer_dirac.md

orphan

DIRAC computer exercises

The following exercises are to be done in groups of 2-3 students each covering different groups/atoms of the periodic table of elements. At the end of the day, each group should give a short presentation on their findings to the other groups.

In order to run a DIRAC calculation you need an input file of arbitrary name, e.g. `dirac.inp` which will be setup in the following and which contains the computational directives. In addition, a second file is needed providing the molecular structure. DIRAC can handle two formats but for convenience we here focus on the standard **XYZ** molecular input file, named for example `molecule.xyz`. The run-command inside your cluster run script looks then as follows:

```
$ pam --noarch --inp=dirac.inp --mol=molecule.xyz
```

where the DIRAC output is re-directed to the file `dirac_molecule.out`. Make sure that you rename the output file with a proper name if you want to take a look at what you did back home.

More information on running DIRAC calculations can be found in the `FirstSteps` section of this documentation.

If not stated otherwise we will always use a Gaussian nuclear model in DIRAC.

Exercises - Day 3

In the following exercises we will do our (first) calculations in DIRAC looking at the effect on the total energy and the valence-shell spin-orbit splitting of a heavy-element atom while picking a different nuclear model, varying the actual speed of light as well as going to the non-relativistic limit.

the contender: Pb atom

A general .xyz file (see molecule_using_xyz) for an atom reads (replace here XX with Pb):

```
1
XX atom
XX 0.0 0.0 0.0
```

Atomic basis set calculations

In order to get started we calculate the ground state configuration and lowest excited states within the p s^2 manifold (s^3 P, s^1 D, s^1 S) of the Pb 6p block atom, using DIRAC and a double- ζ basis set.

The corresponding DIRAC input file (for downloading Pb.dirac.inp <Pb.dirac.inp>) is:

Pb.dirac.inp

[!NOTE] We explicitly give the electronic occupation (.CLOSED SHELL / .OPEN SHELL) which will active the average-of-configuration SCF. DIRAC has an automatic occupation mode but one should always carefully check the output for correctness.

[!NOTE] The convergence threshold (.EVCCNV) is reduced in order to speed-up the exercises. Note the keyword “.DOSSSS” which activates the (SSISS) integral classes to be explicitly calculated.

[!NOTE] “.RESOLVE” activates the open-shell state analysis module and the line “&GOSCIPIPRNT=5 &END” raises the print flag within this module.

Nuclear models in DIRAC

DIRAC offers two different nuclear models, run the p-block atom for the point-nucleus model as well as with a finite nucleus (Gaussian model).

- point nucleus:

```
**INTEGRALS
.NUCMOD
1
```

- finite nucleus (Gaussian, default in DIRAC except for .NONREL and .ECP):

```
**INTEGRALS
.NUCMOD
2
```

Question:

- Do you observe the same trends (total energy, spin-orbit splitting) as with the numerical atomic GRASP calculations on Day 1?

Speed of light and the non-relativistic limit

- vary speed of light c between $Z < c < \infty$:

```
**GENERAL
.CVALUE
137.000
```

- run a true non-relativistic calculation:

```
**HAMILTONIAN
.NONREL
```

Question:

- How does the latter calculation compare with a calculation using a large c value, $c \gg 137$?
- Does the p-shell become degenerate in the non-relativistic limit?

Exercises - Day 4

The Hamiltonian families

DIRAC offers a variety of Hamiltonians, ranging from non-relativistic, scalar-relativistic to “fully” relativistic. In the following we will examine the dependence of the ground state configuration, spin-orbit splitting and lowest excited state manifold of Pb on the choice of Hamiltonian.

- non-relativistic Hamiltonian(s):

```
**HAMILTONIAN
.NONREL
```

```
**HAMILTONIAN
.LEVY-LEBLOND
```

- scalar-relativistic Hamiltonians (just a selection, look at the DIRAC manual page for even more choices):


```

**HAMILTONIAN
.SPINFREE      ! Ken Dyall's spinfree Hamiltonian

**HAMILTONIAN
.DKH2          ! spinfree Douglas-Kroll-Hess 2nd order
.SPINFREE

**HAMILTONIAN
.X2C           ! spinfree exact two-component
.SPINFREE

**HAMILTONIAN
.BSS           ! spinfree exact two-component
109
.SPINFREE

**HAMILTONIAN
.BSS           ! spinfree Douglas-Kroll-Hess 2nd order
102
.SPINFREE

```

- relativistic (including spin-orbit coupling) Hamiltonians:

```

**HAMILTONIAN
! Dirac-Coulomb Hamiltonian in which (SS|SS) integrals are neglected
! and replaced by an interatomic SS correction (calculated as a
! classical repulsion term of (tabulated) small component atomic charges)

**HAMILTONIAN
.LVNEW ! Dirac-Coulomb Hamiltonian in which (SS|SS) integrals are neglected
! and replaced by an interatomic SS correction (with atomic small component
! charge calculated via a Mulliken analysis instead of a table look-up

**HAMILTONIAN
.DOSSSS ! the "full" Dirac-Coulomb Hamiltonian

**HAMILTONIAN
.DKH2 ! Douglas-Kroll-Hess second order;
! implies inclusion of 2-electron spin-same-orbit corrections (AMFI)

**HAMILTONIAN
.X2C ! "exact" two-component Hamiltonian;
! implies the inclusion of 2-electron spin-same-orbit corrections (AMFI).

**HAMILTONIAN
.X2C
.GAUNT ! include 2-electron spin-same and spin-other-orbit corrections.

**HAMILTONIAN
.X2C
.NOAMFI ! neglect 2-electron spin-orbit corrections.

**HAMILTONIAN
.DKH2
.NOAMFI ! neglect 2-electron spin-orbit corrections.

```

[!NOTE] We don't include the zeroth-order approximation (ZORA) to the Dirac-Coulomb Hamiltonian as in its present implementation in DIRAC it only works for closed-shell systems. In case you are interested to run ZORA calculations have a look at the HAMILTONIAN_.ZORA keyword in this documentation.

- *special* cases - the molecular-mean-field Hamiltonians:

[!NOTE] At present DIRAC does not have the functionalities to transform the "Gaunt integrals" to the molecular basis needed for post-Hartree-Fock methods. The following Hamiltonian combinations allow to approximate the Gaunt interaction in a "molecular-mean-field" fashion. We can likewise use this approach to approximate the Dirac-Coulomb Hamiltonian ["mean-field" for (SS|LL)+(SS|+ (SS|)SS) integrals]. In fact, there are even more molecular-mean-field Hamiltonian available (X2Cmmf) but current implementation restrictions allow those only for the Coupled Cluster module (see Day 4).

```

**HAMILTONIAN ! molecular mean-field approximation for the Dirac-Coulomb Hamiltonian: 4DC**
.DOSSSS
**MOLTRA
.INTFL2
1 1 1 0
.INTFL4
1 0 0 0

**HAMILTONIAN ! molecular mean-field approximation for the Dirac-Coulomb-Gaunt Hamiltonian: 4DCG**
.DOSSSS
.GAUNT
**MOLTRA
.INTFL2
1 1 1 1
.INTFL4
1 0 0 0

```

Questions:

1. How do the two-component (X2C, DKH2) and four-component results compare?
2. Does the inclusion of spin-other-orbit 2-electron interaction (Gaunt contributions) alter the spin-orbit splitting/excitation energies?
3. Is the inclusion of 2-electron spin-orbit corrections (here via "AMFI") important in 2-component spin-orbit calculations (X2C, DKH2)?
4. How do the non-relativistic and various scalar-relativistic ("SPINFREE") results compare?
5. There are two exact-two-component spinfree Hamiltonians listed. Do you obtain slightly different results and if, why?
6. Is the molecular-mean-field approach a reasonable approximation to the "full" 4-component Hamiltonian?

Further reading and literature

1. Dyll1994
2. Visscher2000
3. Ilias2007
4. Sikkema2009

Exercises - Day 5

The valence ground- and excited state manifold of Se 4^2

After the intense study of atoms with GRASP and DIRAC, we are ready to run our first molecular calculations. In this exercise we look at the zero-field splitting of the ground state and valence excitation manifold of Se 4^2 which is determined by (in approximate spin-orbit free notation) $(\pi^*)^2$ manifold: $3\Sigma^-_g$, $1\Delta_g$, $1\Sigma^+_g$.

Task:

1. Draw a molecular orbital diagram for the valence part of the Se 4^2 molecule. Hint: the valence configuration of Se is $4p^4$.

The xyz-molecular input file looks as follows (Se2.xyz <Se2.xyz>):

Se2.xyz

The SCF input reads as (Se2.dirac.inp <Se2.dirac.inp>)

Se2.dirac.inp

Repeat the calculation with the “X2C”, “X2C+Gaunt” and “Gaunt molecular-mean-field” Hamiltonian.

Questions

1. How does the zero-field splitting change with the inclusion of the Gaunt interaction?
2. Does the X2C Hamiltonian provide accurate results (compared to the fully relativistic calculation)?

A selection of properties in DIRAC

In our final part of the computer exercises we will look at some property calculations in DIRAC.

NMR shieldings

The first part concerns the calculation of NMR shieldings at the DFT level using the recently implemented simple magnetic balance scheme and the use of London orbitals in comparison with RKB and URKB calculations.

An illustrative tutorial including a brief introduction to the theory and a guideline for the exercise can be found on the DIRAC tutorial web page, [NMRshieldSimpleMagBal](#).

Density at the nucleus and the effective density

The second part illustrates the calculation of the density at the nucleus in comparison with the effective density (average density over the finite nucleus). We will look at the molecule HI (HI.xyz <HI.xyz>)

HI.xyz

The dirac input for the density at the nucleus then reads as (HI.dc.dirac.inp <HI.dc.dirac.inp>)

HI.dc.dirac.inp

Let's repeat this exercise with the X2C Hamiltonian, including AMFI corrections (HI.x2c.dirac.inp <HI.x2c.dirac.inp>)

HI.x2c.dirac.inp

As the last exercise in this course we will look at picture change errors when transforming from a 4-component to a 2-component formalism. This illustrates the importance of transforming the property operator when you change the “picture” you are working in! In DIRAC we have the option to **NOT** transform the property operators when doing an X2C calculation. The keyword is “NOPCTR” under “**PROPERTIES**”. The input file is (for download, HI.x2c.nopctr.dirac.inp <HI.x2c.nopctr.dirac.inp>):

HI.x2c.nopctr.dirac.inp

Questions:

1. Compare the effective density and density at the iodine center? What do you observe? Do they agree?
2. Picture-change errors: are they large for both, iodine and hydrogen?
3. Picture-change errors: is there a difference in order of magnitude of the effects for the dipole moment compared with the densities, and if so why?

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/Corrientes2012/computer_grasp.md

orphan

GRASP computer exercises

The following exercises are to be done in groups of 2-3 students. Each group shall cover different groups/atoms of the periodic table of elements. At the end of the day, all groups should give a short presentation on their findings to the other groups.

In order to run a GRASP calculation you need an input file of arbitrary name, e.g. `grasp.inp` which will be setup in the following. The run-command inside your cluster run script looks as:

```
$ grasp.x < grasp.inp > grasp.out
```

where the GRASP output is re-directed to the file `grasp.out`. Make sure that you give a proper name to your output file if you want to take a look at what you did back home.

Unless otherwise stated we will always use a Gaussian nuclear model in GRASP.

Exercises - Day 1

Nuclear models in GRASP

The electron-nucleus interaction in the electronic Hamiltonian is $V_{en} = -e\phi_{nuc}$ where ϕ_{nuc} is the nuclear scalar potential

$$\phi_{nuc}(\mathbf{r}_1) = \frac{1}{4\pi\epsilon_0} \int \frac{\rho_{nuc}(\mathbf{r}_2)}{|\mathbf{r}_1 - \mathbf{r}_2|} d^3\mathbf{r}_2$$

GRASP offers four different models for the nuclear charge distribution ρ_{nuc} : the point charge, the homogeneously charged sphere, the Fermi 2-parameter charge distribution and the Gaussian charge distribution. You find more information about these models [at this URL](#). (Note that there is a systematic error in this source in that the potentials come with a negative sign whereas this comes from multiplication of the electron charge to give the electron-nucleus interaction).

We will start with calculations on a one-electron system for each of the four nuclear models. The corresponding downloadable Grasp input file for is `grasp_1el_atom.inp` <`grasp_1el_atom.inp`>.

This is how you change the nucleus model: * point nucleus:

```
NUC 0
```

- finite nucleus (Uniform Sphere):

```
NUC 1
```

- finite nucleus (Fermi):

```
NUC 2
```

- finite nucleus (Gaussian):

```
NUC 3
```

Questions:

1. How does the use of a finite-nucleus model affect the total energy of an atom compared to the point-nucleus model?
2. What is the effect of nucleau model on the spin-orbit splitting?
3. Do differences between finite-nuclear models increase or decrease with the increasing atom's nuclear charge Z ?

Exercises - Day 2

[!NOTE] A database for atomic data can be found [here](#).

Atomic numerical calculations

In order to get started on “real” atoms we calculate the average level structure of two possible ground state configurations of coinage-metals (Cu, Ag, Au, Rg), using GRASP. The input file for download is `grasp_Cu.inp` <`grasp_Cu.inp`>.

Run all atoms of the coinage-metal series:

```
Cu: Z = 29, A = 63
Ag: Z = 47, A = 107
Au: Z = 79, A = 197
Rg: Z = 111, A = 283
```

In order to generate a less biased average set of orbital solutions (degeneracy of the $d^{10}s^2$: 10; $d^{10}s^1$: 2), compare your results with results from an additional calculation using:

```
AL 10 2
```

[!NOTE] We explicitly set the maximum number of SCF iterations (“SCF NITIT=10”) in order to allow Cu to converge (default SCF iteration = 6)

Questions:

1. The two candidates for the ground state configuration are $(n-1)d^{10}ns^1$ and $(n-1)d^9ns^2$. For each configuration what are the possible values of total angular momentum J ? Your answer will help you identify the ground state configuration by looking at the list of energies of the individual atomic levels.
2. Does the ground state configuration vary down the coinage-metal series?
3. How does the d-s excitation energy ($(n-1)d^{10}ns^1 \rightarrow (n-1)d^9ns^2$) change down the coinage-metal series?

Spin-orbit mixing within a group

In this exercise we shall investigate how spin-orbit mixing evolves as we descend a column of the periodic table. We will consider group 14, that is, the carbon group. A sample input, for silicium is given `grasp_Si.inp <grasp_Si.inp>`.

The $3p^2$ electron configuration of silicium gives rise the three LS-states: 3P , 1D and 1S . With the introduction of spin-orbit coupling they are split into 3P_2 , 3P_1 , 3P_0 , 1D_2 and 1S_0 (verify this !) and states with the same J may interact and mix. This means for instance that 3P_2 and 1D_2 interact, leading to singlet-triplet mixing, forbidden in a non-relativistic framework. There is also possibility of mixing between 3P_0 and 1S_0 .

In order to understand how to proceed, let us first look at what an average level (AL) calculation means:

In the GRASP input we specify one or more electron configurations. Assume that we generate all possible Slater determinants $\left|\Phi_I\right\rangle$ corresponding to the configurations, and then build and diagonalize the Configuration Interaction (CI) matrix in the space of the Slater determinants

$$H_{\mathbf{K}=\mathbf{E}_\mathbf{K}} \quad H_{\{IJ\}} = \left\langle \Phi_I \left| H \right| \Phi_J \right\rangle$$

This gives us a set of electronic states expressed in terms of atomic state functions

$$\Psi_K = \sum_I^N \Phi_I c_{IK}$$

whose number is determined by the degeneracy of the state and where N refers to the total number of Slater determinants and therefore number of microstates/atomic state functions.

In order to build the CI matrix we need orbitals. Rather than optimizing them for an individual electronic state we shall determine them by optimizing the *average energy*

$$E^{\text{av}} = \frac{1}{N} \sum_K \left\langle \Psi_K \left| H \right| \Psi_K \right\rangle$$

This looks rather impossible because the average energy contains the atomic state function that we do not know until we have diagonalized the CI matrix, which we can not build before we have orbitals available. However, if we expand the atomic state functions in terms of determinants we obtain

$$E^{\text{av}} = \frac{1}{N} \sum_K \sum_I^N \sum_J^N \left\langle \Phi_I \left| H \right| \Phi_J \right\rangle c_{IK}^* c_{JK} = \frac{1}{N} \sum_K \left\langle \Phi_K \left| H \right| \Phi_K \right\rangle$$

We arrive at the final result because the atomic state functions are orthonormal such that the expansion coefficients c_{IK} form a unitary matrix. Now the average energy is expressed in terms of our set of Slater determinants and we may proceed. In fact, GRASP will first optimize the orbitals by an SCF procedure based on the above expression and then solve the CI problem to get the individual atomic state functions and atomic level energies.

The atomic state functions have well-defined total angular momentum J . In the “ANG” input section of GRASP we specify the desired J for each electron configuration. If we want all possible values, we simply set “-1” as in the present case. A single Slater determinant is in general not an eigenfunction of J^2 . GRASP therefore constructs configuration functions (CSFs), that is, linear combinations of Slater determinants with well-defined J and parity. These are by default listed in the output, but in the exercises above we had set “ANG 7” to suppress this output. With the above input we find for instance that the first CSF is generated from the valence configuration $3p_{1/2}^2$ and has $J=2$ and even parity. At the end of the GRASP output the composition of atomic state functions in terms of CSFs is listed. Since the CSFs are orthonormal the weight CSF of index I to an atomic state function of index K is simply c_{IK}^2 .

In order to investigate singlet-triplet mixing we also need to know the composition of atomic state functions in terms of LS-coupled CSFs. The third number “2” in line 2 in the above input makes GRASP also set up CSFs in LS-coupling. The non-relativistic CSFs are listed after the relativistic ones. With the present input we find that the first non-relativistic CSF couples to 3P_2 . The weight of non-relativistic CSFs to atomic state functions is listed after the relativistic ones.

Questions:

1. Does the singlet-triplet mixing (*state interaction*) increase or decrease with increasing Z ? You can investigate this by monitoring the mixing of 3P_2 and 1D_2 in the lowest atomic level of $J=2$, making a graph showing how the weight of 1D_2 evolves with increasing Z .
2. How much of the ground state is described by the $np_{1/2}^2$ configuration with increasing Z ? You can investigate this by monitoring the composition of the ground-state in terms of relativistic CSFs. You can calculate the effective occupation number of the $np_{1/2}$ orbital by, for each CSF, multiplying its occupation by the weight of the CSF, then summing up these numbers. You can then plot the effective occupation number of the $np_{1/2}$ orbital as a function of increasing Z . How do you interpret your result ?

Speed of light and the non-relativistic limit

- Run an average-of-configuration SCF calculation on the gold atom; downloadable input `grasp_Au.inp <grasp_Au.inp>`.

- GRASP allows you to vary the speed of light, c , between $Z < c < \infty$:

```
SCF CON=x, xDx
```

- perform a non-relativistic calculation using a very large c value, $c \gg 137$:

```
SCF CON=1.0D6
```

Questions:

1. Is it possible to set $c < Z$?
2. Can you argue from a comparison of the relativistic (“regular” speed-of-light) and non-relativistic calculation why gold is *golden*? What about Ag?
3. Which spinors are stabilized by relativistic effects and which are destabilized?

Radial densities for the d-/p block

- Plot radial density for heavy elements, compare s and d functions (d block) resp. $p_{1/2}$ and $p_{3/2}$ (p block).

A simple FORTRAN code `orbpri.F` for doing it is available. Here we shall demonstrate how to use it for the copper atom. First we remind you that the form of relativistic atomic orbitals is

$$\begin{aligned} \left\langle \begin{array}{c} r \\ \chi_{\kappa, m} \end{array} \right| \chi_{\kappa, m} \rangle &= \frac{1}{r} \left\langle \begin{array}{c} r \\ \chi_{\kappa, m} \end{array} \right| \chi_{\kappa, m} \rangle \end{aligned}$$

so the radial density is

$$n(r) = \left(\left(R^L \right)^2 + \left(R^S \right)^2 \right) r^2 = \left(P^2 + Q^2 \right).$$

The input file for download is grasp_Cu_aver.inp <grasp_Cu_aver.inp>.

The line ORBOUT specifies that the radial functions \$P\$ and \$Q\$ (and other information) is printed to file. This file will have unit number 11, as specified by the final number following the MCDF line.

We start up orbpri.x:

```
$orbpri.x
*** OUTPUT from ORBPRI ***
Give unit number of orbital file to read
```

and we enter unit number 11:

```
11
*** OUTPUT from ORBPRI ***
* Atomic number:      29.0
* Number of grid points: 2080
* First grid point:   0.345E-07
* Grid step size :    0.100E-01
```

```
* Number of orbitals read: 10
```

Do you want info on orbitals (y/n) ?

Well, why not ?:

y	Orb.	E	P0	Q0
1	1s	-332.845231246527	353.946570713740	0.000000000000
2	2s	-41.883350988707	110.925718644588	0.000000000000
3	2p-	-36.527466295898	0.975314932490	9.217465299856
4	2p	-35.763812849690	769.447328175348	0.000000000000
5	3s	-5.353239297458	41.725471888184	0.000000000000
6	3p-	-3.614445494458	0.362775735823	3.428505650010
7	3p	-3.511783710417	286.145123083999	0.000000000000
8	3d-	-0.674391837066	0.265542615160	5.019157934192
9	3d	-0.661421437065	264.801381125017	0.000000000000
10	4s	-0.289259556935	9.151716417086	0.000000000000

Select one of the following:

1. Print radial functions P and Q.
2. Print radial functions R(large) and R(small).
3. Print radial distribution of individual orbitals.
4. Exit.

We choose to look at the density of individual orbitals:

```
3
Name atom (A3)
```

The rest is then rather straightforward:

```
Cu
Outer radial point = : 36.862
Give index of orbital to print(-1 to quit):
6
Give index of orbital to print(-1 to quit):
7
Give index of orbital to print(-1 to quit):
8
Give index of orbital to print(-1 to quit):
9
Give index of orbital to print(-1 to quit):
10
Give index of orbital to print(-1 to quit):
-1
*** EXITING ORBPRI ***
```

Looking in your working directory you now see a series of files with self-explanatory titles. With your favorite plotting program you may now make a plot like this one

 Copper radial density
Copper radial density

Each group should pick one of their favorite block/group.

```
calculate the 3d, 4d, 5d, 6d block (coinage metals)
calculate the 3d, 4d, 5d, 6d block (group: "Zn, Cd, Hg, Cn")
calculate the 2p, 3p, 4p, 5p, 6p, 7p block (group 14)
calculate the 2p, 3p, 4p, 5p, 6p, 7p block (group 16)
```

Questions:

1. How does the radial overlap between s and d resp. $p_{1/2}$ and $p_{3/2}$ change down a group?
2. Where would you expect the most open-shell character (at the top or bottom of a group)?

Using a non-relativistic or relativistically optimized basis set for heavy atoms?

- run the Thallium atom with a non-relativistic double- ζ basis set (download the input file grasp_Tl_dz_nonrel.inp <grasp_Tl_dz_nonrel.inp>)
- run the Thallium atom with a relativistic double- ζ basis set: (download the input file grasp_Tl_dz_rel.inp <grasp_Tl_dz_rel.inp>)

Questions:

1. How does the spin-orbit splitting from both basis set calculation compare with the experimental reference?
2. Can you recommend to use a non-relativistically optimized basis set in relativistic (four-component) calculation? Why/why not?
3. What are the major qualitative differences between both basis sets?

Basis set quality

- run the Bi atom numerically (if not already done in [Radial densities for the d-/p block](#)) (input file for download, grasp_Bi.inp <grasp_Bi.inp>);
- repeat the calculation using Ken Dyall's
 - double- ζ basis set (input file for download, grasp_Bi_dz.inp <grasp_Bi_dz.inp>)
 - triple- ζ basis set (input file for download, grasp_Bi_tz.inp <grasp_Bi_tz.inp>)
 - quadruple- ζ basis set (input file for download, grasp_Bi_qz.inp <grasp_Bi_qz.inp>)

Questions:

1. Look at the ground and lowest excited states - does a double- ζ basis set suffice to reach a good agreement with the numerical solution?
2. Compare with experiment!
3. R.95 values as a measure for the density at the nucleus: would a quadruple- ζ basis set be large enough to reach agreement with the numerical reference value? What may be missing?

Using *j*- or *l*-optimized basis sets ?

- use a *j*-optimized basis set to calculate the spin-orbit splitting of the ground state of element 113. (input file for download, grasp_113_j-dz.inp <grasp_113_j-dz.inp>)
- use an *l*-optimized double- ζ basis set to calculate the spin-orbit splitting of the ground state of element 113. (input file for download, grasp_113_l-dz.inp <grasp_113_l-dz.inp>)

Questions:

1. Can you come up with an explanation for the differences in the spin-orbit splitting calculated with an *j*- and *l*-optimized basis set ?
2. Which calculation yields a value closer to the numerical reference and why ?

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/Corrientes2012/computer_introduction.md

orphan

Introduction to Computer Exercises

The following exercises are to be done in groups of 2-3 students each covering different groups/atoms of the periodic table of elements. At the end of a day, each group should give a short presentation on their findings to the other group.

The computer exercises comprise atomic numeric and basis set calculations with the computer code GRASP. In addition, we will use the computer code DIRAC to perform atomic and molecular energy and property calculations. The scope of the course is to introduce the use and handling of general-purpose relativistic quantum chemistry codes to the participants allowing them to use these codes in their every-day research work. Questions for each exercises will guide the students to extract the essential results from the calculations.

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/Corrientes2012/index.md

orphan

PhD course - Introduction to Relativistic Quantum Chemistry

These exercises are based on the PhD course developed and held by S. Knecht, H. J. Aa. Jensen, and K. G. Dyall in Corrientes, Argentina, 2012, prior to the REHE 2012 conference.

This electronic web material can be part of any workshops on relativistic quantum chemistry, where theory lessons are coupled with Grasp and DIRAC computer exercises.

Pen-and-Paper Exercises

pen_and_paper.rst

Computer Exercises

computer_introduction.rst computer_grasp.rst computer_dirac.rst

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/Corrientes2012/pen_and_paper.md

orphan

The course comprises two pen-and-paper exercises which are based on materials of the morning-session lectures of Day 1.

Dirac's relation

A relation that is often exploited throughout the course, e.g. to achieve a separation of spin-dependent and independent terms in the Dirac equation (see lecture on Day 2), is Dirac's relation Dirac1928, which can be written for two arbitrary vector operators $\vec{u}$ and $\vec{v}$ as:

$$(\vec{\sigma} \cdot \vec{u})(\vec{\sigma} \cdot \vec{v}) = \vec{u} \cdot \vec{v} I_2 + i \vec{\sigma} \cdot (\vec{u} \times \vec{v})$$

where $\vec{\sigma}$ are the Pauli spin matrices and I_2 is a 2×2 unit matrix. Note that $\vec{u}$ and $\vec{v}$ do not necessarily commute.

- Problem 1:** Verify the relation expanding the left-hand side of Dirac's relation. Hint: use the relations for the Pauli spin matrices:

$$\begin{aligned} \vec{\sigma} &= (\sigma_x, \sigma_y, \sigma_z) \\ \sigma_x^2 &= \sigma_y^2 = \sigma_z^2 = I_2 \\ \sigma_i \sigma_j &= \delta_{ij} I_2 + i \epsilon_{ijk} \sigma_k \end{aligned}$$

Note that we use the Einstein summation convention; in the final expression above the index k appears twice in the same term, which implies that we sum over it.

- Problem 2:** derive a final expression inserting for $\vec{u} = \vec{v}$ the kinematical momentum operator $\vec{\pi} = \vec{p} + e\vec{A}$ (where $\vec{A}$ is an external electromagnetic vector potential).

Two-component Pauli equation (0th order Pauli equation)

From the one-electron Dirac equation in external magnetic fields we may derive the Pauli equation (also known as 0th order Pauli equation) by considering the non-relativistic limit $c \rightarrow \infty$.

$$\left[\frac{(\vec{p})^2}{2m_e} + \frac{e^2}{2m_e} \vec{A}^2 + \frac{e}{m_e} (\vec{\sigma} \cdot \vec{B}) + V \right] \psi = i \hbar \frac{\partial}{\partial t} \psi$$

- Problem 3:** Derive the above equation starting from the one-electron Dirac equation in external magnetic fields written in two-spinor form:

$$\begin{aligned} & i \hbar \frac{\partial}{\partial t} \begin{pmatrix} \psi^L \\ \psi^S \end{pmatrix} = c \begin{pmatrix} \vec{\sigma} \cdot \vec{\pi} \\ \vec{\pi} \cdot \vec{\sigma} \end{pmatrix} \begin{pmatrix} \psi^L \\ \psi^S \end{pmatrix} \\ & + m_e c^2 \begin{pmatrix} \psi^L \\ -\psi^S \end{pmatrix} + V \begin{pmatrix} \psi^L \\ \psi^S \end{pmatrix} \end{aligned}$$

The following hints may be useful:

1. shift the zero of energy to the non-relativistic limit one. This corresponds to the elimination of rest mass and is achieved by the substitution

$$V \rightarrow V - m_e c^2$$

2. eliminate the small-component ψ^S from the upper component using the magnetic balance condition:

$$\psi^S \approx \frac{\vec{\sigma} \cdot \vec{\pi}}{2m_e} \psi^L$$

3. use $\vec{A} \cdot \vec{p} = \vec{p} \cdot \vec{A} - (\vec{p} \times \vec{A}) \cdot \vec{\sigma}$

and remember that in Coulomb gauge we have

$$\vec{\nabla} \cdot \vec{A} = 0$$

4. for a constant and homogeneous magnetic field $\vec{B} = \vec{\nabla} \times \vec{A}$, we may write $\vec{A} = \frac{1}{2} (\vec{B} \times \vec{r})$

- Problem 4:** define the *Bohr magneton* and the *gyromagnetic ratio* g of the electron according to the Pauli Hamiltonian.

Literature and further reading

- Dyall2007, Chapter 4.
- Reiher2009, Chapter 5.

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/TCCM2021/computer_dirac.md

orphan

Introduction

In order to run a DIRAC calculation you need an input file of arbitrary name, e.g. `dirac.inp` which will be setup in the following and which contains the computational directives. In addition, a second file is needed providing the molecular structure. DIRAC can handle two formats but for convenience we here focus on the standard **XYZ** molecular input file, named for example `molecule.xyz`. The run-command inside your cluster run script looks then as follows:

```
$ pam --noarch --inp=dirac.inp --mol=molecule.xyz
```

where the DIRAC output is re-directed to the file `dirac_molecule.out`. Make sure that you rename the output file with a proper name if you want to take a look at what you did back home.

More information on running DIRAC calculations can be found in the **FirstSteps** section of this documentation.

If not stated otherwise we will always use a Gaussian nuclear model in DIRAC.

[!NOTE] It is important that you understand the inputs. Before running calculations, carefully go through the input with reference to the manual that you find on the [DIRAC pages](#)

[!NOTE] You are expected to make a report presenting your results, also including some theoretical background as well as computational details.

Atomic calculations

[!NOTE] **Exercise A** In these exercises we look at relativity in atomic systems. Three columns of the periodic table will be explored: 1. From group 2: Mg, Sr and Ra 2. From group 12: Zn, Cd and Hg 3. From group 18: Ne, Kr and Rn

The exercises are: 1. Perform 4-component HF-calculations on your series of three atoms. We focus on the *outermost* p orbitals: i) How does spin-orbit splitting vary as a function of nuclear charge Z ? ii) How do spin-orbit interaction affect orbital sizes $\langle r^2 \rangle^{1/2}$ of spin-orbit partners down the series ? 2. Perform 4-component *spin-free* and *non-relativistic* HF-calculations on the heaviest atom of your series in order to investigate the effects of spin-orbit coupling and scalar relativistic effects. i) Make a table showing orbital energies and orbital sizes of all orbitals for each of the three Hamiltonians. ii) What orbitals contract (stabilize) and expand (destabilize) ? iii) Plot the densities of the *outermost* p orbitals with the three Hamiltonians. 3. Perform 2-component relativistic HF-calculations on the heaviest atom of your series using the following Hamiltonians: i) X2C ii) second-order Douglas-Kroll-Hess iii) ZORA and iv) scaled ZORA. Make a table of energies of all orbitals obtained with these Hamiltonians comparing with those obtained with the 4-component Dirac-Coulomb Hamiltonian. Helpful information for these exercises is given below !

We shall start gently by looking at atoms. It will be useful to recall the form and notation for atomic orbitals. In the non-relativistic case atomic orbitals have the general form

$$\psi_{n\ell m}(\mathbf{r}) = R_{n\ell}(r) Y_{\ell m}(\theta, \phi)$$

where appears the principal quantum number n , the azimuthal quantum number ℓ (orbital angular momentum) and the magnetic quantum number m . Orbitals are indicated using the letter code $s, p, d, f, \dots$ for orbital angular momentum $\ell = 0, 1, 2, 3, \dots$. Complete specification requires multiplying these spatial functions with spin α or β functions

In the 2-component relativistic case, the orbitals have the general form

$$\psi_{n\kappa m_j}(\mathbf{r}) = R_{n\kappa}(r) \chi_{\kappa m_j}(\theta, \phi)$$

where $\chi_{\kappa m_j}$ are 2-component angular functions. The quantum number κ requires some explanation. Spin-orbit interaction leads to a coupling of orbital and spin angular momentum to form total angular momentum $\mathbf{j} = \mathbf{\ell} + \mathbf{s}$. The associated quantum number j can take the values $j = \ell + 1/2$ and $j = \ell - 1/2$. However, just giving j does not specify an orbital uniquely, since it can for instance not distinguish $s_{1/2}$ and $p_{1/2}$ orbitals (both have $j = 1/2$). This is possible with the quantum number κ according to the following table:

	$s_{1/2}$	$p_{1/2}$	$p_{3/2}$	$d_{3/2}$	$d_{5/2}$	$f_{5/2}$	$f_{7/2}$	
κ	-1	+1	-2	+2	-3	+3	-4	

4-component atomic orbitals have separate radial and angular parts for both the large and small components

$$\begin{aligned} \psi_{n\kappa m_j}(\mathbf{r}) &= \begin{bmatrix} \psi_{n\kappa m_j}^L \\ \psi_{n\kappa m_j}^S \end{bmatrix} \\ &= \begin{bmatrix} R_{n\kappa}^L(r) \chi_{\kappa m_j}(\theta, \phi) \\ R_{n\kappa}^S(r) \chi_{-\kappa m_j}(\theta, \phi) \end{bmatrix} \end{aligned}$$

with the notation for orbitals dictated by the quantum number of the large component.

With these introductory remarks in mind, we are ready to do our first calculation.

We will run a Hartree-Fock calculation on the neon atom. We shall use the input file `Ne.inp` (download [Ne.inp](#)):

```
Ne.inp
```

and the xyz-file `Ne.xyz` (download [Ne.xyz](#)):

```
Ne.xyz
```

Neon has the ground-state configuration $1s^2 2s^2 2p^6$. Note that under the keyword (SCF .CLOSED SHELL) we specify the occupation of *gerade* and *ungerade* orbitals separately, that is:

```
.CLOSED SHELL
4 6
```


For this particular calculation we have chosen to use the dyall.2zp, which is a double zeta basis set for SCF calculations:

```
*BASIS
.DEFAULT
dyall.2zp
```

The minimal command for running this calculations is:

```
pam --inp=Ne.inp --mol=Ne.xyz
```

Looking in the output, we find information about the SCF-cycles and the final energy

Ne.scf

The entry *SS Coulombic correction* refers to a default approximation in which the calculations of the expensive $\langle SS|SS \rangle$ two-electron integrals are replaced by a simple energy correction see Visscher1997a for further details. It is strictly zero for atoms.

The final energy is followed by a list of orbital eigenvalues. In the present calculation we have activated Mulliken population analysis and find further information about the occupied orbitals in the output

Ne.pop

Note again the separation on *gerade* and *ungerade* orbitals. We shall focus on the valence p orbitals of neon and the heavier homologues. In the present calculation we note that the $2p_{1/2}$ and $2p_{3/2}$ orbitals have energies $-0.851172 E_h$ and $-0.846688 E_h$, respectively, corresponding to a spin-orbit splitting of $0.004484 E_h$, or 984.2 cm^{-1} (the conversion factor is $2.19474625E+5$).

We have also chosen to have get some information about orbital sizes. We do not have direct access to radial expectation values $\langle r \rangle$, but can calculate the rms $\langle r^2 \rangle^{1/2}$ by calculating $\langle r^2 \rangle = \langle x^2 \rangle + \langle y^2 \rangle + \langle z^2 \rangle$. This is specified in the input as:

```
*EXPECTATION VALUE
.ORBANA
.OPERATOR
'XXSECMOM'
.OPERATOR
'YYSECMOM'
.OPERATOR
'ZZSECMOM'
```

(see `one_electron_operators` for syntax). We have also used the keyword `.ORBANA` to ask for contributions from individual orbitals. In the output we therefore for instance find

Ne.xx

One column gives the matrix elements $\langle \varphi_i | \hat{r}^2 | \varphi_j \rangle$ for occupied orbitals φ_i , whereas the final column gives the matrix element multiplied with the occupation number of the orbital.

We are interested in the rms of the valence p orbitals. We make the following table

Orb.	$\langle x^2 \rangle$	$\langle y^2 \rangle$	$\langle z^2 \rangle$	$\langle r^2 \rangle$	$\langle r^2 \rangle^{1/2}$
E1u 1	0.405856904615	0.405856904615	0.405856904615	1.218	1.103
E1u 2	0.489542686491	0.489542686491	0.244771343246	1.223	1.106
E1u 3	0.326361790990	0.326361790990	0.571133134233	1.223	1.106

From the Mulliken analysis output we see that the three listed orbitals correspond to $2p_{1/2,1/2}$, $2p_{3/2,1/2}$, and $2p_{3/2,1/2}$, respectively, where the second subscript corresponds to the value of l_m . We see that the $2p_{3/2}$ orbitals all have the same size ($\langle r^2 \rangle^{1/2} = 1.106 a_0$) and are slightly bigger than the spin-orbit partner $2p_{1/2}$ ($\langle r^2 \rangle^{1/2} = 1.103 a_0$).

It may also be of interest to have a look at the shape of the atomic orbitals. Here things are somewhat more complicated than in the non-relativistic case since we have seen that the orbitals are complex 4-component vector functions. What is possible is to plot orbital densities, see `orbital_densities` for instructions.

So far we have done 4-component relativistic calculations. It is possible to successively turn off spin-orbit interaction and scalar relativistic effects using the keywords `HAMILTONIAN_.SPINFREE` and `HAMILTONIAN_.LEVY-LEBLOND`. The latter calculation is based on the Lévy-Leblond Hamiltonian, which is a 4-component *non-relativistic* Hamiltonian.

It is also possible to explore various approximate 2-component Hamiltonians. We shall be interested in the following ones:

1. The eXact 2-Component Hamiltonian (X2X). Specification:

```
**HAMILTONIAN
.X2C
```

2. The ZORA Hamiltonian. Specification:

```
**HAMILTONIAN
.ZORA
0 0
```

3. The scaled ZORA Hamiltonian. Specification:

```
**HAMILTONIAN
.ZORA
0 1
```

4. The second-order Douglas-Kroll-Hess Hamiltonian Specification:

```
**HAMILTONIAN
.DKH2
```

Molecular calculations

[!NOTE] **Exercise M** : In these exercises we look at relativistic effects on the spectroscopic constants of noble metal monohydrides. Experimental data can be found at [these NIST pages](#) by giving the chemical formula and selecting *Constants of diatomic molecules*. Note that we are looking at spectroscopic constants for the ground state, normally denoted X and found at the bottom of the table. Three different DFT functionals will be explored: 1. B3LYP 2. PBE 3. PBE0

The exercises are: 1. Perform 4-component DFT-calculations on the monohydrides of the noble metals Cu, Ag and Au in order to investigate the effects of spin-orbit coupling and scalar relativity (again using the keywords HAMILTONIAN_.SPINFREE and HAMILTONIAN_.LEVY-LEBLOND) on equilibrium distances r_e , harmonic frequencies ω_e and anharmonic constants $\omega_e x_e$. 2. For AuH, explore the non-relativistic limit by recalculating spectroscopic constants setting $c=4000$ a.u. 3. For CuH, explore the *ultra-relativistic* limit by recalculating spectroscopic constants setting $c=20$ a.u. Helpful information for these exercises is given below !

As a sample calculation, let us look at CuH. These are simple diatomic species, so we shall simply obtain spectroscopic constants by calculating the energy at a selection of interatomic distances about the expected minimum. We can then calculate the equilibrium distance r_e , harmonic frequency ω_e and anharmonic constant $\omega_e x_e$ using the utility program `twofit`. The manual pages provide theoretical background. The executable `twofit.x` is in the same directory as the DIRAC executable.

We will run a DFT calculation on CuH, here using the B3LYP functional. We shall use the input file `CuH.inp` (download <CuH.inp>):

`CuH.inp`

and the xyz-file `CuH.xyz` (download <CuH.xyz>):

`CuH.xyz`

The minimal command for running this calculations is:

```
pam --inp=CuH.inp --mol=CuH.xyz
```

Looking in the output, we find information about the SCF-cycles and the final energy

`CuH.scf`

We may now see that the entry *SS Coulombic correction* is non-zero, albeit very small. In this particular case the energy correction is calculated as

$$E_{\text{SS}}^{\text{inter}} = \frac{q^{\text{S}}_{\text{Cu}} q^{\text{S}}_{\text{H}}}{R_{\text{Cu-H}}}$$

where q^{S} is the contribution of the small components to the atomic charge.

In passing we note that the output contains an xyz-file

`CuHmod.xyz`

which is modified compared to the input file. This is because DIRAC checked the original xyz-input and found linear symmetry

`CuH.sym`

DIRAC uses *linear supersymmetry* in these calculations; this means that the underlying integrals are adapted to the C_{2v} subgroup, but the Fock matrix has the full block structure due to linear symmetry. In the same manner, the atomic calculations above uses atomic supersymmetry.

We are now ready to scan the potential surface of CuH in the vicinity of the expected minimum. A pedestrian way is to run the calculation point by point, modifying the input xyz-file for each new geometry. However, it is also possible to automatize the scan, e.g. (using bash)

`CuH.script`

Total energies can then be extracted for instance using:

```
grep 'Total energy' CuH.*out > CuH.pot
```

After editing the file `CuH.pot` (download <CuH.pot>) reads:

`CuH.pot`

A polynomial fit of order 6, using `twofit` (output here <CuH.pot.twofit>), gives the spectroscopic constants $r_e = 1.4604$ Å, $\omega_e = 1958.2$ cm⁻¹ and $\omega_e x_e = 34.6$ cm⁻¹. From the NIST page we find $r_e = 1.46263$ Å, $\omega_e = 1941.26$ cm⁻¹ and $\omega_e x_e = 37.51$ cm⁻¹, so our results are not bad !

We have seen that relativistic effects are dictated by the Lorentz factor

$$\gamma = \frac{1}{\sqrt{1 - \frac{v^2}{c^2}}}$$

where v is the speed of the particle and c is the speed of light. We have also seen that for hydrogen-like atoms the speed of the $1s$ orbital is Z (nuclear charge) in atomic units, to be compared with the speed of light which is around 137 in the same units. The non-relativistic limit can be attained by setting the speed of light to infinity. This is not possible on a computer, so we settle for a large, but finite value. It is also possible to investigate the *ultra-relativistic* case by lowering the speed of light. In DIRAC the speed of light can be adjusted using the GENERAL_.CVALUE keyword.

[!NOTE] When extracting spectroscopic constants using `twofit` it is important that the minimum r_e is in the interval of selected points, ideally somewhere in the middle, so you may have to add further points to your scan to achieve this. Relativistic effects are important, so when searching for an

unknown minimum, you may first make a rather coarse grid of points (spacing 0.1 Å) and use `twofit` to guide you in the right direction, where you can put a finer grid (spacing 0.01 Å).

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/TCCM2021/index.md

orphan

Exercises - Relativistic Quantum Chemistry (TCCM 2021)

These exercises are based on the course given by Trond Saue for The European Master in Theoretical Chemistry and Computational Modelling (TCCM).

This electronic web material can be part of any workshops on relativistic quantum chemistry, where theory lessons are coupled with DIRAC computer exercises.

Before embarking on these exercises, it is strongly recommended to review the lectures notes ! Further theoretical background, in particular about Hamiltonians, is found in Trond Saue, *ChemPhysChem* **12** (2011) 3077 ([pdf](#)).

Pen-and-Paper Exercises

pen_and_paper.rst

Computer Exercises

computer_dirac.rst

FILE: www.diracprogram.org/doc/release-26/_sources/exercises/TCCM2021/pen_and_paper.md

orphan

In the text full units are used for clarity, but in practice one can employ SI-based atomic units, setting $\hbar = m_e c = 4\pi \text{varepsilon}_0 = 1$.

Pauli spin matrices

The Pauli spin matrices

```
\begin{aligned}
\sigma_x &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \\
\sigma_y &= \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \\
\sigma_z &= \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}
\end{aligned}
```

are representation matrices of the operator $\mathbf{s}$ in the basis $\left\{ \left| \alpha \right\rangle, \left| \beta \right\rangle \right\}$ of spin-1/2 functions (*spin-up* and *spin-down*, respectively), that is $\mathbf{s} \rightarrow \hbar \boldsymbol{\sigma}$. These spin functions obey the following relations

```
\begin{aligned}
&\begin{array}{l}
\hat{s}_z \left| \alpha \right\rangle = +\frac{1}{2} \hbar \left| \alpha \right\rangle \quad \hat{s}_z \left| \beta \right\rangle = -\frac{1}{2} \hbar \left| \beta \right\rangle \\
\hat{s}_+ \left| \alpha \right\rangle = 0 \quad \hat{s}_+ \left| \beta \right\rangle = \hbar \left| \alpha \right\rangle \\
\hat{s}_- \left| \alpha \right\rangle = \hbar \left| \beta \right\rangle \quad \hat{s}_- \left| \beta \right\rangle = 0
\end{array}
\end{aligned}
```

where appears the ladder operators $s_{\pm} = s_x \pm i s_y$. The spin-1/2 functions are furthermore orthonormal. Let us see how the Pauli σ_x matrix is generated:

- Element (1,1): $\langle \alpha | \sigma_x | \alpha \rangle = \langle \alpha | \frac{1}{2} (\sigma_+ + \sigma_-) | \alpha \rangle = \frac{1}{2} (\langle \alpha | \sigma_+ | \alpha \rangle + \langle \alpha | \sigma_- | \alpha \rangle) = \frac{1}{2} (0 + \hbar) = \frac{\hbar}{2}$
 - Element (1,2): $\langle \alpha | \sigma_x | \beta \rangle = \langle \alpha | \frac{1}{2} (\sigma_+ + \sigma_-) | \beta \rangle = \frac{1}{2} (\langle \alpha | \sigma_+ | \beta \rangle + \langle \alpha | \sigma_- | \beta \rangle) = \frac{1}{2} (0 + \hbar) = \frac{\hbar}{2}$
 - Element (2,1): $\langle \beta | \sigma_x | \alpha \rangle = \langle \beta | \frac{1}{2} (\sigma_+ + \sigma_-) | \alpha \rangle = \frac{1}{2} (\langle \beta | \sigma_+ | \alpha \rangle + \langle \beta | \sigma_- | \alpha \rangle) = \frac{1}{2} (\hbar + 0) = \frac{\hbar}{2}$
 - Element (2,2): $\langle \beta | \sigma_x | \beta \rangle = \langle \beta | \frac{1}{2} (\sigma_+ + \sigma_-) | \beta \rangle = \frac{1}{2} (\langle \beta | \sigma_+ | \beta \rangle + \langle \beta | \sigma_- | \beta \rangle) = \frac{1}{2} (0 - \hbar) = -\frac{\hbar}{2}$
- In the same manner construct representation matrices for the ladder operators $s_{\pm}$ and from this σ_x and σ_y .
 - Demonstrate, for general vector operators $\mathbf{A}$ and $\mathbf{B}$, the Dirac identity
$$\langle \mathbf{A} | \boldsymbol{\sigma} | \mathbf{B} \rangle = \langle \mathbf{A} | \boldsymbol{\sigma} | \mathbf{B} \rangle + i \langle \mathbf{A} | \boldsymbol{\sigma} | \mathbf{B} \rangle$$
 - Use the Dirac identity to calculate

- $\left(\boldsymbol{\sigma} \cdot \hat{\mathbf{p}}\right) \left(\boldsymbol{\sigma} \cdot \hat{\mathbf{p}}\right) \boldsymbol{\sigma} \cdot \hat{\mathbf{p}}$
- $\left(\boldsymbol{\sigma} \cdot \hat{\mathbf{p}}\right) \left(\boldsymbol{\sigma} \cdot \hat{\mathbf{p}}\right) \boldsymbol{\sigma} \cdot \hat{\mathbf{p}}$, where $\hat{\mathbf{p}}$ is mechanical momentum $\hat{\mathbf{p}} = \hat{\mathbf{p}} + e\mathbf{A}$ and $\mathbf{A}$ is a vector potential. Show how this expression is simplified in Coulomb gauge: $\boldsymbol{\nabla} \cdot \mathbf{A} = 0$.
- Show that $V_{eN} \left(\boldsymbol{\sigma} \cdot \hat{\mathbf{p}}\right) = \boldsymbol{\sigma} \cdot \hat{\mathbf{p}} V_{eN}$ where V_{eN} is the electron-nucleus interaction $V_{eN} = -\frac{Ze^2}{4\pi\epsilon_0 r}$ in an atom

FILE: www.diracprogram.org/doc/release-26/_sources/getting_help/known_problems.md

orphan

Known problems

Modules not running in parallel

- RELADC is not parallelized

Modules not running with 32 bit integers

- LUCITA
- VERY large Coupled-Cluster calculations

gfortran on Apple MX (X= 1,2,3,..)

- The default optimization level 3 gives wrong results for ExaCorr runs with the TALSH library. See issue #155 for updates on this.

FILE: www.diracprogram.org/doc/release-26/_sources/getting_help/memory.md

orphan

Help with memory problems

DIRAC will allocate the following memory segments:

- Static allocation (approx. 130 MB; you can verify it using “make info”)
- WORK array for F77 “dynamic” allocation (specified by `./pam --mb`)
- Additional F90 dynamic allocations (depends on the calculation)

MEMGET errors

They look like this:

```
--- SEVERE ERROR, PROGRAM WILL BE ABORTED ---
```

```
Date and time (Linux) : Fri Nov 26 10:48:19 2010
MEMGET ERROR, insufficient work space in memory
```

This means that you need to increase the WORK array (increase `--mb`).

MEMCHK errors

They look like this:

```
MEMCHK ERROR, not a valid memget id in work(kfree-1)
Text from calling routine : PSIDHF.DHFSFCF (called from MEMREL)
KFIRST,KFREE,IALLOC =      1      2      1
found memory check :      0
expected :      1234567890
```

This means that you have found a bug in the code. Please contact the developers.

Other memory allocation errors

If you see a error message about memory problems in the queuing system output (and not in the DIRAC output) it typically means that there is not enough memory for F90 dynamic allocations.

You need to increase the memory limit that you specify to the queuing system or perhaps decrease `--mb` (if possible).

FILE: www.diracprogram.org/doc/release-26/_sources/index.md

Project website: <http://diracprogram.org>

Setting up DIRAC

- **Installation:** Requirements <installation/requirements>|EasyBuild <installation/easybuild>|System administrators <installation/sysadmins>|Linux, Unix, Mac <installation/general>|Windows <installation/windows>|Expert options <installation/expert>
- **Math libraries:** Detection and linking <installation/math>|MKL environment variables <installation/mkl>|Tensor libraries <installation/tensor_libraries>
- **MPI:** Forwarding environment variables <installation/mpi>
- **64-bit integer support:** Do I need it? <installation/int64/faq>|Math libraries <installation/int64/math>|64-bit OpenMPI <installation/int64/mpi64>|Troubleshooting <installation/int64/troubleshooting>
- **Testing:** Running the test set <installation/testing>
- **Example installations and run scripts:** HPC cluster (Strasbourg) <installation/examples/hpc-unistra>
- **pam script:** Setting the scratch directory <pam/scratch>|Alternative MPI launcher and passing arguments for it <pam/mpi_command>|String replacement <pam/replace>

Updates

ChangeLog <patches/CHANGELOG>

First steps in DIRAC territory

- **Getting started:** First calculation <tutorials/getting_started>
- **Molecule input formats:** mol format <molecule_and_basis/molecule_using_mol>|xyz format <molecule_and_basis/molecule_using_xyz>|ecp input <molecule_and_basis/molecule_with_ecp>|Troubleshooting <molecule_and_basis/troubleshooting>
- **Basis sets:** General information <molecule_and_basis/basis>|Howto uncontract basis sets <molecule_and_basis/howto_uncontract>
- **Data storage:** Extracting data from the checkpoint file <manual/data>
- **Troubleshooting:** Known problems <getting_help/known_problems>|Memory problems <getting_help/memory>

Reference manual

manual/index.rst

Tutorials and walkthrus

- **Basis sets:** Basis sets for relativistic calculations <tutorials/relbas>|Augmenting basis sets <tutorials/augmentation>
- **SCF start guess:** Atomic start <tutorials/start_guess/atomic_start>|Extended Hückel start <tutorials/start_guess/atomic_huckel>|
- **Restarting and multi-step jobs:** SCF <tutorials/restart/scf>|X2C <tutorials/restart/x2c>|Coupled Cluster restart <tutorials/restart/coupled_cluster>|DFCOEF and DFPCMO <tutorials/restart/dfcoef_and_dfpcmo>|Troubleshooting <tutorials/restart/troubleshooting>
- **Python interface with ASE:** ASE-DIRAC <tutorials/ase/ase>
- **2-component Hamiltonians:** xamfX2C: 2e picture change corrections for the X2C Hamiltonian <tutorials/two_component_hamiltonians/xamfX2C>|X2C and local X2C <tutorials/two_component_hamiltonians/x2c_mol_loc>|Molecular mean-field X2C <tutorials/two_component_hamiltonians/molecular_mean_field>|Selecting a 2-component Hamiltonian other than X2C <tutorials/two_component_hamiltonians/hamiltonians>|Case study <tutorials/two_component_hamiltonians/example>
- **Relativistic effective core potentials:** Getting started <tutorials/ecp/first>
- **Nonrelativistic limit:** Reproducing nonrelativistic results <tutorials/reproducing_nr_results>
- **DFT:** TDDFT <tutorials/dft/tddft>|BSSE <tutorials/dft/bsse>|CAM functional <tutorials/dft/cam>|Troubleshooting <tutorials/dft/troubleshooting>
- **Orbital manipulations:** Frozen orbitals <tutorials/frozen_orbitals/tutorial>|Atomic shift operator <tutorials/atomic_shift/tutorial>
- **Long-range WFT/short-range DFT:** General <tutorials/srDFT/general>
- **Property calculations:** NMR shieldings using simple magnetic balance <tutorials/simple_magnetic_balance/tutorial>|An introduction to complex reponse <tutorials/properties/complex_response>|Magnetizabilities with London Atomic Orbitals <tutorials/lao_magnetizabilities/lao-magnetizabilities>|Dipole moment and polarizability of open-shell molecule <tutorials/properties/dipmom-polariz>|Nuclear spin-rotation constants <tutorials/spinrot/tutorial>|Molecular rotational g-tensors <tutorials/rotdg/tutorial>|Parity-violation contribution to nuclear spin-rotation constants <tutorials/pvcsr/tutorial>|Parity-violation contribution to electric field gradient <tutorials/pvcefg/tutorial>
- **X-ray spectroscopy:** Core ionization in the CO and N2 molecules <tutorials/x_ray/CO_N2_IP1s/tutorial>|Core electron excitations and ionization in water at the HF and DFT levels <tutorials/x_ray/water_Kedge/tutorial>|X-ray absorption spectroscopy beyond the electric-dipole approximation <tutorials/x_ray/BED_Ba/tutorial>
- **Spectroscopy:** Excitation energies at the SCF-level <tutorials/properties/scfexc/tutorial>|Electronic excitations using the POLPRP module <tutorials/polprp/general>|Full scope application: Excitation spectrum of an osmium complex <tutorials/polprp/osmium>
- **Analysis:** Projection analysis <tutorials/analysis/projection_analysis>|Localization of orbitals <tutorials/localization/tutorial>
- **Visualization:** General overview <tutorials/visual/general/tutorial>|Orbital densities <tutorials/visual/orbital_densities/tutorial>|Magnetizability density <tutorials/visual/magnetizability_density/tutorial>|Molecular electrostatic potential <tutorials/visual/mep/tutorial>|Radial distributions <tutorials/visual/radial_distributions/tutorial>
- **Open-shell SCF:** Basics <tutorials/open_shell_scf/aoc>|Converging atoms <tutorials/open_shell_scf/converging_atoms>
- **Coupled-Cluster:** The high spin oxygen molecule <tutorials/highspin_cc/02>|Coupled Cluster memory count <tutorials/cc_memory_count/count_cc_memory>|Hybrid-parallel run <tutorials/hybrid_parallel_cc_run/hybrid_parallel_cc_run>
- **Case studies:** Ir(16+) cation <tutorials/case_studies/Ir/Ir_16plus>|CmF molecule <tutorials/case_studies/CmF/CmF>|MnO6 system <tutorials/case_studies/MnO6>|UF6 molecule <tutorials/case_studies/UF6_molecule/UF6>|UF6(-) anion

- <tutorials/case_studies/UF6_anion/UF6_anion>|U06(-6) anion <tutorials/case_studies/U06_6minus/U06_6minus>|LuF3 molecule <tutorials/case_studies/LuF3/LuF3>
- **ADC:** Triatomic molecule <tutorials/reladc/triatomic_molecule>|Atom <tutorials/reladc/atom>
- **ECP:** First calculation <tutorials/ecp/first>|Correlation calculations <tutorials/ecp/combine>
- **Polarizable continuum model:** Basics <tutorials/pcm/pcm_basics>|Hartree-Fock and DFT calculations in solution with the polarizable continuum model <tutorials/pcm/pcm_scf>|Calculation of polarizabilities in solution: response theory approach <tutorials/pcm/pcm_response>
- **Polarizable embedding (PE) model:** PE-HF calculations on micro-solvated H2O <tutorials/polarizable_embedding/pe_scf>|PE-TDDFT calculations on micro-solvated H2O <tutorials/polarizable_embedding/pe_response>
- **Frozen density embedding (FDE):** NMR shieldings in Frozen Density Embedding (FDE) scheme with London atomic orbitals (LAOs) <tutorials/fde_nmr/tutorial>
- **Davidson corrections for relCI (LUCITA and KRCI with LUCIAREL):** +Q corrections <tutorials/relci_q_corrections/q_corrections>
- **Quantum electrodynamics (QED):** effective QED potentials <tutorials/qed/effqed>
- **Utility programs:** TWOFIT <tutorials/utis/twofit>|VIBCAL <tutorials/utis/vibcal>|CFREAD <tutorials/utis/cfread>|LABREAD <tutorials/utis/labread>
- **Python interface with OpenFermion:** Openfermion-Dirac <tutorials/openfermion/openfermion-dirac>
- **Outdated tutorials (need update):** DIRRCI <tutorials/dusty_attic/dirrci>|GOSCIP <tutorials/dusty_attic/goscip>|LUCITA <tutorials/dusty_attic/lucita>|MOLTRA <tutorials/dusty_attic/moltra>|MP2 <tutorials/dusty_attic/mp2>|Open shells <tutorials/dusty_attic/open_shells>

Exercises

exercises/Corrientes2012/index.rst exercises/TCCM2021/index.rst

Development

- **Public and private repositories:** Working with multiple remotes <development/multiple-remotes>|Contributing changes <development/contributing-changes>
- **Code review:** Code review workflow <development/code_review>
- **Releasing:** Release preparation <development/release_preparation>|Patch preparation <development/patch_preparation>|Beta testing <development/beta_testing>|Where to commit after the release is out <development/commit_policy>
- **Programming:** Rules <development/rules>|How to add new tests <development/testing>|How to add/move/remove sources <development/adding_moving_removing_sources>|[runtest_dirac.py](#)|Input parsing <development/input_reading>|Git submodules <development/external_projects>|Nightly tests <development/cdash>|Dirac on Windows <development/windows>|Profiling <development/profiling>|Debugging <development/debugging>|History <development/museum>|Further development <development/further_development>|Good Fortran 90 practices <development/good_fortran_90_practices>|FAQ <development/faq>
- **Data schema and the CHECKPOINT file:** How data is organized on the checkpoint file <development/checkpoint>
- **Basis sets:** Testing basis sets in DIRAC <development/test_basis_sets>
- **Moving code between machines:** Transferring uncommitted code <development/transferring_uncommitted_code>
- **Notes:** Screening <development/screening>|64bit integers <development/int8>|Numerical constants <development/num_constants>|XML <development/xml>|Static linking <development/static_linking>|Problems with lapack's DSYEVR <development/diag>|How this documentation works <development/documentation_howto>

Bibliography

- References <zreferences>

FILE: www.diracprogram.org/doc/release-26/_sources/installation/easybuild.md

orphan

Installation using EasyBuild

We recommend using EasyBuild to build DIRAC on HCP systems.

If you or your cluster administrator are unfamiliar with it, we recommend following the [installation](#) and [usage](#) instructions provided on the [EasyBuild website](#). Additionally, there is an instructive [tutorial](#) that can be used to configure it.

Once EasyBuild is installed on your system, as a user, you can install DIRAC employing, for example, the following commands:

```
$ module load EasyBuild/5.2.0
$ module use /path-to-your-modules-directory/software/modules/all
$ eb -r --buildpath=/your-build-directory/ --installpath=/path-to-your-modules-directory/software --repositorypath=/path-to-your-modules-dire
```

Once installed, you can use DIRAC simply by typing in your runfile, for example: :

```
$ module load DIRAC/26.0-intel-2025a-int64
```

and to run DIRAC you must use: :

```
$ pam-dirac --inp=... --mol=... [...]
```

Current versions of DIRAC supported by EasyBuild can be found [here](#) and [here](#).

However, if you still want to compile the program manually, you can follow the instructions provided in the Installation instructions for system administrators <sysadmins> section.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/examples/hpc-unistra.md

orphan

Installation on [HPC cluster](#) in Strasbourg

Configuration and compilation

In order to find the proper software needed for configuration and compilation the following line is added to .bashrc :

```
source $HOME/modules
```

where the file modules read (as of June 2013):

```
module delete mpi/openmpi-1.4.i11
module delete compilers/intel11
module delete libs/mkl11
module load mpi/openmpi-1.6.i13
module load compilers/intel13
module load libs/mkl13
```

To properly set environmental variables for the Intel MKL math library we follow instructions [here](#) and add to .bashrc the following lines:

```
export MKL_NUM_THREADS=1
export MKL_DYNAMIC="FALSE"
export OMP_NUM_THREADS=1
```

Finally we configure and build the binary:

```
$ ./setup --fc=mpif90 --cc=mpicc
$ cd build
$ make
```

Setting up pam

In your top directory you may configure the pam run script by adding a file .diracrc with contents:

```
--scratch=/scratch
--noarch
--debugger=/usr/bin/gdb
--basis="$HOME/Dirac/basis:$HOME/Dirac/basis_dalton"
--profiler=/usr/bin/gprof
```

Example run script

Here is a simple example of a run script:

```
#!/bin/bash
#SBATCH -p pri2008
#SBATCH -A qossaue
#SBATCH -t 00:10:00
#SBATCH -n 24 # xx Coeurs
#SBATCH --mail-type=END
#SBATCH --mail-user=trond.saue@irsamc.ups-tlse.fr
. ~/.bashrc
scontrol show hostname > machinelist
pam --inp=HF --mol=H2O --mpi=$SLURM_NPROCS --machfile=machinelist
exit 0
```

FILE: www.diracprogram.org/doc/release-26/_sources/installation/expert.md

orphan

Expert options

Compiling in verbose mode

Sometimes you want to see the actual compiler flags and definitions when compiling the code:

```
$ make VERBOSE=1
```

How can I change optimization flags?

You can turn optimization off (debug mode) like this:

```
$ ./setup --type=debug [other flags]
$ cd build
```

```
$ make
```

You can edit compiler flags under the path `cmake/custom/compiler_flags-`

Alternatively you can edit compiler flags through `ccmake`:

```
$ cd build
$ ccmake ..
```

FILE: www.diracprogram.org/doc/release-26/_sources/installation/general.md

orphan

Basic installation

Getting the Code

We recommend users who do not plan to make any code changes to download from <https://doi.org/10.5281/zenodo.18662050> (DIRAC26)

In this case the release tarballs are self-contained, and in addition we get useful download metrics which we can use in future funding applications.

If you clone the code directly from GitLab, please clone with `--recursive`, e.g. the latest master:

```
$ git clone --recursive --branch master https://gitlab.com/dirac/dirac.git
```

By replacing master with a version number, for instance `v26.0`, one can select a specific version. In order to get the latest revision only:

```
$ git clone --depth 1 --recursive git@gitlab.com:dirac/dirac.git
```

If you have downloaded a zip/tar.gz/tar.bz2/tar archive directly from GitLab, you will require Git installed to build the code. After extracting the archive you will need to run this additional step in order to fetch and update external dependencies:

```
$ git submodule update --init --recursive
```

Building

The default installation (sequential) proceeds through four commands:

```
$ cd dirac
$ ./setup
$ cd build
$ make -j
```

DIRAC is configured using [CMake](#), typically via the `setup` script, and subsequently compiled using `make` (or `gmake`). The `setup` script is a useful front-end to `CMake`. You need python to run `setup`. To see all options, run:

```
$ ./setup --help
```

The `setup` script creates the directory “build” and calls `CMake` with appropriate environment variables and flags. By default `CMake` builds out of source. This means that all object files and the final binary are generated outside of the source directory. Typically the build directory is called “build”, but you can change the name of the build directory (e.g. “build_gfortran”):

```
$ ./setup [--flags] build_gfortran
$ cd build_gfortran
$ make
```

You can compile the code on all available cores:

```
$ make -j
```

Testing

We strongly recommend that the installation is followed by testing your installation:

```
$ ctest
```

or:

```
$ make test
```

For more information about testing, see [here](#).

If you encounter any problems check the issues under [DIRAC](#) and create a new one if your problem is not mentioned.

Contributing

- [Working with the public and private repositories](#)
- [External contributions](#)
- [DIRAC testing dashboard](#)

Parallel build

Building DIRAC for parallel runs using [MPI](#) is in principle a very simple modification of the above procedure:

```
$ ./setup [--flags] --mpi
$ cd build
$ make
```

Once again we recommend [testing the code](#) afterwards.

Typical examples

In order to get familiar with the configuration setup, let us demonstrate some typical configuration scenarios.

Configure for parallel compilation using MPI (make sure to properly export MPI paths):

```
$ ./setup --mpi --fc=mpif90 --cc=mpicc --cxx=mpicxx
```

These compiler names are in fact default for MPI in Dirac, so there is a shortcut if these are the compiler wrappers you want to use:

```
$ ./setup --mpi
```

Configure for sequential compilation using ifx/icx/icpx and link against parallel mkl:

```
$ ./setup --fc=ifx --cc=icx --cxx=icpx --qmk=parallel
```

Configure for sequential compilation using gfortran/gcc/g++:

```
$ ./setup --fc=gfortran --cc=gcc --cxx=g++
```

You get the idea. The configuration is usually good at detecting math libraries automatically, provided you export the proper environment variable MATH_ROOT, see [linking_to_math](#).

What to do if CMake is not available or too old?

If it is your machine and you have an Ubuntu or Debian-based distribution:

```
$ sudo apt-get install cmake
```

On Fedora:

```
$ sudo yum install cmake
```

Similar mechanisms exist for other distributions or operating systems. Please consult Google.

If it is a cluster, please ask the Administrator to install/upgrade CMake.

If it is a cluster, but you prefer to install it yourself (it's easy):

1. [Download](#) the latest pre-compiled CMake tarball
2. Extract the tarball
3. Set correct PATH variable

What to do if hdf5 is not installed on your machine

You can download the latest version of hdf5 from [the hdfgroup website](#)

FILE: www.diracprogram.org/doc/release-26/_sources/installation/int64/faq.md

orphan

I am about to install DIRAC on a 64-bit machine, should I compile using 64-bit integers or 32-bit integers?

DIRAC runs smoothly (with few exceptions; see below) on 64-bit platforms using either 32-bit or 64-bit integers.

It is easier to install for 32-bit integers because MPI and math libraries are typically built for 32-bit integers. Therefore DIRAC is by default built for 32-bit integers.

If you decide to build for 32-bit integers, you will **not** be able to:

- Safely allocate more than 16 GB of memory **per cpu-core**
- Run LUCITA
- Run really, really large Coupled-Cluster calculations

If you want to be able to run the above calculations, you have to build for 64-bit integers.

If you decide to build for 64-bit integers, you **have to** make sure that the math library you link to can handle 64-bit integers.

Otherwise DIRAC will stop.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/int64/math.md

orphan

How to build math libraries for 64-bit integers

Before you continue please verify whether you really need 64-bit integers. It is extra work and perhaps not needed.

If you decide to build DIRAC for 64-bit integers, you have to link to 64-bit integer aware math library/libraries providing both BLAS and LAPACK.

The following things can happen:

- Your math library is compiled for 64-bit integers and DIRAC will link and run properly.
- Your math library is not compiled for 64-bit integers and DIRAC will not link.
- Your math library is not compiled for 64-bit integers and DIRAC will link but stop at runtime.
- Your math library is not compiled for 64-bit integers and DIRAC will link and not stop at runtime because you deactivated the self-test and may produce wrong numbers.

For cases 2-4 you may have to either:

- Verify linking.
- Compile your own math library.
- Use DIRAC's internal math implementation (slow).
- Go back to 32-bit integers.

MKL

MKL provides bindings for 64-bit integers and normally DIRAC should correctly link if you have specified MATH_ROOT correctly.

Atlas

It is not trivial but doable to compile your own Atlas library with 64-bit integer support.

The difficulty is that you need to compile both LAPACK and Atlas.

To do this we recommend to follow the nice example: [Installing ATLAS with full LAPACK on Linux/AMD64](#).

FILE: www.diracprogram.org/doc/release-26/_sources/installation/int64/mpi64.md

orphan

How to build MPI libraries for 64-bit integers

If you want to use 64-bit integers in parallel DIRAC, you also need 64-bit integers in your MPI installation.

Important The ExaCorr core will not be available in compilations with 64-bit integers.

64-bit OpenMPI

Please check the integer type of your currently available OpenMPI installation (perhaps it can do 64-bit integers already). For this, type:

```
ompi_info -a | grep 'Fort integer size'
```

if the output contains 8 for 64-bit integers

```
Fort integer size: 8
```

then you have a suitable 64-bit OpenMPI installation which uses 64-bit (or 8-bytes) integers by default. If the output shows 4, you have 32-bit integers.

If you decided that you want to build 64-bit OpenMPI yourself (what is not difficult), then download the latest stable source from <http://www.open-mpi.org/> and extract it. Then enter the directory and configure OpenMPI (edit prefix path).

For Intel compilers use:

```
./configure CXX=icpx CC=icx F77=ifx FC=ifx FFLAGS=-i8 FCFLAGS=-i8 CFLAGS=-m64 CXXFLAGS=-m64 --prefix=/path/to/your_openmpi
```

For GNU compilers type:

```
./configure CXX=g++ CC=gcc F77=gfortran FC=gfortran FFLAGS="-m64 -fdefault-integer-8" FCFLAGS="-m64 -fdefault-integer-8" CFLAGS=-m64 CXXFLAG
```

Once the configure step is finished (lot of output, at the end there should be no error), build the library:

```
make -jN
make all install
```

Finally export the path locations in .bashrc (or in the corresponding file if you don't use bash-shell):

```
export PATH=$PATH: '/path/to/your_openmpi/bin'
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH: '/path/to/your_openmpi/lib'
export MANPATH=$MANPATH: '/path/to/your_openmpi/share/man'
```

FILE: www.diracprogram.org/doc/release-26/_sources/installation/int64/troubleshooting.md

orphan

DFT runs stop with “programming error in distribution of points in parallel DFT” with 64bit integer Intel MPI

This is not a programming error in DIRAC but a bug in 64bit integer Intel MPI mpi_reduce. The Intel developers are informed and working on a patch. In the meantime, for DFT calculations using Intel MPI you should use 32bit integers.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/math.md

orphan

Linking to math libraries

General

The DIRAC program uses libraries like BLAS or LAPACK for which a standard (but slow) implementation is found among the DIRAC source codes. To achieve optimal performance one should use optimized libraries at the linking stage and turn to the included versions (builtin math) only when no optimized version is available.

DIRAC requires BLAS and LAPACK libraries. Typically you will want to link to external math (BLAS and LAPACK) libraries, for instance provided by MKL or Atlas.

By default the CMake configuration script will automatically detect these libraries:

```
$ ./setup --blas=auto --lapack=auto          # this is the default
```

if you define MATH_ROOT, for instance:

```
$ export MATH_ROOT=/opt/intel/mkl
```

Do not use full path MATH_ROOT='/opt/intel/mkl/lib/ia32'. CMake will append the correct paths depending on the processor and the default integer type. If the MKL libraries that you want to use reside in /opt/intel/mkl/10.0.3.020/lib/em64t, then MATH_ROOT is defined as:

```
$ export MATH_ROOT=/opt/intel/mkl/10.0.3.020
```

Then:

```
$ ./setup [--flags]
$ cd build
$ make
```

Intel/MKL

If you compile with Intel compilers and have the MKL library available, you should use the -qmk flag which will automatically link to the MKL libraries (in this case you do not have to set MATH_ROOT). You have to specify whether you want to use the sequential or parallel (threaded) MKL version. For a parallel DIRAC runs you should probably link to the sequential MKL:

```
$ ./setup --fc=mpif90 --cc=mpicc --cxx=mpicxx --qmk=sequential
```

For a sequential compilation you may want to link to the parallel MKL:

```
$ ./setup --fc=ifx --cc=icx --cxx=icpx --qmk=parallel
```

The more general solution is to link to the parallel MKL and control the number of threads using MKL environment variables.

Explicitly specifying BLAS and LAPACK libraries

If automatic detection of math libraries fails for whatever reason, you can always call the libraries explicitly like here:

```
$ ./setup --blas=/usr/lib/libblas.so --lapack=/usr/lib/liblapack.so
```

Alternatively you can use the -explicit-libs option. But in this case you should disable BLAS/LAPACK detection:

```
$ ./setup --blas=none --lapack=none --explicit-libs="-L/usr/lib -lblas -llapack"
```

Builtin BLAS and LAPACK implementation

If no external BLAS and LAPACK libraries are available, you can use the (slow) builtin BLAS/LAPACK implementation. However note that these are not optimized and you will sacrifice performance. This should be the last resort if nothing else is available:

```
$ ./setup --blas=builtin --lapack=builtin
```

Linking to Atlas libraries on SUSE or Fedora

DIRAC may not be able to detect Atlas BLAS/LAPACK libraries on SUSE or Fedora because of nonstandard suffix:

```
$ cd /usr/lib64/atlas
$ ls -l

lrwxrwxrwx. 1 root root 17 Oct 30 14:41 libatlas.so.3 -> ./libatlas.so.3.0
-rwxr-xr-x. 1 root root 5.0M Sep 2 2011 libatlas.so.3.0
lrwxrwxrwx. 1 root root 17 Oct 30 14:41 libcblas.so.3 -> ./libcblas.so.3.0
-rwxr-xr-x. 1 root root 128K Sep 2 2011 libcblas.so.3.0
lrwxrwxrwx. 1 root root 19 Oct 30 14:41 libclapack.so.3 -> ./libclapack.so.3.0
-rwxr-xr-x. 1 root root 96K Sep 2 2011 libclapack.so.3.0
lrwxrwxrwx. 1 root root 19 Oct 30 14:41 libf77blas.so.3 -> ./libf77blas.so.3.0
-rwxr-xr-x. 1 root root 117K Sep 2 2011 libf77blas.so.3.0
lrwxrwxrwx. 1 root root 18 Oct 30 14:41 liblapack.so.3 -> ./liblapack.so.3.0
-rwxr-xr-x. 1 root root 5.4M Sep 2 2011 liblapack.so.3.0
lrwxrwxrwx. 1 root root 19 Oct 30 14:41 libptcblas.so.3 -> ./libptcblas.so.3.0
-rwxr-xr-x. 1 root root 128K Sep 2 2011 libptcblas.so.3.0
lrwxrwxrwx. 1 root root 21 Oct 30 14:41 libptf77blas.so.3 -> ./libptf77blas.so.3.0
-rwxr-xr-x. 1 root root 118K Sep 2 2011 libptf77blas.so.3.0
```

This can be fixed by creating soft links with .so suffix:

```
$ su
$ for file in *3; do ln -s $file ${file%.3}; done
$ ls

lrwxrwxrwx. 1 root root 13 Oct 30 17:45 libatlas.so -> libatlas.so.3
lrwxrwxrwx. 1 root root 17 Oct 30 14:41 libatlas.so.3 -> ./libatlas.so.3.0
-rwxr-xr-x. 1 root root 5225848 Sep 2 2011 libatlas.so.3.0
lrwxrwxrwx. 1 root root 13 Oct 30 17:45 libcblas.so -> libcblas.so.3
lrwxrwxrwx. 1 root root 17 Oct 30 14:41 libcblas.so.3 -> ./libcblas.so.3.0
-rwxr-xr-x. 1 root root 130824 Sep 2 2011 libcblas.so.3.0
lrwxrwxrwx. 1 root root 15 Oct 30 17:45 libclapack.so -> libclapack.so.3
lrwxrwxrwx. 1 root root 19 Oct 30 14:41 libclapack.so.3 -> ./libclapack.so.3.0
-rwxr-xr-x. 1 root root 97752 Sep 2 2011 libclapack.so.3.0
lrwxrwxrwx. 1 root root 15 Oct 30 17:45 libf77blas.so -> libf77blas.so.3
lrwxrwxrwx. 1 root root 19 Oct 30 14:41 libf77blas.so.3 -> ./libf77blas.so.3.0
-rwxr-xr-x. 1 root root 119760 Sep 2 2011 libf77blas.so.3.0
lrwxrwxrwx. 1 root root 14 Oct 30 17:45 liblapack.so -> liblapack.so.3
lrwxrwxrwx. 1 root root 18 Oct 30 14:41 liblapack.so.3 -> ./liblapack.so.3.0
-rwxr-xr-x. 1 root root 5566480 Sep 2 2011 liblapack.so.3.0
lrwxrwxrwx. 1 root root 15 Oct 30 17:45 libptcblas.so -> libptcblas.so.3
lrwxrwxrwx. 1 root root 19 Oct 30 14:41 libptcblas.so.3 -> ./libptcblas.so.3.0
-rwxr-xr-x. 1 root root 130824 Sep 2 2011 libptcblas.so.3.0
lrwxrwxrwx. 1 root root 17 Oct 30 17:45 libptf77blas.so -> libptf77blas.so.3
lrwxrwxrwx. 1 root root 21 Oct 30 14:41 libptf77blas.so.3 -> ./libptf77blas.so.3.0
-rwxr-xr-x. 1 root root 119824 Sep 2 2011 libptf77blas.so.3.0
```

Finally set:

```
$ export MATH_ROOT=/usr/lib64/atlas
```

Now DIRAC will correctly detect them.

Linking to Atlas libraries on Linux Mint

DIRAC may not be able to detect Atlas BLAS/Lapack libraries on Linux Mint because of some nonstandard (?) suffix.

After having installed them:

```
$ sudo apt-get install liblapack3 libatlas3-base
```

the libraries were not detected by DIRAC. I had to do this:

```
$ cd /usr/lib/atlas-base
$ su
$ for file in *3; do ln -s $file ${file%.3}; done
```

and this:

```
$ cd /usr/lib
$ sudo ln -s liblapack.so.3 liblapack.so
```

With this the automatic detection of math libraries works.

Linking to the ACML library

The Core Math Library (ACML) is distributed for most used Fortran compilers, both for 4-byte and 8-byte integers, as dynamic and static, see the [ACML web-page](#).

The user has to point the MATH_ROOT variable to the proper library directory. For instance, here one wants to use 64-bit integer version of ACML, in connection with Intel compilers:

```
$ export MATH_ROOT='/home/milias/bin/acml5.3.1_ifort_int64/ifort64_int64/lib'
```

The complete variety of compiler- and integer-specific ACML's clones can be found on the [dedicated web-page](#).

We report, however, that the gfortran generated ACML library can give these linking errors:

```
Linking Fortran executable dirac.x
/home/milias/bin/acml5.3.1_gnu_int64/gfortran64_int64/lib/libacml.so: undefined reference to `_gfortran_transfer_integer_write@GFORTRAN_1.4'
/home/milias/bin/acml5.3.1_gnu_int64/gfortran64_int64/lib/libacml.so: undefined reference to `_gfortran_transfer_character_write@GFORTRAN_1.4'
/home/milias/bin/acml5.3.1_gnu_int64/gfortran64_int64/lib/libacml.so: undefined reference to `_gfortran_transfer_real_write@GFORTRAN_1.4'
collect2: ld returned 1 exit status
```

If this happens, we recommend user to prefer some non-gfortran ACML version.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/mkl.md

orphan

How to set good environment variables for the Intel MKL library

By default DIRAC links to the multi-threaded Intel MKL library.

The pam script sets MKL_NUM_THREADS=1, MKL_DYNAMIC="FALSE", OMP_NUM_THREADS=1, and OMP_DYNAMIC="FALSE", unless these variables are set by the user. In order to benefit from the parallelization of MKL, the user should provide appropriate environment variables.

Be very careful when running MPI calculations and using threaded MKL. If all cores are already taken by the MPI (or by other users on the same node), you may observe a significant slow down of the code's execution run.

Currently DIRAC cannot benefit from OMP_NUM_THREADS other than 1 so this variable should always be set to 1 for DIRAC calculations.

Examples

Allow for 4 threads per core in MKL routines:

```
export MKL_NUM_THREADS=4
export MKL_DYNAMIC="FALSE"
```

Allow for 4 threads per core in MKL/BLAS routines:

```
export MKL_NUM_THREADS=4
export MKL_DOMAIN_NUM_THREADS="MKL_BLAS=4"
export MKL_DYNAMIC="FALSE"
```

As an example of proper splitting of CPUs between MPI and MKL, assume a 16 core node where 8 cores are assigned to MPI and the rest to MKL threads:

```
export MKL_NUM_THREADS=2
export MKL_DOMAIN_NUM_THREADS="MKL_BLAS=2"
export MKL_DYNAMIC="FALSE"
```

If you would like to run a sequential job on the same node it would then read:

```
export MKL_NUM_THREADS=16
export MKL_DOMAIN_NUM_THREADS="MKL_BLAS=16"
export MKL_DYNAMIC="FALSE"
```

You can find recommended settings for calling Intel MKL routines from multi-threaded applications on the [Intel web page](#).

Troubleshooting

Sometimes you could get this error:

```
OMP: Error #18: Setting environment variable "__KMP_REGISTERED_LIB_12973" failed:
OMP: Hint: Seems application required too much memory.
```

This can happen with statically linked multi-threaded MKL library, see [here](#).

FILE: www.diracprogram.org/doc/release-26/_sources/installation/mpl.md

orphan

Introduction

The DIRAC program suite is parallelized using the MPI2 protocol.

See related works involving parallelization implementation, Pernpointner2003, Saue1997, Knecht2008, Knecht2010a, Fleig2006, Fleig2001, Fleig2003.

Parallel run information

My shell environment variables are not forwarded to the compute nodes. What do I have to do?

Assuming you are using the pam script to submit your parallel calculation you can forward any environment variable, e.g., LD_LIBRARY_PATH, to the slave-node shell by adding to you pam command line:

```
./pam --mpiarg="-x LD_LIBRARY_PATH"    # for OpenMPI
./pam --mpiarg="-envall"               # for IntelMPI and MPICH(2)
```

In the latter case ALL environment variables will be forwarded in one shot whereas for **OpenMPI** you may need to provide the most essential environment variables individually. These may be for a typical parallel DIRAC run:

```
LD_LIBRARY_PATH
BASDIR
PATH
```

FILE: www.diracprogram.org/doc/release-26/_sources/installation/requirements.md

orphan

Requirements

DIRAC requires CMake, Fortran 90, C, and C++ compilers and a Python environment for the setup and run scripts. The program is designed to run on a unix-like operating system, and uses MPI for parallel calculations. Optionally DIRAC can (and should) be linked to platform specific BLAS and LAPACK libraries. Installing HDF5 is strongly recommended as not all functionality will work without it.

Minimum requirements

- CMake 3.23 - or later
- Python 3.6 - or later
- GFortran/GCC/G++ 4.9 - 5.2 or Intel compilers 13.0 - 15.0.6
- HDF5 1.10 - or later (note that patch levels 1.14.0 and 1.14.1 should be avoided due to a bug). The installation of the h5py Python package is not mandatory but recommended.

If your distribution does not provide a recent CMake, you can install it without root permissions within few minutes. Download the binary distribution from <https://cmake.org/download/>, extract it, and set the environment variables PATH and LD_LIBRARY_PATH to point to the bin/ and lib/ subdirectories of the extracted path, respectively. Verify that your new CMake is recognized with `cmake --version`.

Requirements for building with the parallel library ExaTensor

- CMake 3.23 - or later
- Python 3.6 - or later
- GFortran/GCC/G++ 8.x or 11+ (so neither 9.x nor 10.x), Intel compilers 18 or later
- OpenMPI 4.1.5 or later, or MPICH 3.2.1 or later. Compilation with Intel MPI is possible but should be checked for performance.

Further information on supported environments can be found at the [DIRAC fork of ExaTensor website](#). MPI is not required for building only the TAL-SH (single-node) component of ExaTensor.

MacOS X users are advised to use GCC 13 instead of GCC 14/15 until an [issue with compiler optimization level affecting the ExaCorr code is resolved](#).

At runtime you need to set environment variables to inform the library about the optimal parallel setup in both the installation as well as the execution stage. This is explained in detail at the [DIRAC fork of ExaTensor website](#).

The setup script of DIRAC should usually take care of the installation step, but the appropriate set up at runtime should be carefully considered by the user to obtain optimal performance. Examples of setup for running the standalone ExaTENSOR (Qforce.x) test utility for different architectures are given [here](#).

Users interested in this functionality should read the section on [tensor libraries](#) for DIRAC-specific instructions and known issues.

Supported, but optional dependencies

- OpenMPI 1.6.2 - 1.8.5 (MPI parallelization)
- IntelMPI 4.1 - 5.0 (MPI parallelization)
- Boost 1.54.0 - 1.60.0 (PCMSolver; installed along DIRAC if below 1.54.0)
- Alps 2.2 (DMRG code)
- [TBLIS](#) 2+ (ExaCorr code, see the [Tensor libraries section](#)). As for TAL-SH/ExaTensor, MacOS X users are advised to use GCC 13 instead of GCC 14/15 until an [issue with compiler optimization level affecting the ExaCorr code is resolved](#).

Additional dependencies due to PCMSolver

It is optionally possible to enable the polarizable continuum model functionality. The functionality is provided using the external module [PCMSolver](#). The module requires additional dependencies:

- the zlib compression library, version 1.2 or higher;

- the Boost set of libraries, version 1.54 or higher.

The module performs checks on the system to find an installation of Boost that has the suitable version. The search is directed towards conventional system directories. If the search fails, the module will build a copy of Boost on the spot. If your system has Boost installed in a nonstandard location, e.g. /boost/location, you have to direct the search by adding the following flags to the setup script:

```
$ ./setup -DBOOST_INCLUDEDIR=/boost/location/include -DBOOST_LIBRARYDIR=/boost/location/lib
```

By default, compilation of the module is enabled. CMake will check that the additional dependencies are met. In case they are not, compilation of PCMSolver will be disabled. CMake will print out which dependencies were not satisfied. Passing the option `-DENABE_PCMSOLVER=OFF` to the setup script will disable compilation of the module and skip detection of additional dependencies.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/sysadmins.md

orphan

Installation instructions for system administrators

We recommend using EasyBuild to compile DIRAC in HCP systems. However, if you still want to compile it by yourself, you can follow the following instructions.

Please read the other installation sections for details but the installation procedure is typically this:

```
$ ./setup --mpi --qmk=parallel --prefix=/full/install/path/
$ cd build
$ make [-j]
$ export DIRAC_TMPDIR=/full/path/scratch
$ export DIRAC_MPI_COMMAND="mpirun -np 8" # make test will run with MPI using 8 processes
$ make test
$ make install
```

This will install binaries, run scripts, the basis set library, as well as tools into the install path.

Advise users to always set a suitable scratch directory:

```
$ export DIRAC_TMPDIR=/full/path/scratch
```

If your system runs MPI jobs with mpirun, the pam script can be called with the `--mpi` flag:

```
$ pam [other flags] --mpi=8      # will run "mpirun -np 8 dirac.x"
```

You can define a custom MPI launcher with `DIRAC_MPI_COMMAND`, in this case do not use `--mpi` flag:

```
$ export DIRAC_MPI_COMMAND="mpirun -np 8"
$ pam [other flags]          # no --mpi flag
```

FILE: www.diracprogram.org/doc/release-26/_sources/installation/tensor_libraries.md

orphan

Notes on setting up and using tensor libraries in Dirac

The ExaCorr code (see `**EXACC`) is built around the use of tensor contract libraries.

ExaCorr will not be available in compilations with 64-bit integers

Originally based on the tensor libraries [TAL-SH](#) and [ExaTENSOR](#) by Dmitry Lyakh (which support both CPU and GPU architectures), through the Tensor Algebra Processing Primitives ([TAPP](#)) API the code now supports additional tensor libraries: [TBLIS](#) (CPU architectures), and the [TAPP reference implementation](#) (CPU architectures, testing only).

Important Only the [fork maintained by the DIRAC team](#) (which contains both ExaTENSOR and the TAL-SH codes) should be used with DIRAC, since it incorporates enhancements to the build system not found in the original, no longer maintained repositories by Dmitry Lyakh.

We are currently working towards supporting other distributed memory tensor libraries such as the [Cyclops Tensor Framework](#), to provide users with additional choice.

The choice of tensor library used as computational backend is made at compile time, via the `TENSOR_EXECUTOR` environment variable.

Configuring for TBLIS (build will fetch TAPP and TBLIS from their repositories):

```
export TENSOR_EXECUTOR=1 # enables TBLIS via the TAPP API
```

The variable `ENABLE_TBLIS` can be toggled (=ON/OFF) through CMake.

Configuring for the TAPP reference implementation (build will fetch TAPP and the reference implementation from its repository):

```
export TENSOR_EXECUTOR=4 # enables the TAPP reference implementation via the TAPP API
```

Configuring for TAL-SH (build will fetch the code from the ExaTENSOR repository):

```
export TENSOR_EXECUTOR=2 # enables TAL-SH
```

Configuring for ExaTENSOR (build will fetch the code from its repository):

```
export TENSOR_EXECUTOR=3 # enables ExaTENSOR
```

Important By default, both TAL-SH / ExaTENSOR and TBLIS will be fetched and built, even if e.g. TBLIS is the default executor.

Configuring and building TAL-SH / ExaTENSOR for ExaCorr

For TAL-SH / ExaTENSOR, it is possible to avoid fetching via the network by setting the EXATENSOR_GIT_REPO_LOCATION variable on CMake to the path to a local clone of the ExaTENSOR git repository. This is aimed as a workaround in systems which do not allow network access, such as some supercomputer centers. We are currently working to [support a similar capability for TAPP and TBLIS](#).

The setup for building TAL-SH / ExaTENSOR in this version of DIRAC is driven by DIRAC's CMake system but not fully integrated with it, with environment variables being passed on to the TAL-SH / ExaTENSOR Make system.

Upon starting the build for TAL-SH / ExaTENSOR, the file Exatensor_ENV containing TAL-SH / ExaTENSOR environment variables is created by DIRAC's CMake system. The file Exatensor_ENV_UP is also created as a backup of the last configuration, as Exatensor_ENV is re-generated and overwritten at each CMake configuration. The values of the variables in Exatensor_ENV can be changed (e.g. BUILD_TYPE=OPT to BUILD_TYPE=DEV to toggle debug flags on), or valid variables in the the ExaTENSOR build system (found under exatensor/src/exatensor in the build directory) can be added, for additional customization.

For example, the contents of Exatensor_ENV for a Mac OSX build with GNU compilers and the OpenBLAS library could look like the following:

```
WRAP=NOWRAP           # relevant for Cray compilers
BUILD_TYPE=OPT         # optimization configuration
EXA_TALSH_ONLY=YES     # only build the TAL-SH component, for a sequential build
# DIRAC compilers, passed on
CMAKE_Fortran_COMPILER=/opt/local/bin/gfortran-mp-14
CMAKE_C_COMPILER=/opt/local/bin/gcc-mp-14
CMAKE_CXX_COMPILER=/opt/local/bin/g++-mp-14
TOOLKIT=GNU           # GNU compilers
EXA_OS=NO_LINUX        # Mac OSX
GPU_CUDA=NOCUDA        # CPU-only configuration
MPILIB=NONE           # no MPI
BLASLIB=OPENBLAS
PATH_BLAS_OPENBLAS=/opt/local/lib
EXA_NO_BUILD=YES       # skips building ExaTENSOR and TAL-SH
```

Note The variable EXA_NO_BUILD=YES is added to Exatensor_ENV at the end of complete build, so if one wishes to change any configuration variables and recompile, this has to be removed.

Before configuring, the environment variable EXA_GPUS should be set in order to enable GPU support for TAL-SH or ExaTENSOR.

For NVIDIA GPUs:

```
export EXA_GPUS=NVIDIA
```

It is important for NVIDIA architectures to see whether the GPU_SM_ARCH is correctly set in Exatensor_ENV. Examples are GPU_SM_ARCH=70 for V100 (sm_70) GPUs, GPU_SM_ARCH=80 for A100 GPUs (sm_80), and GPU_SM_ARCH=90 for H100 GPUs (sm_90). For additional information please consult the NVIDIA documentation, or [other online resources](#).

For AMD GPUs:

```
export EXA_GPUS=AMD
```

For AMD GPUs, Exatensor_ENV will contain GPU_CUDA=CUDA and USE_HIP=YES. The GPU_SM_ARCH will be disregarded if present, and the build system should detect the target architecture from environment variables; if not, the GPU type can be passed on via the environment variable HIP_TARGET.

Running ExaCorr in parallel and on GPU nodes

DIRAC can be compiled and run both in sequential and in parallel with all of the above tensor libraries, provided that the code is compiled with 32-bit integers.

However, unless the ExaTENSOR library is used to distribute tensors over multiple nodes, the tensor libraries will only use a single node. Furthermore, ExaTENSOR-based parallel calculations with less than four (4) MPI ranks will actually run as a single node calculation.

In the case ExaCorr has been compiled with GPU support for TAL-SH, and ExaTENSOR is not in use, the environment variable TALSH_GPUS can be used to control how many of the GPUs accessible to TAL-SH runs will be used (if TALSH_GPUS is not defined, all GPUs available to the process will be used).

OpenMP parallelization is by default enabled in TAL-SH / ExaTENSOR, and is controlled by the appropriate environment variables. The use of a threaded math library is highly recommended.

ExaTENSOR runs will require the definition of another set of environment variables, for example as in:

```
export QF_NUM_PROCS=4           # total number of MPI ranks in the calculation. Must be at least 4 for ExaTENSOR to work properly.
export QF_PROCS_PER_NODE=4      # number of MPI ranks executing per node. in thi case all MPI ranks run on the same node.
export QF_CORES_PER_PROCESS=2   # Number of CPU cores attached to each MPI rank
export QF_NVMEM_PER_PROCESS=0   #
export QF_HOST_BUFFER_SIZE=1000 # Memory available on the node, divided by the number of MPI ranks
export QF_MEM_PER_PROCESS=750   # about 75% of QF_HOST_BUFFER_SIZE

export QF_GPUS_PER_PROCESS=2 # Number of NVIDIA or AMD GPUs to be used per MPI rank; here set to 2 GPUs per rank
export QF_MICS_PER_PROCESS=0 # this should always be set to zero
```



```
export QF_AMDS_PER_PROCESS=0 # this should always be set to zero, even in the case of AMD GPU systems

export OMP_NUM_THREADS=$QF_CORES_PER_PROCESS
export OMP_DYNAMIC=false
export OMP_MAX_ACTIVE_LEVELS=3
export OMP_THREAD_LIMIT=256
export OMP_WAIT_POLICY=PASSIVE
export OMP_PROC_BIND="spread"
export OMP_PLACES="{0:4},{4:4},{8:4},{12:4}"
```

please consult the examples for runtime configurations in the source tree, and [the documentation of ExaTensor](#) for further details.

These environment variables can also be added to the `~/diracrc` file for convenience (see `pam --help`).

Important In addition to no longer being supported by its original authors, ExaTensor exploits MPI-3 features that are not always well-supported by the underlying software stack (MPI implementation, communication API etc). As such it can be rather non-trivial to arrive at a configuration suitable for production runs. For example, at the time of the DIRAC26 release, we have been able to compile and run ExaCorr with ExaTensor over multiple computing nodes at the [Zeus HPC cluster \(Lille/Fr\)](#) using GNU 11 compilers and OpenMPI 4.1.5; runs on other resources with similar software stacks were not successful.

With that, users are encouraged to periodically consult the collection of build and runtime configuration examples for the [main development](#) and [DIRAC26 release](#) branches in the [DIRAC Git repository](#) for hints on their systems.

OpenMP and GPU bindings

GPU bindings or OpenMP thread placement are not necessarily easy to get right, so users should consult the documentation of the machine the jobs will run on. Additionally, tools such as [hello_jobstep](#) or [hello_cpu_binding](#) or [hello_gpu_binding](#) may be used, or serve as a starting point to be adapted to your needs.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/testing.md

orphan

Testing the installation

It is very important that you verify that your DIRAC installation correctly reproduces the reference test set before running any production calculations. But also with all tests passing you are strongly advised to always carefully verify your results since we cannot guarantee a complete test coverage of all possible keyword combinations.

The test set driver is CTest which can be invoked with “make test” after building the code.

Environment variables for testing

Before testing with “make test” you should export the following environment variables:

```
$ export DIRAC_TMPDIR=/scratch # scratch space (adapt the path of course)
$ export DIRAC_MPI_COMMAND="mpirun -np 8" # only relevant if you compile with MPI
```

Note that if you set the DIRAC_MPI_COMMAND the pam script will assume you that this is a parallel calculation. Before you test an MPI build, you have to set DIRAC_MPI_COMMAND otherwise the tests will be executed without MPI environment and many of the tests will fail.

Tests based on the ExaTensor library

For an MPI build and if the ExaCorr module is activated (which is the case by default), additional environment variables have to be set in order to run the tests checking the ExaTensor based functionality, see [tensor libraries.html](#) section for additional details. Note a minimal MPI configuration to execute ExaCorr in this case requires 4 (four) MPI ranks.

Running the test set

You can run the whole test set either using:

```
$ make test
```

or directly through CTest:

```
$ ctest
```

Both are equivalent (“make test” runs CTest) but running CTest directly makes it easier to run sequential tests on several cores:

```
$ ctest -j4 # only for sequential tests
```

You can select the subset of tests by matching test names to a regular expression:

```
$ ctest -R dft
```

or matching a label:

```
$ ctest -L short
```

To print all labels:

```
$ ctest --print-labels
```

How to get information about failing tests

We do our best to make sure that the release tarball is well tested on our machines. However, it is impossible for us to make sure that all functionalities and tests will work on all systems and all compiler versions. Therefore, some tests may fail and this is how it can look:

Total Test time (real) = 258.17 sec

The following tests FAILED:

```
34 - dft_response (Failed)
73 - xyz_input (Failed)
75 - dft_ac (Failed)
```

Errors while running CTest

The first place to look for the reasons is build/Testing/Temporary:

```
$ ls -l Testing/Temporary/
total 108
-rw-rw-r-- 1 bast bast 2247 Dec 11 10:19 CTestCostData.txt
-rw-rw-r-- 1 bast bast 94798 Dec 11 10:19 LastTest.log
-rw-rw-r-- 1 bast bast 39 Dec 11 10:19 LastTestsFailed.log
```

LastTest.log contains a log of all tests that were run. Search for the test name or “failed”. If tests do not produce any output, then this is the place to look for reasons (environment variables, problems with pam).

In addition, for each test (whether passed or failed), a directory is created under build/test. So in this case you can go into build/test/dft_response to figure out what went wrong. The interesting files are the output files:

```
-rw-rw-r-- 1 bast bast 89538 Dec 11 10:17 lda_hf.out
-rw-rw-r-- 1 bast bast 1570 Dec 11 10:17 lda_hf.out.diff
-rw-rw-r-- 1 bast bast 841 Dec 11 10:17 lda_hf.out.reference
-rw-rw-r-- 1 bast bast 841 Dec 11 10:17 lda_hf.out.result
```

The *.out file is the output file of the test. The *.result file contains numbers extracted by the test script. The *.reference file contains numbers extracted from the reference file. The *.diff file is empty for a test where result and reference match. If they do not match, the *.diff file will list the lines of output where the results differ.

FILE: www.diracprogram.org/doc/release-26/_sources/installation/windows.md

orphan

DIRAC on Windows

We offer native support for Microsoft Windows 7/8 and for Cygwin.

Dependencies

1. The CMake system (<http://www.cmake.org>). Add path to CMake executables into your PATH environment variable. Example of path to add: “C:.2-win32-x86”
2. Fortran, C and C++ compilers. We recommend to use compilers from <http://mingw-w64.sourceforge.net/> for both 32/64 bit Windows. You can download 32-bit version from [this URL](#). And 64-bit version from [this page](#). In both cases we recommend to download archive file instead of installer and then just unpack it somewhere. Add path to compilers into your PATH environment variable. Example of path to add: “C:.”
3. The Python interpreter of version at least 2.7 (<http://www.python.org>). Install 32-bit version of Python on 32-bit Windows and 64-bit Python on 64-bit Windows. Into PATH variable add paths to both Python and its scripts. Example of paths to add: “C:;C:.”
4. Zlib library (<http://www.zlib.net/>)

For 32-bit Windows (if you place zlib libraries at the same place as we do then CMake should find them automatically):

[a] download zlib in archive from <http://zlib.net/zlib128-dll.zip>

[b] unpack downloaded archive (Extract all)

[c] rename unpacked folder to “zlib” (note: that folder must contain folders: “include”, “lib”, “test”, file “zlib1.dll” and some other files)

[d] move it to “C:Files”

[e] run in command line these commands :

```
cd C:\Program Files\zlib
dlltool -D zlib1.dll -d lib/zlib.def -l lib/libzdll.a
```

[f] add “C:Files” into PATH

[g] if CMake will have problems to find zlib libraries you can use this (available since CMake 2.8.7): :

```
python setup -DZLIB_ROOT=[path to zlib folder]
```

For 64-bit Windows (if you place zlib libraries at the same place as we do then CMake should find them automatically):

[a] download zlib in archive from [here](#)

[b] unpack downloaded archive (Extract all)

[c] in unpacked folder find folder named “zlib” and move it to “C:\Files” (note: that folder must contain folders: “bin”, “include” and “lib”)

[d] add “C:\Files” into PATH

[e] if CMake will have problems to find zlib libraries you can use this (available since CMake 2.8.7): :

```
python setup -DZLIB_ROOT=[path to zlib folder]
```

5. Boost (<http://www.boost.org/>). Download Boost in archive file from <http://sourceforge.net/projects/boost/files/boost/> Then follow this instructions (if you place Boost libraries at the same place as we do then CMake should find them automatically):

[a] unpack downloaded archive (Extract all)

[b] in unpacked folder (for example: “C:_1_56_0_1_56_0”) run this command in command line:

:

```
bootstrap.bat mingw
```

[c] in the same directory run: :

```
b2 --with-system --with-fileSYSTEM --with-test --with-chrono --with-timer toolset=gcc install
```

[d] in “C:” you will find “include” and “lib” directories

[e] if CMake will have problems to find your Boost libraries yo can use: :

```
python setup -DBOOST_INCLUDEDIR=[path]\include -DBOOST_LIBRARYDIR=[path]\lib
```

6. To speed-up your calculations you can download OpenBLAS (it includes BLAS and LAPACK implementation) library from <http://www.openblas.net/>. There are available prebuilt binaries for 32/64 bit Windows for Intel NEHALEM architecture with threading capability. You can specify number of threads with “OPENBLAS_NUM_THREADS” environment variable. On 64-bit Windows you can choose from “Win64-int32” and “Win64-int64” versions. Use “Win64-int32” version when building without “-int64” setup option and “Win64-int64” version when building with “-int64” setup option. Add path to “libopenblas.dll” file into PATH. Example of path to add: “C:\v0.2.14-Win64-int64”.

[!NOTE] To use OpenBLAS libraries you have to set them explicitly with “-blas” and “-lapack” setup options. See Build examples below.

[!NOTE] For detailed info about how to use threads and for detailed install guide go to <https://github.com/xianyi/OpenBLAS/wiki>.

[!NOTE] If you need you can compile your own OpenBLAS binaries, for example for different CPU architecture. See <https://github.com/xianyi/OpenBLAS/wiki/How-to-use-OpenBLAS-in-Microsoft-Visual-Studio> - do not be afraid it is built by MinGW in MSYS!

7. How to install software needed to create documentation see windows_developers

Important notes

1. You can modify or create PATH environment variable in “Control Panel” -> “User Accounts” and click “Change my environment variables”. Please add or change variables only in the part for User variables not in the part for System variables. When you want to add several paths into PATH environment variable then you can use “;” as a separator.
2. Please avoid to have spaces in your paths (also in name of your user’s home directory). The zlib library in “C:\Files” is an exception from this rule.
3. Avoid to have path to program “sh.exe” in your PATH. See also http://www.cmake.org/Wiki/CMake_MinGW_Compiler_Issues. Examples of not good paths: “C:\I.0”, “C:.” or “C:.”. These paths are causing conflict with CMake programs!
4. Upon installing Python for all users on the Windows computing server it happens that registers are not under HKEY_CURRENT_USER. One has to export the “Python” section under HKEY_LOCAL_MACHINE, modify the exported file and import it again. See [this post](#).
5. To check which dynamic libraries are used by your program you can use “Process Explorer” utility from <https://technet.microsoft.com/en-us/sysinternals/bb896653.aspx>. This utility can also be used to show number of threads used by your program, its I/O operations and so on. You should care about your program is loading correct “dll” libraries during execution. For example: you want to build Dirac with “-int64” option and you specify to use “Win64-int64” version of OpenBLAS with “-blas” option - this is correct. But in your PATH “Win64-int32” version of OpenBLAS is found before “Win64-int64” version and “Win64-int32” version is used - this is not correct and Dirac program fails.

Windows PowerShell

If you are using Windows PowerShell instead of command line (“cmd.exe”) you should realise that commands look little bit different: :

```
bootstrap.bat mingw
```

looks in PowerShell like: :

```
.\bootstrap.bat mingw
```

If you have problems to build Dirac in PowerShell try to use command line (type “cmd.exe” in PowerShell).

Installation and running

Configuration, installation and own execution of DIRAC is possible from the Windows (DOS) command line or Windows PowerShell. Windows operating systems are generating executables with the “.exe” suffix, like dirac.x.exe.

To execute Python scripts please type “python” command before script name as Windows can not determine scripting language.

Finally, instead of “make” type its MinGW64 version, what is “mingw32-make”.

Build examples

For specifying one or more parameters for Python scripts (either in the “*diracrc*” configuration file, or directly by typing in the command line), you should envelope them into inverted commas, “...”. In “*diracrc*” you can use backslash in Windows paths. For example:

```
--basis="C:\Dirac\my-dirac\basis;C:\Dirac\my-dirac\basis_dalton;C:\Dirac\my-dirac\basis_ecp"
--scratch="D:\dirac\tmp"
```

[!NOTE] If you want to use “*diracrc*” configuration file it must be placed in your user’s home directory, for example: “*C:\diracrc*”.

If you want to specify “*setup*” options (“*-fc*”, “*-cc*”, “*-cxx*”, “*-blas*”, “*-lapack*” and others) you have to use slash sign in paths like in Linux, if you want to specify *-D* options you have to use backslash sign in paths. See example below.

[!NOTE] If your compilers are in PATH you do not need to specify full path to “*-fc*”, “*-cc*”, “*-cxx*”.

[!NOTE] To show all setup options run in command line: `python setup --help`.

Configure DIRAC:

```
C:\Dirac\my-dirac> python setup --generator="MinGW Makefiles" --int64 --blas="C:/libraries/OpenBLAS/OpenBLAS-v0.2.14-Win64-int64/bin/libopenb
```

Compile the executable (in the default *build* directory):

```
cd build
C:\Dirac\my-dirac\build> mingw32-make -j 4
```

Run your selected test set together with uploading results onto the CDash web:

```
C:\Dirac\my-dirac\build> ctest -D ExperimentalTest -L short -D ExperimentalSubmit -j 4
```

Intel (not available yet)

We tried to test Dirac with *Intel Parallel Studio XE 2015 Update 3 Cluster Edition* (<https://software.intel.com/en-us/intel-parallel-studio-xe>) which contains Intel C/C++ and Fortran compilers, Intel MKL and Intel MPI libraries in virtual machine with *Microsoft Windows Server 2012 R2* running on AMD Athlon II 64-bit processor.

Our installation steps:

1. Install *Visual Studio Professional 2013 with Update 3* (<https://www.visualstudio.com/>)
2. Install *Intel Parallel Studio XE 2015 Update 3 Cluster Edition*
3. Initialize the Cluster edition tools:

```
cd C:\Program Files (x86)\Intel\Parallel Studio XE 2015
psxevars.bat intel64
```

4. Add “*C:\XE 2015_mic;C:\Files (x86)\XE*” into PATH environment variable.
5. Make own *cmd.exe* shortcut, open its properties and put into “*Target*” field (then you can use Intel compilers from this custom command line):

```
%comspec% /k ""C:\Program Files (x86)\Microsoft Visual Studio 12.0\VC\vcvarsall.bat" amd64 && "C:\Program Files (x86)\Intel\Composer XE 2015\
```

Intel compiler names:

C/C++ compiler: *icl* Fortran compiler: *ifort* CMake Generator: NMake Makefiles MPI compilers for C/C++ and Fortran (*.bat* wrappers): *mpicc*, *mpicxx*, *mpif90* MPI execution program: *mpiexec* (when first time run after Windows start/user login it asks for user password)

We have used this setup command:

```
python setup --cc=icl --cxx=icl --fc=ifort --type=debug --int64 --generator="NMake Makefiles" -D ENABLE_PCMSOLVER=OFF
```

and then this command for building:

```
cd build
nmake
```

Cygwin

Regarding the Microsoft Windows platform, DIRAC is able to run under the Cygwin (<https://cygwin.com/>) intermediating environment, which has almost all necessary Linux substitutes. Thus it is considered as the Windows non-native installation, because the Cygwin layer ensures full Linux workplace on the top of the ground Windows operating system.

In order to build Dirac on Cygwin you have to install these packages:

1. compilers: *gcc-core*, *gcc-fortran*, *gcc-g++* (also these we need to get all header files: *gcc-ada*, *gcc-objc*, *gcc-objc++*)
2. *make* and *cmake* (it includes *ctest*..)
3. *python* interpreter
4. *zlib0*, *zlib-devel*
5. *git* version control system
6. optionally you can install *OpenBLAS* (*libopenblas*) - recommended for 32 and 64 integer builds or *BLAS* and *LAPACK* (together in *liblapack0* package) - but these are only 32-bit versions

[!NOTE] *OpenBLAS* library is placed in “*/usr/bin/cygbblas-0.dll*”.

[!NOTE] You have to explicitly specify to use *OpenBLAS* library with “*-blas*” setup option.

[!NOTE] With 64 bit integers enabled use builtin *LAPACK* with *OpenBLAS* (*OpenBLAS* on Cygwin does not contain *LAPACK*).

[!NOTE] To run *OpenBLAS* in parallel export *OPENBLAS_NUM_THREADS* environment variable.

[!NOTE] *BLAS* and *LAPACK* libraries from *liblapack0* package are placed in “*/usr/lib/lapack*” (“*cygblas-0.dll*” and “*cyglapack-0.dll*”).

7. optionally you can install *Open MPI*

[!NOTE] If you want to use *Open MPI* you have to run *Cygserver* as an administrator, see <https://cygwin.com/cygwin-ug-net/using-cygserver.html>.

8. to be able to build documentation (*make html*, *make doxygen* - problematic, *make slides*) install also: *pkg-config*, *ghostscript*, *libfreetype-devel*, *libpng-devel*, *python-gtk2.0*, *libgtk2.0-devel*, *doxygen* (why so many see <http://blogs.bu.edu/mhirsch/2014/06/matplotlib-in-cygwin-64-bit/>) and get and install *pip* from <https://pip.pypa.io/en/latest/installing.html> and in *Cygwin* terminal run:

```
pip install sphinx python-dateutil pyparsing sphinxcontrib-bibtex sphinx_rtd_theme numpy matplotlib hieroglyph
```

[!NOTE] Previous *pip* install command usually does not work if you have installed *libopenblas*. You should remove it, install *liblapack0* and try again.

Always check if you are using *Cygwin* programs and libraries (placed in for example: “*/usr/bin*”, “*/usr/include*”, “*/usr/lib*”, “*/usr/share*”) instead of programs/libraries from *Windows* (those are placed in “*/cygdrive/...*”) for example by this commands :

```
which gcc g++ gfortran make cmake ctest ar ranlib python git doxygen
whereis zlib cygblas-0.dll
```

Installation of packages can be done by running *Cygwin* installer (there is nothing like *sudo apt-get*). You can use *Cygwin* installer also to update installed packages. Sometimes new version of installer is released and then you must install and update your packages with its new version but there is no need to completely reinstall your *Cygwin* installation.

[!NOTE] Ask your administrator to install *Cygwin* packages, “*pip*” and other software into *Cygwin* environment.

[!NOTE] You can have also your own *Cygwin* installation and then you do not need to ask anybody.

Cygwin Troubleshooting

When using *Cygwin* programs and environment variables from *Windows* are also available in *Cygwin*. There can be problems for example with *Boost* and *zlib* libraries from *Windows* which can be detected by *CMake* when run from *Cygwin*. Those libraries can make configuration or build fail. In order to avoid this temporarily move folders with those libraries to some another place which is not investigated by *CMake* or change their names (then if you want to build from *Windows* you can return everything back or see *windows_users* how to learn setup where to look for *zlib* and *Boost*). Also you should check environment variables.

Example how to point to correct *Boost* and *zlib* libraries :

```
python setup -DBOOST_INCLUDEDIR=[install_prefix]/include -DBOOST_LIBRARYDIR=[install_prefix]/lib -DZLIB_ROOT=/usr/lib
```

[!NOTE] *Boost* libraries which can be installed from *Cygwin* installation are not recognized by *Dirac* setup.

You can try to compile your own (download from <http://sourceforge.net/projects/boost/files/boost/>): :

```
bootstrap.sh gcc --prefix=[install_prefix]
b2 --with-system --with-filesystem --with-test --with-chrono --with-timer toolset=gcc install
```

To more clearly see (in case when problems occur) what happens it is possible to run *Dirac* configuration directly by *CMake* when you create a build directory in *Dirac* top directory then move into it and run: :

```
cmake ..
```

If there is need to remove *Cygwin* in an environment with multiple users then all users must delete their own home directories and then administrator can delete whole *Cygwin* installation/folder.

Sometimes you can get errors like *os.fork()* is not available. In that case try to rebase *Cygwin*: <http://cygwin.wikia.com/wiki/Rebaseall>.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/amfi.md

orphan

*AMFI

*AMFI

[!NOTE] We highly recommend to use the *XAMFI module rather than the present *AMFI module as highlighted by the numerical examples provided in Knecht2022.

Specification of the AMFI, RELSCF directives.

The AMFI program of B.Schimmelpfennig serves for calculating spin-orbit contributions to various quasirelativistic two-component Hamiltonians. Its important part is the independent scalar atomic code - RELSCF (named as AT34), which is providing atomic orbital coefficients for the AMFI mean-field summation.

The user may control not only the extent of the output, but also provide here some parameters for the RELSCF code. Note that closed/open-shell HF-SCF code gives both the nonrelativistic and the spin-free second-order Douglas-Kroll-Hess SCF energies, which can be compared with DIRAC results.

RELSCF is using only decontracted basis sets restricted to occupied shells only; for example, if you set some spdf-basis for a DIRAC BSS+AMFI run of Magnesium (12 electrons), RELSCF will use, for the sake of computational effectivity, this basis set with only s- and p- exponents.

.PRNT_A

.PRNT_A

Print level for AMFI calculations.

Default:

```
.PRNT_A
0
```

```
.PRNT_S
```

.PRNT_S

Print level for the RELSCF part.

Default:

```
.PRNT_S
0
```

```
.MXITER
```

.MXITER

Maximum number of iterations in RELSCF.

Default:

```
.MXITER
50
```

```
.AMFICH
```

.AMFICH

Set up an artificial charge of the calculated system which has effect only on the AMFI mean-field summation(s). This does not change the electronic occupation for the DIRAC SCF procedure.

The reason is that AMFI-attached scalar relativistic Hartree-Fock SCF module might not be converging for certain high-spin open-shell atoms (like Pt,Yb), what is anyway expected for a single-determinant code. A small positive charge (+1, +2) usually helps to get converged molecular orbitals of given atom with a negligible change in results due to AMFI mean-field contribution. The AMFI mean-field contributions are large for core shells, but small for valence orbitals.

In the case of polyatomic systems the overall AMFI-charge is proportionally distributed over individual atoms and rounded to the closest integer value.

Example:

```
.AMFICH
+1
```

The user can check the mean-field occupation/charge of each handled atom in the AMFI/RELSCF output, together with the RELSCF convergence status.

```
.NOAMFC
```

.NOAMFC

Discard selected centers from AMFI calculations. Related the HAMILTONIAN_ .BSS keyword, whose numerical value *axyz* must have y=0 (i.e. ‘spin-free’ from the DIRAC BSS side, while AMFI provides SO terms of first order in potential).

First line specifies how many atoms are neglected, the second line gives centers to be omitted. Number for each center of calculated system is determined in the DIRAC output file.

Infinite order scalar terms “from the end”, and the (one-electron) spin orbit term (SO1) from AMFI. Centers 1 and 3 are neglected.

```
.BSS
5009
.NOAMFC
2
1 3
```

```
.ONLSCF
```

.ONLSCF

Stop DIRAC calculation after RELSCF program has finished. The (converged) AO coefficients are written to the file *RELSCF_COEF* and can be used to restart from.

This may be useful in combination with the keyword AMFI_ .AMFICH, *i.e.* the user may first run the RELSCF step for an atom with an artificial charge.

The obtained coefficients can then be used to start the RELSCF step for the neutral atom for which otherwise one would not obtain convergence.

Input example:

```
.ONLSCF
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze.md

orphan

****ANALYZE**

****ANALYZE**

Wave function analyzing tool.

This section is aimed to analyze the final Hartree-Fock wave function by using one or more analysis modules. By default none of them is activated.

.PRIVEC

.PRIVEC

Print vectors. Activates the *PRIVEC subsection.

.MULPOP

.MULPOP

Perform Mulliken population analysis Mulliken1955. Continues to *MULPOP subsection.

.PROJECTION

.PROJECTION

Perform projection analysis Faegri2001 Activates the *PROJECTION subsection.

.DENSITY

.DENSITY

Write density to a formatted file in Gaussian cube format. Activates the *DENSITY subsection.

.LOCALIZATION

.LOCALIZATION

Localize orbitals using the Pipek-Mezey criterion Dubillard2006 .

Continues to the activated *LOCALIZATION subsection. Note that in the present implementation this only works in \$C_1\$ symmetry.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze/density.md

orphan

***DENSITY**

***DENSITY**

The density can be dumped to formatted file in [CUBE file format](#) for subsequent visualization (e.g. using [Molekel](#) or [MOLDEN](#) or other programs; see also [POV-Ray](#)).

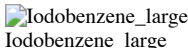
The large and small component density is written to files rhoL.cube and rhoS.cube, respectively. They can be recovered from the work area by the command

```
pam -get ``rhoL.cube rhoS.cube ...
```

Note also that the coefficient file DFCOEF must be present in order to generate the density.

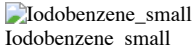
```
pam --incmo ...
```

Large component density of iodobenzene plotted using [Molekel](#) (cutoff: 0.01):



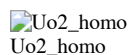
Iodobenzene_large

Small component density of iodobenzene plotted using [Molekel](#) (cutoff: 0.0001):



Iodobenzene_small

It is also possible to plot the density of an orbital by subtracting densities generated from a given coefficient file, varying the occupation. The figure shows the large component density of the HOMO of the uranyl cation obtained by subtracting from the density generated from regular occupation the density generated by reducing the occupation by two electrons.



.ORBITALS

.ORBITALS

This keyword is only available in the Development version of Dirac. Make individual cube files for each give orbital, instead of the total density.

Example:

```
.ORBITALS
4,.9
1,3
```

This generates cube files for gerade orbitals 4-9 and ungerade orbitals 1 and 3. The numbering starts from the first occupied orbital, so “positronic” orbitals have negative indices. Be aware that the cube generation can be rather slow for large basis sets. Writing hundreds of cube files can take much more time than a SCF calculation.

.DOCUBE

.DOCUBE

Specify large/small component density generation (1 = on; 0 = off).

Default:

```
.DOCUBE
1 1
```

.NCUBE

.NCUBE

Number of cells along the sides of the cube.

Default:

```
.CUBE
80 80 80
```

.CUBADJ

.CUBADJ

Adjust the volume of the cube.

Arguments: Real CUBADJ(1), CUBADJ(2)

Default:

```
.CUBADJ
4.0 8.0
```

DIRAC will look at atomic positions and then try to place the cube such that most density is included. The origin of the cube is determined by finding the minimum atomic position in the x-, y- and z-direction separately and then add CUBADJ(1). The length of the vectors spanning the cube (parallelepiped) are analogously defined by maximal atomic positions, with CUBADJ(2) added.

.PRINT

.PRINT

Print level.

Default:

```
.PRINT
0
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze/localization.md

orphan

*LOCALIZATION

*LOCALIZATION

Performs localization of molecular orbitals localization using the Pipek-Mezey criterion Pipek:Mezey. Since localized orbitals in general do not span single irreps, this module only works with C_{1v} symmetry.

The implementation uses an exponential parametrization combined with second-order minimization using the Newton trust-region method (see Sections 10.8 and 12.3 of Helgaker:book for general principles).

In short, we have some function $T(\mathbf{kappa})$ that we seek to minimize with respect to variational parameters $\mathbf{kappa}$, where $T^{\{0\}} = T(\mathbf{kappa}^{\{0\}})$ is our current expansion point. In iteration n we consider a second-order expansion of the target function

$$Q_n(\mathbf{kappa}_n) = T_n^{\{0\}} + \mathbf{T}_n^{\{1\}} \mathbf{kappa}_n + \frac{1}{2} \mathbf{kappa}_n^T \mathbf{T}_n^{\{2\}} \mathbf{kappa}_n$$

To the extent this expansion is valid, the (Newton) step to be taken is given by the linear equation

$$\mathbf{T}_n^{\{2\}} \mathbf{kappa}_n = -\mathbf{T}_n^{\{1\}}$$

We assume the second-order expansion to be valid within a trust radius h . If the proposed Newton step goes outside this radius, we will instead take a step onto the border of our trust region in an optimal direction. To find it, we set up a Lagrangian

$$L(\mathbf{kappa}, \mu) = Q(\mathbf{kappa}) - \frac{1}{2} \mu \left(\|\mathbf{kappa}\|^2 - h^2 \right)$$

where minimization in iteration n now gives

$$\left(\mathbf{T}_n^{\{2\}} - \lambda_n \mathbf{I} \right) \mathbf{kappa}_n = -\mathbf{T}_n^{\{1\}}$$

An important quantity in the algorithm is the ration between the observed and predicted changes in the target function

$$r = \frac{T_{n+1}^{\{0\}} - T_n^{\{0\}}}{Q_n - T_n^{\{0\}}} = \frac{\text{DIFFG}}{\text{DIFFQ}}$$

In Helgaker:book the following scheme for the update of the trust radius is proposed (we quote):

1. If $r > 0.75$, increase the trust radius $h_{n+1} = 1.2h_n$.
2. If $0.25 < r < 0.75$, do not change the trust radius $h_{n+1} = h_n$.
3. If $r < 0.25$, reduce the trust radius $h_{n+1} = 0.7h_n$.
4. If $r < 0$, reject the step $\mathbf{kappa}_n$ and generate a new step of trust radius $h_n \rightarrow 0.7h_n$.

.PRJLOC

.PRJLOC

Localization from projection analysis instead of Mulliken analysis. [Default: False]:

.PRJLOC

.OWNBAS

.OWNBAS

Calculate fragments in their own basis. For now it is the only option which can be used with projection analysis option (.PRJLOC) [Default: False]:

.OWNBAS

.C1READ

.C1READ

Read C1 coefficients for fragments.

.VECREF

.VECREF

First give number of fragments to project onto. Then for each fragment give filename of MO coefficients and string specifying which orbitals will be used in the projection (orbital_strings):

```
.VECREF
3
DF01AT
1..6
DFH1AT
1
DFH2AT
1
```

.HESLOC

.HESLOC

Define model of the Hessian [Default: FULL].

Use full Hessian:

```
.HESLOC
FULL
```

Use diagonal approximation of the Hessian. If using this approximation, there is danger of converging to the stationary point instead of minima:

```
.HESLOC
DIAG
```

First the diagonal approximation of the Hessian will be used and after reaching convergence criterion the localization will switch to the construction of the full Hessian:

```
.HESLOC
COMB
```

```
.CHECK
```

.CHECK

This keyword can be used only in combination with .HESLOC = COMB. The full Hessian will be calculated only at the last iteration. This way one can check if the procedure converged to the minimum.

```
.THFULL
```

.THFULL

Convergence threshold for the localization process. It applies on the functional value of the localization criteria when full Hessian is calculated. It is used in .HESLOC = FULL scheme and in the second part of the .HESLOC = COMB scheme. When not specified gradient criterion or criterion of the number of negative eigenvalues equals zero will be used:

```
.THFULL
1.0D-13
```

```
.THGRAD
```

.THGRAD

Convergence threshold for the localization process. It applies on the gradient of the localization functional. It is used in .HESLOC = FULL or DIAG scheme and in the second part of the .HESLOC = COMB scheme. When not specified the functional value criterion or criterion of the number of negative eigenvalues equals zero will be used:

```
.THGRAD
1.0D-17
```

```
.THDIAG
```

.THDIAG

Convergence threshold for the localization process. It applies on the functional value of the localization criteria when diagonal Hessian is calculated. It can be used with the .HESLOC = DIAG keyword. If .HESLOC = COMB it is used to switch from the first to the second stage of the convergence process:

```
.THDIAG
1.0D-05
```

```
.SELECT
```

.SELECT

Select subset of MOs for localization (use `orbital_strings` syntax). Currently only Pipek-Mezey localization is implemented, therefore localize only occupied orbitals. Localization of virtual orbitals is not recommended.

```
.SELECT
1..5
```

```
.MAXITR
```

.MAXITR

Maximum number of iterations. Default:

```
.MAXITR
100
```

```
.PRINT
```

.PRINT

Set print level. Default:

```
.PRINT
1
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze/mulpop.md

orphan

*MULPOP

*MULPOP

Mulliken population analysis of molecular orbitals is performed in AO basis. The analysis is based on the concept of *labels*. Each basis function is labeled by its functional type and center (and boson irrep when in SO basis, the default). The labels are given in the output. A set of primitive labels may be collected to group labels as specified by the user.

.VECPOP

.VECPOP

For each fermion irrep, give an `orbital_strings` of molecular orbitals to analyze.

Default: Analyze the occupied electronic molecular orbitals.

.NETPOP

.NETPOP

Give Mulliken net/overlap populations in addition to gross populations.

.LABEL

.LABEL

Use pre-defined labels for use in Mulliken population analysis. The following definitions are recognized by DIRAC:

.LABEL
ATOM

Provide labels for individual atoms. This is useful for getting Mulliken charges.

.LABEL
SHELL

Provide labels for individual orbital types.

.LABDEF

.LABDEF

Define labels for use in Mulliken population analysis.

Give first the number of labels to define, then the label name (12 characters) and `orbital_strings` for each fermion irrep.

Example:

```
.LABDEF
4 (total number of lines to follow)
Re s      1
Re p      2..4
Re d      5..10
Re f      11..20
...       21..35
```

Pitfall:

If you decide to redefine labels for the population analysis you have to reassign **all** of them, otherwise Dirac is unable to continue. The list of functions with the initial label definitions (*default: SO-labels*) can be drawn from the DIRAC output starting at the section :

GETLAB: SO-labels

```
Large components: 35
1  L Ag AR s      2  L Ag AR dxx      3  L Ag AR dyy      4  L Ag AR dzz      etc.
```

.AOLAB

.AOLAB

Base definition of labels on AO basis.

Default: Base definition of labels on SO basis.

.INDSML

.INDSML

Use individual small component labels.

Default: Gather all small component functions belonging to a given center and irrep, irrespective of function type.

.PRINT

.PRINT

Print level.

Default:

.PRINT
0

.PR TOL

.PR TOL

Only labels with contribution greater than this tolerance value are printed.

Default:

.PR TOL
0.01

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze/privec.md

orphan

*PRIVEC

*PRIVEC

Print vectors read from DFCOEF.

.VECPRI

.VECPRI

For each fermion irrep, give an `orbital_strings` of orbitals to print.

Default: Print the occupied electronic solutions.

.PRICMP

.PRICMP

Separate control for printing of large and small components.

Default: Print large components.

.PRICMP
1 0

Print only small components:

.PRICMP
0 1

.AOLAB

.AOLAB

Print vectors in AO basis.

Default: Print vectors in SO basis.

.PRINT

.PRINT

Print level.

Default:

```
.PRINT
0
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze/prjana.md

orphan

*PROJECTION

***PROJECTION**

The current solution may be projected down onto another set of coefficients generated from the basis, e.g. one may project molecular solutions down onto atomic solutions in order to evaluate atomic contributions. The coefficients of the fragments are read simultaneously and the overlap between them is taken into account Faegri2001 .

The projection analysis can be thought of as a Mulliken analysis based on the atomic (fragment) orbitals; it is therefore very much less sensitive to the basis set.

Normally, the fragments are calculated in the full molecular basis. This is done by zeroing charges of the remaining atoms of the molecule in the basis set input and adjusting occupation in the menu file. It is, however, possible to calculate the fragments in the own basis (a subset of the full molecular basis) using `PROJECTION_.OWNBAS`. The advantage is faster calculations and conservation of atomic symmetry for atomic fragments. To avoid working with symmetry-combinations of atomic centers for the fragments, it may sometimes be advantageous to dump molecular coefficients in C_1 symmetry using `GENERAL_.ACMOUT` under `**GENERAL` and do the analysis without symmetry. When `PROJECTION_.OWNBAS` is used, one then needs to calculate each atomic type only once in its own basis in order to do the complete analysis.

```
.VECPRJ
```

.VECPRJ

For each fermion irrep, give an `orbital_strings` of orbitals to analyze.

Default: Analyze the occupied electronic solutions.

```
.VECREF
```

.VECREF

First give number of fragments to project onto, and then for each fragment give filename of MO coefficients and the number of symmetry-independent nuclei in this fragment and for each fermion irrep, give an `orbital_strings` of reference orbitals.

Example:

```
.VECREF
4
AFH1XX
1
1
AFH2XX
1
1
AFX1XX
1
1..43
AFX2XX
1
1..43
```

```
.OWNBAS
```

.OWNBAS

Calculate fragments in their own basis.

This keyword must be used with some care as the list of fragments is assumed to be identical to that of symmetry independent centers.

```
.C1READ
```

.C1READ

Read C1 coefficients for fragments.

```
.ATOMS
```

.ATOMS

Analyze molecular orbitals in terms of atomic fragments. For each atomic type give filename of AO coefficients and an orbital string (see `orbital_strings`) of atomic orbitals to include. DIRAC assumes that the atomic orbitals are calculated in their proper atomic basis *without* symmetry, that is, using the `GENERAL_.ACMOUT` keyword and the resulting DFACMO file.

Example:

```
.ATOMS
AFHXXX
1
AFXXX
1..43
```

```
.POLREF
```

.POLREF

If the polarization contribution is too big, the projection analysis loses its meaning. By activating this keyword Intrinsic Atomic Orbitals (IAOs), as [formulated](#) by Gerald Knizia, are generated, thus eliminating completely the polarization contribution.

```
.PROTHR
```

.PROTHR

Set threshold for absolute value of projection coefficients to be printed.

Default:

```
.PROTHR
0.01
```

```
.WGPOP
```

.WGPOP

Split overlap densities according to weight of contributions.

```
.PRINT
```

.PRINT

Print level.

Default:

```
.PRINT
0
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/analyze/rho1.md

orphan

```
*RHO1
```

*RHO1

For each unique pair (A,B) of centers a formatted file is created, containing four columns of numbers: The origin is placed at atom A, and the first column gives the coordinate in Angstroms along the line between centers A and B. The following columns contain the total, large and small component density, respectively (in atomic units). The file is named 'RH',NAMN(ICENTA),NAMN(ICENTB),IDEGB where ICENTB is the name of symmetry-independent center B and IDEGB the numbering of the dependent center. Underscores are inserted wherever there are blanks. The generation of the density requires the coefficient file DFCOEF. Note that one can in principle obtain the density along any line simply by introducing ghost centers.

For uranyl one may use the command:

```
pam --incmo --get ``RHU___0___1 RHU___0___2 ...
```

```
.PRINT
```

.PRINT

Print level. Default:

```
.PRINT
0
```

```
.MESH
```

.MESH

Step length for grid. Default:

```
.MESH
0.01
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/data.md

orphan

Data Handling in DIRAC

Since release DIRAC23 all data suitable for restart and analysis purposes is stored on the file CHECKPOINT.h5

This file is in hdf5 format allowing for easy export of the generated data to other formats. The structure is given by the data schema of which some important elements are highlighted below. This is usually so-called compound data, also called data types, consisting of multiple individual elements. A full description can be found in the file utils/DIRACschema.txt, we will merely summarize the most useful data here. For users familiar with Jupyter notebooks we recommend having a look at the directory demo_notebooks that contains the Jupyter Notebook DiracDataTutorial.ipynb that demonstrates how data can be imported in Python for further processing. Similarly you may consult the notebook TrexioConverter.ipynb to see how data can be converted to the TREXIO format. For developers we furthermore recommend consulting the section on the CHECKPOINT file in the developer manual.

input

This section contains of two sets of data: the molecule data type and the aobasis data type.

- The molecule data type contains the topology of the molecule with coordinates in xyz format, nuclear symbols and charges. - The aobasis data type contains all information regarding the employed atomic orbital basis: location of the expansion centers, exponential parameters, contraction coefficients, etc.

result

The amount of data in this section depends on the calculation that is carried out. Of interest for analysis is typically the data type mobasis that contains the molecular orbitals and their energies. This data type is found for instance in the result/wavefunctions/scf subsection that also contains the total energy of the SCF wavefunction.

Not all data is yet written to the CHECKPOINT file, for energies of wave functions other than SCF or MP2 it is still necessary to consult the standard output file. In future releases the amount of data stored on CHECKPOINT.h5 will likely increase, but the aim is to keep this file concise, large datasets like transformed 2-electron integrals will still be stored in a different file.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/dft.md

orphan

*DFT

*DFT

Introduction

In this link the directives for modifying the DFT calculation are described (e.g. modifying grid, using ALDA and much more)

typical input (here correcting LDA):

```
**HAMILTONIAN
.DFT
LDA
*DFT
.SAOP
LBA $\alpha$ 
```

Single line following .SAOP : asymptotic potential (currently LB94 or LBA α).

The specification of the functional is described in connection with the .DFT option in the HAMILTONIAN_.DFT section

The DFT module is described in references Schipper2000, Saue2002, Fossgaard2003.

It uses standard non-relativistic functionals. This is justified in many cases since various studies, like refs. Varga1999, Varga2000, Mayer1996, indicate that relativistic corrections to the exchange-correlation functionals have a negligible effect on spectroscopic constants; however, for example for core properties further studies are necessary.

Integration algorithm and thresholds

.SCREENING

.SCREENING

Specify the screening threshold used in the BLAS-3 integration scheme.

Default:

```
.SCREENING
1.0D-18
```

```
.POINTWISE
```

.POINTWISE

Switch to old (unscreened) BLAS-2 integration scheme (default before DIRAC11) instead of the new BLAS-3 scheme (default since DIRAC11).

```
.TINYDENS
```

.TINYDENS

Specify the tiny density threshold. Grid points with a density smaller than this threshold do not contribute to XC matrix elements.

Default:

```
.TINYDENS
1.0D-14
```

Asymptotic correction

```
.GRAC
```

.GRAC

Activate the gradient regulated asymptotic correction scheme (GRAC), Gruning2001, followed by two lines of input.

First line: Asymptotic potential (currently LB94 or LBalpha).

Second line (free format): Parameters α and β (see reference Gruning2001) the ionization potential, and the threshold for difference in HOMO eigenvalue below which asymptotic correction is switched on. A sufficiently converged density is needed before the correction is activated (before the bulk potential is shifted in order to reproduce the desired IP (see reference Gruning2001))

Typical input (here correcting PBE0):

```
**HAMILTONIAN
.DFT
PBE0
*DFT
.GRAC
LB94
0.5 40.0 0.79248 1.0D-6
```

```
.SAOP!
```

.SAOP!

Activate the statistical averaging of (model) orbital potentials (SAOP) as defined in reference Schipper2000. This implies (and activates) the ALDA kernel. The functional under HAMILTONIAN_.DFT is expected to be GLLBhole. The asymptotic potential is LBalpha.

Recommended input:

```
**HAMILTONIAN
.DFT
GLLBhole
*DFT
.SAOP!
```

```
.SAOP
```

.SAOP

The more general version of DFT_.SAOP!

Followed by a single line of input: asymptotic potential (currently LB94 or LBalpha).

Typical input (here correcting LDA):

```
**HAMILTONIAN
.DFT
LDA
*DFT
.SAOP
LBalpha
```

Spin magnetization TDDFT

```
.NOSAOP
```


.NOSAOP

Turn off spin density contribution to XC response.

.COLLINEAR

.COLLINEAR

Use the collinear approximation as a definition of the spin density.

$$s = \{m_z\} = \{\sum \lim_{i \rightarrow \infty} \phi_i^\dagger \{\sigma_z\} \phi_i\}$$

instead of the default noncollinear definition

$$s = |\mathbf{m}|$$

.BETASIGMA

.BETASIGMA

use

$$\mathbf{m} = \sum_i \phi_i^\dagger \beta \mathbf{\sigma} \phi_i$$

instead of the default form

$$\mathbf{m} = \sum_i \phi_i^\dagger \mathbf{\sigma} \phi_i$$
Adiabatic local density approximation (ALDA)

.ALDA

.ALDA

Approximate all functional derivatives beyond the xc potential by SVWN derivatives. For hybrid functionals exact exchange is switched off in the solution of the response equation.

.XALDA

.XALDA

Same as DFT_.ALDA but keep the fraction of exact exchange of the xc functional under HAMILTONIAN_.DFT.

.ALDA+

.ALDA+

Use DFT_.ALDA+ only for the Hermitian contribution (density contribution) and use the proper xc kernel for the anti-Hermitian part (spin density contribution).

.ALDA-

.ALDA-

Use DFT_.ALDA- only for the anti-Hermitian contribution (spin density contribution) and use the proper xc kernel for the Hermitian part (density contribution).

.XALDA+

.XALDA+

Use DFT_.XALDA+ only for the Hermitian contribution (density contribution) and use the proper xc kernel for the anti-Hermitian part (spin density contribution).

.XALDA-

.XALDA-

Use DFT_.XALDA- only for the anti-Hermitian contribution (spin density contribution) and use the proper xc kernel for the Hermitian part (density contribution).

Other functionality

.GAUNTSSCALE

.GAUNTSSCALE

Scale Gaunt integrals (if included) with the same factor as for Hartree-Fock exchange. This means that hybrid functionals include fractional HF Gaunt interaction, and pure functionals no HF Gaunt interaction at all. If this option is not given the Gaunt integrals will be included to 100%, meaning that even an LDA calculation will include full Hartree-Fock Gaunt interaction when the .GAUNT keyword is given in **HAMILTONIAN.

.OVLDIAG

.OVLDIAG

Activate the overlap diagnostic for TD-DFT calculations of excitation energies according to Peach2008. This is only available in the development version.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/dftcfun.md

orphan

Density-functionals available in DIRAC using the HAMILTONIAN_.DFT keyword

Below is a list of density functionals available with the keyword HAMILTONIAN_.DFT in DIRAC:

Exchange functionals

- LDA exchange Dirac1930
- Becke 1988 Becke1988
- The correction term to LDA proposed by Becke Becke1988
- Perdew–Wang 1986 Perdew1986

Correlation functionals

- The Vosko–Wilk–Nusair LDA correlation functional (VWN5) Vosko1980
- The Lee–Yang–Parr functional Lee1988
- Perdew 1986 Perdew1986

Predefined combinations**LDA functionals**

The explicit definitions in terms of GGAKEY is also given.

LDA
GGAKEY Slater=1.0 VWN5=1.0

SVWN5
GGAKEY Slater=1.0 VWN5=1.0

SVWN3
GGAKEY Slater=1.0 VWN3=1.0

Xalpha
...

GGA functionals

The explicit definitions in terms of GGAKEY is also given.

BLYP
GGAKEY Slater=1.0 Becke=1.0 LYP=1.0

PBE
GGAKEY PBEx=1.0 PBEC=1.0

RPBE
GGAKEY RPBEX=1.0 PBEC=1.0

BP86
GGAKEY Slater=1.0 Becke=1.0 P86c=1.0 PZ81=1.0

BPW91
GGAKEY Slater=1.0 Becke=1.0 PW91c=1.0

PP86
GGAKEY PW86x=1.0 P86c=1.0

KT1
GGAKEY Slater=1.0 VWN5=1.0 KT=-0.006

KT2
GGAKEY Slater=1.07173 VWN5=0.576727 KT=-0.006

KT3
GGAKEY Slater=1.092 OPTX=-0.925452 LYP=0.864409 KT=-0.004

OLYP
GGAKEY Slater=1.05151 OPTX=-1.43169 LYP=1.0

Hybrid functionals

The explicit definitions in terms of GGAKEY is also given.

B3LYP
GGAKEY Slater=0.8 Becke=0.72 HF=0.2 VWN5=0.19 LYP=0.81

B3LYP-G
GGAKEY Slater=0.8 Becke=0.72 HF=0.2 VWN3=0.19 LYP=0.81

B3P86
GGAKEY Slater=0.8 Becke=0.72 VWN5=0.19 P86c=0.81

B3P86-G
GGAKEY Slater=0.8 Becke=0.72 VWN3=0.19 P86c=0.81

PBE0
GGAKEY PBEx=0.75 PBEC=1.0 HF=0.25 HF

PBE38
GGAKEY PBEx=0.625 PBEx HF=0.375 PBEC=1.0

Long-range corrected functionals

The explicit definitions in terms of CAM is also given.

CAMB3LYP
CAM p:alpha=0.19 p:beta=0.46 p:mu=0.33 x:slater=1 x:becke=1 c:lyp=0.81 c:wn5=0.19

FILE: www.diracprogram.org/doc/release-26/_sources/manual/dirac.md

orphan

****DIRAC**

****DIRAC**

By default DIRAC only activates the generation of the basis set (e.g. kinetic balance conditions for small component functions) and the one-electron modules. Two-electron integrals over the atomic basis functions will be calculated when needed by the job modules given below.

We recommend that you use the [Dalton](http://daltonprogram.org) program package if you want to e.g. save the two-electron integrals to disk for another purpose, this is not possible with DIRAC.

.WAVE FUNCTION

.WAVE FUNCTION

Activates the wave function module(s). This activates the reading of the ****WAVE FUNCTION** section, where the desired wave function type(s) must be specified. If you read in converged MO coefficients from the CHECKPOINT file and you want to skip the SCF step and proceed directly to properties or the post-SCF step, then a trick to do this is to comment out (or remove) this keyword.

.ANALYZE

.ANALYZE

Activates the Hartree–Fock or Kohn–Sham analysis module. This activates the reading of the ****ANALYZE** section.

.PROPERTIES

.PROPERTIES

Activates the property module (which will call the integral module for property integrals). This activates the reading of the ****PROPERTIES** section.

.OPTIMIZE

.OPTIMIZE

Activates the geometry optimization. This activates the reading of the ***OPTIMIZE** subsection.

.4INDEX

.4INDEX

Explicitly activates the transformation of integrals to molecular orbital basis. This activates the reading of the ****MOLTRA** section.

These transformed integrals are currently only used by the ****RELCC**, **RELADC**, ****POLPRP**, **DIRRCI**, and ***LUCITA** modules, and if one of these three modules are requested under ****WAVE FUNCTION**, then this flag is automatically activated unless **.NO4INDEX** is specified in this input module.

By default, molecular orbitals with orbital energy between -10 and +20 hartree are included, this can be modified in the ****MOLTRA** section.

.NO4INDEX

.NO4INDEX

Do not automatically activate integral transformation to molecular orbital basis if any of ****RELCC**, **RELADC**, ****POLPRP**, **DIRRCI**, and ***LUCITA** modules are requested under ****WAVE FUNCTION**.

This keyword is utilized when repeating correlated CC or CI calculations (with different parameters for instance) based on saved files after the integral transformation.

.TITLE

.TITLE

Title line (max. 50 characters). Example:

```
.TITLE  
my first DIRAC calculation
```

.INPTEST

.INPTEST

Input test - no job modules are called, only verification of DIRAC input files. It is often useful to start a new set of calculations with an input test in order to check that input file processing is correct before submitting your (long-term run) job.

.ONLY INTEGRALS

.ONLY INTEGRALS

Stop after the calculation of the one-electron integrals for the Hamiltonian and the one-electron integrals specified under ****INTEGRALS**. The integrals are written to disk.

.XMLOUT

.XMLOUT

Create the output xml-file containing selected data (in development).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/effqed.md

orphan

***EFFQED**

***EFFQED**

Options for the numerical integration of effective QED potentials described in Sunaga2022. An example of the input is shown in [tutorial](#).

.GRASP

.GRASP

Activate the radial grid of GRASP code.

Default:

Lindh's radial grid

.ANGQED

.ANGQED

Angular momentum of Lebedev quadrature for the numerical intebration of effective QED potential.

Default:

```
.ANGQED  
15
```

.UEHCUT

.UEHCUT

Due to the very local nature of the effective QED potentials, the upper limit of the radial integration over a potential associated with atomic center A is cut off. The radial integration is cut off based on the Uehling potential and is performed in a shorter region.

Default:

Cutoff based on Flambaum-Ginges low-frequency term

.MINIMUM

.MINIMUM

Decision of the smallest nuclear charge Z to include effective QED potentials.

Default:

.MINIMUM
19

The low-frequency (LF) term of the Flambaum-Ginges potential has the range of the 1s-orbital of a hydrogen-like atom, so the LF term of light atoms are rather widely distributed and may overlap with the neighboring atoms in the molecule. This is the reason why light elements are skipped in the default.

.ALLATOM

.ALLATOM

Incorporating effective QED potentials for all atoms in the molecule. *Default:*

The element with Z < 19 is not calculated.

.SCREEN

.SCREEN

The AO basis set less than the threshold in a grid point is omitted in the numerical integration, which is done in the region close the nucleus. For example, in AB molecule, tight exponent of atom B would not be considered in the numerical integration of atom A's effQED potentials. *Default:*

.SCREEN
1.0E-10

FILE: www.diracprogram.org/doc/release-26/_sources/manual/fde.md

orphan

*FDE

*FDE

Frozen density embedding (FDE) directives.

General information on the FDE method can be found in recent reviews (e.g. Jacob2013, Gomes2012a), whereas details concerning the Dirac implementation are found in Gomes2008, Hofener2012 and Hofener2013.

Data import

If Dirac is used to calculate the subsystem of interest, data from the subsystem(s) making up the environment must be fed into the program.

.EBPOT

.EBPOT

Specifies the name of the file containing the embedding potential over an integration grid.

Default:

.EBPOT
EBPOT

.FRDENS

.FRDENS

Specifies the name of the file containing the “frozen” quantities (integration grid, electrostatic potential, electrostatic potential due to external fields (e.g. point charges), density and (x,y,z) components of the density gradient).

Default:

.FRDENS
FRZDNS

Embedding potential

If the FDE_.EMBPOT option was selected, the program will simply compute matrix elements of the the AO basis used by the program, and add it to the one-electron Fock matrix. Because of that, this option can be used with any wavefunction in Dirac. The ground-state calculation in this case corresponds to the “fixed potential” scheme of Gomes2008.

However, if FDE_.EMBPOT is not specified, the program implicitly selects the FDE_.UPDATE option, and assumes a frozen density has been provided (see FDE_.FRDENS).

.UPDATE

.UPDATE

This option requires that the embedding potential for the ground-state be calculated using the “frozen” quantities and those for the subsystem treated with Dirac. The non-additive contributions are calculated with the orbital-free kinetic energy and exchange-correlation density functionals specified under FDE_.NAKEF and FDE_.NAXCF, respectively.

Currently this option is only supported during the SCF procedure, so it does not have any effect for correlated wavefunctions (e.g. MP2, CI, CC).

The FDE_.EMBPOT and this option cannot be specified at the same time.

.NAKEF

.NAKEF

Specifies the orbital-free kinetic energy density functional used to calculate the non-additive kinetic energy contributions to the ground-state embedding potential (see FDE_.UPDATE) and/or response contributions (see FDE_.RSP or FDE_.RSPLDA)

Default:

```
.NAKEF
kin_tf
```

Which corresponds to the Thomas-Fermi kinetic energy functional. Other available functionals are: PW91k (kin_pw91).

.NAXCF

.NAXCF

Specifies the exchange-correlation density functional used to calculate the non-additive kinetic energy contributions to the ground-state embedding potential (see FDE_.UPDATE) and/or response contributions (see FDE_.RSP or FDE_.RSPLDA)

Default:

```
.NAXCF
lda
```

Where lda is equivalent to specifying “slaterx=1.0 vwn=1.0” (without quotes). Other available functionals are: pbe (equivalent to “pbex=1.0 pbex=1.0”), blyp (equivalent to “beckex=1.0 lypc=1.0”), pp86 (equivalent to “pw86x=1.0 p86c=1.0”)

Response contributions

For response-based approaches (TDHF, TDDFT), contributions from FDE to the active subsystem can be included through the keywords below (the default is to not include any). For correlated wavefunctions (e.g. MP2, CI, CC) these do not yet have any effect.

These options can be used together with either FDE_.UPDATE or FDE_.EMBPOT, but require that “frozen” quantities are present (see FDE_.FRDENS)

.RSP

.RSP

Specifies that FDE response contributions should be calculated employing the density functionals specified under FDE_.NAKEF and FDE_.NAXCF, if any.

.RSPLDA

.RSPLDA

Specifies that the non-additive exchange-correlation and kinetic energy FDE response contributions should be calculated with the LDA and Thomas-Fermi functionals, respectively.

Data export

Dirac can also provide ground-state data (density and density gradient, electrostatic potential etc) from the subsystem in question over a grid, so that these can be used by other programs.

.GRIDOU

.GRIDOU

[!WARNING] Development version only.

Specifies the grid over which the quantities will be calculated and exported, and enables the export. The input is a XYZW file, and the output is in XML format. The original file is overwritten.

Default:

.GRIDOU
GRIDOUT

.LEVEL

.LEVEL

[!WARNING] Development version only.

Specifies which kind of wavefunction will be used to obtain the exported quantities

Default:

.LEVEL
DHF

which corresponds to SCF (Hartree-Fock, DFT) ones. Also supported: MP2.

.EXONLY

.EXONLY

[!WARNING] Development version only.

This option forces the program to skip the calculation of any FDE contributions, and proceed to exporting the

[!WARNING] Development version only.

Magnetic properties with London atomic orbitals - FDE contributions to the property gradient

In calculations of magnetic properties with London atomic orbitals (LAOs), additional contributions from FDE to the active subsystem should be included in the property gradient (:cite: olejniczak2017).

$$E_{[1],\text{emb}}^{\{B\}} = - \int v_{\text{emb}}^I \Omega_{ia}^{\{B,I\}} - \iint w_{\text{emb}}^{\{I,I\}} \Omega_{ia}^{\{I\}} \Omega_{jj}^{\{B,I\}} - \iint w_{\text{emb}}^{\{I,II\}} \Omega_{$$

where $\Omega_{pq}^{\{B\}}$ is the first-order perturbed overlap distribution, which in a basis of London orbitals consists of two terms

$$\Omega_{pq}^{\{B\}} = \frac{i}{2} (R_{MN} \times r) \Omega_{pq} + (T_{pt}^{B*} \Omega_{tq} + \Omega_{pt} T_{tq}^B)$$

“direct” LAO term (the first term) and “reorthonormalization” term (two last terms in brackets).

FDE contributions to property gradient can be included by the keywords presented below.

.LAO11

.LAO11

This keyword turns on the FDE-LAO contributions to the property gradient dependent on the embedding potential (v_{emb}^I) and the uncoupled embedding kernel ($w_{\text{emb}}^{\{I,I\}}$). This is the advised option. Other contributions to the property gradient can be included or excluded by using expert keywords (below).

.LDPT

.LDPT

This keyword turns on the term dependent on the embedding potential (v_{emb}^I) only. Calculations with the FDE_.LDPT keyword are less demanding than calculations with the FDE_.LAO11 keyword, but may lead to inaccurate results, especially for heavy elements.

.NOLDPT

.NOLDPT

This keyword turns off the term dependent on the embedding potential (v_{emb}^I).

.L11KR

.L11KR

This keyword turns on the terms dependent on the uncoupled embedding kernel ($w_{\text{emb}}^{\{I,I\}}$).

.NL11KR

.NL11KR

This keyword turns off the terms dependent on the uncoupled embedding kernel ($w_{\text{emb}}^{\{I,I\}}$).

.LDKR

.LDKR

This keyword corresponds to the term dependent on the uncoupled embedding kernel ($w_{\text{emb}}^{I,I}$), which involves only the “direct” LAO contribution.

.NOLDKR

.NOLDKR

This keyword excludes the “direct” LAO contribution to the uncoupled embedding kernel ($w_{\text{emb}}^{I,I}$).

.LRKR

.LRKR

This keyword corresponds to the term dependent on the uncoupled embedding kernel ($w_{\text{emb}}^{I,I}$), which involves only the “reorthonormalization” LAO contribution.

.NOLRKR

.NOLRKR

This keyword excludes the “reorthonormalization” LAO contribution to the uncoupled embedding kernel ($w_{\text{emb}}^{I,I}$).

.LAO12

.LAO12

This keyword turns on the FDE-LAO contributions to the property gradient dependent on the coupled embedding kernel ($w_{\text{emb}}^{I,II}$). This option includes both, the term dependent on the non-additive part of the coupled embedding kernel and the Coulomb term. Each of these terms can be turned on by separate keywords (below). If this keyword is employed, it is also necessary to import perturbed densities using the FDE_.PERTIM keyword.

.LAOFRZ

.LAOFRZ

This keyword corresponds to the term dependent on the non-additive part of embedding kernel involving the coupling between two subsystems ($w_{\text{emb}}^{I,II}$). In order to calculate this term, it is necessary also to use the keyword FDE_.PERTIM.

.LFCOUL

.LFCOUL

This keyword corresponds to the term dependent on the Coulomb part of embedding kernel involving the coupling between two subsystems ($w_{\text{emb}}^{I,II}$). In order to calculate this term, it is necessary also to use the keyword FDE_.PERTIM.

.PERTIM

.PERTIM

This keyword should be used whenever FDE_.LAOFRZ keyword is present. It commands the program to read the contributions (“direct” LAO contribution and “reorthonormalization” contribution) to the perturbed density in LAO basis from files.

Default:

```
.PERTIM
pertden_direct_lao.FINAL
pertden_reorth_lao.FINAL
```

.FRZNOS

.FRZNOS

This keyword means that no spin-density contributions to the perturbed density will be used in FDE calculations.

Magnetizability with London atomic orbitals - FDE contributions to the expectation value of the magnetizability tensor

The FDE calculations of the magnetizability tensor with London atomic orbitals (LAOs), require the FDE-LAO contributions to the property gradient (presented above) and the FDE-LAO contributions to the expectation value of the magnetizability tensor. The latter involve the terms dependent on the embedding potential (v_{emb}^T), uncoupled embedding kernel ($w_{\text{emb}}^{I,I}$) and coupled embedding kernel ($w_{\text{emb}}^{I,II}$).

.EMAFDE

.EMAFDE

This keyword turns on all FDE-LAO contributions to the expectation value part of the magnetizability tensor. This is the advised option. It is possible to exclude each contribution by using expert keywords (presented below).

.MNOPOT

.MNOPT

This keyword turns off the contribution to the expectation value part of the magnetizability tensor dependent on the embedding potential (v_{emb}^{Γ}).

.MNOUKE

.MNOUKE

This keyword turns off the contribution to the expectation value part of the magnetizability tensor dependent on the uncoupled embedding kernel ($w_{\text{emb}}^{\{I,I\}}$).

.MNONKE

.MNONKE

This keyword turns off the contribution to the expectation value part of the magnetizability tensor dependent on the non-additive part of the coupled embedding kernel ($w_{\text{emb}}^{\{I,II\}}$).

Debug/expert options

.SKIPX

.SKIPX

Specifies that the non-additive exchange-correlation contributions are not to be calculated.

.SKIPK

.SKIPK

Specifies that the non-additive kinetic energy contributions are not to be calculated.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/general.md

orphan

**GENERAL

****GENERAL**

.DIRECT

.DIRECT

Direct evaluation of two-electron integrals for Fock matrices (all two-electron integrals for other uses, e.g. CI, CCSD, MCSCF, are always evaluated directly, i.e. never read from disk).

The default is to evaluate LL, SL, and SS integrals directly (1 = evaluate directly; 0 = do not evaluate directly):

```
.DIRECT
1 1 1
```

.SPHTRA

.SPHTRA

Transformation to spherical harmonics embedded in the transformation to orthonormal basis; totally symmetric contributions are deleted.

The default is a spherical transformation of large and small components, respectively (1 = on; 0 = off):

```
.SPHTRA
1 1
```

The transformation of the large components is a standard transformation from Cartesian functions to spherical harmonics. The transformation of the small components is modified, however, in accordance with the restricted kinetic balance relation.

.PCMOUT

.PCMOUT

Write MO coefficients to the formatted file DFPCMO. This is useful for porting coefficients between machines with different binary structure. For reading the DFPCMO file, there is no keyword - simply copy this file to the working directory.

.ACMOUT

.ACMOUT

Write coefficients in C1 symmetry to the unformatted file DFACMO.

.ACMOIN

.ACMOIN

Import coefficients in C1 symmetry from the unformatted file DFACMO to current symmetry. This assumes that the current symmetry is lower than the symmetry used for obtaining the original coefficients.

.LOWJACO

.LOWJACO

Use Jacobi diagonalization in the Löwdin orthogonalization procedure (subroutine LOWGEN). This is much slower than the default Householder method but does not mix AOs in the case of block-diagonal overlap matrix.

.DOJACO

.DOJACO

Use the Jacobi method for matrix diagonalization (currently limited to real matrices). The default [Householder](#) method is generally more efficient, but may mix degenerate eigenvectors of different symmetries.

.QJACO

.QJACO

Employ pure Jacobi diagonalization of quaternion matrixes. Properly handles degenerate eigenvectors. Slower than .DOJACO and exclusive. Experimental option.

.LINDEP

.LINDEP

Thresholds for linear dependence in large and small components; refer to the smallest acceptable values of eigenvalues of the overlap matrix. The default is:

```
.LINDEP
1.0D-6 1.0D-8
```

.RKBIMP

.RKBIMP

Import SCF coefficients calculated using restricted kinetic balance (RKB) and add the UKB component (all small component). This option is a nice way to get (unrestricted) magnetic balance in response calculations involving a uniform magnetic field (e.g. NMR shielding and magnetizability), in particular when combined with London orbitals, which makes the magnetic balance *atomic*.

.PRJTHR

.PRJTHR

RKBIMP projects out the RKB coefficients transformed to orthonormal basis and then adds the remained, corresponding to the UKB complement. With the keyword you can set the threshold for projection. The default is:

```
.PRJTHR
1.0D-5
```

.PRINT

.PRINT

General print level. The higher the number is, the more output the user gets. Option of this type is useful for code debugging. By default set to:

```
.PRINT
0
```

.CVALUE

.CVALUE

Speed of light in a.u. Default (from CODATA22):

```
.CVALUE
137.035999177
```

.LOGMEM

.LOGMEM

Write out a line in the output for each memory allocation done by DIRAC. This is mainly useful for programmers or for solving out-of-memory issues.

.CODATA

.CODATA

Select the fundamental physical constants used throughout the code. The values are taken from different CODATA sets. Current available data sets are requested by using the keywords CODATA86, CODATA98, CODATA02, CODATA06, CODATA10, CODATA14, CODATA18, or CODATA22. For the weak mixing angle and the Fermi coupling constant, there are also values reported in 1994 by the Particle Data Group (PDG) that were not included in any of those CODATA sets. Nevertheless, they can be used by employing the keyword PDG94. When PDG94 is used, all other constants are taken from the default CODATA set. Default:

```
.CODATA
CODATA22
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/grid.md

orphan

**GRID

**GRID

The numerical integration scheme uses the Becke partitioning Becke1988a of the molecular volume into atomic ones, for which the quadrature is performed in spherical coordinates. Radial integration is carried out using the scheme proposed by Lindh *et al.* Lindh2001, while the angular integration is handled by a set of highly accurate Lebedev grids. Note that the Becke partitioning scheme generally uses Slater-Bragg radii for atomic size adjustments. For the heavier elements, with their more variable oxidation states, this can lead to errors. If the user invokes the GRID_.ATSIZE keyword, DIRAC tries to deduce relative atomic sizes from available densities, if it can find coefficients.

The **radial integration** employs an exponential grid

$$r_k = e^{kh}; \quad w_k = r_k; \quad S = \sum_{k=-\infty}^{\infty} w_k f(r_k)$$

and is specified in terms of step size h , the inner point r_1 and the outermost point r_{k_H} , chosen to provide the required relative precision R (equal to the discretization error R_D).

.RADINT

.RADINT

Specify the maximum error in the radial integration.

Default:

```
.RADINT
1.0D-13
```

.ANGINT

.ANGINT

Specify the precision of the Lebedev angular grid. The angular integration of spherical harmonics will be exact to the L-value given by the user. By default the grid will be pruned. The highest implemented value is 64.

Default:

```
.ANGINT
41
```

.NOPRUN

.NOPRUN

Turn off the pruning of the angular grid.

.ANGMIN

.ANGMIN

Specify the minimum precision of the Lebedev angular grid after pruning.

```
.ANGMIN
LEBMIN
```

Close to the nucleus, the precision of the angular grid will be less than L-value given by GRID_.ANGINT, but it will not be less than the integer LEBMIN. Note that giving a LEBMIN value greater than or equal to the L-value given by GRID_.ANGINT is equivalent to turning the pruning off.

Default:

```
.ANGMIN
15
```

```
.ATSIZE
```

.ATSIZE

Generate new estimates for atomic size ratios for use in the Becke partitioning scheme. Relative sizes for a pair of atoms A and B are calculated from their contribution to the large component density along the line connecting A and B.

```
.IMPORT
```

.IMPORT

```
.IMPORT
numerical_grid
```

Import previously exported numerical grid.

For debugging you can also create your own grid file. The grid file is formatted - in free format. A grid file for three points could look like this:

```
3
0.1  0.1 0.1    1.0
0.01 0.2 0.4    0.9
9.9  9.9 9.0    0.8
-1
```

The first integer is the number of points, then come the x-, y-, and z-coordinate, and the weight for each point. The last line is a negative integer. Works also in parallel.

You can export the DIRAC grid simply by copying back the file “numerical_grid” from the scratch directory.

```
.NOZIP
```

.NOZIP

DIRAC will try to remove redundant grid points when symmetry is present. Each reflection plane reduces the number of grid points by a factor of two. With this keyword you can turn this default symmetry grid compression off.

```
.4CGRID
```

.4CGRID

Include the small component basis in the generation of the DFT grid even if you run a 1- or 2-component calculation.

This can be useful if you want to compare to a 4-component calculation using the identical integration grid.

By default the small component is ignored in the grid generation when running 1- or 2-component DFT calculations.

```
.DEBUG
```

.DEBUG

Very poor grid - corresponds to

```
.RADINT
1,0D-3
.ANGINT
10
```

```
.COARSE
```

.COARSE

Coarse grid - corresponds to

```
.RADINT
1,0D-11
.ANGINT
35
```

```
.ULTRAFINE
```

.ULTRAFINE

A better grid than default - corresponds to

```
.RADINT
2,0D-15
.ANGINT
64
```

.INTCHK

.INTCHK

Test the performance of the grid by computing the overlap matrix numerically and analyzing the errors. In addition, the error matrix can be printed.

Default (no test):

```
.INTCHK
0
```

Error analysis:

```
.INTCHK
1
```

Print error matrix:

```
.INTCHK
2
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/groupchain.md

orphan

Symmetry-handling at the correlated level**Introduction**

The DIRAC code can handle symmetries corresponding to D_{2h} and subgroups (denoted binary groups) as well as linear supersymmetry. At the SCF level DIRAC employs a unique quaternion symmetry scheme for the fermion symmetry of the spinors which combines time reversal and spatial symmetry (Saue1999). A particularity of this scheme is that symmetry reductions due to spatial symmetry is translated into a reduction of algebra, from quaternion down to complex and possibly real algebra. This leads to a classification of the binary groups as:

- Quaternion groups: C_{1^*} , C_{i^*}
- Complex groups: C_{2^*} , C_{s^*} , C_{2h^*}
- Real group: D_{2^*} , C_{2v^*} , D_{2h^*}

In general, at the correlated level, the highest Abelian subgroup of the point group under consideration is used. All quaternion and complex subgroups are Abelian. The fermion irreps of the real groups are non-Abelian and so an Abelian complex subgroup is chosen for calculations on a correlated level.

```
\begin{aligned}
\begin{array}{lcl}D_{2^*} & C_{2v^*} & \rightarrow C_{2^*} \rightarrow D_{2h^*} \rightarrow C_{2h^*}\end{array}
\end{aligned}
```

Character tables for Abelian double groups

Below we give character tables for the Abelian double groups handled by DIRAC. The irreps are given in the same order as internally in the correlation modules of the program. This means that fermion irreps come before boson irreps.

The tables are given according to the conventions of S. L. Altmann and P. Herzog, *Point-Group Theory Tables*, Clarendon Press, Oxford, 1994 (A second corrected edition is now available free of charge [here](http://www.diracprogram.org/doc/release-26/_sources/manual/groupchain.md)). The final column of the tables give the irrep labels employed by DIRAC.

- Real groups :

```
$\mathbf{C_{1^*}}$ $E$
$A_{1/2}$ $1$ $A$
$A$ $1$ $a$
$\mathbf{C_{i^*}}$ $E$ $i$
$A_{1/2,g}$ $1$ $\phantom{-}1$ $AG$
$A_{1/2,u}$ $1$ $-1$ $AU$
$A_g$ $1$ $\phantom{-}1$ $ag$
$A_u$ $1$ $-1$ $au$
```

- Complex groups:

```
$\mathbf{C_{2^*}}$ $E$ $C_{2^*}$
$\wedge^1 E_{1/2}$ $1$ $\phantom{-}i$ $1E$
$\wedge^2 E_{1/2}$ $1$ $-i$ $2E$
$A$ $1$ $\phantom{-}1$ $a$
$B$ $1$ $-1$ $b$
$\mathbf{C_{s^*}}$ $E$ $\sigma_h$
$\wedge^1 E_{1/2}$ $1$ $\phantom{-}i$ $1E$
$\wedge^2 E_{1/2}$ $1$ $-i$ $2E$
$A^{\prime}$ $1$ $\phantom{-}1$ $a$
```

$A^{\prime}$	b				
$\mathbf{C}_{2h}$	E	C_2	i	σ_h	
$\Lambda_{1/2,g}$	1	i	1	i	$1Eg$
$\Lambda_{2/2,g}$	1	$-i$	1	$-i$	$2Eg$
$\Lambda_{1/2,u}$	1	i	-1	$-i$	$1Eu$
$\Lambda_{2/2,u}$	1	$-i$	-1	i	$2Eu$
A_g	1	1	1	1	ag
B_g	1	-1	1	-1	bg
A_u	1	1	-1	-1	au
B_u	1	-1	-1	1	bu

FILE: www.diracprogram.org/doc/release-26/_sources/manual/hamiltonian.md

orphan

**HAMILTONIAN

**HAMILTONIAN

Introduction

This section defines the electronic Hamiltonian that is to be used. Within the Born-Oppenheimer approximation the generic form of the electronic Hamiltonian is

$$H = \sum_i h(i) + \frac{1}{2} \sum_{i \neq j} g(i, j) + V_{NN}; \quad V_{NN} = \frac{1}{2} \sum_{A \neq B} \frac{Z_A Z_B}{r_{AB}}$$

where V_{NN} is the operator of the repulsion of classical nuclei. The one-electron hamiltonian $h(i)$ splits into the free-electron Hamiltonian h_0 and the electron-nucleus interaction V_{eN} . In the non-relativistic case the free-electron Hamiltonian is simply the kinetic energy operator, whereas a rest mass term is added in the relativistic case, e.g. for the Dirac (bare-nucleus) Hamiltonian

$$h_D = \beta mc^2 + c \left(\alpha \cdot \mathbf{p} \right) + V_{eN}$$

In the non-relativistic case the two-electron operator $g(i, j)$ is the instantaneous Coulomb interaction.

$$g_{\text{Coulomb}}(1, 2) = \frac{1}{r_{12}}$$

In the relativistic case, the two-electron interaction is vastly more complex, including magnetic interactions as well as retardation effects. In the relativistic framework the instantaneous Coulomb-interaction is the zeroth-order term in an expansion in c^{-2} of the full Lorentz invariant two-electron interaction. Note, however, that although the mathematical form of the Coulomb term is the same as in the non-relativistic domain, the physical content is different. For instance, in the relativistic domain the Coulomb term contains the spin-same orbit (SSO) interaction. The first-order term is the Breit interaction

$$g_{\text{Breit}}(1, 2) = -\frac{c \alpha_1 \cdot \alpha_2}{2c^2 r_{12}} - \frac{\left(c \alpha_1 \cdot \mathbf{r}_{12} \right) \left(c \alpha_2 \cdot \mathbf{r}_{12} \right)}{2c^2 r_{12}^3} + \frac{\mathbf{r}_{12} \cdot \mathbf{r}_{12}}{2c^2 r_{12}^3}$$

which can be rearranged to

$$g_{\text{Breit}}(1, 2) = g_{\text{Gaunt}}(1, 2) + g_{\text{gauge}}(1, 2) = -\frac{c \alpha_1 \cdot \alpha_2}{2c^2 r_{12}} - \frac{\mathbf{r}_{12} \cdot \mathbf{r}_{12}}{2c^2 r_{12}^3} + \frac{\mathbf{r}_{12} \cdot \mathbf{r}_{12}}{2c^2 r_{12}^3}$$

The Gaunt term, which contains the spin-other orbit interaction, is implemented at the SCF level in DIRAC.

The Dirac Hamiltonian (or effective one-electron Hamiltonians such as the Fock or Kohn-Sham operators) give electronic solutions of both positive and negative energy. 2-component relativistic Hamiltonians can be generated by a unitary decoupling transformation. The exact decoupling gives the eXact 2-Component Hamiltonian (X2C) (we use Ilias2007), whereas the Zeroth-Order Regular Approximation (ZORA), Douglas-Kroll-Hess (DKH) and Barysz-Sadlej-Snijders (BSS) Hamiltonians are generated by approximate decouplings. For more a detailed discussion of relativistic Hamiltonians, see Saue2011.

Internally the program will always work with 4-component operators that are expanded using distinct large and small component basis sets. In the transformation to an orthogonal basis set one may, however, combine large and small component functions and/or functions of different symmetry in order to obtain a matrix expansion of e.g. the spin-free modified Dirac equation or the Lévy-Leblond equation Visscher2000.

In addition one can also modify the Hamiltonian by introduction of an additional operator, e.g. describing an external field. Any additional operator defined in this section must be totally symmetric under both the molecular point group and time reversal symmetry. The latter requirement precludes the introduction of external magnetic fields.

General

.PRINT

.PRINT

Print level. Default:

.PRINT
0

.ONESYS

.ONESYS

Ignore two-particle interactions.

The keyword ensures the diagonalization of the bare nuclei Hamiltonian matrix without proceeding to the iterative SCF method.

.PHASEORIGIN

.PHASEORIGIN

Origin appearing in the London atomic orbital phase-factors. Default:

```
.PHASEORIGIN
0.0 0.0 0.0
```

[!WARNING] This is the text from the DALTON manual. We wonder if it is used for other integrals, in the code in her1int.F it is the final “else” option.

.GAUGEORIGIN

.GAUGEORIGIN

Set the gauge origin (in bohr) for an external magnetic field, e.g. for NMR shielding calculations. To set the gauge origin in angstrom use HAMILTONIAN_.GO ANG. By default gauge origin is set to the coordinate origin:

```
.GAUGEORIGIN
0.0 0.0 0.0
```

.GO ANG

.GO ANG

Same as HAMILTONIAN_.GAUGEORIGIN but reads gauge origin coordinates in angstrom instead of bohr.

.DIPORG

.DIPORG

Origin for all moment integrals, including the dipole (dipole is independent of origin only for neutral systems). By default it is set to the coordinate origin:

```
.DIPORG
0.0 0.0 0.0
```

4-component Hamiltonians

The default Hamiltonian of DIRAC is the Dirac-Coulomb Hamiltonian using the Simple Coulombic Correction (see HAMILTONIAN_.LVCORR). The Dirac-Coulomb Hamiltonian formally has no bound solution and is therefore embedded in projection operators. By default, these are the projection operators obtained iteratively in the SCF process.

.GAUNT

.GAUNT

Add the Gaunt interaction to the Hamiltonian. This will increase the computational time significantly but is important when studying spin-orbit splittings and/or performing accurate studies of light molecules. The current implementation is limited to including the Gaunt interaction in the construction of the Fock matrix and works for Hartree-Fock and DFT. For using Gaunt in combination with DFT see the *DFT section of the manual. Transformation of the Gaunt part of the two electron operator to the MO basis is not yet implemented, for this purpose we recommend the use of a molecular mean field approximation. This can be used for example in MP2/CC/FSCC/IHFSCC calculations with RELCCSD by means of the HAMILTONIAN_.X2Cmmf Hamiltonian (see also our FAQ/Tutorial pages).

For the relativistic two-component mode (see HAMILTONIAN_.X2C keyword) with AMFI contributions it uses both the spin-same and the spin-other orbit mean-field parts.

.DOSSSS

.DOSSSS

This keyword gives unmodified Dirac-Coulomb Hamiltonian which was the default until the DIRAC11 release. Explicitly including (SSISS) type Coulomb integrals does give the most accurate description of the system but does increase computational cost significantly. Use this option for high-accuracy calculations, preferably in conjunction with HAMILTONIAN_.GAUNT to also include the Gaunt correction to the two-electron interaction.

.LVCORR

.LVCORR

This keyword activates the Dirac-Coulomb Hamiltonian in which (SSISS) integrals are neglected and replaced by an interatomic SS correction (calculated as a classical repulsion term of (tabulated) small component atomic charges) Visscher1997a .

This is currently the most economical and accurate approximation to the full Dirac-Coulomb Hamiltonian and can certainly be used for the calculation of spectroscopic constants and valence properties; for core properties, testing is recommended (see also HAMILTONIAN_.LVNEW). This is the default Hamiltonian choice since DIRAC11.

.LVNEW

.LVNEW

Modification of HAMILTONIAN_.LVCORR that obtains the atomic small component charge via a Mulliken analysis instead of the original table look-up. (The problem with the table look-up is that the electrostatics in the molecule will be wrong if you have specified a basis set which does not give the correct small-electron charge because of deficiencies in the core region.)

.URKBAL

.URKBAL

Unrestricted kinetic balance.

The default is restricted kinetic balance. This is imposed by deleting unphysical solutions from the free particle positronic spectrum. This leads to a 1:1 ratio of electronic and positronic solutions. This preprojection is sensitive to linear dependencies and should therefore preferably be used in conjunction with the spherical transformation of both large and small components.

.INTFLG

.INTFLG

Specify what two-electron integrals to include. All other modules use this as the default value. By default include LL and SL, exclude SS integrals (1 = include; 0 = do not include):

```
.INTFLG
1 1 0
```

.ESHIFT

.ESHIFT

Specify an energy shift in units of mc^2 . By default the zero of the energy is aligned with the non-relativistic energy scale. To shift back to the relativistic scale, one can specify:

```
.ESHIFT
1.000
```

Turning off spin dependence

.SPINFREE

.SPINFREE

Use Dyall's spin-free Hamiltonian, Ref. Dyall1994, to obtain results without spin-orbit coupling for the four-component Hamiltonian in the default restricted kinetic balance scheme. This keyword works also for two-component relativistic Hamiltonians where one can choose between two spin-free schemes - see the HAMILTONIAN_.BSS keyword.

Note that this option should not be used for response calculations with time-antisymmetric (magnetic) operators as it will eliminate important contributions.

.NOSPIN

.NOSPIN

Implies HAMILTONIAN_.SPINFREE, but also remove all spin-symmetry-breaking (quaternion “imaginary” or “triplet” terms) from property gradients in response calculations. Used for analyzing magnetic properties similarly to how it is done with non-relativistic methods.

.NOSFMU

.NOSFMU

In spin-free correlated calculations group multiplication tables are by default set up as direct products of spatial and spin symmetries. This flag turns off this, and so the spin-free case is treated similar to the spin-orbit case.

.SPINF2

.SPINF2

[!WARNING] documentation missing

Other projection operators

.ASHIFT

.ASHIFT

Define atomic shift operator

$$V = \sum_A \sum_{I \in A} \sum_{i \in I} |\varphi^A_i\rangle \langle \varphi^A_i| \epsilon^A_I$$

where $\sum_A$ is a sum over atomic types. For each atomic types a group Γ^A of orbitals may be specified with an associated shift parameter ϵ^A_I . Example:

```
.ASHIFT
AFCXXX
1
-9.328
1,2
```


AF0XXX
0

Following the keyword there is a loop over the atomic types of the molecule under consideration, in this case carbon monoxide. For each atomic type one must specify the name of the coefficient file for the atom and the number of groups Γ . For each group one then specifies the shift parameter ϵ^A_Γ and then `orbital_strings` for each group. In the example above we specify one group for carbon. It involves a shift parameter -9.328 and the first and second atomic orbitals, that is, $C1s$ and $C2s$. For oxygen there are no groups.

.FREEPJ

.FREEPJ

Project out all negative-energy solutions of the free-particle one-electron Dirac Hamiltonian from the MO space. This corresponds to the Feynmann (1948) basis for QED and no-pair Hamiltonians.

.VEXTPJ

.VEXTPJ

Project out all negative-energy solutions of the bare-nucleus one-electron Dirac Hamiltonian from the MO space. This corresponds to the Furry (1951) basis for QED and no-pair Hamiltonians.

Exact 2-component (X2C) Hamiltonians

.X2C

.X2C

This keyword activates the Exact 2-Component one-electron Hamiltonian Ilias2007 based on its implementation in the module X2Cmod, Refs. Knecht2010 and Knecht2014. It yields exactly the same energies as the 4-component one-electron Hamiltonian in the same large component basis set with restricted kinetic balance, and it yields the same division in spinfree (scalar relativistic) and spin-orbit like contributions as the 4-component one-electron Hamiltonian. The two-electron part of the Hamiltonian is treated on a non-relativistic level.

One should therefore combine X2C with a correction to the unscreened one-electron spin-orbit operator. It is default with .X2C to include the atomic mean field two-electron spin-orbit correction “AMFI”, unless HAMILTONIAN_.NOAMFI is specified. To use the spinfree version of X2C with only scalar relativistic terms one needs to add the keyword HAMILTONIAN_.SPINFREE (.SPINFREE implies .NOAMFI). The spinfree X2C is equivalent to and numerically yields the same numbers as X2C in the quantum chemistry packages CFour, Turbomole, and Molcas.

An overview of the (local) X2C approach is given in the corresponding tutorial section X2Clocal.

NOTE: From DIRAC2023 onwards, we highly recommend to use the **(e)amf** module described in *XAMFI in replacement of the *AMFI module as highlighted by the numerical examples provided in Knecht2022.

.X2Cmmf

.X2Cmmf

This keyword activates the 2-component molecular-mean-field (X2C) Hamiltonian approach Sikkema2009 within the module X2Cmod, Ref. Knecht2014.

DIRAC starts with a 4c-SCF run and performs a transformation to 2-component mode (based on the converged Fock operator) prior to a post-HF correlation step. One can combine this option with HAMILTONIAN_.GAUNT which activates the inclusion of spin-other-orbit contributions in the Hamiltonian. The X2Cmmf-Hamiltonian can at present only be used for post-HF calculation within the RELCCSD module. Patches for other correlation modules in DIRAC will be part of the Dirac2014 release. See also the Molecular mean-field X2C tutorial, section mmf_X2C, for further information.

.BSS

.BSS

Use the 2-component relativistic Hamiltonian obtained after the Barysz–Sadlej–Snijders transformation of the Dirac Hamiltonian in the finite basis set, see Ref. Ilias2005. Calculations using the 2-component BSS Hamiltonian are running only with large component basis functions.

Approximate 2-component Hamiltonians

Please note that these are only tested for energies and generally do not work for properties !

.ZORA

.ZORA

Use the zeroth-order regular approximation (vanLenthe1994, vanLenthe1996, Visscher2000) of the Dirac Hamiltonian in the Hartree-Fock procedure. Works only for closed-shell systems. The implementation offers only little computational advantages and is intended chiefly for comparisons of approximate Hamiltonians methodologies. Note that the combination HAMILTONIAN_.SPINFREE and HAMILTONIAN_.ZORA gives a spin-free formalism that differs from the conventional spin-free ZORA formulation. Two integers should be specified in free format on the line following HAMILTONIAN_.ZORA:

.ZORA
1 1

The first number indicates whether the density is to be normalized over the 2-component (0; ZORA) or 4-component metric (1; ZORA4).

The second number specifies whether the orbital energies should be unmodified (0; normal ZORA) or scaled (1; scaled ZORA).

.DKH1

.DKH1

First-order Douglas-Kroll Hamiltonian

.DKH2

.DKH2

Second-order Douglas-Kroll Hamiltonian

Non-relativistic Hamiltonians

.LEVY-LEBLOND

.LEVY-LEBLOND

Use the nonrelativistic Levy-Leblond Hamiltonian Levy1967.

Use this option before any additional one-electron operators are specified, because it redefines the metric used in the calculation.

.NONREL

.NONREL

Standard nonrelativistic calculation based on the Schrodinger equation. Should give identical energy results as with the HAMILTONIAN_.LEVY-LEBLOND keyword, if same nucleus model is chosen. Point nucleus model is default for NONREL.

DIRAC runs in the 2-component spin-free mode, which in fact represents the traditional one-component mode (Pauli Hamiltonian).

Effective core potentials

.ECP

.ECP

Perform a relativistic effective core potential calculation. The ECP parameters should be set in the MOL file. Both spin-orbit and spin-free calculations are available by specifying spin-orbit (SO) parameters in the MOL file. With SO parameters, the 2-component spin-orbit calculation is conducted, whereas the spin-free (1-component) calculation is performed by omitting the SO part in the ECP parameter. Point nucleus model is default for ECP. (See `ecp_input`)

External fields/Environment

.OPERATOR

.OPERATOR

Specification of an additional one-electron operator in the Hamiltonian. The operator must be totally symmetric both under the molecular point group and time reversal symmetry. The field strength of the operator is specified with COMFACTOR. The keyword can be repeated for addition of more than one operator.

See the `one_electron_operators` section for more information and explicit examples.

.PCM

.PCM

Model solvent effects by placing the molecule in a cavity in a dielectric continuum. The cavity is shaped on the actual geometry of the solute, the full molecular electrostatic potential is used. DIRAC makes use of the external [PCMSolver](#) module.

.SOLVENT

.SOLVENT

Model solvent effects by placing the molecule in a spherical cavity in a dielectric continuum. The solute electrostatic potential is represented in terms of a truncated multipolar expansion.

.FDE

.FDE

Activates the frozen density embedding (FDE) functionality. Options can be specified under the `\*FDE` menu.

In order to use FDE the user must have generated an embedding potential and/or frozen densities for the environment, either directly with the ADF code (see the developer's website <http://www.scm.com> for further information) or via the PyADF scripting framework (see Jacob2011 or visit the developer's website <http://pyadf.org> for further information).

.PELIB

.PELIB

Activates polarizable embedding using PELib, website <https://gitlab.com/pe-software/pelib>. Which model and other options must be specified under the *PELIB menu.

Three embedding models are defined (all of which can be included in a single calculation)

- polarizable density embedding (PDE) - polarizable embedding (PE) - continuum solvation (FixSol)

.CAP

.CAP

Complex Absorption Potential (only in the development version).

Kohn-Sham Hamiltonian

.DFT

.DFT

Perform a Kohn–Sham density functional theory calculation. In the following line you must specify the desired DFT functional.

The functional can either be selected from a set of predefined combinations of exchange and correlation functionals <dftcfun>, e.g.:

```
.DFT
B3LYP
```

Alternatively, it can be composed by specifying GGAKEY followed by a list of the desired functionals together with their weights:

```
.DFT
GGAKEY PW86X=1.0 P86C=1.0
```

GGAKEY also allows the definition of (global) hybrid functionals, for instance B3LYP can be specified as:

```
.DFT
GGAKEY Slater=0.8 Becke=0.72 HF=0.2 VWN=0.19 LYP=0.81
```

where HF indicates weight of Hartree-Fock exchange 20%. It is also possible to specify long-range corrected or Coulomb-attenuated functionals using the CAM keyword, e.g. CAMB3LYP is predefined but can also be specified as:

```
.DFT
CAM p:alpha=0.19 p:beta=0.46 p:mu=0.33 x:slater=1 x:becke=1 c:lyp=0.81 c:vwn5=0.19
```

Coulomb-attenuated functionals are based on a separation of the two-electron interaction into a long- and short-range part

$$\frac{1}{r_{12}} = \frac{\left[\alpha + \beta \operatorname{erf}(\mu r_{12}) \right]}{r_{12}} - \frac{1 - \left[\alpha + \beta \operatorname{erf}(\mu r_{12}) \right]}{r_{12}}$$

For CAM or long-range corrected functionals this separation is only invoked in the evaluation of exchange. The above CAM input first reads the three parameters(p) α (alpha), β (beta) and μ (mu), followed by exchange (x) and correlation (c) functionals with their respective weights. Standard exchange functionals are automatically short-range corrected following the approach of Iikura2001. The activation of this separation for correlation as well leads to long-range WFT/short-range DFT methods <../tutorials/srDFT/general>, as represented by MP2-srDFT in DIRAC.

.DFTAUTO

.DFTAUTO

Perform a Kohn–Sham calculation using functionals provided by the XCFun library. In the following line you must specify the desired DFT functional.

.HFXFAC

.HFXFAC

Weight of exchange in Fock matrix construction. Default:

```
.HFXFAC
1.0
```

.HFXATT

.HFXATT

[!WARNING] documentation missing

.HFXMU

.HFXMU

[!WARNING] documentation missing

Advanced modification of the 4-component Hamiltonian

.NOSMLV

.NOSMLV

Delete SS nuclear attraction integrals. This will take out contributions to the one-electron spin-orbit interaction and the Darwin interaction.

.SMLV1C

.SMLV1C

Neglect potential for multi-center SS blocks, i.e. multi-center SS nuclear attraction integrals and multi-center SS two-electron integrals.

.JZOUT

.JZOUT

Print out Jz MO matrices for the diagonalization into own formatted files (suitable for testing). Only for the linear symmetry. Programmer's option.

.ONECAP

.ONECAP

Consider taking the HAMILTONIAN_, SMLV1C model one step further. Only one-center contributions to the LS and SS two-electron integrals and SS nuclear attraction integrals are calculated explicitly. The electrostatic effects of the terms neglected this way are included by calculating the classical repulsion from small component charges based on a Mulliken population analysis. Note that we therefore only need to calculate the derivative of the LL integrals when calculating the molecular gradient.

.ONECNV

.ONECNV

Employ the one-center model as given by HAMILTONIAN_.ONECAP until convergence to a specified THRESHOLD, whereafter the full set of two-electron integrals will be used:

.ONECNV
THRESHOLD

This threshold applies to whatever convergence criteria has been selected (SCF_.EVCCNV, SCF_.ERGCNV or SCF_.FCKCNV).

Advanced BSS keywords/Experimental

.X2COLD

.X2COLD

One-step Exact (infinite order) 2-Component relativistic Hamiltonian Ilias2007. This keyword was called .X2C prior to DIRAC10. DIRAC runs in the (memory saving) 2-component mode. Note that one should combine this option with the spin-free option as X2C will only provide an unscreened (bare nucleus) spin-orbit operator that gives unphysically large spin-orbit contributions. To get a realistic screened spin-orbit operator AMFI is added in the development version unless HAMILTONIAN_.NOAMFI specified.

.X2C4

.X2C4

One-step Exact (infinite order) 2-Component relativistic Hamiltonian Ilias2007 .

DIRAC runs in the 4-component mode. This mode is useful if you wish to restart from a previous 4-component calculation and vice versa. AMFI is added in the development version unless HAMILTONIAN_.NOAMFI specified.

.IOTC4

.IOTC4

[!WARNING] documentation missing

.BSS4

.BSS4

[!WARNING] documentation missing

.CMPEIG

.CMPEIG

When some two-component relativistic Hamiltonian is chosen, compare eigenvalues between the 'parent' four-component Dirac and derived two-component one-electron Hamiltonians.

For the infinite order (one- and two-step) two-component Hamiltonians eigenvalues are identical with four-component Dirac counterparts. For the second-order (and lower order) Douglas-Kroll-Hess Hamiltonian they slightly differ.

.BEG_2C

.BEG_2C

[!WARNING] documentation missing

.ONESTEP

.ONESTEP

Together with the HAMILTONIAN_.BSS keyword - for the infinite order only - invokes the one-step infinite order method (which is otherwise called by HAMILTONIAN_.X2C, HAMILTONIAN_.X2C4 keywords).

.NOAMFI

.NOAMFI

Do not include the AMFI contribution where AMFI is the default. This holds also for keywords HAMILTONIAN_.X2C and HAMILTONIAN_.X2C4). In the DIRAC08 distribution version .NOAMFI was the default.

[!WARNING] missing AMFI contribution (for HAMILTONIAN_.X2C, HAMILTONIAN_.X2C4 and HAMILTONIAN_.BSS keywords) may lead to overestimation of spin-orbit effects since these would be represented by one-electron terms only, without the two-electron shielding.

.DO2C4C

.DO2C4C

After iterations at the two-component level ascend to the four-component level.

[!WARNING] only in development version

.DO4C2C

.DO4C2C

After iterations at the four-component level do the relativistic transformation to the two-component level.

[!WARNING] only in development version

.USE_DF

.USE_DF

[!WARNING] only in development version

After the four-component DC-SCF method do the infinite order transformation (either one- or two-step) upon the Fock-Dirac matrix. Otherwise it is transforming Dirac bare nucleus.

Used only with keyword HAMILTONIAN_.DO4C2C.

.CONT2C

.CONT2C

[!WARNING] only in development version

After four-component DC-SCF continue with two-component iterations.

Used only with keyword HAMILTONIAN_.DO4C2C.

Integer (3,4,5) should be specified in free format on the line following HAMILTONIAN_.CONT2C:

```
.CONT2C
3
```

[!WARNING] and what does this integer mean?

.MO4C2C

.MO4C2C

[!WARNING] documentation missing

Various

.SCQSET

.SCQSET

[!WARNING] documentation missing

.YREQ1

.YREQ1

[!WARNING] documentation missing

.BLOCKD**.BLOCKD**

[!WARNING] documentation missing

.QDOTS**.QDOTS**

[!WARNING] documentation missing

FILE: www.diracprogram.org/doc/release-26/_sources/manual/index.md

orphan

Keyword reference

- ****DIRAC** <dirac> — Job specification
 - ***OPTIMIZE** <optimize> — Geometry optimization directives
- ****GENERAL** <general> — General input
 - ***PARALLEL** <parallel> — Parallel run directives
- ****MOLECULE**<molecule> — Specification of basis set and other atom-specific information
 - ***CHARGE** <molecule> — Define the molecular charge
 - ***BASIS** <molecule> — Define default and atom-specific basis sets
 - ***CENTERS** <molecule> — Define specific properties for atoms (e.g. ghost centers, point charges)
 - ***SYMMETRY** <molecule> — Set symmetry manually (when reading from xyz file)
 - ***COORDINATES** <molecule> — Set unit for the input coordinates
- ****HAMILTONIAN** <hamiltonian> — Specify the Hamiltonian
 - ***DFT** <dft> — DFT directives
 - ***EFFQED** <effqed> — effective QED potential directives
 - ***XAMFI** <xamfi> — X-AMFI directives
 - ***AMFI** <amfi> — AMFI directives
 - ***X2C** <x2c> — X2C directives
 - ***FDE** <fde> — Frozen Density Embedding directives
 - ***PCM** <pcm> — Polarizable Continuum Model directives
 - ***PCMSOL** <pcmsolver> — PCMSolver module directives
 - ***PELIB** <pelib> — Polarizable (density) embedding model directives
- ****WAVE FUNCTION** <wave_function> — Method specification
 - ***SCF** <wave_function/scf> — SCF module (Hartree-Fock/Kohn-Sham)
 - ***MP2CAL** <wave_function/mp2cal> — second-order Møller-Plesset perturbation theory
 - ***RESOLVE** <wave_function/resolve> — resolve open-shell states
 - ***GOSCI** <wave_function/goscp> — Complete Open-Shell module
 - ***COSCI** <wave_function/cosci> — Complete Open-Shell CI module
 - ***DIRRCI** <wave_function/dirrci> — Direct Relativistic CI module
 - ***LUCITA** <wave_function/lucita> — Spinfree large-scale CI module
 - ***KR** <wave_function/krci> — Kramers-restricted relativistic large-scale CI module
 - ***KRMCSF** <wave_function/krmcsf> — Kramers-restricted relativistic large-scale MCSCF module
 - ****RELCCSD** <wave_function/relccsd> — Coupled cluster module
 - ****RELADC** <wave_function/reladc> — Propagator module (ADC) for single and double ionizations
 - ***POLPRP** <wave_function/polprp> — Polarization Propagator module (ADC) for excitations
 - ***MVOCAL** <wave_function/mvocal> — Modified virtual orbitals
 - ***MP2 NO** <wave_function/mp2no> — MP2 natural orbitals module
 - ***LAPLCE** <wave_function/laplce> — Laplace transformation of orbital-energy denominators
 - ****EXACC** <wave_function/exacorr> — Parallel implementation of coupled cluster methods based on ExaTensor library.
 - ***CASPT2** <wave_function/caspt2> — Relativistic IVO/CASCI/CASPT2 and IVO/RASCI/RASPT2 module
- ****ANALYZE** <analyze> — Analyze the wave function
 - ***MULPOP** <analyze/mulpop> — Mulliken population analysis
 - ***PRIVEC** <analyze/privec> — Print coefficients
 - ***PROJECTION** <analyze/prjana> — Projection analysis
 - ***LOCALIZATION** <analyze/localization> — Localization
 - ***DENSITY** <analyze/density> — Density
 - ***RH01** <analyze/density> — Rho1

- ****PROPERTIES** <properties> — Property module
 - ***EXPECTATION VALUES** <properties/expectation> — Expectation values
 - ***EXCITATION ENERGIES** <properties/excitation> — Excitation energies
 - ***LINEAR RESPONSE** <properties/linear_response> — Linear response
 - ***QUADRATIC RESPONSE** <properties/quadratic_response> — Quadratic response
 - ***MOLGRD** <properties/molgrd> — Molecular gradient
 - ***NMR** <properties/nmr> — NMR directives
 - ***STEX** <properties/stex> — Static exchange (STEX) directives
 - ***NQCC** <properties/nqcc> — NQCC directives
 - ***ESR** <properties/esr> — ESR/EPR directives
- ****VISUAL** <visual> — Visualization module
- ****INTEGRALS** <integrals> — Integral directives
- ****GRID** <grid> — Numerical integration grid
- ****MOLTRA** <moltra> — Integral transformation module

Notes:

Data handling <data> | One-electron operators <one_electron_operators> | Orbital strings <orbital_strings> | Symmetry-handling at the correlated level <groupchain>

FILE: www.diracprogram.org/doc/release-26/_sources/manual/integrals.md

orphan

****INTEGRALS**

****INTEGRALS**

General directives

.NUCMOD

.NUCMOD

Specify nuclear model.

Point nucleus:

```
.NUCMOD
1
```

Gaussian charge distribution (default):

```
.NUCMOD
2
```

The point nucleus model is useful to compare Lévy-Leblond type calculations with regular nonrelativistic calculations done with another code, e.g. [Dalton](#). The two methods should give precisely the same energies.

For the Gaussian charge distribution the default exponents are in accordance with values proposed by Visscher and Dyall Visscher1997b.

.SELECT

.SELECT

Restrict range of nuclei in one-electron integrals involving single atomic centers, for example electric field gradients.

Example: Restrict range to four nuclei (first number), the nuclei 1, 3, 7, and 8:

```
.SELECT
4
1
3
7
8
```

.MAGCOR

.MAGCOR

Print the symmetrized nuclear magnetic moments. This corresponds to taking symmetry combinations of rotations, not coordinates, at each nuclear center. The numbering is used in labels of various magnetic integrals.

.PRINT

.PRINT

General print level in the integral module. Default:

.PRINT
1

*ONEINT

ONEINT*One-electron integrals**

This subsection gives directives for the generation of one-electron integrals. Based on input in the other section, the program will determine what integrals to calculate.

.PRINT

.PRINT

Print level in one-electron integral routines. By default print level is taken from **INTEGRALS.

*READIN

READIN*The mol file**

This subsection allows changes of defaults in the reading of the mol file.

.UNCONTRACT

.UNCONTRACT

Decontract basis sets specified as contracted (when being read from the library, for example). In the case of using non-relativistically contracted basis set the decontraction is necessary for heavy elements. Two-component quasirelativistic Hamiltonians (like X2C) work only with decontracted basis set. By default only the small component is decontracted.

.PRINT

.PRINT

Print level in the reading of the mol file. By default print level is taken from **INTEGRALS.

.MAXPRI

.MAXPRI

Maximum number of primitive functions in a given block in basis file. Default:

.MAXPRI
35

*TWOINT

TWOINT*Two-electron integrals**

This subsection gives directives for the generation of two-electron integrals. It also gives directives for the construction of Fock matrices, such as screening.

.SCREEN

.SCREEN

Screening threshold for integral direct calculations of Fock matrices Saue1997. Default:

.SCREEN
1.0D-12

Note that the screening threshold may influence the convergence. In general, the screening threshold must be about three orders of magnitude smaller than the desired norm of the electronic gradient at convergence.

Choosing a negative value for the screening turns the integral screening completely off.

.ICEDIF

ICEDIF

Separate screening of Coulomb and exchange contributions Saue1997. Useful for fine regulation of the convergence process. Default: Coulomb and exchange on (1 = on; off = 0):

```
.ICEDIF
1 1
```

```
.THRAC
```

.THRAC

Adjust the integral thresholds for SL and SS integrals. For conventional integral calculations only integrals above the threshold given in the mol file are written to disk. The thresholds for the SL and SS integrals are divided by the factors given here. Default:

```
.THRAC
1.0 1.0
```

```
.AOFCK
```

.AOFCK

Do direct Fock matrix construction in non-symmetry-adapted basis (AO basis).

The direct Fock matrix construction is performed in AO basis using the skeleton matrix approach. This may give better screening, and does give more tasks for better parallelization, but AOFCK is more memory intensive than SOFCK. Default is AOFCK if 25 or more MPI nodes.

```
.SOFCK
```

.SOFCK

Do direct Fock matrix construction in symmetry-adapted basis (SO basis). This is the default setting.

AOFCK may give better screening, and does give more tasks for better parallelization, but AOFCK is more memory intensive than SOFCK. Default is SOFCK if at most 24 MPI nodes.

```
.PRINT
```

.PRINT

Set the print level in two-electron integral routines for the calculation of a particular shell quadruplet. The print level is changed only for the given shell quadruplet. A zero matches all shells, thus:

```
.PRINT
4 0 0 0 0
```

or just:

```
.PRINT
4
```

sets the print level to 4 for all shell quadruplets.

Use with care to avoid massive output! At print level 15 the individual integrals are printed.

```
.TIME
```

.TIME

Give detailed timing for integral calculation.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/molecule.md

orphan

****MOLECULE**

****MOLECULE****Specification of molecular symmetry and basis sets**

This section is needed for calculations using the simple .xyz input format for the molecular structure. For calculations with the .mol input format, basis set information is to be specified together with the atomic coordinates.

***CHARGE**

***CHARGE**

Molecular charge

Allows explicit specification of the molecular charge.

.CHARGE

.CHARGE

Sets the value of the molecular charge. For example:

```
.CHARGE
-2
```

will set the molecular charge to -2. The default value is 0.

*SYMMETRY

*SYMMETRY

Molecular symmetry

Allows explicit specification of the molecular symmetry that is to be used in the calculations. This will then overrule the automatically detected point group.

.NOSYM

.NOSYM

Switches off all use of symmetry (this is currently the only option).

.LINEAR

.LINEAR

Forces to use the linear symmetry even for the one-atom case. When you want to remove the inversion symmetry (e.g., an atom under an external electric field), a ghost center must be included.

*BASIS

*BASIS

Basis set

This section should contain the keywords needed to specify the basis set. The minimum specification is a default basis set for all atoms in the system.

.DEFAULT

.DEFAULT

The name of the basis set file that is to be searched for basis sets.

Example: use the double zeta basis sets optimized by Dyall:

```
.DEFAULT
dya11.cv2z
```

.SPECIAL

.SPECIAL

Special basis sets for specific atom types.

The simplest is to specify an alternative basis set library file. One then lists the atomtype, the keyword BASIS, and the name of the alternate file.

Example: use Dunning's cc-pVDZ basis for the fluorine atom:

```
.SPECIAL
F BASIS cc-pVDZ
```

Basis sets may also be explicitly typed using the EXPLICIT keyword and the format that is also used in the .mol file.

Example: specification of a basis set for the atomtype HB:

```
.SPECIAL
HB EXPLICIT
  2      1      1
  4      0      3
13.0100000
 1.9620000
 0.4446000
```

```

0.1220000
1 0 3
0.7270000

```

Note that you may use different basis sets for the same element, in the example above you may have most hydrogen atoms labeled as H and have a few special ones labeled as HB.

We can also introduce point charges that have no basis set attached by specifying the keyword NOBASIS after the atomtype.

Example: a point charge at the position of an oxygen:

```

.SPECIAL
O.PC NOBASIS

```

The keyword .SPECIAL can be repeated as often as you like, atomtypes not listed as special will receive the default basis set.

Example: EuF₃ with dyall set on Eu and the cc-pVDZ on the fluorides:

```

**MOLECULE
*BASIS
.DEFAULT
dyall.cv2z
.SPECIAL
F BASIS cc-pVDZ

*CENTERS

```

*CENTERS

Centers

This optional section allows modification of the properties of a center. This makes it possible to specify fractional charges (for embedding point charges) or zero charge (for ghost centers used in counterpoise calculations).

```
.NUCLEUS
```

```
.NUCLEUS
```

The atom type followed by the modified nuclear charge.

Example: a fractional charge of -0.6 for an oxygen point charge:

```
.NUCLEUS
O.PC -0.6

```

In case the center with the modified charge should contain a basis set there is a second keyword to specify the real nuclear charge that is to be used when searching the basis set file for the element-specific basis set.

Example: an oxygen ghost center (zero charge):

```
.NUCLEUS
O.Gh 0.0 8.0

```

Like the .SPECIAL keyword described above also the .NUCLEUS keyword can be specified as often as necessary. Note that all atomtypes should have exactly the same name as they appear in the xyz file, it is thereby good practice to use a logical naming scheme like the one used above (with a suffix Gh to indicate ghost atoms and a suffix pc to indicate point charges) but this is not enforced by the program.

```
*COORDINATES
```

*COORDINATES

Coordinates

```
.UNITS
```

```
.UNITS
```

With xyz type input coordinates are expected to be defined in terms of Angstrom units. If you want to need to deviate from this standard you can alter the unit type by specifying the keyword UNITS.

Example: specification of coordinates in atomic units:

```
.UNITS
AU

```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/moltra.md

orphan

```
**MOLTRA
```

****MOLTRA**

Integral transformation module.

This module transforms integrals from the scalar AO basis to integrals in the molecular spinor basis. The algorithm is not yet described in detail but it may be helpful to read Ref. Visscher2002 to find information about the double quaternion formalism that is used. This module is usually invoked by one of the correlation modules but can also be run as a stand-alone module by specifying `DIRAC_.4INDEX` in the `**DIRAC` section of input. The input serves to change the default active space and to specify special wishes (e.g modification of `MOLTRA_.INTFLG`).

Three separate transformations are considered:

- Transformation of a core Fock matrix, giving effective one-electron matrices.
- Transformation of two-electron Coulomb integrals.
- Transformation of integrals over general (one-electron) operators.

The output is written to the files `MDCINT`, `MRCONEE`, and `MDPROP`, respectively, and can be read in by the `**RELCC` and `DIRRCI` and programs to perform correlated calculations and by the `RELADC` and `**POLPRP` modules to perform propagator calculations. Some common input needs to be specified both in this section and in the `DIRRCI` sections.

`.ACTIVE`

.ACTIVE

Specify the active set of spinors.

Arguments: One line with an `orbital_strings` for each fermion irrep.

Default:

energy `-10.0 20.0 1.0`

Advanced options

`.THROUT`

.THROUT

Threshold value for writing transformed 2-electron integrals to file. A higher value will decrease the size of the `MDCINT` files but will decrease the accuracy of the calculation.

Default:

`.THROUT`
`1.0D-14`

`.PRPTRA`

.PRPTRA

Perform transformation of property integrals. The type(s) of integrals that are to be transformed can be specified with the `.OPERATOR` keyword.

Default: Do not transform property integrals.

`.SCHEME`

.SCHEME

Selects the integral transformation algorithm (“scheme”). Several schemes have been implemented, but users are to consider only “scheme 4” and “scheme 6” for production calculations. Other schemes refer to outdated to special-purpose algorithms, and should be used with care. The most suitable scheme to use depends to a significant extent on the type of calculation in question (size of the active space etc), and on the computational resources used (compilers, amount of memory/scratch disk space, whether or not disks are shared to local to compute nodes etc).

Scheme 4 (Default):

`.SCHEME`
`4`

Scheme 4 is designed to reduce the communication between different processors at the expense of increasing I/O operations. This increase in I/O operations stems from the integrals being replicated for all MPI processes, so that doubling the number of MPI processes doubles the total disk usage but not the usage per MPI process.

Scheme 6:

`.SCHEME`
`6`

Scheme 6 is designed to decrease I/O operations at the expense of more communication between processes, something which is more often than not advantageous in today’s architectures. It should be noted, however, that the significant amount of communication between processes happens for the so-called “1HT” step (“1HT” standing for first half-transformation, since it is where the first two indexes of the electron repulsion integrals are transformed to MO basis and these intermediate, half-transformed entities are stored on disk), with no communication on the “2HT” step (which will yield the final integrals in MO basis). Due to the large amount of half-transformed entities, this scheme may demand large scratch spaces (e.g. a Dirac-Coulomb calculation on water with the aug-cc-pV5Z basis, where the full spinor space is active and

LL and SL integrals are transformed can reach nearly 1Tb of disk use) but with the advantage that the full size of the problem is exactly divided between processes - so the larger the number of MPI processes, the less disk space is used per process.

.HTSORT

.HTSORT

In connection to the scheme 6 the user can also choose whether or perform an intermediate sorting step prior to the second half-transformation step, via the keyword MOLTRA_.HTSORT. In this step the 1HT entities on disk are rearranged from their “natural” order (arising from the two-electron integral algorithm) to an order which will minimize the amount of I/O operations taking place at the 2HT step.

HT sort disabled (Default):

```
.SCHEME
6
.HTSORT
0
```

Sorting of HT intermediates is disabled by default. Without such sorting, scheme 6’s performance can be negatively affected on computer systems where reading data is relatively expensive (one example is for IBM p4/p5/p6 systems and the XLF compiler) even for relatively modest calculations Dirac-Coulomb calculations (where LL and SL integrals are considered). However, in two-component calculations (where only LL integrals are transformed and therefore there are much less 1HT entities on disk) of similar or larger size, in terms of active space, good performance can still be achieved in the same architecture.

HT sort (strategy #1) enabled:

```
.SCHEME
6
.HTSORT
1
```

This sort strategy minimizes the amount of read/write operations by taking up as much 1HT data in memory and sorting it before writing it to disk again. This provides efficiency for systems (such as the IBM systems mentioned above) where reading is expensive, at the expense of larger memory usage (rule of thumb: the extra memory required *per MPI process* will be about 1-2% of the *total disk* space taken up by the 1HT quantities - so if that is 1Tb, the extra memory used will be of of about 1-2Gb; see the calculation output for more details).

.INTFLG

.INTFLG

Specify which classes of integrals should contribute to the transformed two-electron integrals and the effective (core) Hamiltonian. Default is to follow the definitions given in **HAMILTONIAN. If you want different integral classes in the 2- and 4-index transformation, then use the MOLTRA_.INTFL2 and MOLTRA_.INTFL4 keywords below (default: HAMILTONIAN_.INTFLG from **HAMILTONIAN).

.INTFL2

.INTFL2

Specify what two-electron integrals to include in the 2-index transformation ,i.e., in the two-electron part of the Fock core matrix (default: HAMILTONIAN_.INTFLG from **HAMILTONIAN).

.INTFL4

.INTFL4

Specify what two-electron integrals to include in the 4-index transformation (default: HAMILTONIAN_.INTFLG from **HAMILTONIAN).

.CORE

.CORE

Specify the frozen core spinors,

For each fermion irrep, give an orbital_strings of active orbitals.

Default: All occupied orbitals that are not active.

.CORE2

.CORE2

Specify the frozen core spinors for open (average of configuration) shell,

In the first line give the number of electrons in the open shell. For each fermion irrep, give an orbital_strings of active orbitals. ATTENTION: A continous set of orbitals is assumed.

Default: No frozen open shell.

.POSITRONS

.POSITRONS

Allow negative energy (“positronic”) spinors in active set. If this keyword is not specified then only electronic spinors will be included and any positronic spinors specified with MOLTRA_.ACTIVE are ignored.

.NO2IND

.NO2IND

Skip the 2-index transformation of the effective Fock matrix.

.NO4IND

.NO4IND

Skip the 4-index transformation.

.SCREEN

.SCREEN

Screening threshold in 4-index transformation (a negative value disables screening).

Default:

.SCREEN
1.0D-14

.ASCII

.ASCII

Write integrals to the ASCII file MO_integrals.txt to facilitate interfaces with correlation codes. Warning: only meant for initial interfacing as this format is very inefficient.

Default: Do not write the ASCII file.

.RCORBS

.RCORBS

Recanonize orbitals before transforming.

Default: Do not recanonize orbitals before transforming.

Programmers options

.PRINT

.PRINT

Print level.

Default:

.PRINT
0

.2INDEX

.2INDEX

Ranges of active orbitals in 2-index transformation module specified for index 1 and 2 and fermion irrep 1 and 2.

Default: Ranges set by MOLTRA_.ACTIVE.

.4INDEX

.4INDEX

Ranges of active orbitals in 4-index transformation module specified for index 1 to 4 and fermion irrep 1 and 2.

Default: Ranges set by MOLTRA_.ACTIVE.

.MDCINT

.MDCINT

Write 4-index transformed integrals to MDCINT file (default).

.NOMDCI

.NOMDCI

Skip writing 4-index transformed integrals to MDCINT file. Instead, keep 4IND* files and write file 4INDINFO, which contains info about how to read the 4IND* files.

.SCATTER

.SCATTER

Scatter 4-index transformed integrals to all nodes (default).

.NOSCAT

.NOSCAT

Do not scatter 4-index transformed integrals to all nodes

.MOFILE

.MOFILE

Name of file with MO coefficients to be used for integral transformation. Valid options: “CHECKPOINT”, “KRMCSF”, “KRMOLD”, and “UNKNOWN”. The “UNKNOWN” option means use the first one found of “KRMCSF”, “KRMOLD”, “CHECKPOINT” in this order.

Default:

.MOFILE
UNKNOWN

.PAR4BAS

.PAR4BAS

Specify number of batches for each node in 4-index transformation. Only working for parallel distributions.

Default:

.PAR4BAS
-1

*PRPTRA

***PRPTRA**

Property integrals transformation

This section defines the property operators that should to be transformed. Such integrals are only used in an experimental section of **RELCC, and this subsection can thus usually be omitted.

Programmers options

.PRINT

.PRINT

Print level.

Default:

.PRINT
0

.OPERATOR

.OPERATOR

Specification of general one-electron operators.

See the `one_electron_operators` section for more information and explicit examples.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/one_electron_operators.md

orphan

One-electron operators

Syntax for specifying one-electron operators

In DIRAC, a general one-electron 4-component operator is generated from linear combinations of operators having the basic form

$$\hat{\mathbf{P}} = f \, \mathbf{M} \, \hat{\Omega},$$

where f is a scalar factor, $\hat{\Omega}$ is a scalar operator, and $\mathbf{M}$ is one of the following 4×4 time-reversal symmetric matrices:

$$\begin{aligned} & \mathbf{I}, \mathbf{\beta}, \mathbf{\gamma}^5, \mathbf{\beta} \mathbf{\gamma}^5, \\ & i \mathbf{\alpha}_x, i \mathbf{\alpha}_y, i \mathbf{\alpha}_z, i \mathbf{\beta} \mathbf{\alpha}_x, i \mathbf{\beta} \mathbf{\alpha}_y, \\ & i \mathbf{\beta} \mathbf{\alpha}_z, i \mathbf{\beta} \mathbf{\gamma}_x, i \mathbf{\beta} \mathbf{\gamma}_y, i \mathbf{\beta} \mathbf{\gamma}_z, i \mathbf{\beta} \mathbf{\Sigma}_x, i \mathbf{\beta} \mathbf{\Sigma}_y, \\ & i \mathbf{\beta} \mathbf{\Sigma}_z, i \mathbf{\beta} \mathbf{\alpha} \mathbf{\Sigma}_x, i \mathbf{\beta} \mathbf{\alpha} \mathbf{\Sigma}_y, i \mathbf{\beta} \mathbf{\alpha} \mathbf{\Sigma}_z, \end{aligned}$$

One thing to notice is that $\mathbf{I}$, $\mathbf{\beta}$, $\mathbf{\gamma}^5$, and $\mathbf{\beta} \mathbf{\gamma}^5$ are time-reversal symmetric operators, and they are implemented with such symmetry in DIRAC. On the other hand, $\mathbf{\alpha}_k$, $\mathbf{\Sigma}_k$, $\mathbf{\beta} \mathbf{\alpha}_k$, and $\mathbf{\beta} \mathbf{\Sigma}_k$ (with $k=x,y,z$) are time-reversal antisymmetric operators. This information is saved by the program, but these matrices (i.e., $\mathbf{\alpha}_k$, $\mathbf{\Sigma}_k$, $\mathbf{\beta} \mathbf{\alpha}_k$, and $\mathbf{\beta} \mathbf{\Sigma}_k$) are multiplied by the imaginary unit $i = \sqrt{-1}$ in order to make them time-reversal symmetric and hence fit into the quaternion symmetry scheme of DIRAC (see Saue1999 and Salek2005 for more information).

Another important aspect to note is that DIRAC does not use the 4×4 $\mathbf{M}$ matrices in the standard representation. These matrices are reordered following a criterion described in Section [Quaternion algebra and time-reversal symmetry](#). Further details about the implemented operators are provided in that Section, where the connection between the Dirac matrices and the quaternion algebra is given.

Operator specification

Operators are specified by the keyword HAMILTONIAN_.OPERATOR with the following arguments:

```
.OPERATOR
'user defined operator name'
operator type keyword
one-electron scalar operator label for each component
CMULT
FACTORS
factor for each component
COMFACTOR
common factor for all components
```

Note that the arguments following the keyword HAMILTONIAN_.OPERATOR must start with a blank.

The arguments are optional, except for the one-electron scalar operator label. If no operator type keyword is given, DIRAC assumes the operator type to be $\mathbf{I}$, the diagonal 4×4 matrix. Component factors under the keyword FACTORS, as well as the common factor under the keyword COMFACTOR, are all equal to one if not specified.

The keyword CMULT assures multiplication of the individual factors (or common factor) by the speed of light in vacuum, c . This option has the further advantage that CMULT follows any user-specified modification of the speed of light, as provided by GENERAL_.CVALUE.

Operator type keywords

There are 23 basic operator types implemented in DIRAC. They are listed in the following Table, containing their keywords, operator forms, associated number of factors (NF), and time-reversal symmetry (TRS) without imaginary phase:

Keyword	Operator form	NF	TRS (without i)
DIAGONAL	$f \mathbf{I} \hat{\Omega}$	1	Symmetric
BETA	$f \mathbf{\beta} \hat{\Omega}$	1	Symmetric
GAMMA5	$f \mathbf{\gamma}^5 \hat{\Omega}$	1	Symmetric
BETAGAMM	$f \mathbf{\beta} \mathbf{\gamma}^5 \hat{\Omega}$	1	Symmetric
XALPHA	$f i \mathbf{\alpha}_x \hat{\Omega}$	1	Antisymmetric
YALPHA	$f i \mathbf{\alpha}_y \hat{\Omega}$	1	Antisymmetric
ZALPHA	$f i \mathbf{\alpha}_z \hat{\Omega}$	1	Antisymmetric
XVECTOR	$f_1 i \mathbf{\alpha}_y \hat{\Omega}_z - f_2 i \mathbf{\alpha}_z \hat{\Omega}_y$	2	Antisymmetric
YVECTOR	$f_1 i \mathbf{\alpha}_z \hat{\Omega}_x - f_2 i \mathbf{\alpha}_x \hat{\Omega}_z$	2	Antisymmetric

Keyword	Operator form	NF	TRS (without i)
ZAVECTOR	$\hat{\Omega}_x - \hat{\Omega}_y - \hat{\Omega}_z$	2	Antisymmetric
ALPHADOT	$\hat{\Omega}_x + \hat{\Omega}_y + \hat{\Omega}_z$	3	Antisymmetric
XBETAALP	$\hat{\Omega}_x$	1	Antisymmetric
YBETAALP	$\hat{\Omega}_y$	1	Antisymmetric
ZBETAALP	$\hat{\Omega}_z$	1	Antisymmetric
XSIGMA	$\hat{\Omega}_x$	1	Antisymmetric
YSIGMA	$\hat{\Omega}_y$	1	Antisymmetric
ZSIGMA	$\hat{\Omega}_z$	1	Antisymmetric
SIGMADOT	$\hat{\Omega}_x + \hat{\Omega}_y + \hat{\Omega}_z$	1	Antisymmetric
XBETASIG	$\hat{\Omega}_x$	1	Antisymmetric
YBETASIG	$\hat{\Omega}_y$	1	Antisymmetric
ZBETASIG	$\hat{\Omega}_z$	1	Antisymmetric
BEALDOT	$\hat{\Omega}_x + \hat{\Omega}_y + \hat{\Omega}_z$	3	Antisymmetric
iBETAGAM	$\hat{\Omega}_x^5$	1	Antisymmetric

Note for advanced users: iBETAGAM corresponds to the time-reversal symmetric operator $\hat{\Omega}_x^5$, but has been assigned a time-reversal antisymmetry. This operator can be used, for example, for calculations of the molecular enhancement factors of the electron electric dipole moment (eEDM). For further details, see Section [Quaternion algebra and time-reversal symmetry](#).

One-electron scalar operator labels

Operator label	Integral description	Symmetry	Components	Operators
KINENER	NR kinetic energy	S	KINENER	$\hat{\Omega}_1 = -\frac{1}{2}\nabla^2$
MOLFIELD	Nuclear attraction	S	MOLFIELD	$\hat{\Omega}_1 = \sum_K Z_K \mathbf{r} - \mathbf{R}_K ^{-1}$
OVERLAP	Overlap	S	OVERLAP	$\hat{\Omega}_1 = 1$
FERMI C	Fermi contact	S	FC NAMab	$\hat{\Omega}_1 = \frac{4}{\pi} \delta(\mathbf{r}) \delta(\mathbf{r} - \mathbf{R}_K)$
PVC	Gaussian nuclear density normalized to 1, $\rho_K^G(\mathbf{r})$	S	PVCNAMab	$\hat{\Omega}_1 = \rho_K^G(\mathbf{r}) = (\zeta_K/\pi)^{3/2} \exp(-\zeta_K \mathbf{r} - \mathbf{R}_K ^2)$
GRAD FC	Gradient of the Dirac delta distribution	S	XGFNAMab	$\hat{\Omega}_1 = \frac{\partial}{\partial x} \delta(\mathbf{r} - \mathbf{R}_K)$
			YGFNAMab	$\hat{\Omega}_1 = \frac{\partial}{\partial y} \delta(\mathbf{r} - \mathbf{R}_K)$
			ZGFNAMab	$\hat{\Omega}_1 = \frac{\partial}{\partial z} \delta(\mathbf{r} - \mathbf{R}_K)$
DRHO	Gradient of the Gaussian nuclear density $\rho_K^G(\mathbf{r})$	S	DRHO lmn	$\hat{\Omega}_1 = \frac{\partial}{\partial x} \rho_K^G(\mathbf{r})$
			DRHO opq	$\hat{\Omega}_1 = \frac{\partial}{\partial y} \rho_K^G(\mathbf{r})$
			DRHO rst	$\hat{\Omega}_1 = \frac{\partial}{\partial z} \rho_K^G(\mathbf{r})$
ANGMOM	Orbital angular momentum around CM	A	XANGMOM	$\hat{\Omega}_1 = -(\mathbf{r} \times \nabla)_{CM,x}$
			YANGMOM	$\hat{\Omega}_1 = -(\mathbf{r} \times \nabla)_{CM,y}$

Operator label	Integral description	Symmetry	Components	Operators
DIPLN	Dipole length	S	ZANGMOM	$\hat{\Omega}_3 = -[\langle \mathbf{r} \mathbf{r} \rangle] \times \hat{\nabla}_z$
			XDIPLN	$\hat{Q}_1 = x$
			YDIPLN	$\hat{Q}_2 = y$
			ZDIPLN	$\hat{Q}_3 = z$
DIPVEL	Dipole velocity	A	XDIPVEL	$\hat{\Omega}_1 = \frac{\partial}{\partial x}$
			YDIPVEL	$\hat{\Omega}_2 = \frac{\partial}{\partial y}$
			ZDIPVEL	$\hat{\Omega}_3 = \frac{\partial}{\partial z}$
			XXQUADRU	$\hat{\Omega}_1 = \frac{1}{4} (x^2 - r^2)$
QUADRU	Quadrupole moment	S	XYQUADRU	$\hat{\Omega}_2 = \frac{1}{4} xy$
			XZQUADRU	$\hat{\Omega}_3 = \frac{1}{4} xz$
			YYQUADRU	$\hat{\Omega}_4 = \frac{1}{4} (y^2 - r^2)$
			YZQUADRU	$\hat{\Omega}_5 = \frac{1}{4} yz$
			ZZQUADRU	$\hat{\Omega}_6 = \frac{1}{4} (z^2 - r^2)$
			XXSECMOM	$\hat{Q}_1 = -x^2$
			XYSECMOM	$\hat{Q}_2 = -xy$
			XZSECMOM	$\hat{Q}_3 = -xz$
SECMOM	Second moment	S	YYSECMOM	$\hat{Q}_4 = -y^2$
			YZSECMOM	$\hat{Q}_5 = -yz$
			ZZSECMOM	$\hat{Q}_6 = -z^2$
			XXTHETA	$\hat{\Omega}_1 = -\frac{3}{2} (x^2 - \frac{1}{3} r^2)$
THETA	Traceless quadrupole	S	XYTHETA	$\hat{\Omega}_2 = -\frac{3}{2} xy$
			XZTHETA	$\hat{\Omega}_3 = -\frac{3}{2} xz$
			YYTHETA	$\hat{\Omega}_4 = -\frac{3}{2} (y^2 - \frac{1}{3} r^2)$
			YZTHETA	$\hat{\Omega}_5 = -\frac{3}{2} yz$
			ZZTHETA	$\hat{\Omega}_6 = -\frac{3}{2} (z^2 - \frac{1}{3} r^2)$
			XSEFGMG	
SEFGMG	Magnetic term of Flambaum-Ginges self-energy potential	A	YSEFGMG	
			ZSEFGMG	

where $\mathbf{R}_{CM}$ and $\mathbf{R}_K$ are the position vectors of the nuclear center of mass and the nucleus K with respect to the origin of coordinates, respectively. In addition, the operators x , y , and z are the Cartesian components of the electron position vector operator with respect to the origin of coordinates.

Note that, in the table above, g_e is the electronic g-factor (i.e., $g_e = 2$), 'NAM' are the first three letters of the name of atom K as given in the .mol file, 'ab' denotes the two-digit index of the symmetry-adapted nucleus K , and 'lmn', 'opq', and 'rst' denote three-digit indices identifying, respectively, the x , y , and z Cartesian components of the given operator associated with nucleus K in the symmetry-independent (non-symmetry-adapted) representation.

For the normalized Gaussian nuclear density distribution, the default exponents (ζ_K) are the values proposed by Visscher and Dyall in Visscher1997b.

The matrix representations of the operators shown in the previous Table are either symmetric (S) or antisymmetric (A). This information is provided in the third column of the Table.

Other one-electron operator labels are:

Keyword	Description
ANGLON	Angular momentum around the nuclei
CARMOM	Cartesian moments (symmetric) $(l+1)(l+2)/2$ components ($l = i + j + k$)
CM1	First order magnetic field derivatives of electric field
CM2	Second order magnetic field derivatives of electric field
DARWIN	Darwin-type integrals
DIASUS	Angular London orbital contribution to diamagnetic susceptibility
DSO	Diamagnetic spin-orbit integrals
DSUSCGO	Diamagnetic susceptibility with common gauge origin
DSUSLH	Angular London orbital contribution to diamagnetic susceptibility
DSUSNOL	Diamagnetic susceptibility without London contribution
ELFGRDC	Electric field gradient at the individual nuclei, cartesian
ELFGRDS	Electric field gradient at the individual nuclei, spherical
EXPIKR	Cosine and sine
HBDO	Half B-differentiated overlap matrix
HDO	Half-derivative overlap integrals
HDOBR	Ket-differentiation of HDO-integrals with respect to magnetic field
LONMOM	London orbital contribution to angular momentum
MAGMOM	One-electron contributions to magnetic moment
MASSVEL	Mass velocity
NEFIELD	Electric field at the individual nuclei
NSTCGO	Nuclear shielding with common gauge origin

Keyword	Description
NUCPOT	Potential energy of the interaction of electrons with individual nuclei, divided by the nuclear charge
NUCSHI	Nuclear shielding tensor
NUCSLO	London orbital contribution to nuclear shielding tensor
NUCSNLO	Nuclear shielding integrals without London orbital contribution
PSO	Paramagnetic spin-orbit
S1MAG	Second order contribution from overlap matrix to magnetic properties
S1MAGL	Bra-differentiation of overlap matrix with respect to magnetic field
S1MAGR	Ket-differentiation of overlap matrix with respect to magnetic field
SDFC	Spin-dipole + Fermi contact
SOLVENT	Electronic solvent integrals
SPHMOM	Spherical moments (real combinations), symmetric, ($2l+1$) components ($m = 0, -1, +1, \dots, -l, +l$)
SPIN-DI	Spin-dipole integrals
SPNORB	Spatial spin-orbit
SQHDO	Half-derivative overlap integrals not to be antisymmetrized
SQHDOR	Half-derivative overlap integrals not to be anti-symmetrized
SQOVLAP	Second order derivatives overlap integrals
UEHLING	Uehling potential
SEFGLF	Low-frequency term of Flambaum-Ginges self-energy potential
SEFGHF	High-frequency term of Flambaum-Ginges self-energy potential
SEPZ	Pyykko-Zhao self-energy model potential

Examples of using various operators

We give here several concrete examples on how to construct operators for various properties.

Kinetic part of the Dirac Hamiltonian

The kinetic part of the electronic Dirac Hamiltonian is given (in a.u.) by

$$\hat{H}^K = c \vec{\alpha} \cdot \hat{\vec{p}} = c \left(\alpha_x \hat{p}_x + \alpha_y \hat{p}_y + \alpha_z \hat{p}_z \right) = -c \left[i \alpha_x \frac{\partial}{\partial x} + i \alpha_y \frac{\partial}{\partial y} + i \alpha_z \frac{\partial}{\partial z} \right]$$

This operator may be specified by:

```
.OPERATOR
'Kin energy'
ALPHADOT
XDIPVEL
YDIPVEL
ZDIPVEL
COMFACTOR
-137.03599926085
```

where 137.03599926085 is the CODATA22 value for c .

The speed of light c is an important parameter in relativistic theory, but its explicit value in atomic units is not necessarily remembered. A simpler way to specify the kinetic energy operator is therefore:

```
.OPERATOR
'Kin energy'
ALPHADOT
XDIPVEL
YDIPVEL
ZDIPVEL
CMULT
COMFACTOR
-1
```

Magnetic interactions

Another example is the operator that describes the electromagnetic interaction between electrons and a uniform external magnetic field: $\hat{H}^B = -\frac{c}{2} \vec{B} \cdot \vec{\alpha} \times \hat{\vec{r}}$.

The operator to use in this case is $-\frac{c}{2} \vec{B} \cdot \vec{\alpha} \times \hat{\vec{r}}$, and in DIRAC each component is calculated individually. For example, the x -component of this operator is $-\frac{c}{2} \left(\alpha_y \hat{p}_z - \alpha_z \hat{p}_y \right)$, and can be requested using:

```
.OPERATOR
'B_x'
XAVECTOR
ZDIPLN
YDIPLN
CMULT
COMFACTOR
-0.5
```

The program will assume all operators to be Hermitian. The operator XAVECTOR is defined as $\hat{H}^B = -\frac{c}{2} \vec{B} \cdot \vec{\alpha} \times \hat{\vec{r}}$ (see [Operator type keywords](#)), and in this particular case the scalar operators are $\hat{H}^B_{zz} = -\frac{c}{2} B_z \alpha_z$ and $\hat{H}^B_{yy} = -\frac{c}{2} B_y \alpha_y$ (see ZDIPLN and

YDIPLN in [One-electron scalar operator labels](#)). Additionally, $f_1 = f_2 = \frac{c}{2}$. Then, DIRAC will assume the operator to be $\frac{c}{2} \left(\alpha_y z - \alpha_z y \right)$.

The imaginary phase i makes the operator time-reversal symmetric.

Diagonal operators

If no other argument is given, the program assumes the operator to be diagonal and expects the operator name to be the component label, for instance:

```
.OPERATOR
OVERLAP
```

Finite-field perturbation: Electric dipole moment

Another example is a finite-perturbation calculation where the perturbed Hamiltonian operator arising from the interaction between the electrons and a uniform external electric field oriented along the z -axis ($\hat{H} = -q E_z \hat{z} = e E_z \hat{z}$) is added to the unperturbed Hamiltonian $\hat{H}_0$ defined in **HAMILTONIAN** (default: Dirac-Coulomb).

If we take $e E_z = 0.01$ (in a.u.), the $\hat{z}$ dipole length operator will be used to build the total Hamiltonian $\hat{H} = \hat{H}_0 + 0.01 \hat{z}$ by adding the perturbed Hamiltonian $\hat{H}^{\text{pert}}$ to the Dirac-Coulomb Hamiltonian $\hat{H}_0$ using:

```
**HAMILTONIAN
.OPERATOR
ZDIPLN
COMFACTOR
0.01
```

Note: Don't forget to decrease the symmetry of your system.

Fermi-contact integrals

Here is an example where the Fermi-contact (FC) integrals for a certain nucleus K are added to the Hamiltonian in a finite-field calculation.

Let's assume we are looking at a PbX dimer (order in the .mol file: 1. Pb, 2. X) and we want to add (under **HAMILTONIAN**) the FC interaction for the Pb nucleus to the unperturbed Hamiltonian $\hat{H}_0$ as a perturbation with a given field strength of 10^{-9} a.u..

The FC integrals are called using the label 'FC NAB' (see [One-electron scalar operator labels](#)), and the total perturbed Hamiltonian $\hat{H} = \hat{H}_0 + \hat{H}^{\text{pert}}$ (where, in a.u., $\hat{H}^{\text{pert}} = 10^{-9} \frac{4\pi}{3} \rho_e \delta(\mathbf{r} - \mathbf{R}_{\text{Pb}})$) can be requested using:

```
**HAMILTONIAN
.OPERATOR
'Density at Pb nucleus'
DIAGONAL
'FC Pb 01'
FACTORS
0.000000001
```

Important note: The values obtained after the fit of the finite-field energies need to be scaled by $\frac{3}{4\pi \rho_e} = \frac{1}{8.3872954891254192}$ in order to get the raw density $\delta(\mathbf{r} - \mathbf{R}_{\text{Pb}})$.

The next example shows how to calculate the electronic density at the different nuclei in the molecule, as the expectation values $\langle \delta(\mathbf{r} - \mathbf{R}_K) \rangle$ (for each nucleus K in the system) for a Dirac-Coulomb Hartree-Fock wave function including a decomposition of the molecular orbital contributions to the density:

```
**DIRAC
.WAVE FUNCTION
.PROPERTIES
**HAMILTONIAN
**WAVE FUNCTION
.SCF
**PROPERTIES
.RHONUC
*EXPECTATION
.ORBANA
*END OF
```

For further details, see PROPERTIES_.RHONUC.

Cartesian moment expectation value

In the following example we calculate the cartesian multipole moment expectation value $\langle x^1 y^2 z^3 \rangle$ for a Levy-Leblond Hartree-Fock wave function:

```
**DIRAC
.WAVE FUNCTION
.PROPERTIES
**HAMILTONIAN
.LEVY-LEBLOND
**WAVE FUNCTION
.SCF
**PROPERTIES
*EXPECTATION
.OPERATOR
CM010203
*END OF
```

Quaternion algebra and time-reversal symmetry

The sixteen 4×4 matrices listed in Section [Syntax for specifying one-electron operators](#), i.e., $\boldsymbol{I}$, $\boldsymbol{\beta}$, $\boldsymbol{\gamma}^5$, $\boldsymbol{\beta}\boldsymbol{\gamma}^5$, $\boldsymbol{\alpha}_x$, $\boldsymbol{\alpha}_y$, $\boldsymbol{\alpha}_z$, $\boldsymbol{\beta}\boldsymbol{\alpha}_x$, $\boldsymbol{\beta}\boldsymbol{\alpha}_y$, $\boldsymbol{\beta}\boldsymbol{\alpha}_z$, $\boldsymbol{\Sigma}_x$, $\boldsymbol{\Sigma}_y$, $\boldsymbol{\Sigma}_z$, $\boldsymbol{\beta}\boldsymbol{\Sigma}_x$, $\boldsymbol{\beta}\boldsymbol{\Sigma}_y$, and $\boldsymbol{\beta}\boldsymbol{\Sigma}_z$, are usually expressed in Molecular Physics using the standard representation where, for example,

```
\begin{aligned}
\tilde{\boldsymbol{\beta}} =
\begin{pmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & -1 & 0 \\
0 & 0 & 0 & -1
\end{pmatrix}, \quad
\tilde{\boldsymbol{\gamma}^5} =
\begin{pmatrix}
0 & 0 & 1 & 0 \\
0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0
\end{pmatrix}, \quad
i\tilde{\boldsymbol{\alpha}}_z =
\begin{pmatrix}
0 & 0 & i & 0 \\
0 & 0 & 0 & -i \\
i & 0 & 0 & 0 \\
0 & -i & 0 & 0
\end{pmatrix}.
\end{aligned}
```

Here and in what follows, we use the notation $\tilde{\boldsymbol{U}}$ to refer to any of these sixteen matrices when they are expressed in the standard representation.

It can be seen that all these matrices can be written as the Kronecker product of two 2×2 matrices $\boldsymbol{A}$ and $\boldsymbol{B}$, such that $\tilde{\boldsymbol{U}} = \boldsymbol{A} \otimes \boldsymbol{B}$. In particular, $\boldsymbol{A}$ is one of the real matrices $\boldsymbol{I}$, $\boldsymbol{\sigma}_x$, $\boldsymbol{\sigma}_y$, or $\boldsymbol{\sigma}_z$, whereas $\boldsymbol{B}$ is $\boldsymbol{I}$, $i\boldsymbol{\sigma}_x$, $i\boldsymbol{\sigma}_y$, or $i\boldsymbol{\sigma}_z$. Here, $\boldsymbol{\sigma}_x$, $\boldsymbol{\sigma}_y$, and $\boldsymbol{\sigma}_z$ are the Pauli matrices, and i is the imaginary unit.

The individual matrix elements of $\tilde{\boldsymbol{U}}$ are given as

$$\tilde{U}_{pq} = A_{ij} \quad \text{,} \quad B_{kl} \quad \text{,} \quad ,$$

where $p=2(i-1)+k$ and $q=2(j-1)+l$.

In the DIRAC code, the structure of the Dirac equation with respect to time-reversal symmetry is displayed by using reordered Dirac 4-spinors, where the components are grouped on spin labels (α , β) rather than large (upper) and small (lower) components (L , S),

```
\begin{aligned}
\begin{pmatrix}
\psi^L_\alpha \\
\psi^L_\beta \\
\psi^S_\alpha \\
\psi^S_\beta
\end{pmatrix}
&\rightarrow
\begin{pmatrix}
\psi^L_\alpha \\
\psi^S_\alpha \\
\psi^L_\beta \\
\psi^S_\beta
\end{pmatrix}.
\end{aligned}
```

The same reordering applies to all the matrices $\tilde{\boldsymbol{U}}$, which are then transformed accordingly. In the original ordering (i.e., in the standard representation), these matrices are given as

```
\begin{aligned}
\tilde{\boldsymbol{U}} =
\begin{pmatrix}
\tilde{U}_{L\alpha,L\alpha} & \tilde{U}_{L\alpha,L\beta} & \tilde{U}_{L\alpha,S\alpha} & \tilde{U}_{L\alpha,S\beta} \\
\tilde{U}_{L\beta,L\alpha} & \tilde{U}_{L\beta,L\beta} & \tilde{U}_{L\beta,S\alpha} & \tilde{U}_{L\beta,S\beta} \\
\tilde{U}_{S\alpha,L\alpha} & \tilde{U}_{S\alpha,L\beta} & \tilde{U}_{S\alpha,S\alpha} & \tilde{U}_{S\alpha,S\beta} \\
\tilde{U}_{S\beta,L\alpha} & \tilde{U}_{S\beta,L\beta} & \tilde{U}_{S\beta,S\alpha} & \tilde{U}_{S\beta,S\beta}
\end{pmatrix}
= \boldsymbol{A} \otimes \boldsymbol{B}.
\end{aligned}
```

However, after reordering one clearly has the transformed matrices

```
\begin{aligned}
\boldsymbol{U} =
\begin{pmatrix}
\tilde{U}_{L\alpha,L\alpha} & \tilde{U}_{L\alpha,S\alpha} & \tilde{U}_{L\alpha,L\beta} & \tilde{U}_{L\alpha,S\beta} \\
\tilde{U}_{S\alpha,L\alpha} & \tilde{U}_{S\alpha,S\alpha} & \tilde{U}_{S\alpha,L\beta} & \tilde{U}_{S\alpha,S\beta} \\
\tilde{U}_{L\beta,L\alpha} & \tilde{U}_{L\beta,L\beta} & \tilde{U}_{L\beta,S\alpha} & \tilde{U}_{L\beta,S\beta} \\
\tilde{U}_{S\beta,L\alpha} & \tilde{U}_{S\beta,S\alpha} & \tilde{U}_{S\beta,L\beta} & \tilde{U}_{S\beta,S\beta}
\end{pmatrix},
\end{aligned}
```

which satisfy

$$\boldsymbol{U} = \boldsymbol{P} \quad \text{,} \quad \tilde{\boldsymbol{U}} \quad \text{,} \quad \boldsymbol{P}^{-1} = \boldsymbol{P} \quad \text{,} \quad (\boldsymbol{A} \otimes \boldsymbol{B}) \quad \text{,} \quad \boldsymbol{P}^{-1} = \boldsymbol{B} \otimes \boldsymbol{A},$$

where the 4×4 Kronecker permutation matrix $\boldsymbol{P}$ is a special permutation matrix that swaps the order of the Kronecker product of two 2×2 matrices, and is given by

```

\begin{aligned}
\boldsymbol{P} &= \\
\begin{pmatrix}
1 & 0 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & 0 & 1
\end{pmatrix}.
\end{aligned}

```

[Note that applying $\boldsymbol{P}$ on the left permutes the rows, while applying $\boldsymbol{P}^{-1}$ on the right permutes the columns, ensuring the correct transformation from $\boldsymbol{A} \otimes \boldsymbol{B}$ to $\boldsymbol{B} \otimes \boldsymbol{A}$.]

In other words, $U_{PQ} = B_{kl} A_{ij}$, with $P=2(k-1)+i$ and $Q=2(l-1)+j$. This reordering guarantees that all the matrices U are time-reversal symmetric.

Additionally, following the arguments given in Ref. Saue1999, it is possible to make the following mapping involving the quaternion units:

```

\boldsymbol{I} \quad \rightarrow \quad 1
\boldsymbol{\sigma}_z \rightarrow \quad \check{\imath}
\boldsymbol{\sigma}_y \rightarrow \quad \check{\jmath}
\boldsymbol{\sigma}_x \rightarrow \quad \check{k}

```

or, in a general way, the 2×2 matrices B can be mapped as $B \rightarrow b$, where b is a quaternion unit (i.e., 1 , $\check{\imath}$, $\check{\jmath}$, or $\check{k}$) arising from the relationships given above.

Thus, any linear combination of the 16 reordered matrices U can be mapped to a linear combination of 2×2 matrices u , each of which contains a quaternion unit.

To make this discussion clearer, in the following we discuss two examples. In first place, we transform the 4×4 matrix $\tilde{\boldsymbol{\beta}}$. We start by writing this matrix as the following Kronecker product of two 2×2 matrices: $\tilde{\boldsymbol{\beta}} = \boldsymbol{\sigma}_z \otimes \boldsymbol{I}$. In the reordered representation, this matrix will be expressed as $\boldsymbol{\beta} = \boldsymbol{I} \otimes \boldsymbol{\sigma}_z$. Finally, the corresponding B matrix (in this case, $\boldsymbol{I}$) is transformed to a quaternion unit (in the current example, 1). In this way, the 4×4 reordered matrix $\boldsymbol{\beta}$ can be written in quaternion form as the 2×2 matrix $\boldsymbol{\sigma}_z$. In other words:

```

\begin{aligned}
\tilde{\boldsymbol{\beta}} &= \\
\begin{pmatrix}
1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 \\
0 & 0 & -1 & 0 \\
0 & 0 & 0 & -1
\end{pmatrix} \\
&\rightarrow \\
\boldsymbol{\beta} &= \\
\boldsymbol{P} \tilde{\boldsymbol{\beta}} \boldsymbol{P}^{-1} &= \\
\begin{pmatrix}
1 & 0 & 0 & 0 \\
0 & -1 & 0 & 0 \\
0 & 0 & 1 & 0 \\
0 & 0 & 0 & -1
\end{pmatrix} \\
&\rightarrow \\
\boldsymbol{\beta}^{\text{quat}} &= \\
\boldsymbol{\sigma}_z.
\end{aligned}

```

As a second example, we take $\tilde{\boldsymbol{\alpha}}_z$. This 4×4 matrix, in its original ordering (according to the standard representation), can be written as the Kronecker product $\tilde{\boldsymbol{\alpha}}_z = \boldsymbol{\sigma}_x \otimes \boldsymbol{\sigma}_z$. This means that in the reordered representation it will be given as $\boldsymbol{\alpha}_z = \boldsymbol{\sigma}_z \otimes \boldsymbol{\sigma}_x$. In this particular case, the corresponding matrix B is $\boldsymbol{\sigma}_z$, so that the quaternion unit b is $\check{\imath}$. Therefore, the quaternion form of this reordered matrix will be $\boldsymbol{\alpha}_z^{\text{quat}} = \check{\imath} \boldsymbol{\sigma}_x$. Equivalently:

```

\begin{aligned}
\tilde{\boldsymbol{\alpha}}_z &= \\
\begin{pmatrix}
0 & 0 & i & 0 \\
0 & 0 & 0 & -i \\
i & 0 & 0 & 0 \\
0 & -i & 0 & 0
\end{pmatrix} \\
&\rightarrow \\
\boldsymbol{\alpha}_z &= \\
\boldsymbol{P} \tilde{\boldsymbol{\alpha}}_z \boldsymbol{P}^{-1} &= \\
\begin{pmatrix}
0 & i & 0 & 0 \\
i & 0 & 0 & 0 \\
0 & 0 & 0 & -i \\
0 & 0 & -i & 0
\end{pmatrix} \\
&\rightarrow \\
\boldsymbol{\alpha}_z^{\text{quat}} &= \\
\check{\imath} \boldsymbol{\sigma}_x.
\end{aligned}

```

The relationships between matrices $\tilde{U}$ (i.e., the sixteen 4×4 matrices in original ordering, which corresponds to the standard representation) and the quaternion form of their reordered equivalent 2×2 matrices u is given in the following Table. They follow the transformation rule $\tilde{U} = \boldsymbol{A} \otimes \boldsymbol{B} \rightarrow \boldsymbol{B} \otimes \boldsymbol{A}$, $\tilde{U} \rightarrow \boldsymbol{B} \otimes \boldsymbol{A}$, $\boldsymbol{P}^{-1} \tilde{U} \boldsymbol{P} = \boldsymbol{P} \otimes \boldsymbol{P}$, $\boldsymbol{A} \otimes \boldsymbol{B} \rightarrow \boldsymbol{B} \otimes \boldsymbol{A}$, $\boldsymbol{P}^{-1} \tilde{U} \boldsymbol{P} = \boldsymbol{B} \otimes \boldsymbol{A}$.

$\tilde{\mathbf{U}}$	$\mathbf{A} \otimes \mathbf{B}$	$\mathbf{u}$
$\tilde{\mathbf{I}}$	$\mathbf{I} \otimes \mathbf{I}$	$\mathbf{I}$
$\tilde{\Sigma}_z$	$\mathbf{I} \otimes \mathbf{i} \Sigma_z$	$\check{\imath} \Sigma_z$
$\tilde{\Sigma}_y$	$\mathbf{I} \otimes \mathbf{i} \Sigma_y$	$\check{\jmath} \Sigma_y$
$\tilde{\Sigma}_x$	$\mathbf{I} \otimes \mathbf{i} \Sigma_x$	$\check{k} \Sigma_x$
$\tilde{\beta}$	$\Sigma_z \otimes \mathbf{I}$	Σ_z
$\tilde{\beta} \tilde{\Sigma}_z$	$\Sigma_z \otimes \mathbf{i} \Sigma_z$	$\check{\imath} \Sigma_z$
$\tilde{\beta} \tilde{\Sigma}_y$	$\Sigma_z \otimes \mathbf{i} \Sigma_y$	$\check{\jmath} \Sigma_y$
$\tilde{\beta} \tilde{\Sigma}_x$	$\Sigma_z \otimes \mathbf{i} \Sigma_x$	$\check{k} \Sigma_x$
$\tilde{\beta} \tilde{\gamma}^5$	$(\mathbf{i} \Sigma_y) \otimes \mathbf{I}$	$(\check{\imath} \Sigma_y)$
$\tilde{\beta} \tilde{\alpha}_z$	$(\mathbf{i} \Sigma_y) \otimes \mathbf{i} \Sigma_z$	$\check{\imath} (\check{\imath} \Sigma_y)$
$\tilde{\beta} \tilde{\alpha}_y$	$(\mathbf{i} \Sigma_y) \otimes \mathbf{i} \Sigma_y$	$\check{\jmath} (\check{\imath} \Sigma_y)$
$\tilde{\beta} \tilde{\alpha}_x$	$(\mathbf{i} \Sigma_y) \otimes \mathbf{i} \Sigma_x$	$\check{k} (\check{\imath} \Sigma_y)$
$\tilde{\gamma}^5$	$\Sigma_x \otimes \mathbf{I}$	Σ_x
$\tilde{\alpha}_z$	$\Sigma_x \otimes \mathbf{i} \Sigma_z$	$\check{\imath} \Sigma_x$
$\tilde{\alpha}_y$	$\Sigma_x \otimes \mathbf{i} \Sigma_y$	$\check{\jmath} \Sigma_y$
$\tilde{\alpha}_x$	$\Sigma_x \otimes \mathbf{i} \Sigma_x$	$\check{k} \Sigma_x$

It is important to note that although the time-reversal antisymmetric operators $\tilde{\alpha}_k$, $\tilde{\Sigma}_k$, $\tilde{\beta} \tilde{\alpha}_k$, and $\tilde{\beta} \tilde{\Sigma}_k$ (with $k=x,y,z$ and z') become time-reversal symmetric once they are multiplied by the imaginary unit $\check{\imath}$ and written in quaternion form as described above, DIRAC saves the information that these operators were originally T-reversal antisymmetric (before adding the imaginary phase).

For further details on the quaternion symmetry scheme used in DIRAC, see Ref. Saue1999.

Note: For complete consistency, the names of the operators related to $\tilde{\alpha}_k$, $\tilde{\Sigma}_k$, $\tilde{\beta} \tilde{\alpha}_k$, and $\tilde{\beta} \tilde{\Sigma}_k$ ($k=x,y,z$) could have been $\mathbf{X}iALPHA$, $\mathbf{Y}iALPHA$, $\mathbf{Z}iALPHA$, $\mathbf{X}iAVECTOR$, $\mathbf{Y}iAVECTOR$, $\mathbf{Z}iAVECTOR$, $\mathbf{i}ALPHADOT$, $\mathbf{X}iBETAALP$, $\mathbf{Y}iBETAALP$, $\mathbf{Z}iBETAALP$, $\mathbf{X}iSIGMA$, $\mathbf{Y}iSIGMA$, $\mathbf{Z}iSIGMA$, $\mathbf{X}iBETASIG$, $\mathbf{Y}iBETASIG$, $\mathbf{Z}iBETASIG$, $\mathbf{i}SIGMADOT$, and $\mathbf{i}BEALDOT$ (see [Operator type keywords](#)). However, due to historical reasons, this convention is not used in DIRAC.

Note for advanced users: The operator $\mathbf{i}BETAGAM$ is implemented in the code just as $\tilde{\beta} \tilde{\gamma}^5$, but its time-reversal symmetry without imaginary phase is set as odd.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/optimize.md

orphan

*OPTIMIZE

*OPTIMIZE

Geometry optimization directives.

This section controls the geometry optimization. The geometry optimization algorithm is based on the one of [Dalton](#) implemented by Vebjørn Bakken, University of Oslo. The directives are therefore similar except for these few changes:

- No second-order algorithms are available since the molecular Hessian is not implemented.
- A few new keywords have been introduced (see below).

If ****PROPERTIES** or ****ANALYZE** is specified in the job input section together with ***OPTIMIZE**, then the property or analysis module is called in each optimization iteration and at the converged geometry.

Depending on convergence criteria for energies and gradients the values of the thresholds may be automatically adjusted.

Geometry optimization works for closed-shell and single open-shell Hartee-Fock method with analytical molecular gradient both with and without symmetry and for closed-shell DFT method without symmetry. Geometry optimization for other SCF methods (e.g. DFT with symmetry or open shells; and HF with Gaunt or open shells with more than one electron in one orbital) works with numerical gradient (.NUMGRA). Non-SCF methods should also work with numerical gradient, but not all possibilities are tested. Please report to [dirac-users mailing list](#) if you encounter problems.

.NO SKIP

.NO SKIP

Don't skip SSSL or SSSS gradient contributions when small. This keyword forces the LS and SS two-electron gradient to always be evaluated in all geometry iterations (depending on the integral flag).

The SL and SS two-electron integral contributions to the gradient are normally skipped if their contribution is estimated to be small. An "empirical" estimate of the norms of the LS and SS two-electron gradients based on the norm of the LL two-electron gradient. This trick is by default activated in a geometry optimization, since when the current geometry is far away from the equilibrium, e.g. the norm of the gradient is, say, 1.0, then there is no need to calculate the LS and/or SS two-electron gradient because they have a norm of, say, 0.001.

.NUMGRA

.NUMGRA

Force the use of numerical gradient in geometry optimization.

.TWOGRD

.TWOGRD

Include LL, SL, and SS integral contributions to the gradient (1 = on; 0 = off). The default is to turn all on:

```
.TWOGRD
1 1 1
```

.BAKER

.BAKER

Baker's convergence criteria Baker1993 will be used. The minimum is then said to be found when the largest element of the gradient vector (in Cartesian or redundant internal coordinates) falls below 3.0D-4 and either the energy change from the last iteration is less than 1.0D-6 or the largest element of the predicted step vector is less 3.0D-4.

.GRADIENT

.GRADIENT

Convergence threshold for the molecular gradient. The default is 1.0D-4 Hartree/bohr.

.ENERGY

.ENERGY

Convergence threshold for the energy change. The default is 1.0D-6 Hartree.

.STEP THRESHOLD

.STEP THRESHOLD

Convergence threshold for the geometry step length. The default is 1.0D-4 bohr.

.PRINT

.PRINT

Set print level for this module. Read one more line containing print level. Default value is 0, any value higher than 12 gives debugging level output.

.MAX IT

.MAX IT

Read the maximum number of geometry iterations. The default value is 25.

.TRUSTR

.TRUSTR

Set initial trust radius for calculation. This will also be the maximum step length for the first iteration. The trust radius is updated after each iteration depending on the ratio between predicted and actual energy change. The default trust radius is 0.5 a.u.

.TR FAC

.TR FAC

READ(LUCMD,*) TRSTIN, TRSTDE

Read two factors that will be used for increasing and decreasing the trust radius respectively. Default values are 1.2 and 0.7.

.TR LIM

.TR LIM

```
READ(LUCMD,*) RTENBD, RTENG, RTRJMN, RTRJMX
```

Read four limits for the ratio between the actual and predicted energies. This ratio indicates how good the step is?that is, how accurately the quadratic model describes the true energy surface. If the ratio is below RTRJMN or above RTRJMX, the step is rejected. With a ratio between RTRJMN and RTENBD, the step is considered bad an the trust radius decreased to less than the step length. Ratios between RTENBD and RTENG are considered satisfactory, the trust radius is set equal to the norm of the step. Finally ratios above RTENG (but below RTRJMX) indicate a good step, and the trust radius is given a value larger than the step length. The amount the trust radius is increased or decreased can be adjusted with .TR FAC. The default values of RTENBD, RTENG, RTRJMN and RTRJMX are 0.4, 0.8, -0.1 and 3.0 respectively.

.MAX RE

.MAX RE

```
READ(LUCMD,*) MAXREJ
```

Read maximum number of rejected steps in each iterations, default is 3.

.NOTRUS

.NOTRUS

Turns off the trust radius, so that a full Newton step is taken in each iteration. This should be used with caution, as global convergence is no longer guaranteed. If long steps are desired, it is safer to adjust the initial trust radius and the limits for the actual/predicted energy ratio.

.CONDIT

.CONDIT

```
READ (LUCMD,*) ICONDI
```

Set the number of convergence criteria that should be fulfilled before convergence occurs. There are three different convergence thresholds, one for the energy, one for the gradient norm and one for the step norm. The possible values for this variable is therefore between 1 and 3. Default is 2. The three convergence thresholds can be adjusted with the keywords OPTIMIZE_.ENERGY, OPTIMIZE_.GRADIENT and OPTIMIZE_.STEP THRESHOLD.

.NOBREA

.NOBREA

Disables breaking of symmetry. The geometry will be optimized within the given symmetry, even if a non-zero molecular Hessian index is found. The default is to let the symmetry be broken until a minimum is found with a molecular Hessian index of zero. This option only has effect when second-order methods are used.

.SP BAS

.SP BAS

```
READ(LUCMD,*) SPBSTX
```

Read a string containing the name of a basis set. When the geometry has converged, a single-point energy will be calculated using this basis set.

.PREOPT

.PREOPT

```
READ (LUCMD,*) NUMPRE  
DO I = 1, NUMPRE  
  READ (LUCMD,*) PREBTX(I)  
END DO
```

First we read the number of basis sets that should be used for preoptimization, then we read those basis set names as strings. These sets will be used for optimization in the order they appear in the input. One should therefore place the smaller basis at the top. After the preoptimization, optimization is performed with the basis specified in the molecule input file.

.VISUAL

.VISUAL

Specifies that the molecule should be visualized, writing a VRML file of the molecular geometry. No optimization will be performed when this keyword is given. See also related keywords .VR-BON, .VR-COR, .VR-EIG and .VR-VIB.

.VRML

.VRML

Specifies that the molecule should be visualized. VRML files describing both the initial and final geometry will be written (as initial.wrl and final.wrl). The file final.wrl is updated in each iteration, so that it always reflects the latest geometry. See also related keywords .VR-BON, .VR-COR, .VR-EIG and .VR-VIB.

.SYMTHR

.SYMTHR

READ(LUCMD,*) THRSYM

Determines the gradient threshold (in a.u.) for breaking of the symmetry. That is, if the index of the molecular molecular Hessian is non-zero when the gradient norm drops below this value, the symmetry is broken to avoid unnecessary iterations within the wrong symmetry. This option only applies to second-order methods and when the keyword .NOBREA is not present. The default value of this threshold is 0.005.

.TRSTRG

.TRSTRG

Specifies that the level-shifted trust region method should be used to control the step. This is the default, so the keyword is actually redundant at the moment. Alternative step control methods are OPTIMIZE_.RF and OPTIMIZE_.GDIIS.

.VR-BON

.VR-BON

Only has effect together with .VRML or .VISUAL. Specifies that the VRML files should include bonds between nearby atoms. The bonds are drawn as grey cylinders, making it easier to see the structure of the molecule. If .VR-BON is omitted, only the spheres representing the different atoms will be drawn.

.VR-EIG

.VR-EIG

Only has effect together with .VRML or .VISUAL. Specifies that the eigenvectors of the molecule (that is the eigenvectors of the molecular Hessian, which differs from the normal modes as they are not mass-scaled) should be visualized. These are written to the files [eigv###.wrl](#).

.INITHE

.INITHE

Specifies that the initial molecular Hessian should be calculated (analytical molecular Hessian), thus yielding a first step that is identical to that of second-order methods. This provides an excellent starting point for first-order methods, but should only be used when the molecular Hessian can be calculated within a reasonable amount of time. It has only effect for first-order methods and overrides the keywords .INITEV and .INIRED. It has no effect when .HESFIL has been specified.

.INITEV

.INITEV

READ(LUCMD,*) EVLINI

The default initial molecular Hessian for first-order minimizations is the identity matrix when Cartesian coordinates are used, and a diagonal matrix when redundant internal coordinates are used. If .INITEV is used, all the diagonal elements (and therefore the eigenvalues) are set equal to the value EVLINI. This option only has effect when first-order methods are used and .INITHE and .HESFIL are non-present.

.HESFIL

.HESFIL

Specifies that the initial molecular Hessian should be read from the file DALTON.HES(?). This applies to first-order methods, and the Hessian in the file must have the correct dimensions. This option overrides other options for the initial Hessian. Each time a Hessian is calculated or updated, it's written to this file (in Cartesian coordinates). If an optimization is interrupted, it can be restarted with the last geometry and the molecular Hessian in DALTON.HES, minimizing the loss of information. Another useful possibility is to transfer the molecular Hessian from a calculation on the same molecule with another (smaller) basis and/or a cheaper wave function. Finally, one can go in and edit the file directly to set up a specific force field.

.REJINI

.REJINI

Specifies that the molecular Hessian should be reinitialized after every rejected step, as a rejected step indicates that the molecular Hessian models the true potential surface poorly. Only applies to first-order methods.

.STEEP

.STEEP

Specifies that the first-order steepest descent method should be used. No update is done on the molecular Hessian, so the optimization will be guided by the gradient alone. The 'pure' steepest descent method is obtained when the molecular Hessian is set equal to the identity matrix. Each step will then be the negative of the gradient vector,

and the convergence towards the minimum will be extremely slow. However, this option can be combined with other initial molecular Hessians in Cartesian or redundant internal coordinates, giving a method where the main feature is the lack of molecular Hessian updates (static molecular Hessian).

.RANKON

.RANKON

Specifies that a first-order method with the rank one update formula should be used for optimization. This updating is also referred to as symmetric rank one (SR1) or Murtagh-Sargent (MS).

.PSB

.PSB

Specifies that a first-order method with the Powell-Symmetric-Broyden (PSB) update formula should be used for optimization.

.DFP

.DFP

Specifies that a first-order method with the Davidon-Fletcher-Powell (DFP) update formula should be used for optimization. May be used for both minimizations and transition state optimizations.

.BFGS

.BFGS

Specifies the use of a first-order method with the Broyden-Fletcher-Goldfarb-Shanno (BFGS) update formula for optimization. This is the preferred first-order method for minimizations, as this update is able to maintain a positive definite Hessian. Note that this also makes it unsuitable for transitions state optimization (where one negative eigenvalue is sought).

.NEWTON

.NEWTON

Specifies that a second-order Newton method should be used for optimization that is, the analytical molecular Hessian will be calculated at every geometry. By default the level-shifted trust region method will be used, but it is possible to override this by using one of the two keywords OPTIMIZE_.RF or OPTIMIZE_.GDIIIS. Not implemented in DIRAC.

.QUADSD

.QUADSD

Use the 2nd order quadratic steepest descent method (missing in Dalton manual).

.SCHLEG

.SCHLEG

Specifies that a first-order method with Schlegel's updating scheme [dalton ref 152] should be used. This makes use of all previous displacements and gradients, not just the last, to update the molecular Hessian.

.HELLMA

.HELLMA

Use gradients and Hessians calculated using the Hellmann-Feynman approximation. Currently not working properly.

.M-BFGS

.M-BFGS

A list of old geometries and gradients are kept. At each new point, displacements and gradient difference for the last few steps are calculated, and all of these are then used to sequentially update the molecular Hessian, the most weight being given to the last displacement and gradient difference. Each update is done using the BFGS formula, and it's thus only suitable for minimizations. Only applies to first-order methods.

.CARTES

.CARTES

Indicates that Cartesian coordinates should be used in the optimization. This is the default for second-order methods.

.REDINT

.REDINT

Specifies that redundant internal coordinates should be used in the optimization. This is the default for first-order methods.

.INIRED

.INIRED

Use a simple model Hessian [dalton ref 18] diagonal in redundant internal coordinates as the initial Hessian. All diagonal elements are determined based on an extremely simplified molecular mechanics model, yet this model provides Hessians that are good starting points for most systems, thus avoiding any calculation of the exact Hessian. This is the default for first-order methods.

.1STORD

.1STORD

Use default first-order method. This means that the BFGS update will be used, and that the optimization is carried out in redundant internal coordinates. Same effect as the combination of the two keywords OPTIMIZE_.BFGS and OPTIMIZE_.REDINT. Since the BFGS method ensures a positive definite Hessian, the OPTIMIZE_.BOFILL optimization method is used by default in case of searches for transition states.

.2NDORD

.2NDORD

Use default second-order method. Molecular Hessians will be calculated at every geometry. The level-shifted Newton method and Cartesian coordinates are used. Identical to specifying the keywords OPTIMIZE_.NEWTON and OPTIMIZE_.CARTES. Not implemented in DIRAC.

.GRDINI

.GRDINI

Specifies that the molecular Hessian should be reinitialized every time the norm of the gradient is larger than norm of the gradient two iterations earlier. This keyword should only be used when it's difficult to obtain a good approximation to the molecular Hessian during optimization. Only applies to first-order methods.

.DISPLA

.DISPLA

```
READ (LUCMD,*) DISPLA
```

Read one more line containing the norm of the displacement vector to be used during numerical evaluation of the molecular gradient, as is needed when doing geometry optimizations with CI or MP2 wave functions. Default is 0.001 a.u.

.CONSTR

.CONSTR

```
READ (LUCMD, *) NCON
DO I = 1, NCON
  READ(LUCMD,*) ICON
  ICNSTR(ICON) = 1
END DO
```

Request a constrained geometry optimization. Only works when using redundant internal coordinates. The number of primitive coordinates that should be frozen has to be specified (NCON), then a list follows with the individual coordinate numbers. The coordinate numbers can be found by first running Dalton(DIRAC?) with the .FINDRE keyword. Any number of bonds, angles and dihedral angles may be frozen. NOTE: Symmetry takes precedence over constraints, if you e.g. want to freeze just one of several symmetric bonds, symmetry must be lowered or switched off.

.MODHES

.MODHES

Determine a new model molecular Hessian (see .INIMOD) at every geometry without doing any updating. The model is thus used in much the same manner as an exact molecular Hessian, though it is obviously only a relatively crude approximation to the analytical molecular Hessian.

.REMOVE

.REMOVE

```
READ (LUCMD, *) NREM
DO I = 1, NREM
  READ(LUCMD,*) IREM
  ICNSTR(IREM) = 2
END DO
```

Only has effect when using redundant internal coordinates. Specifies internal coordinates that should be removed. The input is identical to the one for .CONSTRAINT, that is one has to specify the number of coordinates that should be removed, then the number of each of those internal coordinates. The coordinate numbers can first be

determined by running with .FINDRE set. Removing certain coordinates can sometimes be useful in speeding up constrained geometry optimization, as certain coordinates sometimes “struggle” against the constraints. See also .NODIHE.

.INIMOD

.INIMOD

Use a simple model Hessian [dalton ref 18] diagonal in redundant internal coordinates as the initial Hessian. All diagonal elements are determined based on an extremely simplified molecular mechanics model, yet this model provides Hessians that are good starting points for most systems, thus avoiding any calculation of the exact Hessian. This is the default for first-order methods.

.FINDRE

.FINDRE

Determines the redundant internal coordinate system then quits without doing an actual calculation. Useful for setting up constrained geometry optimizations, where the numbers of individual primitive internal coordinates are needed.

.CMBMOD

.CMBMOD

Uses a combination of the BFGS update and the model molecular Hessian (diagonal in redundant internal coordinates). The two have equal weight in the first iteration of the geometry optimization, then for each subsequent iteration the weight of the model Hessian is halved. Only suitable for minimizations.

.RF

.RF

Use the rational function method [dalton ref.12] instead of level-shifted Newton which is the default. The RF method is often slightly faster than the level-shifted Newton, but also slightly less robust. For saddle point optimizations there's a special partitioned rational function method (used automatically when both .RF and .SADDLE are set). However, this method is both slower and less stable than the default trust-region level-shifted image method (which is the default).

.GDIIS

.GDIIS

Use the Geometrical DIIS[dalton ref 151] algorithm to control the step. Works in much the same way as DIIS for wave functions. However, the rational function and level-shifted Newton methods are generally more robust and more efficient. Can only be used for minimizations.

.DELINT

.DELINT

Use delocalized internal coordinates. These are built up as non-redundant linear combinations of the redundant internal coordinates. Performance is more or less the same as for the redundant internals, but the transformation of displacements (step) is slightly less stable.

.NODIHE

.NODIHE

No dihedral angles will be used as coordinates (just bonds and angles).

.VR-COR

.VR-COR

Draws x-, y- and z-axis in the VRML scenes with geometries. Somewhat useful if one is struggling to build a reasonable geometry by adjusting coordinates manually.

.VR-VIB

.VR-VIB

Similar to .VR-EIG, but more useful as it draws the actual normal mode vectors (the mass-weighted eigenvectors). These are written to the files [norm###.wrl](#). Keyword only has effect when a vibrational analysis has been requested.

.VR-SYM

.VR-SYM

Draws in all symmetry elements of the molecule as vectors (rotational axes) and semi-transparent planes (mirror planes).

.M-PSB

.M-PSB

This identical to .M-BFGS, except the PSB formula is used for the updating. Only applies to first-order methods, but it can be used for both minimizations and saddle point optimizations.

.LINE S

.LINE S

Turns on line searching, using a quartic polynomial. By default this is turned off, as there seems to be no gain in efficiency. Can only be used for minimizations.

.SADDLE

.SADDLE

Indicates that a saddle point optimization should be performed rather than a minimization. The default method is to calculate the molecular Hessian analytically at the initial geometry, then update it using Bofill's update. The optimization is performed in redundant internal coordinates and using the trust-region level-shifted image method to control the step. That is by default all the keywords .INITHE, .FILL and .REDINT are already set, but this can of course be overridden by specifying other keywords. If locating the desired transition state is difficult, and provided analytical molecular Hessians are available, it may sometimes be necessary to use the .NEWTON keyword so that molecular Hessians are calculated at every geometry.

.MODE

.MODE

READ(LUCMD,*) NSPMOD

Only has effect when doing saddle point optimizations. Determines which molecular Hessian eigenmode should be maximized (inverted in the image method). By default this is the mode corresponding to the lowest eigenvalue, i.e. mode 1. If an optimization does not end up at the correct transition state, it may be worthwhile following other modes (only the lower ones are usually interesting).

.BOFILL

.BOFILL

Bofill's update [dalton ref 20] is the default updating scheme for transition state optimizations. It's a linear combination of the symmetric rank one and the PSB updating schemes, automatically giving more weight to PSB whenever the rank one potentially is numerically unstable.

.LINDH

.LINDH

Use original Roland Lindh r_ref with .MODHES (missing in Dalton manual).

.GRD IN

.GRD IN

Specifies that the molecular Hessian should be reinitialized every time the norm of the gradient is larger than norm of the gradient two iterations earlier. This keyword should only be used when it's difficult to obtain a good approximation to the molecular Hessian during optimization. Only applies to first-order methods.

.GRD SCREEN

.GRD SCREEN

READ (LUCMD,*) SCRGRD

Read the screening threshold for gradient calculations (missing in Dalton manual).

.NOAUX

.NOAUX

Only has effect when using redundant internal coordinates. The default for minimizations is to add auxiliary bonds between atoms that are up to two and half times further apart than regular (chemical) bonds. This increases the redundancy of the coordinate system, but usually speeds up the geometry optimization slightly. .NOAUX turns this off. For saddle point optimizations and constrained geometry optimization this is off by default (cannot be switched on).

.BFGSR1

.BFGSR1

Use a linear combination of the BFGS and the symmetric rank one updating schemes in the same fashion as Bofill's update. Only suitable for minimizations.

.IPRGRD

.IPRGRD

Print level for DIRAC molecular gradient evaluation (missing in Dalton manual).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/orbital_strings.md

orphan

Specification of orbital strings

A subset of orbitals can be specified in two ways. Usually it suffices to specify the keyword energy, followed by three real numbers to specify a lower limit, upper limit and quasi-degeneracy threshold. All orbitals that have an energy between the two thresholds are then selected. The quasi-degeneracy threshold is used to extend the range if the lower or upper threshold cuts through a set of closely energy-spaced orbitals. It will move the upper or lower limit until it encounters a large enough gap between orbital energies. The default value used in the AO-MO integral transformation program to specify the active space for correlated calculations is e.g. given by the string:

```
energy -10.0 20.0 1.0
```

This picks out all orbitals that have an energy between -10.0 and +20.0 hartree (a.u.), with a minimum gap of 1.0 hartree.

More detailed control is possible by using an orbital string. Here a character string is given where individual orbitals are listed separated by a comma or as a range of orbitals. The range limit is then separated by “..”. Positive energy and negative energy orbitals are indicated using positive and negative numbers, respectively. For instance the orbital string:

```
-5..5,7
```

This picks out the five upper negative energy (“positronic”) and five lower positive energy (“electronic”) orbitals plus positive energy orbital number seven.

If all orbitals (both positive and negative energy) are wanted, then one may write:

```
all
```

The concept of orbitals strings gives great flexibility in the specification of orbitals and serves as an alternative to editing of vector files. A drawback is, however, that one needs to count the number of orbitals manually, which may become cumbersome for cases with large ranges of virtual orbitals.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/parallel.md

orphan

*PARALLEL

*PARALLEL

Parallelization directives.

.PRINT

.PRINT

Print level for the parallel calculation. Default:

```
.PRINT
0
```

A print level of at least 2 is needed in order to be able to evaluate the parallelization efficiency. A complete timing for all nodes will be given if the print level is 4 or higher.

.NTASK

.NTASK

Number of tasks to send to each node when distributing the calculation of two-electron integrals. Default:

```
.NTASK
1
```

A task is defined as a shell of atomic integrals, a shell being an input block. One may therefore increase the number of shells given to each node in order to reduce the amount of communication. However, the program uses dynamical allocation of work to each node, and thus this option should be used with some care, as too large tasks may cause the dynamical load balancing to fail, giving an overall decrease in efficiency. The parallelization is also very coarse grained, so that the amount of communication seldom represents any significant problem.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/pcm.md

orphan

*PCM

*PCM

Polarizable continuum model (PCM) directives.

General information on the PCM can be found in Tomasi2005, whereas details concerning the Dirac implementation are found in DiRemigio2015.

[!WARNING] Calculations with the polarizable continuum model **cannot** exploit molecular point group symmetry and/or 2-component Hamiltonians.

General

.SKIPSS

.SKIPSS

Performs a PCM-SCF calculation with the PCM-SCC approximation. The small-small block of the electrostatic potential is neglected, in the spirit of the SCC approximation described in Visscher1997a. It is not employed by default.

[!WARNING] Currently not working for response calculations.

.PRINT

.PRINT

Print level in the PCM subroutines. Default:

```
.PRINT
0
```

Advanced/Debug

.SEPARA

.SEPARA

Performs PCM-SCF separating the nuclear and electronic electrostatic potential and apparent surface charge. Results are unaffected. *For debug purpose only*

.DOSPF

.DOSPF

Remove spin-orbit dependent part from PCM potential. *For debug purpose only*

.SKIPPOI

.SKIPPOI

Skips the calculation of the one-index transformed apparent surface charge in the linear response function. *For debug purpose only*

FILE: www.diracprogram.org/doc/release-26/_sources/manual/pcmsolver.md

orphan

*PCMSOL

*PCMSOL

PCMSolver module configuration directives.

General information on the PCM can be found in Tomasi2005. Details on the module can be found [here](#) The module source code is freely available under the LGPLv3 license [on this URL](#).

There are two ways of providing input to the PCMSolver module:

1. by means of an additional input file, parsed by the pcmsolver.py script;
2. by means of a special section in the DIRAC input.

Method 1 is more flexible: all parameters that can be modified by the user are available. Method 2 just gives access to the core parameters. We will here describe Method 2, for a description of Method 1 refer to the [PCMSolver input documentation](#) Examples of both methods are given in the PCM tutorial section. All keywords expecting a numerical value are intended to be in Angstrom. The module will perform conversion to atomic units internally.

[!WARNING] Do **NOT** truncate keyword values to six characters. It will result in a crash with an obscure error message.

[!WARNING] Some keywords can be set but won't have any effect on the module as their respective functionality is not in the released version of PCMSolver.

Cavity directives

The generating spheres for the cavity are obtained by centering spheres on the atoms making up the molecule. Radii are automatically selected from a built-in set and are scaled by 1.2. For a finer control over the cavity creation details use Method 1.

.CAVTYP

.CAVTYP

The type of the cavity. Completely specifies type of molecular surface and its discretization. Allowed values are GEPOL and RESTART. If RESTART is given, the module will read the file specified by the RESTART keyword and create a GePol cavity from that. Default:

```
.CAVTYP
GEPOL
```

.PATCHL

.PATCHL

[!WARNING] not available in the current release of the module.

.COARSI

.COARSI

[!WARNING] not available in the current release of the module.

.AREATS

.AREATS

Average area of the surface partition for the GePol cavity. Default:

```
.AREATS
0.3
```

.MINDIS

.MINDIS

[!WARNING] not available in the current release of the module.

.DERORD

.DERORD

[!WARNING] not available in the current release of the module.

.NOSCAL

.NOSCAL

If present the radii for the spheres will **not** be scaled by 1.2. Radii are scaled by default.

.RADIIS

.RADIIS

The set of built-in radii to be used. Allowed values are BONDI and UFF. Refer to the [PCMSolver input documentation](#) for further details about the radii sets. Default:

```
.RADIIS
BONDI
```

.RESTAR

.RESTAR

The name of the .npz file to be used for the GePol cavity restart.

.MINRAD

.MINRAD

Minimal radius for additional spheres not centered on atoms. An arbitrary big value is equivalent to switching off the use of added spheres. By default, no spheres not centered on atoms are added. Default:

```
.MINRAD
100.0
```

Solver directives

.SOLVER

.SOLVER

Type of solver to be used. All solvers are based on the Integral Equation Formulation of the Polarizable Continuum Model:

- IEFPCM. Collocation solver for a general dielectric medium;
- CPCM. Collocation solver for a conductor-like approximation to the dielectric medium.

.SOLVNT

.SOLVNT

Specification of the dielectric medium outside the cavity. This keyword **must always** be given a value. Allowed values are:

- WATER
- METHANOL
- ETHANOL
- CHLOROFORM
- METHYLENECHLORIDE
- 1,2-DICHLOROETHANE
- CARBON TETRACHLORIDE
- BENZENE
- TOLUENE
- CHLOROBENZENE
- NITROMETHANE
- N-HEPTANE
- CYCLOHEXANE
- ANILINE
- ACETONE
- TETRAHYDROFURANE
- DIMETHYLSULFOXIDE
- ACETONITRILE
- EXPLICIT

For further details on the dielectric properties of the available built-in solvents refer to the [PCMSolver input documentation](#). If the solvent name given is different from Explicit any other settings in the Green's function section will be overridden by the built-in values for the solvent specified. Solvent = Explicit, triggers parsing of the Green's function directives.

.EQNTYP

.EQNTYP

[!WARNING] not available in the current release of the module.

.CORREC

.CORREC

Correction, k for the apparent surface charge scaling factor in the CPCM solver $\epsilon(\text{varepsilon}) = \frac{\text{varepsilon} - 1}{\text{varepsilon} + k}$. Default:

.CORREC
0.0

.PROBER

.PROBER

Radius of the spherical probe approximating a solvent molecule. Used for generating the solvent-excluded surface (SES) or an approximation of it. Overridden by the built-in value for the chosen solvent. Default:

.PROBER
1.0

Green's function directives

If solvent is Explicit, **both** the Green's function inside and outside must be specified. The Green's function inside will always be the vacuum, while the Green's function outside might vary. Currently, only isotropic uniform dielectrics are implemented.

.GINSID

.GINSID

Type of Green's function inside the cavity. Default:

.GINSID
VACUUM

.GOUTSI

.GOUTSI

Type of Green's function inside the cavity. Default:

```
.GINSID
UNIFORMDIELECTRIC
```

```
.EPSILO
```

.EPSILO

Static dielectric permittivity for the medium outside the cavity. Default:

```
.EPSILO
1.0
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/pelib.md

orphan

*PELIB

*PELIB

Polarizable embedding model

This input section controls calculations using the polarizable embedding (PE) or polarizable density embedding (PDE) models. See Steinmann2019 for a tutorial review of how to use the PE model.

Citations for DIRAC implementations:

- PE implementation in DIRAC Hedegaard2017.
- PDE implementation in DIRAC Larsson2025.

For examples of calculations with different response methods and Hamiltonians, see the tutorial `pe_response`).

In DIRAC it is possible to include the effects from a structured environment on a core molecular system using the PE or PDE models. Note that the PDE model is currently only implemented with the X2C Hamiltonian. The current implementation is a layered QM/MM-type embedding model capable of using advanced potentials that include an electrostatic component as well as an induction (polarization) component. The effects of the environment are included through effective operators that contain an embedding potential, which is a representation of the environment, thereby directly affecting the molecular properties of the core system. The wave function of the core system is optimized while taking into account the explicit electrostatic interactions and induction effects from the environment in a fully self-consistent manner. The electrostatic and induction components are modeled using Cartesian multipole moments and anisotropic dipole-dipole polarizabilities, respectively. The electrostatic part models the permanent charge distribution of the environment and will polarize the core system, while the induction part also allows polarization of the environment. The environment response is included in the effective operator as induced dipoles, which arise due to the electric fields from the electrons and nuclei in the core system as well as from the environment itself. It is therefore necessary to recalculate the induced dipoles according to the changes in the electron density of the core system as they occur in a wave function optimization. Furthermore, since the induced dipoles are coupled through the electric fields, it is necessary to solve a set of coupled linear equations. This can be done using either an iterative or a direct solver. This also means that we include many-body effects of the total system.

The multipoles and polarizabilities can be obtained in many different ways. It is possible to use the molecular properties, however, usually distributed/localized properties are used because of the better convergence of the multipole expansion. These are typically centered on all atomic sites in the environment (and sometimes also bond-midpoints), however, the implementation is general in this sense so they can be placed anywhere in space. Currently, the PE library supports multipole moments up to fifth order and anisotropic dipole-dipole polarizabilities are supported. For multipoles up to and including third order (octopoles) the trace will be removed if present. Note, that the fourth and fifth order multipole moments are expected to be traceless. In case polarizabilities are included it might be necessary to use an exclusion list to ensure that only relevant sites can polarize each other. The format of the POTENTIAL.INP file is demonstrated below.

A PDE calculation involves two main stages, first a preparation stage, which is used to derive the embedding operators and parameters (electrostatic and repulsion operators; distributed polarizabilities), and, second, the final embedding calculation. The preparation stage consists of two parts. Monomer calculations are used to obtain the fragment densities of the environment, and these are followed by dimer calculations to evaluate operator contributions to the core region from each fragment. The calculations needed for the setup are somewhat involved, but they are automated in the PyFraME Python package `pyframe:0.4.0`, and this is the recommended way of preparing such calculations. PyFraME uses the Dalton program for the monomer and dimer calculations, thus it is assumed that all relativistic effects are in the QM region. PyFraME will provide the HDF5 file with the embedding potential parameters. A minimal example input for deriving a PDE potential with PyFraME is given below.

Keywords

.PDE

.PDE

This option enables the polarizable density embedding (PDE) model. It requires a valid HDF5 file generated by PyFraME containing the embedding potential parameters, e.g.:

```
.PDE
h2o.h5
```

There is no default name for this file. The PDE file needs to be placed into the work directory with the “`-put=h2o.h5`” option when running DIRAC. Note that PDE currently only works with the X2C Hamiltonian. This file only contains the frozen density. Polarizabilities and other multipole moments are kept in the POTENTIAL file (see .POTENT keyword).

Note that the PDE option is only available, if DIRAC has been compiled with `gfortran`. (Technical note. This is so because the standard `hdf5-fortran` library on linux systems have been compiled with `gfortran`, and therefore it cannot be used by other Fortran compilers as Intel `ifx` or Nvidia `nvfortran`.)

.POTENTIAL

POTENTIAL

This option can be used to specify a non-default name of the POTENTIAL.INP input file that contains the embedding potential parameters, e.g.:

```
.POTENT
h2o.pot
```

The potential file needs to be placed into the work directory with the “-put=h2o.pot” option when running DIRAC. Alternatively, this file can be added by using the command “-pot=h2o.pot” which renames it to POTENTIAL.INP.

```
.DIRECT
```

DIRECT

Use the direct solver to determine the induced dipole moments. It will explicitly build a classical response matrix of size $\frac{3S(3S+1)}{2}$, where S is the number of polarizable sites and is therefore only recommendable for smaller molecular systems. Note that this solver is not parallelized. The default is to use the iterative solver

```
.ITERATIVE
```

ITERATIVE

Use the iterative solver to determine the induced dipole moments. This is the default. The convergence threshold defaults to $1.0 \times 10^{-8} \sum_{s=1}^S |\mu^{(k)}_s - \mu^{(k-1)}_s|$, where k is the current iteration, but can also be provided with this option:

```
READ (LUCMD, *) THRITER (optional)
```

```
.BORDER
```

BORDER

Controls the handling of the border between the core molecular system and its environment described by the embedding potential.:

```
1) READ (LUCMD, *) BORDER_TYPE, RMIN, AUORAA
2) READ (LUCMD, *) BORDER_TYPE, REDIST_ORDER, RMIN, AUORAA, NREDIST
```

There are two mutually exclusive schemes:

```
1) BORDER_TYPE = REMOVE 2) BORDER_TYPE = REDIST
```

The first option will remove all multipoles and polarizabilities that are within the given distance “RMIN” from any atom in the core molecular system. The “AUORAA” variable specifies whether the distance threshold is given in ångström (“AA”) or bohr (“AU”). The second option will redistribute parameters that are within the given threshold “RMIN” from any atom in the core system to nearest sites in the environment. The order of multipoles up to which will be redistributed is determined by the “REDIST_ORDER” variable, e.g. “REDIST_ORDER = 1” means that only charges will be redistributed and all other parameters removed, “REDIST_ORDER = 2” means charges and dipoles are redistributed and so on. The sign of “REDIST_ORDER” specifies if the polarizabilities are redistributed. Positive means that the polarizabilities are removed and negative means redistributed. The number of sites that parameters on a given site are redistributed to is determined by the “NREDIST” variable which can be between 1 and 3. The default is to redistribute charges within 2.2 bohr to its nearest site and removing all other parameters.

```
.DAMP INDUCED
```

DAMP INDUCED

Damp the electric fields from induced dipole moments using Thole’s exponential damping scheme.:

```
READ (LUCMD, *) IND_DAMP (optional)
```

The default damping coefficient is the standard 2.1304.

```
.DAMP MULTIPOLE
```

DAMP MULTIPOLE

Damp the electric fields from permanent multipole moments using Thole’s exponential damping scheme.:

```
READ (LUCMD, *) MUL_DAMP (optional)
```

This option requires polarizabilities on all sites with permanent multipole moments. The default damping coefficient is the standard 2.1304.

```
.DAMP CORE
```

DAMP CORE

Damp the electric fields from the electrons and nuclei in the core region based on Thole’s exponential damping scheme:

```
READ (LUCMD, *) CORE_DAMP (optional)
READ (LUCMD, *) NALPHAS
DO I = 1, NALPHAS
  READ (LUCMD, *) ISO_ALPHA(I)
END DO
```

The damping coefficient is optional unless isotropic polarizabilities are present. The default damping coefficient is the standard 2.1304. Standard polarizabilities from vanduijnen1998 have been implemented and will be used if none are given as input. However, only H, C, N, O, F, S, Cl, Br and I are available.

```
.GSPOL
```

.GSPOL

Activate the ground-state polarization approximation, i.e. freeze the embedding potential according to the ground-state density. This means that the polarizable environment is self-consistently relaxed during the optimization of the ground-state density/wave function of the core molecular system and then kept frozen in any following response calculations

.NOMB

.NOMB

Remove many-body effects in the environment. This is done by deactivating interactions between inducible dipole moments.

.RESTART

.RESTART

Use any existing files to restart calculation.

.CUBE

.CUBE

Create cube file for the core molecular system containing the electrostatic potential due to the final converged polarizable embedding potential.:

```
1) READ (LUCMD, *) STD_GRID
2) READ (LUCMD, *) OPTION
2) IF (OPTION == 'GRID') THEN
    READ (LUCMD, *) XSIZE, XGRID, YSIZE, YGRID, ZSIZE, ZGRID
2) END IF
IF (OPTION == 'FIELD') THEN
    FIELD = .TRUE.
END IF
```

The grid density can be specified either using 1) standard grids (COARSE (3 points/bohr), MEDIUM (6 points/bohr) or FINE (12 points/bohr) and in all cases 8.0 bohr are added in each direction) or 2) the GRID option which gives full control of the cube size and density, and requires an additional input line specifying the extent added (in bohr) in each direction and density (in points/bohr). The default grid density is MEDIUM and default cube size is the extent of the molecule plus 8.0 bohr in plus and minus each Cartesian coordinate. If the FIELD option is given then also three cube files containing the Cartesian components of the electric field from the embedding potential will be created.

.ISOPOL

.ISOPOL

Converts all anisotropic polarizabilities into isotropic polarizabilities.

.ZEROPOL

.ZEROPOL

Remove all polarizabilities

.ZEROMUL

.ZEROMUL

Remove all multipoles of order ZEROMUL_ORDER and up.:

```
READ (LUCMD, *) ZEROMUL_ORDER (optional)
```

Remove all multipoles of order ZEROMUL_ORDER and up. The default is to remove all dipoles and higher-order multipoles (i.e. ZEROMUL_ORDER = 1).

.VERBOSE

.VERBOSE

Verbose output. Currently this will print the final converged induced dipole moments.

.DEBUG

.DEBUG

Debug output. Prints the total electric field and the induced dipole moments in each iteration. WARNING: for large systems this will produce very large output files.

The potential input format

The POTENTIAL.INP file is split into three sections: @COORDINATES, @MULTIPOLES and @POLARIZABILITIES. The format is perhaps best illustrated using an example:

../tutorials/polarizable_embedding/3_h2o.pot

@COORDINATES

The coordinates section follows the standard XYZ file format so that the environment can be easily visualized using standard programs. The first line in gives the total number of sites in the environment and the second line specifies whether the coordinates are given in ångström (AA) or bohr (AU). The rest of the coordinates section is a list of the sites in the environment where each line contains the element symbol and x-, y- and z-coordinates of a site. If a site is not located on an atom, e.g. if it is a bond-midpoint, then the element symbol should be specified as X. The listing also gives an implicit numbering of the sites, so that the first line is site number one, the second line is site number two and so on. This numbering is important and used in the following sections.

@MULTIPOLES

The multipoles section is subdivided into the orders of the multipoles, i.e. ORDER 0 for monopoles/charges, ORDER 1 for dipoles and so on. For each order there is a number specifying the number of multipoles of that specific order. Note, that this number does not have to be equal to the total number of sites. This is followed by a list of multipoles where each line gives the multipole of a site. The lines begin with a number that specifies which site the multipole is placed. Only the symmetry-independent Cartesian multipoles (given in a.u.) should be provided using an ordering such that the components are stepped from the right, e.g. xx xy xz yy yz zz or xxx xxy xxz xyy xyz xzz yyy yyz yzz zzz. Note, that the multipoles should in general be traceless, however, for multipoles up to and including octopoles (i.e. ORDER 3) the trace is removed if present. Furthermore, the current implementation is limited to fifth order multipoles.

@POLARIZABILITIES

The polarizabilities section is also subdivided into orders, i.e. ORDER 1 1 for dipole-dipole polarizabilities, which is the only type supported in the current release. The format is the same as for multipoles, i.e. first line contains number of polarizabilities which is followed by a list of the polarizabilities using the same ordering as the multipoles. The polarizabilities should also be given in a.u. In addition, there is also the exclusion lists (EXCLISTS section). Here the first line gives the number of lists (i.e. the number of lines) and the length of the exclusion lists (i.e. the number of entries per line). The exclusion lists specify the polarization rules. There is a list attached to each polarizable site that specifies which sites are not allowed to polarize it, e.g. 1 2 3 4 5 means that site number 1 cannot be polarized by sites 2, 3, 4 and 5.

The potential density embedding potential

A minimal example input for deriving a PDE potential with PyFraME can be found here [<PDE-example>](#).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties.md

orphan

**PROPERTIES

**PROPERTIES

This section allows for the evaluation of a large number of molecular properties. Available properties include:

- Expectation values (e.g. dipole moment and electric field gradients).
- Linear response properties (e.g. polarizability and NMR parameters).
- Quadratic response properties (e.g. hyperpolarizabilities).
- Quasi-degenerate CI perturbation theory for ESR parameters (g-tensor and hyperfine coupling tensors).

For convenience some common properties can be specified directly in this section, which means that the user in principle does not need to know how they are calculated. Note, however, that response functions are by default static, but frequencies can be added in the relevant subsections.

Properties which are not predefined must be specified in detail in the relevant input section (see [one_electron_operators](#)).

By default no properties are calculated.

General control statements

.PRINT

.PRINT

Print level.

Default:

```
.PRINT
0
```

.ABUNDANCIES

.ABUNDANCIES

For properties that make reference to isotopes, give threshold level (in % abundance) for isotopes to print.

Default:

```
.ABUNDANCIES
1.0
```

.RKBIMP

.RKBIMP

Import coefficients calculated with restricted kinetic balance (RKB) in a calculation using unrestricted kinetic balance (UKB). This option is a simple way to generated restricted magnetic balance for the calculation of NMR shieldings. This option works in the general SO case, but not in the spinfree case since spinfree calculations are not possible with UKB.

.NOPCTR

.NOPCTR

In two-component infinite-order relativistic calculations (with HAMILTONIAN_.X2C) take only LL block of four-component property operators to avoid the picture change transformation. Experimental option, use with care.

.RDCCDM

.RDCCDM

Activates the reading of the file CCDENS obtained from a previous CC calculation with either the WAVE_FUNCTION_.RELCCSD or with WAVE_FUNCTION_.EXACC modules. It is not necessary to use this keyword in runs in which the correlated and property modules are both activated.

CCDENS is not saved by pam, so unless the scratch directory from the previous calculation is kept (see `--keep_scratch` in pam), you should retrieve it after a correlated calculation, e.g. :

```
pam --get=CCDENS ...
```

and then copy it back for the property calculation e.g. :

```
pam --put=CCDENS ...
```

Predefined electric properties

.DIPOLE

.DIPOLE

Evaluate the electronic electric dipole moment

Expectation values: $\langle \hat{\mu}_{\alpha} \rangle = -e \langle r_{\alpha} \rangle$

Note: for charged molecules the total electric dipole moment will depend on the gauge origin. It is possible to set the nuclei charge barycenter as the gauge origin so that the computed (total) dipole magnitude is representative for the electronic system as the nuclei electric dipole moment is constrained to vanish. Check for USEBC under `*NMR`.

.QUADRUPOLE

.QUADRUPOLE

Evaluate the electronic traceless electric quadrupole moment

Expectation values: $\langle \Theta_{\alpha\beta} \rangle = -e \frac{3}{2} \langle r_{\alpha} r_{\beta} - \frac{1}{3} \delta_{\alpha\beta} r^2 \rangle$

.EFG

.EFG

Evaluate electric field gradients at nuclear positions, see Visscher_JCP1998 .

Electronic contribution to center K (expectation values) :

$$\phi_{\alpha\beta}^{[2]e}(\mathbf{R}_K) = \frac{-e}{4\pi\epsilon_0} \left\langle \frac{3r_{K\alpha}r_{K\beta} - \delta_{\alpha\beta}r_K^2}{r_K^5} \right\rangle$$

Nuclear contributions to center K :

$$\phi_{\alpha\beta}^{[2]nuc}(\mathbf{R}_K) = \sum_{A \neq K} \frac{Z_A e}{4\pi\epsilon_0} \left[\frac{3r_{KA\alpha}r_{KA\beta} - \delta_{\alpha\beta}r_{KA}^2}{r_{KA}^5} \right]$$

Results are also reported with respect to a principal axis system for each center.

Atomic centers may be restricted with INTEGRALS_.SELECT under `**INTEGRALS`.

.NQCC

.NQCC

Evaluate nuclear quadrupole coupling constants (NQCC) (expectation values). The NQCC is formally defined as

$$\frac{e^2 q Q}{h}$$

where Q is the electric quadrupole moment of the nucleus and $q = \phi_{zz}^{[2]} / e$ (in the principal axis system) is the field gradient. The NQCC may be extracted from experiment, whereas electronic structure calculations may provide the field gradient q . The two quantities are related as

$$\{NQCC \text{ [in MHz]}\} = 234.9647 \times \{Q \text{ [in b]}\} \times \{q \text{ [in atomic units]}\} \frac{E_h}{e a_0^2}$$

The calculations proceed similar to PROPERTIES_.EFG. The total electric field gradients for each center are transformed to a principal axis system for which

$|\psi^{[2]}_{zz}| \geq |\psi^{[2]}_{yy}| \geq |\psi^{[2]}_{xx}|$

DIRAC reports the more general expressions

$\text{NQC}_{\alpha\alpha} \text{ [in MHz]} = 234.9647 \times Q_{\text{b}} \times \psi^{[2]}_{\alpha\alpha} / e \text{ [in atomic units]}$

The asymmetry factor is defined as

$\eta = \frac{\psi^{[2]}_{xx} - \psi^{[2]}_{yy}}{\psi^{[2]}_{zz}}$

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

.POLARIZABILITY

.POLARIZABILITY

Evaluate the electronic dipole polarizability tensor, see Saue2003 (HF) and Salek2005 (DFT).

Linear response function: $\text{quad}\alpha_{\alpha\beta}(-\omega;\omega) = \langle \mu_{\alpha}; \hat{\mu}_{\beta} \rangle_{\omega}$

.FIRST ORDER HYPERPOLARIZABILITY

.FIRST ORDER HYPERPOLARIZABILITY

Evaluate static electronic dipole first-order hyperpolarizability tensor, see Norman_JCP2004 (HF) and Henriksson:2008 (DFT).

Quadratic response function: $\text{quad}\beta_{\alpha\beta\gamma}(-\omega_{\sigma};\omega_1,\omega_2) = \langle \mu_{\alpha}; \hat{\mu}_{\beta}, \hat{\mu}_{\gamma} \rangle_{\omega_1,\omega_2}$

Results are also given for the static electronic dipole polarizability.

.TWO-PHOTON

.TWO-PHOTON

Evaluate two-photon absorption cross sections Henriksson:2005, obtained as a first-order residue of the first-order hyperpolarizability. Give the number of desired states in each boson symmetry. Cannot be specified in combination with other quadratic response calculations.

Example: Point group with four boson irreps, (e.g. C_{2v})

.TWO-PHOTON
5 5 5 0

Predefined magnetic properties

.NMR

.NMR

Evaluate nuclear magnetic shieldings and indirect spin-spin couplings (linear response functions), see Visscher_jcc1999 and Ilias2009.

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

See below for advice on calculation of diamagnetic terms.

.SHIELDING

.SHIELDING

Evaluate nuclear magnetic shieldings (linear response), see Visscher_jcc1999 and Ilias2009 .

Elements of the shielding tensor for center K are given by

$\sigma_{K;\mu\nu} = \frac{\partial^2}{\partial m_{K;\mu} \partial B_{0;\nu}} \langle \hat{h}^{hfs}_{K;\mu\nu} \rangle$

where $\hat{h}$ appears the relativistic hyperfine operator

$\hat{h}^{hfs}_{K;\mu\nu} = -\sum_i \mathbf{m}_K \cdot \mathbf{B}^{el}_{K;\mu\nu}(i); \text{quad } \mathbf{B}^{el}_{K;\mu\nu}(i) = -\frac{1}{4\pi\epsilon_0} \frac{\partial^2}{\partial x_i \partial x_j} \frac{1}{r_{iK}}$

expressed in terms of the nuclear magnetic dipole $\mathbf{m}_K$ and the operator $\mathbf{B}^{el}_{K;\mu\nu}$ giving the magnetic field due to the electrons at the nuclear position, and the relativistic Zeeman operator

$\hat{h}^Z = -\mathbf{m}_K \cdot \mathbf{B}^{el}_{K;\mu\nu}; \text{quad } \mathbf{B}^{el}_{K;\mu\nu} = -\sum_i \frac{e}{r_{iK}^3} (\mathbf{r}_i \times \mathbf{G}_i)$,

expressed in terms of the operator $\mathbf{B}^{el}_{K;\mu\nu}$ associated with the magnetic dipole moment of the electrons and the external magnetic field $\mathbf{B}_0$. Note reference to the gauge origin $\mathbf{G}$.

Note that PROPERTIES_.PRINT 2 gives the full tensor and longer output. The PROPERTIES_.PRINT 4 gives the raw values in symmetry coordinates as well.

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

.MAGNET

.MAGNET

Evaluate the (static) magnetizability tensor Ilias2013

Linear response function

$$\chi_{\{K;\alpha\beta\}} = - \frac{\partial^2}{\partial B_{\{\alpha\}} \partial B_{\{\beta\}}} \langle \hat{h} \rangle^Z; \hat{h}^Z \rangle_{\text{range}_0}$$

where appears the relativistic Zeeman operator

$$\hat{h}^Z = -\hat{\mathbf{m}}_e \cdot \mathbf{B}_0; \quad \hat{\mathbf{m}}_e = -\sum_i \frac{e}{2} (\mathbf{r}_i \times \mathbf{c} \boldsymbol{\alpha}(i)),$$

expressed in terms of the operator $\hat{\mathbf{m}}_e$ associated with the magnetic dipole moment of the electrons and the external magnetic field $\mathbf{B}_0$. Note reference to the gauge origin $\mathbf{G}$.

.ROTG

.ROTG

Evaluate rotational g-tensors: linear response and nuclear contributions, see Aucar_JCP2014.

Elements of the rotational g-tensor are given by

$$g_{\mu\nu} = g^{\text{nuc}}_{\mu\nu} + g^{\text{elec}}_{\mu\nu}$$

with

$$g^{\text{elec}}_{\mu\nu} = - \frac{m_p}{e} \frac{\partial^2}{\partial L_{\mu} \partial B_{\nu}} \langle \hat{h} \rangle^{B0}; \hat{h}^Z \rangle_{\text{range}_0}$$

where appears the first order correction to the Born-Oppenheimer (BO) approximation

$$\hat{h}^{B0} = \boldsymbol{\omega} \cdot \mathbf{J}_e; \quad \boldsymbol{\omega} = \mathbf{L} \cdot \mathbf{I}^{-1},$$

expressed in terms of the angular velocity $\boldsymbol{\omega}$ associated with the total angular momentum of the electrons $\mathbf{J}_e = \mathbf{L}_e + \mathbf{S}_e$, and the relativistic Zeeman operator

$$\hat{h}^Z = -\hat{\mathbf{m}}_e \cdot \mathbf{B}_0; \quad \hat{\mathbf{m}}_e = -\sum_i \frac{e}{2} (\mathbf{r}_i \times \mathbf{c} \boldsymbol{\alpha}(i)),$$

expressed in terms of the operator $\hat{\mathbf{m}}_e$ associated with the magnetic dipole moment of the electrons and the external magnetic field $\mathbf{B}_0$. Note reference to the gauge origin $\mathbf{G}$.

Results are dimensionless.

The total g-tensor, as well as its linear response and nuclear contributions are always given separately.

Using PROPERTIES_.PRINT 1, the paramagnetic (e-e) and diamagnetic (e-p) parts of the linear response contributions are given separately, together with results for the $\mathbf{L}$ and $\mathbf{S}$ parts of the linear response.

.SPIN-SPIN COUPLING

.SPIN-SPIN COUPLING

Evaluate indirect spin-spin couplings Visscher_jcc1999. The indirect spin-spin tensor $J_{\{KL\}}$ associated with nuclei K and L may be expressed as

$$J_{\{KL\}} = \frac{\hbar^2}{2} \gamma_K \gamma_L K_{\{KL\}}$$

where appears gyromagnetic ratios γ_K . The elements of the reduced tensor $K_{\{KL\}}$ are expressed in terms of linear response functions as

$$K_{\{KL;\mu\nu\}} = \frac{\partial^2}{\partial m_{\{K;\mu\}} \partial m_{\{L;\nu\}}} \langle \hat{h} \rangle_{\{K\}^{\text{hfs}}}; \langle \hat{h} \rangle_{\{L\}^{\text{hfs}}} \rangle_{\text{range}_0}$$

where appears the relativistic hyperfine operator

$$\hat{h}^{\text{hfs}}_{\{K\}} = -\sum_i \mathbf{m}_{\{K\}} \cdot \mathbf{B}_i^{\text{el}}_{\{K\}}(i); \quad \mathbf{B}_i^{\text{el}}_{\{K\}}(i) = -\frac{1}{4\pi\epsilon_0} \frac{\mathbf{r}_i}{r_i^3} \times \mathbf{c} \boldsymbol{\alpha}(i)$$

expressed in terms of the nuclear magnetic dipole $\mathbf{m}_{\{K\}}$ and the operator $\mathbf{B}_i^{\text{el}}_{\{K\}}$ giving the magnetic field due to the electrons at the nuclear position.

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

The default is to calculate the diamagnetic term via occupied positive energy to virtual negative energy orbital rotations (also called electron-positron rotations), see Aucar1999 for the theory. The quality of this is very basis set dependent. It is generally more accurate to use the non-relativistic expectation value expression for the diamagnetic term, activated with keyword .DSO in this section. You must also add LINEAR_RESPONSE_.SKIPEP under *LINEAR_RESPONSE to exclude the diamagnetic term from the linear response calculation.

.DSO

.DSO

Evaluate the diamagnetic contribution to indirect spin-spin couplings as an expectation value of the non-relativistic DSO operator.

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

.NSTDIAMAGNETIC

.NSTDIAMAGNETIC

Evaluate the diamagnetic contribution to nuclear magnetic shielding tensor as an expectation value of the non-relativistic operator.

Atomic centers may be restricted with INTEGRALS_. SELECT under **INTEGRALS.

.ESR

.ESR

Evaluate ESR parameters – g-tensors and hyperfine coupling tensors – using first-order quasi-degenerate perturbation theory based on configuration interaction. No default, it is required to specify the actual calculation under *ESR.

Mixed electric and magnetic properties

.OPTROT

.OPTROT

Calculate optical rotation. The most common experimental setup uses light with a frequency corresponding to the sodium D-line (589.29 nm). The optical rotation is reported as the number of degrees of rotation of the plane of polarization per mole of sample for a sample cell of length 1~dm, and at a temperature of 25°C

$$\left[\alpha\right]_{\text{D}^{25}} = -288 \cdot 10^{-30} \cdot \frac{\pi^2 \text{N} a_0^4 \omega}{3M} \sum_{\alpha} G'_{\alpha} G'_{\alpha} ; \quad G'_{\alpha}$$

where M is the molecular mass in g mol^{-1} and N is the number density.

.VERDET

.VERDET

Evaluate Verdet constants Ekstrom2005 for a dynamic electric field corresponding to Ruby laser wavelength of 694 nm and a static magnetic field along the propagation direction of the light beam (in this case, the default frequencies of the quadratic response function thus become $\omega_B = 0.0656$ and $\omega_C = 0.0$).

The Verdet constant is given in terms of quadratic response functions

$$V(\omega) = \epsilon_{\alpha\beta\gamma} \text{Im} \langle \mu_{\alpha}; \hat{\mu}_{\beta}, \hat{m}_{\gamma} \rangle_{\omega}$$

where $C = eN / (24c_0 \epsilon_0 e)$ and N is the number density of the gas. A Verdet calculation cannot be specified in combination with other quadratic response calculations.

The frequencies can be changed using QUADRATIC_RESPONSE_.B FREQ in *QUADRATIC RESPONSE.

Other predefined properties

.MOLGRD

.MOLGRD

Evaluate the molecular gradient, i.e.

$$\frac{\partial E}{\partial \mathbf{X}_A}$$

where $\mathbf{X}_A$ are the coordinates of the nuclei. This is an expectation value of one- and two-electron operators. Normally the molecular gradient evaluation is not invoked explicitly with this keyword but rather implicitly in the geometry optimization module.

.PVC

.PVC

Calculate matrix elements over the nuclear spin-independent parity-violating operator, e.g. calculate energy differences between enantiomers, see Laerdahl1999 and Bast2011.

The parity-violating energy is calculated as the expectation value

$$E_{\text{PV}} = \sum_A \langle H_{\text{PV}} \rangle_A = \frac{G_F}{\sqrt{2}} \sum_i \gamma_5(i) \rho_A$$

where ρ_A is the nuclear charge density of nucleus A normalized to unity, and G_F is the Fermi coupling constant. The weak nuclear charge

$$Q_A = Z_{\text{AC}} - N_{\text{AC}} = Z_A (1 - 4 \sin^2 \theta_W) - N_A$$

is given in terms of both the number of protons – Z_A – and N_A – in the nucleus A and the Weinberg angle θ_W , which describes the rotation of B^0 and W^0 bosons by spontaneous symmetry breaking to form photons and Z^0 bosons.

Using GENERAL_.CODATA, G_F can be chosen according to the data reported in different CODATA sets. In the particular case in which .PDG94 is used under GENERAL_.CODATA, $G_F = 2.22255 \times 10^{-14} \text{E}_{\text{ha}_0^3}$. In addition, DIRAC uses $\sin^2 \theta_W = 0.2319$ when .PDG94 is employed under GENERAL_.CODATA. In other cases, the value reported in the latest available CODATA set is used.

.PVCSHI

.PVCSHI

Calculate parity-violating contribution to the NMR shielding tensor, see Barra1988 and Bast:2006. Elements of the parity-violating contribution to the shielding tensor for center '\$K\$' are given by

$$\sigma^{\text{PV}}_{\{K;\mu\nu\}} = \frac{\partial^2}{\partial m_{\{K;\mu\}} \partial B_{\{0;\nu\}}} \langle \hat{H}^{\text{PV2}}_{\{K\}} | \hat{H}^Z \rangle_0$$

where the nuclear spin-dependent parity-violating operator is given as

$$\hat{H}^{\text{PV2}}_{\{K\}} = -\frac{G_F}{2} (1 - 4 \sin^2 \theta_W) \frac{1}{\sqrt{2}} \sum_i \frac{1}{\gamma_K} \boldsymbol{\alpha} \cdot \mathbf{m}$$

where '\$G_F\$' is the Fermi coupling constant, '\$\theta_W\$' is the Weinberg angle, '\$\rho^A\$' is the nuclear charge density of nucleus '\$A\$' normalized to unity, '\$\gamma_K\$' is the gyromagnetic ratio, and '\$\mathbf{m}_K = \gamma_K \mathbf{I}_K\$' is the nuclear magnetic dipole moment.

The relativistic Zeeman operator

$$\hat{H}^Z = -\hat{\mathbf{m}} \cdot \mathbf{B}_0; \quad \frac{1}{\gamma_K} = \sum_i \frac{e}{2} \langle \mathbf{r}_i | \mathbf{G} | \mathbf{r}_i \rangle \times c \boldsymbol{\alpha}(i),$$

is expressed in terms of the operator '\$\hat{\mathbf{m}} \cdot \mathbf{B}_0\$' associated with the magnetic dipole moment of the electrons and the external magnetic field '\$\mathbf{B}_0\$'. Note reference to the gauge origin '\$G\$'.

Results are given in ppm.

Using GENERAL_.CODATA, '\$G_F\$' can be chosen according to the data reported in different CODATA sets. In the particular case in which .PDG94 is used under GENERAL_.CODATA, '\$G_F = 2.2225 \times 10^{-14} \text{E}_{\text{ha}_0^3}\$'. In addition, DIRAC uses '\$\sin^2 \theta_W = 0.2319\$' when .PDG94 is employed under GENERAL_.CODATA. In other cases, the value reported in the latest available CODATA set is used.

Using PROPERTIES_.PRINT 1, the paramagnetic-like (e-e) and diamagnetic-like (e-p) parts of the linear response are given separately.

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

.PVCSSR

.PVCSR

Calculate parity-violating contribution to the nuclear spin-rotation tensor, see Aucar_JCP2021. Elements of the parity-violating contribution to the nuclear spin-rotation tensor for center '\$K\$' are given by

$$M^{\text{PV}}_{\{K;\mu\nu\}} = -\frac{\partial^2}{\partial I_{\{K;\mu\}} \partial L_{\{\nu\}}} \langle \hat{H}^{\text{PV2}}_{\{K\}} | \hat{H}^{\text{B0}} \rangle$$

where the nuclear spin-dependent parity-violating operator is given as

$$\hat{H}^{\text{PV2}}_{\{K\}} = -\frac{G_F}{2} (1 - 4 \sin^2 \theta_W) \frac{1}{\sqrt{2}} \sum_i \frac{1}{\gamma_K} c \boldsymbol{\alpha} \cdot \mathbf{I}_K$$

where '\$G_F\$' is the Fermi coupling constant, '\$\theta_W\$' is the Weinberg angle, '\$\rho^A\$' is the nuclear charge density of nucleus '\$A\$' normalized to unity, and '\$I_K\$' is the nuclear spin of nucleus '\$K\$'.

The first order correction to the Born-Oppenheimer (BO) approximation

$$\hat{H}^{\text{B0}} = -\boldsymbol{\omega} \cdot \hat{\mathbf{J}}_e; \quad \boldsymbol{\omega} = \mathbf{L} \cdot \hat{\mathbf{I}}^{-1},$$

is expressed in terms of the angular velocity '\$\boldsymbol{\omega}\$' associated with the total angular momentum of the electrons '\$\hat{\mathbf{J}}_e = \hat{\mathbf{L}}_e + \hat{\mathbf{S}}_e\$'. Note that the origin of the orbital angular momentum is the molecular center of mass.

Results are given in Hz.

Using GENERAL_.CODATA, '\$G_F\$' can be chosen according to the data reported in different CODATA sets. In the particular case in which .PDG94 is used under GENERAL_.CODATA, '\$G_F = 2.2225 \times 10^{-14} \text{E}_{\text{ha}_0^3}\$'. In addition, DIRAC uses '\$\sin^2 \theta_W = 0.2319\$' when .PDG94 is employed under GENERAL_.CODATA. In other cases, the value reported in the latest available CODATA set is used.

Using PROPERTIES_.PRINT 1, the paramagnetic-like (e-e) and diamagnetic-like (e-p) parts of the linear response are given separately, together with results for the '\$\mathbf{L}\$' and '\$\mathbf{S}\$' parts of the linear response.

For more details, the user is welcome to see the Tutorial Section.

Atomic centers may be restricted with INTEGRALS_.SELECT under **INTEGRALS.

.PVCEFG

.PVCEFG

Calculate parity-violating contribution to the electric field gradient tensor, see Aucar2025_PVEFG. Elements of the parity-violating contribution to the EFG tensor for center '\$K\$' are given by

$$q^{\text{PV}}_{\{ij\}}(\mathbf{R}_K) = \langle \hat{H}^{\text{PV}}_{\{K\}} | q_{\{ij\}}(\mathbf{R}_K) \rangle; \quad \hat{H}^{\text{PV}}_{\{K\}} \rangle_0$$

where the contribution from the \$K\$ nucleus to the nuclear spin-independent parity-violating operator

$$\hat{H}^{\text{PV}}_{\{K\}} = \frac{G_F}{2\sqrt{2}} \sum_{i,K} Q_{\{w,K\}} \gamma_i^5 \rho_K(\mathbf{r}_i)$$

is expressed in terms of the Fermi coupling constant '\$G_F = 2.222516 \times 10^{-14} \text{E}_{\text{ha}_0^3}\$' (this value corresponds to CODATA 2022, but it can change if you use other CODATA set), the weak nuclear charge '\$Q_{\{w,K\}}\$' and the normalized nuclear charge density '\$\rho_K\$' (in units of the inverse of cube distances). Results are given in a.u.

Using `PROPERTIES_.PRINT 1`, the paramagnetic-like (e-e) and diamagnetic-like (e-p) parts of the linear response are given separately.

Atomic centers may be restricted with `INTEGRALS_.SELECT` under `**INTEGRALS`.

`.RHONUC`

.RHONUC

Calculate electronic density at the nuclear positions, also known as the contact density (see Knecht2011 and Almoukhalalati:2016b).

It is formally the expectation value

$$\langle \rho_e^K = -e \int \delta^3(\mathbf{r} - \mathbf{R}_K) \rangle$$

An important observation: In view of picture change effects, see Knecht2011.

Atomic centers may be restricted with `INTEGRALS_.SELECT` under `**INTEGRALS`.

`.EFFDEN`

.EFFDEN

Calculate effective electronic density associated with nuclei, see Knecht2011. This quantity appears in expressions for the Mössbauer isomer shift. Starting from the electrostatic electron-nucleus interaction

$$E^{\text{el}}(R) = \int \rho_e(\mathbf{r}) \phi_n(\mathbf{r}; R) d^3 \mathbf{r},$$

we consider the change in the electrostatic energy upon a change of nuclear radius. If we ignore any change in the electronic density ρ_e , we may express this as

$$\Delta E_{\text{el}} = \left. \frac{\partial E^{\text{el}}}{\partial R} \right|_{R=R_0} \Delta R = \int \rho_e(\mathbf{r}) \frac{\partial \phi_n(\mathbf{r})}{\partial R} d^3 \mathbf{r}$$

where the effective density $\bar{\rho}_e$ is introduced in the last step. It is often approximated by the contact density, see `PROPERTIES_.RHONUC`, but this is discouraged since it may introduce errors on the order of 10%.

Atomic centers may be restricted with `INTEGRALS_.SELECT` under `**INTEGRALS`.

`.SPIN-ROTATION`

.SPIN-ROTATION

Evaluate nuclear spin-rotation constants: linear response, expectation value and nuclear contributions, see Aucar_JCP2012 and AucarChap2019.

Elements of the nuclear spin-rotation tensor for center K in a molecule in equilibrium are given by

$$M_{K;\mu\nu} = M^{\text{nuc}}_{K;\mu\nu} + M^{\text{elec}}_{K;\mu\nu}$$

with

$$M^{\text{elec}}_{K;\mu\nu} = - \frac{\hbar^2}{2} \frac{\partial^2}{\partial I_K \partial L_\nu} \langle \hat{h}^{\text{hfs}}_K; \hat{h}^{\text{B0}} \rangle$$

where appears the relativistic hyperfine operator

$$\hat{h}^{\text{hfs}}_K = - \sum_i \mathbf{B}_K \cdot \hat{\mathbf{B}}^{\text{el}}_K(i); \quad \hat{\mathbf{B}}^{\text{el}}_K(i) = - \frac{1}{4\pi\epsilon_0 c^2} \nabla \times \mathbf{A}^{\text{el}}_K(i)$$

expressed in terms of the nuclear magnetic dipole $\mathbf{B}_K = \gamma_K \mathbf{I}_K$ and the operator $\hat{\mathbf{B}}^{\text{el}}_K$ giving the magnetic field due to the electrons at the nuclear position, and the first order correction to the Born-Oppenheimer (BO) approximation

$$\hat{h}^{\text{B0}} = - \boldsymbol{\omega} \cdot \hat{\mathbf{J}}_e; \quad \boldsymbol{\omega} = \mathbf{L} \cdot \mathbf{I}^{-1},$$

expressed in terms of the angular velocity $\boldsymbol{\omega}$ associated with the total angular momentum of the electrons $\hat{\mathbf{J}}_e = \hat{\mathbf{L}} + \hat{\mathbf{S}}$. Note that the origin of the orbital angular momentum is the molecular center of mass.

[!NOTE] The current implementation gives by default (`PROPERTIES_.PRINT` values up to 3) results only for molecules in equilibrium.

Results are given in kHz, but for particular `PROPERTIES_.PRINT` values they could also be given in ppm (to compare results with `PROPERTIES_.SHIELDING`).

The total spin-rotation tensors, as well as their electronic (linear response) and nuclear contributions are always given separately.

Using `PROPERTIES_.PRINT 0` or `1`, results are only given in kHz, whereas employing `PROPERTIES_.PRINT 2` or `3` they are also shown in ppm.

In addition, when `PROPERTIES_.PRINT 1` or `3` are used, the paramagnetic-like (e-e) and diamagnetic-like (e-p) parts of the linear response contributions are given separately, together with results for the $\mathbf{L}$ and $\mathbf{S}$ parts of the linear response.

Finally, employing `PROPERTIES_.PRINT 4` expectation value and nuclear contributions are given separately, with the inclusion of Thomas precession effects, in order to properly include contributions to nuclear spin-rotations out of the equilibrium geometry of the molecular system.

For more details, the user is welcome to see the Tutorial Section.

Atomic centers may be restricted with `INTEGRALS_.SELECT` under `**INTEGRALS`.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/esr.md

orphan

*ESR

*ESR

This section gives directives for the calculation of ESR/EPR parameters, g-tensors and Hyperfine Coupling tensors (HFC).

General control statements

.PRINT**.PRINT**

Print level.

Default:

```
.PRINT
0
```

Definition of the multiplicity and CI

.MULTIP**.MULTIP**

```
.MULTIP
2
```

Default: Doublet for an odd number of electrons, triplet for an even number of electrons.

.CI ROOTS**.CI ROOTS**

```
.CI ROOTS
5
```

Default: The multiplicity.

.ESR CI**.ESR CI**

General specification of the ESR CI space with GAS, generalized active spaces. For easier description they are divided in three groups: RAS1-GAS spaces, defining max number of holes in the doubly occupied orbitals in the reference wave function; RAS2-GAS, one “CAS” space with the open shells from the reference wave functions; RAS3-GAS spaces, defining max number of electrons in the empty (virtual) orbitals in the reference wave function.

Default: “RESOLVE”, that is, CI in the open shell orbitals from the reference wave function.

Example: S, SD, and SDT from the occupied orbitals, S and SD to the empty orbitals. The RAS2-GAS space is defined implicitly by the reference wave function, typically and average-of-configurations (AOC) Hartree-Fock. The example is for a case with inversion symmetry, if no inversion symmetry there is only one set orbitals in each GAS space.

```
.ESR CI
3 2      ! 3 RAS1-GAS spaces and 2 RAS3-GAS spaces
1        ! max 1 hole in first RAS1-GAS space
2 2      ! 2 g and 2 u orbitals in RAS1-GAS
2        ! max 2 holes in second RAS1-GAS space
4 2      ! 4 g and 2 u orbitals in RAS1-GAS
3        ! max 3 holes in third and last RAS1-GAS space
4 1      ! 4 g and 1 u orbitals in RAS1-GAS
2        ! max 2 electrons in first RAS3-GAS space
4 1      ! 4 g and 1 u orbitals in RAS3-GAS
1        ! max 1 electron in second and last RAS3-GAS space
4 1      ! 4 g and 1 u orbitals in RAS3-GAS
```

.THRCI**.THRCI**

```
.THRCI
1,d-5
```

Default: 1.d-4 for ppt accuracy. Can also be modified with .G10PPM keyword.

.G10PPM**.G10PPM**

Converge CI for 10 ppm accuracy in g-tensor values, default is ppt accuracy.

.STATE

.STATE

The first CI root belonging to the multiplet of quasi-degenerate states. For example, if the ground state is a non-degenerate singlet state (non-relativistically a spin singlet) followed by the triplet state you want to study, then you should specify state no. 2 here.

Default:

.STATE
1

.KR-CON

.KR-CON

Use Kramers conjugation for odd-number of electrons. For a doublet state this means the program only needs to converge one CI state instead of two CI states - reduces the CI time with a factor 2.

Definition of additional properties

The g-tensor and HFC-tensors for all selected nuclei are always calculated.

.SELECT

.SELECT

Select which nuclei to calculate HFC for. Default is *all*, which can be quite time-consuming if the molecule has many atoms. (You cannot use INTEGRALS_.SELECT under **INTEGRALS if you use this keyword, they do the same thing. It is included here for convenience.) To select three nuclei, no. 3, 7, and 8:

.SELECT
3
3 7 8

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/excitation.md

orphan

*EXCITATION ENERGIES

*EXCITATION ENERGIES

Calculate excitation energies using time dependent Hartree-Fock or DFT. The excitation energies are found as the lowest generalized eigenvalues of the electronic Hessian. DIRAC supports TDDFT kernels from all ground state functionals included in the code. Currently the iterative eigenvalue solver may fail to converge more than about twenty roots per symmetry.

Define excitations and transition moments

.EXCITA

.EXCITA

.EXCITA
SYM N

Number of excitation energies N calculated in boson symmetry no. SYM. This keyword can be repeated if you want excitation energies in more than one boson symmetry.

.OPERATOR

.OPERATOR

Specification of a transition moment operator (see `one_electron_operators` for details). This keyword can be given multiple times to add more operators.

.EPOLE

.EPOLE

Specification of electric Cartesian multipole operators of order \$n`\$

$$\hat{Q}_{\{j_1\}\ldots j_n\}^{\left[n\right]} = -e r_{\{1\}} r_{\{2\}} \ldots r_{\{j_n\}}$$

for the calculation of transition moments (note that they contribute to one order less in the wave vector). Specify order.

Example: Electric dipole operators:

.EPOLE
1

.MPOLE

.MPOLE

Specification of magnetic Cartesian multipole operators of order n

$$\hat{m}_{j_1 \dots j_{n-1}; j_n}^{\leftarrow n} = \frac{n}{n+1} r_{j_1} r_{j_2} \dots r_{j_{n-1}} (\mathbf{r} \times \hat{\mathbf{m}}_j)_n$$

for the calculation of transition moments (note that they contribute to the same order in the wave vector). Specify order.

Example: Magnetic dipole operators:

.MPOLE
1

.VPOLE

.VPOLE

Specification of electric Cartesian multipole operators of order n in the velocity representation

$$\hat{\mathcal{Q}}_{j_1 \dots j_{n-1}; j_n}^{(n)} = \frac{ie}{\omega} r_{j_1} \dots r_{j_{n-2}} (c \alpha_{j_n} r_{j_{n-1}} + n \alpha_{j_{n-1}} r_{j_n})$$

Example: Velocity representation electric dipole operator:

.VPOLE
1

.ANALYZE

.ANALYZE

Analyze solution vectors and show the most important excitations at the orbital level.

.INTENS

.INTENS

Invoke calculation of oscillator strengths to order k in the wave vector. Default order is zero, which corresponds to the widely used electric-dipole approximation. By default results are given both in the length and the velocity representation, but a selection can be made using keywords EXCITATION_ENERGIES_.NOLENR and EXCITATION_ENERGIES_.NOVELR. Only even orders contribute. For further details, see List_JCP2020

Example: :

.INTENS
2

.BED

.BED

Invoke calculation of oscillator strengths using the full operator coupling the molecule to an electromagnetic plane wave. The oscillator strength is then given by

$$f_{n \leftarrow 0} = \frac{2\omega}{\hbar} e^2 \left| \left\langle n \left| \left(\frac{\omega}{c} \right) \cdot \mathbf{r} \right| 0 \right\rangle \right|^2$$

where appears the effective interaction operator

$$T(\omega) = \frac{e}{\omega} \left(\boldsymbol{\alpha} \cdot \boldsymbol{\epsilon} \right) e^{+i \left(\mathbf{k} \cdot \mathbf{r} \right)}$$

In the above expression $\mathbf{k}$ refers to the wave vector of length $k = \omega/c$ and direction $\mathbf{m}$, whereas the polarization of the electric component is specified by $\boldsymbol{\epsilon}$.

Since experiment is typically carried out in an isotropic medium, rotational average is performed. Rather than rotate the molecule we shall rotate the experimental apparatus. To rotate we use the unit vectors of the spherical coordinates

$$\begin{aligned} \begin{array}{l} \mathbf{e}_r = \sin\theta \cos\phi \mathbf{e}_x + \sin\theta \sin\phi \mathbf{e}_y + \cos\theta \mathbf{e}_z \\ \mathbf{e}_\theta = \cos\theta \cos\phi \mathbf{e}_x + \cos\theta \sin\phi \mathbf{e}_y - \sin\theta \mathbf{e}_z \\ \mathbf{e}_\phi = -\sin\theta \cos\phi \mathbf{e}_x - \sin\theta \sin\phi \mathbf{e}_y + \cos\theta \mathbf{e}_z \end{array} \end{aligned}$$

We now choose to align the wave vector with the $\mathbf{e}_r$ unit vector, that is

$$\mathbf{k} = k \mathbf{e}_r$$

This means that the polarization vector $\boldsymbol{\epsilon}$ is in the plane spanned by the unit vectors $\mathbf{e}_\theta$ and $\mathbf{e}_\phi$. We therefore set

$$\boldsymbol{\epsilon} = \cos\chi \mathbf{e}_\theta + \sin\chi \mathbf{e}_\phi$$

We see the solid angle $d\Omega = \sin\theta d\theta d\phi$ gives all possible directions of the wave vector $\mathbf{k}$, whereas the angle χ provides all possible orientations of the polarization vector $\boldsymbol{\epsilon}$ in the plane perpendicular to $\mathbf{k}$.

The general expression for the rotational average will be

$$\left\langle f \left(\boldsymbol{r} \right) \right\rangle_{\theta, \phi, \chi} = \frac{1}{8\pi^2} \int_0^{2\pi} \int_0^{2\pi} \int_0^\pi f \, d\chi \, d\phi \, d\theta$$

In our case we have

$$\left\langle f_{\boldsymbol{n}} \right\rangle_{\theta, \phi, \chi} = \frac{2\omega}{\hbar e^2} \left\langle \boldsymbol{\epsilon}_{\alpha} \boldsymbol{\epsilon}_{\beta} \right\rangle$$

which simplifies to

$$\left\langle f_{\boldsymbol{n}} \right\rangle_{\theta, \phi, \chi} = \frac{2\omega}{\hbar e^2} \left\langle \boldsymbol{\epsilon}_{\alpha} \boldsymbol{\epsilon}_{\beta} \right\rangle$$

since only the polarization vectors depend on the angle χ . The average over the angle χ can be expressed compactly in terms of the wave unit vector $\boldsymbol{m}$ ($\boldsymbol{k} = k\boldsymbol{m}$)

$$\left\langle \boldsymbol{\epsilon}_{\alpha} \boldsymbol{\epsilon}_{\beta} \right\rangle_{\chi} = \frac{1}{2} \left(\delta_{\alpha\beta} - m_{\alpha} m_{\beta} \right),$$

whereas the average over angles θ and ϕ is handled by [Lebedev quadrature](#).

A full account is given in List_JCP2020

.BEDECD

.BEDECD

Invoke calculation of the differential oscillator strengths using the full interaction operator. This quantity is calculated as the difference between the oscillator strengths corresponding to left- and right-handed circularly polarized light. The differential oscillator strength is given by

$$\Delta f = f_L - f_R = -i \frac{2m_e \omega}{\hbar e^2} \boldsymbol{e}_k \cdot \left(\boldsymbol{T} \times \boldsymbol{T}^* \right),$$

where the transition moments are written in vector form, $\boldsymbol{T} = \langle \boldsymbol{e} | \boldsymbol{\epsilon}_{\alpha} \boldsymbol{e}^{\dagger} \boldsymbol{k} \cdot \boldsymbol{r} | \rangle$, which allows the compact cross-product form in previous equation.

Isotropic averaging of the differential oscillator strength is handled in an analogous manner as EXCITATION_ENERGIES_.BED, with the main difference being that in the current case, only two angles are required. The redundancy of the angle χ is a manifestation of axial symmetry. The rotational average can be written as

$$\left\langle \Delta f_{\boldsymbol{n}} \right\rangle_{\theta, \phi} = -i \frac{2m_e \omega}{\hbar e^2} \left\langle \boldsymbol{n} \cdot \boldsymbol{e}_k \right\rangle$$

For a full account see vanHorn2021probing.

.ANGPLOT

.ANGPLOT

Print the anisotropic differential oscillator strength (see. EXCITATION_ENERGIES_.BEDECD) for every point on the Lebedev grid. These points can be used to plot the differential oscillator strength as a function of the incident angle of the radiation. A full account is given in vanHorn2021probing.

.ORIENT

.ORIENT

Specify fixed experimental configuration (no rotational average). The orientation of the wave and polarization vector is given by specification of the angles θ , ϕ and χ , see the EXCITATION_ENERGIES_.BED keyword for more details. For instance, to specify that the wave vector is along the z -axis and the polarization vector along the x -axis, we set

```
.ORIENT
0.0 0.0 0.0
```

.NROTAV

.NROTAV

As described under the .BED keyword, [Lebedev quadrature](#) is employed for rotational average. This quadrature over the solid angles can integrate a spherical harmonic to high accuracy with a maximum angular momentum L_{max} . The default value of L_{max} is presently 5, but can be reset with this keyword.

.BEDCON

.BEDCON

Specification of contributions of the full light-matter interaction of order n in the wave vector

$$\hat{T}_{\text{full}}(\omega) = \frac{k^n}{n!} \frac{d^n}{dk^n} \left[\frac{\boldsymbol{\epsilon}}{\omega} \left(\boldsymbol{\epsilon}_{\alpha} \cdot \boldsymbol{\epsilon}_{\beta} \right) e^{+i \left(\boldsymbol{k} \cdot \boldsymbol{r} \right)} \right]_{k=0} = \frac{\boldsymbol{\epsilon}}{\omega} \frac{i^n}{n!} \left(\boldsymbol{\epsilon}_{\alpha} \cdot \boldsymbol{\epsilon}_{\beta} \right) \left(\boldsymbol{k} \cdot \boldsymbol{r} \right)^n$$

.NOLENR

.NOLENR

Deactivate length representation.

.NOVELR

.NOVELR

Deactivate velocity representation.

Control variational parameters

.OCCUP

.OCCUP

For each fermion ircop give an orbital_strings of inactive orbitals from which excitations are allowed. By default excitations from all occupied orbitals are included in the generalized eigenvalue problem.

Example: :

```
.OCCUP
1..3
7,8
```

This would include excitations from gerade orbitals 1,2,3, and ungerade orbitals 7 and 8.

.VIRTUA

.VIRTUA

For each fermion ircop give an orbital_strings of virtual orbitals to which excitations are allowed. By default excitations to all virtual orbitals are included in the generalized eigenvalue problem.

.SKIPEE

.SKIPEE

Exclude all rotations between occupied positive-energy and virtual positive-energy orbitals.

.SKIPEP

.SKIPEP

Exclude all rotations between occupied positive-energy and virtual negative-energy orbitals.

Control reduced equations

.MAXITR

.MAXITR

Maximum number of iterations.

Default: :

```
.MAXITR
30
```

.MAXRED

.MAXRED

Maximum dimension of matrix in reduced system.

Default: :

```
.MAXRED
200
```

.THRESH

.THRESH

Threshold for convergence of reduced system.

Default: :

```
.THRESH
1.0D-5
```

Control integral contributions

The user is encouraged to experiment with these options since they may have an important effect on run time.

.INTFLG

.INTFLG

Specify what two-electron integrals to include (default: HAMILTONIAN_ . INTFLG under **HAMILTONIAN).

.CNVINT

.CNVINT

Set threshold for convergence before adding SL and SS integrals to SCF-iterations.

2 (real) Arguments: :

```
.CNVINT
CNVXQR(1) CNVXQR(2)
```

Default: Very large numbers.

.ITRINT

.ITRINT

Set the number of iterations before adding SL and SS integrals to SCF-iterations.

Default: :

```
.ITRINT
1 1
```

Advanced/debug flags

.E2CHEK

.E2CHEK

Generate a complete set of trial vector which implicitly allows the explicit construction of the electronic Hessian. Only to be used for small systems !

.ONLYSF

.ONLYSF

Only call FMOLI in sigmavector routine: only generate one-index transformed Fock matrix Saue2003.

.ONLYSG

.ONLYSG

Only call FMOLI in sigmavector routine: 2-electron Fock matrices using one-index transformed densities Saue2003.

.GNOISE

.GNOISE

To test the robustness of property gradients to numerical noise artificial noise is added to the MO-coefficients. More precisely, the user activates noise and provides a “noise level”. Then, for each element of the coefficient array, a pseudo-random number in the interval (-1,+1] is selected, multiplied with the “noise level” and added to the element.

Example: :

```
.GNOISE
1.0D-10
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/expectation.md

orphan

*EXPECTATION VALUES

***EXPECTATION VALUES**

This section gives directives for the calculation of expectation values.

.OPERATOR

.OPERATOR

Specification of general one-electron operators.

See the one_electron_operators section for more information and explicit examples.

Advanced options**.ORBANA****.ORBANA**

Analyze contributions to an expectation value from the individual orbitals.

.PROJEC**.PROJEC**

Decompose an expectation value in contribution from fragment orbitals. This action presupposes that a projection analysis has been performed, see *PROJECTION section for details.

Programmers options**.PRINT****.PRINT**

Print level.

Default:

```
.PRINT
0
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/linear_response.md

orphan

*LINEAR RESPONSE

***LINEAR RESPONSE**

Linear Response module written by T. Saue and H. J. Aa. Jensen Saue2003.

General control statements**.PRINT****.PRINT**

Print level.

Default:

```
.PRINT
0
```

Definition of the linear response function**.A OPERATOR****.A OPERATOR**

Specification of the A operator (see one_electron_operators for details).

.B OPERATOR**.B OPERATOR**

Specification of the B operator (see one_electron_operators for details).

.OPERATORS**.OPERATORS**

Specification of both the A and B operators (see one_electron_operators for details).

.EPOLE

.EPOLE

Specification of electric Cartesian multipole operators of order L . Specify order.

Example: Electric dipole operators

```
::
.EPOLE 1

.MPOLE
```

.MPOLE

Specification of magnetic Cartesian multipole operators of order L . Specify order. *Example:* Magnetic dipole operators

```
::
.MPOLE 1

.TRIAB
```

.TRIAB

Enforce triangularity of response function.

$\langle A; B \rangle_{\omega} = \langle B; A \rangle_{\omega}$

Only one function is calculated.

Default: Deactivated.

```
.B FREQ
```

.B FREQ

Specify frequencies of operator B.

Example: 3 different frequencies.

```
.B FREQ
3
0.001
0.002
0.01
```

Default: Static case.

```
.B FREQ
1
0.0
```

```
.IMAGIN
```

.IMAGIN

Employ imaginary frequencies.

```
.ALLCMB
```

.ALLCMB

Form all possible combinations even if imaginary.

Default: Deactivated.

```
.UNCOUP
```

.UNCOUP

Uncoupled calculation.

Control variational parameters

```
.OCCUP
```

.OCCUP

For each fermion irco give an orbital_strings of inactive orbitals to include in the linear response calculation.

```
.VIRTUA
```

.VIRTUA

For each fermion ircomp give an orbital_strings of virtual orbitals to include in the linear response calculation.

.SKIPEE

.SKIPEE

Exclude all rotations between occupied positive-energy and virtual positive-energy orbitals.

.SKIPEP

.SKIPEP

Exclude all rotations between occupied positive-energy and virtual negative-energy orbitals.

Control reduced equations

.MAXITR

.MAXITR

Maximum number of iterations.

Default:

.MAXITR
30

.MAXRED

.MAXRED

Maximum dimension of matrix in reduced system.

Default:

.MAXRED
200

.THRESH

.THRESH

Threshold for convergence of reduced system.

Default:

.THRESH
1.0D-5

Control integral contributions

The user is encouraged to experiment with these options since they may have an important effect on run time.

.INTFLG

.INTFLG

Specify what two-electron integrals to include (default: HAMILTONIAN_.INTFLG under **HAMILTONIAN).

.CNVINT

.CNVINT

Set threshold for convergence before adding SL and SS integrals to SCF-iterations.

2 (real) Arguments:

.CNVINT
CNVXQR(1) CNVXQR(2)

Default: Very large numbers.

.ITRINT

.ITRINT

Set the number of iterations before adding SL and SS integrals to SCF-iterations.

Default:

```
.ITRINT
1 1
```

Control trial vectors

```
.REAXVC
```

.REAXVC

Read solution vectors from file XVCFIL

```
.REAXVC
XVCFIL
```

Default: No restart on solution vectors. The file has to have six characters. Make sure there is no blank character in front of the file name.

For a restart on solution vectors it is useful to set

```
.REAXVC
XVCFIL
.ITRINT
0 0
```

otherwise LS-integrals (and SS-integrals) are switched on later and one may first iterate away and then back to a possibly converged response vector.

Often you have a converged SCF wave function along with a response vector. In this case make sure that

```
**DIRAC
#.WAVE FUNCTION
```

is commented out. Make then also sure that you use the CHECKPOINT file which has been obtained in the *same* calculation as the response vector file. Otherwise you may observe more response solver iterations than necessary.

```
.XLRNRM
```

.XLRNRM

Normalize trial vectors. Using normalized trial vectors will reduce efficiency of screening. CLARIFY!

Default: Use un-normalized vectors.

Analysis

```
.ANALYZ
```

.ANALYZ

The linear response function is obtained by contracting the property gradient $\mathbf{E}_{A[1]}$ associated with property A with the solution vector $\mathbf{X}_B$ associated with property B :

$$\langle \hat{A}; \hat{B} | \mathbf{r}_{\omega_b} \rangle = \mathbf{E}_{A[1]}^\dagger \mathbf{X}_B$$

The indices of the two vectors run over orbital rotation indices, that is, one occupied and one virtual index. This analysis shows the most important contributions from orbital pairs to the dot product as well as the most important occupied and virtual orbitals. More information about these orbitals can then be gleaned from Mulliken population analysis.

```
.ANATHR
```

.ANATHR

This keyword adjusts the number of terms shown in the above analysis. The threshold is in terms of percentage value to the total value of the linear response function.

Default: 2.0% .

Advanced/debug flags

```
.E2CHEK
```

.E2CHEK

Generate a complete set of trial vector which implicitly allows the explicit construction of the electronic Hessian. Only to be used for small systems !

```
.ONLYSF
```

.ONLYSF

Only call FMOLI in sigmavector routine: only generate one-index transformed Fock matrix Saue2003.

```
.ONLYSG
```

.ONLYSG

Only call FMOLI in sigmavector routine: 2-electron Fock matrices using one-index transformed densities Saue2003.

.STERNHEIM

.STERNHEIM

Set diagonal elements of orbital part of Hessian equal to

-2 m c^2

for rotations between occupied positive-energy and virtual negative-energy orbitals.

Default: Deactivated.

.STERNC

.STERNC

(Sternheim complement) allows to separate basis set incompleteness from the replacement of an inner sum over negative-energy orbitals only by the full sum. In order to benefit from this functionality (only for specialists !), you should run with print level 2 under properties.

Then you can do a sequence of calculations: 1) .SKIPEP 2) .STERNH 3) .STERNC The diamagnetic contribution of 1) is the non-relativistic expectation value, whereas 2) is the Sternheim approximation, that is replacing orbital energy differences with

-2 m c^2

With no basis set incompleteness the sum of the diamagnetic contribution 2) and the paramagnetic contribution 3) should equal the diamagnetic contribution of 1).

Default: Deactivated.

.COMPRESSION

.COMPRESSION

Reduce number of orbital variation parameters by checking corresponding elements of gradient vector against a threshold. This may reduce memory.

Default: No compression.

.COMPRESSION
0.0

.NOPREC

.NOPREC

No preconditioning of initial trial vectors.

Default: Preconditioning of trial vectors.

.RESFAC

.RESFAC

New trial vector will be generated only for variational parameter classes whose residual has a norm that is larger than a fraction 1/RESFAC of the maximum norm.

Default:

.RESFAC
1000.0

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/molgrd.md

orphan

*MOLGRD

***MOLGRD**

The section gives directives for control of a single point evaluation of the molecular gradient activated with PROPERTIES_.MOLGRD under **PROPERTIES.

The molecular gradient module is usually called from within the geometry optimization module.

Advanced options

.TRICK

.TRICK

If activated the calculation of the SL and SS two-electron integral contributions to the gradient is skipped if their contribution is estimated to be small. An “empirical” estimate of the norms of the LS and SS two-electron gradients based on the norm of the LL two-electron gradient. If deactivated all contributions to gradient are calculated and if activated then only “necessary” contributions are calculated.

Default: Deactivated.

.INTFLG

.INTFLG

Specify what two-electron integrals to include (default: HAMILTONIAN_, INTFLG under **HAMILTONIAN).

.SCREEN

.SCREEN

Screening threshold.

Default:

.SCREEN
1.00-17

Which means that screening is turned off by default. This is due to an enormous demand of memory.

Programmers options

.PRINT

.PRINT

Print level.

A print level of less than 3 gives only the total gradient. Print levels from 3+ gives individual contributions to the gradient (kinetic energy gradient, nuclear attraction gradient etc.). Print levels of 5+ print some matrices, and 10+ give massive output!

Default: Print level GENERAL_.PRINT from **GENERAL.

.NUMGRA

.NUMGRA

Calculate the molecular gradient using numerical differentiation.

Default: Analytical evaluation if implemented.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/nmr.md

orphan

*NMR

***NMR**

This section gives directives for the calculation of NMR parameters.

If common gauge origin (CGO) is used, i.e. :LONDON not specified, then the user can define the gauge origin used for the external magnetic field with HAMILTONIAN_.GAUGEORIGIN or HAMILTONIAN_.GO ANG under **HAMILTONIAN. If NMR_.USECM is specified then center-of-mass is used as gauge origin. Default is (0, 0, 0).

Advanced options

.LONDON

.LONDON

Activate calculations of magnetic properties (NMR shielding constants and magnetizabilities) with London atomic orbitals.

Default: Use conventional atomic orbitals.

.USECM

.USECM

Use the center of mass as the gauge origin.

.USEBC

.USEBC

Use the nuclei charge barycenter as the gauge origin.

.INTFLG

.INTFLG

Specify what two-electron integrals to include in the two-electron London contributions to the magnetic field property gradient (default: HAMILTONIAN_ .INTFLG under **HAMILTONIAN).

.NOTWO

.NOTWO

Do not calculate the two-electron London contributions for the magnetic field property gradient when London atomic orbitals are used.

.NOONEI

.NOONEI

Do not calculate the $\{H(0), T(B)\}$ reorthonormalization terms for the magnetic field property gradient when London atomic orbitals are used.

.NOORTH

.NOORTH

Do not calculate the $\{T(B), h(mK)\}$ reorthonormalization contributions for the expectation value term when London atomic orbitals are used.

.SYMCON

.SYMCON

Employ the symmetric connection for reorthonormalization terms when using London atomic orbitals.

Default: Use the natural connection.

.EXPPED

.EXPPED

Keyword used in the DFT calculations only. The contributions to the density perturbed by an external magnetic field in LAO basis (“direct” LAO term and “reorthonormalization” term) are exported on files, pertden_direct_lao.FINAL and pertden_reorth_lao.FINAL.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/nqcc.md

orphan

*NQCC

***NQCC**

This section gives directives for the calculation of Nuclear Quadrupole Coupling Constants (NQCC). By default they are given in the EFG principal axis system.

Advanced options

.CARTESIAN

.CARTESIAN

Activate the calculation of NQCC in the cartesian axis system. If symmetry detection is activated, the molecule will be placed into the principal axis system of the moment of inertia tensor and centered at center of mass so that the NQCC will be given in that axis system, following the same axis convention used in Refs. Bauder1997, Soulard2006, Gaul2020.

Default: Use the EFG principal axis system.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/quadratic_response.md

orphan

*QUADRATIC RESPONSE

*QUADRATIC RESPONSE

This section gives directives for the calculation of quadratic response functions Saue2002a.

General control statements

.PRINT

.PRINT

Print level.

Default:

```
.PRINT
0
```

Definition of the quadratic response function

.DILEN

.DILEN

Specification of dipole operators for A, B, and C (see `one_electron_operators` for details).

.A OPERATOR

.A OPERATOR

Specification of the A operator (see `one_electron_operators` for details).

.B OPERATOR

.B OPERATOR

Specification of the B operator (see `one_electron_operators` for details).

.C OPERATOR

.C OPERATOR

Specification of the C operator (see `one_electron_operators` for details).

.B FREQ

.B FREQ

Specify frequencies of operator B.

Example: 3 different frequencies.

```
.B FREQ
3
0.001
0.002
0.01
```

Default: Static case.

```
.B FREQ
1
0.0
```

.C FREQ

.C FREQ

Specify frequencies of operator C (see `QUADRATIC_RESPONSE_.B FREQ`).

.ALLCMB

.ALLCMB

Evaluate all nonzero quadratic response functions and thereby disregarding analysis of overall permutational symmetry.

Default: Evaluate only unique, nonzero, response functions.

Excited state properties

This page describes unreleased functionality. The keywords may not be available in your version of DIRAC.

First order properties of excited states can be computed from the quadratic response function.

.EXCPRP

.EXCPRP

Give the number of “left” and “right” states in each boson symmetry.

Example:

```
.EXCPRP
5 5 5 5
0 0 0 0
```

Compute the excited state expectation values $\langle \hat{A}_i \rangle$, where i goes from 1 to 5 in each symmetry (four symmetries in this case). The zeros can be substituted with positive integers to generate transition state moments $\langle \hat{A}_i \rangle$.

Control variational parameters

.SKIPEE

.SKIPEE

Exclude all rotations between occupied positive-energy and virtual positive-energy orbitals.

.SKIPEP

.SKIPEP

Exclude all rotations between occupied positive-energy and virtual negative-energy orbitals.

Control reduced equations

.MAXITR

.MAXITR

Maximum number of iterations.

Default:

```
.MAXITR
30
```

.MAXRED

.MAXRED

Maximum dimension of matrix in reduced system.

Default:

```
.MAXRED
100
```

.THRESH

.THRESH

Threshold for convergence of reduced system.

Default:

```
.THRESH
1.0D-5
```

Control integral contributions

The user is encouraged to experiment with these options since they may have an important effect on run time.

.INTFLG

.INTFLG

Specify what two-electron integrals to include (default: HAMILTONIAN_ . INTFLG under **HAMILTONIAN).

.CNVINT

.CNVINT

Set threshold for convergence before adding SL and SS integrals to SCF-iterations.

2 (real) Arguments:

```
.CNVINT
CNVXQR(1) CNVXQR(2)
```

Default: Very large numbers.

.ITRINT

.ITRINT

Set the number of iterations before adding SL and SS integrals to SCF-iterations.

Default:

```
.ITRINT
1 1
```

Control trial vectors

.XQRNRM

.XQRNRM

Normalize trial vectors. Using normalized trial vectors will reduce efficiency of screening.

Default: Use un-normalized vectors.

Advanced/debug flags

.NOPREC

.NOPREC

No preconditioning of initial trial vectors.

Default: Preconditioning of trial vectors.

.RESFAC

.RESFAC

New trial vector will be generated only for variational parameter classes whose residual has a norm that is larger than a fraction 1/RESFAC of the maximum norm.

Default:

```
.RESFAC
1000.0
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/properties/stex.md

orphan

*STEX

*STEX

Core excitation spectra in the Static Exchange approximation

A well known limitation of the time dependent Hartree-Fock approximation is that electronic relaxation is not accounted for in the calculation of excitation energies. This relaxation can be understood as a correlation effect, since it is due to the simultaneous excitation of an electron and the relaxation of the remaining N-1 electrons (or rather orbitals) due to the “hole” left behind by the excited electron. On the other hand the relaxation is well described already at the Hartree-Fock level when the excited states are optimized separately, in the so-called Δ SCF approach.

The Static Exchange (STEX) approximation is a cheap way to incorporate relaxation effects into the calculation of core excitation spectra, where it is particularly important for excitation energies and transition probabilities. The method works by making a configuration interaction singles expansion of the excited states with a reference determinant optimized for core holes in a particular set of orbitals. The excited states are then not orthogonal to the Hartree-Fock ground state, and wavefunction overlaps have to be explicitly taken into account.

A STEX calculation typically proceeds in the following way:

1. The Hartree-Fock groundstate is calculated with Dirac, and the wavefunction saved in the DFC0EF file.
2. The groundstate output is examined and the indices of the core orbital of interest are noted. If the core orbitals are not well localized to individual atoms it may be necessary to add a small fractional charge to break a symmetry of the molecule. This change can then be removed in the final calculation.
3. An average-of-configuration open shell Hartree-Fock calculation is performed to get an approximate wavefunction of the core ionized molecule. This wavefunction should then be saved to a file DFC0EF.ION. In order for the core ionized state to converge it is necessary to use overlap selection and possibly orbital freezing.
4. The STEX calculation is run with the specified hole orbitals, starting from the previously calculated DFC0EF and DFC0EF.ION files.

please provide an example !

Directives

Basic

.HOLES

.HOLES

Indicate (singly occupied) hole orbitals. Example:

```
.HOLES
! Expects format like (NFSYM=2)
2      ! nr of holes in fermion symmetry 1.
5 9    ! holes in fs 1
1      ! nr of holes in fermion symmetry 2.
1      ! holes in fs 2
```

.PRINT

.PRINT

Print level. *Default:* 0

Advanced keywords

.BATCH

.BATCH

Number of simultaneous Fock matrices to build. *Default:* 4

.CUBE

.CUBE

Deactivated ?

.CUTOFF

.CUTOFF

Set default cutoff in eV. *Default:* 100 eV

.SCREEN

.SCREEN

Screening threshold.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/visual.md

orphan

**VISUAL

**VISUAL

This section describes the functionalities of the VISUAL module, which calculates various molecular densities from the 1-electron density matrices and atomic orbitals. These densities (here called grid functions) are calculated on numerical grids and can be integrated or exported as scalar, vector, or tensor fields (e.g., for visualization or further processing). The VISUAL module is not activated by default.

The main setup for these calculations should be provided by the .GRIDS and .GRIDFUNCTIONS blocks, as described below.

GRIDS

.GRIDS

.GRIDS

Grid(s) specification block. Next line should contain the number of grids to be used. Then, the following lines should provide information about each grid (as a key=value list).

For example, to import the grid from the grid.h5 file, use:

```
.GRIDS
1
id=1 input=import file_inp=grid.h5
```

Available key=value options

- id=int

The ID of a grid. Required, used as a reference for VISUAL_.GRIDFUNCTIONS specification.

- npoints=[Nx,Ny,nz]

Specify the number of grid points in x, y, and z directions. Either npoints or spacing is required for grid generation.

- spacing=[Sx,Sy,Sz] or spacing=S

Specify the grid resolution, i.e., the distance between two consecutive grid points. Use spacing=[Sx,Sy,Sz] to specify the distance between grid points in x, y, and z directions, or spacing=S to specify the same distance between grid points for all directions. Either npoints or spacing is required for grid generation.

- input=option

Specify grid input. Available options:

- input=create : create grid; the default is a 3D rectilinear grid; requires the specification of npoints or spacing
- input=import : import grid from an external file
- input=dft : use the DFT numerical grid; requires numerical_grid file
- input=list : use the list of grid points; should be provided in the input (see VISUAL_.LIST)
- input=line : create grid points on line; should be provided in the input (see VISUAL_.LINE keyword)
- input=2d : create grid points on a 2D plane; should be provided in the input (see VISUAL_.2D keyword)
- input=2d_int : create grid points on a 2D plane for integration; should be provided in the input (see VISUAL_.2D_INT keyword)
- input=3d : create grid points on a 3D surface; should be provided in the input (see VISUAL_.3D keyword)
- input=3d_int : create grid points on a 3D surface for integration; should be provided in the input (see VISUAL_.3D_INT keyword)
- input=radial : create radial grid; should be provided in the input (see VISUAL_.RADIAL keyword)

- margin=f

Add space around the 3D rectilinear grid. Applies only to generated 3D rectilinear grids. Default is f=0.0 (value in a.u.), which creates grid point spanning the space between the minimum and maximum coordinates of all atoms, i.e., $x_{\text{grid}} = [\min(x_{\text{a}}), \max(x_{\text{a}})]_{\text{a in atoms}}$, $y_{\text{grid}} = [\min(y_{\text{a}}), \max(y_{\text{a}})]_{\text{a in atoms}}$, $z_{\text{grid}} = [\min(z_{\text{a}}), \max(z_{\text{a}})]_{\text{a in atoms}}$. Therefore, for typical applications, choose some $f > 0$ (e.g., margin=4.0 is typically OK for the electron density visualization).

- export=option

If the grid should be exported to a file, this allows to define the file format for export. Supported formats:

- export=hdf5 : export to hdf5 file
- export=txt : export to text file, no header, space as a separator (for backward compatibility with old VISUAL code)
- export=csv : export to text file (csv): header line, coma as a separator

- file_inp=string

The name of a file, from which the grid should be read.

- file_out=string

The name of a file, to which the grid should be exported

Notes:

- *current limitations:*
 - make sure there is one space between each key=value entry in the grid specification line.
 - the maximum line length in the input file is 200 characters
- for the specific examples, see the corresponding test in DIRAC test suite: test/visual_grid_options

Additional keywords

The selected grid inputs (see input=option) require additional specification. This should be provided as a separate keyword block **following** the .GRID keyword. Most of these keywords are the same as used in the old VISUAL code.

.LIST

.LIST

Calculate various densities in few points. Scalar and vector densities are written to the standard output file. Example (3 points; coordinates in bohr):

```
.LIST
3
1.0 0.0 0.0
0.0 1.0 0.0
0.0 0.0 1.0
```

In the DIRAC input file, this should be provided as:

```
.GRIDS
id=1 input=list
.LIST
3
1.0 0.0 0.0
0.0 1.0 0.0
0.0 0.0 1.0
```

.LINE

.LINE

Calculate various densities along a line. Example (line connecting two points; 200 steps; coordinates in bohr):

```
.LINE
0.0 0.0 0.0
0.0 0.0 5.0
200
```

.RADIAL

.RADIAL

Compute radial distributions

$$f(r) = \int_0^{2\pi} \int_0^\pi f(\mathbf{r}) r^2 \sin\theta \, d\theta \, d\phi$$

by performing Lebedev angular integration over a specified number of even-spaced radial shells out to some specified distance from a specified initial point. Example (coordinates and distance in bohr):

```
.RADIAL
0.0 0.0 0.0
10.0
200
```

The first line after the keyword specifies the initial point, here chosen to be the origin. The second and third line is the distance and step size, respectively.

.2D

.2D

Calculate various densities in a plane. The plane is specified using 3 points that have to form a right angle. Example (coordinates in bohr):

```
.2D
0.0 0.0 0.0 !origin
0.0 0.0 10.0 !"right"
200 !nr of points origin-"right"
0.0 10.0 0.0 !"top"
200 !nr of points origin-"top"
```

.2D_INT

.2D_INT

Integrate various densities in a plane using Gauss-Lobatto quadrature. The plane is specified using 3 points that have to form a right angle. Example (coordinates in bohr):

```
.2D_INT
0.0 0.0 0.0 !origin
0.0 0.0 10.0 !"right"
10 !nr of tiles to the "right"
0.0 10.0 0.0 !"top"
10 !nr of tiles to the "right"
5 !order of the Legendre polynomial for each tile
```

.3D

.3D

Calculate various densities in 3D and write to cube file format. Example (coordinates in bohr):

```
.3D
40 40 40 ! 40 x 40 x 40 points
```

Note: this is the same as using npoints=[40,40,40] in the grid specification line.

.3D_INT

3D_INT

Integrate various densities in a volume.

Grid functions

.GRIDFUNCTIONS

.GRIDFUNCTIONS

Grid function(s) specification block. Next line should contain the number of grid functions to be calculated. Then, the following lines should provide information about each grid function (as a key=value list).

For example, to calculate the values of the electron density (*ed*), its laplacian (*ed_laplacian*), and the reduced density gradient (*rdg*) in the grid points of the grid supplied with the index \$id=1\$ (specified in the VISUAL_.GRIDS section of the input) and exporting them to the respective files, use:

```
.GRIDFUNCTIONS
3
name=ed id_grid=1 purpose=visualization export=cube file_out=ed.cube
name=ed_laplacian id_grid=1 purpose=visualization export=hdf5 file_out=ed_laplacian.h5
name=rdg id_grid=1 purpose=visualization export=csv file_out=rdg.csv
```

Available grid functions

The table below summarizes the grid functions currently implemented in DIRAC with their respective test directories in DIRAC test suite.

name	property	DIRAC tests directories
ed	electron density, $\rho(\vec{r})$	visual_3d_electronden*
ed_gradient	electron density gradient, $\nabla \rho(\vec{r})$	visual_3d_electronden*
ed_hessian	electron density Hessian, $\nabla^2 \rho(\vec{r})$	visual_3d_electronden*
ed_laplacian	electron density Laplacian, $\nabla^2 \rho(\vec{r})$	visual_3d_electronden*
ed_sign_lambda2	$\text{sign}(\lambda^2 \rho(\vec{r}))$	visual_3d_electronden*
rdg	reduced density gradient	visual_3d_electronden*
elf	electron localization function	visual_3d_elf*
kin	kinetic energy density (1)	visual_3d_kinden*
kinls	kinetic energy density (2)	visual_3d_kinden*
kinsl	kinetic energy density (2)	visual_3d_kinden*
kinlap	the Laplacian of the kinetic energy density	visual_3d_kinden*
kintau	kinetic energy density (3)	visual_3d_kinden*
kinnr	the non-relativistic kinetic energy density	visual_3d_kinden*
s	spin density, $\rho_s(\vec{r})$	visual_3d_spinden*
divs	$\Delta \rho_s(\vec{r})$	visual_3d_spinden*
rots	$\Delta \rho_s(\vec{r})$	visual_3d_spinden*
gamma5	γ^5 density	visual_3d_gamma5*
edip	the electric dipole density	visual_3d_rspE*
bdip	the magnetic dipole density	visual_3d_rspB*
ndip	the nuclear magnetic dipole density	visual_3d_rspB*
j	the probability current density, $\vec{j}(\vec{r})$	visual_3d_rspB*
rotj	$\nabla \times \vec{j}(\vec{r})$	visual_3d_rspB*
divj	$\nabla \cdot \vec{j}(\vec{r})$	visual_3d_rspB*
gradj	$\Delta \vec{j}(\vec{r})$	visual_3d_rspB*

DIRAC properties

Comments:

- the kinetic energy density (1) and its LS- and SL-components (2) are calculated as $\alpha \psi^\dagger(\mathbf{r}) (\alpha \cdot \mathbf{p}) \psi(\mathbf{r})$
- the kinetic energy density (3) calculated as $\tau = \nabla_i \phi_k \nabla_i \phi_k$

Available key=value options

Notes:

- *current limitations:*
 - make sure there is one space between each key=value entry in the grid function specification line.
 - the maximum line length in the input file is 200 characters

General key=value options

- name=string

The name of the grid function to calculate; see the available options in the table above.

- id_grid=int

The ID of a grid on which the grid function should be calculated (see the grid ID specification in VISUAL_.GRIDS).

- input=option

Specify grid function input. Available options:

- input=calculate : calculate grid function; the default option
- input=import : import grid function from file

- purpose=option

Specify the type of calculations. Available options:

- purpose=visualization : calculate grid function in grid points; possible export as scalar/vector/tensor field for a subsequent visualization; the default option
- purpose=integration : integrate grid function; for the specific examples, see the corresponding test in DIRAC test suite: test/visual_integration.

- outpri=option

Specify whether the grid function values should also be printed out to DIRAC output. Available options: outpri=no (default if purpose=visualization), outpri=yes (default if purpose=integration).

- export=option

If the grid function should be exported to a file, this allows to define the file format for export. Supported formats:

- export=hdf5 : export to hdf5 file
- export=txt : export to text file, no header, space as a separator (for backward compatibility with old VISUAL code)
- export=cube : export to gaussian cube file; available only for 3D rectilinear grids and scalar fields (for backward compatibility with old VISUAL code)
- export=csv : export to text file (csv): header line, comma as a separator

- file_inp=string

The name of a file, from which the grid should be read.

- file_out=string

The name of a file, to which the grid should be exported

key=value options allowing for pointwise modification of grid functions

- dscale=f

Scale densities *down* by a factor f. Default: dscale=1.0

- uscale=f

Scale densities *up* by a factor f. Default: uscale=1.0

- carpow=[x,y,z]

Scale densities by Cartesian product $x^i y^j z^k$. The x,y,z values are three integers specifying the exponents (i,j,k) . For example, carpow=[1,0,0] is equivalent to calculating the x-component of the electric dipole moment density (specification name=edip).

- radpow=f

Scale densities by a radial power r^n . The keyword is followed by three integers specifying the exponent n . Example: radpow=1 allows to the calculation of radial expectation values $\langle r \rangle$ with respect to the origin.

Notes:

- for the specific examples, see the corresponding test in DIRAC test suite: test/visual_grid_function_modifications

Additional keywords affecting grid functions

.OCCUPATION

.OCCUPATION

Specify occupation of orbitals. Example (neon atom):

```
.OCCUPATION
2
1 1-2 1.0
2 1-3 1.0
```

The first line after the keyword gives the number of subsequent lines to read. In each line, the first number is the fermion ircop. In molecules with inversion symmetry there are two fermion ircops: gerade (1) and ungerade (2). Otherwise there is a single fermion ircop (1). The specification of the fermion ircop is followed by the range of selected orbitals and their occupation. If a single orbital is specified a single number is given instead of the range.

Another example (water):

```
.OCCUPATION
1
1 1-5 1.0
```

Another example (nitrogen atom):

```
.OCCUPATION
2
1 1-2 1.0
2 1-3 0.5
```

Warning: this keyword affects *all* grid functions specified in the input.

.CVALUE

.CVALUE

Set the speed of light to be used in the VISUAL module.

Warning: this keyword affects *all* grid functions specified in the input.

General setup

.PRINTL

.PRINTL

Control the print level:

```
.PRINTL
print_level
```

Predefined print_level values:

- print_level = 0: only the basic information from the VISUAL calculations is printed out; the default option
- print_level = 1 : more details on the methods used are printed out
- print_level = 2 : additional print of matrices (warning: large output!)
- print_level > 2 : the development prints (warning: large output!)

All information is printed out to DIRAC output.

Additional notes (untested functionalities; notes from the old VISUAL code)

[!WARNING] Only the calculation of the density is tested for open shell configurations (and relies on a correct .OCCUPATION). All other properties are only tested for closed shell systems and should not be trusted for open shell systems without a thorough testing.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function.md

orphan

**WAVE FUNCTION

****WAVE FUNCTION**

Get the wave function

This section allows the specification of which wave function module(s) to activate. By default no modules are activated. To activate any of these modules you must also specify DIRAC_.WAVE FUNCTION under **DIRAC, otherwise this input is not read.

Note that the order below specifies the order in which the different modules are called if you ask for more than one.

.SCF

.SCF

Activates the Hartree-Fock/Kohn-Sham module.

Specification of the SCF module can be given in the *SCF subsection.

If HAMILTONIAN_.DFT has been specified under **HAMILTONIAN, then a Kohn-Sham calculation will be performed, otherwise a Hartree-Fock calculation will be performed.

.RESOLVE

.RESOLVE

Resolve open-shell states: do a small CI calculation to get the individual energies of the states present in an average-of-configurations open-shell Hartree-Fock calculation (see *RESOLVE).

.COSCI

.COSCI

Activates advanced COSCI method, see *C0SCI.

.MP2

.MP2

Activates *MP2CAL.

.MVO

.MVO

Calculate modified virtual orbitals (see *MV0CAL). Default after open-shell SCF is modified virtual orbitals based on the closed-shell molecular orbitals. There is no default for closed-shell SCF.

.MP2 NO

.MP2 NO

Activates the *MP2 NO module to calculate MP2 natural orbitals.

.RELCCSD

.RELCCSD

Activates the **RELCC (and the **MOLTRA module to get 4-index transformed integrals).

By default, molecular orbitals with orbital energy between -10 and +20 hartree (a.u.) are included, this can be modified in the **MOLTRA section.

.RELADC

.RELADC

Activates the RELADC and calculates the single and double ionization spectra by the (A)lgebraic (D)iagrammatic (C)onstruction ADC. Also activates the **MOLTRA module to get 4-index transformed integrals.

.POLPRP

.POLPRP

Activates the **POLPRP module for calculation of the excitation spectrum by the strict or extended second order (A)lgebraic (D)iagrammatic (C)onstruction ADC. Also activates the **MOLTRA module to get 4-index transformed integrals.

.DIRRCI

.DIRRCI

Activates the MOLFDIR CI module (and also the **MOLTRA module to get 4-index transformed integrals).

Specification of input for the MOLFDIR CI module is given in the DIRRCI and G0SCIP sections.

By default, molecular orbitals with orbital energy between -10 and +20 hartree (a.u.) are included, this can be modified in the **MOLTRA section.

.LUCITA

.LUCITA

Activates the *LUCITA (and the **MOLTRA module to get 4-index transformed integrals).

By default, molecular orbitals with orbital energy between -10 and +20 hartree (a.u.) are included, this can be modified in the **MOLTRA section.

.EXACC

.EXACC

Activates the **EXACC module, the new coupled cluster implementation based on the ExaTensor library.

.CASPT2

.CASPT2

Activates the *CASPT2 module. complete active space second-order perturbation theory calculation.

Pre-SCF orbital manipulations

.REORDER MO

.REORDER MO

Interchange initial molecular orbitals prior to the SCF-calculation. The start orbitals from DFCOEF are read and reordered.

For each fermion irrep give the new order of orbitals.

Example:

```
.REORDER MO'S
1..8,10,9
```

.ORBROT

.ORBROT

Jacobi rotations between pairs of orbitals.

On the line following the keyword, give first the rotation angle, then on the following line(s) for each fermion irrep, give an `orbital_strings` of orbitals to rotate.

Post-SCF orbital manipulations

.POST SCF REORDER MO

.POST SCF REORDER MO

Interchange converged molecular orbitals. The orbitals from DFCOEF are read and reordered just before exiting the SCF subroutine.

For each fermion irrep give the new order of orbitals.

Example:

```
.POST DHF REORDER MO'S
1..8,10,9
```

.PHCOEF

.PHCOEF

Phase adjustment of coefficients DFCOEF: make the largest element of a given orbital real and positive.

.KRCI

.KRCI

Activates the *KRCI module for the calculation of ground and excited states at the relativistic CI level.

.KRMCSF

.KRMCSF

Activates the *KRMCSF module for the optimization of ground and excited states (in other than the ground state symmetry) at the relativistic MCSF level.

.LAPLCE

.LAPLCE

Activates the *LAPLCE module to compute weights for Laplace transformation of orbital energy denominators with the algorithm of Helmich-Paris. No subsequent calculations, only output of the Laplace points and weights.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/caspt2.md

orphan

*CASPT2

*CASPT2

Relativistic IVO/CASCI/CASPT2 and IVO/RASCI/RASPT2 module, Ref. Masuda2025, Ref. Abe2006, Ref. Anderson1990, Ref. Huzinaga1970

- Initial implementation: Minori Abe
- Parallelisation, DIRAC integration, and RASCI/RASPT2 extension: Kohei Noda

- **MOLTRA interface and IVO functionality: Sumika Iwamuro
- Code optimisation: Kohei Noda and Yasuto Masuda
- RAS3 minimum electron option: Taichi Inoue

The program is based on the CASPT2D formalism reported in Anderson1990.

[!NOTE] Currently, IVO is not supported when the AO coefficients in checkpoint.[no]h5 are complex or quaternion numbers, such as for certain C_2 or C_1 symmetry molecules.

In the input file of IVO, CASCI, and CASPT2 programs, orbitals must be numbered in ascending order of orbital energy, and if multiple orbitals share the same energy, they are numbered by increasing symmetry index. Since DIRAC's SCF output often lists orbitals grouped by symmetry rather than energy, it is convenient to use the following input generator to prepare your CASCI/CASPT2 input.

GUI program to support creating CASPT2 input

You can use the CASPT2 input support GUI program dcaspt2_input_generator <https://github.com/RQC-HU/dcaspt2_input_generator> if your DIRAC output was generated using an input that includes the ANALYZE_.PRIVCEC keyword in the **ANALYZE module.

This program creates the necessary DIRAC input fragment for the *CASPT2 section.

If you are not familiar with writing CASPT2 input files, we recommend using this tool.

For detailed instructions, see https://github.com/RQC-HU/dcaspt2_input_generator/wiki.

Required keywords

.NINACT

.NINACT

Specify the number of inactive spinors.

.NACT

.NACT

Specify the number of active (CAS or RAS) spinors.

.NELEC

.NELEC

Specify the number of electrons in the active space.

.NSEC

.NSEC

Specify the number of secondary spinors.

.SUBPROGRAMS

.SUBPROGRAMS

Specify the subprograms to be used in the CASPT2 calculation.

Currently the following subprograms are available:

- IVO
- CASCI (perform RASCI or CASCI calculation based on your input)
- CASPT2 (perform RASPT2 or CASPT2 calculation based on your input)

For example, if you want to carry out an IVO/CASCI/CASPT2 calculation sequentially, you must specify the sub-programs as follows:

```
.SUBPROGRAMS
IVO
CASCI
CASPT2
```

Required keywords for CASCI/CASPT2 and RASCI/RASPT2

.CASPT2_CIROOTS

.CASPT2_CIROOTS

Specify the total symmetry index and the roots to be included in the CASCI/CASPT2 calculation.

For example, to compute the A_g symmetry of a C_{2h} molecule (symmetry index 5) and request the 1st, 2nd, 4th, 5th, and 6th lowest-energy roots, you need to set the keyword as follows:

```
.CASPT2_CIROOTS
5 1 2 4..6
```

The first number is the total-symmetry index; the remaining numbers specify the roots to be calculated. Additional lines can be added to define roots for other symmetries.

For example:

```
.CASPT2_CIR00TS
5 1 2 4..6
6 1 2
```

Required keywords for IVO

.NOCCG

.NOCCG

Specify the number of occupied gerade Kramers-pair orbitals (count pairs, not individual spinors).
This keyword must be specified for molecules that possess an inversion center.
Conversely, it should not be used for molecules without inversion symmetry.

.NOCCU

.NOCCU

Specify the number of occupied ungerade Kramers-pair orbitals (count pairs, not individual spinors).
This keyword must be specified for molecules that possess an inversion center.
Conversely, it should not be used for molecules without inversion symmetry.

.NOCC

.NOCC

Specify the number of occupied Kramers-pair orbitals (count pairs, not individual spinors).
This keyword must be specified for molecules that lack an inversion center;
it should not be used for molecules with inversion symmetry.

Required keyword for RASCI/RASPT2

.RAS2

.RAS2

Specify the RAS2 spinor indices.
For example, to assign the spinors ranked 7th, 8th, 11th, 12th, 13th, and 14th lowest in energy to the RAS2 space,
set the keyword as follows:

```
.RAS2
7,8,11..14
```

By using this keyword, you can select spinors that are not contiguous in energy for inclusion in the CAS of a CASCI/CASPT2 or RASCI/RASPT2 calculation.
For example, to include the spinors ranked 7th, 8th, 11th, 12th, 13th, and 14th lowest in energy,
set the keyword as follows:

```
.NINACT
6
.NACT
6
.RAS2
7,8,11..14
.SECONDARY
6
```

In this case, spinors 1–6 are inactive;
spinors 7, 8, 11, 12, 13, and 14 are active;
and spinors 9, 10, 15-18 are secondary.

Optional keywords for RASCI/RASPT2

.RAS1

.RAS1

Specify the RAS1 spinor indices on row 1 and the maximum number of holes allowed in RAS1 on row 2.
For example, to assign spinors 3, 4, 5, and 6 (counted in ascending energy) to RAS1 and to allow up to two holes,
set the keyword as follows:

```
.RAS1
3..6
2
```

.RAS3

.RAS3

Specify the RAS3 spinor indices on row 1 and the maximum number of electrons allowed in RAS3 on row 2.
For example, to assign spinors 15, 16, 17, and 18 (counted in ascending energy) to RAS3 and to allow up to two electrons, set the keyword as follows:

```
.RAS3  
15. . 18  
2
```

Optional keywords for IVO

.NHOMO

.NHOMO

Specify the number of HOMO-like spinors.

By default (NHOMO = 0), this value is determined automatically by counting all occupied spinors whose orbital energies lie within 0.1 a.u. of the HOMO energy. If desired, you can override the default by specifying NHOMO explicitly in the input file (e.g., 2, 4, or 6).

Default:

```
.NHOMO  
0
```

.NVCUTG

.NVCUTG

Specify the number of gerade virtual Kramers-pair orbitals to be excluded in the IVO scheme.

This keyword is applicable only to molecules with an inversion center; do not use it for molecules without inversion symmetry.

Default:

```
.NVCUTG  
0
```

.NVCUTU

.NVCUTU

Specify the number of ungerade virtual Kramers-pair orbitals to be excluded in the IVO scheme.

This keyword is applicable only to molecules with an inversion center; do not use it for molecules without inversion symmetry.

Default:

```
.NVCUTU  
0
```

.NVCUT

.NVCUT

Specify the number of virtual Kramers-pair orbitals to be excluded in the IVO scheme.

This keyword should be used only for molecules without an inversion center; do not use it for molecules with inversion symmetry.

Default:

```
.NVCUT  
0
```

general optional keywords for this module

.EShift

.EShift

Specify the energy shift for the CASPT2 calculation.

Default: :

```
.EShift  
0. 0
```

.MINHOLERAS1

.MINHOLERAS1

Specify the minimum number of holes allowed in RAS1.

This keyword is used only in RASCI/RASPT2 calculations.

Note: Setting this parameter may be helpful for excited-state calculations (including core excitations) performed with RASCI/RASPT2.

Default: :

.MINHOLERAS1
0

.MINELECRAS3

.MINELECRAS3

Specify the minimum number of electrons in RAS3.
This keyword is used only in RASCI/RASPT2 calculations.

Default: :

.MINELECRAS3
0

.SCHEME

.SCHEME

Specify the MOLTRA scheme to be used for the CASPT2 calculation.
In most cases, you can simply reuse the scheme defined under ****MOLTRA** (See **MOLTRA_**.SCHEME).

Default:

.SCHEME
4

.COUNTNET

.COUNTNET

Count and print the number of determinants within each irreducible representation as defined by the current CASPT2 input, skipping all other calculations in this module.
(Default: false — no determinant counting is performed.)

.DEBUGPRINT

.DEBUGPRINT

Print debugging information.
(Default: false — no debugging information is printed.)

.RESTART

.RESTART

Restart the CASPT2 calculation from an existing checkpoint file.
(Default: false — the calculation starts from scratch.)
Note: Restart capability is currently implemented only for the CASPT2 or RASPT2 sub-programs.
To create a valid restart file, run the helper script
`gen\_caspt2\_restart <[https://raw.githubusercontent.com/RQC-HU/dirac\\_caspt2/refs/heads/main/tools/gen\\_dcaspt2\\_restart](https://raw.githubusercontent.com/RQC-HU/dirac_caspt2/refs/heads/main/tools/gen_dcaspt2_restart)>`_ .
The typical workflow is:

```
$ gen_caspt2_restart <prev_output_file>
$ pam --inp <input_file> --mol <molfile> --put "caspt2_restart_*
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/cosci.md

orphan

*COSCI

***COSCI**

Introduction

CI module written by S.Yamamoto.

Keywords

.PRINT

.PRINT

Print level.

.NREP

.NREP

Number of irreducible representations. For example, D2h-> 1Eg,2Eg,1Eu,2Eu has 4.

.NOPEN

.NOPEN

...

.INACT

.INACT

Number of inactive MOs (INACT(IFRP),IFRP=1,NFSYM).

.ACTIVE

.ACTIVE

Number of open-shell MOs (ACTIVE(IFRP),IFRP=1,NFSYM).

.GASO

.GASO

Number of spinors for each REP ((NGASO(IREP,IOPEN),IREP=1,NREP),J=1,NOPEN) .

.IELC

.IELC

(IELC(IOPEN),J=1,NOPEN)

.TRDM

.TRDM

Number of roots (TRDM(IREP),I=1,NREP).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/dirrci.md

orphan

DIRRCI – Direct CI module

Relativistic RASCI module written by Lucas Visscher

This section allows for Kramers unrestricted relativistic Configuration Interaction calculations, Ref. MOLFDIR.

Note that when the namelist `goscip` is also present in the input, **GOSCIP** will be called instead of **DIRRCI** !

The input should be given in namelist form.

&CIROOT – Select the state to converge on

IREPNA

Abelian symmetry group of the desired state(s).

Default: First abelian symmetry in the list

NROOTS

Number of states to optimize on.

Default:

NROOTS=1

Advanced options

ISTART

Start vector method.

COSCI start vectors:

ISTART=1

Determinant with lowest eigenvalue:

ISTART=2

First (reference) determinant:

ISTART=3

If the COSCI start vectors are not available, then the default will be 2.

NSEL(NROOTS)

Rank number of states to be optimized.

Default:

NSEL=1, 2, 3, . . .

SELECT

Select wave functions on basis of largest overlap with the start wave function.

Default:

SELECT=F

&DIRECT – Convergence control**CONVERE**

Convergence threshold for energy.

Default:

CONVERR=1.0D-9

MAXITER

Maximum number of direct CI iterations.

Default:

MAXITER=10

Advanced options**CONVERR**

Convergence threshold for residual vector.

Default:

CONVERR=1.0D-10

RESTART

Restart on CI-vectors present in MRCFINV.

Default:

RESTART=F

CPUMAX

Maximum amount of CPU-seconds to be used.

Default:

CPUMAX=604800

&LEADDET – Analyze the CI wave function**Advanced options**

GETDET

Get the list of dominant determinants.

Default:

GETDET=T

COMIN

Print contributions of determinants only if the square of the coefficients is larger than COMIN.

Default:

COMIN=0.1

&OPTIM – Fine tuning of the algorithm**Programmers options****IGENEX**

Write coupling coefficients to file (default):

IGENEX=2

Calculate coupling coefficients when needed:

IGENEX=1

&RASORB – Specify the type of CI and the active space**NELEC**

Number of electrons (excluding frozen core electrons).

Default:

NELEC=0

NRAS1

Number of spinors in the RAS1 space for each abelian irrep.

Default:

NRAS1=NSYMRP*0

NRAS2

Number of spinors in the RAS2 space for each abelian irrep.

Default:

NRAS2=NSYMRP*0

MAXH1

Maximum number of holes in RAS1 spinors.

Default:

MAXH1=0

MAXE3

Maximum number of electrons in RAS3 spinors.

Default:

MAXE3=0

&CIFOPR – Specify different options in CI Property Module**PROPER**

Activate the property module to evaluate expectation values of one_electron_operators defined inside

PRPTR under **MOLTRA**, over the DIRRCI wavefunction Nayak2006, Nayak2007, Nayak2009.

Default:

PROPER=F

NEOPER

Define the number of property operators one needs to calculate expectation values.

Default:

NEOPER=0

NAMEE

Mention the names of one_electron_operators as defined inside **PRPTRA**.

For example, Electric dipole moment and magnetic hyperfine structure constants

can be defined as follows. Of couses, the given names are users own choice.

NAMEE='Z-DIP', 'X1-HYP', 'Y1-HYP', 'Z1-HYP'

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/exacorr.md

orphan

Highly parallelised relativistic Coupled Cluster Code

****EXACC**

****EXACC**

This section discusses the highly parallelised relativistic Coupled Cluster Code (ExaCorr) Pototschnig2021.

In this release only computations using the X2C Hamiltonian (with either HAMILTONIAN_.X2Cmmf or HAMILTONIAN_.X2C) are possible.

Originally based on the tensor libraries [TAL-SH](#) and [ExaTENSOR](#) by Dmitry Lyakh (which support both CPU and GPU architectures), through the Tensor Algebra Processing Primitives (TAPP TAPP2026) API the code now supports additional tensor libraries: [TBLIS](#) (CPU architectures), and the [TAPP reference implementation](#) (CPU architectures, testing only).

The choice of tensor library is made at compile time, see the section on [tensor library installation and use](#) for further information on choosing the library and setting up parallel runs.

In all supported tensor libraries, the tensors used in ExaCorr are currently kept in working memory. As such, sufficient RAM needs to be available.

In ExaTENSOR the memory is distributed, so each additional node used in the calculation will contribute its memory to the memory pool accessible by the library. Currently, it is recommended to use enough nodes that the tensors fit, but not substantially more. In the current release, if the library runs out of memory the code will not stop but enter a blocked state and calculations will not advance. So carefully control the advancement of your calculations, stopping them if they appear to hang.

For the other tensor libraries, calculations can only be carried out on a single node and as such are bound to the memory available in the underlying hardware. For TAL-SH runs a global memory pool size is controlled by the keyword EXACC_.TALSH_BUFF. In the case of TAL-SH and ExaTENSOR, instructions/suggestions for tesing memory requirements can be found in the exacorr_talsh_memory and exacorr_exatensor_memory tests.

Mandatory keywords

.OCCUPIED**.OCCUPIED**

Defines occupied orbitals. Specification of list or energy range (see orbital_strings).

```
.OCCUPIED
energy -1.0 0.0 0.00001
```

.VIRTUAL**.VIRTUAL**

Defines virtual orbitals. Specification of list or energy range (see orbital_strings).

```
.VIRTUAL
20..30
```

Optional keywords

All of the following keywords are optional. For convenience, these have been separated into different classes though in the input such distinctions are not made. For practical examples please consult the inputs for the exacorr tests.

General

.OCC_BETA

.OCC_BETA

Can be used to specify a “high-spin” reference determinant with a different number of “barred” occupied orbitals, than “unbarred” occupied spinors. If .OCC_BETA is specified .OCCUPIED is interpreted as a list of unbarred (alpha) spinors. NB: alpha and beta are used in a loose sense in relativistic calculations to indicate the (un)barred spinors.

```
.OCC_BETA
energy -1.0 0.0 0.00001
```

.VIR_BETA

.VIR_BETA

Can be used to specify a different number of “barred” virtual orbitals than “unbarred” occupied spinors. If .VIR_BETA is specified .VIRTUAL is interpreted as a list of unbarred (alpha) spinors. NB: alpha and beta are used in a loose sense in relativistic calculations to indicate the (un)barred spinors.

```
.VIR_BETA
20.30
```

.PRINT

.PRINT

Print level.

Default:

```
.PRINT
0
```

.EXATENSOR

.EXATENSOR

This keyword activates the full multinode EXATENSOR library, which is designed for massively parallel supercomputers. The additional infrastructure needed for parallel communication makes this implementation inefficient when used for single node runs. For such purposes the use of the supported single-node tensor libraries is recommended, as it avoids the computational overhead in setting up the parallel environment, and for TAL-SH or some TAPP backends that may enable the use of GPUs when available.

Default:

Do not use EXATENSOR

.LSHIFT

.LSHIFT

Expert option: Level shift of orbital energies, ignored for values smaller 0.

Default:

```
.LSHIFT
0.000
```

.EXECUTOR

.EXECUTOR

Selects the tensor library interface between TAPP (1), TAL-SH (2) and EXATENSOR (3). In the case of TAPP, the tensor library backend (reference implementation, TBLIS, etc) is preselected at compilation time.

Default:

```
.EXECUTOR
2
```

.AUTOKERNELS

.AUTOKERNELS

Activates the use of automatically generated code for the solution of amplitude, multipliers etc equations. The use of wavefunctions with triples and higher excitations requires this keyword.

Default:

Do not use automatically generated equations code

.AUTOScheme

.AUTOScheme

Selects automatically generated code. Supported options are (1) for mbauto mbauto and (2) for tenpi Brandejs2025.

Default:

```
.AUTOScheme
2
```

Integral transformation

.MOINT_Scheme

.MOINT_Scheme

Expert option to choose another AO to MO integral transformation scheme. Change at your own risk.

In TALSH only schemes 3 (default) and 42 (using Cholesky decomposition) are available.

In ExaTensor schemes 1-4 and 42 are available with 42 using Cholesky decomposition. Scheme 4 is default for ExaTensor as it reduces the memory footprint by only keeping part of the AO integrals in memory. The other methods keep all AO integrals in memory. Scheme 0 prints the memory requirements and attempts to allocate the memory without doing the calculation.

Default:

```
.MOINT_Scheme
3
```

.CHOLESKY

.CHOLESKY

Threshold to define the accuracy of the Cholesky decomposition (MOINT scheme 42), resulting in inaccuracies of the computed energy of this order of magnitude (in Hartree units).

Default:

```
.CHOLESKY
1.0D-9
```

Tensor library specific

.EXA_BLOCKSIZE

.EXA_BLOCKSIZE

Expert option: Number to tune the parallel distribution (branching) of the spinor spaces.

Default:

```
.EXA_BLOCKSIZE
75
```

.TALSH_BUFF

.TALSH_BUFF

Maximum memory (in gigabytes) used in TALSH, aim at about 80% of available memory on your machine.

Default:

```
.TALSH_BUFF
50
```

Ground-state calculation general controls

.FF_PROP

.FF_PROP

Carries out finite-field calculations for one or more operators (defined in the *PRPTRA section) and one or more perturbation strengths. The input expects a first line with the number of properties and operators, followed by blocks containing the operator label and a list of real and imaginary components of the perturbations strengths

```
.FF_PROP
2 3
[operator label 1]
re(P1), im(P1)
re(P2), im(P2)
```

```
re(P3), im(P3)
[operator label 2]
re(P4), im(P4)
re(P5), im(P5)
re(P6), im(P6)
```

where P1,P2,... are the perturbation strenghts. The perturbations are only applied to the energy calculation, not to any subsequent steps.

Example The input below defines the Heff_EDM operator and carries out finite-field calculations for two imaginary frequencies

```
**MOLTRA
.ACTIVE
energy -1.00 2.0 0.001
.NO4IND
.PRPTRA
*PRPTRA
# eEDM effective operator
.OPERATOR
'Heff_EDM'
iBETAGAM
EDM
COMFACTOR
1.0D0
**EXACC
.OCCUPIED
energy -1 0.0 0.001
.VIRTUAL
energy 0.0 2 0.001
.OCC_BETA
energy -1 -0.2 0.001
.VIR_BETA
energy -0.2 2 0.001
.FF_PROP
1 2
Heff_EDM
0.0000,0.00000
0.0000, 0.005
```

Default:

Finite field calculations are not enabled

.TCONVERG

.TCONVERG

Set convergence criteria (CC iterations, Lambda equations)

Default:

.TCONVERG
1.0D-9

.NCYCLES

.NCYCLES

Maximum number of allowed CC iterations to solve the CC and LAMBDA equations.

Default:

.NCYCLES
30

.LAMBDA

.LAMBDA

Solve Lambda-equations, needs to be activated in order to compute the one particle density matrix and molecular properties.

This calculation generates the file CCDENS, which contains the CC ground-state density matrix in AO basis. In this release, CCDENS is used by the property module to calculate ground-state expectation values.

If saved, CCDENS can be used in a property calculation (see PROPERTIES_.RDCCDM) without the need to invoke this module.

Default:

Lambda equations are not solved

.GS2RDM

.GS2RDM

Forms the 2-body reduced density matrix (2RDM) for the ground-state CCSD wavefunction. When generated, the 2RDM and the 1RDM generated after solving the Lambda-equations are used to calculate the correlation energy (which can be compared to the one calculated with the CCSD method).

The 2RDM is saved to a binary file in MO basis format (CCSD2RDM) written to the scratch directory. If this file is of interest the user should take action to copy it back to the work directory. There is no functionality yet in DIRAC to transform the 2RDM to AO basis.

Default:

2-body reduced density matrix is not formed

Wavefunctions for iterative methods

The CCSD wavefunction is the general default for the code (ground-state energies and expectation values, excited states and response properties).

For ground-state amplitude and Lagrange multiplier calculations other models can be chosen.

.CC2

.CC2

Performs a CC2 calculation instead of the default CCSD. Currently supported only for energies.

Default:

CC2 is not activated

.CCDOUBLES

.CCDOUBLES

Performs a CCD calculation instead of the default CCSD (switch off the contributions of single excitations).

Default:

CCDOUBLES is not activated

.CCiT

.CCiT

Defines CCSDT as the wavefunction for iterative ground-state amplitudes Brandeys2025,mbauto and Lagrange multipliers Fabbro2025 calculations.

.CCiT**.CCiQ**

.CCiQ

Defines CCSDTQ as the wavefunction for iterative ground-state amplitudes Brandeys2025,mbauto and Lagrange multipliers Fabbro2025 calculations.

.CCiQ**.CCiT1a**

.CCiT1a

[!WARNING] This code is experimental.

Defines CCSDT-1a as the wavefunction for iterative grond-state amplitudes. Currently only supported for amplitude code generated with the mbauto tool.

.AUTOScheme

1

.CCiT1a**.CCiT1b**

.CCiT1b

[!WARNING] This code is experimental.

Defines CCSDT-1b as the wavefunction for iterative grond-state amplitudes. Currently only supported for amplitude code generated with the mbauto tool.

.AUTOScheme

1

.CCiT1b**.CCiT2**

.CCiT2

[!WARNING] This code is experimental.

Defines CCSDT-2 as the wavefunction for iterative grond-state amplitudes. Currently only supported for amplitude code generated with the mbauto tool.

.AUTOScheme

1

.CCiT2**.CCiT3**

.CCiT3

[!WARNING] This code is experimental.

Defines CCSDT-3 as the wavefunction for iterative ground-state amplitudes. Currently only supported for amplitude code generated with the mbauto tool.

```
.AUTOScheme
1
.CCiT3
```

Perturbative triples corrections for ground-state energies

```
.NOTRIPLES
```

.NOTRIPLES

Deactivates computation of triples energy corrections. This is useful for ExaTENSOR (as the current implementation is not efficient) and if only excited states or properties are sought.

Default:

CCSD(T) energies are calculated if CCSD wavefunctions are employed.

```
.TRIPL_BLOCK
```

.TRIPL_BLOCK

Defines the number of blocks in the CCSD(T) calculation. Splitting the evaluation of the triples correction to the energy reduces the memory footprint of calculations at the expense of potentially longer execution times.

```
.TRIPL_BLOCK
5
```

Default:

```
.TRIPL_BLOCK
1
```

Virtual space compression and natural orbital generation

[!WARNING] The virtual space compression code requires a two-step workflow: a first calculation to generate one of the flavors for the frozen virtual natural orbitals, and a second one restarting from these orbitals to carry out the CC calculations. For examples please consult the testset.

```
.MP2NO
```

.MP2NO

Perform a closed shell MP2 calculation and generate the frozen natural orbitals (FNOs) Yuan2022 by diagonalization of the virtual-virtual block of the MP2 density matrix, outputting the original occupied orbitals and a set of truncated virtuals in AO basis (the FNOs are transformed into AO basis and saved on file MP2NOs_AO, which has the same structure as DFCOEF.)

The user must specify a threshold indicating a NO occupation number, and orbitals with occupation below that will not be retained in the truncated AO basis.

```
.MP2NO
1.0d-3
```

If one would like to use the new set including the FNOs to do higher-level computation like CC, the MP2NOs_AO should be retrieved from the work directory for the calculation, and used as a standard DFCOEF file in subsequent calculations (currently a run in which FNOs are generated and used in the same post-SCF calculation is not supported).

Apart from MP2NOs_AO, this option also outputs the full, untruncated FNOs space (MO basis) in file NOs_MO. The Fock matrix (FNO basis) is also outputted to file FM_in_NatOrb.

The NOs_MO is provided so that users wishing to change the truncation threshold above don't have to repeat the same MP2 calculation (If present in the scratch directory, the NOs_MO will make the code skip the MP2 calculation).

Default:

MP2FNO is not activated.

```
.MP2NO++
```

.MP2NO++

Performs a calculation to generate perturbed frozen virtual natural orbitals (MP2FNO++) Le2026, based on the perturbation sensitive natural spinors approach Chakraborty2025.

```
.MP2NO++
```

Default:

MP2NO++ is not activated.

`.NO_PROP`

.NO_PROP

Defines which properties will be taken into account in MP2FNO++ calculations. The number of properties considered is followed by their index in the definition of the MO property operations in *PRPTRA.

```
.NO_PROP
3
1 2 3
```

`.NO_WEIGHTED_SUM`

.NO_WEIGHTED_SUM

Defines whether to weight out properties in the generation of MP2FNO++. When enabled,

```
.NO_WEIGHTED_SUM
1
```

the properties will be weighted by the number of properties (1/number of properties defined in NO_PROP). Otherwise, all weights are set to 1.

Default:

All properties used to define the MP2FNO++ have weights set to 1.

`.NO_OMEGA`

.NO_OMEGA

Defines the real and imaginary parts of the perturbing frequency ω used in the MP2FNO++ calculation. For instance, to define a perturbing frequency with 0.1 au for the real part and 0.003 for the imaginary part:

```
.NO_OMEGA
0.100 0.003
```

The default is to carry out MP2FNO++ calculations for static perturbations.

Default:

```
.NO_OMEGA
0.000 0.000
```

`.DONATORB`

.DONATORB

Calculate natural occupation numbers and orbitals in AO (quaternion) basis, from a density matrix in the same basis (such as the one generated by the first-order property code, which saved by default in file CCDENS), and store them in file DFNOSAO.

This option assumes that CCDENS is in the work directory for the calculation.

Default:

DONATORB is not activated.

Laplace transform options

`.MP2LAP`

.MP2LAP

Enables the use of the Laplace transform formalism in MP2 calculations.

Default:

Laplace transform MP2 is disabled

`.MP2_LAP_NO`

.MP2_LAP_NO

Enables the use of the Laplace transform formalism in the generation of MP2 frozen virtual natural orbitals.

Default:

Laplace transform MP2 in FNOs generation is disabled

`.NLAPLACE`

.NLAPLACE

Selects the number of integration points for Laplace transform calculations.

.NLAPLACE
15

Default:

.NLAPLACE
10

.LAP_TRIPL

.LAP_TRIPL

Enables the use of the Laplace transform formalism in CCSD(T) calculations.

CC response theory calculations

.CCCILR

.CCCILR

Calculates the orbital-unrelaxed coupled cluster linear response (LR) function Yuan2024

$\langle A; B \rangle_{\omega_B}$

where A and B represent one-body operators, and ω_B the perturbing frequency associated with B . Each of these operators can correspond to electric or magnetic perturbations, though for the latter it is only possible to carry out calculations in a common gauge origin.

The current implementation can handle both static and frequency-dependent (with either real or complex frequencies) response calculations.

For linear response calculations two wave-function models can be used: the standard linear response CC model (also referred to in the following as CC-CC) and the EOM model (also referred to in the following as CC-CI); we note EOM-LR is an approximation to LR-CC.

.CCCILR
1

where 1 indicates CC-CC type wave-function, and other numbers indicate CC-CI type wave-function.

Apart from this keyword, additional inputs are needed to define a linear response calculation : first users need also to define which operators will be transformed to the MO basis, by specifying the appropriate input for the *PRPTR section of **MOLTRA. Second, the EXACC_.LAMBDA keyword has to be used, to activate the calculation of ground-state Lagrange multipliers.

.CCRASYM

.CCRASYM

Calculates the orbital-unrelaxed coupled cluster linear response (LR) function with the asymmetric formulation Yuan2024 for the CC (CC-CC) approach.

With the asymmetric formulation only the responses to the B perturbation are determined. As such, requiring it can be useful when there are more A perturbations than B ones.

.CCRASYM

The asymmetric formulation, which is not defined for LR-EOM (CC-CI), is disabled by default.

Default:

CCRASYM is deactivated

.CCCIQR

.CCCIQR

Calculates the orbital-unrelaxed coupled cluster quadratic response (QR) function for EOM (QR-EOMCC) Yuan2023 and coupled cluster (QR-CC) Yuan2025. As for linear response, one must have in mind that QR-EOMCC is an approximation to QR-CC

$\langle A; B, C \rangle_{\omega_B, \omega_C}$

As in the case of linear response, the code can currently be used in static and frequency-dependent (with either real or complex frequencies) response calculations.

.CCCIQR
1

1 indicates a QR-CC type calculation, and other numbers indicate a QR-EOMCC type calculation.

The same requirements with respect to transforming operators to MO basis and calculating ground-state Lagrange multipliers as for linear response apply here.

.LRFRE

.LRFRE

Specify ω_B (in atomic unit) in the coupled cluster linear response calculation

.LRFRE
3

0.122,0.124,0.126
0.010,0.010,0.010

The first line contains the number of frequencies to be used. The second line indicates the real part of each frequency. The third line indicates the imaginary part of each frequency.

.QR1FRE

.QR1FRE

Specify ω_B (in atomic unit) in the coupled cluster quadratic response calculation

.QR1FRE
1
0.1
0.0

The setup for ω_B is the same as LRFRE above.

.QR2FRE

.QR2FRE

Specify ω_C (in atomic unit) in the coupled cluster quadratic response calculation

.QR2FRE
1
0.15
0.0

The setup for ω_C is the same as LRFRE above.

.AProp

.AProp

Specify the A operator(s) in linear and quadratic response function calculations. This keyword must always be specified in response calculations.

.AProp
3
1,2,3

The first line contains the number of operators that will be considered as A , and the second line specifies a list of numerical operator indexes corresponding to the order in which these appear in the *PRPTRA section of **MOLTRA.

.BProp

.BProp

Specify the B operator(s) in linear and quadratic response function calculations. This keyword must always be specified in response calculations.

.BProp
3
1,2,3

The input follows the same structure as that of EXACC_.AProp above.

We note that in the case of linear response, the code will calculate response functions for all possible combinations of A , B operators.

.CProp

.CProp

Specify the C operator in quadratic response functions.

.CProp
3
1,2,3

The input follows the same structure as that of EXACC_.AProp above.

We note that the the code will calculate response functions for all possible combinations of A , B and C operators.

.DProp

.DProp

Specify the D operator in two-photon absorption cross-section calculations based on quadratic response, see EXACC_.CCTPA below for details:

.DProp
3
1,2,3

The input follows the same structure as that of EXACC_.AProp above.

.NTA

.NTA

Indicate the number of time-antisymmetric operators in the response function. From this information, the code will determine if the response function is real or imaginary (see for instance Saue2002a pages 390-393).

```
.NTA
1
```

Default:

NTA is 0

This determination is of importance for instance in the case of response functions containing an odd number of time-antisymmetric operators (e.g. optical rotation in the case of linear response, of Verdet constants in the case of quadratic response).

In future releases this determination will be done automatically and this feature will be deprecated.

Example:

The fragment of input below demonstrates the definition of the components of the dynamic ($\omega_B = (0.1, 0.0)$) dipole-dipole polarizability. Note the definition of the transformation to MO of the x, y, z components of the dipole operator via the `*PRPTRA` section of `**MOLTRA` (we have respectively indexes 1, 2 and 3 when defining the A, B operators), and the request for Λ equation calculations for the ground state.

Note that it is very important to define the same range of active orbitals in `**MOLTRA` and in `**EXACC`, though in the former the range is continuous and in the latter it is subdivided into occupied and virtual.

```
**MOLTRA
.ACTIVE
3..10
.NO4IND
.PRPTRA
*PRPTRA
.OPERATOR
XDIPLN
COMFACTOR
1.000
.OPERATOR
YDIPLN
COMFACTOR
1.000
.OPERATOR
ZDIPLN
COMFACTOR
1.000
**EXACC
.OCCUPIED
3..5
.VIRTUAL
6..10
.EXEOM
1
.PRINT
1
.LAMBDA
.CCCILR
20
.LRFRE
1
0.1
0.0
.AProp
3
1,2,3
.BProp
3
1,2,3
*CCDIAG
.CONVERG
1.0d-6
.MAXSIZE
20
.MAXITER
50
```

Note: When EOM or response modules are activated, a `*CCDIAG` section should also be defined under `**EXACC`. This menu allows one to control the parameters of the iterativel solver in EOM or response calculations (see Yuan2024 and Yuan2024b for a discussion of implementation details). The options available can be found under the `*CCDIAG` section from `**RELCC`.

.CCTPA

.CCTPA

Evaluate the coupled-cluster two-photon scattering amplitudes $S_{\{AB,CD\}}$, obtained from quadratic response and EOM-EE left and right eigenvectors:

$$S_{\{AB,CD\}} = T_{\{AB\}}^{\{0f\}}(\omega) T_{\{CD\}}^{\{f0\}}(\omega) = \frac{1}{2} [T_{\{AB\}}^{\{0f\}}(\omega) T_{\{CD\}}^{\{f0\}}(\omega) + (T_{\{CD\}}^{\{0f\}}(\omega) T_{\{AB\}}^{\{f0\}}(\omega) + T_{\{AB\}}^{\{f0\}}(\omega) T_{\{CD\}}^{\{0f\}}(\omega) = \sum_n \left[\frac{\langle f | \hat{A} | n \rangle \langle n | \hat{B} | 0 \rangle}{\omega_n - (\omega + i\gamma)} + \frac{\langle f | \hat{B} | n \rangle \langle n | \hat{A} | 0 \rangle}{\omega_n - (\omega + i\gamma)} \right]$$

```
.CCTPA
20
4
```

where the first line indicates the type of wavefunction (here CC-CI; for CC-CC, which is currently not supported, the number would be equal to one), and the second line specifies the number of the $\langle f |$ obtained from an EOM-EE calculation.

The ω, ω' frequencies do not need to be specified since for this keyword they are both taken to be equal to half of the excitation energy of the target state $\langle f |$:

$$\omega = \omega' = \frac{1}{2}\omega_f$$

Example:

requesting CC-CI (EOM) two-photon scattering amplitudes between the reference state (ground state) and the target state (EOM-EE excited state number 4 out of 5 requested):

```
.LAMBDA
.CCTPA
20
4
.QR1FRE
1
0.0
0.0
.AProp
1
1
.BProp
1
3
.CProp
1
1
.DProp
1
3
*EXEOM
.NROOTS
5
.FLAVOR
EE
```

The ω, ω' frequencies do not need to be specified since for this keyword they are both taken to be equal to half of the excitation energy of the target state $\langle f |$:

$$\omega = \omega' = \frac{1}{2}\omega_f$$

[!WARNING] This code is experimental. At the moment, we only can do calculations for the target state, which is non-degenerate, and r. Only CC-CI (EOM) version works, CC-CC is in development.

Excited state calculation options

.CIS

.CIS

Activates the CIS module to obtain singly excited states Yuan2026. Currently the code will form and diagonalize the full CI singles matrix, so it is not necessary to specify the number of states to calculate.

.CIS

Default:

CIS is not activated.

.CIS(D)

.CIS(D)

Activates the CIS(D) module Yuan2026 to obtain perturbation corrections to a number of CIS excited state energies. The following input requests corrections for the 10 lowest excited states.

```
.CIS(D)
10
```

Default:

CIS(D) is not activated.

.EXEOM

.EXEOM

Activate the equation of motion (EOMCC) module Yuan2024b. This option can be used to obtain excitation energies (EOM-EE), singly ionized (EOM-IP) and electron attached (EOM-EA) states starting from a closed shell reference and at the CCSD level, by solving for the right-hand eigenvalues and eigenvectors of the appropriate similarity-transformed Hamiltonian. Users should specify the number of eom runs.

```
.EXEOM
1
```

Default:

EXEOM is not activated.

*EXEOM

*EXEOM

If the EOM calculation is activated, This menu controls the parameters for excited state calculations with the equation of motion coupled cluster (EOM-CC) theory.

For EOM-CC, currently the implementation supports the excitation energy (EE), single electron attachment (EA) and detachment (IP) models for CCSD wavefunctions only. Furthermore, in the current release is not possible to calculate oscillator strengths.

Note that there is no default, if no options are selected no calculations will be performed.

When EOM or response modules are activated, a *CCDIAG section should also be defined under **EXACC. This menu allows one to control the parameters of the iterative solver in EOM or response calculations (see Yuan2024 and Yuan2024b for a discussion of implementation details). The options available can be found under the *CCDIAG section from **RELCC.

Mandatory keywords

.NROOTS**.NROOTS**

Specify the number of roots in the calculation.

.NROOTS
4**.FLAVOR****.FLAVOR**

Specify the flavor of EOM: EE (excitation energy), EA (single electron attachment energies), and IP (single electron detachment energies) :

. FLAVOR
EE

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/goscip.md

orphan

GOSCIP – COSCI module

General Open Shell CI Program written by Olivier Visser, Ref. Visser1992.

This module allows for small full configuration interaction calculations. It is invoked from within DIRRCI if the namelist GOSCIP is present in the input, or if *RESOLVE is specified when doing open shell HF calculations.

The input is given in namelist form.

Historical note: This program was originally written for the MOLFDIR suite and was later also included in DIRAC.

&GOSCIP

Specify the CI space.

NELEC

Number of electrons (excluding frozen core electrons).

Default:

NELEC=0

NOTE: in the developer's version this line reads as

NELACT

Number of electrons (excluding frozen core electrons).

Default:

NELACT=0

Programmers options**IPRNT**

Print level.

Default:

IPRNT=0

&POPANA

Analyze the CI wave function.

Advanced options**THRESH**

Print only determinants with coefficients higher than THRESH.

Default:

THRESH=1.0D-3

DEGEN

Threshold for when several states are considered to be degenerate.

Default:

DEGEN=1.0D-10

SELPOP

Select only states with relative energies lower than SELPOP.

Default:

SELPOP=1.0D2

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/krci.md

orphan

*KRCI

KRCI*2- and 4-component relativistic GASCI module**

Written by Timo Fleig, based on LUCIA by Jeppe Olsen, parallelization by Stefan Knecht and Hans Joergen Aagaard Jensen.

The KRCI module is a string-based Hamiltonian-direct configuration interaction (CI) program LUCIAREL Fleig2003, Fleig2006 , based on the LUCIA code Olsen1990. It is capable of doing efficient CI computations at arbitrary excitation level, e.g. FCI, SDCI, RASCI, and MRCI using general active spaces. The code is interfaced to molecular integrals obtained in the relativistic 2c- and 4-component framework and uses double-point group symmetry. It is implemented as a full parallel version Knecht2010a .

A central feature of the program is the Generalized Active Space (GAS) concept, in which the underlying total orbital space is subdivided into a basically arbitrary number (save for an upper limit) of subspaces with arbitrary occupation constraints. This is a very general approach to orbital space subdivisions. The program uses DIRAC Kramers pairs from either a closed- or an open-shell calculation in a relativistic two- and four-component formalism (see **HAMILTONIAN).

The technical limitations are roughly set by several 100 million determinants in the CI expansion on PCs and common computing clusters and several billions of determinants on supercomputers with ample memory.

If desired, the program also computes 1-particle densities from optimized CI wave functions from which the natural orbital occupations are printed.

The program can also be used for the computation of various one-electron properties at the GASCI level Knecht2009 .

Alternatively, the CI program GASCIP can also be used in the KRCI module for convergence of CI energies. Many of the other properties are not implemented for GASCIP.

Mandatory keywords

.CI PROGRAM

.CI PROGRAM

specifies the CI module behind KRCI. Two choices: LUCIAREL and GASCIP. *default*: GASCIP

```
.CI PROGRAM
LUCIAREL
```

```
.CIROOTS
```

.CIROOTS

specifies the number of states (roots) Y in irrep X to converge. This keyword may be repeated several times for multi-root multi-symmetry calculations. *default*: one root in symmetry 1.

Example for calculating $Y = 5$ roots in irrep $X = 1$ and $Y = 2$ roots in irrep $X = 3$:

```
::
.CIROOTS 1 5 .CIROOTS 3 2
```

The KRCI module exploits the character tables for the Abelian double groups handled by DIRAC as detailed here, `Symm_handling_corr_level`.

If the calculation is run in **non-linear symmetry** irrep X will be fermion irrep no. X for odd number of electrons and boson irrep no. X for even number of electrons.

If you run the calculation in **linear symmetry** you have to specify the **2 x OMEGA** value of the state to optimize on, and the .CI PROGRAM must be LUCIAREL. The doubling (2 x) stems from the fact that we want to avoid non-integer input, e.g. in case of an odd number of electrons we might have $\Omega = 1/2, 3/2, 5/2$, etc. values and the corresponding input for one root with the $\Omega = 1/2$ would then read as

```
.CIROOTS
1 1
```

If we have a system with an even number of electrons and inversion symmetry the input for two $\Omega = 2g$ states read as

```
.CIROOTS
4g 2
```

```
.GAS SHELLS
```

.GAS SHELLS

Specification of CI calculation with electron distribution in orbital (GAS) spaces. The first line contains the number of GA spaces to be used (1-7), followed by one line per GAS with a separation by a “/” of the min/max number of electrons in each GAS and the number of orbitals per fermion corep (either one (no inversion symmetry) or two (inversion symmetry: *gerade ungerade*) entries per line).

The first entry before the “/” gives the minimal number of accumulated (!) electrons after consideration of this GAS, the second the corresponding maximum number, separated by blanks. The minimum and maximum accumulated occupations allow for a very flexible parameterization of the wave function. All determinants fulfilling the occupation constraints will be constructed. The second entry after the “/” gives the number of orbitals per fermion corep. See also the open-shell input in *SCF which is similar to the syntax used here. The design of a GAS scheme is non-trivial and should be motivated by the electronic structure of the system (e.g. inner core, outer core, valence, virtual space). Sometimes it is useful to subdivide the valence space, for scientific reasons, or/and the virtual space, for technical reasons (save core memory). See reference Fleig2006a, pp. 27 for more details. example for 4 GAS spaces with an advanced excitation pattern:

```
.GAS SHELLS
4
1 2 / 1 0
6 8 / 1 2
14 16 / 7 7
16 16 / 30 24
```

if all remaining orbitals (fullCI) should be included in the last GAS space, use:

```
.GAS SHELLS
4
1 2 / 1 0
6 8 / 1 2
14 16 / 7 7
16 16 / all
```

```
.INACTIVE
```

.INACTIVE

Inactive orbitals per fermion corep. *default*: all orbitals active. example for 4 *gerade* and 2 *ungerade* orbitals frozen in a molecule with inversion center:

```
.INACTIVE
4 2
```

Optional keywords

```
.NOOCCN
```

.NOOCCN

compute natural orbital occupation numbers for each electronic state. *default*: do not compute natural orbital occupation numbers.

```
.ANALYZ
```

.ANALYZ

analyze the final CI wave function printing the coefficients for each determinant above a given threshold 10^{-2} . *default*: do not analyze the final CI wave function.

.THRSCI

.THRSCI

set CI convergence threshold based on the norm of the CI gradient (aka CI residual). *Default* is 2.0×10^{-3} for energy calculations, giving ca. 0.01 mHartree accuracy, but only 2-3 digits in properties. *Default* is 1.0×10^{-4} if properties have been requested, giving ca. 4 digits in properties and ca. 8 decimal places in energies.

```
.THRSCI
1.0e-3
```

.MAX CI

.MAX CI

maximum number of CI iterations. *default*:

```
.MAX CI
40
```

.MXCIVE

.MXCIVE

maximum size of Davidson subspace in LUCIAREL (not used in GASCIP). *default*: 3 times the number of eigenstates (see KRCI_.CIR00TS) to optimize on. example:

```
.MXCIVE
24
```

.RSTRCI

.RSTRCI

Restart CI from vector(s) on file KRCI_CVECS.x where **x** is determined by the symmetry of the wave function.

```
.RSTRCI
1
```

default: no restart.

```
.RSTRCI
0
```

further infomation: the convention in linear symmetry for KRCI_CVECS.x is the following where the offset/range for systems with a gerade number of electrons is for x: offset $\rightarrow$ 1; range: x = [1-64] and with an ungerade number of electrons x: offset $\rightarrow$ 65; range: x = [65-128]. In more detail (with MJ == Ω):

1. for systems with a gerade number of e- and no inversion center:

```
MJ = 0: KRCI_CVECS.1
MJ = +1: KRCI_CVECS.2
MJ = -1: KRCI_CVECS.3
MJ = +2: KRCI_CVECS.4
MJ = -2: KRCI_CVECS.5
...
up to
MJ = +32: KRCI_CVECS.64
```

2. for systems with a gerade number of e- and an inversion center:

```
MJ = 0g: KRCI_CVECS.1
MJ = +1g: KRCI_CVECS.2
MJ = -1g: KRCI_CVECS.3
MJ = +2g: KRCI_CVECS.4
MJ = -2g: KRCI_CVECS.5
...
up to
MJ = +16g: KRCI_CVECS.32
```

```
and
MJ = 0u: KRCI_CVECS.33
MJ = +1u: KRCI_CVECS.34
MJ = -1u: KRCI_CVECS.35
MJ = +2u: KRCI_CVECS.36
MJ = -2u: KRCI_CVECS.37
...
up to
MJ = +16u: KRCI_CVECS.64
```

3. for systems with an ungerade number of e- and no inversion center:

```
MJ = +1/2: KRCI_CVECS.65
MJ = -1/2: KRCI_CVECS.66
MJ = +3/2: KRCI_CVECS.67
MJ = -3/2: KRCI_CVECS.68
...
```

```
up to
MJ = +63/2: KRCI_CVECS.128
```

4. for systems with an ungerade number of e- and an inversion center:

```
MJ = +1/2g: KRCI_CVECS.65
MJ = -1/2g: KRCI_CVECS.66
MJ = +3/2g: KRCI_CVECS.67
MJ = -3/2g: KRCI_CVECS.68
```

```
...
up to
MJ = -31/2g: KRCI_CVECS.96
```

and

```
MJ = +1/2u: KRCI_CVECS.97
MJ = -1/2u: KRCI_CVECS.98
MJ = +3/2u: KRCI_CVECS.99
MJ = -3/2u: KRCI_CVECS.100
```

```
...
up to
MJ = -31/u: KRCI_CVECS.128
```

.CHECKP

.CHECKP

enables a check point write of the current solution vectors to the file KRCI_CVECS.x (see above in KRCI_.CIR00TS for an explanation of how **x** is supposed to be replaced) during the Davidson iterations. A checkpoint file will be written roughly every 6th iteration. *default*: do not write check points.

Advanced options

.IJKLRO

.IJKLRO

enables the storage of the resorted two-electron integrals on file IJKL_REOD which can then be read-in in a restart. This avoids a multiple reading from the 4IND* files and subsequent resorting. The keyword has to be present also in the restart to enable the potential read-in procedure. Hint: This keyword may be combined with KRCI_.MAX CI == 0 in a precedent step in order to save memory for the actual production run. This is due to the fact that the resorting step itself requires the in-core storage of the unsorted and sorted integrals. *default*: do not write the resorted integrals to file IJKL_REOD.

.IJKLSP

.IJKLSP

enables the splitting of the resorted integrals among the co-workers according to their needs in the computation of the sigma vector. It can only be used in combination with KRCI_.IJKLRO. *default*: do no split the resorted integrals among the co-workers.

KRCI properties

This part refers to the 2- and 4-component relativistic KR-CI property module written by Stefan Knecht and Hans Joergen Aa. Jensen, parallelization by Stefan Knecht

The KR-CI property module Knecht2009 takes advantage of the 2- and 4-component KRCI module. It can be used for the computation of:

- permanent dipole moments in ground and excited states,
- transition dipole moments
- computation of $\langle \hat{1}_z \rangle$, $\langle \hat{s}_z \rangle$ and $\langle \hat{j}_z \rangle$ (Ω) expectation values (only for linear molecules).

Upon request the analysis of other one-electron properties may also be implemented. The module could in principle be used for each one-electron operator that is specified in one_electron_operators.

.OPERATOR

.OPERATOR

Further modification by Malaya K. Nayak for the proper functioning of .OPERATOR keyword

Here is some specific examples of one_electron_operators using .OPERATOR keyword.

For the electron electric dipole moment (eEDM) effective electric field one can use

```
.OPERATOR
'A2-EDM'
iBETAGAM
EDM
```

The nucleus-electron scalar-pseudoscalar (Ne-SPS) interaction in BeH can be defined as

```
.OPERATOR
'A1-SPS'
iBETAGAM
'PVCBe 01'      for the first atom (as specied in .mol file) Be in BeH molecule
.OPERATOR
'A2-SPS'
```

```
iBETAGAM
'PVCH 02'          for the second atom (as specied in .mol file) H in BeH molecule
```

Similarly, the magnetic hyperfine-structure constants in BeH can be defined as follows

```
.OPERATOR
'X1-HYP'          (X1-HYP, Y1-HYP and Z1-HYP are for the first atom as specied in .mol file)
XAVECTOR
'NEF 001'
'NEF 005'
.OPERATOR
'Y1-HYP'
YAVECTOR
'NEF 003'
'NEF 001'
.OPERATOR
'Z1-HYP'
ZAVECTOR
'NEF 005'
'NEF 003'
.OPERATOR
'X2-HYP'          (X2-HYP, Y2-HYP and Z2-HYP are for second atom as specied in the .mol file)
XAVECTOR
'NEF 002'
'NEF 006'
.OPERATOR
'Y2-HYP'
YAVECTOR
'NEF 004'
'NEF 002'
.OPERATOR
'Z2-HYP'
ZAVECTOR
'NEF 006'
'NEF 004'
```

The nuclear Magnetic-Quadrupole-Moment (MQM) constants in BeH can be defined by defination as

```
.OPERATOR
'Z1-MQM'          (Z1-MQM, for the first atom as specified in the .mol file)
ZAVECTOR
YZEFG011
XZEFG011
COMFACTOR
-0.333333333D0
.OPERATOR
'Z2-MQM'          (Z2-MQM, for the second atom as specified in the .mol file)
ZAVECTOR
YZEFG021
XZEFG021
COMFACTOR
-0.333333333D0
```

One can define many more one_electron_operators as per their requirements.

.DIPMOM

.DIPMOM

compute the permanent dipole moments in electronic ground and excited states.

.TRDM

.TRDM

compute the transition dipole moments between electronic states.

.OMEGAQ

.OMEGAQ

Note: this keyword may only be used for linear molecules.

compute the expectation values of the spin- and angular momentum operator in z-direction and print the total $\hat{j}_z = \hat{s}_z + \hat{l}_z$ expectation value (Ω value) for each electronic state.

.MHYP

.MHYP

compute magnetic hyperfine interaction constants as expectation values over Fermi's four-component operator (Z. Phys. 60 (1930) 332). Example for the hydrogen atom (proton nucleus):

```
.MHYP 0.5 nuclear spin quantum number I 2.793 nuclear magnetic moment
```

In the molecular case add a column with I and μ for every atom in the order given by the basis file. Reference: Fleig2014

.EEDM

.EEDM

compute electron electric dipole moment effective electric field as an expectation value over the effective one-electron momentum-form operator (J. Phys. B: At. Mol. Opt. Phys. 22 (1989) 559, stratagem II). Reference: Fleig2013

.ENSPS

.ENSPS

compute nucleon-electron scalar-pseudoscalar interaction constant as an expectation value over the corresponding effective four-fermion operator in the limit for an infinitely heavy nucleon (Sov. Phys. JETP 62 (1985) 872). Reference: Fleig2015

.NMQM

.NMQM

compute nuclear Magnetic-Quadrupole-Moment (MQM) Interaction constant as an expectation value over the corresponding effective four-component operator (Sov. Phys. JETP 60 (1984) 873), Reference: Fleig2016

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/krmcscf.md

orphan

*KRMCSF

*KRMCSF

2- and 4-component relativistic KR-MCSCF module

The original implementation written by Joern Thyssen, Timo Fleig and Hans Joergen Aagaard Jensen, parallelization and continuous improvement by Stefan Knecht and Hans Joergen Aagaard Jensen. Linear symmetry adaptation by Stefan Knecht and Hans Joergen Aagaard Jensen.

For more details on the theory and actual implementation see Jensen1996, Thyssen2004, Thyssen2008, Knecht2009.

Mandatory keywords under *KRMCSF

.GAS SHELLS

.GAS SHELLS

Specification of CI calculation with electron distribution in orbital (GAS) spaces. The first line contains the number of GA spaces to be used (1-7), followed by one line per GAS with a separation by a “/” of the min/max number of electrons in each GAS and the number of orbitals per fermion corep (either one (no inversion symmetry) or two (inversion symmetry: *gerade ungerade*) entries per line).

The first entry before the “/” gives the minimal number of accumulated (!) electrons after consideration of this GAS, the second the corresponding maximum number, separated by blanks. The minimum and maximum accumulated occupations allow for a very flexible parameterization of the wave function. All determinants fulfilling the occupation constraints will be constructed. The second entry after the “/” gives the number of orbitals per fermion corep. See also the open-shell input in *SCF which is similar to the syntax used here. The design of a GAS scheme is non-trivial and should be motivated by the electronic structure of the system (e.g. inner core, outer core, valence, virtual space). Sometimes it is useful to subdivide the valence space, for scientific reasons, or/and the virtual space, for technical reasons (save core memory). See reference Fleig2006a, pp. 27 for more details.

Example for a CASSCF run with 10 electrons in 10 orbitals, also known as CAS(10,10):

```
.GAS SHELLS
1
10 10 / 5 5
```

if all orbitals (fullMCSCF, feasible only for systems with 2-3 active electrons) should be included, use:

```
.GAS SHELLS
1
2 2 / all
```

.INACTIVE

.INACTIVE

Inactive orbitals per fermion corep. *default*: all orbitals are active. The example below for a molecule with inversion center marks the lowest 4 *gerade* and 2 *ungerade* orbitals as inactive, i.e. they are always kept doubly occupied in all determinants of the CI expansion:

```
.INACTIVE
4 2
```

All remaining orbitals above the inactive + active space are considered as secondary orbitals, i.e. they are kept empty in all determinants of the CI expansion. By default all orbital rotations between the inactive-active and active-secondary space are included.

Optional keywords under *KRMCSF

.CI PROGRAM

.CI PROGRAM

specifies the CI module behind KRMCSF, choices are *LUCIAREL* and *GASCIP*. *Note*: GASCIP does not work with linear symmetry and for large-scale MCSCF ($> 1.0 \times 10^7$ determinants) only LUCIAREL can be used efficiently. *default*:

```
.CI PROGRAM
GASCIP
```

.THRESH

.THRESH

convergence threshold in the MCSCF gradient. The present default value is quite tight and might be adapted if one is only interested in MCSCF energies, e.g. $1.e-03$. *default*:

```
0.5e-05
```

.SYMMETRY

.SYMMETRY

integer value giving the (boson/fermion) symmetry of the state to optimize on. *default*:

```
1
```

If you run the calculation in **linear symmetry** you have to specify $2 \times \Omega$ value of the state to optimize on and use LUCIAREL as CI PROGRAM. The doubling stems from the fact that we want to avoid non-integer input, e.g. in case of an odd number of electrons we might have $\Omega = 1/2, 3/2, 5/2$, etc. values and the corresponding input for the $\Omega = 1/2$ would then read as

```
1
```

If we have a system with an even number of electrons and inversion symmetry the input for an $\Omega = 2g$ state would read as

```
4g
```

.MAX MACRO

.MAX MACRO

integer value giving the maximum number of MACRO iterations in the MCSCF optimization. *default*:

```
25
```

.MAX MICRO

.MAX MICRO

integer value giving the maximum number of MICRO iterations in the MCSCF optimization. *default*:

```
50
```

.FROZEN

.FROZEN

specifies how many of the inactive orbitals for each fermion corep should be *frozen*, that is, not change during the KRMCSF optimization. The frozen orbitals are not included in 2-electron integral transformation, which saves some CPU time. *default*: include all inactive orbital rotations. *example*: Freeze the first two inactive orbitals in fermion corep 1 (*gerade*) and the first inactive orbital in fermion corep 2 (*ungerade*):

```
.FROZEN
2 1
```

.DELETE

.DELETE

delete occupied-secondary e-e rotations (rotations between electronic-electronic spinors) in the gradient and Hessian calculation specified by an orbital string of virtual (secondary) orbitals for each fermion corep. *default*: include all active-secondary e-e rotations. *example*: Delete all e-e rotations for secondary orbitals 20,21 and 22 in fermion corep 1 (*gerade*) and 24,25,26,27 and 28 in fermion corep 2 (*ungerade*):

```
.DELETE
20,21,22
24..28
```

.SKIPEE

.SKIPEE

skip e-e rotations (rotations between electronic-electronic spinors) in the gradient and Hessian calculation. *default*: include e-e rotations.

.WITHEP

.WITHEP

include e-p rotations (rotations between electronic-positronic spinors) in the gradient and Hessian calculation. *default*: skip e-p rotations.

.SKIPEP

.SKIPEP

skip e-p rotations (rotations between electronic-positronic spinors) in the gradient and Hessian calculation. *default*: skip e-p rotations.

.MVOFAC

.MVOFAC

generate modified inactive and virtual orbitals based on block-diagonalization of FC + “MVOFAC”*FV. The value does not change the MCSCF wave functions, but the non-active orbitals for post-MCSCF calculations (e.g. *LUCITA or *KRCI) are modified. *default*: MVOFAC = 1.0 *example*: Mimic Bauchschrlicher’s MVOs for HF-CI for the MCSCF wave function:

```
.MVOFAC
0.0
```

.PRINT

.PRINT

raise default print level to the given integer value. Please use with care as you may get millions of output lines if you choose a too high value. *default*:

```
.PRINT
0
```

*OPTIMI

***OPTIMI**

optional keywords under *OPTIMI

NOTE: the following keywords must be placed under the input deck

*OPTIMI

.NOOCCN

.NOOCCN

compute natural orbital occupation numbers for the final optimized electronic state. *default*: do not compute natural orbital occupation numbers.

.ANALYZ

.ANALYZ

analyze the final CI wave function printing the coefficients for each determinant above a given threshold 10^{-2} . *default*: do not analyze the final CI wave function.

.MAX CI

.MAX CI

maximum number of initial CI iterations. *default*:

```
.MAX CI
5
```

.MXCIVE

.MXCIVE

maximum size of Davidson subspace. *default*: 3 times the number of eigenstates (see KRCI_.CIR00TS) to optimize on. example:

```
.MXCIVE
3
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/laplce.md

orphan

*LAPLCE

*LAPLCE

Introduction

Laplace transformation module by B. Helmich-Paris.

The Laplace transform of orbital energies denominators can evaluated numerically by quadrature procedures. This is an interesting reformulation for perturbation-theory based approaches like Green's functions methods, Møller-Plesset perturbation theory etc. Here the minimax algorithm is employed, which is very fast and gives also a good accuracy for vanishing HOMO-LUMO gaps. See also Ref. Takatsuka2008 and HelmichParis2016 for more details.

Keywords

.PRINT

.PRINT

Print level.

Default:

.PRINT
0

.NUMPTS

.NUMPTS

A fixed number of quadrature points given by the user. If not given the number of quadrature points to reach a given accuracy TOLLAP is estimated.

Default:

.NUMPTS
0

.MXITER

.MXITER

Maximum number iterations in the Newton-Maehly algorithm that searches for roots and extrema of the numerical error distribution.

Default:

.MXITER
100

.STEPMX

.STEPMX

Maximum step length of a line search part of the Newton-Maehly algorithm when far away from quadratic convergence region.

Default:

.STEPMX
0.3

.TOLRNG

.TOLRNG

Tolerance threshold for the Newton-Maehly procedure that determines the extremum points. You should not touch that.

.TOLRNG
1.D-10

.TOLPAR

.TOLPAR

Tolerance threshold for the Newton procedure that computes the Laplace parameters at each extremum point. You should not touch that.

.TOLPAR
1.D-15

.XPNTS

.XPNTS

You can start the iterative quadrature algorithm with some guess for the exponents. This only works if you are close to the final exponents. Also you have to provide the weights (WGHTS) and the number of quadrature points (NUMPTS).

.WGHTS

.WGHTS

You can start the iterative quadrature algorithm with some guess for the weights. This only works if you are close to the final weights. Also you have to provide the exponents (XPNTS) and the number of quadrature points (NUMPTS).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/lucita.md

orphan

*LUCITA

***LUCITA**

Spin-free relativistic GASCI module written by Jeppe Olsen, adaptation to DIRAC by Timo Fleig, parallelization by Stefan Knecht and Hans Joergen Aagaard Jensen.

LUCITA is a string-based Hamiltonian-direct configuration interaction (CI) program, based on the LUCIA code Olsen1990. It is capable of doing efficient CI computations at arbitrary excitation level, e.g. FCI, SDCI, RASCI, and MRCI using general active spaces. The code is interfaced Fleig2005 to molecular integrals obtained in the spin-free Dirac formalism and uses non-relativistic point group symmetry. It is implemented as a full parallel version Knecht2008.

A central feature of the program is the Generalized Active Space (GAS) concept, in which the underlying total orbital space is subdivided into a basically arbitrary number (save for an upper limit) of subspaces with arbitrary occupation constraints. This is the most general approach to orbital space subdivisions. The program uses DIRAC orbitals from either a closed- or an open-shell calculation in a spin-free relativistic formalism or the non-relativistic Lévy-Leblond formalism (see ****HAMILTONIAN**).

The technical limitations are roughly set by several 100 million determinants in the CI expansion on PCs and common computing clusters and several billions of determinants on supercomputers with ample memory. The program also computes 1- and, if requested 2-particle densities from optimized CI wave functions. The density matrices may be printed along with natural orbital occupations and the corresponding eigenvectors (NOs).

General information

Please add the lines

```
.SCHEME
4
```

to the ****MOLTRA** input section for the integral transformation.

Mandatory keywords

.INIWFC

.INIWFC

specifies the initial SCF wave function.

closed-shell:

```
. INIWFC
DHFSCF
```

open-shell:

```
. INIWFC
OSHSCF
```

.CITYPE

.CITYPE

Type of CI calculation. Typical multi-reference CI calculations should be defined by RASCI or GASCI.

valid choices are: FCI, SDCI, SDTQ, RASCI, GASCI. If you choose RASCI or GASCI please take a look at the *RASCI/GASCI* keyword section below.

example:

```
.CITYPE
FCI
```

.MULTIP

.MULTIP

specifies the target state spin multiplicity ($2S+1$) of the desired eigenstates.

example for singlet state(s) ($S=0$):

```
.MULTIP
1
```

Optional keywords

```
.DENSI
```

.DENSI

Level of computed density matrices. *default:* one-electron density matrices and natural orbital occupation numbers. To compute one- and two-particle density matrices:

```
.DENSI
2
```

```
.NROOTS
```

.NROOTS

Number of states(roots) to optimize. *default:*

```
.NROOTS
1
```

```
.SYMMET
```

.SYMMET

Spatial symmetry of the desired state(s). This is the boson symmetry label referring to an irrep ordering as defined by the group generators. *default:*

```
.SYMMET
1
```

```
.MAXITR
```

.MAXITR

maximum number of CI iterations. *default:*

```
.MAXITR
100
```

```
.INACTI
```

.INACTI

Inactive orbitals per boson symmetry, separated by commas. This keyword is not allowed in connection with the citype choice LUCITA_ .CITYPE GASCI. *default:* all orbitals active. example for D_{2h} :

```
.INACTI
1,1,0,0,1,0,0,2
```

```
.RSTRCI
```

.RSTRCI

Restart CI from vector(s) on file LUCVECT resp. LUCVECT.0 (parallel calculation; until including DIRAC v13.1):

```
.RSTRCI
1
```

default: no restart.

```
.RSTRCI
0
```

Convergence threshold for energy (double-precision value):

```
.CONVER
x.xd-0x
```

default: 1.0d-08

```
.CONVER
1.0d-08
```

```
*LUCITA GASCI
```

*LUCITA GASCI

Specific input – GASCI

.NACTEL

.NACTEL

Number of active electrons. *default*: none. example:

```
.NACTEL
10
```

.GASSHE

.GASSHE

Number and specification of GAS orbitals. Line with the number of GA spaces used (1-7), followed by one line per GAS with number of orbitals per boson symmetry, separated by commas. *default*: none. example for 2 GAS spaces and active orbital distribution when running in the D_{2h} symmetry:

```
.GASSHE
2
2,0,4,4,8,2,1,1
8,2,6,6,19,5,3,3
```

.GASSPC

.GASSPC

Number and specification of sequential CI calculations. Line with the number of CI calculations with given GA spaces (currently only 1 is allowed), followed by one line per GAS with 2 numbers each: The first gives the minimal number of accumulated electrons after this GAS, the second the corresponding maximum number, separated by blanks (defining occupation constraints of each GAS). *default*: None. example for an excitation pattern in connection with the 2-GAS input from LUCITA_GASCI_, GASSHE above and 10 active electrons:

```
.GASSPC
1
8 10
10 10
```

*LUCITA RASCI

*LUCITA RASCI

Specific input – RASCI

.NACTEL

.NACTEL

Number of active electrons. *default*: none. example:

```
.NACTEL
10
```

.RAS1

.RAS1

RAS1 specification and maximum number of holes. Line with orbitals per boson symmetry, separated by commas, followed by a line with the maximum number of holes in RAS1. *default*: none. Example for an active orbital distribution when running in the D_{2h} symmetry and max 2 holes:

```
.RAS1
2,0,4,4,2,2,1,1
2
```

.RAS2

.RAS2

RAS2 specification. Line with orbitals per boson symmetry, separated by commas. Example for an active orbital distribution when running in the D_{2h} symmetry:

```
.RAS2
1,0,1,1,1,1,0,1
```

.RAS3

.RAS3

RAS3 specification and maximum number of electrons. Line with orbitals per boson symmetry, separated by commas, followed by a line with the maximum number of electrons in RAS3. Example for an active orbital distribution when running in the D $_{2h}^{\infty}$ symmetry and max 2 electrons in RAS3:

```
.RAS3
2,0,4,4,2,2,1,1
2
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/mp2cal.md

orphan

*MP2CAL

*MP2CAL

Introduction

This section gives directives for the integral-direct closed-shell second-order Møller-Plesset perturbation theory (MP2) calculation. The default algorithm is described [here](#) and is a modification of the original algorithm published in Ref. Laerdahl1997.

Note that default is to include all positive-energy (electronic) orbitals in the MP2 calculation. You specify .OCCUPIED and .VIRTUAL if you want to do the MP2 calculation with a subset of these orbitals.

Basic keywords

.OCCUPIED

.OCCUPIED

Specify the active subset of the doubly occupied positive-energy (electronic) orbitals from the SCF.

For each fermion irrep, give an `orbital_strings` of active occupied orbitals.

Default: All the occupied electronic orbitals.

.VIRTUAL

.VIRTUAL

Specify the active subset of the empty virtual orbitals from the SCF.

For each fermion irrep, give an `orbital_strings` of active virtual orbitals.

Default: All the unoccupied virtual orbitals.

Advanced options

.INTFLG

.INTFLG

Specify what two-electron integrals to include in the direct MP2 calculation (default: HAMILTONIAN_.INTFLG under **HAMILTONIAN).

.SCREEN

.SCREEN

Screening threshold in the 4-index transformation.

A negative number deactivates screening.

Default:

```
.SCREEN
1.0D-14
```

Programmers options

.PRINT

.PRINT

Print level.

*Default:***.PRINT**
0**.VIRTHR****.VIRTHR**

Threshold for neglecting high-lying virtual orbitals. Neglect all orbitals with energy above this threshold. This option is retained for backwards compatibility, it is advised to use the more general energy option in the `orbital_strings` input.

Arguments: Real DMP2_VIRTHR.

Default: No threshold.

.IJTSK**.IJTSK**

Max. number of I (or J) orbitals (occupied orbitals) in each calculation batch. Usually one batch is sufficient but larger calculations may be possible by reducing the batch size. Reducing the batch size to half will approximately reduce the memory requirements to 1/4 and increase the CPU time by a factor 4.

Arguments: Integer IJTSK.

Default: All active occupied electronic solutions.

.SCLMEM**.SCLMEM**

Internal memory available for scalar untransformed integrals. Larger batches will be written to disk.

Arguments: Integer MAXSCL.

Default: Set by program and is usually sufficient.

.ORGALG**.ORGALG**

Use original algorithm as described in Ref. Laerdahl1997.

Default: Use [new algorithm](#).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/mp2no.md

orphan

*MP2 NO

*MP2 NO

Introduction

MP2 natural orbitals module by S. Knecht and H. J. Aa. Jensen.

The MP2 calculation will produce the MP2 energy and the natural orbitals for the density matrix through second order. The primary purpose of this option is to generate good starting orbitals for CI or MCSCF but also CC wave functions, but it may of course also be used to obtain the MP2 energy, perhaps with frozen core orbitals. For valence correlation calculations it is recommended that the core orbitals are excluded via the `MP2_NO_.ACTIVE` in order to obtain the appropriate correlating orbitals as start for an MCSCF calculation. As the commonly used basis sets do not contain correlating orbitals for the core orbitals and as the core correlation energy therefore becomes arbitrary, a thoughtfully chosen `MP2_NO_.ACTIVE` option can also be of benefit in MP2 energy calculations. See also Ref. Jensen1988 for more details.

Note: the module works at present only for closed-shell Hartree-Fock reference wave functions.

Keywords

.PRINT**.PRINT**

Print level.

Default:

```
.PRINT
0
```

```
.ACTIVE
```

.ACTIVE

Read orbital_strings of orbitals to be included in the MP2 calculation when constructing the MP2 density and natural orbitals.

```
.SEL NO
```

.SEL NO

Select a minimal set of natural orbitals given as a string analog to the orbital_strings defined as active orbitals in subsequent correlation calculations. The numbers are: max. occupation, min occupation, safety tolerance (here +-10%).

Default:

```
.SEL NO
NO-occ 1.99 0.001 0.1
```

```
.SCHEME
```

.SCHEME

Sets the integral transformation scheme for the **MOLTRA part. *Default:* default scheme of MOLTRA (scheme 6):

```
.SCHEME
6
```

```
.MULPOP
```

.MULPOP

Perform a Mulliken population analysis (see MP2_NO_.MULPOP) of the MP2 natural orbitals. This can be useful for a comparison with a Mulliken population analysis of the Hartree-Fock orbitals.

```
.INTFLG
```

.INTFLG

Specify what two-electron integrals to include during the construction of the MP2 natural orbitals (default: HAMILTONIAN_.INTFLG under **HAMILTONIAN).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/mvocal.md

orphan

```
*MVOCAL
```

***MVOCAL**

Introduction

Modified virtual orbitals.

The virtual canonical Hartree-Fock orbitals are not the optimal choice for correlated calculations in which the virtual space is truncated; the lower orbitals tend to be diffuse since they see the $N + 1$ electron system. One may exploit the freedom of rotation amongst the virtual orbitals to obtain orbitals better suited for correlated calculations. The default option in DIRAC when WAVE_FUNCTION_.MVO has been specified is to generate the modified virtual orbitals by construction of a Fock operator for the system with some electrons removed, as first proposed by Bauschlicher Bauschlicher1980. The orbital string specified with MVOCAL_.VECMVO specifies which occupied orbitals to include in the construction of the modified Hartree-Fock potential for the virtual orbitals.

Note that the resulting modified virtual orbitals are no longer canonical and can *not* be used in MP2 calculations; they can be used in the various CI and CC modules, and as start orbitals for *KRMSCF.

Keywords

```
.PRINT
```

.PRINT

Print level.

Default:

```
.PRINT
0
```

.VECMVO

.VECMVO

Read orbital_strings of orbitals to be included in the Hartree-Fock potential when constructing the cationic Fock operator. Default is the doubly-occupied molecular orbitals from the SCF.

.INTFLG

.INTFLG

Specify what two-electron integrals to include during the construction of the modified virtual orbitals (default: HAMILTONIAN_ .INTFLG under **HAMILTONIAN).

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/polprp.md

orphan

Kramers restricted Polarization Propagator in the ADC framework

See Pernpointner2014 for the initial POLPRP implementation. The transition moment implementation and parallelization is described in Pernpointner2017.

**POLPRP

****POLPRP**

Specification of the overall calculation. Davidson diagonalizer (see DAVIDSON section) is automatically invoked.

.STATES

.STATES

Specifies the symmetries of the excited final states to be calculated. If you leave out this keyword all symmetries are checked for the occurrence of possible final states and those are calculated in the respective perturbational order. If the keyword is present, states have to be specified according to the following input example

Input example:

```
.STATES
4
1,3,17,19
```

This determines calculation for all eigenstates in the symmetries (irreducible representations) 1,3,17 and 19. Keep in mind that the numbering scheme for the double group ireps is a bit unconventional and depends on the type of Hamiltonian used.

.DOEXTE

.DOEXTE

Presence of this keyword triggers an extended ADC(2) calculation in the sense that first-order contributions are included in the 2p2h (satellite) block. This often helps to localize states having a sizeable amount of double excitation character because those will alter their energy considerably when calculated in the extended scheme.

.WRITET

.WRITET

Determines the threshold for writing matrix elements into the ADC matrix (default = 0.0). Can save considerable numerical effort with negligible loss of accuracy. Thresholds of 1.0E-06 are in order but it also depends a bit on the system. Value follows the keyword.

.NODIAG

.NODIAG

Diagonalization is skipped if keyword is present and only the eigenvectors of the particle-hole block are written to disk. This is mainly for test purposes since getting full eigenvectors is required for the calculation of transition moments.

.DOTRMO

.DOTRMO

Transition moments are calculated if keyword is present, otherwise no TMs are calculated.

.PRINT

.PRINT

Printlevel (default = 0) number follows the keyword. More output is produced if print level > 0.

****DAVIDSON**

****DAVIDSON**

Specifications for the Davidson diagonalizer. Holds for POLPRP and for other propagators if activated. POLPRP relies on Davidson exclusively and does not address other diagonalizers such as Lanczos.

.DVROOTS

.DVROOTS

Number of desired roots. Remember that Davidson is an iterative diagonalizer that will not be able to generate thousands of roots. Normally one asks for 10 to 50 roots at the spectral boundary that will be well converged. This is a typical situation occurring for the valence excitation spectra.

.DVMAXSP

.DVMAXSP

This is the maximum size of the Krylov space generated internally and where the full Hamiltonian is projected on, in other words, this is the dimension of your trial space. If the dimension is reached during the iterations one macroiteration cycle is finalized and the subspace will be collapsed to the number of roots. The resulting new set of vectors is already a much better approximation to the true eigenvectors entering the next macrocycle.

.DVMAXIT

.DVMAXIT

Number of macrocycles you want to allow. For large problems requiring a lot of memory, DVMAXSP can be reduced leading to more macrocycles. If convergence is still unsatisfactory one slowly increases DVMAXSP.

.DVCONV

.DVCONV

Requested convergence of the eigenvalues. Usually a convergence of 1.0E-06 is absolutely sufficient.

.DVREORT

.DVREORT

Force reorthogonalization of ADCEVEC.XX start vectors initially and in each macrocycle. This may be useful if eigenvectors of a previous incomplete run that are not perfectly orthogonal shall be reused. This is unnecessary in a regular run since the trial space during a Davidson run is kept strictly orthogonal and serves as the construction space for the true eigenvectors of the Hamilton matrix.

.SKIPCC

.SKIPCC

With this keyword you can restart the calculation at a very advanced stage after the sorting step of the two-electron integrals. Therefore you skip SCF, MOLTRA and integral sorting (OOOO, VOOO, ...). This is if a POLPRP run only needs to be executed. Works in serial and parallel. The only files you should provide are the ftXX.YY files containing the sorted integral batches, MDPROP (needed for transition moments) and MRCONEE among the .inp and .mol file. In parallel run this works only if you have an underlying shared file system, so each node can access their own ftXX.nodenum file.

The default values are

```
::
DVROOTS 5 DVMAXSP 200 DVMAXIT 5 DVCONV 1.0E-05 DVREORT = .false. SKIPCC = .false.
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/reladc.md

orphan

RELADC, FANOADC and LANCZOS

This section lists the available keywords for RELADC, FanoADC and the closely connected iterative LANCZOS diagonalizer in DIRAC.

If you want to perform RELADC/LANCZOS calculations for the one- and two-particle propagator or a FanoADC calculation you invoke all of them by setting the .RELADC keyword in the ****WAVE FUNCTION** Section.

The RELADC/LANCZOS calculation is then individually controlled in the **RELADC and **LANCZOS input sections.

The FanoADC calculation is a special case of a one-particle propagator calculation. Therefore the keywords are part of the **RELADC section.

**RELADC

****RELADC**

.DOSIPS

.DOSIPS

Do Single Ionization Potentials. If you intend to do single ionization spectra calculations then set this keyword. Closely related to SIP calculations is the keyword

.ADCLEVEL

.ADCLEVEL

Here you determine the perturbational order for a SIP calculation.

```
ADCLEVEL = 1  strict second order
ADCLEVEL = 2  extended second order
ADCLEVEL = 3  third order + constant diagrams
```

Default:

ADCLEVEL = 3 (including constant diagrams)

Input example:

```
.ADCLEVEL
2
```

.SIPREPS

.SIPREPS

Integer array of length 32. Specifies the symmetries of the one-hole final states (SIP) to be calculated. If you do not enter this array all symmetries are checked for possible final states and those are calculated in the respective perturbational order. You can also specify symmetries individually if you are interested only in a few by listing the number of requested symmetries followed by the individual irrep numbers.

Default:

SIPREPS(1:32) = 0 (no symmetries preselected, all are calculated)

Input example:

```
.SIPREPS
8
1,3,5,7,17,19,21,23
```

.READQKL

.READQKL

Read back previously calculated constant diagrams in SIP runs. This is a restart option to avoid the most time consuming step in SIP runs. The constant diagrams for all symmetries are stored in the file QKLVAL.

.NOCONS

.NOCONS

Block calculation of constant diagrams in third-order SIP calculations. ADC is still executed up to third order but in the hole/hole block the (time consuming) constant diagrams are omitted. Skipping constant diagrams reduces accuracy but increases speed considerably. Not recommended for production runs.

.VCONV

.VCONV

Determines convergence of the inverse iteration in the calculation of the constant diagrams. If these iterations take much time you can reduce tightness by setting VCONV to 1.0E-05, 1.0E-04 asf. Check accuracy of results when you activate this option.

Default:

VCONV=1.0E-06

Input example:

```
.VCONV
1.0E-04
```

.DOFULL

.DOFULL

Lanczos is automatically invoked in a single and double ionization calculation. If you want to invoke an additional full diagonalizer set this keyword. Attention: ADC matrices can become very large. A full diagonalization is therefore supported only for matrix dimensions up to 5000 x 5000 and is useful for test purposes only.

.DODIPS

.DODIPS

Do Double Ionization Potentials. If you intend to do double ionization spectra calculations then set this keyword. The keyword .ADCLEVEL does not apply to DIP runs because the perturbational order is set fixed to 'extended second order'.

Default:

DODIPS = F

.DIPREPS

.DIPREPS

Integer array of length 32. Specifies the symmetries of the two-hole final states (DIP) to be calculated. If you do not enter this array all symmetries are checked for possible final states and those are calculated in the respective perturbational order. You can also specify symmetries individually if you are interested only in a few. Input is analogous to SIPREPS.

Default:

DIPREPS(1:32) = 0 (no symmetries preselected, all are calculated)

.ADCTHR

.ADCTHR

In DIP calculations only matrix elements whose amount is larger than ADCTHR will be written to disk. This is due to the large matrices occurring in DIP runs. Tests have shown that you can reduce matrix size by a factor of two setting ADCTHR to 1.0E-05 with an accuracy loss in the meV region. Perform accuracy tests for production runs since the behavior is system-dependent.

Default:

ADCTHR = 0.0 (all nonzero matrix elements are written to disk)

Input example:

```
.ADCTHR
1.0E-05
```

.FANO

.FANO

This keyword activates the FanoADC module. It requires the specification of the initial and final states to be investigated. By running the default calculation settings, specified ionization spectra are calculated as well. This can be turned off by using the keyword .FANOONLY.

Default:

False

For a Fano calculation it is crucial to use a proper (and normally large) basis set. It has been shown, that it is beneficial to add KBJ exponents to the atoms themselves or to ghost atoms surrounding the system in order to span a basis for the interaction region of the final state and the outgoing electron. By this the user should take care not to loose symmetry by specifying the positions of ghost atoms.

More information about the proper selection of basis sets can be found in the PhD thesis of Elke Fasshauer ([link](#)).

.FANOIN

.FANOIN

Here the initially ionized state is specified by symmetry and relative spinor number. The following example chooses the first spinor in the first symmetry to be the initial state:

```
.FANOIN
1 # symmetry
1 # relative spinor number
```

You can find a table of spinors, their symmetries and relative and absolute spinor numbers at the beginning of every ADC calculation with the *same* active space. For the case of the Auger process of a neon atom, this looks like:

Spinor	Abelian Rep.	Energy	Recalc. Energy
0	1 1 1g	-32.8174679811	-32.8174680470
0	2 2 1g	-1.9358495110	-1.9358495652
0	1 3 -1g	-32.8174679811	-32.8174680470
0	2 4 -1g	-1.9358495110	-1.9358495652
0	1 5 1u	-0.8528295909	-0.8528295926
0	2 6 1u	-0.8482685119	-0.8482685445
0	1 7 -1u	-0.8528295909	-0.8528295926
0	2 8 -1u	-0.8482685119	-0.8482685445

```

0   1   9   3u   -0.8482685155   -0.8482685461
0   1   10  -3u   -0.8482685155   -0.8482685461
.
.
virtual orbitals

```

which means, that the first spinor of the first symmetry (1g) which represents the Ne1s orbital is chosen as the initially ionized state.

.FANOCHNL

.FANOCHNL

Autoionization processes decay via several channels into different final states. These are specified in this section. In the first line the number of channels, let us call it N, needs to be given. In the second line the number of two-hole configurations for each channel is specified. If you chose 2 channels, you need to put two numbers here!

The following lines give details about the channels. For every channel a maximum 4 letter shortcut has to be specified followed by the two-hole configurations of describing the final state. They have to be pairs of absolute spinor numbers.

Take care: The number of 2-hole-configurations of a channel has to be the same as you specified in the upper part. The 4 letter shortcut has to start at the beginning of the line. Do not include a space.

This part is at the moment no black box method and the user is required to think carefully about the selection of these final states and not to forget a possible channel. On the other hand, this gives a lot of control to the user to specify exactly what he/she wants to calculate.

Again the Neon atom as example:

```

.FANOCHNL
3          # number of channels N
1,12,15    # number of hole configurations of the different channels
s2         # descriptor of 2s-2 final state
2 4        # two-hole configuration of 2s-2 final state
sp
2 5
2 6
2 7
2 8
2 9
2 10
4 5
4 6
4 7
4 8
4 9
4 10
p2
5 6
5 7
5 8
5 9
5 10
6 7
6 8
6 9
6 10
7 8
7 9
7 10
8 9
8 10
9 10

```

If you change the Hamiltonian you will have to change the channel specification as well!

.FANOONLY

.FANOONLY

This keyword enforces to run only a FanoADC run and overwrite every other calculation of the one-particle propagator.

Default:

False

**LANCZOS

**LANCZOS

Once the ADC matrices are stored in packed form on disk they are diagonalized by the iterative Lanczos algorithm. The spectral information is written to the files SSPEC.#irrep (SIPs) and DSPEC.#irrep (DIPs). Hereby the ionization potential, the pole strength and the error estimate are written in a line terminated by the '@' for grep purposes. Immediately after this line follows the (indented) configuration information belonging to this final state. This is imaginable as a one-hole or two-hole Slater-determinant forming this state in zeroth order. Strictly speaking, the configuration coefficients refer to the intermediate state representation basis.

.SIPITER, .DIPITER

Determines the number of Lanczos iterations in a SIP or DIP calculation for all symmetries. There is no need to specify the number of iterations per symmetry because the convergence behaviour is similar within a specific ionization class. However, DIP calculations can require substantially more iterations for a comparable accuracy. Due to the iterative nature of the Lanczos diagonalizer the edge values converge very fast and some may be reproduced if SIPITER or DIPITER are set to high values. These

reproduced eigenvalues are spurious and will be projected out from the final result. If one observes very many spurious solutions (mainly in the SIP case) it is recommended to reduce SIPITER accordingly.

Default:

SIPITER = 500, DIPITER = 500.

Input example:

```
.SIPITER
1000
.DIPITER
2500
```

.SIPRNT, .DIPRNT

Real values. These two variables only control screen output of the calculated eigenvalues and have no influence on the results in the (SD)SPEC files. You can enter the threshold in eV up to which computed IPs (SIPs, DIPs) will be printed on screen. Sometimes one is only interested in a few lowest IPs and the screen output suffices.

Default:

SIPRNT = 50.0, DIPRNT = 50.0.

Input example:

```
.SIPRNT
20.0
.DIPRNT
100.0
```

.SIPIGV, .DIPIGV

For each chosen symmetry selected by the XXXREPS array a lower and upper energy boundary value for the corresponding method are specified. Only within this energy range the long eigenvectors are calculated. This is more user-friendly since one can not anticipate the number of eigenvectors to be expected in a certain energy range.

Default:

0.0 0.0 (no eigenvectors calculated)

Input example:

```
.SIPIGV #(same for .DIPIGV)
4 # number of lines of ranges to follow
10.0 20.0
20.0 30.0
0.0 0.0
10.0 15.0
```

.DOINCORE

.DOINCORE

This keyword activates an incore Lanczos diagonalization. If you run jobs on machines with large core memory you can speed up the diagonalization considerably by transferring large parts of the ADC matrix to memory. This is especially noticeable in DIP jobs because the matrices are much larger than in the single ionization case. Attention: Be aware that the operating system allows you to allocate as much memory as you want. If there is not enough physical memory the OS starts to swap portions of the memory to disk. You have to avoid this situation since your job will not terminate in time. Check carefully that the memory you request is physically available!

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/relccsd.md

orphan

Kramers unrestricted Coupled-Cluster methods

See Visscher1996 for the initial CCSD(T) implementation. The Fock-space CC implementation is described in Visscher2001. The ground-state expectation value implementations are described in vanStralen2005 for MP2 and Shee2016 for CCSD. The base EOM CC implementation is described in Shee2018.

****RELCC**

****RELCC**

Specification of reference determinant, type of calculation, and general settings.

.FOCKSP

.FOCKSP

Activate the Fock space module. This option should be used for multireference calculations. See further *CCF5PC. Because the first sector of Fock space gives the same result as a regular CCSD calculation, the latter calculation is switched off. If you do wish to perform also a regular CC calculation (e.g. to get the CCSD(T) energy) you

need to activate this explicitly via RELCC_.ENERGY (see below).

.ENERGY

.ENERGY

Activate the energy calculation. This is the preferred option for calculations on closed shell or simple open shell systems and need not be specified explicitly in such cases. For Fock space calculations the keyword switches on a separate single reference calculation done prior to the FS calculation.

Default:

Perform energy calculation.

.GRADIENT

.GRADIENT

Calculate the effective 1-particle density matrix. This option can be used to calculate molecular properties. For closed-shell MP2 see vanStralen2005. For closed-shell CC see Shee2016.

This calculation generates the file CCDENS, which contains the MP2 or CC ground-state density matrix in AO basis. In this release, CCDENS is used by the property module to calculate ground-state expectation values.

If saved, CCDENS can be used in a property calculation (see PROPERTIES_.RDCCDM) without the need to invoke this module.

Default:

No gradient calculation.

.EOMCC

.EOMCC

Activate the equation of motion (EOMCC) module. This option can be used to obtain excitation energies (EOM-EE), singly ionized (EOM-IP) and electron attached (EOM-EA) states starting from a closed shell reference and at the CCSD level, by solving for the right-hand eigenvalues and eigenvectors of the appropriate similarity-transformed Hamiltonian.

Unlike in Fock-space calculations, where the full model Hamiltonian for each sector is diagonalized, in EOMCC the user must specify the desired number of roots per symmetry, (see *EOMCC for further details) and these are determined via matrix-free diagonalization procedures such as the Davidson method (see *CCDIAG for the options available).

Default:

No EOM-CC calculation.

.pEOM

.pEOM

Activate the partitioned EOM approximation to CCSD (P-EOM-CCSD), in which the doubles-doubles block of the similarity transformed Hamiltonian is approximated by diagonal elements given by orbital energy differences. This keyword also activates the RELCC_.EOMCC keyword, and as such it is available for the same types of calculations supported by the latter.

Default:

No P-EOM-CCSD calculation.

.EOM2

.EOM2

Activate the second order perturbation theory based EOM approximation (EOM-MBPT2), in which all of the matrix elements of the the similarity transformed Hamiltonian are approximated. This keyword also activates the RELCC_.EOMCC keyword, and as such it is available for the same types of calculations supported by the latter.

Default:

No EOM-MBPT2 calculation.

.pEOM2

.pEOM2

Activate the partitioned second order perturbation theory based EOM approximation (P-EOM-MBPT2), which combines both approximations mentioned above. This keyword also activates the RELCC_.EOMCC keyword, and as such it is available for the same types of calculations supported by the latter.

Default:

No P-EOM-MBPT2 calculation.

.EOMPROP

.EOMPROP

[!WARNING] Development version only.

Activate the calculation of excited-state first-order properties for the EOMCC module for CCSD wavefunctions, by solving for the left-hand eigenvectors of the corresponding similarity-transformed Hamiltonian. It should be specified along with RELCC_.EOMCC as it requires the right-hand eigenvectors and, as such, the same type of calculation (EOM-EE/IP/EA) will be performed. See *EOMPROP for further details.

Default:

No EOM-CC excited-state expectation value calculation.

.NELEC

.NELEC

Number of correlated electrons. This variable determines the reference determinant to be used in the exponential expansion of the wave function. Since the default values correspond to the information passed on by the MOLTRA code on basis of the Hartree-Fock occupations and chosen range of active orbitals in MOLTRA, **for CLOSED SHELL systems there is usually no need to specify this variable manually**. If you do chose to specify this manually, you should make sure to count electrons carefully as the numbers relate to the number of correlated electrons rather than the total number. This number may therefore change if you change thresholds in the integral transformation.

For OPEN SHELL systems it is usually better to employ the multireference Fock space approach, except for simple cases such as a high-spin open shell state. In this case the single reference CCSD(T) ansatz works rater well. For such cases it is easier to use the RELCC_.NELEC_OPEN keyword that allows to just specify the distribution of open shell electrons over the irreps, but for backwards compatibility we retain the older NELEC option as well.

Arguments:

Integer (NELEC(I), I=1,NFSYM*2).

Default:

Number of correlated electrons in closed shells in these irreps (written by **MOLTRA).

.NELEC_OPEN

.NELEC_OPEN

Distribution of correlated open shell electrons over the irreps. This determines the reference determinant for open shell single reference calculations.

This input should always be given for average-of-configuration SCF calculations that are followed by a CC calculation. The total number of electrons that is given should correspond to the number of open shell electrons.

One should activate either RELCC_.NELEC (total number of correlated electrons) or RELCC_.NELEC_OPEN (only open-shell electrons) for open-shell cases. Do not activate both RELCC_.NELEC and RELCC_.NELEC_OPEN once.

Arguments:

Integer (NELEC_OPEN(I), I=1,NFSYM*2).

Default:

Zero. (note that this default leads to wrong results because a closed shell ion will be calculated if no input is given).

.NEL_F1

.NEL_F1

Number of electrons in the *gerade* irreps of the Abelian symmetry group. This works like the NELEC keyword, but uses the supersymmetry possible for linear groups in which irreps are ordered as 1/2, -1/2, 3/2, -3/2, 5/2, ... (with the number being the m_j value).

.NEL_F2

.NEL_F2

Number of electrons in the *ungerade* irreps of the Abelian symmetry group.

.PRINT

.PRINT

Print level. - Level 0 (default): computed energies and achieved convergence of wave function parameters - Level 1: additional information on the progress of the calculation, useful in case you encounter a problem - Level 2: detailed information on the wave function, useful for analysis but can give very large output files - Level 3 and higher: do not use

Default:

.PRINT
0

.COUNTMEM

.COUNTMEM

Stop CC module after counting the total memory demand. Needs only MRCONEE.

.TIMING

.TIMING

Print detailed timing information.

Default:

Only limited timing information is printed.

.DEBUG

.DEBUG

Print debug information.

Default:

Debug information is not printed.

.RESTART

.RESTART

Reuses information from prior calculations to resume the calculation from the last successfully recorded checkpoint. In order for a restart to be possible, the appropriate files from both the 4-index transformation - e.g. MRCONEE, MDCINT (and MDCINX* for parallel runs) and, if the transformation of property operators was requested, MDPROP - and from the previous coupled cluster run (ft.* for the sorting of integrals, MCCRES* for the amplitudes) have to be present at the scratch directory. Given the size and number of these files, this means in practice one must request the the scratch directory is kept at the end of the runs (see the pam script options for details)

Default:

No restart is performed.

.NOSORT

.NOSORT

Forces the code to skip the sorting of integrals coming from the 4-index transformation into six integral classes. Using this option without the integral sorting step having been properly carried out may produce incorrect results.

Default:

Sorting of integrals into classes is performed.

*CCENER

***CCENER**

Covers options related to energy.

.NOMP2

.NOMP2

Deactivate MP2 calculation.

.NOSD

.NOSD

Deactivate CCSD calculation.

.NOSDT

.NOSDT

Deactivate the calculation of perturbative triples. This is potentially useful when running into memory problems for very big calculations and will also save some CPU time.

.MAXIT

.MAXIT

Set maximum number of iterations allowed to solve the CC equations.

.MAXDIM

.MAXDIM

Set maximum number of amplitude vectors used in the DIIS extrapolation.

.NTOL

.NTOL

Specify requested convergence (10^{-NTOL}) in the amplitudes.

.NOSING

.NOSING

Eliminate T1 amplitudes in the calculation (only interesting for test purposes, this gives no computational speed-up).

.NODOUB

.NODOUB

Eliminate T2 amplitudes in the calculation (only interesting for test purposes, this gives no computational speed-up). Deactivate contribution from doubles; corresponds to a CCS calculation.

*CCFOPR

***CCFOPR**

Calculate first-order properties (expectation values) for the MP2 and CCSD wave function.

.MP2G

.MP2G

Use MP2 wave function.

.CCSDG

.CCSDG

Use CCSD wave function.

.NATORB

.NATORB

Calculate natural orbitals (currently only for MP2 density matrix)

.RELAXED

.RELAXED

Use orbital-relaxed density matrix (currently only for MP2). The current default for MP2 and CC wave functions is to use the unrelaxed density matrix. This is computationally less expensive, but also less accurate.

*EOMCC

***EOMCC**

This menu controls the parameters for the definition of the EOM model used and the number of roots per symmetry.

Currently the implementation supports the excitation energy (EE), single electron attachment (EA) and detachment (IP) models for CCSD wavefunctions only.

Note that there is no default, if no options are selected no calculations will be performed.

.EE

.EE

Selects excitation energy calculations.

Expects two integers, the first specifying the state symmetry number and the second the number of states for this symmetry. The keyword should be repeated multiple times if different symmetries are desired. The implementation does not allow for mixing the different EOM models in the same run.

NB: finding the state symmetry number for EE may require some experimentation, the totally symmetry irrep is always number 1, but the order of the other boson irreps depends on the double group that is used and how the molecule is oriented.

Example: requesting two excitation energies for symmetry 1, four for symmetry 3, and one for symmetry 8:

```
.EE
1 2
.EE
3 4
.EE
8 1
```

Default:

No excitation energies requested

.EA

.EA

Selects single electron attachment calculations.

Expects two integers, the first specifying the state symmetry number and the second the number of states for this symmetry. The keyword should be repeated multiple times if different symmetries are desired. The implementation does not allow for mixing the different EOM models in the same run.

NB: the state symmetry number for EA depends on the ordering of irreps for the spinors, this information is printed at the beginning of the RELCCSD output.

Example: requesting two electron attachment energies for symmetry 1, four for symmetry 3, and one for symmetry 8:

```
.EA
1 2
.EA
3 4
.EA
8 1
```

Default:

No excitation energies requested

.IP

.IP

Selects electron detachment calculations.

Expects two integers, the first specifying the state symmetry number and the second the number of states for this symmetry. The keyword should be repeated multiple times if different symmetries are desired. The implementation does not allow for mixing the different EOM models in the same run.

NB: the state symmetry number for IP depends on the ordering of irreps for the spinors, this information is printed at the beginning of the RELCCSD output.

Example: requesting two single electron detachment energies for symmetry 1, four for symmetry 3, and one for symmetry 8:

```
.IP
1 2
.IP
3 4
.IP
8 1
```

Default:

No excitation energies requested

*EOMPROP

*EOMPROP

.EXCPRP

.EXCPRP

[!WARNING] Development version only.

number of excited states to calculate expectation values for, on a given symmetry.

Arguments: two integers, the first specifying the state symmetry and the second the number of states for this symmetry

*CCDIAG

*CCDIAG

This menu controls the parameters to the iterative Davidson diagonalization procedure.

Note that the implementation currently does not support the saving of eigenvectors, so no restarts are possible.

.CONVERG

.CONVERG

Sets the convergence threshold on the norm of the residual vectors

Default:

.CONVERG
1.0E-8

.MAXSIZE

.MAXSIZE

Sets the maximum size of the subspace

Default:

.MAXSIZE
128

.MAXITER

.MAXITER

Sets the maximum number of iterations

Default:

.MAXITR
80

.TRV_I

.TRV_I

` Creates trial vectors based on unit vectors sorted by lowest energy of the diagonal of the similarity-transoformed Hamitonian (pivoting).

Default:

Enabled

.TRV_FULLMATRIX

.TRV_FULLMATRIX

Creates unit trial vectors from a unit matrix. This roughly means that the first trial vectors will be those for high-lying states.

If combined with a number of EE/EA/IP roots set to -1 and a maximum number of iterations set to 1, the full similarity transformed Hamiltonian for a given symmetry will be diagonalized. This yields the full spectrum, but for anything other than CC singles will require significant amounts of memory, so in practice such a combination is useful for debug purposes only.

Default:

Disabled

.TRV_CCS

.TRV_CCS

Creates trial vectors from the eigenvectors of the singles-singles EOM-EE/IP/EA blocks.

.TRV_R2L

.TRV_R2L

.TRV_NOR2L

.TRV_NOR2L

Control the use the right-hand side (R) vectors as guess for the left-hand side (L). If disabled, (.TRV_NOR2L) trial vector generation will be controlled by the options above.

Default:

.TRV_R2L

.OVERLAP

.OVERLAP

.NOOVERLAP

.NOOVERLAP

Controls the use of root following via overlap in sorting the left/right-hand side eigenvalues and eigenvectors.

Default:

.NOOVERLAP

.DEBUG

.DEBUG

Enables debug printout. Massive output.

Default:

Does not print out debug information

*CCPROJ

***CCPROJ**

This menu controls the use of projection operators in the CC calculations.

These projectors are used to remove terms from the ground state CC equations (frozen core), the EOM-CC similarity transformed Hamiltonian (CVS and restricted excitation window), or a combination of both. The implementation for projection operations is detailed in Halbert2020 and follows the scheme introduced by Coriani and Koch Coriani2015.

.FCORE

.FCORE

Defines which active occupied spinors to consider as part of frozen core within the ground-state CC calculation.

Input is an threshold energy value in atomic units. T amplitudes containing spinors with energies below such value are set to zero.

Note this value does not have to be the same as for the definition of the core spinors for core-valence separation. Assuring the consistency of these choices is up to the user.

Example: requests that all spinors with energies below -10.0 a.u. are frozen:

```
.FCORE
-10.0
```

Default:

No frozen core spinors

.CVS_CORE

.CVS_CORE

Defines which active occupied spinors to consider as core spinors for EOM-CC calculations employing the core-valence separation (CVS) framework. To be used with EOM-IP or EOM-EE, in order to target high-lying excited/ionized states, namely for core spectra.

Input is an threshold energy value in atomic units.

Right/Left-hand side trial vector and similarity transformed Hamiltonian matrix elements representing excited determinants containing at least one occupied spinor with energies below such value are set to zero.

Note this value does not have to be the same as for the definition of the frozen core. Assuring the consistency of these choices is up to the user.

In addition to this, users should make sure to enable root following (see CCDIAG_.OVERLAP) if they want to target states with dominant singly ionized or singly excited character.

When each edge is energetically far apart (e.g., K and L1 edge of heavy elements), one should aim for calculating different edges separately instead of requesting more roots. Otherwise, convergence is likely to be obtained only for the lowest-lying state. Furthermore, if more than one edge is sought, we recommend that is the restart capabilities of the code are employed to avoid the AO to MO transformation step (**MOLTRA) and the integral sorting in RELCCSD (see restart options under *CCRESTART).

Example: requests that all spinors with energies below -10.0 a.u. are considered to be core:

```
.CVS_CORE
-10.0
```

Default:

No core-valence separation is enforced

```
.NODOCC
```

.NODOCC

Requests that, when using core-valence separation, Right/Left-hand side trial vector and similarity transformed Hamiltonian matrix elements representing excited determinants containing both core occupied spinors to be projected out.

Example: requests that double core excited determinants are projected out:

```
.NODOCC
```

Default:

Double core excited determinants are retained in the calculations

```
.REW_OCC
```

.REW_OCC

```
.REW_VIRT
```

.REW_VIRT

Defines a set of active occupied and/or virtual spinors as part of restricted window for EOM-CC calculations. Can be used with any EOM variant.

If an occupied/virtual restricted window is defined, the Right/Left-hand side trial vector and similarity transformed Hamiltonian matrix elements representing excited determinants containing at least one occupied/virtual spinor within the window(s) are retained in the calculation, the rest being projected out.

Input for each keyword is a pair of values defining the threshold energy value in atomic units, one per line. Note that the two keywords can be used independently.

Right/Left-hand side trial vector and similarity transformed Hamiltonian matrix elements representing excited determinants containing at least one occupied spinor with energies below such value are set to zero.

The option CCPR0J_.REW_OCC is incompatible with CCPR0J_.CVS_CORE. It can however be used with CCPR0J_.FCORE, but assuring the consistency of these choices is up to the user.

In addition to this, users should make sure to enable root following (see under *EOMCC) if they want to target states with dominant singly ionized or singly excited character.

Example: requests two restricted windows, one for occupied spinors with energies between -200.0 a.u. and -150.0 a.u. and another for virtual spinors with energies between 0.001 a.u. and 35.0 a.u.:

```
.REW_OCC
-200.0
-150.0
.REW_VIRT
0.001
35.0
```

Default:

No restricted window is defined, all occupied and/or virtual spinors are active

```
*CCFSPC
```

***CCFSPC**

Perform a Fock space MRCC calculation. Fock space allows variable particle number. Sectors (m,n) in Fock space corresponds to $N+n-m$ - electron states obtained by the generation of m holes (electron removal) and n particles (electron attachment) with respect to a closed-shell reference determinant, the $(0,0)$ sector. Within each specified sector (and the lower ones), an effective Hamiltonian is built and diagonalized to give CC energies for a set of states.

```
.DOIH
```

.DOIH

Use the Intermediate Hamiltonian formalism in which an auxiliary space is used to prevent the “intruder state” problem. Default: IH formalism not used.

```
.DOEA
```

.DOEA

Calculate electron affinities (add one electron to the reference state, allowing occupation of the active virtual orbitals, corresponding to the $(0,1)$ sector).

.DOIE

.DOIE

Calculate ionization energies (remove one electron from the reference state, allowing depletion of the active occupied orbitals, corresponding to the $(1,0)$ sector).

.DOEA2

.DOEA2

Calculate second electron affinities (add two electrons to the reference state, allowing occupation of the active virtual orbitals, corresponding to the $(0,2)$ sector).

.DOIE2

.DOIE2

Calculate second ionization energies (remove two electrons from the reference state, allowing depletion of the active occupied orbitals, corresponding to the $(2,0)$ sector).

.DOEXC

.DOEXC

Calculate excitation energies (allow excitation from the set of active occupied orbitals to the set of active virtual orbitals, corresponding to the $(1,1)$ sector).

.NACTH

.NACTH

Specification of the set of active hole orbitals (from which ionization/excitation takes place)

.NACTP

.NACTP

Specification of the set of active particle orbitals (to which electron attachment/excitation takes place)

.MAXIT

.MAXIT

Maximum number of iterations allowed to solve the FSCC equations

.MAXDIM

.MAXDIM

Set maximum number of amplitude vectors used in the DIIS extrapolation.

.NTOL

.NTOL

Specify requested convergence (10^{-NTOL}) in the amplitudes.

.GESTAT

.GESTAT

Specify the state number in the last active sector to pick the energy from (remember to account for degeneracies) for a state-specific FSCC geometry optimization based on a numerical gradient.

*CCIH

***CCIH**

Options for intermediate hamiltonian in FSCC.

.EHMIN

.EHMIN

Minimum orbital energy of occupied orbitals forming the auxiliary (Pi) space. Orbitals with energies lower than this energy are taken in the secondary (Q) space and do not contribute to the model space.

low limit of orbital energies of active occupied orbitals, which constitute the secondary Pi space. Could be used in (1,0), (2,0) and (1,1) sectors. Arguments: real.

.EHMAX

.EHMAX

Maximum orbital energy of occupied orbitals forming the auxiliary (Pi) space. Orbitals with energies higher than this energy are taken in the primary (Pm) space.

This is upper limit of one-electronic energies of active occupied orbitals, which constitute the secondary Pi space. Could be used in (1,0), (2,0) and (1,1) sectors. Arguments: real.

.EPMIN

.EPMIN

Minimum orbital energy of virtual orbitals forming the auxiliary (Pi) space. Orbitals with energies lower than this energy are taken in the primary (Pm) space.

This is the low limit of orbital energies of active virtual orbitals, which constitute the secondary Pi space. Could be used in (0,1), (0,2) and (1,1) sectors. Arguments: real.

.EPMAX

.EPMAX

Maximum orbital energy of virtual orbitals forming the auxiliary (Pi) space. Orbitals with energies higher than this energy are taken in the secondary (Q) space and do not contribute to the model space.

This is the upper limit of one-electronic energies of active virtual orbitals, which constitute the secondary Pi space. Could be used in (0,1), (0,2) and (1,1) sectors. Arguments: real.

Other Intermediate Hamiltonian (IH) input parameters

For *experts* only.

Following keywords belong to the CCIH namelist section.

.IHScheme

.IHScheme

Choose particular IH scheme. Arguments: Integer IHScheme = 1, or 2.

The IHScheme=1 corresponds to the extrapolated IH (XIH) approach, described in the paper Eliav2005.

Main idea: proper modification of the energetic denominators, containing problematic Pi -> Q transition. The original denominator $1/(E_{Pi} - E_Q)$, used during CC iterations, is substituted by the following expression (1)

$$\frac{(1 - \frac{AIH * SHIFT}{(E_{Pi} - E_Q + SHIFT)})^{NIH}}{(1 - AIH * SHIFT / (E_{Pi} - E_Q + SHIFT))},$$

here AIH, SHIFT, NIH are parameters, specially chosen for overcoming of the intruder states problem. These parameters could be used in the procedure of the extrapolation of the “exact” effective Hamiltonian solutions from corresponding IH CC energies and wave functions.

The IHScheme=2 corresponds to the simplified IH-2 approach, described in the paper Landau2004.

Here the problematic denominators $1/(E_{Pi} - E_Q)$ are substituted simply by the factor 0.

Default: IHScheme = 2

Next key options are used only in case of XIH (IHScheme = 1).

.SHIFTH1

.SHIFTH1

Energy shift for the one-electronic excitations in (1,0) sector. Arguments: real.

.SHIFTH2

.SHIFTH2

Energy shift for the two-electronic excitations in (1,0) sector. Arguments: real.

.SHIFTH2

.SHIFTH2

Energy shift for the two-electronic excitations in (2,0) sector. Arguments: real.

.SHIFTP11

.SHIFTP11

Energy shift for the one-electronic excitations in (0,1) sector. Arguments: real.

.SHIFTP12

.SHIFTP12

Energy shift for the two-electronic excitations in (0,1) sector. Arguments: real.

.SHIFTP2

.SHIFTP2

Energy shift for the two-electronic excitations in (0,2) sector. Arguments: real. Usually we choose the approximate difference between the highest orbital energy belonging to Pi and the lowest orbital energy belonging to the Pm space. Works only with the old style of RELCC input.

.SHIFTHP

.SHIFTHP

Energy shift for the two-electronic excitations in (1,1) sector. Arguments: real

.AIH

.AIH

Compensation factor, used in expression (1). Arguments: real positive, not greater then 1.0.

.NIH

.NIH

Compensation power, used in expression (1). Arguments: integer.

In the case of the limit: AIH=1.0 and NIH -> “infinity” (NIH>100, in practice) we have so called “full compensation” method, corresponding to the extrapolation of the effective Hamiltonian from the intermediate one.

*CCSORT

***CCSORT**

Specialist options related to the sorting of two-electron integrals and the calculation of the reference Fock matrix.

.NORECMP

.NORECMP

Do not recompute the Fock matrix, but assume a diagonal matrix with the orbital energies taken from the SCF program on the diagonal. This is usually not recommended as the latter correspond to a restricted open shell expression and RELCCSD uses an unrestricted formalism. For closed shell systems the two expressions are identical and this option merely suppresses a build-in check on the accuracy of transformed integrals.

.USEOE

.USEOE

Ignore recomputed Fock matrix and use orbital energies supplied by the SCF program. This option is sometimes useful for degenerate open shell cases in which case the perturbation theory for the unrestricted formalism is not invariant for rotations among degenerate orbitals. It should only change the outcome of the [T], (T) and -T energy corrections.

*CCRESTART

***CCRESTART**

Control parameters for the restart option. The default behavior of the restart option is to verify whether in the RELCCSD the successive checkpoints have been passed, and restart the calculation at the first one which is not flagged as “Completed, restartable”.

For example, one would have

Status of the calculations	
Integral sort # 1 :	Completed, restartable
Integral sort # 2 :	Completed, restartable
Fock matrix build :	Completed, restartable
MP2 energy calculation :	Completed, restartable
CCSD energy calculation :	Completed, restartable
CCSD(T) energy calculation :	Completed, restartable

if a calculations has successfully has passed through all checkpoints.

For the single-reference calculations the restart is straightforward and in general no additional keywords are necessary. Users must nevertheless be careful that the restart is performed for exactly the same calculation, as there are no interla consistency checks in place in the case of a restart: this means that, for example, if either the geometry or the number of correlated electrons or virtual spinors has been changed after the initial calculations, if the RESTART option is on the code will proceed with the old checkpoint data (sorted integrals, etc) and any results will be erroneous.

Fock-space calculations can also be restarted, but for these additional care must be taken: (1) One must redo the integral sorting steps if the model (P) or correlation (Q) spaces change dimension. A change in the values that define the partition between main (Pm) and intermediate (Pi) model spaces, on the other hand, does not require the integral sorting to be redone. (2) One has to specifically identify which sectors of Fock space are to be (re)calculated and which should be skipped. Some examples of such a procedure can be found in the test set.

Important: for EOMCC calculations it is currently only possible to restart from ground-state calculations. In this case, the “.REDOCCSD” keyword *must* be used, otherwise the code will produce incorrect results.

.UNCONVERGED

.UNCONVERGED

Forces results from unconverged iterative procedures to be considered as converged.

Default:

False

.FORCER

.FORCER

Forces restart even if setup is potentially different (number of electrons, active/inactive spinors etc).

Default:

False

.REDOCCSD

.REDOCCSD

Forces the iterative procedure to solve the CCSD (or CCD or CCS) equations to be performed, even if in a prior run it has been marked as completed.

Default:

False

.REDOSECT

.REDOSECT

In the case of fock-space calculations, indicate which sectors are to be recalculated during the restart.

This keyword expects two lines as input; on the first line, an integer specifying how many sectors are recalculated and in the second line a list of the sectors in question in the RELCC notation.

```
.REDOSECT
2
00 01
```

will redo sectors 0h0p and 0h1p

Default:

All sectors are recalculated

.SKIPSECT

.SKIPSECT

In the case of fock-space calculations, indicate which sectors are to be skipped during the restart

This keyword expects two lines as input; on the first line, an integer specifying how many sectors are recalculated and in the second line a list of the sectors in question in the RELCC notation.


```
.SKIPSECT
2
00 01
```

will bypass the calculation of sectors 0h0p and 0h1p.

Default:

No sectors are skipped

```
.REDOSORT
```

.REDOSORT

Forces the sorting into the six integral classes to be performed again, even if prior sorting was completed.

Default:

False. integrals are not resorted if the prior sorting was completed.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/resolve.md

orphan

```
*RESOLVE
```

***RESOLVE**

Introduction

Resolve open-shell states.

The individual electronic states of an average-of-configurations calculation are resolved by a full CI in the small open-shell manifold(s) using the G0SCIP module.

Note that even if this is a small CI diagonalization one needs in principle to generate *all* AO integrals for the initial 4-index transformation. The cost of calculation can be greatly reduced invoking integral screening (controlled by RESOLVE_.SCREEN) in the 4-index transformation, possibly also by leaving out at least SS integrals (using RESOLVE_.INTFLG).

Keywords

```
.PRINT
```

.PRINT

Print level.

Default:

```
.PRINT
0
```

```
.INTFLG
```

.INTFLG

Specify what two-electron integrals to include (default: HAMILTONIAN_.INTFLG under **HAMILTONIAN).

```
.SCREEN
```

.SCREEN

Screening threshold in the 4-index transformation.

A negative number deactivates screening.

Default:

```
.SCREEN
1.0D-14
```

```
.SCHEME
```

.SCHEME

Choose integral transformation algorithm (PLEASE CLARIFY!)

Default:

```
.SCHEME
4
```

FILE: www.diracprogram.org/doc/release-26/_sources/manual/wave_function/scf.md

orphan

*SCF

*SCF

This section gives directives for Hartree-Fock and Kohn-Sham calculations. Kohn-Sham calculations are activated by invoking the keyword `HAMILTONIAN_.DFT` under `**HAMILTONIAN`.

Open-shell calculations correspond to either average-of-configurations (`SCF_.AOC`) (see `aoc` for theory) or fractional occupation (`SCF_.FOCC`). The former is the default for Hartree-Fock calculations, whereas the latter is default for Kohn-Sham calculations. Note that average-of-configurations Kohn-Sham calculations are not well defined.

Occupation

`.CLOSED SHELL`

.CLOSED SHELL

For each fermion irrep give the number of closed shell electrons.

The specification of the closed shell electrons is simple. For symmetry groups *without* inversion symmetry, there is only one fermion irrep, and you need only to specify the number of electrons.

For symmetry groups *with* inversion symmetry, you need to specify the distribution of the electrons in the two fermion irreps Saue2000.

`.OPEN SHELL`

.OPEN SHELL

Specification of open shell(s).

For each open shell give the number of electrons and the number of active spinors.

Short example:

```
.OPEN SHELL
1
5/0,6
```

1 open shell with 5 electrons in 6 spinors (= 3 Kramers pairs) in irrep 2 (the *ungerade* one). Thus, the fractional occupation is 5/6.

The open shell module in DIRAC is based on average-of-configurations Thyssen1998. The simplest case is one electron in two spinors (= one Kramers pair). For this special case the average-of-configuration calculation gives the same result as the usual restricted open-shell Hartree-Fock. For all other cases the calculation gives the average energy of many states.

Note that the order of closed and open shells are assumed to be as in the following scheme:

Virtual orbitals Not occupied

... **Open shell 2** Fractionally occupied

Open shell 1 Fractionally occupied

Closed shell

Doubly occupied; that is, the lowest molecular orbitals are doubly occupied, the next ones are occupied with the electrons of open shell no. 1, etc.

Other orderings can be achieved by using `WAVE_FUNCTION_.REORDER M0` and `SCF_.OVLSEL`.

To get the energies of the individual states present in the average-of-configurations, specify `WAVE_FUNCTION_.RESOLVE` (see also `*RESOLVE`).

To get the energies of (some) of the individual states present in the average of configurations, you can use the `GOSCIPI`, the `DIRRCI`, or the `*LUCITA`.

`.BOSONS`

.BOSONS

Occupation of boson irreps in spin-free calculation. For example, for the D_{2h} symmetry eight numbers in subsequent line, for the C_{2v} symmetry there four occupation numbers in line.

`.MISELE`

.MJSELE

In the case of linear supersymmetry give occupation for each M_J value. The format of SCF_.MJSELE is illustrated for the case of the CH radical in $^2\Pi_{3/2}$ state. We first provide the usual specification of closed and open shells

```
.CLOSED SHELL
6
.OPEN SHELL
1
1/2
```

One can specify the occupation of the open-shell electron in the MO that mainly consists of $2\pi_{3/2,3/2}$ orbital, as shown below

```
.MJSELEC
2      # Number of the MJ-splitted orbital
1 3    # Absolute values of Mj*2: 1/2, 3/2
6 0    # Distribution of closed shell orbitals
0 2    # Distribution of orbitals of open shell 1
```

Currently SCF_.MJSELE does not work with inversion symmetry.

.KPSELE

.KPSELE

In the case of **atomic supersymmetry** give occupation for each κ - value. This option also works in linear supersymmetry when a single atomic center is combined with a ghost atom.

The format of SCF_.KPSELE is illustrated for the case of Uranium ($[Rn]5f^43d^417s^2$). We first provide the usual specification of closed and open shells

```
.CLOSED SHELL
44 44
.OPEN SHELL
2
3/0,14
1/10,0
```

The closed shells are those of Radon as well as the outer $7s$ shell of Uranium. However, their presence will lead to convergence problems because by default orbitals are ordered according to energy, but also with closed shells before open ones. This means that the outer $7s$ shell of Uranium will end up amongst the open-shell orbitals, whereas some $6d$ orbitals end up being defined as closed shell, thus creating havoc. This is completely avoided by in addition giving the occupation in terms of κ - values as shown below

```
.KPSELEC
7      # Number of the Kappa-splitted orbital
-1  1  -2  2  -3  3  -4      # Values of Kappa: s  p-  p+  d-  d+  f-  f+
14 10  20 12 18  6  8      # Distribution of closed shell orbitals
0  0  0  0  0  6  8      # Distribution of orbitals of open shell 1 (5f^3)
0  0  0  4  6  0  0      # Distribution of orbitals of open shell 2 (6d^1)
```

.AOC

.AOC

Average-of-configuration calculation (default for open-shell Hartree-Fock).

.FOCC

.FOCC

Fractional occupation (default for open-shell Kohn-Sham) CLARIFY

SCF_.FOCC calculations are less memory-intensive than SCF_.AOC calculations. In the latter case one additional AO-Fock matrix is generated for each open shell.

SCF_.FOCC calculations are therefore an interesting option for generating start orbitals for MCSCF as well as initial convergence in open-shell Hartree-Fock.

.AUTOCC

.AUTOCC

Program is allowed to change occupation during SCF cycles. This is deactivated by default. However, the program will still try to do an automatic initial occupation if neither SCF_.CLOSED SHELL nor SCF_.OPEN SHELL is given.

Trial function

An SCF-calculation (HF or DFT) may be initiated in seven different ways:

- Using **MO coefficients** from a previous calculation. This is default when a CHECKPOINT file with MO coefficients is found.
- Using a sum of atomic LDA potentials (see Lehtola2020). This default when no MO coefficients are provided.
- Using coefficients obtained by diagonalization of the one-electron Fock matrix: the **bare nucleus approach**.
- The **corrected bare nucleus approach**. In this approach SCF_.BNCORR screening factors from Slater's rules are used to correct the too attractive bare nucleus potential.
- Using the **two-electron Fock matrix** from a previous calculation; this may be thought of as starting from a converged SCF potential.
- Using an **atomic start** based on densities from atomic SCF runs for the individual centers, see e.g. vanLenthe2006.
- Using an **extended Hückel start** based on *atomic fragments*.

The default is to start from MO coefficients if the CHECKPOINT file is present. Otherwise the corrected bare nucleus approach (SCF_,SCRPOt) is followed. In all cases linear dependencies are removed in the zeroth iteration.

.ATOMST

.ATOMST

Start first SCF iteration with a molecular density matrix constructed from atomic densities. The keyword SCF_.ATOMST is followed by input for each atomic type. The details, orbital strings (see orbital_strings for the syntax) and occupation, usually correspond to those of the atomic runs, but the user may modify this at will. The syntax is explained in the parenthesis "" for each atomic type but we highly recommend to carefully check the tutorial example atomic_start_guess. Please note that the order of atoms corresponds to the order they appear in the molecule file.

```
.ATOMST
"SCF coefficients file name (6 characters)" "integer specifying # of occupation patterns, here: 2"
orbital occupation string #1 for atomic type 1
occupation (real*8 value in the range of 0.0d0 - 1.0d0)
orbital occupation string #2 for atomic type 1
occupation (real*8 value in the range of 0.0d0 - 1.0d0)
"SCF coefficients file name (6 characters)" "integer specifying # of occupation patterns, here: 1"
orbital occupation string #1 for atomic type 2
occupation (real*8 value in the range of 0.0d0 - 1.0d0)
...
```

.AD HOC

.AD HOC

Start first SCF iteration with orbitals generated from an extended Hückel calculation using pre-calculated orbitals for each constituent atomic type of the molecule.

```
.AD HOC
"SCF coefficients file name (6 characters)"
orbital occupation string for atomic type 1
"SCF coefficients file name (6 characters)"
orbital occupation string for atomic type 2
...
```

The order of atomic types follows that of the input. Presently, this functionality only works without symmetry.

.HUCPAR

.HUCPAR

Modify the the Wolfsberg-Helmholtz constant K of the extended Hückel calculation. *Default:* 1.75

.SCRPOt

.SCRPOt

Default start procedure: use sum of atomic potentials to initialize calculation Lehtola2020. The atomic potentials used in DIRAC were derived from numerical spin-restricted four-component average-level Dirac–Coulomb–Hartree–Fock calculations with GRASP for the lowest average energy state. The atomic potential was then approximated with the LDA exchange and fitted to error-function form Lehtola2020.

.BNCORR

.BNCORR

Old version of the start potential. Two-electron repulsion is estimated via nuclear-attraction type integrals:

$$\langle X_A | \sum_C \frac{-Z_C}{r_{Cj}} \cdot \sum_j a_j e^{(-\alpha_{Cj} r_C^2)} | r_C \rangle \langle X_B | \rangle, \quad X = L, S$$

The coefficients a_{*j} and the exponents α_{Cj} in this expression are chosen according to Slater's rules to obtain an approximate atomic electronic density for the initial guess. For example, with one heavy element and without this correction (that is, with the bare nucleus Hamiltonian) all electrons will end up on that heavy element in the initial guess!

.NOBNCR

.NOBNCR

Switch off all bare nucleus corrections (SCRPOt or BNCORR).

.BNC_FORCE

.BNC_FORCE

Force employing the bare nucleus correction (BNC). This keyword is worth when the calculated system is highly positively charged what makes (from defined charge value) switching off the default BNC. The BNC can help to achieve better convergence also for non-neutral systems.

.FOMOUT

.FOMOUT

Print Fock MO matrices for diagonalization (according to symmetries) into own formatted files. Programmer's option suitable for testing. Only in for the linear symmetry.

.TRIVEC

.TRIVEC

Start SCF-iterations from the vector file.

.TRIFCK

.TRIFCK

Start SCF-iterations from the two-electron Fock matrix from previous calculation (stored on file DFFCK2).

.MOSTART

.MOSTART

This keyword collects most start guess possibilities. It is followed by a second line specifying start guess. The available options are:

- Bare nucleus start:

```
.MOSTART
BARNUC
```

- Bare nucleus correction, using sum of atomic potentials generated using LDA on Hartree-Fock densities (numerical 4C, generated by GRASP). *Default:*

```
.MOSTART
SCRPOI
```

- Bare nucleus potential corrected with screening factors based on Slater's rules:

```
.MOSTART
BNCORR
```

- Start SCF-iterations from the MO coefficients on the CHECKPOINT file:

```
.MOSTART
TRIVEC
```

- Start SCF-iterations from the two-electron Fock matrix from previous calculation (stored on file DFFCK2):

```
.MOSTART
TRIFCK
```

Convergence criteria

Three different criteria for convergence may be chosen:

- The norm of the DIIS error vector $\|\mathbf{e}\| = \|\mathbf{F}, \mathbf{D}\|$ (in MO basis). This corresponds to the norm of the electronic gradient and is the recommended convergence criterion. When you are only interested in the energy $\text{SCF_EVCCNV} = 1.0\text{e-}5$ is usually sufficient. For properties and correlated methods you should converge to $\text{SCF_EVCCNV} = 1.0\text{e-}9$. Large negative energy eigenvalues lead to a loss of precision that might lead to convergence problems. Remember also that a too loose screening threshold (too many integrals neglected) will hinder convergence. You should modify TWOINT_SCREEN under *TWOINT if you modify SCF_EVCCNV or one of the other two convergence criteria.
- The difference in total energy between two consecutive iterations.
- The largest absolute difference in the total Fock matrix between two consecutive iterations.

The change in total energy is approximately the square of the largest element in the error vector or the largest change in the Fock matrix. The default is convergence on electronic gradient with $1.0\text{e-}6$ as threshold. Alternatively, the iterations will stop at the maximum number of iterations.

Sometimes it may happen that the specified convergence criterion is too tight for the given basis set and/or other input parameters. In this case one needs to decide whether one should proceed with post-HF steps (like correlation calculations) or not. The program decides this by looking at a secondary convergence criterion that gives the *allowed* convergence. This value is by default the same as first or *desired* convergence criterion but can be made lower to make sure that a calculation does not abort when the convergence is slightly above threshold.

For more detailed help see SCF help on convergence troubleshooting and related pages.

.MAXITR

.MAXITR

Maximum number of SCF iterations.

Default:

```
.MAXITR
50
```

When restarting SCF iterations from previous molecular orbitals file (DFCMO or formatted DFPCMO), we recommend to decrease maximum number of iterations together with readjusting desired and allowed convergence thresholds. By properly set desired and allowed thresholds one can have exact number of iterations specified by *.MAXITR*.

.EVCCNV

.EVCCNV

Converge on error vector (electronic gradient).

2 (real) Arguments:

.EVCCNV
desired threshold allowed threshold

.ERGCNV

.ERGCNV

Threshold for convergence on total energy.

2 (real) Arguments:

.ERGCNV
desired threshold allowed threshold

.FCKCNV

.FCKCNV

Converge on largest absolute change in Fock matrix.

2 (real) Arguments:

.FCKCNV
desired threshold allowed threshold

Note that the *allowed* threshold may be omitted. It is then made equal to the *desired* threshold.

Convergence acceleration

It is imperative to keep the number of SCF iterations at a minimum. This may be achieved by convergence acceleration schemes:

- **Damping:** The simplest scheme is damping of the Fock matrix that may remove oscillations. In $n + 1$ iteration the Fock matrix to be diagonalized is: $\mathbf{F}' = (1-c)\mathbf{F}_{n+1} + c\mathbf{F}_n$, where c is the damping factor.
- **DIIS:** Direct inversion of iterative subspaces, Refs. Pulay1980 , Pulay1982 , Hamilton1986, may be thought of as generalized damping involving Fock matrices from many iterations. Damping factors are obtained by solving a simple matrix equation involving the B-matrix constructed from error vectors (approximate gradients). Linear dependent columns in the B-matrix is removed.

In DIRAC DIIS takes precedence over damping.

.DIISTH

.DIISTH

Change the default convergence threshold for initiation of DIIS, based on largest element of error vector.

Default:

.DIISTH
a very large number

.MXDIIS

.MXDIIS

Maximum dimension of B-matrix in the DIIS module.

Default:

.MXDIIS
10

.DIISMO

.DIISMO

Activate DIIS in orthogonal basis (MO) with the error vector as described above.

.DIISAO

.DIISAO

Activate [DALTON](#)-like DIIS using AO-basis. The error vector is

$$\{\mathbf{e}\} = \{\mathbf{f}\} - \{\mathbf{f}\}^{\dagger}$$

where the term $\mathbf{f}$ is given by

$$\mathbf{f} = \mathbf{C}^{\dagger} \mathbf{S}_{A0} \left[\mathbf{D}_{C_{A0}} \mathbf{F}_{A0}^{\dagger} + \sum_{0 \in \text{mathcal{O}}} \mathbf{f}_0 \right]$$

.NODIIS

.NODIIS

Do not perform DIIS. The default is to activate SCF_.DIISM0 for closed-shell calculations, and to activate SCF_.DIISA0 for average-of-configurations calculations.

.DAMPFC

.DAMPFC

Change the default damping factor.

Default:

.DAMPFC
0.25

.NODAMP

.NODAMP

Do not perform damping of the Fock matrix. Damping is activated by default, but DIIS takes precedence. In case all columns in the B-matrix is removed by linear dependency, damping is activated.

Level shifts

.LSHIFT

.LSHIFT

Activate level shift (for virtual orbitals). Followed by a real argument (level shift).

.OLEVEL

.OLEVEL

Activate level shift (for open-shell orbitals). Followed by a real argument (level shift), one line for each open shell {Please give example}.

.OPENFACTOR

.OPENFACTOR

Change the default factor on an open-shell diagonal contribution to the Fock matrix (see aoc for theory). A factor of one corresponds to a Koopmans interpretation of the orbital energies. However, experience shows convergence can be improved by tuning this factor. DIRAC therefore presently employs a default factor of 1/2.

Default:

.OPENFAC
0.5

2nd-order optimization

.2NDOPT

.2NDOPT

The default SCF of DIRAC uses only gradient information. By adding this keyword 2nd-order optimization, using both gradient and Hessian information, is activated in case the regular SCF does not converge. This scheme is computationally more expensive and so far only available for closed-shell Hartree-Fock.

State selection

Convergence can be improved by selection of vectors based on overlap with vectors from a previous iteration. This method may also be used for convergence to some excited state.

If dynamic overlap selection is used, the vector set from the previous iteration is used as the criterion. For the first iteration either restart vectors or vectors generated by the bare nucleus approach (not*recommended) are used.*

If SCF_.NODYNSEL is given, either the restart vectors or the bare nucleus vectors are used, i.e. the overlap selection vectors are *not* updated in each iteration. Please note, that overlap selection based on vectors from the bare nucleus approach is not recommended.

Overlap selection is very useful together with WAVE_FUNCTION_.REORDER M0. This will reshuffle the vectors within the restart coefficients.

Example: First one might do a open shell calculation on Boron, this would give the $P_{1/2}$ state. But if we restart on the $P_{1/2}$ coefficients, interchange the $p_{1/2}$ with the $p_{3/2}$ orbitals, and request overlap selection, we can converge to the $P_{3/2}$ state.

There also exists a keyword for reordering the converged SCF orbitals. This is useful for reordering the orbitals for the 4-index transformation and subsequent correlation calculations (CCSD, CI etc.) (see WAVE_FUNCTION_.POST SCF REORDER M0).

.OVLSEL

.OVLSEL

Activate dynamic overlap selection. The default is no overlap selection.

.NODYNSEL

.NODYNSEL

No dynamic update of overlap selection vectors. The default is dynamic update.

[!NOTE] Overlap selection is nowadays marketed hard as MOM (Maximum Orbital Method, see Gilbert_JPCA2008), but this method has been included in DIRAC for at least two decades and goes back to the pioneering work of [Paul Bagus](#) It was used in Bagus_JCP1971, but not reported explicitly. However, it is for instance documented in the [1975 manual of the ALCHEMY program](#) (On pdf page 15 you find a description of keyword MOORDR using a “maximum overlap criterion”).

Iteration speedup

The total run time may be reduced significantly by reducing the number of integrals to be processed in each iteration:

- **Screening on integrals:** Thresholds may be set to eliminate integrals below the threshold value, see Saue1997. . The threshold for LL integrals is set in the basis file, but this threshold may be adjusted for SL and SS integrals by threshold factors set in the `\**INTEGRALS` section.
- **Screening on density:** In direct mode further reductions are obtained by screening on the density matrix as well, see Ref. Saue1997. This becomes even more effective if one employs *differential densities*, that is $\Delta \mathbf{D} = \mathbf{D}_{n+1} - \alpha \mathbf{D}_n$. The default value for α is $\alpha = \frac{\mathbf{D}_{n+1} \cdot \mathbf{D}_n}{\mathbf{D}_n \cdot \mathbf{D}_n}$ which corresponds to a Gram-Schmidt orthogonalization. As SCF converges, α goes towards 1, but α can also explicitly be set equal to 1 with `SCF_.FIXDIF`.
- **Neglect of integrals:** The number of integrals to be processed may be reduced even further by adding SL and SS integrals only at an advanced stage in the SCF-iterations, as determined either by the number of iterations or by energy convergence. The latter takes precedence over the former.

.NODSCF

.NODSCF

Do not perform SCF-iterations with differential density matrix.

Default: Use differential density matrix in direct SCF.

.FIXDIF

.FIXDIF

Set α equal to 1.

.CNVINT

.CNVINT

Set thresholds for convergence before adding SL and SS/GT integrals to SCF-iterations.

2 (real) Arguments:

```
.CNVINT
CNVINT(1) CNVINT(2)
```

.ITRINT

.ITRINT

Set the number of iterations before adding SL and SS/GT integrals to SCF-iterations.

Default:

```
.ITRINT
1 1
```

.INTFLG

.INTFLG

Specify what two-electron integrals to include (see HAMILTONIAN_.INTFLG under **HAMILTONIAN).

Default: HAMILTONIAN_.INTFLG from **HAMILTONIAN.

Output control

.PRINT

.PRINT

General print level for the SCF method. For instance, value of 2 prints eignevalues during each iteration.

Default:

```
.PRINT
0
```

```
.EIGPRI
```

.EIGPRI

Controls the print-out of positive energy and negative energy eigenvalues (1 = on; 0 = off).

Default: Only the positive energy eigenvalues are printed.

```
.EIGPRI
1 0
```

Eliminating/freezing orbitals

In studies of electronic structure it may be of interest to eliminate or freeze certain orbitals. This option is furthermore useful for convergence, in particular to excited electronic states. A simple case is the thallium atom. The ground state $^2P_{1/2}$ has the electronic configuration $[\text{Xe}]4f^{14}5d^{10}6s^26p_{1/2}^1$. The first excited state $^2P_{3/2}$ with the electronic configuration $[\text{Xe}]4f^{14}5d^{10}6s^26p_{1/2}^13d^1$ can easily be accessed by first calculating the ground state, then eliminating the $6p_{1/2}^1$ from the ensuing calculation. In the final calculation the excited state is relaxed using overlap selection. The use of frozen orbitals is demonstrated in test/33.frozen: When the geometry of the water molecule is optimized with the oxygen 1s and 2s orbitals frozen, a bond angle of 96.242 degrees is found, contrary to the 90 degrees one might have expected when s-p hybridization is thus blocked Kutzelnigg1984.

The elimination of orbitals is achieved by projecting the selected orbitals out of the transformation matrix to orthonormal basis. The selected orbitals can be expressed either in the full molecular basis or in the basis set of the chosen fragment. In the latter case, the same set of fragment orbitals can in the case of atomic fragments be used at different molecular geometries. One may even perform a geometry optimization, but only using the numerical gradient. When freezing orbitals the selected orbitals are first eliminated from the transformation to orthonormal basis, but then reintroduced in the backtransformation step. They will appear in the output with zero orbital eigenvalues. Note that when freezing orbitals the orbitals to be eliminated must be specified as well. The frozen orbitals must be a subset of the eliminated orbitals.

Fragments are defined with respect to the list of symmetry-independent atoms appearing in the DIRAC basis file: Consider the water molecule in the full C_{2v} -symmetry. Then there are two fragments: the oxygen atom and the H_2 moiety. However, with no symmetry there will be three fragments: the oxygen atom and the two hydrogen atoms. At the moment there are no orthonormalization of fragments on different fragments and so in practice one should only use orbitals from one fragment.

```
.PROJECT
```

.PROJECT

Eliminate orbitals by projecting them out of the transformation matrix to orthonormal basis.

Arguments: Number of fragments (NPRJREF).

Then for each fragment read name PRJFIL of the coefficient file followed by the number of symmetry-independent nuclei in this fragment followed by an orbital_strings of selected orbitals for each fermion irrep.

```
DO J = 1,NPRJREF
  READ(LUCMD,'(A6)') PRJFIL(J)
  READ(LUCMD,'(I6)') NPRJNUC(J)
  READ(LUCMD,'(A72)') (VCPR0J(I,J),I=1,NFSYM)
ENDDO
```

```
.PRJTHR
```

.PRJTHR

Smallest norm accepted when eliminating orbitals.

Default:

```
.PRJTHR
1.0D-10
```

```
.OWNBAS
```

.OWNBAS

Eliminated/frozen orbitals are given in the fragment basis. Note that the list of fragments is assumed to follow the list of symmetry independent nuclei in the DIRAC basis file.

```
.FROZEN
```

.FROZEN

Freeze orbitals. This keyword must only be used in conjunction with the keyword SCF_.PROJECT. The latter keyword eliminates selected orbitals from the variational space. From the list of eliminated orbitals, the user can select those that should be added to the coefficient array after diagonalization of the Fock matrix in orthonormal basis. To do so, the user must, for each fermion irrep, give an orbital_strings giving the position of the eliminated orbital in the coefficient array. If the value zero is given, it means that this eliminated orbital is definitely out and not kept frozen.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/x2c.md

orphan

*X2C

*X2C

Feature and debug options for the Exact 2-Component relativistic Hamiltonian module, Refs. Knecht2010 and Knecht2014.

.fragx2c

.fragx2c

use the local X2C scheme (DLU approach) of Peng and Reiher Peng2012 to construct an approximate molecular U matrix from a superposition of atomic U matrices. This requires to compute in advance atomic U matrices in atomic calculations which are stored in the file X2CMAT. For each atom type (C, H, F, U, ...) the module expects an X2CMAT file with the atomic number as appropriate ending, for example: X2CMAT.092 (for U), X2CMAT.006 (C), X2CMAT.001 (H) etc.. See also the tutorial section for a working example.

.skippc

.skippc

Skip the picture change transformation of four-component operators. This is for restarting X2c calculations after SCF, SCF+MOLTRA steps, when such operators transformations were already carried out.

.DEBUG

.DEBUG

raises the print level in the X2C module to print intermediate matrices and arrays. Use with care as the output might become filled with lots of numbers.

.freeh1

.freeh1

use the free-particle Hamiltonian as defining Hamiltonian to solve the initial matrix problem in the X2C algorithm. for debug purposes mostly.

FILE: www.diracprogram.org/doc/release-26/_sources/manual/xamfi.md

orphan

*XAMFI

*XAMFI

Options for the (extended) atomic-mean field aka **(e)amf** module for the HAMILTONIAN_ .X2C Hamiltonian, developed and written by S. Knecht, M. Repisky, H. J. Aa. Jensen and T. Saue, 2013-2022. The HDF5 interface for XAMFI was developed and implemented by K. N. Spauszus (U Bonn/Germany) and S. Knecht.

[!WARNING] The XAMFI module requires a working HDF5 installation as well as h5py in your python env!

The **(e)amf** module has been described in Knecht2022. Based on atomic calculation it provides two-electron scalar-relativistic **and** spin-orbit picture-change corrections for the two-component X2C Hamiltonian (in a molecular or atomic) framework. Since it is completely integrated in the SCF framework of DIRAC, it can exploit all its capabilities for efficient atomic calculations, for example the use of atomic supersymmetry.

For a detailed example of how to use the *XAMFI module, see the **x_amfi** test of the DIRAC test suite or have a look at the [tutorial](#).

The keywords below are valid for both, atomic (mean-field) as well molecular calculations.

[!WARNING] Do not use approximations like .LVCORR for the parent 4-component Hamiltonian. Always use .DOSSSS in the atomic molecular-mean field calculations.

.AMF

.AMF

Activates the atomic mean-field model for two-electron scalar-relativistic **and** spin-orbit picture-change corrections. See also Algorithm 1 in Knecht2022 for further information.

.EAMF

.EAMF

Activates the **extended** atomic mean-field model for two-electron scalar-relativistic **and** spin-orbit picture-change corrections. See also Algorithm 2 in Knecht2022 for further information.

.ATOMIC

.ATOMIC

Activates a deprecated (!) simplified version of the atomic mean-field model for two-electron scalar-relativistic **and** spin-orbit picture-change corrections. Used only for testing/debugging purposes.

[!WARNING] Do not use this option in production calculations.

.DEBUG

.DEBUG

Activates debug output in the *XAMFI module.

[!WARNING] **CAUTION:** This will lead to extremely verbose output

FILE: www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/basis.md

orphan

Pick the right basis for your calculation

The development of basis sets suitable for use in relativistic calculations reflects the relative lateness of the field’s development. Because the more consistent efforts in method development started at about the mid 1980’s, it wasn’t until well into the late 1990’s that the pioneering works of the early and mid 1990’s were substantially complemented and improved upon.

The availability of basis sets has dramatically improved in recent years, notably with the work of K. G. Dyall, and Dirac users are strongly advised to use Dyall’s basis sets whenever they are available. These sets follow roughly the “correlation-consistent” philosophy introduced by Dunning and coworkers Dunning1989, so they already contain polarization and correlation functions, but the SCF sets are designed for an adequate SCF representation rather than to match correlating sets for the valence shells.

Dyall basis sets

We recommend that you use the Dyall basis set repositories for [double-, triple-, and quadruple-zeta sets](#) and [quintuple-zeta set](#) whenever the basis sets are available for the elements of interest. In order to make that usage as convenient as possible, the following files, containing all sets currently available at the URL above (published or to be published), are made available:

quality	valence	core-valence	all-electron	DFT
double-zeta	dyall.v2z	dyall.cv2z	dyall.ae2z	dyall.2zp
+diffuse functions	dyall.av2z	dyall.acv2z	dyall.aae2z	
triple-zeta	dyall.v3z	dyall.cv3z	dyall.ae3z	dyall.3zp
+diffuse functions	dyall.av3z	dyall.acv3z	dyall.aae3z	
quadruple-zeta	dyall.v4z	dyall.cv4z	dyall.ae4z	dyall.4zp
+diffuse functions	dyall.av4z	dyall.acv4z	dyall.aae4z	
quintuple-zeta	dyall.v5z	dyall.cv5z	dyall.ae5z	
+diffuse functions	dyall.av5z	dyall.acv5z	dyall.aae5z	

The basis sets with diffuse functions are available for the s, p, and d blocks. The diffuse functions consist of one additional function in each symmetry that has functions for valence correlation.

While the division into “valence” and “core-valence” can be at times not so clear-cut as for lighter elements, the option was made to stick to the usual jargon of non-relativistic theory, particularly in relation to the “correlation-consistent” family of basis sets.

The valence basis sets are defined differently for each block, and include functions for the correlation of the following shells:

block	shells included
s	ns shell, (n-1)s, p shells
p	ns, np shells
d	ns, np, nd shells, (n+1)s and p shells

block	shells included
f	ns, np, nd, nf, (n+1)s, p, d shells, (n+2) s, p shells

The choice for the f block is necessary to cover correlation of the open f shell, which becomes a semicore shell towards the end of the row. The reason for including the outer core shells for the s and d blocks is that the correlation of these shells is usually necessary for accurate results.

The core-valence basis sets include the (n-2) shell for the s elements, the (n-1) shell for the p elements, the (n-1) shell for the d elements, and nothing extra for the f elements.

The all-electron basis sets include correlating functions for all shells, down to the 1s for all elements. These are intended for use when correlating all electrons.

The basis sets also include functions for dipole polarization of the outer core shells, as this is important for many elements. For the s block these functions are included for the outer core (n-1)s and p shells; for the p block, an f function is included to polarize the (n-1)d shell; for the d block, polarizing f and higher functions are included for the nd shell; none are added for the f block as the f is very compact.

The DFT basis sets do not contain the correlating functions as these are not necessary for DFT (or Hartree-Fock) calculations, except for the outermost shells where the functions with one unit more of angular momentum are included from the correlating sets as polarization functions. The dipole polarization functions for the outer core are also included. These basis sets are the most economical choice for DFT calculations.

You are encouraged to look in the basis set files and in the original archives published in Theor. Chem. Acc. to get a feel for what is included in each case. The archive files are available on zenodo [here](#).

With the recent addition of Dyall basis sets for the light elements, it is no longer necessary to use the standard non-relativistic basis sets, such as the correlation-consistent sets of Dunning and coworkers. However, because the Dyall basis sets are quite a bit larger, you might want to continue using these basis sets. It is advisable that, in order to have a balanced description when light and heavy elements are present, that one uses either contracted or uncontracted sets throughout. For 4-component calculations it is strongly recommended to use the basis sets uncontracted (which is the case for the files listed above).

See the [Dyall basis set repository](#) for the latest updates and the appropriate basis set references. In case of errors or omissions on any of the files in this directory, users are kindly asked to contact the authors of DIRAC.

Other relativistic basis sets

Apart from Dyall's sets, you can choose several different basis sets based upon geometric progressions of exponents. One such set is that of K. Faegri, also available in the basis set library, but with the drawback that you may need to extend it by adding polarization functions.

Non-relativistic and scalar-relativistic basis sets

The DIRAC distribution shares a large library of standard non-relativistic and scalar-relativistic basis sets with the [Dalton](#) program. These basis sets can be found in the directory **basis_dalton** of the DIRAC distribution.

These basis sets are not all suitable for relativistic calculations, especially not for the heavier elements. Basis sets developed for full relativistic calculations (including spin-orbit coupling) can be found in the directory **basis**.

FILE: www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/howto_uncontract.md

orphan

How to uncontract basis sets

It is boring to uncontract basis sets manually. There is a keyword for it:

```
**DIRAC
[...]
**INTEGRALS
*READIN
.UNCONTRACT
```

Note that there is the limit for maximum number of exponents in one shell, which specified in the "include/cbirea.h" file (as MAXPRD = 35).

FILE: www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/molecule_using_mol.md

orphan

How to specify the molecule and basis set using the traditional mol files

The procedure used to read mol files is as follows: first, the basis set for the large component will be read in from the the mol file; then the small component basis set is generated automatically via the kinetic balance prescription, but can (by experienced users) also be specified explicitly. Try this only if you have studied the theory and know what the criteria for small component basis sets are, there are many pitfalls that one needs to be aware of!

[!WARNING] The traditional mol file is entered in fixed format (with few exceptions). This means that exact position and spacing matters. If you misplace the charge, the number of centers, etc., then DIRAC will not be able to read the structure and basis set information correctly. Until you get familiar with the format, we recommend to copy existing mol files and adapt them.

We will learn the mol file format using the following example:

DIRAC

```
C      2
      1.      2
H1    -1.4523499293      .0000000000      .8996235720
H2     1.4523499293      .0000000000      .8996235720
LARGE BASIS cc-pVDZ
      8.      1
O      .0000000000      0.0000000000      -.2249058930
LARGE BASIS cc-pVDZ
FINISH
```

This is the water molecule using the cc-pVDZ basis for the large component basis set. The first 3 lines are arbitrary comment lines. You can leave them blank or write some note to yourself but they have to be there. Line 4 specifies a cartesian GTO basis set (C) and 2 atom types (basis set types). So after line 4 DIRAC expects 2 basis set types and here they are - first hydrogen:

```
1.      2
H1    -1.4523499293      .0000000000      .8996235720
H2     1.4523499293      .0000000000      .8996235720
LARGE BASIS cc-pVDZ
```

then oxygen:

```
8.      1
O      .0000000000      0.0000000000      -.2249058930
LARGE BASIS cc-pVDZ
```

and we finish off with a FINISH in the final line.

You can already guess the meaning of the numbers. We have 2 hydrogens with a nuclear charge of 1.0 and one oxygen with a nuclear charge of 8.0. Then come the atom symbols and the coordinates in bohr followed by the large component basis set label. DIRAC will look up this label in the basis set library and complain if it does not exist. Note that the atom label (H1, H2, O) is only for you so that you find it in the output. It does not matter to the code. The code will identify basis sets based on the nuclear charge, not the label. The cartesian coordinates may be given in free format. However, the name of the atom must still be left four places, and no coordinates must enter the four first positions.

DIRAC typically runs in cartesian GTO. Note that by default the transformation to spherical harmonic basis (modified for the small components) is done in the transformation to the orthogonal basis. Spherical Gaussians (S) are allowed in 2-component calculations.

Default coordinates are in bohr, how can I use angstrom instead?

See the “A” in line 4 in position 20? If it is there, DIRAC will read in angstrom, if this position is blank, then DIRAC will read in bohr (default):

```
DIRAC
1234567890123456789012345678901234567890 # this is a comment line
C      2      A      # this is a comment line
      1.      2
[...]
```

Nuclear exponent

You can modify the default gaussian exponent for the nuclear charge distribution by giving a floating point value somewhere within the space specified below:

```
DIRAC
1234567890123456789012345678901234567890 # this is a comment line
C      2      A      # this is a comment line
      1.      2-----here-----
[...]
```

As an example, here is the Ne atom with a custom nuclear exponent of 10.0D8:

```
DIRAC
      -----here-----
C      1
      10.      1      10.0D+08
Ne     0.0  0.0  0.0
LARGE BASIS cc-pVDZ
FINISH
```

Fitting basis set

In the development version of DIRAC it is now possible to make use of density fitting. In such calculations, it is mandatory to include a basis set used to expand the density. This is done adding a line with the FTSET keyword after the coordinates for a given atom type. Basically all the options available in the definition of large or small basis sets are applicable to the fit set (i.e. the use of a set from the library, explicitly typed basis, even/well-tempered etc). Here is an example:

DIRAC

```
C      2
```

```

      1.      2
H1   -1.4523499293      .0000000000      .8996235720
H2    1.4523499293      .0000000000      .8996235720
LARGE BASIS cc-pVDZ
FTSET BASIS Ahlrichs-Coulomb-Fit
      8.      1
O     .0000000000      0.0000000000      -.2249058930
LARGE BASIS cc-pVDZ
FTSET BASIS Ahlrichs-Coulomb-Fit
FINISH

```

Point charges

You can add point charges with the “LARGE NOBASIS” basis set specification (or, equivalently, with “LARGE POINTCHARGE”). For example adding two point charges, both of -1.6, to the water example above:

DIRAC

```

C      3
      1.      2
H1   -1.4523499293      .0000000000      .8996235720
H2    1.4523499293      .0000000000      .8996235720
LARGE BASIS cc-pVDZ
      8.      1
O     .0000000000      0.0000000000      -.2249058930
LARGE BASIS cc-pVDZ
      -1.6      2
XX1   .0000000000      1.0000000000      .0000000000
XX2   .0000000000     -1.0000000000      .0000000000
LARGE NOBASIS
FINISH

```

Multiple basis sets from the basis set library

As an alternative to the BASIS option described above, it is possible to use specify different basis set files via the MULTIBASIS keyword. The MULTIBASIS option is present to allow easier inclusion of different sets of diffuse and/or polarization functions to a reference basis set. The syntax for this keyword is very similar to that of the BASIS:

DIRAC

```

C      2
      1.      2
H1   -1.4523499293      .0000000000      .8996235720
H2    1.4523499293      .0000000000      .8996235720
LARGE MULTIBASIS 2 cc-pVDZ cc-pVDZ-diffuse
      8.      1
O     .0000000000      0.0000000000      -.2249058930
LARGE MULTIBASIS 2 cc-pVDZ cc-pVDZ-diffuse
FINISH

```

After the keyword an integer with the number of files to be read should be specified. It is followed by the name of the different basis set files, each separated by a whitespace. The only limitation for the number of basis set files is that the total length of this line should not exceed Fortran’s maximum allowed line size (80 characters).

Explicitly typed basis sets

Explicitly typed basis sets are best described using an explicit example:

DIRAC

```

C      2
      8.      1
O     .0000000000      0.0000000000      -.2249058930
LARGE EXPLICIT 3 1 1 1
# s functions
f 10 4
11720.0000000 0.00071000 -0.00016000 0.00000000 0.00000000
1759.0000000 0.00547000 -0.00126300 0.00000000 0.00000000
400.8000000 0.02783700 -0.00626700 0.00000000 0.00000000
113.7000000 0.10480000 -0.02571600 0.00000000 0.00000000
37.0300000 0.28306200 -0.07092400 0.00000000 0.00000000
13.2700000 0.44871900 -0.16541100 0.00000000 0.00000000
5.0250000 0.27095200 -0.11695500 0.00000000 0.00000000
1.0130000 0.01545800 0.55736800 0.00000000 0.00000000
0.3023000 -0.00258500 0.57275900 1.00000000 0.00000000
0.0789600 0.00000000 0.00000000 0.00000000 1.00000000
# p functions
f 5 3
17.7000000 0.04301800 0.00000000 0.00000000
3.8540000 0.22891300 0.00000000 0.00000000
1.0460000 0.50872800 0.00000000 0.00000000
0.2753000 0.46053100 1.00000000 0.00000000
0.0685600 0.00000000 0.00000000 1.00000000
# d functions
f 2 2
1.1850000 1.00000000 0.00000000
0.3320000 0.00000000 1.00000000
[...]
```

Following EXPLICIT is the highest angular quantum number plus one. In this case it is 3, since we are using a *spd* basis set. Following this number are the number of blocks for each *l*-value. The memory requirements grow rapidly with the number of basis functions in a block (note for instance that four *g* functions actually are 60 basis functions, as there are 15 cartesian components of each *g* function). Memory requirements can therefore be reduced by splitting basis functions of the quantum number into different blocks. This will, however, decrease the performance of the integral calculation.

Following the LARGE EXPLICIT line we see a line with a comment. Lines starting with either !, \$, or # are comments and are ignored by the code.

The next line starts with a single character describing the input format of the basis set in this block.

The default format is 8F10.4 which will be used if left blank. **Be very careful when using this default format as it will miss any exponential parameter standing to the right of the 10 characters.** In this format the first column is the orbital exponent and the seven last columns are contraction coefficients. If no numbers are given, a zero is assumed. If more than 7 contracted functions occur in a given block, the contraction coefficients may be continued on the next line, but the first column (where the orbital exponents are given) must then be left blank.

An F or f in the first position (like in the example above) will indicate that the input is in free format. This will of course require that all contraction coefficients need to be typed in, as all numbers need to be present on each line. However, note that this options is particularly handy together with completely decontracted basis sets, as described below. Note that the program reads the free format input from an internal file that is 80 characters long, and no line should therefore exceed 80 characters.

The numbers in the line “f 10 4” are the number of of primitive Gaussians in this block (10) and the number of contracted Gaussians in this block (4). If a zero is given for the number of contracted Gaussians, an uncontracted basis set will be assumed, and only orbital exponents need to be given.

One may also give the format H or h. This corresponds to high precision format 4F20.8, where the first column again is reserved for the orbital exponents, and the three next columns are designated to the contraction coefficients. If no number is given, a zero is assumed. If there are more than three contracted orbitals in a given block, the contraction coefficients may be continued on the next line, though keeping the column of the orbital exponents blank.

Specification of how to generate small component functions

Coming back to the example above we can add another integer following “f 10 4”, which will specify how to generate small component functions:

DIRAC

```
C      2
      8.      1
0      .0000000000      0.0000000000      -.2249058930
LARGE EXPLICIT 3      1      1      1
# s functions
f 10      4      0
11720.0000000 0.00071000 -0.00016000 0.00000000 0.00000000
1759.0000000 0.00547000 -0.00126300 0.00000000 0.00000000
400.8000000 0.02783700 -0.00626700 0.00000000 0.00000000
113.7000000 0.10480000 -0.02571600 0.00000000 0.00000000
37.0300000 0.28306200 -0.07092400 0.00000000 0.00000000
13.2700000 0.44871900 -0.16541100 0.00000000 0.00000000
5.0250000 0.27095200 -0.11695500 0.00000000 0.00000000
1.0130000 0.01545800 0.55736800 0.00000000 0.00000000
0.3023000 -0.00258500 0.57275900 1.00000000 0.00000000
0.0789600 0.00000000 0.00000000 0.00000000 1.00000000
# p functions
f 5      3      0
17.7000000 0.04301800 0.00000000 0.00000000
3.8540000 0.22891300 0.00000000 0.00000000
1.0460000 0.50872800 0.00000000 0.00000000
0.2753000 0.46053100 1.00000000 0.00000000
0.0685600 0.00000000 0.00000000 1.00000000
# d functions
f 2      2      0
1.1850000 1.00000000 0.00000000
0.3320000 0.00000000 1.00000000
[...]
```

If a 0 (as in this example) or no number is given, the small component functions are generated both upwards and downwards (default). If the number is 1, the small component functions are generated upwards, If the number is 2, the small component functions are generated downwards. For other values no small components functions generated.

MOLFDIR-type basis sets

The MOLFDIR basis set file(s) (here: Oxygen-xyz.bas) must be copied to the scratch area, for example using the pam script:

DIRAC

```
C      2
      1.      2
H1 -1.4523499293      .0000000000      .8996235720
H2 1.4523499293      .0000000000      .8996235720
LARGE MOLFBAS Hydrogen-xyz.bas
      8.      1
0      .0000000000      0.0000000000      -.2249058930
LARGE MOLFBAS Oxygen-xyz.bas
FINISH
```

Even-tempered basis sets (geometric progressions)

The exponents are generated in an even-tempered series:

$$\eta_{N-k+1} = \alpha \beta^{k-1}$$

with

$k = 1, \dots, N$

```
LARGE EVENTEMP 0.05 3.0 11 3 2 1 1
1..5
6..11
7..11
9..10
```

This means that alpha is 0.05, beta is 3.0, N is 11. Highest angular quantum number plus one is 3 (s, p, and d functions), we will have 2 blocks for the s, 1 block for p, and one block for d. The s blocks go from exponent 1 to 5 and from 6 to 11, p goes from 7 to 11, d goes from 9 to 10.

Well-tempered basis sets

This works very much like LARGE EVENTEMP, except that the exponents are generated in an well-tempered series:

$\eta_N = \alpha$

and

$\eta_{N-k+1} = \eta_{N-k+2} \beta \left[1 + \gamma \left(\frac{k}{N} \right)^n \right]$

with

$k = 1, \dots, N$

```
LARGE WELLTEMP 0.05 2.5 2.0 6.0 11 3 2 1 1
1..5
6..11
7..11
9..10
```

Here alpha is 0.05, beta is 2.5, gamma is 2.0, n is 6.0, and N is 11.

Explicit small component basis set

This is for experts. Following the line or block specifying the LARGE component basis set you can override the default kinetic balance prescription and specify the small component basis set either using SMALL EXPLICIT or using SMALL MOLFBAS, in analogy to LARGE EXPLICIT and LARGE MOLFBAS. After the large component basis set the small component basis set can be specified. If nothing is specified it is equivalent to specifying SMALL KINBAL (see below). There are three possibilities for giving the basis set.

Family basis sets

Input for basis sets where the same set of exponents are used for all functions. This is analogous to the well- and even-tempered basis sets except that the exponents are not calculated from a formula, but must be given in the file. These exponents may come from a basis set optimization with GRASP:

DIRAC

```
C 2
      8.      1
0      .0000000000      0.0000000000      -.2249058930
LARGE FAMILY 10 3 1 1 1
# exponents
6665.000000
1000.000000
228.000000
64.710000
21.060000
7.495000
2.797000
0.521500
0.159600
0.046900
# ranges
1..10
6..10
8..9
[...]
```

Dual family basis sets

A basis set analogous to the family basis set, except one set of exponents are used for s, d, g, ... (gerade) functions, and another set is used for p, f, h, ... (ungerade) functions:

DIRAC

```
C 2
      8.      1
0      .0000000000      0.0000000000      -.2249058930
LARGE DUALFAMILY 10 5 3 1 1 1
# s, d, g, ...
6665.000000
1000.000000
228.000000
64.710000
```



```

21.0600000
7.4950000
2.7970000
0.5215000
0.1596000
0.0469000
# p, f, h, ...
9.4390000
2.0020000
0.5456000
0.1517000
0.0404100
# ranges
1..10
1..5
8..9
[...]
```

How to set the charge of the system

By default the charge of the atom/molecule is zero. You can change this in the mol file:

DIRAC

```

*** <- you can use these positions on next line
C 2 1
    1. 2
H1 -1.4523499293 .0000000000 .8996235720
H2 1.4523499293 .0000000000 .8996235720
LARGE BASIS cc-pVDZ
    8. 1
O .0000000000 0.0000000000 -.2249058930
LARGE BASIS cc-pVDZ
FINISH
```

This is water with charge plus 1. Alternatively you can specify the occupation explicitly in the inp file and leave it blank in the mol file.

Automatic augmentation with diffuse functions

By using a augmentation prefix you can use automatic augmentation to extend Dunning, Dyall, and Turbomole basis sets with diffuse exponents.

The following augmentation prefixes are recognized:

```

d-aug-cc-p...
t-aug-cc-p...
q-aug-cc-p...
5-aug-cc-p...
6-aug-cc-p...
7-aug-cc-p...

s-aug-dyall...
d-aug-dyall...
t-aug-dyall...
q-aug-dyall...

s-a-Turbomole...
d-a-Turbomole...
t-a-Turbomole...
q-a-Turbomole...
```

What the code does is that it sorts exponents for each angular momentum, calculates the factor between the two most diffuse exponents, and adds a new exponent (or new exponents) by keeping this factor (even tempered augmentation). If an angular momentum only has one exponent, then the progression cannot be calculated (obviously), and a factor 3.5 is used. This is a choice.

Automatic augmentation can be useful but it should not be used in a black box manner. You are strongly encouraged to define `DEBUG_PRIMITIVES` in `abacus/herndn.F` and to examine which exponents have been added. The code simply augments and does not consider whether it uses the original SCF set exponents or correlating exponents.

How to force specific symmetry

In all the above examples we have not specified symmetry explicitly. In this case DIRAC will detect and use the highest symmetry available in the code. In doing so it may translate and rotate the molecule (check output).

There are two alternatives to this. The first is that we can turn off symmetry completely and force C1 (by placing a 0 at the right place; the 0 means zero symmetry generators):

DIRAC

```

C 2 0
[...]
```

DIRAC

Alternative two is to specify an explicit symmetry other than C1. For this we need to remove all symmetry-generated centers and specify the coordinates only for the symmetry-unique centers:

DIRAC

```

C   2   2   X   Y
   8.   1
O   .0000000000   0.000000000   -.2249058930
LARGE BASIS cc-pVDZ
   1.   1
H   1.4523499293   .000000000   .8996235720
LARGE BASIS cc-pVDZ
FINISH

```

We give only one hydrogen center, the other is generated by symmetry operations. This molecule has two symmetry generators (second 2 in line 4).

[!WARNING] If you fail to remove the centers generated by symmetry operations, DIRAC will apply symmetry operations to them which will re-generate the symmetry-unique centers. You will get more than one center at the same position and DIRAC will stop with the error “nuclei too close”.

All available symmetry groups and the corresponding symmetry group specification strings are given in the following list:

```

C   ?   3   Z   Y   X   # group:      D2h
                        # operations: reflection(xy)
                        #               reflection(xz)
                        #               reflection(yz)

C   ?   2XY  YZ        # group:      D2
                        # operations: rotation(z)
                        #               rotation(x)

C   ?   2   Y   X        # group:      C2v
                        # operations: reflection(xz)
                        #               reflection(yz)

C   ?   2   ZXYZ        # group:      C2h
                        # operations: reflection(xy)
                        #               inversion

C   ?   1XY             # group:      C2
                        # operations: rotation(z)

C   ?   1   Z           # group:      Cs
                        # operations: reflection(xy)

C   ?   1XYZ            # group:      Ci
                        # operations: inversion

C   ?   0               # group:      C1

```

FILE: www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/molecule_using_xyz.md

orphan

How to specify the molecule and basis set using xyz files

The actual xyz file only contains the number of centers, the atoms, and the corresponding coordinates (by default in angstroms). It could look like this one:

```

6
Methanol, identifying the alcoholic proton as H1
C   -0.000000010000   0.138569980000   0.355570700000
O   -0.000000010000   0.187935770000   -1.074466460000
H   0.882876920000   -0.383123830000   0.697839450000
H   -0.882876940000   -0.383123830000   0.697839450000
H   -0.000000010000   1.145042790000   0.750208830000
H1  -0.000000010000   -0.705300580000   -1.426986340000

```

The first line is the number of atoms. The second line can be blank or an arbitrary comment. It is not read by the program. If you omit this line, then your molecule will be wrong, one atom will be missing! The following lines contain atom types and coordinates. The first entry on each line is the name of the atom. Any string up to eight characters will do, but if the name is present in the periodic table DIRAC will recognize the charge and can find the corresponding basis set.

From the xyz file DIRAC cannot know what basis set you want to use. For this reason, basis set and symmetry specification is then done in the input file using the keywords given below. This is in contrast to the traditional mol files. This means that the keywords below do not make sense when combined with a traditional mol file.

*BASIS

Here the basis set information is specified. .DEFAULT specifies the default large component basis set for all atoms. This can be modified for specific atom by the .SPECIAL keyword. For example, if we use the methanol.xyz file listed above, the following input specifies a cc-pVDZ on carbon and the 3 hydrogens labeled H, while oxygen and H1 will be treated with a cc-pVTZ basis:

```

**MOLECULE
*BASIS
.DEFAULT
cc-pVDZ
.SPECIAL
0 BASIS cc-pVTZ
.SPECIAL
H1 BASIS cc-pVTZ

```

*CENTERS

Here the physical properties of the centers in our molecule can be specified. This is useful if the name of the center differs from the IUPAC element name or in case we want to use nuclei with fractional charges. This is controlled by the keyword `nucleus` that can be repeated as often as is needed. For the geometry file `methanol.xyz` we need to specify:

```
*CENTERS
.NUCLEUS
H1 1.0
```

Another reason to change the charge of the nucleus is to be able to run counterpoise calculations in which a ghost basis set is to be placed without adding a charge. This can be achieved by defining the ghost center as (for instance) `O.Gh` and then putting the nuclear charge to zero using:

```
*CENTERS
.NUCLEUS
O.Gh 0.0 8.0
```

the first number is the charge to be used, the second corresponds to the element number and is used to search the basis set library. In this example you can therefore simply specify the basis set name using the *DEFAULT* or *SPECIAL* keywords and the program will then find the corresponding oxygen basis.

*COORDINATES

Here you can specify the units to be used when reading the coordinates. If nothing is specified, angstrom is the default (this is then a real xyz file). If we specify 'AU' we use atomic units instead:

```
*COORDINATES
.UNITS
AU
```

We can also specify our own unit, by typing the name and the conversion factor when going from this new unit to atomic units:

```
*COORDINATES
.UNITS
nm 18.8972
```

Now all coordinates are entered in nanometer. If we add 'A' at the end, we can enter the new unit in angstrom:

```
*COORDINATES
.UNITS
nm 10.0 A
```

*SYMMETRY

With this keyword we can control symmetry recognition. To turn symmetry detection off and to force C1 symmetry use:

```
*SYMMETRY
.NOSYM
```

Including ghost centers and point charges

In some situations you may want to add a ghost center, for instance in counterpoise calculations (see `bsse`) or to lower the symmetry detected by DIRAC (see `case_CmF`). An example of the latter case is to take out inversion symmetry for an atom:

```
2
Ne      0.0 0.0 0.0
foo     0.0 0.0 10.0
```

We give a non-standard name to the ghost center, so that DIRAC does not find a nuclear charge. This has to be accompanied by telling DIRAC not to add a basis to the ghost center:

```
**MOLECULE
*BASIS
.DEFAULT
dya11.2zp
.SPECIAL
foo NOBASIS
```

In a counterpoise calculation, on the other hand, you want the center to carry a basis set. For a ghost neon atom you would then specify:

```
**MOLECULE
*CENERS
.NUCLEUS
foo 0.0 10.0
```

As already explained, the first number on the last line is the charge of the center, and the second the nuclear charge used to find the basis. If you actually want to add a point charge, let us say of charge -2.3, you can use the xyz-file above, but now you want charge, but not basis, and write:

```
**MOLECULE
*BASIS
.DEFAULT
dya11.2zp
.SPECIAL
foo NOBASIS
*CENERS
```

.NUCLEUS
foo -2.3

FILE: www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/molecule_with_ecp.md

orphan

How to specify ECP parameters in mol files

Effective core potential (ECP) is an efficient method in many quantum mechanical calculations. It reduces basis set demands in heavy elements by replacing core electrons with an effective potential. To account of spin-orbit and other relativistic effects, many recently developed ECP parameters are based on atomic Dirac-Fock calculations, and are sometimes called the relativistic effective core potentials (RECP).

Following the usual practice the point nucleus model is used by default for ECP calculations.

Theory

Originally, the spin-orbit term is included in the RECP (SOREP) as following Lee1977,

$$U^{\text{SOREP}} = U_{\{L\}}^{\text{SOREP}}(r) + \sum_{l=0}^{L-1} \sum_{j=|l-1/2|}^{l+1/2} \sum_{m=-j}^j [U_{\{lj\}}^{\text{SOREP}}(r) - U_{\{L\}}^{\text{SOREP}}(r)] |l j m\rangle \langle l j m|$$

The SOREP is divided into two types of potential - averaged relativistic effective core potential (AREP) and spin-orbit potential. ($U^{\text{SOREP}} = U^{\text{AREP}} + U^{\text{SO}}$) Ermler1981

$$U^{\text{AREP}} = U_{\{L\}}^{\text{AREP}}(r) + \sum_{l=0}^{L-1} \sum_{m=-l}^l [U_{\{l\}}^{\text{AREP}}(r) - U_{\{L\}}^{\text{AREP}}(r)] |l m\rangle \langle l m|$$

where

$$U_{\{l\}}^{\text{AREP}} = \frac{1}{2l+1} [\frac{1}{l} \cdot U_{\{l, l-1/2\}}^{\text{SOREP}}(r) + (l+1) \cdot U_{\{l, l+1/2\}}^{\text{SOREP}}(r)]$$

The spin-orbit potential (U^{SO}) is defined as,

$$U^{\text{SO}} = s \cdot \sum_{l=1}^L \frac{2}{2l+1} \Delta U_{\{l\}}^{\text{SOREP}}(r) \sum_{m=-l}^l \sum_{m'=-l}^l |l m\rangle \langle l m'|$$

with

$$\Delta U_{\{l\}}^{\text{SOREP}}(r) = U_{\{l, l+1/2\}}^{\text{SOREP}}(r) - U_{\{l, l-1/2\}}^{\text{SOREP}}(r)$$

ECP parameters from the library

See the simple example file from the ecp test (HI_ecplib.mol <../test/ecp/HI_ecplib.mol>):

```
../test/ecp/HI_ecplib.mol
```

1. Basis set library for ECP (line 7): The format is same as the large component basis set. (see large component basis set) However, one should use the basis set parameters corresponding to the RECP.

2. ECP parameters from the library (line 8): ECPLIB keyword should be used for the use of RECP parameters from the library.

Note: In DIRAC program, some ECP libraries are provided in basis_ecp directory for your conveniences. However, it is safe to check ECP parameters from authors' webpages. The name of ECP parameters used here is following, ECP(XX)(YY)(ZZ)(SO/SF).

- XX: representative authors, (CE=Christiansen, Ermler, and coworkers), (DS=Dolg, Stoll, and coworkers), (HW=Hay, Wadt, and coworkers), (SB=Stevens, Basch, and coworkers)
- YY: number of core electrons replaced by potential
- ZZ: further information (if exist),
- (SO/SF) : spin-orbit(SOREP) or spin-free(AREP), The difference of SO and SF is the existence of SO parameters.

For example, ECPDS60MDFSO indicates Stuttgart RECP from Dolg, Stoll, and coworkers with 60 electrons replaced by potential which is fitted from relativistic calculation (spin-orbit potential is included).

Correlation-consistent basis sets by Kirk Peterson and co-workers that can be used with the Stuttgart-Cologne ECPs are available [here](#).

Explicitly typed ECP

```
INTGRL
HI MOLECULE
I:Christiansen RECP, H:Aug-cc-pVTZ
C 2 2 Y Z A
53. 1
I 1.59900000 0.00000000 0.00000000
LARGE BASIS ECPCE46_TZ
ECP 46 4 3
# AREP
# f
```

```

4
2      .922500      -1.447005
2      2.569100      -14.188832
2      7.908600      -43.263306
1      25.061100      -27.740856
# s-f
6
2      1.503600      -124.037542
2      1.874600      235.545458
2      2.682800      -261.475363
2      3.446600      144.184313
1      1.124200      31.003269
0      11.458300      6.512373
# p-f
6
2      1.245400      -95.368794
2      1.582600      188.963297
2      2.242900      -221.153567
2      2.930100      104.680452
1      .950300      30.809252
0      12.782000      5.414640
# d-f
6
2      .668500      -50.340552
2      .828000      102.276429
2      1.115700      -133.296812
2      1.419000      75.010821
1      .503000      19.150171
0      4.557600      8.099959
# spin-orbit
# p-f
6
2      1.245400      5.416752
2      1.582600      3.711418
2      2.242900      -25.208180
2      2.930100      27.780892
1      .950300      -2.699622
0      12.782000      .351446
# d-f
6
2      .668500      -.406356
2      .828000      2.040806
2      1.115700      -3.283400
2      1.419000      1.842046
1      .503000      -.126998
0      4.557600      .008442
# f
4
2      .922500      -.019085
2      2.569100      .035451
2      7.908600      -.007995
1      25.061100      .096830
1.      1
H      0.00000000      0.00000000      0.00000000
LARGE BASIS aug-cc-pVTZ
FINISH

```

For the explicitly typed ECP, the ECP keyword is used. Three numbers after the ECP keyword denote the following:

1. Number of core electrons: This number of core electrons is substituted by RECP. In this case, 46 core electrons in iodine atom are omitted and 7 electrons in the valence are described.
2. Number of AREP blocks: The numbers of AREP blocks to be read.
3. Number of SO blocks: The numbers of SOREP blocks to be read. In the case of AREP (spin-free) calculation, the number of SOREP blocks is set to 0.

After reading block numbers, AREP (and SOREP) blocks are read.

1. AREP blocks: Three parameters in each line in AREP blocks are n_{li} , α_{li} , and C_{li} respectively in the following equation.

$$U_{li} = \sum_{li} C_{li} r^{n_{li}-2} e^{-\alpha_{li} r^2}$$

2. SO blocks: Three parameters in each line in SO blocks are same as in AREP blocks.

Note: coefficients in the spin-orbit block are sometimes defined differently by RECP developing groups. Spin-orbit potential in DIRAC is defined as Park2012,

$$U_{li}^{SO, DIRAC} = \frac{2}{2l+1} \Delta U_{li}^{SOREP}$$

For example, factors $(2/2l+1)$ are already multiplied in SO parameters at the [Stuttgart RECP webpage](http://www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/troubleshooting.md). One can use it directly without the modification. There are two kinds of SO factors in the [RECPs of Christiansen and coworkers](http://www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/troubleshooting.md). The SO factor in DIRAC is the same as the ones defined for Columbus program.

FILE: www.diracprogram.org/doc/release-26/_sources/molecule_and_basis/troubleshooting.md

orphan

GETPOT: Nuclei too close

If you see this error then one of the following situations is likely:

- You have two or more atoms on top of each other in your mol or xyz file.
- You specify symmetry operations explicitly and have too many atoms (remove symmetry-

generated centers). - You use .OPTIMIZE together with xyz format (use mol format instead).

FILE: www.diracprogram.org/doc/release-26/_sources/pam/mpi_command.md

orphan

How to define an alternative MPI launcher

The pam script by default launches MPI runs with mpirun. If you cannot use mpirun because you are on a system which does not support it or which offers alternative MPI launchers (SGI, Cray), you can define your own DIRAC_MPI_COMMAND like this:

```
$ export DIRAC_MPI_COMMAND="mpirun -np 8"
$ export DIRAC_MPI_COMMAND="mpprun"
$ export DIRAC_MPI_COMMAND="aprun -n 24"
```

This will then launch:

```
$DIRAC_MPI_COMMAND dirac.x
```

instead of:

```
mpirun -np N dirac.x
```

Note that `--mpi` overrides DIRAC_MPI_COMMAND.

Passing arguments to MPI launcher

You can pass arguments to the MPI launcher, for example:

```
./pam --scratchfull="/tmp/milias/TEST" --noarch --mpiarg="-x PATH -x LD_LIBRARY_PATH --wdir '/tmp/milias/TEST'" --mpi=4 --inp=cc.inp --mol=N
```

Note that you have provide proper workdir (the `--wdir` flag) for your MPI launcher, otherwise it ends with error.

Concerning passing of environmental variables (through the `-x` flag), if the variable is not defined, the MPI launcher gives warning (like `Warning: could not find environment variable "LD_LIBRARY_PATHx"`).

FILE: www.diracprogram.org/doc/release-26/_sources/pam/replace.md

orphan

Replace strings in input files

The “`pam --replace`” is a fantastic option which we encourage you to use it. It replaces any string in input files by user defined string.

Try this:

```
**DIRAC
.WAVE FUNCTION
**HAMILTONIAN
.hamiltonian
**WAVE FUNCTION
.SCF
*END OF
```

and this:

```
DIRAC
.
.
C 1
10. 1
Ne 0.0 0.0 0.0
LARGE BASIS basis
FINISH
```

now run it with:

```
pam --replace hamiltonian=LEVY-LEBLOND --replace basis=cc-pVDZ
```

it will write to the output file:

```
inp_mol_hamiltonian=LEVY-LEBLOND_basis=cc-pVDZ.out
```

you get the point: you can now run whole tables with just two input files!

A nice application of this feature is to make a PES scan, written as a script:

```
for r in `seq 1.00 0.05 2.00`; do
    pam --replace distance=$r ..
done
```

Transfer renamed files

The pam scripts enables putting/getting renamed files. Thanks to this feature we are able to save DIRAC working files under own original names and ensure their renaming upon moving them to and from the scratch directory.

This feature is great for running many calculation at the same time without risking that files will be overwritten by concurrent runs.

Here we extract the DFCOEF file from the scratch directory and save it as DFCOEF.tl2.v3z in the home directory:

```
pam --get "DFCOEF=DFCOEF.tl2.v3z"
```

In next run we copy files to the scratch directory (where calculations occur) with their proper names:

```
pam --put "DFCOEF.tl2.v3z=DFCOEF"
pam --copy="/home/milias/my_scratch/Tl2_fsc02/MDCINT.18corr_el.v3z=MDCINT"
```

FILE: www.diracprogram.org/doc/release-26/_sources/pam/scratch.md

orphan

The scratch directory is the place where DIRAC will write temporary files. At the beginning of the run, DIRAC copies input files and the binary into the scratch space and at the end of the calculation, the output is copied back, possibly together with other useful (restart) files.

How you can set the scratch directory

You can set the scratch space like this:

```
$ pam --scratch=/somepath
```

If `--scratch` is not set, DIRAC will look for the environment variables `DIRAC_TMPDIR`.

If these variables are not set, DIRAC defaults to `$HOME/DIRAC_scratch_directory`.

On your local computer you typically want to use the same scratch space every time and you may be tired of typing `pam --scratch` every time. In this case, put `--scratch=/somepath` in your `~/diracrc` or export `DIRAC_TMPDIR` in your `~/bashrc` or equivalent.

DIRAC will always append `$USER/DIRAC_inp_mol_pid` to the scratch path. This is to prevent users from accidentally wiping out their home directory.

What if you don't want DIRAC to append anything to your scratch path

For this use:

```
$ pam --scratchfull=/somepath
```

In this case DIRAC will use the path as it is and not append anything to it. Be careful with this since DIRAC may remove the path after the calculation has finished.

How to set the scratch path on a cluster

On a cluster you typically want to be able to find the scratch directory based on some job id. If you are on a cluster with a Torque/PBS scheduler then it can be useful to set:

```
$ pam --scratch=/somepath/$USER/$PBS_JOBID
```

Cluster with a global scratch disk

This means that all nodes write temporary files to the same disk.

No additional flags are needed for this run.

Cluster with local scratch disks

This means that each node writes temporary files to its own disk.

For this situation the input files have to be distributed over the nodes based on `--machfile`:

```
$ pam --scratch=/somepath/$USER/$PBS_JOBID --machfile=/path/to/machfile
```

If you don't give `--machfile`, DIRAC assumes that you run using a global scratch disk.

The list of machines is typically generated when the scheduler launches the job but you can also set the list of machines manually.

The input files are by default distributed using rsh/rcp. If your cluster does not support these you can change to ssh/scp protocol:

```
$ pam --scratch=/somepath/$USER/$PBS_JOBID --machfile=/path/to/machfile --rsh=ssh --rcp=scp
```

FILE: www.diracprogram.org/doc/release-26/_sources/patches/CHANGELOG.md

orphan

26.0

New features:

- Basis set library has been extended with 5z basis sets for s- and p-block elements: New dyall.(v5z/cv5z/av5z/ae5z/acv5z/aae5z) basis sets.
 - Contributor: K. Dyall.
 - Zenodo links: [s-block elements](#) and [p-block elements](#).
 - Reference: [M. L. Reitsma, E. H. Prinsen, J. D. Polet, A. Borschevsky, and K. G. Dyall, Relativistic quintuple-zeta basis sets for the s block Available to Purchase, J. Chem. Phys. 163, 144116 \(2025\).](#)
- Merge of the DIRAC_CASPT2 program published by Masuda and coworkers into DIRAC.
 - This adds the following features:
 - IVO calculation (Huzinaga1970).
 - CASCI/CASPT2 calculation (Anderson1990 but the reference function is CASCI not CASSCF).
 - RASCI/RASPT2 calculation.
 - Contributors: K. Noda, S. Iwamuro, Yasuto Masuda and M. Abe, Tokyo University of Agriculture and Technology, University of Hiroshima, Japan.
 - Reference: [Yasuto Masuda, Kohei Noda, Sumika Iwamuro, Masahiko Hada, Naoki Nakatani, and Minori Abe, Relativistic CASPT2/RASPT2 Program along with DIRAC Software, J. Chem. Theor. Comput. 21, 1249 \(2025\).](#)
- Schiff moment operator:
 - Contributor: K. Gaul.
 - Reference: [Konstantin Gaul, Nicholas Hutzler, Phelan Yu, Andrew Jayich, Miroslav Ilias and Anastasia Borschevsky, CP-violation sensitivity of closed-shell radium-containing polyatomic molecular ions, Phys. Rev. A 109, 042819 \(2024\).](#)
- Calculation of parity-violating EFG tensors. Activated with “PVCEFG”
 - Contributor: Juan J. Aucar.
 - Reference: [Juan J. Aucar and Alejandro F. Maldonado, Parity Violation Effects on the Electric Field Gradient, Phys. Chem. Chem. Phys. 27, 7594 \(2025\).](#)
 - Manual: PVCEFG <https://diracprogram.org/doc/release-26/manual/properties.html#pvcefg>.
- ExaCorr: new functionality
 - Support via the TAPP API to tensor libraries (e.g. the TBLIS library for single-node CPU architectures), in addition to TAL-SH (single-node CPU or GPU architectures) and ExaTENSOR (distributed memory CPU or GPU architectures). The choice of tensor library is made on compilation.
 - Contributors: J. Brandejs, A. S. P. Gomes, L. Visscher
 - Reference: [J. Brandejs, N. Hörnblad, E. F. Valeev, A. Heinecke, J. Hammond, D. Matthews, P. Bientinesi, Tensor Algebra Processing Primitives \(TAPP\): Towards a Standard for Tensor Operations arXiv:2601.07827 \(2026\)](#)
 - Manual: https://diracprogram.org/doc/release-26/manual/wave_function/exacorr.html#executor and https://diracprogram.org/doc/release-26/installation/tensor_libraries.html
 - Orbital-unrelaxed quadratic response coupled cluster molecular properties.
 - Contributors: X. Yuan, L. Halbert, L. Visscher, A. S. P. Gomes
 - Reference: [X. Yuan, L. Halbert, L. Visscher, A. S. P. Gomes, A Comparison of Relativistic Coupled Cluster and Equation of Motion Coupled Cluster Quadratic Response Theory J. Phys. Chem. A 129, 11695-11712 \(2025\)](#)
 - Manual: https://diracprogram.org/doc/release-26/manual/wave_function/exacorr.html#ccciqr.
 - Ground-state expectation values with higher-order coupled cluster (CCSDT, CCSDTQ).
 - Contributors: G. Fabbro, J. Brandejs, T. Saue
 - Reference: [G Fabbro, J Brandejs, T Saue, Highly Accurate Expectation Values Using High-Order Relativistic Coupled Cluster Theory, J. Phys. Chem. A 129, 6942 \(2025\)](#)
 - Manual: https://diracprogram.org/doc/release-26/manual/wave_function/exacorr.html#wavefunctions-for-iterative-methods
 - CIS and CIS(D) excitation energies.
 - Contributors: X. Yuan, T. Lejeune, A. S. P. Gomes
 - Manual: https://diracprogram.org/doc/release-26/manual/wave_function/exacorr.html#excited-state-calculation-options
 - Virtual frozen natural orbitals for response properties.
 - Contributors: M. Le, X. Yuan, A. S. P. Gomes
 - Manual: https://diracprogram.org/doc/release-26/manual/wave_function/exacorr.html#id10
- VISUAL: new functionality
 - Improved data handling and more calculation options
 - The real-space data can be exported in TXT/CSV/CUBE/HDF5 formats
 - MPI parallelization
 - Possibility to calculate various densities in the same run on the same or different grids
 - Improved user input: more flexibility to define grids and densities (“grid functions”)
 - Selected tutorials are translated to jupyter notebooks (added to demo_notebooks/)
 - Contributor: Gosia Olejniczak
 - Manual: <https://diracprogram.org/doc/release-26/manual/visual.html>
- RELCCSD: print out largest amplitudes (for analysis) at print level 2.
 - Contributor: Lucas Visscher.
 - Manual: https://diracprogram.org/doc/release-26/manual/wave_function/relccsd.html#print
- TREXIO-converter: Jupyter notebook to convert (non-relativistic) wave function information to the TREXIO format.
 - Contributor: Lucas Visscher.
 - Manual: <https://diracprogram.org/doc/release-26/manual/data.html>

25.0

New features:

- Effective QED potentials:
 - Contributors: A. Sunaga and T. Saue.
 - Reference: [Ayaki Sunaga, Maen Salman and Trond Saue, 4-component relativistic Hamiltonian with effective QED potentials for molecular calculations, J. Chem. Phys. 157, 164101 \(2022\).](#)
 - Vacuum polarization: Uehling.
 - Electron self-energy: Flambaum-Ginges and Pyykkö-Zhao.
 - Tutorial: <http://www.diracprogram.org/doc/release-25/tutorials/qed/effqed.html>.
- Atomic shift operator:
 - Contributor: T. Saue.
 - Reference: [Ayaki Sunaga, Maen Salman and Trond Saue, 4-component relativistic Hamiltonian with effective QED potentials for molecular calculations, J. Chem. Phys. 157, 164101 \(2022\).](#)
 - Manual: [ASHIFT](#).
- New ESR/EPR features:
 - Contributor: H. J. Aa. Jensen.
 - .ESR CI flexible option for defining CI for ESR/EPR calculations.
 - .G10PPM for *ESR : request g-shifts to 10 ppm accuracy instead of default of +/-1 ppt = 1000 ppm accuracy.
 - Added .STATE option for *ESR, defining first CI state belonging to ESR multiplet (for example if a “singlet” state is below the “triplet” multiplet).
 - .MAXMK2 for *ESR.
 - .KR CONJ to generate the second state for doublets by Kramers conjugation.
- KRCI module:
 - Contributor: H. J. Aa. Jensen.
 - *.CIROOTS input has been made easier, description in manual has been updated.
 - The KRCI module now again works with both LUCIAREL and GASCIP (a fix from 2021 fixed symmetry specification for LUCIAREL but made it not work for GASCIP).
 - GASCIP code optimized, much faster now.
- DIRAC calculator for the Atomistic Simulation Environment (ASE) is available.
 - Contributors: Carlos M. R. Rocha and Lucas Visscher.
 - Reference (DIRAC calculator): <https://qc2.readthedocs.io>
 - Reference (ASE): <https://wiki.fysik.dtu.dk/ase/>
- Approximations to EOM-CCSD (Partitioned EOM (P-EOM-CCSD); second-order perturbation theory-based EOM (EOM-MBPT(2) and P-EOM-MBPT(2)) are available in the RELCCSD module for excitation and ionization energies.
 - Contributors: L. Halbert and A. Gomes.
 - Reference: [L. Halbert and A. S. P. Gomes, The performance of approximate equation of motion coupled cluster for valence and core states of heavy element systems, Mol. Phys. e2246592 \(2023\).](#)

Infrastructure changes and fixes:

- Better cmake structure for compiling DIRAC. Now utility programs are small and not the same size as dirac.x.
 - Contributor: H. J. Aa. Jensen.
- Changed all references to ifort etc. to ifx etc from intel oneAPI; ifort is not supported from oneAPI 2025.
 - Contributor: H. J. Aa. Jensen and J. J. Aucar.
- Made all necessary changes for Nvidia’s NVHPC compilers (previously known as PGI compilers).
 - Contributor: H. J. Aa. Jensen.
- Include support in ExaCorr for NVHPC, and added hints for compilation with OneAPI and NVHPC, and a way to control via environment variable CUDA_SM_ARCH which NVIDIA GPU architecture to compile for. Please note that calculations beyond iterative triples (e.g. CCSDTQ) are not supported with NVHPC.
 - Contributor: A. Gomes.
- Quit if LUCIAREL is requested for sequential dirac.x (LUCIAREL only works if compiled with MPI).
 - Contributor: H. J. Aa. Jensen.

Basis set library:

- Basis set library has been extended with “relativistic prolapse-free Gaussian basis sets” RPF-2Z, RPF-3Z, aug-RPF-2Z, and aug-RPF-3Z.
 - Contributor: H. J. Aa. Jensen.
 - Reference: [J. S. Sousa, E. F. Gusmão, A. K. N. R. Dias, and R. L. A. Haiduke, Relativistic Prolapse-Free Gaussian Basis Sets of Double- and Triple-ζ Quality for s- and p-Block Elements: \(aug-\)RPF-2Z and \(aug-\)RPF-3Z, J. Chem. Theor. Comput. 20, 9991 \(2024\).](#)
- Basis set library has been extended with ECP basis sets cc-pVnZ-PP and cc-pwCVnZ-PP, for n = D,T,Q,5, and the augmented versions aug-cc-pVnZ-PP and aug-cc-pwCVnZ-PP
 - Contributor: Kirk Peterson.
- Transferred fixes of Dalton basis sets from Dalton:
 - Corrected some references in correlation consistent basis sets (as e.g. aug-cc-pVTZ).
 - Fixed Scandium in aug-cc-pVTZ-J (two lines missing, would have given error if used).
 - Contributor: H. J. Aa. Jensen.
- Added special basis sets for spin-spin coupling and hyperfine coupling from Frank Jensen group (Aarhus University):
 - pcJ, ccJ and pcH sets.
 - Copied from Dalton basis set directory.
 - Contributor: H. J. Aa. Jensen.
- Corrections to Dyall basis sets:
 - Fixed incorrect definitions for the number of exponents for Zr/Nb/Ru in dyall.acv4z.
 - Fixed missing exponents:
 - 1x f for Lu in cv2z/ae2z
 - 1x d for K in acv4z
 - 3x f, 2x g, 1x h for Rb/Sr in acv4z

- 1x g for Ra in acv4z/aae4z
- 1x f for Rf–Cn in aae4z
- Fixed incorrect exponent values:
 - 1x s, 1x p, 1x d for Zr in av3z/acv3z/aae3z
 - 2x g, 1x h for Pb–Rn in acv4z/aae4z
 - 1x g for Tl–Rn in ae4z
 - 1x g for Lr in v4z
- Fixed 150+ instances of small rounding errors.
- Contributor: Lukas F. Pasteka.

Bugfixes:

- Fix of symmetry problem for KRMSCF with LUCIAREL and odd number of electrons.
 - Contributor: H. J. Aa. Jensen.
- Changes and corrections in the definition of one-electron operators β^5 and α .
 - One-electron operator type ZiBETAAL was not defined properly in previous code versions. From now on, it is replaced by ZBETAALP and represents the operator $\beta \alpha_z$.
 - In previous code versions, the one-electron operator types XiBETAAL and YiBETAAL represented $\beta \alpha_x$ and $\beta \alpha_y$, respectively. From now on, they are replaced by the operator types XBETAALP and YBETAALP, respectively.
 - New one-electron operator type BETAGAMMA5 available, representing the time-symmetric operator β^5 . Please note that the operator type iBETAGAMMA5 was already implemented in DIRAC and it also points to β^5 , but with the assignments of time-reversal antisymmetry.
 - For more details, please check the manual: one_electron_operators.
 - Contributor: I. A. Aucar.
- Set the correct sign of CC linear and quadratic response functions in the case of more than one time-antisymmetric perturbation operator
 - Contributor: A. Gomes.

New defaults:

- The CODATA2022 set of physical constants is now used as default (taken from <http://physics.nist.gov/constants> on March 6, 2025).
 - Contributor: I. A. Aucar.
- The inverse fine-structure constant is now extracted directly from CODATA, rather than being calculated from other fundamental constants.
 - Contributor: I. A. Aucar.
- Planck's constant is taken from CODATA (the reduced Planck constant is now calculated internally).
 - Contributor: I. A. Aucar.
- Tenpi-generated code is now the default choice for automatically generated T equation codes.
 - Contributor: A. Gomes.
- Default CI gradient convergence threshold for *KRCI is now 3.d-3 for energy calculations and 1.d-4 for property calculations. The 3.d-3 value is sufficient for total energies, accurate to ca. 0.01 mHartree, and saves a lot of unnecessary CPU time, but it will only give 2-3 digits in properties, which have linear accuracy in the CI wavefunction.
 - Contributor: H. J. Aa. Jensen.

24.0

New features:

New functionality with ExaCorr:

- Calculation of linear response properties with CCSD wavefunctions. - Contributors: Xiang Yuan, Loic Halbert, Johann Pototschnig, Anastasios Papadopoulos, Lucas Visscher and Andre Gomes. - Reference: Xiang Yuan, Loic Halbert, Johann Valentin Pototschnig, Anastasios Papadopoulos, Sonia Coriani, Lucas Visscher, and Andre Severo Pereira Gomes, Formulation and Implementation of Frequency-Dependent Linear Response Properties with Relativistic Coupled Cluster Theory for GPU-Accelerated Computer Architectures, J. Chem. Theory Comput. 20, 677 (2024) <https://doi.org/10.1021/acs.jctc.3c00812> - Manual: CCCILR https://diracprogram.org/doc/release-24/manual/wave_function/exacorr.html#cccilr - Tutorial: not yet available; please have a look at examples in the testset. - Calculation of quadratic response properties CCSD wavefunctions. - Contributors: Xiang Yuan, Loic Halbert, Lucas Visscher and Andre Gomes. - Reference: Xiang Yuan, Loic Halbert, Lucas Visscher, and Andre Severo Pereira Gomes, Frequency-Dependent Quadratic Response Properties and Two-Photon Absorption from Relativistic Equation-of-Motion Coupled Cluster Theory, J. Chem. Theory Comput. 19, 9248 (2023) <https://doi.org/10.1021/acs.jctc.3c01011> - Manual: CCCIQR https://diracprogram.org/doc/release-24/manual/wave_function/exacorr.html#ccciqr - Tutorial: not yet available; please have a look at examples in the testset. - Calculation of energies with equation of motion coupled cluster for excitation (EOM-EE-CCSD), ionization (EOM-IP-CCSD), and electron affinity (EOM-EA-CCSD) models. - Contributors: Xiang Yuan, Loic Halbert, Lucas Visscher and Andre Gomes. - Reference: Xiang Yuan, PhD thesis (2024), <https://doi.org/10.5463/thesis.505> - Manual: EXEOM https://diracprogram.org/doc/release-24/manual/wave_function/exacorr.html#exeom see also https://www.diracprogram.org/doc/release-24/manual/wave_function/exacorr.html#id10 - Tutorial: not yet available; please have a look at examples in the testset. - Two-body reduced density matrix for CCSD ground-state wavefunctions. - Contributors: Loic Halbert, Andre Gomes - Manual: GS2RDM https://diracprogram.org/doc/release-24/manual/wave_function/exacorr.html#gs2rdm - Tutorial: not yet available; please have a look at examples in the testset.

Revised features:

- Restart from density matrices of CC for first-order properties.
 - Contributors: Trond Saue, Andre Gomes
 - Manual: RDCCDM <https://diracprogram.org/doc/release-24/manual/properties.html#rdccdm>
- Update of DIRAC schema which defines the information stored on CHECKPOINT.h5.
 - Essential symmetry information added.
 - MO coefficients for C₁ symmetry optionally stored for restarts without use of symmetry.
 - Occupation of spinors in SCF added.
 - Non-default property integrals stored on separate file to reduce the size of the checkpoint file.
 - Original input is preserved in the CHECKPOINT file.
 - Contributors: Lucas Visscher, Trond Saue

Improvements:

- Support for AMD GPUs with ExaCorr (TAL-SH, work in progress to enable distributed calculations with ExaTensor), in addition to the NVIDIA GPU support (TAL-SH and ExaTensor).
 - Contributors: Johann Pototschnig, Jan Brandejs and Andre Gomes.
 - For further information on ExaTensor and TAL-SH as used in DIRAC, see <https://github.com/RelMBdev/ExaTENSOR/tree/dirac>
 - Note : ROCM versions 5.7+ should be used. Earlier ROCM versions may result in code that compiles but may show runtime errors (e.g. HSA_STATUS_ERROR_MEMORY_APERTURE_VIOLATION, or values different than reference ones in the tests). A potential workaround for earlier versions is to reduce hipcc optimization level to -O1.

Bugfixes:

- Fixed orbital analysis with .VECPRI for X2C, issue #79 (H. J. Aa. Jensen)
- Fixed Fock-space coupled cluster calculations with finite fields.

23.0

New features:

- Calculation of parity-violating nuclear spin-rotation tensors. Activated with “PVCSSR”
 - Contributor: I. Agustín Aucar.
 - Reference: [Ignacio Agustín Aucar and Anastasia Borschevsky, Relativistic study of parity-violating nuclear spin-rotation tensors. J. Chem. Phys. 155, 134307 \(2021\)](#)
 - Manual: [PVCSSR](#)
 - Tutorial: <http://www.diracprogram.org/doc/release-23/tutorials/pvcssr/tutorial.html>
- Calculation of oscillator strengths beyond the electric dipole approximation in the generalized velocity representation
 - Contributors: Martin van Horn, Trond Saue, and Nanna Holmgaard List.
 - Reference: [M. van Horn, T. Saue, N. H. List. Probing Chirality across the Electromagnetic Spectrum with the Full Semi-Classical Light-Matter Interaction. J. Chem. Phys. 156, 054113 \(2022\)](#)
 - Manual: [VPOL](#)
 - Tutorial: http://www.diracprogram.org/doc/release-23/tutorials/x_ray/BED_Ba/tutorial.html
- (extended) atomic-mean-field two-electron scalar and spin-orbit picture-change corrections for two-component Hamiltonian calculations
 - Contributors: Stefan Knecht, Michal Repisky, Hans Joergen Aa. Jensen and Trond Saue.
 - Reference: [Stefan Knecht, Michal Repisky, Hans Jørgen Aagaard Jensen, and Trond Saue, Exact two-component Hamiltonians for relativistic quantum chemistry: Two-electron picture-change corrections made simple. J. Chem. Phys. 157, 114106 \(2022\)](#)
 - Manual: [XAMFI](#)
 - Tutorial: http://www.diracprogram.org/doc/release-23/tutorials/two_component_hamiltonians/xamfX2C.html
- ExaCorr module works now also with contracted basis functions.
 - Contributor: Lucas Visscher.
- Basis set library has been extended with basis sets with diffuse functions for s and d blocks, dyall.aXNz basis sets.
 - Contributor: Ken Dyall.
 - Reference: [K. G. Dyall, P. Tecmer, and A. Sunaga, Diffuse basis functions for relativistic s and d block Gaussian basis sets, J. Chem. Theor. Comput. 19, 198 \(2023\)](#).

Infrastructure changes and fixes:

- HDF5 checkpoint file is now default for data handling
 - Contributor: Lucas Visscher
 - User manual: [Data storage in DIRAC](#)
 - Developer manual: [The CHECKPOINT.h5 file](#)

Revised features:

- Added a CI gradient convergence threshold option ‘.THRGCI’ to ‘*KRCICALC’ (could not be changed by user previously).
 - Contributor: H. J. Aa. Jensen.

Performance improvements:

- Improved code in luciarel CI program. Note that KRMC and KRCI by default uses luciarel.
 - Contributor: Andreas Nyvang.
 - numerical stability has been improved in the complex parallel diagonalization routine
 - convergence algorithm has also been changed, with considerable improvement in convergence rate. In particular, the truncation of the reduced space in the Davidson iterations is now done in a much better way, leading to the improvement in convergence rate. Both for real and complex parallel cases.

New defaults:

- Default of MOLTRA changed from scheme 6 to scheme 4 (A. Sunaga).

Bugfix:

- Default for .MVO has been corrected, so it now behaves as expected: modified virtual orbitals based on the Fock potential from the doubly-occupied molecular orbitals after an open-shell SCF calculation (H. J. Aa. Jensen).

22.0 (2022-02-08)

New features:

- New licence: GNU Lesser General Public License v2.1

- Electronic circular dichroism (ECD) beyond lowest-order – full light-matter interaction
 - Contributors: Martin van Horn, Trond Saue, and Nanna Holmgaard List
 - Reference: [M. van Horn, T. Saue, N. H. List, Probing Chirality across the Electromagnetic Spectrum with the Full Semiclassical Light-Matter Interaction. J. Chem. Phys. \(in press\) \(2022\), ChemRxiv](#)
- MP2 frozen virtual natural orbitals via the ExaCorr module
 - Contributors: Xiang Yuan, Lucas Visscher, André Severo Pereira Gomes.
 - Reference: [X. Yuan, L. Visscher, A. S. P. Gomes, Assessing MP2 frozen natural orbitals in relativistic correlated electronic structure calculations.](#)
 - Manual: [MP2NO](#) and [DONATORB](#)
- Polarizable Embedding with Complex Polarization Propagator
 - Contributors: Joel Creutzberg, Erik D. Hedegård
 - Reference: [Polarizable embedding complex polarization propagator in four- and two-component frameworks.](#)

Infrastructure changes and fixes:

- HDF5 checkpoint file and DIRAC data scheme, python utilities to extract data
 - Contributor: Lucas Visscher
 - Manual: [The CHECKPOINT.h5 file](#)

Improvements:

- DIIS replaced by [CROP](#) algorithm in ExaCorr module.
 - Contributors: Chima Chibueze and Lucas Visscher.
 - Reference: [Pototschnig et al., J. Chem. Theory Comput. 17 \(2021\) 5509–5529.](#)
 - Manual: [EXACC](#)

Bugfix:

- There was an error in the construction of the projection operator for secondary orbitals using density matrices for the very special case of 4-component relativistic average-of-configuration HF calculations on systems with inversion symmetry and no occupation in ungerade symmetry.

21.1 (2021-09-16)

Revised features:

- Enhanced pam script: now it also looks in job directory for .diracrc, and it looks there first. pam looks for .diracrc three places: 1) job directory (“local .diracrc”), 2) home directory (“global .diracrc”), 3) pam script directory (“system .diracrc”). The first .diracrc found is used. (H. J. Aa. Jensen).
 - added ENABLE_EXATENSOR to the options you can change with ‘ccmake –’ (H. J. Aa. Jensen).
 - enable talsh only with unsupported mpi libraries (Andre Gomes)
 - improved exatensor MPI identification (Johann Pototschnig)

Error corrections:

- Fixed pam script so it can find dirac.x and basis set directories in an installed version (i.e. after make install or e.g. “ccmake –install build –prefix <installation root dir>”). H.J.Aa.Jensen.
- Fixed memory management in LUCIA (Lucas Visscher)
- atomic supersymmetry now works with X2Cmmf (Ayaki Sunaga, Stefan Knecht, Trond Saue)

21.0 (2021-05-28)

New features:

- New parallel coupled cluster implementation ExaCorr. Contributors: J.V. Pototschnig, A. Papadopoulos, D. I. Lyakh, M. Repisky, L.Halbert, A.S.P. Gomes, H.J.Aa. Jensen, and L. Visscher. To be published.
- Improved start potential with far fewer functions and better accuracy. Contributor: Lucas Visscher. S. Lehtola, L. Visscher and E. Engel. Efficient implementation of the superposition of atomic potentials initial guess for electronic structure calculations in Gaussian basis sets.

10. Chem. Phys. 152, 144105 (2020); <http://dx.doi.org/10.1063/5.0004046> doi: 10.1063/5.0004046
- Calculation of Rotational Molecular g-tensors. Activated with “.ROTG”. Contributor: I. Agustín Aucar. I. A. Aucar, S. S. Gómez, C. G. Giribet and M. C. Ruiz de Azúa. Theoretical study of the relativistic molecular rotational g-tensor.

10. Chem. Phys. 141, 194103 (2014); <http://dx.doi.org/10.1063/1.4901422> doi: 10.1063/1.4901422
- Improved reading of basis sets to be able to read all EMSL basis sets of type DALTON. Contributor: Hans Joergen Aa. Jensen.
- “make latex” for making a latex version of the documentation in “build/latex”, used e.g. pdflatex to make a pdf version of the manual. Contributor: Hans Joergen Aa. Jensen
- Efficient parallel diagonalization of quaternion Fock matrices using openMP. Contributor: Hans Joergen Aa. Jensen.
- Utility (dirac_data.py) to export DIRAC data into a Python dictionary. Contributor: Lucas Visscher.

Revised features:

- Gauge origin (.GAUGEO and .GO ANG) now can only be set under **HAMILTONIAN.
- Phase and dipole origins (.PHASEO and .DIPORG, respectively) now can only be set under **HAMILTONIAN.
- EOM-CC excitation energies collected for all symmetries and summarized at the end.

New defaults:

- One-electron operator ANG MOM's origin was moved from the gauge-origin to the molecular center of mass.
- The CODATA2018 set of physical constants is now used as default (taken from <http://physics.nist.gov/constants>). Before this version, CODATA1998 was default.
- Non-relativistic and relativistic effective core-potential calculations ("NONREL" and "ECP") now have nuclear point nucleus as default. For non-relativistic calculations this makes it easier to compare to other codes, and for RECP this is the intended behavior which by mistake was not activated.

19.0 (2019-12-12)

New features:

- Calculation of Nuclear Spin-Rotation tensors. Activated with ".SPINRO". Contributors: Trond Saue and I. Agustín Aucar. I. A. Aucar, S. S. Gómez, M. C. Ruiz de Azúa, and C. G. Giribet. Theoretical study of the nuclear spin-molecular rotation coupling for relativistic electrons and non-relativistic nuclei. 10. Chem. Phys. 136, 204119 (2012); <http://dx.doi.org/10.1063/1.4721627> doi: 10.1063/1.4721627
- Interface with the Quantum Computing package OpenFermion. <https://github.com/bsenjan/Openfermion-Dirac>. Contributors: Bruno Senjean and Luuk Visscher. T. E. O'Brien, B. Senjean, R. Sagastizabal, X. Bonet-Monroig, A. Dutkiewicz, F. Buda, L. DiCarlo and L. Visscher Calculating energy derivatives for quantum chemistry on a quantum computer. Accepted to npj: Quantum Information. <https://arxiv.org/abs/1905.03742>
- Expectation value of Nuclear Magnetic-Quadrupole-Moment interaction constant using ".OPERATOR" and ".NMQM" keyword in KRCI. Contributor: Malaya K. Nayak T. Fleig, M. K. Nayak and M. G. Kozlov. TaN, a molecular system for probing P,T-violating hadron physics. Phys. Rev. A 93, 012505, 2016; <http://dx.doi.org/10.1103/PhysRevA.93.012505> doi: 10.1103/PhysRevA.93.012505

Revised features:

- Maximum number of basis functions increased from 30k to 40k.

Error corrections:

- Fixed use of point charges (do not try to obtain e.g. mass of a point charge).
- Fixed spelling error in keyword .GAUNTSCALE (was .GAUTSCALE).
- Fixed TIEQN1 function call in src/relccsd/ccgrad.F (thanks to Martin Siegert).
- Fixed wrong memory allocation for spin-free or non-relativistic KRMCSF.
- Fixed runtime error for XPROPMAT in src/krmc/krmccan.F property modules.
- Fixed runtime error for calling XPRIND in src/prp/pamprp.F property modules.

Basis set corrections:

- Fixed Be basis set in cc-pV5Z.
- Fixed Al basis set in aug-cc-pV(D+d)Z, aug-cc-pV(T+d)Z and aug-cc-pV(Q+d)Z.
- Fixed Si basis set in aug-cc-pV(D+d)Z and aug-cc-pV(Q+d)Z.

Performance improvements:

- Reduced static memory requirement from common blocks.

Documentation:

- Restore local build of Sphinx documentation.

18.0 (2018-12-13)

New features:

- The visualization module has been extended to generate various radial distributions using the keyword .RADIAL. Contributor: Trond Saue.

Error corrections:

- Fix f correlating exponents for 3d in dyall.cv2z and dyall.ae2z basis sets. Contributor: Ken G. Dyall.
- Fix f polarizing exponents for 3d in dyall.cv3z and dyall.ae3z basis sets. Contributor: Ken G. Dyall.
- Bugfix in KRCI module (subroutine STRTYP_GAS_REL), reported by Shigeyoshi Yamamoto.
- Bugfix in polarizable embedding (the printed electronic part of the energy was not corrected with PE contributions).
- Fixed the formatting of the carbon potential in the file basis_ecp/ECPCE2SO.

Performance improvements:

- New much more efficient quaternion diagonalization routine using openMP for performance. Contributor: Hans Joergen Aa. Jensen.
- New much more efficient complex diagonalization routine using openMP for performance. Contributor: Hans Joergen Aa. Jensen.
- Use MPI_REDUCE for adding Fock matrix contributions from MPI worker nodes on MPI master node. Contributor: Hans Joergen Aa. Jensen.
- Save memory in direct Fock matrix constructions, now >13000 basis functions can be run in 32GB RAM. Contributor: Hans Joergen Aa. Jensen.

- (With these performance improvements, a 4c HF test calculation with 13000 basis functions, 3300 positive energy orbitals (6600 in total) has been run on Titan at ORNL with 800 nodes, each with 8 openMP threads, at 1h59m-2h06m per HF iteration.)

Framework changes:

- Allow to compile with OpenMP support.
- Update in CMake framework: pass compilers as cmake variables, not as environment variables.

17.0 (2017-12-12)

New features:

- Kramers restricted Polarization Propagator in the ADC framework for electronic excitations, activated with “POLPRP”.

13. Pernpointner. The relativistic polarization propagator for the calculation of electronic excitations in heavy systems.

14. Chem. Phys. 140, 084108 (2014); <http://dx.doi.org/10.1063/1.4865964>

M. Pernpointner, L. Visscher and A. B. Trofimov. Four-component Polarization Propagator Calculations of Electron Excitations: Spectroscopic Implications of Spin-Orbit Coupling Effects. J. Chem. Theory Comput. (2017) submitted.

- Polarizable embedding using PELib (<https://gitlab.com/pe-software/pelib-public>). Activated with “PEQM”, additional options under *PEQM.

Reference: E. D. Hedegård, R. Bast, J. Kongsted, J. M. H. Olsen, and H. J. Aa. Jensen: Relativistic Polarizable Embedding., J. Chem. Theory Comput. 13, 2870-2880 (2017); <http://doi.org/10.1021/acs.jctc.7b00162>

- New and numerically stable procedure for elimination/freezing of orbitals at SCF level. Author: T. Saue.
- New “MVOFAC” option in *KRM input section. Author: H. J. Aa. Jensen.
- New easier options for point charges in the .mol file: “LARGE POINTCHARGE” or “LARGE NOBASIS” (the two choices are equivalent).
- Added the RPF-4Z and aug-RPF-4Z basis sets for f-elements to the already existing files with sets for s, p and d elements. Deleted the aug-RPF-3Z set as that was not an official set.
- Write out effective Hamiltonian in Fock space coupled cluster to a file for post processing. Can be used with external code of Andrei Zaitsevskii (St. Petersburg).
- Provided memory counter for RelCC calculations, what is suitable for memory consuming large scale Coupled Cluster calculations. See the web-documentation.
- Updated basis_dalton/ with basis set updates in the Dalton distribution:
- fix of errors in Ahlrichs-pVDZ (several diffuse exponents were a factor 10 too big)
- fix of errors for 2. row atoms in aug-cc-pCV5Z
- added many atoms to aug-cc-pVTZ_J
- added many Frank Jensen “pc” type basis sets
- added Turbomole “def2” type basis sets

Error corrections:

- Fixed errors for quaternion symmetries in 2-electron MO integrals used in CI calculations with GASCIP. It is now possible to do CI calculations with GASCIP for CI symmetry (i.e. no symmetry).
- Fixed the p exponents for Na in the dyall 4z basis sets to match the archive. The changes are small so should not significantly affect results.
- Fixed compilation of XCFun on Mac OS X High Sierra.
- Fixed error for parallel complex CI or MCSCF with GASCIP

New defaults:

- .SKIPPEP is now default for KR-MCSCF, new keyword .WITHEP to include e-p rotations

Performance improvements:

- restored integral screening behavior from DIRAC15

16.0 (2016-12-12)

New features:

- RELCCSD expectation values aka first-order unrelaxed analytic energy derivative. For more information, see J. Chem. Phys. 145 (2016) 184107 as well as test/cc_gradient for an example.
- Calculation of approximate dipole intensities together with the excitation energies in Fock space CC. See test/fscf_intensities for an example.
- Improved start potential for SCF: sum of atomic LDA potentials, generated by GRASP.

New defaults:

- Negative denominators (e.g. appearing in core ionized systems) accepted in RELCCSD
- AOFOCK is now default if at least 25 MPI nodes (parallelizes better than SOFOCK). .AOFOCK documented.

Error corrections and updates in isotope properties for the following atoms:

- Br isotope 2: quadrupole moment .2620 -> .2615 - Ag isotope 2: magnetic moment .130563 -> -.130691 (note sign change) - In isotope 2: quadrupole moment .790 -> .799 - Nd magnetic moments of isotopes 4 and 5 were interchanged: -0.065 -> -1.065 and -1.065 -> -0.065 - Gd: quadrupole moments of isotopes 4 and 5 updated: 1.36 -> 1.35 and 1.30 -> 1.27 - Ho isotope 1: quadrupole moment updated 3.49 -> 3.58 - Lu isotope 2: quadrupole moment updated 4.92 -> 4.97 - Hf isotope 1: mass was real*4, not real*8, thus 7 digits instead of 179.9465457D0 (i.e. approx 179.9465) - Ta isotope 1: quadrupole moment added 0.00 -> 3.17 - Tl isotope 1: nuclear moment 1.63831461D0 -> 1.63821461D0 (typo, error 1.d-4) - Pb isotope 3: nuclear moment 0.582583D0 -> 0.592583D0 (typo, error 1.d-2) - Po isotope 1: nuclear moment added: 0.000 -> 0.688

Framework improvements:

- Faster Dirac compilation due to implemented OBJECT libraries (requires CMake of version at least 2.8.10).
- Run benchmark tests and only benchmark tests with “ctest -C benchmarks -L benchmark”.
- Change any tab character in coordinate specifications in .mol file to a blank. (tab characters have been reported to cause errors in reading coordinates).
- Added OPENBLAS in search list for blas and lapack libraries.
- Made keyword .X2Cmmf case insensitive.
- Cache compiler flags.

Bug fixes:

- Bugfix: MCSCF and GASCIP was not correct for odd number of electrons.
- Bugfix: now .CASSCF works again for KRMSCF input.
- Bugfix: KRMSCF was not converging in some cases (problem reported by Miro Ilias for Os atom).
- Bugfix: “all” keyword for “.GAS SHELLS” was not recognized (problem reported by Miro Ilias).
- Bugfix for numerical gradient when automatic symmetry detection: do no move or rotate molecule for finding highest possible symmetry because then the numerical gradient will not be correct (problem reported by Miro Ilias).
- Bugfix for spinfree X2Cmmf and linear symmetry.
- Bugfix: restored “make install” target.
- Bugfix: Test reladc_fano does not report failure if Stieltjes is deactivated.

15.0 (2015-12-16)

- FanoADC-Stieltjes: Calculation of decay widths of electronic decay processes.
- Better convergence for open-shell calculations, using .OPENFAC 0.5 as default. Final orbital energies are recalculated with .OPENFAC 1.0, for IP interpretation.
- Performance improvement for MP2 NO: only transform (G O|G O) integrals instead of (G G| integrals instead of (G G|)G G).
- Geometry optimization with xyz input.
- Performance improvements for determinant generation in GASCIP.
- Use Kramers conjugation on CI vectors (cuts time for CI in half for ESR doublets).
- ANO-RCC basis: Fixed Carbon basis set (wrong contraction coefficients, see [MOLCAS ANO-RCC](<http://www.molcas.org/ANO/>)).
- ANO-RCC basis: Modified the 3 Th h-functions by replacing them with the 3 Ac h-functions to Th. (A mistake was made in the original work when the 3 Th h-functions were made, and this has never been redone. They are too diffuse, exponents (0.3140887600, 0.1256355100, 0.0502542000) for Th, Z=90, compared to (0.7947153600, 0.3149038200, 0.1259615200) for Ac, Z=89, and (0.8411791300, 0.3310795400, 0.1324318200) for Pa, Z=91. We have selected to just replace the 3 Th h-functions with those from the Ac basis set, because the Ac g-functions are quite close to the Th g-functions, closer than Ac g-functions, and therefore differences in results compared to optimized Th h-functions should be minimal.) Thanks to Kirk Peterson for pointing out the Th problem on <http://daltonforum.org>.
- Fixed reading of ANO-RCC and ANO-DK3 basis sets from the included basis set library.
- Update PCMSolver module to its [1.0.4 version](<https://github.com/PCMSolver/pcmsolver/releases>).
- Do not write Hartree-Fock in output if it is a DFT calculation, write DFT.
- Fix a bunch of problems where the number of arguments of the call does not match the subroutine definition (submitted by Martin Siegert).
- Fixed plotting of the electrostatic potential (electronic part was factor 2 too large).
- Configuration framework uses [Autocmake](<http://autocmake.org>).

14.1 (2015-07-20)

- Added OpenBLAS to default blas search list.
- Fixed misleading error message for .OPEN SHELL input.
- Workaround for CMake warning about nonexistent run_pamadm target.
- Intel compilers xHost flag automatic detection only upon request (-D ENABLE_XHOST_FLAG_DETECTION=ON).
- Fixed minor issues to allow compilation using IBM XL Fortran compiler (thanks to Bob Walkup, IBM).
- Update PCMSolver module to its [1.0.3 version](<https://github.com/PCMSolver/pcmsolver/releases>).

14.0 (2014-12-12)

- DIRAC14 release, see doc/release/release-statement.txt

FILE: www.diracprogram.org/doc/release-26/_sources/patches/KNOWN-PROBLEMS.md

orphan

Known problems

14.0

- LAO shieldings are wrong in combination with the .SELECT keyword.
- Analytic molecular gradient is wrong for an open-shell system.
- LAO shieldings are wrong in combination with nonzero .HFXMU (e.g. LCPBE0).
- CAMB3LYP analytic molecular gradient is wrong.
- Several tests show failures on Intel 15 with -int64.
- XC grid generation does not correctly handle ghost atoms.
- .CNVINT keyword is ignored.
- .PROPERTIES can reset OPTIMIZE parameters (e.g. cancel .NUMGRA).

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/analysis/projection_analysis.md

orphan

Projection analysis

Theory

A serious flaw of Mulliken population analysis is its basis set sensitivity. This is well known from the non-relativistic domain. In the relativistic domain a further disadvantage is that the expansion of large and small components in separate basis sets means large and small component populations come separately and it is not easy to see how they are connected. Furthermore, if the large and small component are expanded in scalar basis functions, as is the case of the DIRAC code, then the population analysis can not distinguish distributions from different spin-orbit components of atomic orbitals, for instance, $p_{1/2}$ and $p_{3/2}$.

Projection analysis Dubillard2006 eliminates all these inconveniences. It is based on the simple concept of molecular orbitals as linear combination of atomic orbitals (LCAO) and allows analysis of electronic structure in terms of well-defined atomic (or fragment) orbitals.

Projection analysis requires as input a set of pre-calculated orbitals $\{\psi^A_p\}$ for the constituent atoms (or fragment) of the molecule. Here A refers to the atomic center (or fragment) and p is on orbital index for that center. The molecular orbitals are then expanded as

$$\left| \psi^M_i \right\rangle = \sum_A \sum_{p \in A} \psi^A_p c^A_{pi} + \left| \psi^{\text{pol}}_i \right\rangle$$

Typically one will select the occupied orbitals of the constituent atoms. They are not guaranteed to fully span the molecular orbitals and therefore the expansion includes their orthogonal complement $\left| \psi^{\text{pol}}_i \right\rangle$, denoted the *polarization contribution*. The expansion coefficients c^A_{pi} are found by solving the equation

$$\langle \psi^A_p | \psi^M_i \rangle = \sum_B \sum_{q \in B} \langle \psi^A_p | \psi^B_q \rangle c^B_{qi}$$

obtained by projection.

For projection analysis, arbitrary fragments can in principle be used. There is, however, an important restriction in that DIRAC can only do analysis in terms of fragments complying to the molecular point group. For instance, the two hydrogens in water are related by symmetry in the full C_{2v} molecular point group. They then have to be treated as a single fragment, or they can be separated by lowering the symmetry. Starting from DIRAC14 projection analysis can be carried out in terms of individual atoms using the PROJECTION.ATOMS keyword, as shown in the example below.

Example 1: Methane

Introduction

As a first simple example we shall consider the methane molecule. The molecular input file CH4.xyz (download <../../../../test/tutorial_projection_analysis/CH4.xyz>) is

```
../../../../test/tutorial_projection_analysis/CH4.xyz
```

Suppose that we have run a DFT(PBE) calculation using:

```
pam --inp=PBE --mol=CH4.xyz
```

and the menu file PBE.inp (download <../../../../test/tutorial_projection_analysis/PBE.inp>) is

```
../../../../test/tutorial_projection_analysis/PBE.inp
```

Preparing for analysis

We would now like to analyze the electronic structure of the methane molecule in terms of the orbitals of the constituent atoms. We first generate the atomic fragments. We recommend calculating the atoms in the highest possible symmetry and the converting the coefficients to no (C_1) symmetry. This allows a very precise identification of the individual atomic orbitals (including m_j quantum number). We accordingly run calculations where we save MO files, each as the unique file:

```
pam --inp=H --mol=H
pam --inp=C --mol=C
```

with corresponding input files H.inp (download <../../../../test/tutorial_projection_analysis/H.inp>), H.mol (download <../../../../test/tutorial_projection_analysis/H.mol>), C.inp (download <../../../../test/tutorial_projection_analysis/C.inp>), and C.mol (download <../../../../test/tutorial_projection_analysis/C.mol>).

Running the analysis

We are now ready for the projection analysis itself. We prepare the input file prj.inp (download <../../../../test/tutorial_projection_analysis/prj.inp>)

```
../../../../test/tutorial_projection_analysis/prj.inp
```

In the *PROJECTION section we specify the two atomic types identified by the name of their coefficient files as well as an orbital string (see orbital_strings) listing what orbitals to include for each atom. DIRAC will assume that each atom has been calculated in its proper basis.

We now run the projection analysis using:

```
pam --inp=prj --mol=CH4.xyz --put "C.h5 H.h5 PBE_CH4.h5=CHECKPOINT.h5" --noh5
```


Note that we set the pam flag `--noh5` to avoid generating a new HDF5 checkpoint file, since nothing new is added to the existing one.

Interpreting the output

The first thing to look for in the output (download `<../../../../../test/tutorial_projection_analysis/result/prj_CH4.out>`) is the section:

```
* Total gross contributions:
AFCX 1      6.4301 E -      6.4301 P -      0.0000
AFHX 1      0.8716 E -      0.8716 P -      0.0000
AFHX 2      0.8716 E -      0.8716 P -      0.0000
AFHX 3      0.8716 E -      0.8716 P -      0.0000
AFHX 4      0.8716 E -      0.8716 P -      0.0000
Polarization: 0.0834
```

It shows the gross population (in Mulliken sense, but in terms of the fragment orbitals) of each atom and then, most importantly, the gross population of the polarization contribution. In this example it is small and perfectly acceptable. If it becomes too large the analysis loses meaning because it implies that significant electron density has not been assigned to any fragment. In the case of significant polarization contribution one can either increase the number of atomic orbitals used in the analysis or turn on *repolarization*. For the latter it suffices to add the `PROJECTION_POLREF` keyword (see the corresponding input file `<../../../../../test/tutorial_projection_analysis/prj_polref.inp>`) which activates *Intrinsic Atomic Orbitals (IAOs)*, Knizia2013, thus eliminating completely the polarization contribution. We then get (see the corresponding output `<../../../../../test/tutorial_projection_analysis/result/prj_polref_CH4.out>` file):

```
* Total gross contributions:
AFCX 1      6.4782 E -      6.4782 P -      0.0000
AFHX 1      0.8804 E -      0.8804 P -      0.0000
AFHX 2      0.8804 E -      0.8804 P -      0.0000
AFHX 3      0.8804 E -      0.8804 P -      0.0000
AFHX 4      0.8804 E -      0.8804 P -      0.0000
Polarization: 0.0000
```

from which we can conclude that at this level of theory the carbon and hydrogen charges in methane are $-0.48e$ and $+0.12e$, respectively.

Having concluded that the polarization contribution is acceptable, we can proceed with the analysis. We turn to this section:

```
* Total reference orbital contributions:
AFCX 1 E 1 1.99999 -0.10660951E+02
AFCX 1 E 2 1.27874 -0.97828469E+00
AFCX 1 E 3 1.05077 -0.65580701E+00
AFCX 1 E 4 1.05030 -0.65541446E+00
AFCX 1 E 5 1.05030 -0.65541446E+00
AFHX 1 E 1 0.87162 -0.23118508E+00
AFHX 2 E 1 0.87162 -0.23118508E+00
AFHX 3 E 1 0.87162 -0.23118508E+00
AFHX 4 E 1 0.87162 -0.23118508E+00
```

which gives the accumulated gross population of each fragment orbital. We can identify the fragments from the name of the coefficient file and its orbital energy. For instance the first orbital on the list is clearly carbon $1s$ followed by carbon $2s$. Next comes three orbitals that are almost degenerate. These are clearly the carbon $2p$ orbitals, the first one being $2p_{1/2}$ followed by the two $2p_{3/2}$. By furthermore looking into the output of the carbon atom calculation (note that we asked for Mulliken population analysis in the input) we find that the first $2p_{3/2}$ orbital has $m_j=1/2$ and the second $m_j=3/2$. From these gross population we can deduce the atomic configurations of the atoms in the molecule. For carbon we find $[\text{He}]2s^{\wedge}1.3 2p^{\wedge}3.2$ and for hydrogen $1s^{\wedge}0.9$.

There is also detailed output for each molecular orbital in the output. For the third molecular orbital we find for instance:

```
* Electronic eigenvalue nr. 3: -0.3411330794649 (Occupation : f = 1.0000)
=====
Orbital      Total      Eigenvalue      Kramers partner 1      Kramers partner 2
AFCX 1 E 3 0.57945 -0.65580701E+00 (-1.0000,-0.0000) ( 0.0000, 0.0000)
AFHX 1 E 1 0.31559 -0.23118508E+00 ( 0.0000,-0.5774) (-0.5774,-0.5774)
AFHX 2 E 1 0.31559 -0.23118508E+00 ( 0.0000, 0.5774) ( 0.5774,-0.5774)
AFHX 3 E 1 0.31559 -0.23118508E+00 ( 0.0000, 0.5774) (-0.5774, 0.5774)
AFHX 4 E 1 0.31559 -0.23118508E+00 ( 0.0000,-0.5774) ( 0.5774, 0.5774)
* Gross contributions:
AFCX 1      0.5254 E -      0.5254 P -      0.0000
AFHX 1      0.1165 E -      0.1165 P -      0.0000
AFHX 2      0.1165 E -      0.1165 P -      0.0000
AFHX 3      0.1165 E -      0.1165 P -      0.0000
AFHX 4      0.1165 E -      0.1165 P -      0.0000
Polarization: 0.0085
```

To fully understand the output we have to keep in mind that all calculations are Kramers-restricted such that all orbitals come in pairs. The expansion of the molecular orbitals should therefore rather be written as

$$\left| \psi^{\text{MO}}_i \right\rangle = \sum_A \sum_{p \in A} \left| \psi^A_p c^A_{\pi} + \psi^A_{\overline{p}} c^A_{\overline{\pi}} \right\rangle + \left| \psi^{\text{pol}}_i \right\rangle$$

where c^A_{π} and $c^A_{\overline{\pi}}$ is the expansion coefficient of Kramers partner 1 and 2, respectively. Let us now define the absolute value

$$\left| c^A_{\pi} \right| = \sqrt{\left| c^A_{\pi} \right|^2 + \left| c^A_{\overline{\pi}} \right|^2}$$

This is the real number reported under the heading `Total`. Under the heading `Kramers partner 1` is reported the complex number

$$z_1 = \left(\text{Re}(z_1), \text{Im}(z_1) \right) = \frac{c^A_{\pi}}{\left| c^A_{\pi} \right|},$$

and analogously for Kramers partner 2, from which we get the detailed decomposition of the molecular orbital. We find that this molecular orbital is composed of the carbon $2p_{1/2}$ orbital and the hydrogen $1s$ orbitals.

Exercises

1. What is the effect of increasing the C-H bond distance on the atomic configurations of the C,H atoms in the methane molecule and on the atomic charges of the constituent atoms? Collect your calculated data in a table.
2. How different basis sets affect the atomic configurations of the C,H atoms in the methane molecule and the atomic charges of the same constituent atoms?

- Investigate the effect of various DFT functionals (see *DFT) on the atomic configurations of the C and H atoms in the methane molecule and on atomic charges of these constituent atoms.

Example 2: Uranyl

Introduction

We next turn to a molecule with a heavy atom and a more complicated electronic structure, namely uranyl UO_2^{2+} . Here the two oxygens are related by symmetry, and so one would at first consider running the projection analysis with lower (or no) symmetry or using the `PROJECTION_.ATOMS` keyword, as in the methane example above. However, we can reduce the symmetry from $D_{\infty h}$ to $C_{\infty v}$ by introducing a ghost center. Our molecular input file then looks like

UO2.mol

Now the oxygen atoms are no longer connected by symmetry and we can carry out the projection analysis in $C_{\infty v}$ symmetry.

Preparing the analysis

Using the menu file `HF.inp`

UO2_HF.inp

we run 4-component Hartree-Fock:

```
pam --mw=150 --inp=HF --mol=UO2
```

Coefficients are saved on the h5 checkpoint file. Next we generate atomic reference orbitals. For oxygen this is quite straightforward:

```
pam --inp=0 --mol=0
```

using molecule input `0.mol`

`O.mol`

and the menu file `0.inp`

`O.inp`

For the uranium we must be a bit more careful since the ground state configuration is an open-shell one: $[\text{Rn}]5f^3 6d^1 7s^2$. Using the molecular input `U.mol`

`U.mol`

and the inp file `U.inp`

`U_KPSELE.inp`

Note that `SCF_.KPSELE` is needed for getting the convergence in this case. Convergence is now straightforward :

```
pam --inp=U --mol=U
```

and the correct configuration is confirmed by Mulliken population analysis.

Running the analysis

We are now ready to do the projection analysis. We prepare the input file `prj.inp`

`prj_UO2.inp`

and prepare the coefficient files:

```
$ ln -s 0.h5 01.h5
$ ln -s 0.h5 02.h5
```

We run the projection analysis:

```
pam --inp=prj --mol=UO2 --put="U.h5 01.h5 02.h5 HF_UO2.h5=CHECKPOINT.h5"
```

Interpreting the output

As in the methane case above we first look at the polarization contribution:

```
* Total gross contributions:
AFUXXX      89.0635  E -    89.0635  P -    0.0000
AF01XX       8.3098  E -    8.3098  P -    0.0000
AF02XX       8.3098  E -    8.3098  P -    0.0000
Polarization:    0.3169
```

We see that it is more sizable than before. If we eliminate the polarization contribution using the `PROJECTION_.POLREF` keyword, we obtain:

```
* Total gross contributions:
AFUXXX      89.1586  E -    89.1586  P -    0.0000
AF01XX       8.4207  E -    8.4207  P -    0.0000
AF02XX       8.4207  E -    8.4207  P -    0.0000
Polarization:    0.0000
```

The atomic gross populations tell us that the oxygen atoms has a charge -0.42, whereas the uranium atom has charge +2.84, considerably far from the formal oxidation state +VI.

We next turn to the electron configuration of the atoms in the molecule by looking at the section:

```
* Total reference orbital contributions:
AFUXXX E 1 2.00000 -0.42818128E+04 1s
AFUXXX E 2 2.00000 -0.80663682E+03 2s
AFUXXX E 3 2.00000 -0.77703498E+03 2p-
AFUXXX E 4 2.00000 -0.63578312E+03 2p+
AFUXXX E 5 2.00000 -0.63578312E+03 2p+
AFUXXX E 6 2.00000 -0.20673001E+03 3s
AFUXXX E 7 2.00000 -0.19325143E+03 3p-
AFUXXX E 8 2.00000 -0.16037784E+03 3p+
AFUXXX E 9 2.00000 -0.16037784E+03 3p+
AFUXXX E 10 2.00000 -0.13906994E+03 3d-
AFUXXX E 11 2.00000 -0.13906994E+03 3d-
AFUXXX E 12 2.00000 -0.13242590E+03 3d+
AFUXXX E 13 2.00000 -0.13242590E+03 3d+
AFUXXX E 14 2.00000 -0.13242590E+03 3d+
AFUXXX E 15 2.00000 -0.54355461E+02 4s
AFUXXX E 16 2.00000 -0.48231971E+02 4p-
AFUXXX E 17 2.00000 -0.39554187E+02 4p+
AFUXXX E 18 2.00000 -0.39554187E+02 4p+
AFUXXX E 19 2.00000 -0.29743891E+02 4d-
AFUXXX E 20 2.00000 -0.29743891E+02 4d-
AFUXXX E 21 2.00000 -0.28130201E+02 4d+
AFUXXX E 22 2.00000 -0.28130201E+02 4d+
AFUXXX E 23 2.00000 -0.28130201E+02 4d+
AFUXXX E 24 2.00000 -0.15202067E+02 4f-
AFUXXX E 25 2.00000 -0.15202067E+02 4f-
AFUXXX E 26 2.00000 -0.15202067E+02 4f-
AFUXXX E 27 2.00000 -0.14785575E+02 4f+
AFUXXX E 28 2.00000 -0.14785575E+02 4f+
AFUXXX E 29 2.00000 -0.14785575E+02 4f+
AFUXXX E 30 2.00000 -0.14785575E+02 4f+
AFUXXX E 31 2.00001 -0.12603340E+02 5s
AFUXXX E 32 1.99989 -0.10135890E+02 5p-
AFUXXX E 33 1.99952 -0.80948982E+01 5p+
AFUXXX E 34 1.99993 -0.80948982E+01 5p+
AFUXXX E 35 1.99983 -0.43524477E+01 5d-
AFUXXX E 36 1.99914 -0.43524477E+01 5d-
AFUXXX E 37 1.99872 -0.40407150E+01 5d+
AFUXXX E 38 1.99956 -0.40407150E+01 5d+
AFUXXX E 39 1.99990 -0.40407150E+01 5d+
AFUXXX E 40 1.98417 -0.21392050E+01 6s
AFUXXX E 41 1.93819 -0.13442976E+01 6p-
AFUXXX E 42 1.75124 -0.98481977E+00 6p+ (1/2)
AFUXXX E 43 1.98199 -0.98481977E+00 6p+ (3/2)
AFUXXX E 44 0.03906 -0.20241412E+00 7s
AFUXXX E 45 0.82745 -0.34596371E+00 5f- (1/2)
AFUXXX E 46 0.15661 -0.34596371E+00 5f- (3/2)
AFUXXX E 47 0.00000 -0.34596371E+00 5f- (5/2)
AFUXXX E 48 0.00000 -0.31822326E+00 5f+ (7/2)
AFUXXX E 49 0.00000 -0.31822326E+00 5f+ (5/2)
AFUXXX E 50 0.33927 -0.31822326E+00 5f+ (3/2)
AFUXXX E 51 0.94161 -0.31822326E+00 5f+ (1/2)
AFUXXX E 52 0.42621 -0.19266767E+00 6d- (1/2)
AFUXXX E 53 0.09667 -0.19266767E+00 6d- (3/2)
AFUXXX E 54 0.34518 -0.18309790E+00 6d+ (1/2)
AFUXXX E 55 0.33432 -0.18309790E+00 6d+ (3/2)
AFUXXX E 56 0.00010 -0.18309790E+00 6d+ (5/2)
AF01XX E 1 2.00005 -0.20696087E+02 1s
AF01XX E 2 1.89709 -0.12503665E+01 2s
AF01XX E 3 1.51690 -0.61398417E+00 2p-
AF01XX E 4 1.46077 -0.61259798E+00 2p+
AF01XX E 5 1.54590 -0.61259798E+00 2p+
AF02XX E 1 2.00005 -0.20696087E+02 1s
AF02XX E 2 1.89709 -0.12503665E+01 2s
AF02XX E 3 1.51690 -0.61398417E+00 2p-
AF02XX E 4 1.46077 -0.61259798E+00 2p+
AF02XX E 5 1.54590 -0.61259798E+00 2p+
```

We can to a large extent deduce the identity of the various atomic orbitals by just looking at orbital energies and I have added it by hand to the above output in a hopefully obvious notation. Even more precise information, such as m_j value (indicated in parenthesis) can be obtained from looking at the Mulliken population analysis of each atom. From this list of gross population we extract the atomic valence configuration $5f^{2.25}6d^{1.16}7s^{0.04}$ for the uranium atom and $2s^{1.90}2p^{4.41}$ for oxygen. An interesting feature is that the uranium $6p$ population adds up to 5.64 and not six electrons. This is a manifestation of the so-called *6p-hole*, due to overlap with the oxygen ligands, and mostly located to the $6p_{3/2,1/2}$ orbital.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/ase/ase.md

orphan

ASE-DIRAC

DIRAC has an interface to the ASE (Atomistic Simulation Environment) workflow software that can be installed following the instructions in the ase_dirac folder. After installation you can use this to build workflows in which DIRAC is run with automatic generation of the required input. A jupyter-notebook tutorial to demonstrate a simple workflow can be found in the folder demo_notebooks.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/atomic_shift/tutorial.md

orphan

Atomic shift operator

DIRAC allows the definition of an atomic shift operator `HAMILTONIAN_.ASHIFT` to be added to the Hamiltonian

$$V = \sum_A \sum_{I \in A} \sum_{i \in I} |\varphi^A_i\rangle \langle \varphi^A_I| \epsilon^A_I \langle \varphi^A_i|$$

where $\sum_A$ is a sum over atomic types. For each atomic types a group Γ of orbitals may be specified with an associated shift parameter ϵ^A_Γ . In this manual we shall see an example of its use.

Degeneracy-driven covalency: the case of CO

We consider carbon monoxide. We run a straightforward HF calculation using the input files `CO.xyz`

`CO.xyz`

and `CO.inp`

`CO.inp`

using the command:

```
pam --inp=CO --mol=CO.xyz
```

Looking at the Mulliken population analysis in the output, we find for the lowest two occupied molecular orbitals:

```
* Electronic eigenvalue no.   1: -20.684805978278      (Occupation : f = 1.0000)  m_j=  1/2
=====

* Gross populations greater than 0.01000

Gross      Total  |      L A1 0  s
-----
alpha      0.9995 |      0.9992
beta       0.0005 |      0.0000

* Electronic eigenvalue no.   2: -11.369805624916      (Occupation : f = 1.0000)  m_j=  1/2
=====

* Gross populations greater than 0.01000

Gross      Total  |      L A1 C  s
-----
alpha      0.9997 |      0.9990
beta       0.0003 |      0.0000
```

We see that the first and second lowest occupied MO are $O1s$ and $C1s$, respectively, separated by a sizable energy of $9.315 E_h$.

Let us now see what happens if we bring the two atomic core orbitals into resonance. First we generate the carbon atomic orbitals, using input files `C.inp`

`C.xyz`

and `C.inp`

`C.inp`

and the command:

```
pam --inp=C --mol=C.xyz
```

The checkpoint file `C_C.h5` contains the C_{1s} coefficients:

```
h5dump -n C_C.h5 | grep C1
dataset /result/wavefunctions/scf/mobasis/eigenvalues_C1
dataset /result/wavefunctions/scf/mobasis/orbitals_C1
```

We next prepare the input file `COshift.inp`

`COshift.inp`

which notably defines an atomic shift operator for carbon which builds a projector from the $C1s$ orbital and shifts it down by $9.315 E_h$. We run our calculation using the command:

```
pam --inp=COshift --mol=CO.xyz --put "ac.C=AFCXXX"
```

The Mulliken population analysis of the two lower orbitals now read:

```
* Electronic eigenvalue no.   1: -20.687541416409      (Occupation : f = 1.0000)  m_j=  1/2
=====
```

* Gross populations greater than 0.00010

Gross	Total		L A1 C s	L A1 O s	A1 O _small	B1 O _small	B2 O _small
alpha	0.9996		0.4790	0.5203	0.0001	0.0000	0.0000
beta	0.0004		0.0000	0.0000	0.0000	0.0001	0.0001

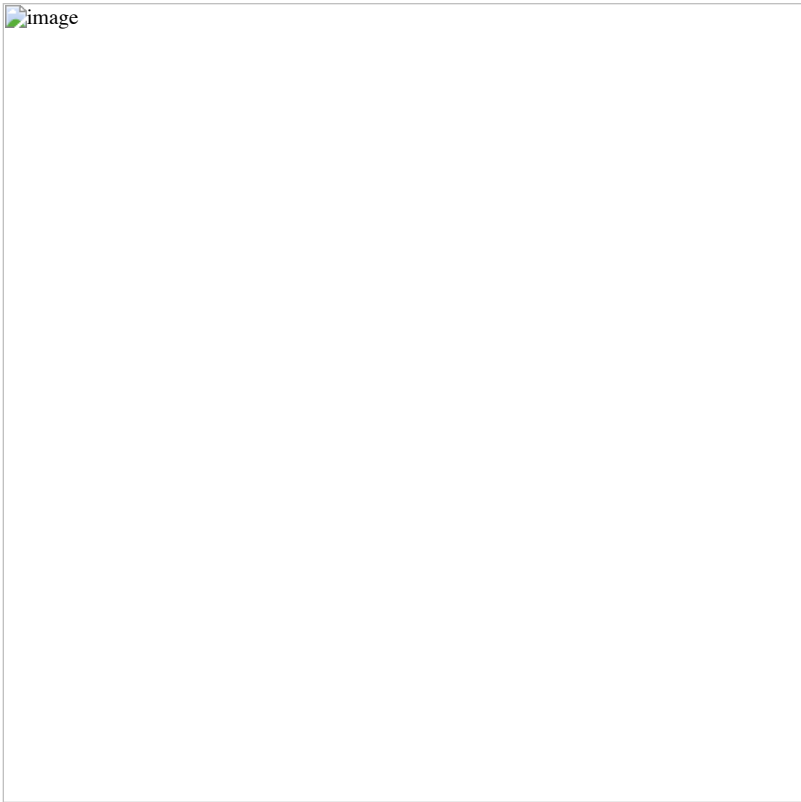
* Electronic eigenvalue no. 2: -20.681935936276 (Occupation : f = 1.0000) m_j= 1/2

* Gross populations greater than 0.00010

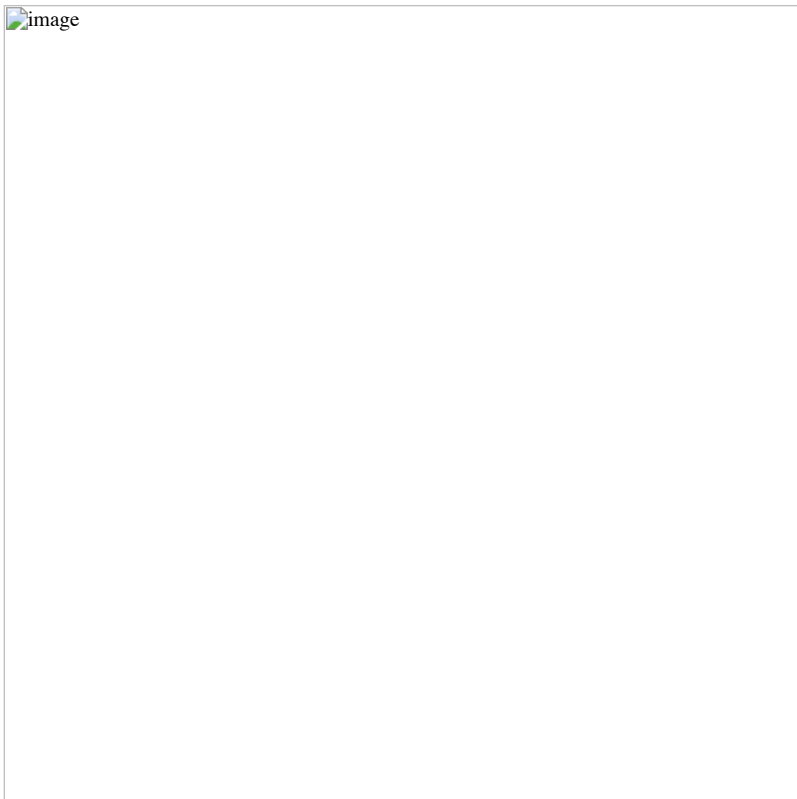
Gross	Total		L A1 C s	L A1 O s	A1 O _small	B1 O _small	B2 O _small
alpha	0.9996		0.5202	0.4790	0.0001	0.0000	0.0000
beta	0.0004		0.0000	0.0000	0.0000	0.0001	0.0001

and we indeed see that the two lowest molecular orbitals now are almost degenerate with concomitant strong mixing of the \$O1s\$ and \$C1s\$ orbitals.

By scanning shift values in the resonance region we obtain the following plot of orbital energies of the two lowest occupied molecular orbitals



as well as the mixing of \$O1s\$ and \$C1s\$ orbitals in the LOMO.



At this point it may be useful to consider the diagonalization of a 2×2 matrix

```
\begin{aligned}
\left[\begin{array}{cc}\varepsilon_1 & \delta \\ \delta & \varepsilon_2\end{array}\right],
\end{aligned}
```

where eigenvalues are given by

$$\lambda_{\pm} = \frac{1}{2} \left[(\varepsilon_1 + \varepsilon_2) \pm \sqrt{(\varepsilon_1 - \varepsilon_2)^2 + 4\delta^2} \right]; \quad \Delta\lambda = \lambda_+ - \lambda_-$$

We notably see that as we bring the two starting energies into resonance, that is $\varepsilon_1 = \varepsilon_2$, the difference $\Delta\lambda$ becomes equal to the twice the interaction 2δ . In this particular case $2\delta = 5.6 \text{ mE}_h$.

What can learn from this ? Neidig, Clark and Martin Neidig_CoorChemRev2013 have introduced the concept of *near-degeneracy driven covalency*, where bonding interaction arises from near-degeneracy, even in the absence of overlap. In the above example we clearly see this mechanism at work: bringing the $\text{O } 1s$ and $\text{C } 1s$ orbitals into resonance causes strong orbital mixing. *However*, it would be absurd to call this bonding. The two atomic core orbitals can be disentangled by localization and we also note that their interaction is extremely weak.

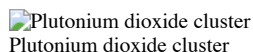
FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/atomic_start2.md

orphan

Atomic start guess

Initial SCF run

We consider the Hartree-Fock calculation of plutonium dioxide cluster



In order to help convergence we will employ an atomic start.

Preparing the atomic start

For each atom we run a SCF calculation in full (linear) symmetry based on the atomic ground state configuration. For open-shell atoms this amounts to an average-of-configuration (AOC) calculation at the HF level, and a fractional occupation calculation at the DFT level.

Hydrogen atom

We run the hydrogen atom as a one-electron system. The input file `H.inp` accordingly reads :

```

**DIRAC
.WAVE FUNCTION
.ANALYZE
**GENERAL
.ACMOUT
**HAMILTONIAN
.ONESYS
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
2 0
**ANALYZE
.MULPOP
*END OF

```

The molecular input H.mol is :

BASIS

```

C 1      1      A
H      1.      1
0.0000000000      0.0000000000      0.0000000000
LARGE BASIS cc-pVDZ
FINISH

```

We run pam :

```

pam --inp=H.inp --mol=H.mol --get=ACMOUT
mv DFACMO ac.H

```

Oxygen atom

The input file 0.inp reads: :

```

**DIRAC
.WAVE FUNCTION
.ANALYZE
**GENERAL
.ACMOUT
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
4 0
.OPEN SHELL
1
4/0,6
**ANALYZE
.MULPOP
*END OF

```

The molecular input 0.mol is :

BASIS

```

C 1      8      1      A
O      0.0000000000      0.0000000000      0.0000000000
LARGE BASIS cc-pVDZ
FINISH

```

We run the calculation :

```

pam --inp=0.inp --mol=0.mol --get=DFACMO
mv DFACMO ac.O

```

Plutonium atom

For plutonium we employ the $[Rn]5f^4$ configuration of the Pu^{4+} cation. The input file Pu.inp accordingly reads: :

```

**DIRAC
.WAVE FUNCTION
.ANALYZE
**GENERAL
.ACMOUT
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
42 44
.OPEN SHELL
1
4/0,14
**INTEGRALS
*READIN
.UNCONTRACT
**ANALYZE
.MULPOP
*END OF

```

The molecular file 'Pu.mol' reads :

BASIS

```

C      1      A
      94.      1
Pu      0.0000000000      0.0000000000      0.0000000000
LARGE BASIS dyall.v2z
FINISH

```

We run the calculation :

```

pam --inp=Pu.inp --mol=Pu.mol --get=DFACMO
mv DFACMO ac.Pu

```

Running the atomic start

We are now ready to run the atomic start. The input file 'PuO2.inp' reads:

```

**DIRAC
.WAVE FUNC
.ANALYZE
#. INPTES
**WAVE FUN
.SCF
*SCF
.ATOMST
AFPUXX 2
1..43
1.00
44..50
0.286
AF0XXX 2
1,2
1.00
3..5
0.667
AFHXXX 1
1
0.50
.CLOSED SHELL
84 86
**ANALYZE
.MULPOP
**END OF

```

The keyword ATOMST is followed by input for each atomic type. The occupations chosen corresponds to those of the atomic runs, but the user may modify this at will. Please note that the order of atoms corresponds to the order they appear in the molecule file. The molecular input file 'PuO2.mol' reads:

DIRAC

```

C      3      0      A
      94.      1
Pu      0.0000000000      0.0000000000      0.0000000000
LARGE BASIS dyall.v2z
8.      8
O      1.3490000000      1.3490000000      1.3490000000
O      1.3490000000      1.3490000000      -1.3490000000
O      -1.3490000000      1.3490000000      1.3490000000
O      -1.3490000000      1.3490000000      -1.3490000000
O      1.3490000000      -1.3490000000      1.3490000000
O      1.3490000000      -1.3490000000      -1.3490000000
O      -1.3490000000      -1.3490000000      1.3490000000
O      -1.3490000000      -1.3490000000      -1.3490000000
LARGE BASIS cc-pVDZ
0.5      24
H      1.9263502692      0.7716497308      1.9263502692
H      1.9263502692      0.7716497308      -1.9263502692
H      -1.9263502692      0.7716497308      1.9263502692
H      -1.9263502692      0.7716497308      -1.9263502692
H      1.9263502692      -0.7716497308      1.9263502692
H      1.9263502692      -0.7716497308      -1.9263502692
H      -1.9263502692      -0.7716497308      1.9263502692
H      -1.9263502692      -0.7716497308      -1.9263502692
H      1.9263502692      1.9263502692      0.7716497308
H      1.9263502692      1.9263502692      -0.7716497308
H      -1.9263502692      1.9263502692      0.7716497308
H      -1.9263502692      1.9263502692      -0.7716497308
H      1.9263502692      -1.9263502692      0.7716497308
H      1.9263502692      -1.9263502692      -0.7716497308
H      -1.9263502692      -1.9263502692      0.7716497308
H      -1.9263502692      -1.9263502692      -0.7716497308
H      0.7716497308      1.9263502692      1.9263502692
H      0.7716497308      1.9263502692      -1.9263502692
H      -0.7716497308      1.9263502692      1.9263502692
H      -0.7716497308      1.9263502692      -1.9263502692
H      0.7716497308      -1.9263502692      1.9263502692
H      0.7716497308      -1.9263502692      -1.9263502692
H      -0.7716497308      -1.9263502692      1.9263502692
H      -0.7716497308      -1.9263502692      -1.9263502692
LARGE BASIS cc-pVDZ
#SMALL KINBAL
FINISH

```

Before running the calculation the user must make provide links to the atomic coefficient files :


```
ln -s ac.Pu AFPUXX
ln -s ac.C AFCXXX
ln -s ac.H AFHXXX
```

and the calculation was then run as :

```
pam --mol=PuO2.mol --inp=PuO2.inp --copy="AF*" --outcmo --mw=105 --mpi=32
```

This calculation converges smoothly after 9 iterations :

SCF - CYCLE

```
* Convergence on norm of error vector (gradient).
Desired convergence:1.000D-07
Allowed convergence:1.000D-06

* ERGVAL - convergence in total energy
* FCKVAL - convergence in maximum change in total Fock matrix
* EVCVAL - convergence in error vector (gradient)
```

Energy		ERGVAL	FCKVAL	EVCVAL	Conv.acc	CPU	Integrals	Time stamp			
It.	1	-30262.16062810	3.03D+04	0.00D+00	0.00D+00	Atomic s	8min27.160s	LL SL	Sun Dec	2	
It.	2	-29246.11569530	-1.02D+03	-4.18D+01	1.15D+01		18min50.650s	LL SL	Sun Dec	2	
It.	3	-29248.64080628	2.53D+00	4.35D+00	5.49D+00	DIIS 2	18min31.760s	LL SL	Mon Dec	3	
It.	4	-29248.86829228	2.27D-01	-1.32D+00	2.81D-01	DIIS 3	17min35.410s	LL SL	Mon Dec	3	
It.	5	-29248.86879546	5.03D-04	3.32D-02	3.76D-02	DIIS 4	16min51.240s	LL SL	Mon Dec	3	
It.	6	-29248.86880842	1.30D-05	8.21D-03	8.29D-04	DIIS 5	15min15.820s	LL SL	Mon Dec	3	
It.	7	-29248.86880842	3.88D-09	-5.99D-05	5.49D-05	DIIS 6	13min26.070s	LL SL	Mon Dec	3	
It.	8	-29248.86880843	3.43D-09	8.30D-06	1.55D-06	DIIS 7	11min31.390s	LL SL	Mon Dec	3	
It.	9	-29248.86880842	-7.49D-10	-1.04D-06	4.71D-08	DIIS 7	9min 3.550s	LL SL	Mon Dec	3	

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/augmentation.md

orphan

Basis sets augmentation

DIRAC suite offers possibility to easily modify employed basis sets.

The purpose of this tutorial, which is reflected in the test *basis_automatic_augmentation*, is to show nonstandard automatic augmentation of provided standard basis sets, like cc-pVXZ, Turbomole and Dyall's.

In the following table we demonstrate the influence of the (contracted) basis set extension on the total nonrelativistic energy and on the dipole moment, which is simply calculated through SCF wave-function expectation value.

Two molecules were chosen: HF and HBr. For the former molecule, the pre-cc-pVXZ and Turbomole basis sets are used for both F and H atoms. In the latter molecule, the heavy Br atom is described with Dyall's relativistic basis set, while the H atom is provided with the nonrelativistic basis set.

Basis set name	Atom	Basis size	Atom	Basis size	E(NR-SCF)/a.u.	dip. mom/D
aug-cc-pVDZ	F	[10s5p2d14s3p2d]	H	[5s2p13s2p]	-100.0337931	-1.89747844
d-aug-cc-pVDZ	F	[11s6p3d15s4p3d]	H	[6s3p14s3p]	-100.0339787	-1.88741792
t-aug-cc-pVDZ	F	[12s7p4d16s5p4d]	H	[7s4p15s4p]	-100.0341087	-1.88839962
Turbomole-DZP	F	[8s4p1d14s2p1d]	H	[4s1p12s1p]	-100.0028441	-1.94418011
s-a-Turbomole-DZP	F	[9s5p2d15s3p2d]	H	[5s2p13s2p]	-100.0180563	-1.91810891
Turbomole-TZVPP	F	[11s6p2d1f15s3p2d1f]	H	[5s2p1d13s2p1d]	-100.0657904	-1.91940698
s-a-Turbomole-TZVPP	F	[12s7p3d2f16s4p3d2f]	H	[6s3p2d14s3p2d]	-100.0668454	-1.89121376
dyall.v2z	Br	[15s11p7d1f15s11p7d1f]	(H	[4s1p12s1p])	-2572.6361195	-0.97693933
d-aug-dyall.v2z	Br	[17s13p9d3f17s13p9d3f]	(H	[4s1p12s1p])	-2572.6425365	-0.76521484

(H-atom in parenthesis has cc-pVDZ basis sets.)

Note that the nonrelativistic Hamiltonian with contracted basis sets was used in all these examples. Contracted basis sets were set for light elements (F,H), while for Br atom uncontracted scheme with Dyall's basis was preferred.

For relativistic calculation with either 2-component of 4-component Hamiltonians the nonrelativistic contraction of light elements basis sets is no longer suitable, especially if the molecule contains heavy elements. The user is advised to resort to uncontracting his basis sets. For example, for the HBr molecule you can combine decontracted cc-pVDZ basis set for hydrogen with the decontracted (as is) dyall.v2z basis for bromine.

Dyall's basis sets, which are constructed without contractions, can be used also with the X2C Hamiltonian.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/CmF/CmF.md

orphan

Electronic structure of CmF

We want to investigate the structure of the diatomic molecule CmF (Curium Fluoride). Following Mochizuki_JCP2003 we assume that curium has valence configuration in $5f^{77s^2}$ in the molecule, in particular since curium is a late actinide and thus quite similar to lanthanides. The molecule will therefore have an atomic-like open $5f^7$ shell. This may cause convergence problems since DIRAC by default assumes open-shell orbitals to have energies above close-shell ones, which is likely to not be the case here. In order to guide DIRAC we shall therefore employ the following strategy:

1. We calculate the curium atom.
2. We import the $5f^7$ orbitals into the molecular calculation, place them in the open shell and keep them frozen in a first SCF calculation.
3. We then freeze all closed-shell orbitals, allowing the $5f^7$ orbitals to relax in the molecule.

The curium atom

For ease of analysis we shall impose linear symmetry also in the atomic calculation. We achieve this through the introduction of a ghost center

Cm.xyz

The ground state configuration of curium (Cm, Z=96) is $[Rn]5f^{76d^17s^2}$ (see [here](#) for general information and [also here](#)), so with two open shells. We specify this configuration by using the keyword SCF_KPSELE:

Cm_KPSELE.inp

We run the calculation using:

```
pam --inp=Cm --mol=Cm.xyz --get "DFC0EF=cf,Cm"
```

This calculation converges smoothly in 13 iterations, and we are now ready to attack the molecule.

The molecular calculation:

1. Importing curium $5f$ orbitals

The menu file for the first step of the molecular calculation reads

CmF_5f_frz.inp

In the *SCF section it may be noted that we specify 49 closed-shell orbitals, followed by seven open-shell ones, with seven active electrons. Next we specify that we will take orbitals 45 to 51 from the coefficient file of the cerium atom, that we name AFCMXX, put them in position 50 to 56 and keep them frozen. This also implies that we are effectively doing a closed-shell calculation, which helps convergence as well. We run the calculation using the command:

```
pam --mw=200 --inp=CmF_5f_frz --mol=CmF.xyz --put "cf,Cm=AFCMXX" --get "DFC0EF=cf,CmF_5f_frz"
```

The calculation converges in 18 iterations. Here is a snippet from the population analysis:

CmF_5f_frz.pop

It shows that the upper closed-shell orbital (49) is dominated by cerium $(7)s$. The open-shell cerium $5f$ orbitals are frozen and keep their energies from the atomic calculation, with $5f_{5/2}$ and $5f_{7/2}$ orbitals having energies -0.5359 and -0.4840 E_h , respectively.

2. Relaxing the curium $5f$ orbitals

We now want to relax the curium $5f$ orbitals in the molecule whilst keeping the other orbitals frozen. This step is not necessarily required, but allows us to see how the $5f$ orbital energies evolve. We use the following input file

CmF_5f_relax.inp

It will be seen that we now import the coefficient file of the previous run and freeze all closed-shell orbitals. We use the command:

```
pam --mw=200 --inp=CmF_5f_frz --mol=CmF.xyz --put "cf,CmF_5f_frz=DFC0EF" --get "DFC0EF=cf,CmF_5f_relax"
```

The calculation converges in 14 iterations. Again a snippet from the Mulliken population analysis:

CmF_5f_relax.pop

We note that the curium $5f$ orbital energies are split by the molecular field, also that they are somewhat lowered. We also note that orbitals 46 - 48, dominated by fluorine $2p$ orbitals, are almost degenerate with the curium $5f$ orbitals, whereas orbital 50 is an almost pure curium $7s$ orbital.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/Ir/Ir_16plus.md

orphan

Getting excited states of Ir^{16+}

In the following we are interested in getting the ground ($^2S_{1/2}$) and two closest excited states ($^2P_{1/2}$, $^2P_{3/2}$) of the Ir^{16+} cation and their correlation energies. The electronic configurations of these states are

```
(^2S)      : [Xe] 4f(14) 5s(1) 5p1/2(0) 5p3/2(0)
(^2P_1/2) : [Xe] 4f(14) 5s(0) 5p1/2(1) 5p3/2(0)
(^2P_3/2) : [Xe] 4f(14) 5s(0) 5p1/2(0) 5p3/2(1)
```

For simplicity, we will work with the wo-component Hamiltonian (HAMILTONIAN.X2C) and employ the smallest $v2z$ decontracted basis set by K.Dyall. Due to the convergence problem of the standalone AMFI atomic SCF code we keep the +2 charge (AMFI.AMFICh) for mean-field orbitals.

There are two ways to obtain excited states - (i) from the converged SCF state, and, (ii) only at the correlated level from the 2P_aver SCF state.

For subsequent Coupled Cluster (CC) correlated calculations please soften the DHOLU variable in the subroutine DENOM (file *src/relecsd/cceqns.F*) to the value of 5.0D-4 and recompile DIRAC. Note that this is not recommended approach as the $p32$ states are in general not well described at the CC level. Nevertheless, with this little trick we can compare desired excited states calculated with both CC and Fock-space CC methods.

The $^2S_{1/2}$ ground state

Thanks to the linear symmetry, having two irreps, one can place the unpaired electron the 1st irrep to get the $^2S_{1/2}$ ground state. SCF calculations are followed by two CC calculations, where in the second one uses orbital energies for denominators.:

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2S12.scf_cc33e.2fs.inp
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2S12.scf_cc33e_oe.2fs.inp
```

Input files to download are Ir.dyall_v2z.lsym.mol <Ir.dyall_v2z.lsym.mol>, Z61.x2c.2S12.scf_cc33e.2fs.inp <Z61.x2c.2S12.scf_cc33e.2fs.inp>, Z61.x2c.2S12.scf_cc33e_oe.2fs.inp <Z61.x2c.2S12.scf_cc33e_oe.2fs.inp>. Corresponding output files are Z61.x2c.2S12.scf_cc33e.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2S12.scf_cc33e.2fs_Ir.dyall_v2z.lsym.out>, Z61.x2c.2S12.scf_cc33e_oe.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2S12.scf_cc33e_oe.2fs_Ir.dyall_v2z.lsym.out>.

The SCF $^2P_{1/2}$, $^2P_{3/2}$ excited states

By placing the unpaired electron into 2nd irrep one gets the $^2P_{1/2}$ first excited state: :

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2P12.scf_cc33e.2fs.inp --get "DFC0EF=DFC0EF.v2z.2P12"
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2P12.scf_cc33e_oe.2fs.inp
```

Input files to download Z61.x2c.2P12.scf_cc33e.2fs.inp <Z61.x2c.2P12.scf_cc33e.2fs.inp>, Z61.x2c.2P12.scf_cc33e_oe.2fs.inp <Z61.x2c.2P12.scf_cc33e_oe.2fs.inp>. Corresponding output files are Z61.x2c.2P12.scf_cc33e.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2P12.scf_cc33e.2fs_Ir.dyall_v2z.lsym.out>, Z61.x2c.2P12.scf_cc33e_oe.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2P12.scf_cc33e_oe.2fs_Ir.dyall_v2z.lsym.out>.

How to obtain the $^2P_{3/2}$ second excited state at the SCF level, and, consequently, at the CC level ? For that, we utilize the WAVE_FUNCTION.REORDER MO keyword with reading of the *DFC0EF.v2z.2P12* file from the previous run. Likewise one uses overlap selection in the SCF step: :

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2P32.scf_cc33e.2fs.inp --put "DFC0EF.v2z.2P12=DFC0EF"
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2P32.scf_cc33e_oe.2fs.inp --put "DFC0EF.v2z.2P12=DFC0EF"
```

Corresponding input files to download are Z61.x2c.2P32.scf_cc33e.2fs.inp <Z61.x2c.2P32.scf_cc33e.2fs.inp>, Z61.x2c.2P32.scf_cc33e_oe.2fs.inp <Z61.x2c.2P32.scf_cc33e_oe.2fs.inp>. Output files are Z61.x2c.2P12.scf_cc33e.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2P12.scf_cc33e.2fs_Ir.dyall_v2z.lsym.out>, Z61.x2c.2P12.scf_cc33e_oe.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2P12.scf_cc33e_oe.2fs_Ir.dyall_v2z.lsym.out>.

The CCSD(T) $^2P_{1/2}$, $^2P_{3/2}$ excited states

The other option is to start from the $^2P_{\text{aver}}$ averaged single determinant state and distinguish between individual $^2P_{1/2}$ and $^2P_{3/2}$ states at the Coupled Cluster correlated level thanks to the linear symmetry.

First we test the averaged, $^2P_{\text{aver}}$ state: :

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2Paver.scf_cc33e.2fs.inp
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2Paver.scf_cc33e_oe.2fs.inp
```

Files to download are Z61.x2c.2Paver.scf_cc33e.2fs.inp <Z61.x2c.2Paver.scf_cc33e.2fs.inp>, Z61.x2c.2Paver.scf_cc33e_oe.2fs.inp <Z61.x2c.2Paver.scf_cc33e_oe.2fs.inp>.

Afterwards we can proceed to the individual spin-orbit distinguished states, based on $M_{\{J\}}$ splitted occupation of each fermion irrep at the Coupled Cluster level.

First the first excited state, $^2P_{1/2}$:

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2Paver.scf_cc33e_2P12.2fs.inp
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2Paver.scf_cc33e_oe_2P12.2fs.inp
```

Input files to download are Z61.x2c.2Paver.scf_cc33e_2P12.2fs.inp <Z61.x2c.2Paver.scf_cc33e_2P12.2fs.inp>, Z61.x2c.2Paver.scf_cc33e_oe_2P12.2fs.inp <Z61.x2c.2Paver.scf_cc33e_oe_2P12.2fs.inp>. Corresponding output files are Z61.x2c.2Paver.scf_cc33e_2P12.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2Paver.scf_cc33e_2P12.2fs_Ir.dyall_v2z.lsym.out>, Z61.x2c.2Paver.scf_cc33e_oe_2P12.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2Paver.scf_cc33e_oe_2P12.2fs_Ir.dyall_v2z.lsym.out>.

Then we proceed to the second excited state, $^2P_{3/2}$:

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2Paver.scf_cc33e_2P32.2fs.inp
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.2Paver.scf_cc33e_oe_2P32.2fs.inp
```

Files to download are Z61.x2c.2Paver.scf_cc33e_2P32.2fs.inp <Z61.x2c.2Paver.scf_cc33e_2P32.2fs.inp>, Z61.x2c.2Paver.scf_cc33e_oe_2P32.2fs.inp <Z61.x2c.2Paver.scf_cc33e_oe_2P32.2fs.inp>. Corresponding output files are Z61.x2c.2Paver.scf_cc33e_2P32.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2Paver.scf_cc33e_2P32.2fs_Ir.dyall_v2z.lsym.out>, Z61.x2c.2Paver.scf_cc33e_oe_2P32.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.2Paver.scf_cc33e_oe_2P32.2fs_Ir.dyall_v2z.lsym.out>.

The $^2S_{1/2}$, $^2P_{1/2}$ and $^2P_{3/2}$ FSCCSD states

Simple and very stable approach to obtain ground and multiple excited states in one step is through the Fock-space Coupled Cluster method. Starting from the closed-shell system, Ir^{17+} , one gets - by solving (01) sector all three correlated states of interest, $^2S_{1/2}$, $^2P_{1/2}$ and $^2P_{3/2}$:

```
pam --noarch --mw=120 --mol=Ir.dyall_v2z.lsym.mol --inp=Z61.x2c.scf_fsc01_33ce_5s5p.2fs.inp
```

The input file to download is Z61.x2c.scf_fsc01_33ce_5s5p.2fs.inp <Z61.x2c.scf_fsc01_33ce_5s5p.2fs.inp>. Corresponding output file is Z61.x2c.scf_fsc01_33ce_5s5p.2fs_Ir.dyall_v2z.lsym.out <Z61.x2c.scf_fsc01_33ce_5s5p.2fs_Ir.dyall_v2z.lsym.out>.

Overview of excitation energies

In the following table we summarize excitation energies. All values are in a.u. Energies in the Table are not rounded, the are cut to 8 decimal places (“oe” means orbital energies used in CC denominators, otherwise recalculated diagonal Fock matrix elements).

Method	$^2S_{1/2}$	$^2P_{1/2}$	$^2P_{3/2}$	2S12-2P12	2S12-2P32
(SCF ref)					
SCF	-17751.10181462	-17749.67796221	-17748.96107014	1.42385	2.14074
CCSD	-17751.90589433	-17750.48885480	-17749.77167089	1.41704	2.13422
CCSDoe	-17751.90589433	-17750.48885479	-17749.77167088	1.41704	2.13422
CCSD(T)	-17751.90998637	-17750.49777405	-17749.78015403	1.41221	2.12983
CCSD(T)oe	-17751.90999205	-17750.49779342	-17749.78020119	1.41220	2.12979
(CC ref)					
CCSD	-17751.90589433	-17750.48895732	-17749.77154045	1.41694	2.13435
CCSDoe	-17751.90589433	-17750.48895728	-17749.77154043	1.41694	2.13435
CCSD(T)	-17751.90998637	-17750.49779338	-17749.78012458	1.41219	2.12986
CCSD(T)oe	-17751.90999205	-17750.49784867	-17749.78024342	1.41214	2.12974
FSCCSD	-17751.90324662	-17750.48431436	-17749.76770431	1.41893	2.13554

It seems that quality of computed excitation energies increases in the line SCF-FSCCSD-CCSD-CCSD(T). Triple excitations (CCSD(T) results) are significant.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/LuF3/LuF3.md

orphan

Investigating covalency of LuF_3

The problem

This case study concerns whether the lutetium 4f orbitals participate in bonding in lutetium trifluoride. This is very much a leading question in that it supposes that we can identify these orbitals in the molecular wave function and also judge whether they are chemically active.

Concerning the latter point, one should note that canonical Hartree-Fock or Kohn-Sham orbitals are in general not suitable for ‘seeing’ chemical bonds since they follow irreps of point groups and tend to delocalize over the molecule. One option is therefore to exploit the invariance of the electronic energy upon rotations amongst the occupied orbitals of a closed-shell molecule to localize molecular orbitals

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/MnO6.md

orphan

The MnO_6 system

The MnO_6 compound is good example of the oxide crystal structure.

In the following we show how one can get converged molecular spinors (or molecular orbitals) of its ground state using single determinant SCF method.

We take into account two parameters for controlling the SCF convergence - the SCF_OPENFACTOR and the SCF_0LEVEL keywords.

Here the noAcAc abbreviation (“no active-active”) means value of OPENFAC=0, what is no active-active interaction among open-shell spinors. The wAcAc (“with active-active”) means OPENFAC=1, that is full active-active interaction (or full open-shell contribution to the Fock matrix).

For simplicity, we employ the two-component Hamiltonian, see the keyword `HAMILTONIAN_X2C`.

The way to the convergence

With the aim to achieve the SCF convergence we investigated these three ways:

1. Starting from noAcAc we get the SCF convergence. We save the DFCOEF file with molecular spinors. Input files to download are `Mn06.x2c.bare_noacac.inp` <../../../../test/tutorial_Mn06/Mn06.x2c.bare_noacac.inp> and `Mn06.crystal.mol` <../../../../test/tutorial_Mn06/Mn06.crystal.mol>:

```
pam --inp=Mn06.x2c.bare_noacac.inp --mol=Mn06.crystal.mol --get "DFCOEF=DFCOEF.Mn06.noAcAc"
```

2. Using AcAc with molecular spinors (the DFCOEF file) from the previous SCF run the `$MnO_6$` system does not converge with the default DIIS accelerator. The new input file to download is `Mn06.x2c.wacac.inp` <../../../../test/tutorial_Mn06/Mn06.x2c.wacac.inp>:

```
pam --inp=Mn06.x2c.wacac.inp --mol=Mn06.crystal.mol --put "DFCOEF.Mn06.noAcAc=DFCOEF"
```

3. Using the AcAc interaction with the parameter of `OLEVEL=0.2`, and, with the DFCOEF file from the point 1, we finally obtain the convergence. The total SCF energy of the system is *-1617.5839185788545 a.u.* (see the corresponding output file, `Mn06.x2c.wacac_olvlshift_Mn06.crystal.out` <../../../../test/tutorial_Mn06/result/Mn06.x2c.wacac_olvlshift_Mn06.crystal.out>) The new DIRAC input file to download is `Mn06.x2c.wacac.inp` <../../../../test/tutorial_Mn06/Mn06.x2c.wacac_olvlshift.inp>:

```
pam --inp=Mn06.x2c.wacac_olvlshift.inp --mol=Mn06.crystal.mol --put "DFCOEF.Mn06.noAcAc=DFCOEF" --get "DFCOEF=DFCOEF.Mn06.wAcAc"
```

This tutorial shows that it is important to try all possible means to achieve the SCF convergence if the default setting is not working.

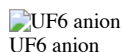
FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/UF6_anion/UF6_anion.md

orphan

The \$UF_6\$ anion

The problem

This case study, suggested by [Kirk Peterson](#) concerns the calculation of the `$UF_6$` anion using effective core potentials (ECP)



UF6 anion

We start from the molecular input `UF6.mol`

`UF6.mol`

and the menu file `UF6.inp`

`UF6.inp1`

The problem is that this calculation simply does not converge.

Analysis

The neutral molecule is a simple closed shell molecule and using the menu file

`UF6neutral.inp`

the SCF converges nicely in 26 iterations. Before proceeding we investigate the electronic structure of this species using projection analysis, see Dubillard2006 and [this tutorial](#). Looking at the input file above, one may note that we already prepared for this by invoking the keyword `GENERAL_ACMOUT` in order to recover the `$C_1$ MO` coefficients:

```
pam --inp=UF6 --mol=UF6 --outcmo --get "DFACM0=ac.UF6"
```

We now generate orbitals for the constituent atoms of `$UF_6$` in their electronic ground state. For the fluorine atoms this is straightforwardly obtained using the molecular input `F.mol`

`F.mol`

and the menu file `F.inp`

`F.inp`

using the command:

```
pam --inp=F --mol=F --outcmo --get "DFACM0=ac.F"
mv DFCOEF cf.UF6
```

Note that since no effective core potential has been given for the fluorine we prefer to calculate it using the X2C Hamiltonian.

The corresponding molecular input for the uranium atom is

U.mol

For the uranium atom, the calculation of the occupation easily converges by using the keyword SCF_.KPSELE introduced in DIRAC21.

U_KPSELE.inp

using the command:

```
pam --inp=U --mol=U --get "DFACM0=ac.U"
```

The projection analysis is carried out in C_{1v} symmetry to make all fluorines symmetry-independent. The proper molecular input for UF_6 is found as an xyz-file in the DIRAC output

UF6.xyz

whereas the menu file prj.inp is given by

prj.inp

Before running the analysis we have to create symbolic links for the reference files:

```
ln -s ac.U AFUXXX
ln -s ac.F AFF1XX
ln -s ac.F AFF2XX
ln -s ac.F AFF3XX
ln -s ac.F AFF4XX
ln -s ac.F AFF5XX
ln -s ac.F AFF6XX
```

The analysis is now run using:

```
cp ac.UF6 DFC0EF
pam --inp=prj --mol=UF6.xyz --incmo --copy="AF*"
```

Looking at the end of the output we find

prj_UF6.out

showing that the polarization contribution is rather small. It can be eliminated completely by adding the keyword ,POLREF under the *PROJEC section (valid from DIRAC13). The gross population of the uranium valence orbitals is then given by

prjpol_UF6.out

We can identify the various orbitals from orbital energies and degeneracies or by looking in the Mulliken output of the atomic calculation. It is seen that the fluorines have charges -0.57 and uranium +3.45. The electronic configuration of uranium in the neutral molecule is $5f^{1.23}6d^{1.32}7s^{0.06}$.

Calculating the anion

Now let us consider adding an electron. From the output of the neutral molecule we find that the HOMO (*ungerade*) has energy -0.642 E_h and the LUMO (*ungerade*) comes in at -0.115 E_h . In fact, Mulliken population analysis shows that the first seven virtual orbitals are indeed basically f orbitals. We therefore attempt a straightforward calculation starting from the coefficients of the neutral molecule and using the menu file UF6a.inp

UF6anion.inp

and the command:

```
pam --copy="cf.UF6=DFC0EF" --inp=UF6a --mol=UF6 --outcmo
```

Note that we activate static overlap selection to keep occupied orbitals in place. However, this calculation does not converge. Also, in the Mulliken population analysis we activate the SHELL keyword to simplify identification of atomic orbitals, see MULPOP_.LABEL

Looking at the open-shell orbitals we find that they are essentially f orbitals. However, we also note that the HOMO of the closed shell orbitals comes in at -0.40353915 E_h and the lowest occupied open-shell orbitals is almost degenerate, having orbital energy -0.39634956 E_h . Clearly this almost degeneracy creates artificial mixing which perturbs convergence. We therefore add an open-shell level shift:

```
.0LEVEL
0.200
```

and now the SCF converges in 38 iterations. Curiously, convergence is extremely sensitive to the choice of the level shift. Even changing it by 0.005 breaks convergence !

orphan

The UF_6 anion - another approach

Different and more robust solution for obtaining the SCF convergence of the UF_6 anion is provided here. It is based on the damping of the Fock matrix from SCF iteration to iteration.

Starting from neutral UF_6

In the first step we follow the previous approach of starting from the closed-shell (neutral) UF_6 system. (We also may utilize a shift in the virtual space to reduce mixing between occupied and virtual spinors.) In any case, we get smooth convergence with the default DIIS method. Files to download are UF6neutral.inp, UF6.mol, cc-pVDZ-PP and ECP60MDF: :

```
pam --noarch --inp=UF6neutral.inp --mol=UF6.mol --get "DFC0EF=DFC0EF.UF6neutral"
```

The \$UF₆\$ anion from neutral

In the next step we compare two calculations starting from neutral molecular spinors of UF₆ (the file DFCOEF.UF6neutral). One calculation uses the DIIS method (not shown here) and the other uses the damping with two damping factors of 0.90 and 0.50. (File to download is UF6_anion_scf damp.inp): :

```
pam --noarch --inp=UF6_anion_scf damp.inp --mol=UF6.mol --put "DFCOEF.UF6neutral=DFCOEF" --get "DFCOEF=DFCOEF.UF6anion_damp_pass1" --replace d
pam --noarch --inp=UF6_anion_scf damp.inp --mol=UF6.mol --put "DFCOEF.UF6anion_damp_pass1=DFCOEF" --get "DFCOEF=DFCOEF.UF6anion_damp_pass2" --
```

Concerning the DIIS speedup method, the SCF iterations initially converge to ca 10^{-5} but then began to diverge and after about 150 iterations the error gradient of the Fock matrix is still around 10^{-5} .

However, with the damping convergence speedup the calculation converges to ca 10^{-8} after about 50 iterations. The damping scheme uses more of the input Fock matrix than the output Fock matrix. Concerning the damping factor, the lower than 1 it is, we should get faster convergence. However, when the factor is getting too small, iterations diverge. The closer we are to the converged states, the smaller the damping factor can be.

If we begin calculations from the converged solutions from the damping, the DIIS iterations converge rapidly after few iterations. But this is only with the deactivated active-active shells interaction (new file to download is UF6anion2.inp): :

```
pam --noarch --inp=UF6anion2.inp --mol=UF6.mol --put "DFCOEF.UF6anion_damp_pass2=DFCOEF" --get "DFCOEF=DFCOEF.UF6anion_diis"
```

The \$UF₆\$ anion with OPENFAC=1

Unfortunately, with the factor OPENFAC=1 (what is the keyword SCF_.OPENFACTOR) we were not able to get the convergence, neither with the default DIIS, nor with damping factor (to download, UF6anion.inp and UF6anion3.inp): :

```
pam --noarch --inp=UF6anion.inp --mol=UF6.mol --put "DFCOEF.UF6anion_diis=DFCOEF"
pam --noarch --inp=UF6anion3.inp --mol=UF6.mol --put "DFCOEF.UF6anion_diis=DFCOEF" --replace dampfactor=0.8
```

More effort is required to obtain converged molecular spinors of the \$UF₆\$ anion with the active-active open-shell interaction (that is OPENFAC=1). We leave this task to the reader.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/UF6_anion/UF6_anion2.md

orphan

The \$UF₆\$ anion - another approach

Different and more robust solution for obtaining the SCF convergence of the \$UF₆\$ anion is provided here. It is based on the damping of the Fock matrix from SCF iteration to iteration.

Starting from neutral \$UF₆\$

In the first step we follow the previous approach of starting from the closed-shell (neutral) \$UF₆\$ system. (We also may utilize a shift in the virtual space to reduce mixing between occupied and virtual spinors.) In any case, we get smooth convergence with the default DIIS method. Files to download are UF6neutral.inp, UF6.mol, cc-pVDZ-PP and ECP60MDF: :

```
pam --noarch --inp=UF6neutral.inp --mol=UF6.mol --get "DFCOEF=DFCOEF.UF6neutral"
```

The \$UF₆\$ anion from neutral

In the next step we compare two calculations starting from neutral molecular spinors of UF₆ (the file DFCOEF.UF6neutral). One calculation uses the DIIS method (not shown here) and the other uses the damping with two damping factors of 0.90 and 0.50. (File to download is UF6_anion_scf damp.inp): :

```
pam --noarch --inp=UF6_anion_scf damp.inp --mol=UF6.mol --put "DFCOEF.UF6neutral=DFCOEF" --get "DFCOEF=DFCOEF.UF6anion_damp_pass1" --replace d
pam --noarch --inp=UF6_anion_scf damp.inp --mol=UF6.mol --put "DFCOEF.UF6anion_damp_pass1=DFCOEF" --get "DFCOEF=DFCOEF.UF6anion_damp_pass2" --
```

Concerning the DIIS speedup method, the SCF iterations initially converge to ca 10^{-5} but then began to diverge and after about 150 iterations the error gradient of the Fock matrix is still around 10^{-5} .

However, with the damping convergence speedup the calculation converges to ca 10^{-8} after about 50 iterations. The damping scheme uses more of the input Fock matrix than the output Fock matrix. Concerning the damping factor, the lower than 1 it is, we should get faster convergence. However, when the factor is getting too small, iterations diverge. The closer we are to the converged states, the smaller the damping factor can be.

If we begin calculations from the converged solutions from the damping, the DIIS iterations converge rapidly after few iterations. But this is only with the deactivated active-active shells interaction (new file to download is UF6anion2.inp): :

```
pam --noarch --inp=UF6anion2.inp --mol=UF6.mol --put "DFCOEF.UF6anion_damp_pass2=DFCOEF" --get "DFCOEF=DFCOEF.UF6anion_diis"
```

The \$UF₆\$ anion with OPENFAC=1

Unfortunately, with the factor OPENFAC=1 (what is the keyword SCF_.OPENFACTOR) we were not able to get the convergence, neither with the default DIIS, nor with damping factor (to download, UF6anion.inp and UF6anion3.inp): :

```
pam --noarch --inp=UF6anion.inp --mol=UF6.mol --put "DFCOEF.UF6anion_diis=DFCOEF"
pam --noarch --inp=UF6anion3.inp --mol=UF6.mol --put "DFCOEF.UF6anion_diis=DFCOEF" --replace dampfactor=0.8
```

More effort is required to obtain converged molecular spinors of the UF_6^- anion with the active-active open-shell interaction (that is $\text{OPENFAC}=1$). We leave this task to the reader.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/UF6_molecule/UF6.md

orphan

UF6 molecule

UF_6 (146 electrons, O_h symmetry) is known compound connected with the uranium enrichment. We shall investigate UF_6 as the closed-shell (singlet) state and in the highest D_{2h} computational symmetry (`UF6.v2z.D2h.mol <../../../../test/tutorial_UF6/UF6.v2z.D2h.mol>`).

Note that tutorial shows different ways for obtaining convergence than another one for UF_6^- , UF_6^- anion, which is based upon relativistic effective core potentials.

Atomic start

We demonstrate the atomic start combined with the automatic occupation scheme, which successfully gives converged state with the occupation of $(E_l, E_u)=(72, 74)$, see the input file, `x2c_a4p.atstart_scf_autoocc_2fs.inp <../../../../test/tutorial_UF6/x2c_a4p.atstart_scf_autoocc_2fs.inp>`, and the output file `<../../../../test/tutorial_UF6/result/x2c_a4p.atstart_scf_autoocc_2fs_UF6.v2z.D2h.out>`.

Input files for corresponding F,U atomic runs are `F.v2z.D2h.mol <../../../../test/tutorial_UF6/F.v2z.D2h.mol>`, `F.x2c.scf_2fs.inp <../../../../test/tutorial_UF6/F.x2c.scf_2fs.inp>`, `U.v2z.D2h.mol <../../../../test/tutorial_UF6/U.v2z.D2h.mol>`, `x2c.scf_autoocc.inp <../../../../test/tutorial_UF6/x2c.scf_autoocc.inp>`. Output files from these (F,U) atomic runs are `F.x2c.scf_2fs_F.v2z.D2h.out <../../../../test/tutorial_UF6/result/F.x2c.scf_2fs_F.v2z.D2h.out>` and `x2c.scf_autoocc_U.v2z.D2h.out <../../../../test/tutorial_UF6/result/x2c.scf_autoocc_U.v2z.D2h.out>`.

Note that it is not necessary to specify proper atomic configuration of the U atom as the auto-occupation scheme does the job and produces suitable starting MOs.

Autooccupation

It is possible to obtain proper orbital occupation thanks to the implemented atomic potential start and avoid the lengthy atomic start procedure given above.

The autooccupation gives the lowest SCF energy, see the output file `x2c_a4p.scf_autoocc_UF6.v2z.D2h.out <../../../../test/tutorial_UF6/result/x2c_a4p.scf_autoocc_UF6.v2z.D2h.out>`, the input file, `x2c_a4p.scf_autoocc.inp <../../../../test/tutorial_UF6/x2c_a4p.scf_autoocc.inp>`, and resulting MO coefficients, `DFPCMO.UF6.x2c_a4p.scf.v2z <../../../../test/tutorial_UF6/DFPCMO.UF6.x2c_a4p.scf.v2z>`.

Verification of the occupation

Next, we have to verify that this occupation of spinors gives the lowest energy. For that, we perform several atomic-start based SCF calculations with different occupation schemes. Results - occupations and SCF energy - are collected in the following table:

occupation	SCF energy
68, 78	not converged
70, 76	-28636.011248088082
72, 74	-28636.624880692940
74, 72	-28635.715232162675
76, 70	-28634.793937853457

We see that manually assigned occupation of 72,74 gives the lowest energy, what is the proof of the correct autooccupation settings (see one input file, `UF6.x2c_a4p.atstart_scf_autoocc_72_74 <../../../../test/tutorial_UF6/UF6.x2c_a4p.atstart_scf_autoocc_72_74.inp>`).

Geometry optimization

The UF_6 geometry optimization, thanks to the high O_h symmetry, requires change of one parameter only - the U-F bond distance.

To carry out the UF_6 structure optimization with the BP86 functional effectively, let us first obtain starting set of (DFT) molecular orbitals. For that, we first run single point DFT step with molecular orbitals file from the previous SCF step and reuse them for the DFT-geometry optimization. See the BP86 input file, `UF6.x2c_a4p.bp86_72_74.inp <../../../../test/tutorial_UF6/UF6.x2c_a4p.bp86_72_74.inp>`, and the geometry optimization input, `UF6.geomopt.x2c_a4p.bp86_72_74.inp <../../../../test/tutorial_UF6/UF6.geomopt.x2c_a4p.bp86_72_74.inp>`.

Thanks to this MO start, number SCF iterations during the geometry optimization cycle is less than 10, see the corresponding output file `<../../../../test/tutorial_UF6/result/UF6.geomopt.x2c_a4p.bp86_72_74_UF6.v2z.D2h.out>`.

The final U-F bond distance obtained with the X2c-BP86-v2z method is 2.020Å (experimental distance is 1.999Å).

Test script

The Dirac Python test script for this tutorial is `test <../../../../test/tutorial_UF6/test>`. Pay attention to comments and commented lines to be able to reproduce this tutorial's demonstrative calculations.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/case_studies/UO6_6minus/UO6_6minus.md

orphan

The $\text{UO}_6^{(6-)}$ anion

Another (difficult) case study example is the $\text{UO}_6^{(6-)}$ anion. The molecule is in squashed geometry (not fully octahedral). Axial and equatorial bond distances are not equal.

The $\text{UO}_6^{(6-)}$ ground state

The ground state of $\text{UO}_6^{(6-)}$ converges fine, but not with the default DIIS and starting from screened bare nucleus. Rather one has to use the robust damping scheme. Try first the DAMPFC factor of 0.9 :

```
pam --noarch --get "DFCOEF=DFCOEF.UO6_6m_ground_1pass" --replace dampfactor=0.90 --inp=UO6_6minus.x2c.scf_damp.inp --mol=UO6.mol
```

(Files to download are UO6_6minus.x2c.scf_damp.inp and UO6.mol.)

Then, after we get the *DFCOEF.UO6_6m_ground_1pass* file, we decrease the damping level to 0.65. One could try also some lower value: :

```
pam --noarch --put "DFCOEF.UO6_6m_ground_1pass=DFCOEF" --get "DFCOEF=DFCOEF.UO6_6m_ground_2pass" --replace dampfactor=0.50 --inp=UO6_6minus.
```

In this way we get converge molecular spinors saved in the *DFCOEF.UO6_6m_ground_2pass* file.

The $\text{UO}_6^{(5-)}$ anion

Now to the next state - the $\text{UO}_6^{(5-)}$ system. With $2p_{3/2}$ electron out of the U atom there is a difficult convergence. There are (at least) five subsequent runs for that.

First run cca 500 iterations with the damping factor of 0.65 and with the shift of virtual spinors of +0.65. This goes very slowly. The pass two continues from pass one. Then pass three from previous pass up to pass 4, which has another 500 iteration. Damping factors of these passes were decreasing from 0.65 to 0.25. Finally, in pass 5 the convergence was reached with the default DIIS method: :

```
pam --noarch --put "DFCOEF.UO6_6m_ground_2pass=DFCOEF" --get "DFCOEF=DFCOEF.UO6_5m_2p3_1pass" --replace dampfactor=0.65 --inp=UO6_5m.x2c.scf
pam --noarch --put "DFCOEF=DFCOEF.UO6_5m_2p3_1pass=DFCOEF" --get "DFCOEF=DFCOEF.UO6_5m_2p3_2pass" --replace dampfactor=0.55 --inp=UO6_5m.x2c
pam --noarch --put "DFCOEF=DFCOEF.UO6_5m_2p3_2pass=DFCOEF" --get "DFCOEF=DFCOEF.UO6_5m_2p3_3pass" --replace dampfactor=0.45 --inp=UO6_5m.x2c
pam --noarch --put "DFCOEF=DFCOEF.UO6_5m_2p3_3pass=DFCOEF" --get "DFCOEF=DFCOEF.UO6_5m_2p3_4pass" --replace dampfactor=0.25 --inp=UO6_5m.x2c
pam --noarch --put "DFCOEF=DFCOEF.UO6_5m_2p3_4pass=DFCOEF" --get "DFCOEF=DFCOEF.UO6_5m_2p3_diiis_5pass" --inp=UO6_5m.x2c.scf_2p3.inp --mol=UO
```

(For downloading is the file UO6_5m.x2c.scf_2p3_damp.inp). Note that user may encounter difficulties when reproducing SCF convergence of this tutorial case.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/cc_memory_count/count_cc_memory.md

orphan

Counting the RelCC memory consumption

It is of great importance to know the exact number of the memory needed for a high memory and time demanding RelCC job.

SCF

The first step is the SCF run with saving molecular orbitals, preferably as DFPCMO text file. Afterwards, using obtained molecular orbitals, prepare the active space for MOLTRA and RelCC.

MOLTRA & RelCC

Equip the MOLTRA input with the MOLTRA_.N04IND keyword and RelCC input with the RELCC_.COUNTMEM keyword. Run SCF part with previously saved converged molecular orbitals, with one iteration. The job passes quickly through MOLTRA integrals transformations as it produces only two-index quantities in the MRCONEE file.

In the RelCC module, the program counts the memory need and leaves.

Example

Demonstration of this feature is in the DIRAC *test/count_cc_memory*, see test <test>

We found the almost minimal setting - via the *-ag* parameter flag - for the memory counting run:

```
pam --noarch --gb=0.20 --ag=0.366 --inp=H20.ae4z.x2c.scf_countmem-relcc.inp --mol=H20.xyz --put "DFPCMO.H20.x2c.ae4z=DFPCMO"
```

Input files are H20.ae4z.x2c.scf_countmem-relcc.inp <H20.ae4z.x2c.scf_countmem-relcc.inp>, H20.xyz <H20.xyz>, producing the output file <H20.ae4z.x2c.scf_countmem-relcc_H20.out>.

We are to look the “Peak memory usage” to get reasonable estimate of dynamic memory:

Predicted RelCC memory demand: 0.335 GB
Peak memory usage (Gb) : 0.363

Next step is the sharp run based on the previous RelCC memory conuting. The serial job needs at least 0.366 GB, rounding to 0.4 GB of the total memory:

pam --noarch --gb=0.20 --ag=0.366 --inp=H20.ae4z.x2c.scf_relcc.inp --mol=H20.xyz --put "DFPCM0.H20.x2c.ae4z=DFPCM0"

We also perform the parallel run, which consumes at least 16*0.366 = 5.856 GB of the memory, rounding to 5.9 GB. This amount of memory must be free at the running machine equipped with at least 16 CPUs:

pam --noarch --mpi=16 --gb=0.20 --ag=0.366 --inp=H20.ae4z.x2c.scf_relcc.inp --mol=H20.xyz --put "DFPCM0.H20.x2c.ae4z=DFPCM0"

Have a look at the output <H20.ae4z.x2c.scf_relcc_H20.out> file. We search there the peak memory usage which is cca 99.2% of the --ag parameter.

Predicted RelCC memory demand: 0.335 GB
Peak memory usage: 0.363 GB

Therefore in practical RelCC memory demanding calculations watch for the **Peak memory usage** value and set the --ag parameter accordingly. This may take few iterations, especially when you want to find the real memory minimum. Don’t forget to multiply your memory amount with the number of threads in parallel calculations.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dft/bed.md

orphan

Beyond the electric dipole approximation: the full light-matter interaction

Introduction

write me ...

Case study 1: The radon atom

Case study 2: Titanium tetrachloride

As a molecular example we will consider titanium tetrachloride. We use the structure employed in Lestrangle_JChemPhys2015

TiCl4.xyz

The full molecular symmetry is \$T_d\$, but DIRAC will work in the \$D_2\$ subgroup. We focus on excitations from the Cl \$1s\$ to vacant Ti \$3d\$ orbitals. The relation between irreps of \$T_d\$ and \$D_2\$ is

\$T_d\$ \$D_2\$
\$A_1\$ \$A\$ \$x^2+y^2+z^2\$
\$A_2\$ \$A\$
\$E\$ \$A\oplus A\$ \$(2z^2-x^2-y^2, x^2-y^2)\$
\$T_1\$ \$B_1\oplus B_2\oplus B_3\$ \$(R_x, R_y, R_z)\$
\$T_2\$ \$B_1\oplus B_2\oplus B_3\$ \$(x, y, z)\$

The table also shows that the components of the electric dipole operator transforms as \$T_2\$.

We first perform a spin-orbit free DFT/PBE0 calculation of the molecule using the input file

TiCl4_PBE0.inp

Curiously the DFT calculation does not converge, but by first running HF and restart we converge (HOMO-LUMO gap 0.219 \$E_h\$). The Cl \$1s\$ orbitals span \$A_1\oplus T_2\$. Looking at the Mulliken population analysis:

* Electronic eigenvalue no. 2: -102.11926181928 (Occupation : f = 1.0000) sym= B2

Gross	Total		L B2 Cl s	A Cl _small	B3 Cl _small	B1 Cl _small
alpha	0.0024		0.0000	0.0012	0.0000	0.0012
beta	0.9976		0.9963	0.0000	0.0012	0.0000

* Electronic eigenvalue no. 3: -102.11926181926 (Occupation : f = 1.0000) sym= B3

Gross	Total		L B3 Cl s	A Cl _small	B2 Cl _small	B1 Cl _small
alpha	0.0024		0.0000	0.0012	0.0000	0.0012
beta	0.9976		0.9963	0.0000	0.0012	0.0000

* Gross populations greater than 0.00010

Gross	Total		L B3 Cl s	A Cl _small	B2 Cl _small	B1 Cl _small
alpha	0.0024		0.0000	0.0012	0.0000	0.0012
beta	0.9976		0.9963	0.0000	0.0012	0.0000

* Electronic eigenvalue no. 4: -102.11926181917 (Occupation : f = 1.0000) sym= B1

=====

* Gross populations greater than 0.00010

Gross	Total		L B1 Cl s	A Cl _small	B2 Cl _small	B3 Cl _small
alpha	0.9976		0.9963	0.0012	0.0000	0.0000
beta	0.0024		0.0000	0.0000	0.0012	0.0012

* Electronic eigenvalue no. 5: -102.11926179218 (Occupation : f = 1.0000) sym= A

=====

* Gross populations greater than 0.00010

Gross	Total		L A Cl s	B2 Cl _small	B3 Cl _small	B1 Cl _small
alpha	0.9976		0.9963	0.0000	0.0000	0.0012
beta	0.0024		0.0000	0.0012	0.0012	0.0000

we find from degeneracy and symmetry that orbitals 2,3 and 4 span \$T_2\$, whereas orbital 5 span \$A_1\$. Likewise, looking at the Mulliken population analysis for the lower virtuals:

* Electronic eigenvalue no. 46: -0.1347912984154 (Occupation : f = 0.0000) sym= A

=====

* Gross populations greater than 0.00010

Gross	Total		L A Ti dxx	L A Ti dyd	L A Ti dzd	L A Ti g202	L A Ti g022	L A Cl px	L A Cl py
alpha	0.9999		0.3412	0.4428	0.0066	0.0004	0.0003	0.0748	0.0971
beta	0.0001		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Gross		L A Cl pz	L A Cl dxx	L A Cl dxy	L A Cl dxz	L A Cl dyd	L A Cl dyz	L A Cl dzd	L A Cl fxxx
alpha		0.0014	0.0074	0.0001	0.0092	0.0096	0.0071	0.0001	-0.0003
beta		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Gross		L A Cl fxy	L A Cl fxz	L A Cl fyy	L A Cl fyzz	L A Cl fyyz	L A Cl fyzz
alpha		0.0009	0.0003	0.0011	-0.0005	-0.0004	0.0007
beta		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

* Electronic eigenvalue no. 47: -0.1347912984122 (Occupation : f = 0.0000) sym= A

=====

* Gross populations greater than 0.00010

Gross	Total		L A Ti dxx	L A Ti dyd	L A Ti dzd	L A Ti g220	L A Ti g022	L A Cl px	L A Cl py
alpha	0.9999		0.1859	0.0843	0.5205	0.0004	0.0002	0.0407	0.0185
beta	0.0001		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Gross		L A Cl pz	L A Cl dxx	L A Cl dxy	L A Cl dxz	L A Cl dyd	L A Cl dyz	L A Cl dzd	L A Cl fxxx
alpha		0.1141	0.0040	0.0109	0.0018	0.0018	0.0039	0.0113	-0.0002
beta		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Gross		L A Cl fxy	L A Cl fxz	L A Cl fyy	L A Cl fyzz	L A Cl fyyz	L A Cl fyzz
alpha		-0.0002	0.0004	-0.0004	0.0012	0.0011	-0.0004
beta		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

* Electronic eigenvalue no. 48: -0.1090420939155 (Occupation : f = 0.0000) sym= B2

=====

* Gross populations greater than 0.00010

Gross	Total		L B2 Ti py	L B2 Ti dxz	L B2 Ti fxy	L B2 Ti fyy	L B2 Ti fyzz	L B2 Ti g301	L B2 Ti g121
alpha	0.0001		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
beta	0.9999		0.0565	0.6691	0.0023	-0.0014	0.0023	0.0002	-0.0017

Gross		L B2 Ti g103	L B2 Cl s	L B2 Cl px	L B2 Cl py	L B2 Cl pz	L B2 Cl dxx	L B2 Cl dxy	L B2 Cl dxz
alpha		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
beta		0.0002	0.0048	0.0107	0.2054	0.0107	0.0040	0.0091	0.0012

Gross		L B2 Cl dyd	L B2 Cl dyz	L B2 Cl dzd	L B2 Cl fxxx	L B2 Cl fxy	L B2 Cl fxyz	L B2 Cl fyy	L B2 Cl fyzz
alpha		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
beta		0.0122	0.0091	0.0040	0.0004	0.0004	0.0004	-0.0007	0.0004

Gross | L B2 Cl fzzz

alpha		0.0000
beta		0.0004

* Electronic eigenvalue no. 49: -0.1090420939055 (Occupation : f = 0.0000) sym= B1

=====

* Gross populations greater than 0.00010

Gross	Total		L B1 Ti pz	L B1 Ti dxy	L B1 Ti fxz	L B1 Ti fyy	L B1 Ti fzz	L B1 Ti g310	L B1 Ti g130
alpha	0.9999		0.0565	0.6691	0.0023	0.0023	-0.0014	0.0002	0.0002
beta	0.0001		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Gross		L B1 Ti g112	L B1 Cl s	L B1 Cl px	L B1 Cl py	L B1 Cl pz	L B1 Cl dxx	L B1 Cl dxy	L B1 Cl dxz
alpha		-0.0017	0.0048	0.0107	0.0107	0.2054	0.0040	0.0012	0.0091
beta		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Gross		L B1 Cl dyy	L B1 Cl dyz	L B1 Cl dzz	L B1 Cl fxxx	L B1 Cl fxxz	L B1 Cl fxyz	L B1 Cl fyyy	L B1 Cl fyyz
alpha		0.0040	0.0091	0.0122	0.0004	0.0004	0.0004	0.0004	0.0004
beta		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Gross		L B1 Cl fzzz							
alpha		-0.0007							
beta		0.0000							

* Electronic eigenvalue no. 50: -0.1090420938994 (Occupation : f = 0.0000) sym= B3

* Gross populations greater than 0.00010

Gross	Total		L B3 Ti px	L B3 Ti dyz	L B3 Ti fxxx	L B3 Ti fxyy	L B3 Ti fxzz	L B3 Ti g211	L B3 Ti g031
alpha	0.0001		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
beta	0.9999		0.0565	0.6691	-0.0014	0.0023	0.0023	-0.0017	0.0002
Gross		L B3 Ti g013	L B3 Cl s	L B3 Cl px	L B3 Cl py	L B3 Cl pz	L B3 Cl dxx	L B3 Cl dxy	L B3 Cl dxz
alpha		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
beta		0.0002	0.0048	0.2054	0.0107	0.0107	0.0122	0.0091	0.0091
Gross		L B3 Cl dyy	L B3 Cl dyz	L B3 Cl dzz	L B3 Cl fxxx	L B3 Cl fxyy	L B3 Cl fxyz	L B3 Cl fxzz	L B3 Cl fyyy
alpha		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
beta		0.0040	0.0012	0.0040	-0.0007	0.0004	0.0004	0.0004	0.0004
Gross		L B3 Cl fzzz							
alpha		0.0000							
beta		0.0004							

we find that the orbitals 46 and 47 span E^- , whereas orbitals 48, 49 and 50 span T_2^- .

We are now ready to consider the simulation of ligand K-edge X-ray spectroscopy. We consider excitations from 4 occupied orbitals to 5 virtual ones. Each excitation gives rise to a singlet and a triplet, so there will be in all $4 \times 5 \times (1+3) = 80$ excitations. The singlets are totally symmetric (A_1), whereas the triplets span the rotations (T_1). From a T_d direct product table (see for instance [here](#)) we find the symmetries of the singlet excitations to be

$$\begin{aligned} & \begin{array}{l} \text{\texttt{\textbackslash begin{aligned}}} \\ \text{\texttt{\textbackslash begin{array}{lcll}}} T_2 \text{\texttt{\textbackslash rightarrow}} E : \text{\texttt{\textbackslash quad}} T_1 \text{\texttt{\textbackslash oplus}} T_2 \text{\texttt{\textbackslash T_2}} \text{\texttt{\textbackslash rightarrow}} T_2 : \text{\texttt{\textbackslash quad}} A_1 \text{\texttt{\textbackslash oplus}} E \text{\texttt{\textbackslash oplus}} T_1 \text{\texttt{\textbackslash oplus}} T_2 \text{\texttt{\textbackslash A_1}} \text{\texttt{\textbackslash rightarrow}} E : \text{\texttt{\textbackslash quad}} \\ \text{\texttt{\textbackslash end{aligned}}} \end{array} \end{aligned}$$

For triplet excitations we get

$$\begin{aligned} & \begin{array}{l} \text{\texttt{\textbackslash begin{aligned}}} \\ \text{\texttt{\textbackslash begin{array}{lcll}}} T_2 \text{\texttt{\textbackslash rightarrow}} E : \text{\texttt{\textbackslash quad}} A_1 \text{\texttt{\textbackslash oplus}} A_2 \text{\texttt{\textbackslash oplus}} 2E \text{\texttt{\textbackslash oplus}} 2T_1 \text{\texttt{\textbackslash oplus}} 2T_2 \text{\texttt{\textbackslash T_2}} \text{\texttt{\textbackslash rightarrow}} T_2 : \text{\texttt{\textbackslash quad}} A_1 \text{\texttt{\textbackslash oplus}} A_2 \text{\texttt{\textbackslash oplus}} 2E \text{\texttt{\textbackslash opl}} \\ \text{\texttt{\textbackslash end{aligned}}} \end{array} \end{aligned}$$

Translated to D_2 we find that we need 20 excitations per symmetry and so we set up the following input

TiCl4_exc.inp

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dft/bsse.md

orphan

Basis set superposition error in the DFT

The basis set superposition error (BSSE, also known as counterpoise correction, see Boys:1970) can be estimated by performing subsystem calculations using ghost atoms which carry basis sets of the full system.

This tutorial describes how to use ghost atoms in DIRAC and how to avoid certain pitfalls when working with ghost centers. As an example we shall consider the helium dimer, having linear symmetry $D_{\infty h}$. For the counterpoise correction calculation one beryllium atom is replaced by a ghost atom without charge, but carrying the full beryllium basis. The symmetry is now reduced to $C_{\infty v}$. DIRAC can automatically detect and exploit the linear symmetry, thus providing computational savings. In the case of DFT there is, however, a complication: The ghost center needs not only the basis of a beryllium atom, but also the same numerical grid. This will force us to reduce the symmetry of the molecular calculation to $C_{\infty v}$. When carrying out the counterpoise correction we shall furthermore have to make sure that atoms are in the same place as in the molecular calculation. Below we see how to handle these issues, first with the use of xyz-files, next using mol-files.

Another matter is that conventional DFT based on local functionals is not a good choice for the description of a van der Waals complex like the helium dimer. We shall therefore combine short-range DFT with MP2.

Using mol-files

We start with the molecular calculation. Without consideration of counterpoise we would use

He2.mol

However, DIRAC will not detect the full molecular symmetry $D_{\infty h}$, which is different from the counterpoise system and so the same numerical grid can not be used. We therefore add a ghost center, with no basis, to break inversion symmetry.

He2Gh.mol

The input file for our calculation combining MP2 with short-range LDA reads

MP2srLDA.inp

Note a final touch: We use the keyword NORTSD to avoid that our molecule is moved (placing origin at center of mass) during symmetry detection. We run the molecular calculation using:

```
pam --inp=MP2srLDA --mol=He2Gh --get=numerical_grid
```

Note that we save the file containing the numerical grid. Naming the ghost center 'Gh' assures that no grid is generated for it, as can be seen from the output:

Atom	Deg	Rmin	Rmax	Step size	#rp	#tp
He1	1	0.998E-05	0.157E+02	0.123E+00	99	8552
He2	1	0.998E-05	0.157E+02	0.123E+00	99	8552

We now turn to the counterpoise calculation. We replace one helium atom by a ghost atom, but this time we want a helium basis set for it. We therefore specify

HeGh.mol

DIRAC uses nuclear charge to find basis sets from libraries so we have to add "Q=2.0" for the ghost atom in the mol-file to provide this information. Note also that we again call the ghost atom 'Gh', but it will get a grid since we import it from the previous calculation. The corresponding menu file reads

MP2srLDA_cp.inp

We have adjusted the electron number to that of the atom and also ask for the import of the numerical grid. We run the calculation using:

```
pam --inp=MP2srLDA_cp --mol=HeGh --put=numerical_grid
```

Using xyz-files

write me...

Symmetry recognition

If you leave the symmetry text in the molecule specification blank, DIRAC will try to evaluate the symmetry of the system automatically. Note that the symmetry recognition uses the atom types and coordinates but not the respective basis sets. This means that if you use ghost atoms with different basis sets, you should give the symmetry explicitly or check the symmetry recognized by DIRAC.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dft/cam.md

orphan

CAM functional

In DIRAC, you can calculate with the CAMB3LYP functional using:

```
.DFT
CAMB3LYP
```

It is also possible to change the default parameters. The following line will reproduce the above functional:

```
.DFT
CAM p:alpha=0.19 p:beta=0.46 p:mu=0.33 x:slater=1 x:becke=1 c:lyp=0.81 c:vwn5=0.19
```

but it will give you complete freedom over the parameters. As always, please verify your results carefully when changing these parameters.

Approaching the B3LYP limit

CAM uses the following partitioning of the two-electron interaction:

$$\frac{1}{r_{12}} = \frac{1 - [\alpha + \beta \operatorname{erf}(\mu r_{12})]}{[\alpha + \beta \operatorname{erf}(\mu r_{12})]} + \frac{[\alpha + \beta \operatorname{erf}(\mu r_{12})]}{r_{12}}$$

For testing purposes we can try to approach the B3LYP limit using the CAM code, in order to check that the alpha/beta limits work well.

In B3LYP, "HF" admixture is 0.2 so in CAM this can be obtained with alpha=0.2 and beta=0.0.

So this is B3LYP:

```
.DFT
GGAKEY Slater=0.8 Becke=0.72 HF=0.2 LYP=0.81 VWN=0.19
```

Now naively we could try to obtain B3LYP results like this (**this is wrong**):

```
.DFT
CAM p:alpha=0.2 p:beta=0.0 p:mu=0.0 x:slater=0.8 x:becke=0.72 c:lyp=0.81 c:vwn5=0.19
```

This does not work in DIRAC and the correct B3LYP expressed using CAM in DIRAC can be obtained like this:

```
.DFT
CAM p:alpha=0.2 p:beta=0.0 p:mu=0.0 x:slater=1.0 x:becke=0.9 c:lyp=0.81 c:vnw5=0.19
```

This is because x:slater and x:becke get scaled inside fun-cam.c by (1-alpha). The x:becke=0.9 can be surprising if you read the paper by Yanai et al. [Chem. Phys. Lett. 393 (2004) 51] and follow their examples.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dft/tddft_1score_N2.md

orphan

1s core-ionization of N2 by TD-DFT

Introduction

We want to study the excitation of a 1s electron of the N_2 molecule to an empty orbital. More precisely we shall look at the excitation of an electron from the bonding $1s\sigma_g$ or anti-bonding $1s\sigma_u$ orbitals to the vacant $2p\pi_g$ or $2p\sigma_u$ orbitals (see MO-diagram below).



Note that this diagram does not take spin-orbit into account, but we shall consider this interaction later on. Let us first consider the possible final states. One electron leaves from one of four spin-orbitals and enters one of six spin-orbitals. This gives 24 determinants which translates into the following states:

Configuration	States
$1s\sigma_g^{-1}2p\pi_g$	$^1,3\Pi_g$
$1s\sigma_g^{-1}2p\sigma_u$	$^1,3\Sigma_u$
$1s\sigma_u^{-1}2p\pi_g$	$^1,3\Pi_u$
$1s\sigma_u^{-1}2p\sigma_u$	$^1,3\Sigma_g$

Spin-orbit free calculation

Preparing the input files

We employ the following molecular input file N2.mol

N2.mol

Here we do not provide any symmetry information, meaning that we ask DIRAC to detect it. DIRAC will find that the full group is $D_{\infty h}$. With spin-orbit coupling DIRAC will then activate linear supersymmetry, but in the spin-orbit free case it will simply use the highest Abelian single point group, that is D_{2h} . For the final states DIRAC will employ the *total* symmetry, that is the combined spin and spatial symmetry. Here we shall keep in mind that the singlet spin function is totally symmetric (A_g), whereas the triplet spin functions transform as rotations. We know the triplet functions as:

$$T_{-1} = \alpha_1\alpha_2; \quad T_0 = \frac{1}{\sqrt{2}}(\alpha_1\beta_2 - \beta_1\alpha_2); \quad T_{+1} = \beta_1\beta_2$$

but for our purposes it will be more convenient to form the combinations:

$$T_x = \frac{1}{\sqrt{2}}(T_{-1} - T_{+1}); \quad T_y = \frac{1}{\sqrt{2}}(T_{-1} + T_{+1}); \quad T_z = T_0$$

which transform as rotations $R_x(B_{3g})$, $R_y(B_{2g})$ and $R_z(B_{1g})$, respectively.

We can now set up the following correlation of states:

State	Spin	Spatial	Spin $\otimes$ Spatial
$^1\Sigma_g$	A_g	A_g	A_g
$^1\Sigma_u$	A_g	B_{1u}	B_{1u}
$^1\Pi_{x,y,g}$	A_g	B_{3g}, B_{2g}	B_{3g}, B_{2g}
$^1\Pi_{x,y,u}$	A_g	B_{3u}, B_{2u}	B_{3u}, B_{2u}
$^3\Sigma_g$	B_{3g}, B_{2g}, B_{1g}	A_g	B_{3g}, B_{2g}, B_{1g}
$^3\Sigma_u$	B_{3g}, B_{2g}, B_{1g}	B_{1u}	B_{2u}, B_{3u}, A_u
$^3\Pi_{x,y,g}$	B_{3g}, B_{2g}, B_{1g}	B_{3g}, B_{2g}	$(A_g, B_{1g}, B_{2g}), (B_{1g}, A_g, B_{3g})$
$^3\Pi_{x,y,u}$	B_{3g}, B_{2g}, B_{1g}	B_{3u}, B_{2u}	$(A_u, B_{1u}, B_{2u}), (B_{1u}, A_u, B_{3u})$

Counting total symmetries we find the 24 microstates are evenly distributed amongst the eight irreps of D_{2h} :

Irrep	Core-ionized state
A_g	$\Sigma_g^+ \Pi_{xg}^3 \Pi_{yg}^3$ x^2, y^2, z^2
B_{3u}	$\Pi_{xu}^1 \Pi_{yu}^3 \Sigma_u^+ \Pi_{zu}^3$ x
B_{2u}	$\Pi_{yu}^1 \Pi_{xu}^3 \Sigma_u^+ \Pi_{zu}^3$ y
B_{1g}	$\Sigma_g^+ \Pi_{xg}^3 \Pi_{yg}^3$ xy
B_{1u}	$\Sigma_u^+ \Pi_{xu}^3 \Pi_{yu}^3$ z
B_{2g}	$\Pi_{yg}^1 \Pi_{xg}^3 \Sigma_g^+ \Pi_{zg}^3$ xz
B_{3g}	$\Pi_{xg}^1 \Pi_{xg}^3 \Sigma_g^+ \Pi_{yg}^3$ yz
A_u	$\Sigma_u^+ \Pi_{xu}^3 \Pi_{yu}^3$ xyz

From these considerations we now set up the following menu file for our calculation

N2spf.inp

In the *SCF section we give the electron occupation of N_2 : 6 and 8 electrons in *gerade* and *ungerade* orbitals, respectively. We also ask for a Mulliken population analysis (ANALYZE_.MULPOP) for the occupied orbitals and the orbitals involved in the core excitation.

Let us now look at how we set up the calculation of excitation energies under *EXCITATION ENERGIES. We have seen that there are three excitations per boson irrep. Note that the numbering of irreps follow what you for instance find in the D_{2h} direct product table in the output:

	Ag	B3u	B2u	B1g	B1u	B2g	B3g	Au
Ag	Ag	B3u	B2u	B1g	B1u	B2g	B3g	Au
B3u	B3u	Ag	B1g	B2u	B2g	B1u	Au	B3g
B2u	B2u	B1g	Ag	B3u	B3g	Au	B1u	B2g
B1g	B1g	B2u	B3u	Ag	Au	B3g	B2g	B1u
B1u	B1u	B2g	B3g	Au	Ag	B3u	B2u	B1g
B2g	B2g	B1u	Au	B3g	B3u	Ag	B1g	B2u
B3g	B3g	Au	B1u	B2g	B2u	B1g	Ag	B3u
Au	Au	B3g	B2g	B1u	B1g	B2u	B3u	Ag

Note also that we skip excitations in B_{2u} and B_{2g} , since they are related by symmetry to the excitations of B_{3u} and B_{3g} , respectively.

If nothing further is specified the excitation energies are calculated by a “bottoms-up” approach and so we will get valence excitations only, since the core-excitations are much higher in energy. We therefore restrict the excitations to the occupied $1s\sigma_g$ and $1s\sigma_u$ orbitals.

We furthermore ask for transition moments to be calculated with respect to the component of the dipole moment operator. These will be non-zero only for excitations in irreps B_{3u} , B_{2u} and B_{1u} . Finally we ask for analysis of what orbitals contribute to the various excitations. For this the Mulliken population analysis may come in handy as reference.

Looking at the output

After running the calculation, let us now look at the output. The following excitation energies were calculated

N2spf_exc.txt

DIRAC assumes that excitation energies that are within $10^{-9} E_h$ of each other come from the same degenerate state. This threshold is somewhat arbitrary and we shall see that DIRAC is not always correct.

There is sufficient symmetry in the calculation (symmetry distinct rotation) to allow DIRAC to pinpoint the symmetry of the core-ionized state and we therefore find the following distribution

N2spf_exc2.txt

The first and second block refers to singlet and triplet states, respectively. Based on the discussion in the preceeding section we see that levels 1, 2 and 3 all come from a Π_u^3 which in D_{2h} splits into B_{2u}^3 and B_{3u}^3 . After careful inspection we can set up the following table

Level	eigenvalue (eV)	
0	0.000	$1\Sigma_g$
1,2,3	388.780	$3\Pi_u$
4,5,6	388.831	$3\Pi_g$
7	389.916	$1\Pi_u$
8	389.936	$1\Pi_g$
9,10	399.834	$3\Sigma_g$
11,12	399.876	$3\Sigma_u$
13	400.519	$1\Sigma_g$
14	400.568	$1\Sigma_u$

Looking further down in the output we find dominant inactive and virtual orbitals. Restricting attention to B_{3u} total symmetry we find that the first excited state $^3\Pi_u$, at 388.78 eV, is dominated by the excitation $1(i:E1u) \rightarrow 4(v:E1g)$, which, as can be inferred from the Mulliken population analysis, corresponds to $1s\sigma_u \rightarrow 2p\pi_{y;g}$. The second excited state $^1\Pi_u$, at 389.91 eV, corresponds to $1(i:E1u) \rightarrow 5(v:E1g)$ ($1s\sigma_u \rightarrow 2p\pi_{x;g}$), whereas the third excited state $^3\Sigma_u$, at 399.88 eV, is dominated by $1(i:E1g) \rightarrow 5(v:E1u)$ ($1s\sigma_g \rightarrow 2p\sigma_u$).

Within the electric dipole approximation only singlet states get oscillator strengths. In the output we find

N2spf_osc.txt

showing intensity to the $^1\Pi_u$ and $^1\Sigma_u$ states.

Including spin-orbit

Spin-orbit is included by simply commenting out the keyword HAMILTONIAN_.SPINFREE in the input above:

**HAMILTONIAN
!.SPINFREE

This leads to the following states:

N2so_exc.txt

with the following distribution on linear symmetries:

N2so_exc2.txt

Comparing with the preceeding section we see the following spin-orbit decomposition of the Λ -S states:

Level	eigenvalue (eV)		
0	0.000	$^1\Sigma_g^+$	$0_g^+ (0.000)$
1,2,3	388.780	$^3\Pi_u$	$0_u^+ (388.771), 0_u^- (388.771), 1_u (388.780), 2_u (388.788)$
4,5,6	388.831	$^3\Pi_g$	$0_g^+ (388.821), 0_g^- (388.822), 1_g (388.830), 2_g (388.839)$
7	389.916	$^1\Pi_u$	$1_u (389.916)$
8	389.936	$^1\Pi_g$	$1_g (389.936)$
9,10	399.834	$^3\Sigma_g^+$	$0_g^- (399.834), 1_g (399.834)$
11,12	399.876	$^3\Sigma_u^+$	$0_u^- (399.876), 1_u (399.876)$
13	400.519	$^1\Sigma_g^+$	$0_g^+ (400.519)$
14	400.568	$^1\Sigma_u^+$	$0_u^+ (400.568)$

Note that the energies are given relative to the lowest level, that is, the ground state and that it is somewhat stabilized by spin-orbit coupling.

The effect of spin-orbit coupling shows up in the oscillator strengths:

N2so_osc.txt

What we see is the $^0_u^+$ and 1_u components of the $^3\Pi_u$ state stealing intensity from the singlet states. This change is not very spectacular since the nitrogen molecule is composed of light atoms for which relativistic effects are not very strong. We can mimic a more strongly relativistic system by reducing the speed of light to e.g. 20 a.u.:

**GENERAL
.CVALUE
20.0D0

We now see

N2so20_osc.txt

Simulating the core-excitation spectrum using complex response

[Gas Phase Core Excitation Database](#)

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dft/tddft.md

orphan

A TDDFT study of carbon monoxide

Introduction

To illustrate the TDDFT capabilities of DIRAC we will consider excited states of the carbon monoxide molecule. The experimental spectrum can be found [at this page](#) . Note that this source gives only adiabatic excitation energies T_e , that is, the energy difference between the minima of the potential curve of the ground and excited state, whereas the TDDFT calculations give vertical excitation energies T_v . The experimental adiabatic excitation energies has been converted to vertical ones by Nielsen_JCP1980. We will begin this tutorial in spinfree mode to facilitate the connection to the non-relativistic domain. In the following we employ the molecular input file CO.mol (download <CO.mol>):

CO.mol

Here we do not provide any symmetry information, meaning that we ask DIRAC to detect it. DIRAC will find that the full group is $C_{\infty v}$. With the spin-orbit coupling DIRAC will then activate the linear supersymmetry, but in the spin-orbit free case it will simply use the highest Abelian single point group, that is C_{2v} . Irreps of $C_{\infty v}$ and C_{2v} correlate as follows:

$C_{\infty v}$ C_{2v}
 Σ^+ A_1
 Σ^- A_2
 Π B_1, B_2
 Δ A_1, A_2

Ground state electronic structure

Before actually doing TDDFT let us first have a look at the electronic structure of the ground state of CO. We therefore carry out Kohn-Sham DFT (see *DFT) using the PBE functional and the eXact 2-Component (HAMILTONIAN_X2C) Hamiltonian followed by Mulliken population analysis using the input file SCF.inp (download <SCF.inp>):

SCF.inp

Based on orbital energies and the Mulliken population analysis we can set up the following MO diagram:

COModiagram

(Orbital energies for the atoms were obtained from atomic calculations using fractional occupation)

According to the above MO diagram the valence electron configuration of carbon monoxide is $1\sigma^2 2\sigma^2 1\pi^4 3\sigma^2$. From the projection analysis we find that the electron configurations of the carbon and oxygen atoms in the molecule are $1s^2 2s^2 2p^2$ and $1s^2 2s^2 1.7 2p^{4.3}$, respectively, with a slight negative charge of -0.3 on oxygen. We note that the HOMO is a σ orbital dominated (~90 %) by carbon (2s,2p), whereas the doubly degenerate LUMO is a π^* orbital with about 75% C 2p and 25% O 2p. The LUMO+1 is σ^* orbital with a combination of carbon and oxygen 3s dominated by the former atom.

Excited states

Preparing the input files

The excited states will be calculated in a “bottoms-up” fashion for each irrep by expansion of the solution vector in trial vector Bast2009. For each irrep the user specifies the number of desired excitations. It may be a good idea of including some extra excitations in case of root flipping during the iterative solution of the TDDFT equations. We are working within the adiabatic approximation of TDDFT and consider single excitations from a closed-shell ground state. The final states will be therefore be singlet or triplets in the spin-orbit free case. We need to tell DIRAC how these states are distributed on the four irreps of C_{2v} . At this point we should emphasize that DIRAC will employ the *total* symmetry of the final states, that is the combined spin and spatial symmetry. A singlet spin function

$$S_0 = \frac{1}{\sqrt{2}} \left(\alpha_1 \beta_2 - \beta_1 \alpha_2 \right)$$

is totally symmetric (A_1), whereas the triplet spin functions transform as rotations. We know the triplet functions as:

$$T_{-1} = \alpha_1 \alpha_2; \quad T_0 = \frac{1}{\sqrt{2}} \left(\alpha_1 \beta_2 + \beta_1 \alpha_2 \right); \quad T_{+1} = \beta_1 \beta_2$$

but for our purposes it will be more convenient to form the combinations:

$$T_x = \frac{1}{\sqrt{2}} \left(T_{-1} - T_{+1} \right); \quad T_y = \frac{1}{\sqrt{2}} \left(T_{-1} + T_{+1} \right); \quad T_z = T_0$$

which transform as rotations R_x (B₂), R_y (B₁) and R_z (A₂), respectively. For a given point group you find the symmetry of the rotations in most standard character tables (for instance [here](#)).

As an example of how to determine total symmetry of the final states let us consider single excitations HOMO into the LUMO (π^*) and LUMO+1 (σ^*) orbitals. This leads to $2 \times 6 = 12$ determinants which translate into the following final states:

Configuration	States
$3\sigma^{-1} 2\pi^1$	$^1\pi$
$3\sigma^{-1} 4\sigma^1$	$^1\sigma$

We can now set up the following correlation of states:

State	Spin	Spatial	Spin $\otimes$ Spatial
$^1\Sigma$	A_1	A_1	A_1
$^1\Pi_{x,y}$	A_1	B_1, B_2	B_1, B_2
$^3\Sigma$	B_2, B_1, A_2	A_1	B_2, B_1, A_2

State	Spin	Spatial	Spin $\otimes$ Spatial
${}^3\Pi_{x,y}$	B_2, B_1, A_2	B_1, B_2	$(A_2, A_1, B_2), (A_1, A_2, B_2)$

(The group multiplication table is found in the DIRAC output.)

Counting total symmetries we find that the 12 microstates are evenly distributed amongst the four irreps of C_{2v} :

Irrep	Valence-excited state
1 A_1	${}^1\Sigma, {}^3(y)\Pi_x, {}^3(x)\Pi_y$
2 B_1	${}^1\Pi_x, {}^3(y)\Sigma, {}^3(z)\Pi_y$
3 B_2	${}^1\Pi_y, {}^3(x)\Sigma, {}^3(z)\Pi_x$
4 A_2	${}^3(z)\Sigma, {}^3(x)\Pi_x, {}^3(y)\Pi_y$

Note that the order of the irreps follows the output of DIRAC.

We now set up the following menu file TDDFT.inp (download <TDDFT.inp>) for our calculation

TDDFT.inp

We base our TDDFT calculation on the long-range corrected CAMB3LYP functional Yanai_CPL2004 which provides a better description of Rydberg and charge-transfer excitations than standard continuum functionals. We furthermore ask for oscillator strengths within the electric dipole approximation. Finally we ask for analysis of what orbitals contribute to the various excitations. For this the Mulliken population analysis may come in handy as reference.

Looking at the output

Energy levels

After running the calculation, let us now look at the output. There is output for each symmetry, but we go directly to the final output at the end where we first find a list of levels

TDDFT_CO_spf_exc1

The degeneracies indicated in parenthesis for each level generally arise from both spatial and spin degrees of freedom. DIRAC assumes degeneracy when energies are within $1.0 \times 10^{-9} E_h$. As we shall see this is not a foolproof scheme.

To learn more about the nature of the excited states we look at the following list of energy levels, now in other units (eV and cm^{-1}) and with symmetry information:

TDDFT_CO_spf_exc2

The first level is simply the ground state. The reference state for calculation excitation energies is presently limited to closed-shell and so this is 1A_1 (C_{2v}), corresponding to ${}^1\Sigma^+$ of $C_{\infty v}$. Linear (super)symmetry is not implemented for spin-orbit free calculations, so DIRAC gives symmetry information with respect to C_{2v} . DIRAC, however, is able to separate spin and spatial symmetries. We see for instance the next level is six-fold degenerate with equal contributions from 3B_1 and 3B_2 . Looking at the above correspondence table between C_{2v} and $C_{\infty v}$ that a degenerate level of B_1 and B_2 spatial symmetries corresponds to a Π term, so we identify the first excited level of CO as ${}^3\Pi$. The correspondence table also allows us to distinguish mirror symmetry about the molecular axis for Σ states, so we identify the second excited state as ${}^3\Sigma^+$. Proceeding in this manner, we also see some failures of the degeneracy detection in DIRAC. Level 4 and 5 are classified as 3A_1 and 3A_2 , respectively, and are clearly components of a ${}^3\Delta$ state. Reducing the threshold for degeneracies would bring them together, as well as level 7 (1A_2) and 8 (1A_1), which are component of a ${}^1\Delta$ state. However, level 6 is seen to contain both a singlet 1A_2 (${}^1\Sigma^-$) and a triplet 3A_2 (${}^3\Sigma^-$) which are clearly not allowed to mix without spin-orbit interaction, so here a *larger* threshold would be needed to distinguish them.

Below we give a summary of results obtained with a variety of functionals available in DIRAC. The assignment and energy of states may be compared to Tozer_JCP1998.

	T_e	T_v	CAMB3LYP	LDA	PBE	PBE0	BLYP	B3LYP
$a\ ^3\Pi$	6.04	6.32	5.93	5.98	5.77	5.76	5.85	5.88
$a\ ^3\Sigma^+$	6.92	8.51	7.95	8.43	8.12	7.85	8.10	7.93
$A\ ^1\Pi$	8.07	8.51	8.49	8.19	8.26	8.45	8.25	8.41
$d\ ^3\Delta$	7.58	9.36	8.70	9.21	8.78	8.64	8.72	8.66
$e\ ^3\Sigma^-$	7.96	9.88	9.72	9.89	9.87	9.79	9.79	9.73
$f\ ^1\Sigma^-$	8.07	9.88	9.72	9.89	9.87	9.79	9.79	9.73
$D\ ^1\Delta$	8.17	10.23	10.11	10.36	10.21	10.21	10.03	10.05
$b\ ^3\Sigma^+$	10.39	10.4	10.27	9.59	9.35	10.12	9.29	9.95
$B\ ^1\Sigma^+$	10.78	10.78	10.90	9.95	9.78	10.72	9.65	10.43
$j\ ^3\Sigma^+$	11.28	11.3	10.27	10.37	10.09	10.84	10.09	10.71
$C\ ^1\Sigma^+$	11.40	11.4	11.29	10.68	10.56	11.25	10.49	11.05
$E\ ^1\Pi$	11.52	11.53	11.46	10.70	10.58	11.36	10.49	11.13
$c\ ^3\Pi$	11.55	11.55	11.31	10.31	10.31	11.12	10.30	10.97
MAE (all states)			0.29	0.49	0.62	0.29	0.68	0.39
MAE (valence)			0.30	0.15	0.26	0.31	0.31	0.33
MAE (Rydberg)			0.29	0.89	1.05	0.26	1.11	0.46

For all functionals we provide the mean absolute error (MAE), and it can indeed be seen that the performance of CAMB3LYP is quite good. PBE0 is also display nice performance, whereas its GGA equivalent comes out significantly worse. However, if we do separate error analysis of the valence states (the first seven states) and the

Rydberg states (the last six states), we see that PBE performs well for the valence states, but not for the Rydberg states. This is connected to the wrong asymptotic behaviour of LDA/GGA functionals.

The performance of standard LDA/GGA functionals can be improved by invoking asymptotic corrections. One option is the statistical average of orbital potentials (SAOP). Below we show the result of SAOP-corrected functionals.

SAOP-corrected	T_e	T_v	SAOP	LDA	PBE	PBE0	BLYP	B3LYP
$a\{3\}Pi$	6.04	6.32	6.33	6.11	5.76	5.76	5.84	5.87
$a\{3\}Sigma^{+}$	6.92	8.51	8.65	8.47	7.88	7.88	8.19	8.00
$A\{1\}Pi$	8.07	8.51	8.60	8.47	8.57	8.57	8.61	8.64
$d\{3\}Delta$	7.58	9.36	9.39	9.30	8.64	8.64	8.79	8.71
$e\{3\}Sigma^{-}$	7.96	9.88	10.04	10.00	9.77	9.77	9.84	9.76
$E\{1\}Sigma^{-}$	8.07	9.88	10.04	10.00	9.77	9.77	9.84	9.76
$D\{1\}Delta$	8.17	10.23	10.50	10.49	10.27	10.27	10.39	10.31
$b\{3\}Sigma^{+}$	10.39	10.4	10.48	10.87	10.49	10.49	10.21	10.52
$B\{1\}Sigma^{+}$	10.78	10.78	10.97	11.25	11.21	11.21	10.83	11.22
$j\{3\}Sigma^{+}$	11.28	11.3	11.42	11.70	11.34	11.34	11.20	11.43
$C\{1\}Sigma^{+}$	11.40	11.4	11.73	11.97	11.80	11.80		
$E\{1\}Pi$	11.52	11.53	11.89		11.97	11.97		
$c\{3\}Pi$	11.55	11.55	11.76	11.73	11.68	11.68	11.42	11.70
MAE (all states)			0.17	0.24	0.29	0.29	0.20	0.26
MAE (valence)			0.12	0.12	0.32	0.32	0.24	0.29
MAE (Rydberg)			0.21	0.42	0.25	0.25	0.12	0.21

Orbital contributions

We can also get information about dominant (single) excitations leading to each excited state. We then go to the section starting with:

Analysis of response solution vectors

For the first excitation of A_1 total (space+spin) symmetry we find for example

TDDFT_CO_spf_exc3

This is an excitation to the first Δ state, combining a T_y spin function with B_1 spatial symmetry. The dominant inactive orbital is orbital 7 and if we look at the Mulliken population analysis we find

TDDFT_CO_spf_exc4

showing that this is the HOMO, which is a σ orbital with 51 % carbon *s* and 36 % carbon *p* character. The dominant virtual orbital is orbital 8

TDDFT_CO_spf_exc5

which is clearly a π orbital dominated by carbon p_x .

Oscillator strenghts

DIRAC also allows provides oscillator strenghts corresponding to the calculated (single) excitations. The general definition of an oscillator strenght is

$$f_{\{0n\}} = \frac{2m}{e^2\hbar\omega_{\{0n\}}} \left| T_{\{0n\}} \right|^2$$

where $\hbar\omega_{\{0n\}}$ and $T_{\{0n\}}$ are the corresponding excitation energy and transition moment, respectively. The calculation of oscillator strenghts is invoked through:

.INTENS
0

which means that we ask for oscillator strenghts to order zero in the wave vector. This corresponds to the electric dipole approximation. Starting from DIRAC21, oscillator strenghts have been implemented to *arbitrary* order in the wave vector, in both the length and velocity representation, see List_JCP2020. In addition DIRAC also allows the use of the *full* semi-classical light-matter interaction. Presently, we limit ourselves to the electric-dipole approximation. The isotropic dipole length-dipole length oscillator strenghts are given as

$$f^{(\{0\}iso)}_{\{0n\}} = \frac{2m}{3e^2\hbar\omega_{\{0n\}}} \left| \mu_{\{0n\}} \right|^2$$

and are found to be

TDDFT_CO_spf_exc6

Corresponding rates of spontaneous emission are given by

$$A_{\{0n\}} = \frac{2e^2\omega_{\{0n\}}^2}{4\pi\epsilon_0 mc^3} f_{\{0n\}} .$$

This is a spin-orbit free calculation and so we only see oscillator strength to singlet states (clearly dominated by $C\{1\}Sigma^{+}$). Things change is we activate spin-orbit interaction (that is, remove the HAMILTONIAN_.SPINFREE keyword). We see that the energy levels split up

TDDFT_CO_exc

and that oscillator strength goes to more states

TDDFT_CO_exc5

reflecting that triplet states “steal” intensity from singlet ones.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dft/troubleshooting.md

orphan

Troubleshooting

WARNING: error in the number of electrons

Sometimes you may see the following warning:

```
number of grid points           = 54836
DFT exchange–correlation energy: = -230.0368431371773283
number of electrons from numerical integration = 107.9973057254599667
number of electrons from orbital occupations = 108
WARNING: error in the number of electrons = -0.0026942745400333
        is larger than 1.0d-3
        this can happen when starting from coefficients
        from a different geometry
        or it can mean that the quadrature grid is inappropriate
```

What does this mean?

When you run a Kohn–Sham (KS) density functional theory (DFT) calculation, the exchange–correlation (XC) energy and XC potential contributions are integrated numerically. This means that for KS DFT calculations the numerical quadrature grid enters as an additional parameter which you control.

In every iteration, when integrating the XC contributions, the code will also integrate the density at the same time and compare the number of electrons from numerical integration with the number of electrons from orbital occupations as a sanity check and issue the above warning if the difference is larger than a threshold (here 0.001).

If you see this warning, then the number of electrons from numerical integration deviates significantly from the target number of electrons. This is a sanity check and it does not necessarily mean that your results will be useless, since the integrated number of electrons is not a sufficient measure for the quality of the grid for the property that you calculate. If you see this warning in every iteration of your calculation, then it may be a good idea to investigate the quality of the grid. You can either select a finer (and more expensive) grid (see `**GRID`), and/or tighten the screening threshold (`DFT_.SCREENING` under `*DFT`).

When restarting from a `DFC0EF` file from a different geometry, the warning can appear during the first few iterations and then this warning disappears (because the density of the old geometry is then displaced with respect to the grid at new geometry).

If the integrated number of electrons is significantly off, then there is a serious problem, possibly a bug. In this case please contact the developers and send your output.

If you do not see the above warning, then it does not guarantee that your integration grid is useful for the property that you study. It is always a good practice to verify that your results are converged with respect to the grid parameters. When you calculate a table of functionals, it is a good practice for one of the results to re-run with the best integration grid and to check that your results do not change.

DFT nonconverging...

What to do if your DFT run is not converging ? Instead of DFT try first to perform the SCF method. If the previous SCF step does converge, save the MO-coefficient file and use them as starting set for your DFT method.

This trick can help because the HOMO-LUMO gap is higher in the SCF than in the DFT. Restarting the DFT from converged SCF coefficients makes the DFT method converging in many cases.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dmrg/integral_generation.md

orphan

Generate a FCIDUMP file for external DMRG calculations

This tutorial briefly demonstrates how to generate a FCIDUMP integral file which can be used to perform relativistic DMRG calculations with an external DMRG code. The interface and the first relativistic DMRG implementation and calculation are described in Knecht2014a.

We will explain the basic steps to generate a FCIDUMP file by looking at the HF molecule using a small cc-pVDZ basis set for each atom. The molecular input file **hf.xyz** is:

```
2
H      0.0000000000    0.0000000000    0.8547056701
F      0.0000000000    0.0000000000   -0.0453403237
```

and the corresponding wave function input file **fci.inp** is:

```

**DIRAC
.TITLE
  FCIDUMP file tutorial
.WAVE FUNCTION
**HAMILTONIAN
.DOSSSS
**WAVE FUNCTION
.SCF
.KR CI
*SCF
.CLOSED SHELL
10
*KRCI
.CI PROGRAM
LUCIAREL
.CIROOTS
0 1
.MAX CI
100
.INACTIVE
1
.GAS SHELLS
1
8 8 / 9
**MOLECULE
*BASIS
.DEFAULT
cc-pVDZ
*END OF INPUT

```

So let's have a closer look at the above wave function input. The default Hamiltonian (to be specified under ****Hamiltonian**) in Dirac is the Dirac-Coulomb Hamiltonian and we explicitly ask to also include the class of (SSISS) integrals. The wave function input asks for a self-consistent-field Hartree-Fock calculation for the closed-shell HF molecule with 10 electrons followed by a FCI/CASCI calculation for the $\Omega=0$ state using an active space of **8** electrons in **9** Kramer's pairs ($= 18$ spinors). The syntax (min max # of electrons in the CAS space / # of Kramer's pairs) reads in detail as:

```

.GAS SHELLS
1
8 8 / 9

```

Note that for this FCI/CASCI calculation we have frozen the 1s shell of F by setting:

```

.INACTIVE
1

```

To run the above example we use DIRAC's python script *pam* with the following command line:

```
$ $path-to-dirac-build-directory/pam --inp=fci.inp --mol=hf.xyz
```

A closer look at the output of the CI program at the bottom of the Dirac calculation will tell us the final FCI/CASCI energy for the $\Omega=0$ state to be **-100.1956847...** Hartree. This is the reference energy we should expect to get from the DMRG calculation on the same active space.

To obtain now the integral file **FCIDUMP** for the above FCI/CASCI example we first need to modify the above input file by introducing the keyword **FCIDUMP** and change the CI program from *LUCIAREL* to *GASCIP* for technical reasons. Save the file as **fcidump.inp** and run Dirac with the following command line:

```
$ $path-to-dirac-build-directory/pam --inp=fcidump.inp --mol=hf.xyz --get=FCIDUMP
```

The **fcidump.inp** reads as:

```

**DIRAC
.TITLE
  FCIDUMP file tutorial
.WAVE FUNCTION
**HAMILTONIAN
.DOSSSS
**WAVE FUNCTION
.SCF
.KR CI
*SCF
.CLOSED SHELL
10
*KRCI
.CI PROGRAM
GASCIP
.FCIDUMP
.CIROOTS
0 1
.MAX CI
100
.INACTIVE
1
.GAS SHELLS
1
8 8 / 9
**MOLECULE
*BASIS
.DEFAULT
cc-pVDZ
*END OF INPUT

```

Happy DMRG computing!

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dusty_attic/dirrci.md

orphan

DIRRCI:Sample inputs

Calculate the spin-orbit components of the 2P ground state of the fluorine atom using a CISD wavefunction.

The full calculation is given as input example 5.cisd.energy REF? in the test directory:

```
&RASORB NELEC=7, NRAS1=1,1, NRAS2=2*0,3,3, MAXH1=2, MAXE3=2 &END
&CIROOT IREPNA=' 1Eu', NR00TS=3 &END
&DIRECT MAXITER=15 &END
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dusty_attic/goscip.md

orphan

GOSCIP

Let us remind first that DIRAC uses the so-called averaged SCF scheme, which means that all the configurations belonging to the reference space receive the same weight.

In the case when one (or few) of the states from the reference space is (or are) more important than others, than the large open-shell space can lead to dis-balanced treatment and therefore to higher SCF energies than those obtained with a smaller reference space.

The “better” SCF configuration is that one giving the lowest total energy. Upon suitable averaged open-shell SCF occupation one can perform complete open-shell CI (COSCI) calculations, one example is given below.

Sample inputs

Calculate the spin-orbit components of the 2P ground state of the fluorine atom using a COSCI wavefunction. This will represent the “uncorrelated” result since the CI will merely give the energies of the individual determinants in this simple case.

The same calculation can also be done automatically by specifying WAVE_FUNCTION_.RESOLVE in the **WAVE FUNCTION section of the input.

The full calculation is given as input example of the *cosci_energy* test:

```
&GOSCIP NELEC=5, IPRNT=1 &END
&POPANA THRESH=1.0D-4, DEGEN=1.0D-12, SELPOP=50.0 &END
```

NOTE: In the developer’s version the first line reads as

```
&GOSCIP NELACT=5, IPRNT=1 &END
&POPANA THRESH=1.0D-4, DEGEN=1.0D-12, SELPOP=50.0 &END
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dusty_attic/lucita.md

orphan

LUCITA:Sample inputs

Perform SDCI calculations:

```
*LUCITA
.INIWFC
.DHFSCF
.CITYPE
.SDCI
.MULTIP
3
*END OF
```

Perform GASCI calculations:

```
*LUCITA
.TITLE
Aluminum atom in C2h symmetry, 7s5p2d (L) basis, SDT/SD CI (1s inactive).
.INIWFC
.OSHSCF
.CITYPE
.GASCI
```

```
.MULTIP
2
.NACTEL
13
.GASSHE
5
1,0,0,0 ! 1s
2,0,0,0 ! 2s3s
0,0,2,4 ! 2p3p
5,2,1,2 ! 4s5s 4p 3d
5,2,2,4 ! 6s7s 5p6p 4d
.GASSPC
1
2 2
3 6
11 13
11 13
13 13
*END OF
```

In addition, for we provide three LUCITA input templates with increasing complexity (lucita.novice,lucita.expert,lucita.wizard)

The user can download and modify them according to his needs.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dusty_attic/moltra.md

orphan

MOLTRA:Sample inputs

Use all spinors with energies between -12.0 and 20.0 a.u. as active space. Note that one may *not* place a comment line directly after .ACTIVE:

```
**MOLTRA
.ACTIVE
energy -12.0 20.0 1.0
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dusty_attic/mp2.md

orphan

MP2 sample inputs

Valence + subvalence γ calculation. The 19 active occupied Kramers pairs are split in two and the calculation is done in three batches (1..10+1..10, 11..19+1..10, and 11..19+11..19 for I+J). All orbitals with energy above 100 hartree (a.u.) are neglected in the virtual space. No SS integrals are included. Note that the orbitals in the active space need not be consecutive.

```
.NELECT
80 78
.....
*MP2CAL
.OCCUP
24,32..40
24,32..39
.VIRTUAL
all
all
.VIRTHR
100.0
.INTFLG
1 1 0
.IJTSK
10
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/dusty_attic/open_shells.md

orphan

Examples of open-shell systems

Here we provide few examples of open-shell systems.

Boron atom: open-shell SCF and small CI relativistic calculations

An example of one electron in two spinors is the Boron atom.

We seek to get the first excited state, $2P_{3/2}$, of B (X $2P_{1/2}$), and find out the $2P_{1/2}$ - $2P_{3/2}$ fine structure splitting.

We select the Dirac-Coulomb Hamiltonian and neglect the (SSIS) class of integrals.

Complete open-shell CI

Possible way to get the value is performing small CI over the active open-shell space, containing six 2p orbitals, which are split due to the spin-orbital interaction. Note, that for pedagogical purposes we have employed several computational symmetries.

Starting from the average SCF occupation (one electron distributed over $2p_{1/2}$ and $2p_{3/2}$ shells),

```
pam --mol=B.sto-3g.lsym.mol -inp=B.2Pav.dc_rkb.scf_2fs_cosci.inp
```

```
pam --mol=B.sto-3g.D2h.mol --inp=B.2Pav.dc_rkb.scf_2fs_cosci.inp
```

for one-fermion symmetries,

```
pam --mol=B.sto-3g.C2v.mol --inp=B.2Pav.dc_rkb.scf_1fs_cosci.inp
```

we get the value of 17.179488 cm⁻¹ and in the outputs we see the proper degeneracy of resulting total CI energies - 2 and 4.

By averaging the COSCI energies of the ground and the states we get the total SCF energy of the average occupation

```
((2*(-24.249116730099) + 4*(-24.249038454591)))/6 = -24.249064546427
SCF energy: -24.249064546425263
```

MO reordering

To apply the MO reordering scheme, first, one has to obtain the MO coefficients file of the ground state:

```
pam --outcmo --mol=B.sto-3g.D2h.mol --inp=B.2P12.dc_rkb.scf_2fs.inp
```

```
pam --outcmo --mol=B.sto-3g.C1.mol --inp=B.2P12.dc_rkb.scf_1fs.inp
```

and use them for calculation of the excited state:

```
pam --incmo --mol=B.sto-3g.D2h.mol --inp=B.2P32.dc_rkb.scf_2fs_reord_ovlssel.inp
```

```
pam --incmo --mol=B.sto-3g.C1.mol --inp=B.2P32.dc_rkb.scf_1fs_reord_ovlssel.inp
```

For the linear symmetry one can employ the M_J specified occupation of shells both for the ground and the first excited state. Only the preDHF reordering works in this case, not the overlap selection.

```
pam --outcmo --mol=B.sto-3g.lsym.mol --inp=B.2P12.dc_rkb.scf_2fs_mj.inp
```

```
pam --incmo --mol=B.sto-3g.lsym.mol --inp=B.2P32.dc_rkb.scf_2fs_reord_mj.inp
```

Finally, in the output one can see proper order of spinors for the excited state - first $2p_{3/2}$ spinors (four-fold degeneracy), and then the $2p_{1/2}$ spinors (two-fold degeneracy):

```
* Fermion symmetry E1u
* Open shell #1, f = 0.2500
-0.28691011961454 ( 4)
* Virtual eigenvalues, f = 0.0000
0.12724113329935 ( 2)
```

Regarding total energies we get

```
SCF energy 2_P1/2: -24.249116730391307
SCF energy 2_P3/2: -24.249038454664991
FSS : (-24.249116730391307+24.249038454664991)*219474.631280634 cm-1 = 17.179536 cm-1
```

what is very close to the above given COSCI value of 17.179488 cm⁻¹.

Correlated calculations

... to add ...

Conclusion

Further quality improvement of the theoretical data would be in enlarging basis set. The current one, decontracted STO-3G, is too small and is chosen for demonstrative purposes only.

Nitrogen atom open-shell atomic calculations

We set an average occupation of 3 electrons over 6p shells.

```
pam N.sto-3g.C2v.mol N.3os_aver.dc_rkb.scf_1fs_cosci.inp
pam N.sto-3g.lsym.mol N.3os_aver.dc_rkb.scf_2fs_cosci.inp
```

Obtained COSCI states are as follows:

1	0.000000000	0.000000	1	1	1	1	0	0	0	0
2	2.972174508	23972.206548	1	1	1	1	0	0	0	0
3	2.972227617	23972.634901	1	1	1	1	1	1	0	0

4	4.953663077	39953.991305	1	1	0	0	0	0	0	0
5	4.953762831	39954.795879	1	1	1	1	0	0	0	0

Hydroxyl radical

Here we present open-shell SCF calculations on the OH.(known as the hydroxyl radical) system, followed by complete small open shell CI method (cosci) for getting individual states.

The ground state of the OH readical is X 2_Pi_3/2 resulting from the electron configuration of

1s_sig(2) 2s_sig(2) 2pz_sig(2) 2pxy_pi_1/2(2) 2pxy_pi_3/2(1)

Settig the SCF configuration

What does not converge is the 1 electron in 1 open-shell:

```
.CLOSED SHELL
8
.OPEN S
1
1/2
```

Therefore for this case one has to resort to the average-of-configurations (3 electrons over 4 pi shells):

```
.CLOSED SHELL
6
.OPEN S
1
3/4
```

The Dirac-Coulomb Hamiltonian gives noncorrelated fine structure splitting of 143.842458 cm-1 as the result of complete open-shell CI over pi-shells.

```
pam OH.ccpVDZ.lsym.mol OH.dc_rkb.scf_os_res.inp
pam OH.ccpVDZ.C2v.mol OH.dc_rkb.scf_os_res.inp
```

COSCI energies are :

```
-75.446610400724 ( 2 * )
-75.445955006263 ( 2 * )
```

The average SCF energy is

```
::
-75.446282703494589
```

and is equal to the average values of COSCI energies

```
((-75.446610400724-75.445955006263)/2=-75.446282703493
```

Spin-free approach

Utilizing the Dirac-Coulomb spin-free Hamiltonian confirms the four-fold (pi_x=pi_y) degeneracy of the valence open shells:

```
pam OH.ccpVDZ.C2v.mol OH.dc_sf.scf_os_res.inp
```

With the resulting spin-free SCF (and COSCI) energy of

```
-75.446280480624 ( 4 * ).
```

Mj selection

For this heteronuclear diatomic molecule one has the advantage of using the omega-number occupation of shells. One can even place one electron in one shell, because the “MJSELECTION” keyword ensures the convergence of both “X 2Pi_3/2”

```
.CLOSED SHELL
8
.OPEN S
1
1/2
# 2Pi_3/2
.MJSELECTION
3
4 0 0
0 1 0
```

and of the “A 2Pi_1/2” first excited state:

```
.CLOSED SHELL
8
.OPEN S
1
1/2
# 2Pi_1/2
.MJSELECTION
3
3 1 0
1 0 0
```

```
pam OH.ccpVDZ.lsym.mol OH.dc.scf_mj_2Pi32.inp
pam OH.ccpVDZ.lsym.mol OH.dc.scf_mj_2Pi12.inp
```

giving total SCF energies of the ground and the excited states:

```
X 2Pi_3/2 : -75.446717125968732
A 2Pi_1/2 : -75.446063726708829
```

The fine-structure splitting then makes

FSS : $(-75.446717125968732 + 75.446063726708829) \times 219474.631280634 = 143.404561 \text{ cm}^{-1}$

which is very close to the above given COSCI value of 143.842458 cm⁻¹.

Energy order of spinors for the OH “X 2Pi_{3/2}” ground state is:

```
1: -20.623437096718      (Occupation: f = 1.0000) m_j= 1/2; 1s_sig(2)
2: -1.2970494235574      (Occupation: f = 1.0000) m_j= 1/2; 2s_sig(2)
3: -0.6455362075645      (Occupation: f = 1.0000) m_j= 1/2; 2pz_sig(2)
4: -0.5439526090110      (Occupation: f = 1.0000) m_j= 1/2; 2pxy_pi_1/2(2)
5: -0.5863405891346      (Occupation: f = 0.5000) m_j= -3/2; 2pxy_pi_3/2(1)
```

While for the first excited “A 2Pi_{1/2}” state of the OH radical we have this order:

```
1: -20.623901392971      (Occupation: f = 1.0000) m_j= 1/2 ; 1s_sig(2)
2: -1.2971646385507      (Occupation: f = 1.0000) m_j= 1/2 ; 2s_sig(2)
3: -0.6456515101399      (Occupation: f = 1.0000) m_j= 1/2 ; 2pz_sig(2)
4: -0.5431435116553      (Occupation: f = 1.0000) m_j= -3/2 ; 2pxy_pi_3/2(2)
5: -0.5874674338778      (Occupation: f = 0.5000) m_j= 1/2 ; 2pxy_pi_1/2(1)
```

Correlated approach

The user can try to approach experimental [data](#).

Hydroxyl radical cation OH⁺

Again one employs the average-of-configuration:

```
.CLOSED SHELL
6
.OPEN S
1
2/4
```

Number of electronic states is higher.

Electronic states can be found [here](#).

How to handle open-shell systems using CC methods

Coupled-Cluster Singles, Doubles and Noniterative triples (CCSD(T)) and Fock-space Coupled-Cluster methods are powerful correlation methods in the DIRAC program suite.

Coupled Cluster (CC) methods in DIRAC are most powerful ab-initio correlation methods working upon two/four-component Kramers unrestricted spinors. They serve as widely employed relativistic analogue with respect the nonrelativistic realm.

Therefore we feel it is important to present the user few hints on how to fully exhaust CC capabilities for practical calculations.

Even at the Coupled Cluster correlated level the user has certain variability in choosing the occupation of spinors. Thus he may alter the electronic state of a system.

We give you few hints how to employ them in correlated open-shell calculations.

CCSD(T) method

Example 1: FO molecule

Fock space CCSD method

The starting system is always closed shell. One can add one or two-electrons to the N-electron system to iterate into N+1 and/or N+2 systems.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/ecp/combine.md

orphan

Combining other methods with ECP

In DIRAC program, ECP can be incorporated with several ground and excited state calculation methods. The calculation methods can be set in input file. Below is the few examples of hydrogen iodide calculations. (See `ecp_input` for the molecular input)

DFT calculation

See the quick Bi2 molecule test (`DFT.inp <../.../test/ecp/DFT.inp>` and `Bi2.xyz <../.../test/ecp/Bi2.xyz>`)

`../.../test/ecp/DFT.inp`

COSCI calculation

`COSCI.inp`

MP2 and CC calculation

`CC.inp`

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/ecp/first.md

orphan

ECP calculation

In DIRAC, the effective core potential (ECP) method is implemented and various subsequent correlation methods are available within the two-component or one-component effective Hamiltonian.

From the inclusion (exclusion) of spin-orbit potential parameters in the input file, molecular spinors (orbitals) are obtained and this is the starting point of several ground and excited state calculation with the ECP method.

The first example is the SCF calculation of Hydrogen Iodide.

SOREP calculation

The simplest SCF input file (`HF.inp <HF.inp>`) for ECP calculation is

`HF.inp`

together with the molecular input (`HI_sorep.mol <HI_sorep.mol>`)

`HI_sorep.mol`

This is the two-component spin-orbit relativistic effective core potential (SOREP) calculation. (For the detail of input, see `ecp_input`)

In the calculation output, the two-component molecular spinors (see also the `full output <HF_HI_sorep.out>`) are listed as :

```
* Fermion symmetry E1
* Closed shell, f = 1.0000
-0.91849028398625 ( 2) -0.53547268879208 ( 2) -0.40006484189588 ( 2) -0.37307836548798 ( 2)
* Virtual eigenvalues, f = 0.0000
0.03426904787073 ( 2) 0.05875502261229 ( 2) 0.08417741978544 ( 2) 0.08661765422042 ( 2) 0.08729608075284 ( 2)
0.13645335183505 ( 2) 0.20146339739788 ( 2) 0.20209264550882 ( 2) 0.23363818798538 ( 2) 0.23500980856689 ( 2)
0.26681617711053 ( 2) 0.33365537629186 ( 2) 0.34280937791647 ( 2) 0.34322959805083 ( 2) 0.45833702543722 ( 2)
0.48328741534790 ( 2) 0.50649128877099 ( 2) 0.51736684713002 ( 2) 0.61447310303433 ( 2) 0.61538818610103 ( 2)
0.61831033924315 ( 2) 0.61894855200797 ( 2) 0.63389639932323 ( 2) 0.63480881100894 ( 2) 0.63698525034339 ( 2)
0.67886357619745 ( 2) 0.68325244000339 ( 2) 0.75638614817881 ( 2) 0.75745722311601 ( 2) 0.82161646959771 ( 2)
0.92201376113948 ( 2) 0.98455089264787 ( 2) 0.98468688973278 ( 2) 1.01721143371678 ( 2) 1.01778243972275 ( 2)
1.19559448686781 ( 2) 1.32033135104656 ( 2) 1.32175612476556 ( 2) 1.47246800149637 ( 2) 1.47312441361493 ( 2)
1.48573327678144 ( 2) 1.48673373305796 ( 2) 1.52583503659726 ( 2) 1.66340941889701 ( 2) 1.66434543208265 ( 2)
1.68305757276943 ( 2) 1.99128262601425 ( 2) 1.99405394098767 ( 2) 2.00507423191293 ( 2) 2.00643391158823 ( 2)
2.23720017057984 ( 2) 2.96761200756551 ( 2) 3.86994566085372 ( 2) 3.87002312971009 ( 2) 4.03165192324828 ( 2)
4.03197238122657 ( 2) 4.26133522183310 ( 2) 4.26149442194385 ( 2) 4.35237329819399 ( 2) 4.68664685801378 ( 2)
6.91601218905285 ( 2) 7.39252180916936 ( 2) 7.61998005503501 ( 2) 21.10496588735050 ( 2)
```

In this output, the molecular spinors are generated with the inclusion of spin-orbit coupling in the SCF step. The orbital degeneracies showing up in AREP calculation are broken with SOREP (for comparison, see AREP calculation results below).

Note: Compared to all-electron calculations, only small number of occupied molecular spinors are listed in the above example. Here, only four occupied molecular spinors are shown. This is because only eight (upper) electrons are described - seven electrons on the iodine atom and one electron on the hydrogen atom. Remaining forty-six electrons in iodine are omitted and are substituted by relativistic effective core potentials.

AREP calculation

We set the one-component scalar relativistic effective core potential (AREP) calculation by omitting the SO parameters in SOREP (`HI_arep.mol <HI_arep.mol>`).

`HI_arep.mol`

After the SCF step, the following molecular orbitals are generated (see also the `full output <HF_HI_arep.out>`) :

```
* Fermion symmetry E1
* Closed shell, f = 1.0000
-0.91858300709907 ( 2) -0.53332424176610 ( 2) -0.38664035068657 ( 4)
* Virtual eigenvalues, f = 0.0000
0.03415914639101 ( 2) 0.05878598572499 ( 2) 0.08511594399606 ( 2) 0.08655831374568 ( 4) 0.13616239596081 ( 2)
0.20167198225135 ( 4) 0.23444552560595 ( 4) 0.26659777181880 ( 2) 0.33379422568499 ( 2) 0.34298998830327 ( 4)
0.46036400731232 ( 2) 0.49614437446042 ( 4) 0.51225813066935 ( 2) 0.61491923213700 ( 4) 0.61871900761802 ( 4)
0.63458123096312 ( 4) 0.63671447698655 ( 2) 0.68126113881102 ( 4) 0.75708271906160 ( 4) 0.82156075161775 ( 2)
0.92185115859998 ( 2) 0.98434317399476 ( 4) 1.01698231678790 ( 4) 1.19528790938055 ( 2) 1.32084962430606 ( 4)
1.47297468375170 ( 4) 1.48649707416567 ( 4) 1.52558291015577 ( 2) 1.66381720482403 ( 4) 1.68291895964979 ( 2)
1.99293721990332 ( 4) 2.00580508554321 ( 4) 2.23700102551750 ( 2) 2.96678284298137 ( 2) 3.86917720594751 ( 4)
4.03098540560673 ( 4) 4.26054689341582 ( 4) 4.35208715121292 ( 2) 4.68579647949884 ( 2) 7.21075466518969 ( 4)
7.50571888682965 ( 2) 21.10478953048176 ( 2)
```

Compared to molecular spinors from the SOREP calculation above, the spin-orbit coupling is neglected in AREP-based molecular orbitals. As a result, some degeneracies in molecular orbitals appear in the relativistic scalar (spin-free, AREP) only treatment. Detailed comparison of molecular orbitals can be done using the ****ANALYZE** functionality.

Exercises

1. Identify and compare few occupied and virtual orbitals (including HOMO and LUMO) obtained without (AREP) and with (SOREP) spin-orbital relativistic effects.
2. Optimize (numerically, see *OPTIMIZE) the internuclear distance with (SOREP) and without (AREP) spin-orbital interaction. What is effect of spin-orbital coupling on the bond distance? Compare against other published works (cite: Park2012). Help - the input file, `geopt_HF.inp` <`geopt_HF.inp`>.
3. Compute the dipole moment, electric polarizability and first hyperpolarizability of the HI molecule with (SOREP) and without (AREP) spin-orbital relativistic effects. How do spin-orbital effects influence this property? Compare with two-component (X2C) all electron calculations. Input files `HI.acv2z.mol` <`HI.acv2z.mol`>, `x2c.scf_prop.inp` <`x2c.scf_prop.inp`> and `x2c_sf.scf_prop.inp` <`x2c_sf.scf_prop.inp`>.
4. At the X2c-level evaluate the effect of the picture change for the dipole moment, electric polarizability and the first hyperpolarizability (input file `x2c.scf_prop_nopctr.inp` <`x2c.scf_prop_nopctr.inp`>). What is the size of the picture change effect affecting the properties? Apply these findings to the previous ECP property calculations.
5. To evaluate the size of spin-orbital effects on properties, you might use (in addition to the X2C approach of point 3.) the four-component Dirac-Coulomb Hamiltonian.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/fde_nmr/tutorial.md

orphan

NMR shieldings in Frozen Density Embedding (FDE) scheme with London atomic orbitals (LAOs)

In this tutorial we will show how to calculate the NMR shielding tensor with London atomic orbitals within the FDE scheme.

NMR shielding tensor in FDE scheme - theory overview

In order to calculate the NMR shielding tensor with LAOs, one must take into account modifications of the property gradient due to the use of perturbation-dependent basis sets (see Ilias2009). In FDE scheme, further modifications are needed, since the property gradient includes not only terms from isolated subsystems (here marked as “I” and “II”), but also a contribution from the interaction energy between these subsystems (see Hofener2013).

For instance, these new contributions to the property gradient of subsystem “I” can be written as (see :cite: olejniczak2017):

$$E_{\{1\},\text{emb}}^{\{B\}} = - \int v_{\text{emb}}^I \Omega_{\{a\}}^{\{B,I\}} - \iint w_{\text{emb}}^{\{I,I\}} \Omega_{\{a\}}^{\{I\}} \Omega_{\{jj\}}^{\{B,I\}} - \iint w_{\text{emb}}^{\{I,II\}} \Omega_{\{a\}}^{\{I\}} \Omega_{\{jj\}}^{\{B,II\}}$$

where $\Omega_{\{pq\}}^{\{B\}}$ is the first-order perturbed overlap distribution, which in a basis of London orbitals consists of two terms

$$\Omega_{\{pq\}}^{\{B\}} = \frac{i}{2} (R_{\{MN\}} \times r) \Omega_{\{pq\}} + (T_{\{pt\}}^{\{B\}} \Omega_{\{tq\}} + \Omega_{\{pt\}} T_{\{tq\}}^{\{B\}}),$$

“direct” LAO term (the first term) and “reorthonormalization” term (in parenthesis). The v_{emb}^I is the embedding potential of subsystem I, $w_{\text{emb}}^{\{I,I\}}$ is the embedding kernel involving subsystem I and $w_{\text{emb}}^{\{I,II\}}$ is the embedding kernel coupling subsystem I and subsystem II.

In this tutorial we will show how to include each of these contributions to the property gradient in the calculations of the NMR shielding tensor. We will also demonstrate how to calculate and plot the first-order perturbed current density induced by magnetic field and how to use it in analysis of the NMR shielding tensor.

A sample calculation of NMR shielding tensor within FDE

We will consider a simple system of SeH_2 - H_2O , in which SeH_2 constitutes an active subsystem (subsystem I) and H_2O is an environment (subsystem II):



The computational protocol is different depending on whether the coupling between two subsystems is included or not (through the term dependent on $w_{\text{emb}}^{\{I,II\}}$). To avoid confusion, we will describe these two protocols separately.

We will need two molecular input files. File `h2se.mol`, corresponding to an active subsystem I, reads:

```
h2se.mol
```

and file `h2o.mol`, standing for a subsystem II (‘environment’):

h2o.mol

In this example we will use the four-component relativistic DC Hamiltonian and DFT/LDA with small grid and small basis sets.

Calculations without the FDE coupling terms

Here we demonstrate how to include the first two terms of Eq. fde_pg in the property gradient: terms dependent on the embedding potential and the uncoupled embedding kernel.

In this tutorial, we use a coarse grid generated in DIRAC in preceeding calculations on $\text{SeH}_2\text{-H}_2\text{O}$ ‘supermolecule’, which was then exported on a numerical_grid file. In DIRAC the numerical_grid file, which can be retrieved from DFT calculations, is divided into batches of points. However, FDE module is able to read only the continuous list of grid points, therefore such numerical_grid file has to be reformatted. This can be done by the script which can be found in utils directory of DIRAC, dft_to_fde_grid_convert.py, which should be called from the directory where the numerical_grid file is:

```
grid_convert
```

This script will overwrite the numerical_grid and save its original version as numerical_grid.0RIG.

Step 1: SCF calculation on the subsystem II

The first step is the calculation on subsystem II, the aim of which is to export its density and electrostatic potential on a text file.

The menu file getfrozen.inp reads:

```
getfrozen.inp
```

and we make sure to get the text file GRIDOUT (related to the FDE_.GRIDOU keyword). We should also remember to supply two files, related to keywords FDE_.GRIDOU (GRIDOUT) and FDE_.EXONLY (FILEEX), which are copies of a numerical grid file:

```
getfrozen.run
```

After this step, the GRIDOUT file will be updated: additional columns on that file correspond to the electrostatic potential and the density of subsystem II calculated in each point of a grid.

Step 2: SCF and response calculation on the subsystem I

Now we can use the information about the subsystem II written on the GRIDOUT file for constructing the embedding potential, $v_{\text{emb}}^{\text{I}}$, and embedding kernel, $w_{\text{emb}}^{\text{I,I}}$. In order to do that, we use the GRIDOUT file saved from Step 1 and copy it to a FROZEN file, which is given as an argument to the FDE_.FRDENS keyword.

We use the following input file for the SCF step:

```
updatefrozen.inp
```

and the pam command is:

```
updatefrozen.run
```

To calculate the NMR shielding tensor, we use the following input file:

```
shield_v_w11.inp
```

in which the keyword FDE_.LA011 denotes that the contributions from the embedding potential, $v_{\text{emb}}^{\text{I}}$, and the uncoupled embedding kernel, $w_{\text{emb}}^{\text{I,I}}$, will be added to the property gradient. The corresponding pam command is:

```
shield_v_w11.run
```

This is the advised option. However, as the calculations of the uncoupled embedding kernel terms in the property gradient may be expensive, one may want to restrict the FDE-LAO contributions to the part dependent on the embedding potential only. In this case, the input file reads:

```
shield_v.inp
```

and the corresponding pam command is:

```
shield_v.run
```

Additionally, we may want to save the PAMXVC and TBM0 files, if we are interested in visualization of the NMR shielding density and/or magnetically-induced current density. The calculation on our test molecule give the following results for the isotropic part of the shielding tensor of Se atom:

terms included in property gradient	value
None	2469.8091
v	2370.3941
v+w11	2370.2488

None in the table denotes calculations performed with no “new” FDE-LAO terms in the property gradient (the gauge origin is in the center of mass of SeH_2). Also, in this test case the value obtained with the embedding potential and the uncoupled embedding kernel in the property gradient (v+w11) is very close to the result of calculations with the embedding potential only (v), but for heavier nuclei the difference between these two setups may be significant.

Calculations with the FDE coupling terms

Here we will demonstrate how to include the coupling between two subsystems in the property gradient (term dependent on $w_{\text{emb}}^{\text{I,II}}$).

Step 1: SCF and response calculation on the subsystem II

In the first step we perform the calculations on subsystem II, the aim of which is to get its density and electrostatic potential written on a text file. In addition, we will export the perturbed density of subsystem II. We will need the DFCOEF file of subsystem II, which can be taken from `getfrozen.inp` run with the following input:

```
getfrozen.inp
```

with the `pam` command:

```
getfrozen.run
```

(exactly as done in calculations without the FDE coupling terms presented above). From this step we save the `GRIDOUT.h2o` file. In order to get the density perturbed by an external magnetic field in LAO basis, the following input can be used:

```
getfrozenpert.inp
```

with the keyword `NMR_.EXPED` asking the program to save the contributions to the perturbed density on two files `pertden_direct_lao.FINAL` and `pertden_reorth_lao.FINAL`. The corresponding `pam` command is:

```
getfrozenpert.run
```

NOTE: This step has to be run in serial mode for the moment.

Step 2: Response calculation on the subsystem I

Now we can finally calculate the FDE contributions to the NMR shielding tensor of an active subsystem (H_2Se) which depend on the embedding kernel including the coupling terms between two subsystems. If the SCF step on an active subsystem was not done before, we should run the following input:

```
updatefrozen.inp
```

with the `pam` command:

```
updatefrozen.run
```

The response input is then the following:

```
shield_v_w11_w12_nocoulomb.inp
```

with two new keywords: `FDE_.LA0FRZ` and `FDE_.PERTIM`. The corresponding `pam` command is:

```
shield_v_w11_w12_nocoulomb.run
```

This input does not include the computationally expensive Coulomb term in the embedding kernel, only the nonadditive exchange-correlation and kinetic parts of the embedding kernel. If the Coulomb term is to be calculated then an additional keyword is required in the input, `FDE_.LFCOUL`. Instead of using the `FDE_.LFCOUL` and the `FDE_.LA0FRZ` keywords together, it is possible to use a single `FDE_.LA012` keyword, which represents both contributions to the coupled embedding kernel:

```
shield_v_w11_w12.inp
```

with an analogous `pam` command:

```
shield_v_w11_w12.run
```

This input contains all the necessary contributions to the property gradient. Additionally, we may want to save the `PAMXVC` and `TBM0` files, if we are interested in visualization of the NMR shielding density and/or magnetically-induced current density. With this setup the calculations on the test system gave the following results:

terms included in property gradient	value
v+w11+w12, no Coulomb	2370.2536
v+w11+w12, ALL terms	2370.2536

Here the two results are the same within 0.0001 ppm accuracy, as the contributions from the Coulomb embedding contribution to the perturbed Fock matrix are very small (can be printed in the output with the print level set at least to 5 under `**PROPERTIES` section).

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/frozen_orbitals/tutorial.md

orphan

Frozen orbitals

DIRAC allows the freezing (or elimination) of orbitals during Hartree-Fock or Kohn-Sham calculations. We shall illustrate this functionality using bonding in the water molecule as an example and essentially reproducing a calculation reported by Jarvie and co-workers in 1973 Jarvie1973.

The water molecule has a bond angle of 104.5° . This is sometimes rationalized by invoking sp^3 hybridized oxygen. Perfect hybridization would, however, lead to a bond angle of $109.5^\circ = \arccos(-1/3)$, and so the deviation is rationalized by invoking valence shell electron pair repulsion (VSEPR) theory and in particular the stronger repulsion between lone pairs.

Jarvie and co-workers therefore proposed to freeze the oxygen $2s$ orbital in water and see how this affected the bond angle. Let us do this calculation the DIRAC way:

Atomic calculation

First we need an oxygen $2s$ orbital. We therefore set up an atomic calculation using the input files *O.inp*

O.inp

and *O.mol*

O.mol

Our plan now is to import this orbital into a subsequent molecular calculation. There is, however, one important restriction: The atomic calculation has to be carried out in the same symmetry as the molecular one, which in this case will be C_{2v} , hence the explicit symmetry specification in the *O.mol* input file (for more details, see *molecule_using_xyz*). We run the calculation using the command:

```
pam --inp=0 --mol=0 --get "DFCOEF=cf.0"
```

Geometry optimization with frozen orbitals: how it works

For the molecular calculation we use the input files *H2O.inp*

H2O.inp

and *H2O.mol*

H2O.mol

The latter file specifies the experimental geometry of water with bond angle 104.5° and bond distance $0.972 \text{ \AA}$. The former file includes the OPTIMIZE_.NUMGRA keyword to activate geometry optimization using a numerical gradient. This will be needed here since we have included the constraint of a frozen orbital.

Now let us look at the specification of the frozen orbital: We have to tell DIRAC that the orbital was not calculated in the full molecular basis, rather using only the basis of the oxygen atom. This information is indicated by the keyword SCF_.OWNBAS. Next, we have to tell where to find the orbital. This is specified using the keyword SCF_.PROJECT and its arguments. The oxygen $2s$ orbital is specified by the coefficients found on the file *DFCOEF* that we recovered in the atomic calculation and renamed *cf.O*. The oxygen atom is considered a fragment of the water molecule, and the coefficient file therefore constitute a fragment file. The first argument of the SCF_.PROJECT keyword is the specification of the number of fragment files to read, which in this case is just one.

For each fragment we now have to specify the name of the fragment file. As seen in the FORTRAN snippet in the manual under the SCF_.PROJECT keyword, the fragment file should be given a six-character name, so we in this case choose *DFOXXX*. When the fragment has been calculated in its own basis, as specified by the SCF_.OWNBAS keyword, we specify it by indicating the number of *symmetry-independent nuclei* it contains. Let us see how this works: Going back to the molecular input file *H2O.mol* file we see that it gives the coordinate of one oxygen atom and *one* hydrogen atom. We do not need to specify the coordinates of the second hydrogen atom because it is then known from symmetry. The two hydrogen atoms are symmetry-dependent and have to be treated together as a fragment. We have, however, selected the oxygen atom as our fragment and specify that our fragment consists of a single symmetry-independent center. Oxygen will be chosen since it comes first in the molecular input file.

We now have to specify what orbital(s) to select on the fragment file. This is done through an orbital string (see the section *orbital_strings*). The oxygen atom has five occupied orbitals and the $2s$ orbital is the second one, as we can for instance infer from looking at the Mulliken population analysis in the output of the atomic calculation.

Now we have to understand the SCF_.FROZEN keyword. This will take a little bit more theory: At each iteration in the SCF-cycle we have to solve the generalized eigenvalue equation

$$F\mathbf{c} = S\mathbf{c}\epsilon$$

where F is the Fock (or Kohn-Sham) matrix in the AO-basis (symmetry-adapted or not) and $\mathbf{c}$ and ϵ are the coefficients and eigenvalues we seek. The equation also features the overlap matrix S since the AO-basis is generally not orthogonal. A first step towards the solution of this equation is to introduce a non-unitary transformation V that will take us to an orthonormal basis, that is

$$V^\dagger SV = I$$

where I is the identity matrix. We insert $VV^\dagger = I$ in front of the coefficients and pre-multiply with $V^\dagger$ to give

$$V^\dagger F V \mathbf{c} = V^\dagger S V \mathbf{c} \epsilon$$

Since the matrix V transforms the overlap matrix to the identity matrix, the transformed equation simplifies to

$$F^{\prime} \mathbf{c}^{\prime} = \mathbf{c}^{\prime} \epsilon; \quad F^{\prime} = V^\dagger F V; \quad \mathbf{c}^{\prime} = V^{-1} \mathbf{c}.$$

This is a normal eigenvalue problem and it suffices now to give the transformed Fock matrix $F^{\prime}$ to a standard routine for matrix diagonalization in order to obtain eigenvalues ϵ and the transformed eigenvectors $\mathbf{c}^{\prime}$. In a final step we get the MO-coefficients $\mathbf{c}$ by the backtransformation

$$\mathbf{c} = V \mathbf{c}^{\prime}$$

This diagonalization procedure is repeated in each iteration of the SCF-cycle and provides the optimization of the molecular orbitals.

The freezing of orbitals is done in the following manner: The imported fragment orbitals are projected out of the transformation matrix V , which means that they are not present in the orthonormal basis of the transformed Fock matrix $F^{\prime}$. They have in other words been removed from the variational space. If this was our goal we could in fact have stopped here. This is for example what was done in Fossgaard2003b where the effect of the lanthanide contraction on bonding in the CsAu molecule was investigated by projecting out the gold $4f$ orbitals from the variational space (hence the name of the SCF_.PROJECT keyword).

In the present case we do not want to delete the oxygen $2s$ orbital, rather freeze it. Therefore we have to put the oxygen $2s$ orbital back when backtransforming coefficients. The SCF_.FROZEN keyword takes as argument an orbital string which indicates in what position one would like to put the frozen orbitals in the total set of orbitals.

Geometry optimization of the water molecule with a frozen oxygen 2s orbital

We run the geometry optimization with the command:

```
pam --inp=H2O --mol=H2O --put "cf,0=DF0XXX"
```

After optimization we find that the bond distance is 0.972 Å and the bond angle 95.9 $^{\circ}$. This is perhaps a surprising result since one would expect that breaking hybridization would lead to a bond angle of 90.0 $^{\circ}$. Our result suggests that there must be some additional repulsion between the hydrogens of water. The nature of this repulsion (electrostatic vs. Pauli repulsion) has been a subject of considerable discussion in the literature. A full account is found in Dubillard2006.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/getting_started.md

orphan

Getting started with DIRAC

You have installed and tested DIRAC and now it's time to run your first DIRAC calculation!

You have two possibilities to run DIRAC. The traditional uses two input files: the *.inp file, which determines what should be calculated, together with the *.mol file, which defines the molecular geometry and the basis set. Alternatively you can use the *.inp file, file together with a geometry file (*.xyz file.). In this case the *.xyz file provides the nuclear coordinates and the *.inp file contains in addition to the job description also the basis set information (see ***MOLECULE**).

DIRAC writes its output to a text file, named after the molecule and input file names. If this file already exists, for example from a previous calculation, the old file is renamed to make place for the new output. The python script pam that launches the main DIRAC executable also writes some general information to standard output.

Dirac-Coulomb SCF

Let's try it out and run our first DIRAC calculation with the following minimal input (hf.inp) to calculate the Dirac-Coulomb Hartree-Fock ground state (with the default [simple Coulombic correction](#) to approximate the contribution from (SS)SS integrals:

```
**DIRAC
.WAVE FUNCTION
**WAVE FUNCTION
.SCF
**MOLECULE
*BASIS
.DEFAULT
cc-pVDZ
*END OF INPUT
```

together with the following example geometry file (methanol.xyz):

```
6
my first DIRAC calculation # anything can be in this line
C      0.000000000000      0.138569980000      0.355570700000
O      0.000000000000      0.187935770000     -1.074466460000
H      0.882876920000     -0.383123830000      0.697839450000
H     -0.882876940000     -0.383123830000      0.697839450000
H      0.000000000000      1.145042790000      0.750208830000
H      0.000000000000     -0.705300580000     -1.426986340000
```

Now start the calculation by executing:

```
pam --mol=methanol.xyz --inp=hf.inp
```

If everything works fine the results of the calculation will be written to hf_methanol.out. In addition a [HDF5](#) checkpoint file hf_methanol.h5 will be created (see checkpoint for detailed information). To see the actual content of the file you may execute:

```
h5dump -n hf_methanol.h5
```

Closed shell CCSD

Running CCSD(T) calculations is straightforward if the ground state of the molecule can be well-described by a single closed shell determinant. This is the case for many molecules.

The input requires the specification of a basis set (TZ or better is recommended, but for this example we will take DZ to reduce the run time). We take the inter halogen molecule ClF as an example and use the default cut-offs for correlating electrons (include all valence electrons with energy above -10 hartree) and truncation of virtual space (delete virtuals above 20 hartree). The geometry file (clf.xyz) is very simple and only requires to specify the coordinates. The program will then identify the symmetry as $S_{\infty v}$:

```
2
ClF molecule at equilibrium distance taken from NIST
Cl  0.0  0.0  0.0
F   0.0  0.0  1.628
```

We specify the wave function type (SCF, followed by RELCCSD) and basis set in the input file (cc.inp) to calculate the CCSD(T) energy with the Dirac-Coulomb Hamiltonian again with the contribution from (SS)SS integrals approximated by a simple Coulombic correction:

```
**DIRAC
.WAVE FUNCTION
```



```

**WAVE FUNCTION
.SCF
.RELCCSD
**MOLECULE
*BASIS
.DEFAULT
cc-pVDZ
*END OF INPUT

```

Now start the calculation by executing:

```
pam --mol=clf.xyz --inp=cc.inp
```

If everything works fine the results of the calculation will be written to `cc_clf.out`. In addition an HDF5 checkpoint file `cc_clf.h5` will be created, as mentioned above.

You can suppress having the checkpoint file copied back to your home directory using:

```
pam --noh5
```

Cheap relativistic calculations

Relativistic calculations are more expensive than non-relativistic ones, but this is accentuated by the fact that relativistic effects are associated with heavy elements, which also implies many electrons and increased computational cost. However, as explained in Dubillard2006, we can induce strong relativistic effects in molecules of light atoms by playing with the speed of light. The starting point is the observation that a diagnostic of relativistic effects is provided by the Lorentz factor

$$\gamma = \frac{1}{\sqrt{1-(v/c)^2}},$$

where v is the speed of the particle and c is the speed of light. In atomic units we have:

* The speed of light : 137.0359992

as shown in the DIRAC output. Scalar relativistic effects are associated with the high speeds of electrons in the vicinity of heavy nuclei. In fact, for a one-electron atom the average speed of the $1s$ electron in atomic units is $v_{1s}=Z$. This explains why relativistic effects become so pronounced for heavy atoms; what counts is the nuclear charge, not the atomic mass. On the other hand, we see that relativistic effects are sensitive to the ratio (v/c) , so one way to accentuate relativistic effects is to *reduce* the speed of light.

Let us look at an example of this. We shall look at the geometry of the water molecule as a function of the speed of light. The molecular geometry is given by `h2o.xyz`

`h2o.xyz`

For a standard geometry optimization we use the input file `geo.inp`

`geo.inp`

We run DIRAC using :

```
pam --inp=geo --mol=h2o.xyz
```

From the DIRAC output we find that the initial geometry is:

Bond distances (angstroms):

	atom 1	atom 2	distance
bond distance:	H 1	O	0.957897
bond distance:	H 2	O	0.957897

Bond angles (degrees):

	atom 1	atom 2	atom 3	angle
bond angle:	H 2	O	H 1	104.120

whereas at the end of optimization we have:

Bond distances (angstroms):

	atom 1	atom 2	distance
bond distance:	H 1	O	0.942905
bond distance:	H 2	O	0.942905

Bond angles (degrees):

	atom 1	atom 2	atom 3	angle
bond angle:	H 2	O	H 1	105.667

Now let's play. We now do a geometry optimization using the file `geo_ultra.inp`

`geo_ultra.inp`

where the speed of light is reduced all the way down to $10^{-4} E_h/\hbar$. In the DIRAC output the final geometry is now:

Bond distances (angstroms):

	atom 1	atom 2	distance
bond distance:	H 1	0	1.066701
bond distance:	H 2	0	1.066701

Bond angles (degrees):

	atom 1	atom 2	atom 3	angle
bond angle:	H 2	0	H 1	89.505

That is quite a change ! The reader is encouraged to play with other speeds of light. It is also possible to turn off spin-orbit interaction by setting HAMILTONIAN_.SPINFREE.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/highspin_cc/O2.md

orphan

Relativistic Coupled-Cluster calculations on the dioxygen molecule

A simple example of a high-spin open shell state treated with the single reference CCSD(T) method is the dioxygen molecule.

Hartree-Fock reference

We start with the description of the single-determinant reference state. The ground state with two unpaired electrons corresponds to the triplet state. In the $D_{\infty h}$ symmetries the electrons are placed as follows:

```
.CLOSED SHELL
6 8
.OPEN SHELL
1
2/4,0
```

(See the DIRAC.INP <hf.inp> and the MOLECULE.MOL <O2.mol> input files, and the associated output file <hf_O2.out>.)

The overall electronic occupation of the oxygen molecule can be displayed in the following MO diagram ([see also here](#)):

 MO scheme of the oxygen molecule

Coupled-Cluster description

Let us continue with the correlated treatment of the oxygen molecule at the Coupled-Cluster level.

The 1s electrons of each oxygen atom are kept frozen. The active space then consists of 12 electrons, of which there are 10 closed shell electrons.

The closed shell electrons occupy the nonbonding $2s \sigma_g$ and σ_u orbitals and the bonding $2p \sigma_g$ and π_u orbitals, while the 2 open shell electrons are distributed over the two antibonding $2p \pi_g$ orbitals.

Let us suppose that we want to take the $M_S=1$ state as our reference and assign alpha spin to both our open shell electrons. In the double group ($D_{\infty h}^{(2)}$) we do not distinguish between σ_g and π_g , so we need to add all alpha electrons in gerade orbitals ($2s \sigma_g + 2p \sigma_g + 2p \pi_g = 4$), beta electrons in gerade orbitals ($2s \sigma_u + 2p \sigma_u = 2$), alpha electrons in ungerade orbitals ($2s \pi_u + 2p \pi_u = 3$) and beta electrons in ungerade orbitals ($2s \pi_u + 2p \pi_u = 3$). This gives - at the Coupled Cluster level - the following occupation:

```
.NELEC
4 2 3 3
```

(See the input file <uccsd.inp> and the output file <uccsd_O2.out>.)

Note that this determinant does not represent the exact ground state of the oxygen molecule as this triplet is split by a few wave numbers due to spin-spin and (second order) spin-orbit interactions.

The lowest state is the $\Omega=0$ component that cannot be represented by a single determinant. This state can be calculated using the Fock space Coupled Cluster method, see below.

Utilizing the linear symmetry

Coming back to our example, the oxygen molecule, we now show how the Coupled-Cluster occupation can be refined in the linear symmetry.

We again want to take the $M_S=1$ state as our reference. The irreps of $D_{\infty h}^{(2)}$ are ordered as $1/2, -1/2, 3/2, -3/2, \dots$ so we need to consider the Ω value (giving the projection on the molecular axis of both spin and orbital angular momentum) of the occupied orbitals. The σ_g orbitals go in the irreps $1/2$ and $-1/2$ while the π_g orbitals span the four irreps $(1/2, -1/2, 3/2, -3/2)$. Putting an alpha electron in a σ_g orbital will give an Ω value of $1/2$, while putting it in a π_g orbital can either give $-1/2$ (when put in the orbitals with orbital momentum -1) or $3/2$ (when put in the +1 orbital). Similarly the beta electrons go in irreps $-1/2$ (for the σ_g), $-3/2$ and $1/2$ (for the π_g). This makes the input for our example :

```
.NEL_F1
2 3 1 0
.NEL_F2
2 2 1 1
```

(The input file <uccsd_linsym.inp>, and the resulting output file <uccsd_linsym_02.out>.)

Fock-space Coupled-Cluster calculations

An example of the use of the Fock space method concerns the calculation of the lowest three states of the molecular oxygen.

These are obtained by distributing two electrons in the degenerate π^*_g orbitals. The X ground state is a triplet that is split by the second-order spin-orbit coupling and the spin-spin interactions in a lowest $M_S=0$ component and two higher $M_S=\pm 2$ components.

In the Fock space approach we first perform a calculation on the closed-shell O_2^{2+} and calculate the three X states, the degenerate open shell singlet a and the singlet b state in one single Fock space run, which involves sectors sequence of (0,0)-(0,1)-(0,2).

The complete FSCCSD input reads then <fscsd.inp>. The corresponding output file is here <fscsd_02.out>. These FSCCSD correlated energy states can be compared against the uncorrelated COSCI states, see here <hf_02.out>.

Another way of running FSCCSD calculations is starting from the closed-shell anion, O_2^{2-} , and removing two-electrons in the sequence of (0,0)-(1,0)-(2,0) sectors. The FSCCSD input file is here <fscsd_IE2.inp>, and the downloadable output file is here <fscsd_IE2_02.out>.

The reader can check the experimental values of excitation energies in the [NIST web page](#).

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/hybrid_parallel_cc_run/hybrid_parallel_cc_run.md

orphan

DIRAC hybrid parallelization

Here we present the simple performance study of the hybrid (or mixed) Open MPI - OpenMP parallelization.

The **Open MPI** is code's explicit parallelization, while the **OpenMP** is implicit parallelization of the linked mathematical library - MKL or OpenBLAS.

Machine

We employed the Kronos cluster (sitting at the GSI.de).

The SLURM system choosed the working node:

- Intel(R) Xeon(R) CPU E5-2680 v4 @ 2.40GHz
- 40 CPU node with 126 GB of the total memory (checked via `free -g -t`)

System

We use DIRAC benchmark Coupled-Cluster test and two different parallel DIRAC installations. Both consist of the Open MPI framework, the Intel compiler enhanced with highest optimization flag, `-xHost`, and internally threaded mathematical library - the (open) OpenBLAS library and the commercial MKL library.

- Open MPI 2.0.1; Intel-17 with MKL-integer8 internally-threaded(OpenMP) library

```
$PAM1 --mpi=$nOpenMPI --gb=4.60 --ag=5.00 --noarch --inp=$DIRAC/test/benchmark_cc/cc.inp --mol=$DIRAC/test/benchmark_cc/C2H4Cl2_sta_c1.mo
```

- Open MPI 2.0.1; Intel-17 with OpenBLAS-integer8 internally-threaded(OpenMP) library

```
$PAM2 --mpi=$nOpenMPI --gb=4.60 --ag=5.00 --noarch --inp=$DIRAC/test/benchmark_cc/cc.inp --mol=$DIRAC/test/benchmark_cc/C2H4Cl2_sta_c1.mo
```

The variables `-gb=MEM1` and `-ag=MEM2` are to be set carefully with respect to the total node memory and number of OpenMPI-threads. For instance, for 24 OpenMPI threads MEM2 is max. 120/24=5GB; MEM1 is to be lower, 4.60GB. The higher number of threads, the lower assigned memory per thread. We have to be careful to set the suitable value of MEM2 to make the memory demanding job pass.

Results

Wall times depending on number of OpenMPI (mpi) and OpenMP (omp) threads are shown in the following Table:

	mpi	omp	OpenBLAS	MKL
4	1		38min15s	37min4s
4	10		33min18s	29min1s
8	1		30min32s	29min13s
8	5		28min15s	23min36s
16	1	•		48min59s

Discussion

One can see (in Table mytableCC) that better performance is obtained with the Intel17-MKL compilation settings.

For this testing system, the fastest calculation is with 8 OpenMPI threads, where for each mpi-thread there are 5 MKL omp threads by keeping total number of CPUs, $8 \times 5 = 40$.

Higher number of threads may be causing communication overhead and thus performance slowdown, as we see for 16 mpi-thread.

Note that we have not splitted this compact run into separate steps (SCF, MOLTRA, RELCCSD), where each step would have with its specific setting for the hybrid-parallel run.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/lao_magnetizabilities/lao-magnetizabilities.md

orphan

Magnetizabilities with London Atomic Orbitals

Introduction

In this tutorial we will look at the calculation of (static) magnetizabilities with DIRAC. For more details about the method, see Ilias2013.

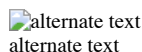
The component $\chi_{\alpha\beta}^{\text{static}}$ of the static 3×3 magnetizability tensor describes connects component α of first-order induced magnetic dipole to component β of the external inducing homogeneous magnetic field

$$m_{\alpha}^{(1)} = \sum_{\beta} \chi_{\alpha\beta}^{\text{static}} B_{\beta} = \frac{1}{2} \int \left(\mathbf{r}_G \times \mathbf{j}^{\text{B}_{\beta}} \right)_{\alpha} d\tau$$

The first-order induced magnetic dipole is proportional to the direct product of the position vector $\mathbf{r}_G$ with respect to some arbitrary (gauge) origin G and the first-order induced current density $\mathbf{j}^{\text{B}}$. Although the static induced magnetic dipole is formally independent of the gauge-origin this does not hold true in the finite basis approximation, where excruciating slow convergence of the magnetizability with respect to basis set is typically observed. London Atomic Orbitals (LAOs), also known as Gauge Including Atomic Orbitals (GIAOs), remove any reference to the arbitrary gauge origin by shifting the gauge origin to the centers of individual basis functions and effectively cure both problems.

Example: NF_3

We shall illustrate these features using the nitrogen trifluoride molecule.



The molecular input file NF3.mol uses the experimental geometry and automatic symmetry detection.

NF3.mol

Upon input one of the nitrogen atom is at the origin with one of the N-F bonds aligned with the z-axis. DIRAC detects the full symmetry C_{3v} , but uses the Abelian subgroup C_s .

NF3sym.txt

The molecule is further centered at the center of mass with the C_3 -axis aligned with the z-axis; this is seen from the xyz-file given in the output

NF3geo.txt

Generating the HF wave function

We first run a Hartree-Fock calculation to generate orbitals and corresponding energies using scf.inp

scf.inp

and the command:

```
pam --inp=scf --mol=NF3 --outcmo
```

You may notice in the output that DIRAC will center and rotate the molecule. It detects the full C_{3v} symmetry, but will run the calculation in the lower C_s symmetry, with reflection in the xy -plane, with the C_3 rotation around the z -axis.

Magnetizabilities: first attempt

We first calculate the magnetizability using the input file cgo.inp

cgo.inp

where the (common) gauge origin has been set to the center of mass using the NMR_USECM keyword. Notice that we use GENERAL_RKBIMP (? PROPERTIES_RKBIMP) to convert our molecular coefficients from restricted to unrestricted kinetic balance (RKB $\rightarrow$ UKB), the former employed for the generation of orbitals, and the latter employed for the response calculations, indicated by HAMILTONIAN_URKBAL. This corresponds to the use of [simple magnetic balance](#). The calculation is run using:

```
pam --inp=cgo --mol=NF3 --incmo
```

and gives the total magnetizability tensor

cgo_NF3_total

here reported in atomic units $e^2a_0^2/m_e$, corresponding to $7.89104 \cdot 10^{-29}$ J/T.

Magnetizabilities using LAOs

We now activate LAOs using the input file lao.inp

lao.inp

which gives the total magnetizability tensor

lao_NF3_total

which is markedly different from the CGO. The question now is: Which result is ‘best’ ? From microwave spectroscopy (see Stone1969) the magnetizability anisotropy, defined as

$$\chi_{ani} = \chi_{\perp} - \chi_{\parallel}$$

has been found to be $-0.63 \pm 0.32 e^2a_0^2/m_e$. With CGO and LAOs we obtain $+0.4421$ and $-0.6457 e^2a_0^2/m_e$, respectively, clearly favoring the LAO calculation. However, it very often happens that one gets the right answer for the wrong reason, so let us investigate the gauge-origin independence of the result as well as basis set convergence.

Gauge-origin dependence

To investigate gauge-origin dependence we do a CGO and LAO calculation with the gauge origin placed along the C_3 axis:

shift.inp

Please note that when shifting the gauge origin in this manner you should limit the gauge origin to symmetry-independent points, that is, you should stay on symmetry elements like the xz mirror plane in this case. The CGO calculation now gives

cgo2_NF3_total

We see that the parallel component $\chi_{\parallel}$ is unchanged, whereas the perpendicular component $\chi_{\perp}$ is dramatically different. With LAOs we get

lao2_NF3_total

where the numerical differences with respect to the original calculation is below the convergence threshold of the linear response calculation. We can therefore see that the use of LAOs removes the gauge dependence in the finite basis approximation.

Basis-set convergence

The basis set convergence is illustrated by the following table, taken from Ilias2013, showing CGO(LAO) magnetizabilities (in $e^2a_0^2/m_e$) for a wide range of basis sets.

Basis	$\chi_{\parallel}$	$\chi_{\perp}$	χ_{iso}	χ_{ani}
DZ	-14.83 (-4.27)	-9.78 (-4.87)	-11.47 (-4.67)	+5.04 (-0.60)
TZ	-7.88 (-4.36)	-6.52 (-4.96)	-6.97 (-4.76)	+1.37 (-0.60)
QZ	-5.65 (-4.43)	-5.54 (-5.04)	-5.58 (-4.83)	+0.11 (-0.61)
aug-DZ	-6.75 (-4.57)	-6.31 (-5.22)	-6.46 (-5.00)	+0.44 (-0.65)
aug-TZ	-5.01 (-4.64)	-5.42 (-5.24)	-5.28 (-5.04)	-0.41 (-0.60)
aug-QZ	-4.70 (-4.64)	-5.26 (-5.24)	-5.08 (-5.04)	-0.56 (-0.60)
d-aug-DZ	-6.37 (-4.57)	-6.18 (-5.22)	-6.24 (-5.00)	+0.19 (-0.65)
d-aug-TZ	-5.01 (-4.69)	-5.45 (-5.27)	-5.31 (-5.08)	-0.44 (-0.58)
d-aug-QZ	-4.76 (-4.68)	-5.31 (-5.28)	-5.13 (-5.08)	-0.55 (-0.60)

The basis set convergence is seen to be dramatically different, again clearly in favour of LAOs.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/localization/tutorial.md

orphan

Localization of orbitals

Example 1: Carbon dioxide

Textbooks and other sources, e.g. [Wikipedia](#) gives a Lewis structure of carbon dioxide with double bond from carbon to each oxygen. Is this the correct bonding picture ?

 Carbon dioxide Lewis structure

To investigate this, we shall carry out Pipek-Mezey localization Pipek:Mezey of the canonical molecular orbitals of carbon dioxide. We start from the molecular input *CO2.xyz*

CO2.xyz

as well as the menu file *CO2.inp*

CO2.inp

Note that we distinguish the two oxygen atoms. Since we do not use the proper element name, the appropriate charges are given in the menu file using the NUCLEUS keyword. We run the SCF calculation using:

```
pam --inp=CO2 --mol=CO2.xyz
```

In preparation for localization we used the keyword GENERAL_.ACMOUT to dump '\$C_1' \$ - coefficients to the checkpoint file. You can check this using:

```
h5dump -n CO2.h5 | grep C1
dataset      /result/wavefunctions/scf/mobasis/eigenvalues_C1
dataset      /result/wavefunctions/scf/mobasis/orbitals_C1
```

Now we are ready to localize the canonical MOs. We use the menu file *loc.inp*

Note that we now have turned off symmetry (NOSYM) and ask to read the '\$C_1' \$ coefficients from the checkpoint file using the GENERAL_.ACM0IN keyword. We run localization using:

```
pam --inp=loc --mol=CO2.xyz --put "CO2.h5=DFACM0.h5"
```

Using grep we find:

```
$grep '*C' loc_CO2.out
*CENTERS
*C ITR      DIFFA      |GRAD|      |KAPPA|      H(nonD/D)      e: +,-,0      H      DIFFG      DIFFQ      HMIN
*C 0 0.00D+00 0.15D-01 0.00D+00 0.00115 158 54 8 FULL 0.00D+00 0.00D+00 -0.36D+00
*C 1 0.21D-01 0.86D-02 0.54D+01 0.00068 183 29 8 FULL -0.21D-01 -0.22D+01 -0.36D+00
*C 2 0.57D-02 0.78D-02 0.34D+00 0.00037 184 28 8 FULL -0.57D-02 -0.72D-02 -0.36D+00
*C 3 0.13D-02 0.53D-02 0.59D+00 0.00035 193 19 8 FULL -0.13D-02 -0.33D-02 -0.37D+00
*C 4 0.88D-03 0.99D-03 0.26D+00 0.00021 199 13 8 FULL -0.88D-03 -0.10D-02 -0.37D+00
*C 5 0.27D-04 0.32D-04 0.49D+00 0.00010 204 8 8 FULL -0.27D-04 -0.27D-04 -0.37D+00
*C 6 0.14D-07 0.28D-07 0.40D-01 0.00008 204 8 8 FULL -0.14D-07 -0.14D-07 -0.37D+00
*C 7 0.98D-01 0.81D-01 0.67D+00 0.00143 203 9 8 FULL -0.98D-01 -0.17D+00 -0.26D+00
*C 8 0.54D-01 0.89D-01 0.30D+00 0.00130 201 11 8 FULL -0.54D-01 -0.64D-01 -0.18D+00
*C 9 0.23D-01 0.52D-01 0.16D+00 0.00116 201 11 8 FULL -0.23D-01 -0.25D-01 -0.18D+00
*C 10 0.78D-02 0.65D-02 0.17D+00 0.00103 203 9 8 FULL -0.78D-02 -0.86D-02 -0.18D+00
*C 11 0.42D-04 0.50D-02 0.40D+01 0.00104 205 7 8 FULL -0.42D-04 -0.10D-03 -0.18D+00
*C 12 0.68D-04 0.73D-04 0.59D+00 0.00104 208 4 8 FULL -0.68D-04 -0.70D-04 -0.18D+00
*C 13 0.64D-06 0.73D-05 0.39D+00 0.00103 208 4 8 FULL -0.64D-06 -0.16D-05 -0.18D+00
*C 14 0.39D-06 0.74D-06 0.13D+00 0.00104 208 4 8 FULL -0.39D-06 -0.41D-06 -0.18D+00
*C 15 0.28D-08 0.41D-08 0.10D-01 0.00103 208 4 8 FULL -0.28D-08 -0.28D-08 -0.18D+00
*C 16 0.25D-01 0.73D-02 0.40D+01 0.00101 206 6 8 FULL -0.25D-01 -0.16D+01 -0.18D+00
*C 17 0.42D-03 0.12D-03 0.62D-01 0.00100 209 3 8 FULL -0.42D-03 -0.42D-03 -0.18D+00
*C 18 0.13D-06 0.10D-07 0.50D-02 0.00100 209 3 8 FULL -0.13D-06 -0.13D-06 -0.18D+00
*C 19 0.58D-05 0.32D-04 0.10D+01 0.00099 210 2 8 FULL -0.58D-05 -0.28D-04 -0.18D+00
*C 20 0.11D-01 0.41D-01 0.25D+00 0.00100 202 16 2 FULL -0.11D-01 -0.11D-01 -0.14D+00
*C 21 0.79D-01 0.70D-02 0.82D+00 0.00033 212 6 2 FULL -0.79D-01 -0.16D+00 -0.11D-03
*C 22 0.27D-03 0.28D-04 0.28D+00 0.00033 212 6 2 FULL -0.27D-03 -0.27D-03 -0.18D-08
*C 23 0.14D-06 0.25D-06 0.72D-01 0.00033 212 0 8 FULL -0.14D-06 -0.14D-06 0.00D+00
```

Let us now have a look at the localized orbitals. The Pipek-Mezey criterion minimizes the number of centers spanned by each MO. We accordingly perform a Mulliken population analysis, asking only for the distribution of density amongst the three atoms. The menu file *atompop.inp* reads

We run the calculation using:

```
pam --inp=atompop --mol=CO2.xyz --put "loc_CO2.h5=CHECKPOINT.h5"
```

In the output we find:

```
*****
***** Mulliken population analysis *****
*****
```

```
Fermion ircop E1
-----
```

```
* Electronic eigenvalue no. 1: -19.228904376854 (Occupation : f = 1.0000)
=====
```

```
* Gross populations greater than 0.01000
```

```
Gross      Total      |      L 01 1
```

alpha	1.0000		1.0086
beta	0.0000		0.0000

* Electronic eigenvalue no. 2: -19.228897406456 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L 02 1
alpha	1.0000		1.0086
beta	0.0000		0.0000

* Electronic eigenvalue no. 3: -10.375624527418 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1
alpha	0.9997		1.0010
beta	0.0003		0.0000

* Electronic eigenvalue no. 4: -1.1701747687870 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L 02 1
alpha	0.9995		0.9988
beta	0.0005		0.0000

* Electronic eigenvalue no. 5: -1.1313373272171 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1	L 01 1
alpha	1.0000		0.4569	0.5497
beta	0.0000		0.0000	0.0000

* Electronic eigenvalue no. 6: -0.5710187998784 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1	L 02 1
alpha	1.0000		0.4569	0.5497
beta	0.0000		0.0000	0.0000

* Electronic eigenvalue no. 7: -0.5281308034633 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L 01 1
alpha	0.9995		0.9988
beta	0.0005		0.0000

* Electronic eigenvalue no. 8: -0.5229733977076 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1	L 01 1
alpha	1.0000		0.2282	0.7706
beta	0.0000		0.0000	0.0000

* Electronic eigenvalue no. 9: -0.5229733976932 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1	L 01 1
alpha	1.0000		0.2282	0.7706
beta	0.0000		0.0000	0.0000

* Electronic eigenvalue no. 10: -0.3795760457099 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1	L 02 1
alpha	1.0000		0.2282	0.7706
beta	0.0000		0.0000	0.0000

* Electronic eigenvalue no. 11: -0.3795760457064 (Occupation : f = 1.0000)

* Gross populations greater than 0.01000

Gross	Total		L C 1	L 02 1
alpha	1.0000		0.2282	0.7706
beta	0.0000		0.0000	0.0000

*** Total gross population ***

Gross	Total		L C 1	L 01 1	L 02 1
total	22.00000		5.62427	8.18536	8.18536

Simple observation shows that there are *three* and not two localized orbitals connecting carbon to each oxygen atom, suggesting *triple* and not double bonds. Before exploring this further, it is important to note that the orbitals energies given in the output are those of the canonical MOs. Localized MOs do not have orbital energies. However, to establish some energetic ordering we can calculate *approximate* orbital energies as expectation values of each localized orbital with respect to the Kohn-Sham matrix (for HF calculations one would use the Fock matrix).

Since we can not use molecular symmetry with the localized orbitals we first have to reconstruct the Kohn-Sham matrix for the '\$C_1'\$ case. We prepare the input file *scf0.inp*

Note that we set the number of SCF iterations to zero such that we only build the initial Kohn-Sham matrix with the input coefficients (no diagonalization). Next we run:

```
pam --inp=scf0 --mol=C02.xyz --put "loc_C02.h5=CHECKPOINT.h5" --get DFFCKT
```

where we recover the total Kohn-Sham matrix in the unformatted file DFFCKT. To see that our procedure actually works let us first calculate the expectation value of this matrix with respect to the original canonical MOs. Using the input file *exp0.inp*

we run:

```
pam --inp=expC --mol=C02.xyz --put "C02.h5=DFACM0.h5 DFFCKT"
```

Note that we use the keyword GENERAL_ACM0IN since we want to use the '\$C_1'\$ coefficients from the original DFT calculation with symmetry. At the end of the generated output file we then find back the orbital energies that are found in the Mulliken population analysis above:

Expectation value for individual orbitals

Matrix element	Occ.		
E1 1	-19.22890437800	2.0000	-38.45780875600
E1 2	-19.22889740759	2.0000	-38.45779481518
E1 3	-10.37562453182	2.0000	-20.75124906363
E1 4	-1.170174770392	2.0000	-2.340349540785
E1 5	-1.131337328982	2.0000	-2.262674657965
E1 6	-0.571018800591	2.0000	-1.142037601183
E1 7	-0.528130804300	2.0000	-1.056261608600
E1 8	-0.522973398844	2.0000	-1.045946797688
E1 9	-0.522973398844	2.0000	-1.045946797688
E1 10	-0.379576046966	2.0000	-0.759152093933
E1 11	-0.379576046966	2.0000	-0.759152093933
Total :	-108.0783738266 a.u. s0 = F t0 = F		

s0 = T : Expectation value zero by point group symmetry.
t0 = T : Expectation value zero by time reversal symmetry.

Now we do the same with the localized orbitals and combine this with a more detailed Mulliken population analysis. We set up the input file *exp.inp*

and run:

```
pam --inp=exp --mol=C02.xyz --put "loc_C02.h5=CHECKPOINT.h5 DFFCKT"
```

In the new Mulliken population analysis we still have the orbital energies of the canonical orbitals, but in the output below I have replaced these with the expectation values for the localized orbitals and also reordered the orbitals on energy. I have kept the original numbering, though. We now have:

* Electronic eigenvalue no.	7:	-17.16989574591	(Occupation : f = 1.0000)
=====			
* Gross populations greater than 0.01000			
Gross	Total		L A 01 s L A 01 pz
alpha	0.9995		0.9858 0.0130
beta	0.0005		0.0000 0.0000
* Electronic eigenvalue no.	4:	-17.14806997582	(Occupation : f = 1.0000)
=====			
* Gross populations greater than 0.01000			
Gross	Total		L A 02 s L A 02 pz
alpha	0.9995		0.9856 0.0132
beta	0.0005		0.0000 0.0000
* Electronic eigenvalue no.	3:	-10.33132604930	(Occupation : f = 1.0000)
=====			
* Gross populations greater than 0.01000			
Gross	Total		L A C s
alpha	0.9997		1.0010

beta	0.0003		0.0000						
* Electronic eigenvalue no. 2: -2.821559678558 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C s	L A 02 s	L A 02 pz				
alpha	1.0000		-0.0108	0.8393	0.1692				
beta	0.0000		-0.0000	0.0000	0.0000				
* Electronic eigenvalue no. 1: -2.799733777067 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C s	L A 01 s	L A 01 pz				
alpha	1.0000		-0.0108	0.8391	0.1694				
beta	0.0000		-0.0000	0.0000	0.0000				
* Electronic eigenvalue no. 5: -0.981751468005 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C s	L A C pz	L A C dzz	L A 01 s	L A 01 pz	L A 01 dzz	
alpha	1.0000		0.2676	0.1794	0.0284	0.0792	0.4654	0.0133	
beta	0.0000		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	
* Electronic eigenvalue no. 6: -0.981751327023 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C s	L A C pz	L A C dzz	L A 02 s	L A 02 pz	L A 02 dzz	
alpha	1.0000		0.2676	0.1794	0.0284	0.0792	0.4654	0.0133	
beta	0.0000		0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	
* Electronic eigenvalue no. 8: -0.451274722906 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C py	L A C dyz	L A 01 py				
alpha	1.0000		0.2034	0.0248	0.7677				
beta	0.0000		0.0000	0.0000	0.0000				
* Electronic eigenvalue no. 9: --0.451274722906 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C px	L A C dxz	L A 01 px				
alpha	1.0000		0.2034	0.0248	0.7677				
beta	0.0000		0.0000	0.0000	0.0000				
* Electronic eigenvalue no. 10: -0.451274722904 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C px	L A C dxz	L A 02 px				
alpha	1.0000		0.2034	0.0248	0.7677				
beta	0.0000		0.0000	0.0000	0.0000				
* Electronic eigenvalue no. 11: -0.451274722904 (Occupation : f = 1.0000)									
=====									
* Gross populations greater than 0.01000									
Gross	Total		L A C py	L A C dyz	L A 02 py				
alpha	1.0000		0.2034	0.0248	0.7677				
beta	0.0000		0.0000	0.0000	0.0000				

We can see that the first three orbitals are clearly the \$1s\$ orbitals of the three atoms of the molecule. Then comes two orbitals dominated by s-orbitals on the oxygens. Finally comes \$\sigma\$-orbitals from carbon to each oxygen atom (\$\angle\epsilon\angle=-0.982\ E_h\$), followed by pairs of \$\pi\$-orbitals to each oxygen (\$\angle\epsilon\angle=-0.451\ E_h\$)

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/nmr_uf3cl3/tutorial.md

orphan

Case study: NMR shielding of UF₃Cl₃

In this tutorial we shall go through the calculation of NMR shieldings of the UF₃Cl₃ molecule using the scheme of simple magnetic balance (SMB) in conjunction with London orbitals Olejniczak2012.

Preparing for the atomic start

To ensure a smooth convergence of the SCF calculation we employ an atomic start whereby an initial molecular density matrix is generated as the direct sum of atomic ones. We therefore have to perform a separate SCF calculation of each atomic type. At the moment the approach is somewhat cumbersome, but will be automatized in future versions of DIRAC. Each atom is calculated in maximal symmetry (linear symmetry) and the coefficients then exported to C1 symmetry using the keyword GENERAL_.ACMOUT.

Uranium atom

We calculate the U+3 cation in the [Rn] 5f³ configuration. The input file U.III.inp reads

```
U.III.inp
```

and the corresponding molecular input file U.mol is

```
U.mol
```

The minimal pam command is:

```
pam --inp=U.III.inp --mol=U.mol --get=DFACM0
```

We save the C1 coefficients and make a symbolic link for the atomic start:

```
mv DFACM0 ac.U.III
ln -s ac.U.III DFUXXX
```

Fluorine atom

We calculate the fluorine atom in the [He] 2s² 2p⁵ ground state configuration using fractional occupation. The input file F.inp reads

```
F.inp
```

and the corresponding molecular input file F.mol is

```
F.mol
```

The minimal pam command is:

```
pam --inp=F.inp --mol=F.mol --get=DFACM0
```

We save the C1 coefficients and make a symbolic link for the atomic start:

```
mv DFACM0 ac.F
ln -s ac.F DFFXXX
```

Chlorine atom

We calculate the chlorine atom in the [Ne] 3s² 3p⁵ ground state configuration using fractional occupation. The input file Cl.inp reads

```
Cl.inp
```

and the corresponding molecular input file Cl.mol is

```
Cl.mol
```

The minimal pam command is:

```
pam --inp=Cl.inp --mol=Cl.mol --get=DFACM0
```

We save the C1 coefficients and make a symbolic link for the atomic start:

```
mv DFACM0 ac.Cl
ln -s ac.Cl DFCLXX
```

Initial SCF calculation

Atomic Hartree–Fock start

In this case the atomic start at the PBE0 level did not converge. It is then recommend to proceed via a Hartree–Fock calculation where the HOMO-LUMO gap is larger. The HF menu file SCFatom.inp reads

```
SCFatom.inp
```

For the atomic start it can be noted that orbitals 1 through 43 of the uranium atom are fully occupied whereas the 5f orbitals (44 through 50) are given an fractional occupation of 3/14.

The corresponding molecular file UF3Cl3.mol reads

UF3Cl3.mol

The minimal pam command is:

```
pam --mol=UF3Cl3.mol --inp=SCFatom.inp --outcmo --copy="DFUXXX DFCLXX DFFXXX"
```

(I also added -mw=120 as well as -mpi=24)

Initial PBE0 run with restricted kinetic balance

Having good start vectors I next ran the PBE0 SCF calculation using the input SCF.inp:

SCF.inp

I am running with a level shift of 0.2 hartree as well as static overlap selection to ensure convergence. I also use the keyword GRID_.ATSIZE to determine the partitioning of the molecular volume into atomic ones. The minimal pam command reads:

```
pam --inp=SCF.inp --mol=UF3Cl3.mol --incmo --outcmo
```

This calculations converges smoothly in 20 iterations to:

TOTAL ENERGY

Electronic energy : -31794.704487527993

Other contributions to the total energy

Nuclear repulsion energy : 2038.659981883082
 SS Coulombic correction : 0.007588768574

Sum of all contributions to the energy

Total energy : -29756.036916876335

Property calculation using London orbitals and simple magnetic balance

Starting from the RKB coefficients we proceed to the actual calculation of NMR parameters using simple magnetic balance and London orbitals. The input file NMR.inp reads

NMR.inp

This calculations gives:

		isotropic shielding					
@		-----					
@atom		total	dia	para	skew	span	
-----		-----					
@U		-5418.3074	288.0612	-5706.3685	-0.1585	13571.5391	9641.0019
@Cl1	1	-1023.9194	67.4482	-1091.3676	0.8381	2875.6960	2759.3185
@Cl1	2	-1023.9194	67.4482	-1091.3676	0.8381	2875.6960	2759.3185
@Cl3		-885.2205	85.8218	-971.0423	0.7133	2705.2946	2511.3830
@F1	1	-690.9071	103.1746	-794.0817	0.8742	1380.6609	1337.2555
@F1	2	-690.9071	103.1746	-794.0817	0.8742	1380.6609	1337.2555
@F2		-787.3430	84.2708	-871.6137	0.7175	1704.0367	1583.6803
-----		-----					
@							

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/open_shell_scf/aoc.md

orphan

Average-of-configuration open-shell Hartree-Fock

This is a short introduction to the theory behind average-of-configuration open-shell Hartree-Fock as implemented in DIRAC. For a more complete description the reader may consult chapter 3 of the PhD thesis of Jørn Thyssen Thyssen2004 .

It should first be noted that there is no restricted open-shell Hartree-Fock (ROHF) code in DIRAC. The reason is that spin-orbit interaction couples spin and spatial degrees of freedom and make the formalism much more complicated since one cannot exploit spin symmetry alone for fixing the expansion coefficients in the reference configuration state function (CSF) which serves as the trial function.

Instead of optimizing the energy for a single open-shell state, we shall optimize the energy for a limited set of open-shell states.

Energy expression

Suppose that we have a set of N_{det} Slater determinants, constituting our N-particle basis. We next construct and diagonalize a CI matrix in this basis. This gives N_{det} solutions of the form

$$|\Psi_I\rangle = \sum_{P=1}^{N_{\text{det}}} |N_{\text{det}}\rangle_P \langle \Phi_P | \Psi_I \rangle C_{PI}$$

We will now find the set of orbitals which minimizes the average energy

$$E_{\text{av}} = \frac{1}{N_{\text{det}}} \sum_{I=1}^{N_{\text{det}}} \langle \Psi_I | H | \Psi_I \rangle$$

Inserting the expansion of the solutions in terms of Slater determinants and using the fact that the expansion coefficients C_{PI} are elements of a unitary matrix we obtain

$$E_{\text{av}} = \frac{1}{N_{\text{det}}} \sum_{P=1}^{N_{\text{det}}} \sum_{Q=1}^{N_{\text{det}}} \langle \Phi_P | H | \Phi_Q \rangle \sum_{I=1}^{N_{\text{det}}} C_{\text{PI}}^* C_{\text{QI}} = \frac{1}{N_{\text{det}}} \sum_{P=1}^{N_{\text{det}}} \langle \Phi_P | H | \Phi_P \rangle$$

showing that the average can also be taken over the N-particle basis itself.

Introducing open shells and active electrons

The above average energy expression is a functional of the orbitals entering the Slater determinants. We will make a distinction between :

- *inactive orbitals*, present in all Slater determinants, represented by indices $ijkl$
- *active orbitals*, present in some, but not all Slater determinants, represented by indices $uvxy$
- *secondary orbitals*, not present in any Slater determinant, represented by indices $abcd$

We shall also employ indices $pqrs$ for general orbitals.

In order to generate our N-particle basis for averaging we will distribute the orbitals into a number of shells. Each shell S is specified by M_S orbital and N_S electrons. Inactive and secondary shells have $N_S = M_S$ and $N_S = 0$, respectively, whereas active shells have $N_S < M_S$. We generate our N-electron basis by distributing all active electrons in all possible ways within their respective shells. The total energy can then be written in terms of orbitals rather than Slater determinants as

$$E_{\text{av}} = \sum_S f_S \left(\sum_{p \in S} \left(h_{pp} + \frac{1}{2} \sum_{S'} Q^{S'}_{pp} + \frac{1}{2} (a_S - 1) Q^S_{pp} \right) \right)$$

where we have introduced

- the fractional occupation $f_S = \frac{N_S}{M_S}$ of shell S
- the coupling coefficient a_S
 - $a_S = \frac{M_S}{N_S}$ for $N_S = M_S$
 - $a_S = 1$ for $N_S = 0$
- two-electron contributions $Q^S_{pq} = f_S \langle \sum_{r \in S} \langle pr | qr \rangle \rangle$

Orbital rotations

We will use an exponential parametrization for the rotation of orbitals

$$\tilde{\phi}_p = \sum_q \phi_q \exp(-\kappa)_{pq}$$

where κ is an anti-Hermitian matrix to ensure unitarity of the transformation. The exponential parametrization allows for unconstrained optimization (no Lagrange multipliers). It also allows the easy identification of redundant variational parameters, that is, parameters whose variation does not change the energy. In this particular case one finds that rotations within shells are redundant and the corresponding matrix elements κ_{pq} can be set to zero.

Gradient elements and off-diagonal blocks of the Fock matrix

The generally non-zero elements of the gradient vector are:

- inactive-secondary rotations:

$$g_{ia} = h_{ai} + \sum_{S'} Q^{S'}_{ai} F^I_{ai}$$

- active-secondary rotations:

$$g_{ua} = f_U \left[F^I_{au} + (a_U - 1) Q^U_{au} \right]; \quad u \in U$$

- inactive-active rotations:

$$g_{iu} = (1 - f_U) \left[F^I_{ui} + f_U \alpha_U Q^U_{ui} \right]; \quad u \in U$$

- inter-shell active-active rotations

$$g_{uv} = (f_U - f_V) F^I_{vu} - (\alpha_U - 1) f_U Q^U_{vu} - (\alpha_V - 1) f_V Q^V_{vu}; \quad u \in U, v \in V$$

where we have introduced

$$\alpha_S = \frac{1 - a_S}{1 - f_S}$$

These gradient elements allow the definition of the off-diagonal elements of the Fock matrix:

- inactive-secondary block:

$$\mathbb{F}_{ia} = F^I_{ia}$$

- active-secondary block:

$$\mathbb{F}_{ua} = F^I_{ua} + (a_U - 1) Q^U_{ua}; \quad u \in U$$

- inactive-active block:

$$\mathbb{F}_{iu} = F^I_{iu} + f_U \alpha_U Q^U_{iu}; \quad u \in U$$

- inter-shell active-active rotations $\langle U_{ne} V \rangle$:

```
\begin{aligned}
\mathbb{F}_{uv} &= \left\{ \begin{array}{l} F^I_{uv} + \frac{(a_U - 1)}{(f_U - f_V)} Q^U_{uv} + \frac{(a_V - 1)}{(f_V - f_U)} Q^V_{uv} \\ (a_U - 1) Q^U_{uv} + (a_V - 1) Q^V_{uv} \end{array} \right\} \text{for } f_U = f_V \\
&\end{aligned}
```

Diagonal blocks of the Fock matrix

The diagonal blocks of the Fock matrix are *a priori* not related to gradient elements and there is therefore freedom of choice in their specification. The specific choice will not affect the total energy, but will affect orbital energies as well as convergence of the AOC HF calculation.

In order to obtain a meaningful definition of the diagonal blocks of the Fock matrix we will consider an extension of Koopmans' theorem to average-of-configuration Hartree-Fock, that is, we consider average energy after removal of an electron from a specific shell T and using the same orbital set as for the original N -electron system.

The energy difference becomes

$$E^{N-1}_{av} - E^{N-1}_{av} = \frac{1}{M_T} \sum_{t \in T} \left[h_{tt} + \sum_S Q^S_{tt} + (a_T - 1) Q_{tt}^T \right]$$

If we now define the diagonal block of the Fock matrix corresponding to shell T as

$$\mathbb{F}_{pq} = h_{pq} + \sum_S Q^S_{pq} + (a_T - 1) Q_{pq}^T$$

the ionization potential associated with the electron removal becomes

$$IP = E^{N-1}_{av} - E^N_{av} = -\frac{1}{M_T} \sum_{t \in T} \epsilon_t$$

In the case of a degenerate shell we simply find

$$IP = E^{N-1}_{av} - E^N_{av} = -\epsilon_t, \quad t \in T$$

identical to the original Koopmans' theorem, whereas one in the general case gets an average over the orbital energies of the shell.

Based on these observations we define the diagonal blocks of the AOC Fock matrix as

- inactive-inactive block:

$$\mathbb{F}_{ij} = F^I_{ij}$$

- secondary-secondary block:

$$\mathbb{F}_{ab} = F^I_{ab}$$

- active-active block:

$$\mathbb{F}_{uv} = F^I_{uv} + (a_U - 1) Q^U_{uv}; \quad u, v \in U$$

These are the definitions employed in DIRAC12 and onwards (and also the definition found in the thesis of Jørn Thyssen Thyssen2004). In previous versions the term $(a_U - 1) Q^U_{uv}$ was missing from the active-active block. Since Q^U_{uu} is positive and for an open shell $a_U < 1$ removal of this term tend to shift orbital energies of the open shell upwards.

Convergence problems typically occur when orbital energies between shells have similar values such that the selection of occupied orbitals for the construction of the Fock matrix becomes ambiguous. In a closed-shell system this will for instance happen when the HOMO-LUMO gap closes. The definition of the active-active block in pre-DIRAC12 version (which was in fact an unintended omittal) can in some instances lead to improved convergence. More specifically, this happens when the orbitals of an open shell and the closed shell (or another open shell) are almost degenerate. However, such situations are often symptomatic for a wrong choice of partitioning of orbitals into closed and open shells. Furthermore, the definition of the active-active block in pre-DIRAC12 versions tend to close the HOMO-LUMO gap which may hamper convergence.

Level shift

Whenever there is almost degeneracy of orbitals between different shells the recommended strategy is to exploit the freedom in the definition of diagonal blocks of the Fock matrix and introduce a *level shift* λ , that is

$$F^U_{uv} \rightarrow F^U_{uv} + \lambda \delta_{uv}$$

The level shift of secondary (virtual) orbitals is controlled by the keyword SCF_.LSHIFT, whereas open shells can be shifted using the keyword SCF_.0LEVEL.

Convergence issues

Open-shell systems tend to be more difficult to converge than closed-shell ones, because of additional orbital classes and more possibilities of near-degeneracies between orbital classes. It is important to understand that DIRAC will generally order orbitals according to energy. Furthermore, DIRAC starts by filling closed-shell orbitals, then open-shell ones. In the case of Uranium ($[Rn]5f^36d^47s^2$) the closed-shell $7s$ orbitals will have higher orbital energies than the $6d$ open-shell ones, and this may lead to convergence problems. Fortunately, since DIRAC21 convergence of open-shell atoms is unproblematic thanks to atomic supersymmetry, see keyword SCF_.KPSELE.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/open_shell_scf/converging_atoms.md

orphan

Converging atoms

DIRAC presently does not perform an open-shell Hartree–Fock in a strict sense, rather an average-of-configuration calculation, which amounts to the optimization of the average energy of a set of configurations (or determinants) generated from the specification of a given number of open shells and their electron occupations. In the case of atoms, a convergence problem can occur when the inner open-shell orbitals are more stable than outer “closed” shell. From DIRAC21 onwards, we have atomic supersymmetry and can specify the occupation number of each κ value. by using the keyword SCF_ κ SELE.

As examples of such calculations, we consider the Nb and Np atoms.

----- Example 1: Nb [Kr]4d⁴5s¹----- :

```

**DIRAC
.WAVE FUNCTION
.ANALYZE
**HAMILTONIAN
.X2C
**INTEGRALS
*READIN
.UNCONTRACT
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
18 18
.OPEN SHELL
2
4/10,0
1/2,0
.KPSELE
5
-1 1 -2 2 -3
8 6 12 4 6
0 0 0 4 6
2 0 0 0 0
**ANALYZE
.MULPOP
.MULPOP
.VECPPOP
1..00
1..00
**MOLECULE
*BASIS
.DEFAULT
dya11.2zp
*END OF

```

----- Example 2: Np [Rn]5f⁴6d¹7s²----- :

```

**DIRAC
.WAVE FUNCTION
.ANALYZE
**HAMILTONIAN
.X2C
**INTEGRALS
*READIN
.UNCONTRACT
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
44 44
.OPEN SHELL
2
4/0,14
1/10,0
.KPSELE
7
-1 1 -2 2 -3 3 -4
14 10 20 12 18 6 8
0 0 0 0 0 6 8
0 0 0 4 6 0 0
**ANALYZE
.MULPOP
.MULPOP
.VECPPOP
1..00
1..00
**MOLECULE
*BASIS
.DEFAULT
dya11.2zp
*END OF

```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/openfermion/openfermion-dirac.md

orphan

Openfermion-Dirac

OpenFermion is a package for Quantum simulation. Dirac has been interfaced with OpenFermion through a python interface, that can be downloaded here <https://github.com/bsenjean/Openfermion-Dirac/>. A jupyter-notebook tutorial is accessible at <https://github.com/bsenjean/Openfermion-Dirac/tree/master/tutorial>.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/pcm/pcm_basics.md

orphan

Polarizable continuum model: some basic remarks

The possibility to perform solvent calculations is currently under development in the pcm branch. We will exploit a *Continuum Solvation Model* (CSM) namely the *Polarizable Continuum Model* (PCM). PCM is a *focused model*: the solute (a single molecule or a cluster containing the solute and some relevant solvent molecules) is described quantum mechanically, while the solvent is approximated as a structureless continuum whose interaction with the solute is mediated by its permittivity, ϵ .

The solute is accomodated inside a molecular cavity, built as a set of interlocking spheres centered on the atoms constituting the molecule under investigation. The current implementation in DIRAC is limited to an SCF description of the solute. For a more in-depth presentation of the PCM, please refer to Tomasi2005, Mennucci2007 and references therein. For a presentation of the details of the implementation in DIRAC, please refer to DiRemigio2015.

Basic Theory

In CSMs we write the Schrödinger equation for the solute as:

$$\left[H_0 + V_{\sigma}(\rho) \right] \psi = E \psi$$

where H_0 is the solute Hamiltonian *in vacuo* and $V_{\sigma}(\rho)$ is the solute-solvent interaction potential, **which depends on the solute wavefunction** through the first-order density matrix. This introduces a *nonlinearity* which must be dealt with appropriately. In standard molecular electronic-structure theory we write the energy as the expectation value of the Hamiltonian (ψ being a trial wave function):

$$\langle \psi | H_0 + V_{\sigma}(\rho) | \psi \rangle$$

An upper-bound estimate to the exact energy of the system can then be obtained by a variational procedure from this functional. When introducing a nonlinearity of the type above, the standard functional **does not** lead to an upper-bound estimate to the exact energy upon minimization. The appropriate functional is instead given by:

$$\langle \psi | H_0 + \frac{1}{2} V_{\sigma}(\rho) | \psi \rangle$$

which has the status of a *free energy* (an extensive justification of this fact is given in Tomasi1994).

Thus in the PCM framework, the basic energetic quantity is the *free energy of solvation* which is conveniently partitioned as follows (Mennucci2007, Tomasi2005, Amovilli1998):

$$\Delta G_{\text{sol}} = \Delta G_{\text{el}} + G_{\text{cav}} + G_{\text{dis}} + G_{\text{rep}} + \Delta G_{\text{Mm}}$$

where:

- ΔG_{el} accounts for the *electrostatic* solute-solvent interaction, arising from mutual polarization in the charge distributions;
- G_{cav} is the *cavitation* energy, needed to form the molecular cavity inside the continuum representing the solvent;
- G_{dis} is the *dispersion* energy, due to the solute-solvent dispersion interactions;
- G_{rep} is the *repulsion* energy, which accounts for Pauli repulsion;
- ΔG_{Mm} is due to molecular motion and accounts for *entropic* contributions to the free energy.

The electrostatic term is, usually, the largest contribution to the solvation energy. Thus we will **exclusively** be concerned with its calculation (see also Amovilli1998 for a discussion of the other terms).

[!WARNING] The non-electrostatic terms are **not implemented** in DIRAC.

In CSMs the calculation of this term requires the solution of the classical Poisson problem nested within the QM calculation. We will define the expectation value of the solvent operator $V_{\sigma}(\rho)$ as the *polarization energy*:

$$U_{\text{pol}} = \langle \psi | V_{\sigma}(\rho) | \psi \rangle$$

Electrostatic term in the Polarizable Continuum Model

In the PCM, the solute-solvent electrostatic interaction is represented by an *apparent surface function* (ASC) σ spread on the cavity surface. Using the integral equation formulation of the Poisson problem, the apparent surface charge is obtained by solving an appropriate integral equation relating σ to the molecular electrostatic potential (MEP) φ evaluated on the cavity surface:

$$\epsilon \sigma = -\varphi$$

The integral operators ϵ and φ are defined as follows:

$$\epsilon = \left(2\pi \frac{\epsilon + 1}{\epsilon - 1} - \mathcal{D} \right) \mathcal{S}$$

$$\varphi = 2\pi - \mathcal{D}$$

in terms of components of the Calderon projector (see Tomasi2005) This equation can be solved numerically by discretization of the cavity surface with a triangular mesh (the finite elements being called *tesserae*). The solution of the electrostatic problem then amounts to solving a linear system of equations:

$$\mathbf{T}\mathbf{q} = -\mathbf{R}\mathbf{v} \rightarrow \mathbf{q} = \mathbf{K}\mathbf{v}$$

where $\mathbf{v}$ and $\mathbf{q}$ are vectors of dimension equal to the number of finite elements. They contain the MEP and the ASC, respectively, sampled at the finite elements centroids. The polarization energy can be expressed as the scalar product of the MEP and ASC sampled at the cavity boundary:

$$U_{\text{pol}} = \mathbf{q} \cdot \mathbf{v}$$

If we split the MEP into its nuclear and electronic parts $\mathbf{v} = \mathbf{v}^{\text{N}} + \mathbf{v}^{\text{e}}$, due to the linearity of Poisson equation, the same separation can be achieved for the ASC $\mathbf{q} = \mathbf{q}^{\text{N}} + \mathbf{q}^{\text{e}}$. Exploiting this separation the polarization energy can be rewritten as:

$$U_{\text{pol}} = U_{\text{NN}} + U_{\text{Ne}} + U_{\text{eN}} + U_{\text{ee}} = U_{\text{NN}} + 2U_{\text{eN}} + U_{\text{ee}}$$

where U_{xy} ($x, y = \text{N}, \text{e}$) is the interaction energy between the x charge distribution and the y -induced ASC. We also exploited the self-adjointness of $\mathbf{K}$ to get $U_{\text{Ne}} = U_{\text{eN}}$.

PCM-SCF

Nesting the PCM inside an SCF calculation requires the calculation of the MEP and ASC at cavity points at every SCF iteration and the update of the Fock matrix to account for the effect of the mutual solute-solvent polarization. The “solvated” Fock matrix is written as:

$$f_{pq} = f_{pq}^{\text{vac}} + \mathbf{q} \cdot \mathbf{v}_{pq}^{\text{e}}$$

The PCM matrix elements are more explicitly given as:

$$\mathbf{q} \cdot \mathbf{v}_{pq}^{\text{e}} = \sum_{\text{I}}^{\text{N}_{\text{ts}}} q_{\text{I}} v_{pq, \text{I}}^{\text{e}}$$

$$v_{pq, \text{I}}^{\text{e}} = \left\langle \phi_{\text{p}} \left| \frac{-1}{|\mathbf{r} - \mathbf{s}_{\text{I}}|} \right| \phi_{\text{q}} \right\rangle$$

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/pcm/pcm_response.md

orphan

Calculation of electric properties in solution: dipole moments and polarizabilities

The current implementation of the PCM in DIRAC allows for the calculation of electric properties up to second order. This means that electric dipole moments and electric dipole polarizabilities are available.

The analytical results can also be checked against a finite-field approach. The Hamiltonian defined in the input the Hamiltonian definition is augmented by an explicit electric field. Repeating the calculation with different coupling strengths and electric field directions, will enable the use of a stencil formula for the finite-difference calculation of energy derivatives, for example using a [Five-point stencil](#):

$$\begin{aligned} f'(x) &\simeq \frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h} \\ f''(x) &\simeq \frac{-f(x+2h) + 16f(x+h) - 30f(x) + 16f(x-h) + f(x-2h)}{12h^2} \end{aligned}$$

Electric dipole moments

We recall that the component μ_I of the electric dipole moment is defined as the first derivative, at zero perturbation strength, of the energy with respect to the component μ_I of an applied external electric field $\mathbf{F}$.

Thanks to the Hellmann-Feynman theorem, this reduces to the calculation of the expectation value of the component μ_I of the electric dipole moment operator:

$$\mu_I = -\left\langle \frac{\partial E(\mathbf{F})}{\partial F_I} \right\rangle_{\mathbf{F}=0} = \langle 0 | \hat{\mu}_I | 0 \rangle$$

Calculation of dipole moments with PCM is quite straightforward. We just need to add the HAMILTONIAN_PCM keyword to a dipole moment calculation input file:

```
**DIRAC
.WAVE FUNCTION
.PROPERTIES
**HAMILTONIAN
.PCM
**WAVE FUNCTION
.SCF
**PROPERTIES
.DIPOLE
*END OF
```

and of course use the appropriate PCMSolver input file:

```
Units = Angstrom
Medium {
  SolverType = CPCM
  Solvent = Cyclohexane
}
```



```
Cavity {
    Type = GePol
    Area = 10.0
    Mode = Explicit
    Spheres = [0.0, 0.0, 0.0, 20.0]
}
```

This PCMSolver input file requires the CPCM model with cyclohexane as solvent. The solute is enclosed in a cavity made from a single sphere, centered at the origin and with a 20 angstrom radius. Consider the following .mol file for water as an example:

```
INTGRL
Water

C 2 0
    8. 1
    .0000000000 0.0000000000 -.2249058930
LARGE BASIS cc-pVDZ
    1. 2
H 1.4523499293 .0000000000 .8996235720
H -1.4523499293 .0000000000 .8996235720
LARGE BASIS cc-pVDZ
FINISH
```

The output for the dipole moment calculation looks just like usual:

```
*****
***** Expectation values *****
*****

s0 t0

-----
Dipole length: X : 0.63076018E-12 a.u. F F
Dipole length: Y : -0.20258107E-15 a.u. F F
Dipole length: Z : 0.813178997624 a.u. F F
-----

s0 = T : Expectation value zero by point group symmetry.
t0 = T : Expectation value zero by time reversal symmetry.
-----

* Dipole moment:

      Electronic      Nuclear      Total
      contribution   contribution   contribution
-----
x      0.00000000 Debye    0.00000000 Debye    0.00000000 Debye
y     -0.00000000 Debye    0.00000000 Debye   -0.00000000 Debye
z      2.06691398 Debye    0.00000000 Debye    2.06691398 Debye
-----
x      0.00000000 a.u.     0.00000000 a.u.     0.00000000 a.u.
y     -0.00000000 a.u.     0.00000000 a.u.    -0.00000000 a.u.
z      0.81317900 a.u.     0.00000000 a.u.     0.81317900 a.u.
-----

1 a.u = 2.54177000 Debye
```

Electric dipole moment polarizabilities

We recall that the electric dipole polarizability is a second order property as is defined as the second derivative, at zero field strength, of the energy:

$$\alpha_{IJ} = -\left.\frac{\partial^2 E(\mathbf{F})}{\partial F_I \partial F_J}\right|_{\mathbf{F}=0}$$

In a response theory framework, the polarizability tensor elements can be identified from the linear response function, see Saue2002a:

$$\alpha_{IJ} = -\langle\langle \mu_I; \mu_J \rangle\rangle$$

The polarizability is a rank-2 tensor and has six unique components. We can extract two invariant quantities from the tensor: the isotropic and anisotropic polarizability:

$$\alpha_{\text{iso}} = \frac{1}{3}(\alpha_{xx} + \alpha_{yy} + \alpha_{zz})$$

$$\alpha_{\text{aniso}} = \frac{1}{\sqrt{2}}\sqrt{(\alpha_{xx} - \alpha_{yy})^2 + (\alpha_{xx} - \alpha_{zz})^2 + (\alpha_{yy} - \alpha_{zz})^2}$$

Again, we just need to add the PCM section to an electric dipole polarizability calculation input file:

```
**DIRAC
.PROPERTIES
**HAMILTONIAN
.PCM
**WAVE FUNCTION
.SCF
**PROPERTIES
.POLARIZABILITY
*END OF
```

the same .mol file as above for water is used in this example. The following PCMSolver input file is used:

```
Units = Angstrom
Medium {
    Solvent = Water
}

Cavity {
    Type = GePol
    Area = 10.0
```

```

    Mode = Implicit
}

```

The output for the dipole moment calculation looks just like usual:

```

+-----+
! Electric dipole polarizability !
+-----+

```

@ Elements of the electric dipole polarizability tensor

```

@  xx      7.76376447 a.u.   (converged)
@  yy      3.21397401 a.u.   (converged)
@  zz      5.57773363 a.u.   (converged)

```

```

@  average  5.51849070 a.u.
@  anisotropy 3.941      a.u.

```

```

@  xx      1.15047118 angstrom**3
@  yy      0.47626180 angstrom**3
@  zz      0.82653484 angstrom**3

```

```

@  average  0.81775594 angstrom**3
@  anisotropy 0.584      angstrom**3

```

Influence of the relative permittivity on electric properties

It is interesting to analyze the effect of varying the relative permittivity of the solvent on the calculation of electric properties. We will use the two input files strategy and exploit the string substitution mechanism of the pam script. Consider the pcm-prop.inp input file:

```

**DIRAC
.WAVE FUNCTION
.PROPERTIES
.PCM
*PCM
*PCMSOLVER
.CAVTYPE
.GEPOL
.NOSCALING
.AREATS
10
.SOLVERTYPE
IEFFPCM
.SOLVNT
solvent
**WAVE FUNCTION
.SCF
**PROPERTIES
.DIPOLE
.POLARIZABILITY
*END OF

```

Notice that instead of specifying a solvent name we put a placeholder string. This can be exploited to run calculations selecting a different solvent:

```
pam --inp=pcm-prop.inp --replace solvent=CYCLOHEXANE --mol=H2O.mol
```

or run this in a loop:

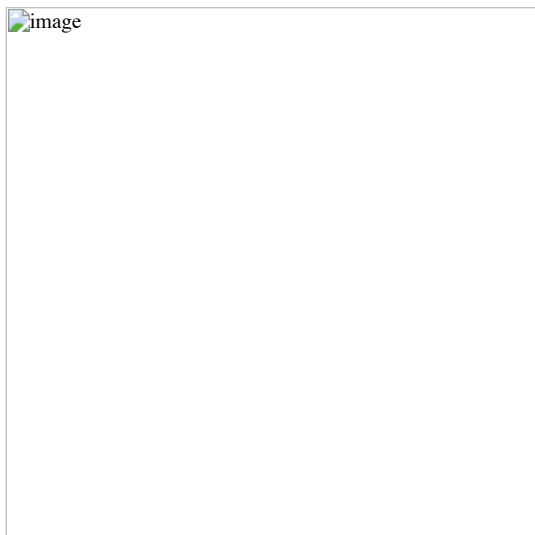
```

# Declare array of solvents. Solvents are ordered by increasing relative permittivity
declare -a solv_array=("N-HEPTANE" "CYCLOHEXANE" "CARBON TETRACHLORIDE" "BENZENE" "TOLUENE" "CHLOROFORM" "CHLOROBENZENE" "ANILINE" "TETRAHYDR

# Now loop over the array elements and substitute them to the string solvent in pcm-prop.inp
for solv in "${solv_array[@]}"
do
    echo "$solv"
    pam --inp=pcm-prop.inp --replace solvent=$solv --mol=H2O.mol
done

```

This will produce a different output file for every different solvent in the array. One can collect the results for dipole moments and isotropic polarizabilities and plot them against the relative permittivities. This was done in DiRemigio2015 for the H_2Po molecule:



where it was also found that the difference between the values of the observables in solution and in vacuo can be fitted to a linear rational function:

$$\alpha_{\mathrm{iso}}(\varepsilon_{\mathrm{r}}) - \alpha_{\mathrm{iso}}(1) = \frac{\varepsilon_{\mathrm{r}} - 1}{a\varepsilon_{\mathrm{r}} + b} \alpha_{\mathrm{iso}}$$

An analogous relation was found to be valid also for the dipole moment. Notice that the \$a, b\$ coefficients are system-dependent.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/pcm/pcm_scf.md

orphan

Hartree-Fock and DFT calculations in solution with the polarizable continuum model

This tutorial will cover the set up of an Hartree-Fock and a DFT calculation using the polarizable continuum model (PCM) to include solvent effects. The implementation of the PCM-SCF algorithm in DIRAC is documented in the recent paper DiRemigio2015, to which the reader is referred for further technical details. The current implementation makes use of an external module for the PCM-related tasks. The module needs some input information to run properly.

As explained in the *PCMSOL section of the reference manual, a PCM calculation can be set up in two ways:

- using **three** input files. The additional input file contains directives for the PCMSolver module;
- using **two** input files. The additional directives in the inp file are forwarded to the PCMSolver module.

The second method is not as flexible as the first one and we will cover both in this tutorial.

For further details, please refer to the PCM documentation in the reference manual. Notice that the fact that a working implementation of the PCM-SCF algorithm gives access to geometry optimizations based on a numerical gradient and finite-field approaches to the calculation of response properties. The input files for DIRAC in these two cases need only be modified to include the directives relevant for the PCM calculation.

We will use the extremely simple examples contained in the test/pcm_energy directory. They are more than enough to show a simple workflow, possible gotchas and what to look for in a PCM output.

[!WARNING] Calculations with the polarizable continuum model **cannot** exploit molecular point group symmetry and/or 2-component Hamiltonians.

The .mol file is prepared in the exact same manner as for an in vacuo calculation. In all the cases examined in the following, we will use the methane molecule with a STO-3G basis set:

```
INTGRL
methane (a standard MOLFDIR test at that geometry)
nonrelativistic contracted STO-3G basis
C 2 0
    6. 1
C 0.0000 0.0000 0.0000
LARGE BASIS STO-3G
    1. 4
H .629889144 .629889144 .629889144
H -.629889144 -.629889144 .629889144
H .629889144 -.629889144 -.629889144
H -.629889144 .629889144 -.629889144
LARGE BASIS STO-3G
FINISH
```

An Hartree-Fock calculation using three input files

In this example, the first input method will be used. We have to prepare three input files. The .inp file will feature some additional keywords:

```

**DIRAC
.WAVE FUNCTION
**HAMILTONIAN
.LEVY-LEBLOND
.PCM
*PCM
.SEPARATE
.PRINT
6
**WAVE FUNCTION
.SCF
*SCF
.PRINT
1
*END OF

```

The .PCM keyword enables the addition of solvent terms to the Fock matrix. Details can be controlled through directives under the *PCM section (to which the reader is referred for further details) Using this input file a calculation using the Levy-Leblond Hamiltonian with PCM will be performed. Notice that the nuclear and electronic molecular electrostatic potential (MEP) and apparent surface charge (ASC) will be treated separately (.SEPARATE keyword) The additional input for the PCMSolver module is:

```

Units = AU
Medium {
    Solvent = Water
}
Cavity {
    Type = GePol
    Area = 357.106482748
    Scaling = False
    Mode = Implicit
}

```

The syntax for the PCMSolver module input is documented [here](#) it is here important to note that the input is specified in atomic units and that the area parameter selects an extremely coarse tessellation. Such a huge value should never be used in production calculations! The above input disables scaling of the radii by the factor 1.2 This is only for testing purposes. In production calculations the scaling by 1.2 should be **always** included.

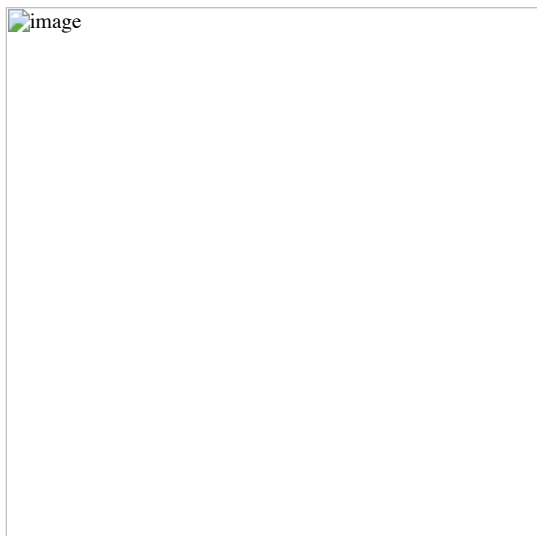
Finally, the calculation can be run by issuing:

```
pam --inp=levy.inp --mol=CH4.mol --pcm=pcmsolver.inp
```

The output file will be named levy_CH4_pcmsolver.out and the corresponding archive levy_CH4_pcmsolver.tgz The archive contains the additional files:

- cavity.off, an input to the [GeomView program](#) to visualize the cavity;
- cavity.npz, a compressed NumPy array file. This can be given as input to PCMSolver to generate a restart cavity. Refer to the PCMSolver documentation for details;
- PEDRA.OUT, a detailed report from the cavity generator;
- PCM_mep_asc, this lists the values of the **converged** MEP and ASC at cavity points.

From the cavity.off file one may generate the following plot. Notice how coarse the tessellation is:



Let us know highlight the additional information contained in the output:

- a summary of the setup of the PCM calculation is printed out when the Hamiltonian definition is completed:

```

===== Polarizable Continuum Model calculation set-up =====
* Polarizable Continuum Model using PCMSolver external module:
  . Converged potentials and charges at tesserae representative points written on file.
  . Calculate the SS block of the electrostatic potential matrix.
  . Separate potentials and apparent charges in nuclear and electronic.
  . Form One-Index Transformed ASC in a Linear Response calculation.
  . Print potentials at tesserae representative points.

* PCMSolver, an API for the Polarizable Continuum Model electrostatic problem. Version 1.0.0
  Main authors: R. Di Remigio, L. Frediani, K. Mozgawa
  With contributions from:

```

R. Bast (CMake framework)
 U. Ekstroem (automatic differentiation library)
 J. Juselius (input parsing library and CMake framework)
 Theory: - J. Tomasi, B. Mennucci and R. Cammi:
 "Quantum Mechanical Continuum Solvation Models", Chem. Rev., 105 (2005) 2999
 PCMSolver is distributed under the terms of the GNU Lesser General Public License.

- the PCMSolver module is initialized. The cavity is formed, discretized and the PCM matrix is created. A report is printed out:

```
~~~~~ PCMSolver ~~~~~
Using CODATA 2010 set of constants.
Input parsing done API-side
===== Cavity
Cavity type: GePol
Average area = 357.106 AU^2
Probe radius = 2.61727 AU
Number of spheres = 5 [initial = 5; added = 0]
Number of finite elements = 80
===== Solver
Solver Type: IEFFPCM, isotropic
PCM matrix hermitivized
===== Medium
Medium initialized from solvent built-in data.
Solvent name: Water
Static permittivity = 78.39
Optical permittivity = 1.776
Solvent radius = 1.385
.... Inside
Green's function type: vacuum
.... Outside
Green's function type: uniform dielectric
Permittivity = 78.39
```

The parameters chosen in the PCMSolver input are printed out. The number of finite elements generated for the cavity surface are also reported. The final section reports the value for the dielectric constant of the medium.

- at each iteration, the time spent in calculating the MEP and ASC is printed out:

```
* NucMEP evaluation (CPU): 0.00000000s
* NucASC evaluation (CPU): 0.00000000s
* EleMEP evaluation (CPU): 0.00265500s
* EleASC evaluation (CPU): 0.00000000s
```

Since the .SEPARATE keyword was given, nuclear and electronic MEP and ASC are handled separately.

- at each iteration, the MEP and ASC at cavity points are printed out. This is because the printlevel was set to 6 Here it's just a sample of the printout:

MEP and ASC at iteration	2			
Finite element #	Nuclear MEP	Nuclear ASC	Electronic MEP	Elect
1	3.124762515511	-0.279550006743	-3.115288065165	0.2774666
2	3.124762515511	-0.279550006743	-3.115288065165	0.2774666
3	3.124762515511	-0.279550006743	-3.115288065165	0.2774666
4	3.100728688114	-0.295176115078	-3.099377710696	0.2967369

- the polarization energy components are also reported:

```
Polarization energy components
U_ee = -30.13519885926173 , U_en = 30.18510529332953 , U_ne = 30.18510529332955 , U_nn = -30.23554041057662
```

together with a more comprehensive report on the energy at the current iteration:

Output from ERGCAL

```
-----
Total electronic energy      : -62.564643386729145
Nuclear potential energy    : 25.365934555796475
Polarization energy         : -0.000264341589633
Total energy                 : -37.198973172522301
```

the polarization energy reported from ERGCAL is the sum of the componets divided by 2.

- at the end of each iteration the time spent in forming the PCM Fock matrix contribution is reported:


```
* PCM Fock matrix contribution (CPU): 0.00269899s
```
- when the calculation is converged, the final energy report lists also the solvation energy, which is the sum of the polarization energy components divided by 2. The total energy already includes this contribution:

```
TOTAL ENERGY
-----
Electronic energy           : -62.567437447656800
Other contributions to the total energy
Nuclear repulsion energy    : 25.365934555796475
Solvation energy            : -0.000171652208014
Sum of all contributions to the energy
Total energy                 : -37.201674544068339
```

A nonrelativistic Hartree-Fock calculation using two input files

In this tutorial, the input to the PCMSolver module will be provided within the DIRAC input file by adding the *PCMSOLVER section. Read the documentation for this section in the reference manual for further details. Notice that this PCMSolver input is identical to the one given above in the separate file:

```
**DIRAC
.WAVE FUNCTION
**HAMILTONIAN
.NONREL      ! activates also point nuclei
*PCM
.PRINT
6
*PCMSOLVER
.CAVTYPE
.GEPOL
.NOSCALING
.AREATS
100
.SOLVERTYPE
.IEFPCM
.SOLVNT
.WATER
**WAVE FUNCTION
.SCF
*SCF
.PRINT
1
*END OF
```

The calculation is now run with:

```
pam --inp=nonrel.inp --mol=CH4.mol
```

The PCMSolver report now is given by:

```
~~~~~ PCMSolver ~~~~~
Using CODATA 2010 set of constants.
Input parsing done host-side
===== Cavity
Cavity type: GePol
Average area = 357.106 AU^2
Probe radius = 2.61727 AU
Number of spheres = 5 [initial = 5; added = 0]
Number of finite elements = 80
===== Solver
Solver Type: IEFPCM, isotropic
PCM matrix hermitivitized
===== Medium
Medium initialized from solvent built-in data.
Solvent name:      Water
Static permittivity = 78.39
Optical permittivity = 1.776
Solvent radius =    1.385
.... Inside
Green's function type: vacuum
.... Outside
Green's function type: uniform dielectric
Permittivity = 78.39
```

the only difference being that the module reports that input parsing has been done host-side. Since we didn't specify the .SEPARATE keyword the total MEP and ASC are reported at tesserae centers for each iteration. The polarization energy components are no longer showed.

A Dirac-Coulomb DFT calculation

In this tutorial we will perform a DFT calculation using the Dirac-Coulomb Hamiltonian and the LDA functional:

```
**DIRAC
.WAVE FUNCTION
**HAMILTONIAN
.DFT
.LDA
.PCM
*PCM
.PRINT
6
**WAVE FUNCTION
.SCF
*SCF
.PRINT
1
*END OF
```

We use the three files input strategy, as we want to customize both the cavity formation and the solvent selection:

```
Units = Angstrom
Medium {
  SolverType = CPCM
  ProbeRadius = 1.385
  Solvent = Explicit
  Green<inside> {Type=Vacuum}
  Green<outside> {
    Type = UniformDielectric
    Eps = 15.04
  }
}
```

```
Cavity {
    Type = GePol
    Area = 100.0
    Mode = Atoms
    Atoms = [1]
    Radii = [1.55]
}
```

Instead of using the IEFPCM we ask for the CPCM (Conductor-PCM), the solvent is also explicitly specified. The module is currently limited to the handling of homogeneous, uniform, isotropic dielectrics. The cavity is also customized. We use the atomic positions provided in the .mol file, but instead of using the built-in value of 1.70 angstrom for the radius of atom 1 (carbon) we use 1.55 angstrom. The calculation is run as usual and there is nothing more to highlight about the output in this case. Notice that the solvent and solver specification could also be given using the two file input strategy. The specification of custom cavities is **only** possible with the three file input strategy.

A Dirac-Coulomb, Hartree-Fock, PCM-SCC calculation

As explained in DiRemigio2015, the calculation of the molecular electrostatic potential integrals:

$$v_{\{\kappa\lambda\}}(\mathbf{s}_I) = \int \mathrm{d}\mathbf{r} \frac{-\omega_{\{\kappa\lambda\}}(\mathbf{r})}{|\mathbf{r} - \mathbf{s}_I|}$$

needed for the PCM can be sped up by neglecting the Small-Small block and replacing the SS block of the potential by a simple Coulombic correction Visscher1997a. This approximation is named PCM-SCC and the user can exploit it by specifying the .SKIPSS keyword in the *PCM section of the input:

```
**DIRAC
.WAVE FUNCTION
**HAMILTONIAN
.PCM
*PCM
.SKIPSS
.PRINT
6
**WAVE FUNCTION
.SCF
*SCF
.PRINT
1
*END OF
```

The PCM-SCC approximation makes sense only when the Dirac-Coulomb Hamiltonian is employed, Notice that **by default** these integrals are never neglected in a PCM calculation. The user should exercise some care in using this option and test, when feasible, the impact on results of the PCM-SCC approximation.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/polarizable_embedding/pe_response.md

orphan

PE-TDDFT calculations of excitation energies and solvent shifts

In this tutorial we investigate property calculations in solution through the polarizable embedding (PE) model (Olsen2010, Olsen2011). The tutorial builds on the implementation reported in Hedegaard2017 which uses [PElib](#) to calculate the environment contributions. This implementation allows calculation of electric properties up to linear response (this corresponds to electric properties up to second order), including an explicit environment. Examples include electric dipole moments and polarizabilities. In addition, the linear response module allows calculation of excitation energies and transition moments, including the effect of a surrounding environment through the PE model. Here we focus on TDDFT including a solvent. A few comments regarding calculations of environment effect on dipole moments and polarizabilities are given in the end of this tutorial.

TDDFT including an explicit environment through the PE model

The generalized eigenvalue problems that is solved when solving the response equations

$$\left(E^{[2]} - \hbar\lambda S^{[2]}\right)X_m = 0$$

which is completely equivalent to what is done in a vacuum calculation. The main difference is in the electronic Hessian, $E^{[2]}$, which is modified by the presence of an environment (cf. Eqs. 36-38) in Hedegaard2017. The Hessian has the structure

$$E^{[2]} = \left(\begin{array}{cc} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^* & \mathbf{A}^* \end{array} \right),$$

with matrix elements

$$\begin{aligned} A_{\{ai;bj\}} &= \langle 0 | \hat{q}_{\dagger ai}, \hat{q}_{bj}, \hat{f}_0 + \hat{v}^{\mathrm{PE}} | 0 \rangle + \langle 0 | \hat{q}_{\dagger ai}, \hat{v}^{\mathrm{ind}} | 0 \rangle \langle 0 | \text{Hessian}_{2a} | 0 \rangle \\ B_{\{ai;bj\}} &= \langle 0 | \hat{q}_{ai}, \hat{q}_{bj}, \hat{f}_0 + \hat{v}^{\mathrm{PE}} | 0 \rangle + \langle 0 | \hat{q}_{ai}, \hat{v}^{\mathrm{ind}} | 0 \rangle \end{aligned}$$

The modification is manifested in the additional terms $\hat{v}^{\mathrm{PE}}$ and $\hat{v}^{\mathrm{ind}}$, while $\hat{f}_0$ is the usual Fock (or Kohn-Sham) operator. Note however, that for Kohn-Sham theory contributions from the exchange-correlation kernel will also be present (these are not shown in the above equation). A brief, physical explanation of the the two additional terms is given below

- $\hat{v}^{\mathrm{PE}} = \hat{v}^{\mathrm{es}} + \hat{v}^{\mathrm{ind}}$ accounts for interactions between multipoles and ground-state density $\hat{v}^{\mathrm{es}}$ as well as mutual polarization between the ground-state density and the environment $\hat{v}^{\mathrm{ind}}$.
- $\hat{v}^{\mathrm{ind}}$ accounts for the differential interaction between the ground- and excited states with a polarizable environment.

A derivation and more thorough explanation of the $E^{[2]}$ term can be found in Hedegaard2017 (cf. Eqs. 36-38).

To demonstrate how to calculate the effect of the environment with PE, we use the same system as in the PE-HF tutorial <pe_scf>. The employed h2o.mol file and potential 3_h2o.pot can be found in that tutorial. The input pe_response.inp is similar to a regular TDDFT calculation but with an additional .PELIB keyword under the **HAMILTONIAN section:

```
**DIRAC
.TITLE
H2O
.WAVE FUNCTION
.PROPERTIES
**HAMILTONIAN
.DFT
B3LYP
.PELIB      ! activate polarizable embedding
**INTEGRALS
.TWOINT
.SCREEN
1,00-12
*READIN
.UNCONT
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
10
**PROPERTIES
*EXCITATION ENERGIES
.EXCITA
1 5
.INTENS
0
**END OF
```

As for the case of PE-HF, the specification of .PELIB will thus make the program calculate the PE-specific terms; in case of linear response that is the additional $\hat{v}^{\mathrm{PE}}$ and $\hat{v}^{\mathrm{ind}}$ terms. Note that use of the .GSPOL keyword under the *PELIB flag will result in a calculation where only the term involving $\hat{v}^{\mathrm{PE}}$ is included (physically, this corresponds to a frozen environment during the excitation process). Now we have all ingredients for starting the calculation:

```
pam --inp=pe_response.inp --mol=h2o.mol --pot=3_h2o.pot
```

The summary will look identical to a vacuum calculation:

```
* Isotropic DL-DL non-zero oscillator strengths (f)
=====
DL = dipole length
Rate = Dipole radiation rate (s-1)
Lifetime = corresponding radiation lifetime (s)

Level  Frequency (eV)    f           Rate           Lifetime    Symmetry
-----
   1      7.00316    0.000001    1.26019E+03    7.93528E-04
   2      7.00317    0.000000    3.70356E+02    2.70010E-03
   3      7.00317    0.000002    3.30199E+03    3.02848E-04
   4      7.30170    0.049543    1.14615E+08    8.72484E-09
   5      8.53008    0.000000    5.32729E+01    1.87713E-02
-----
Sum of oscillator strengths:    0.04955
```

Note that the first three excitations have very low oscillator strength. These excitations corresponds to triplet excitations, which are automatically included in a relativistic framework. The fourth excitation is the observed one. We can calculate the electrostatic part of the solvent shift (i.e. not including the solvent effect on the geometry), by calculating the corresponding excitation energy without PE and subtracting the result from the PE result. We collect results with and without PE (and the obtained solvent shift) in the table below.

QM method (potential method)	Potential	PE	vac.	shift
B3LYP (B3LYP/6-31+G*)	3 H2O	7.30	6.69	0.61
Experimental		8.2	7.4	0.8

The experimental value is taken from discussion given in Christiansen2000. We note that

- The shift is a bit below the experimental one
- The excitation energies are somewhat underestimated (mainly due to the B3LYP functional).

To improve the model, we can either (i) expand the environment, (ii) describe the environment more accurately or (iii) describe the QM system more accurately (but currently DIRAC only includes PE with HF and DFT). Finally, we might also (iv) expand the QM system. Performing these calculations will be not be part of this tutorial. A table with selected results from larger environments, potentials obtained with a more accurate basis set (aug-cc-pVTZ) and a more accurate QM method are compiled below. All these calculations are non-relativistic calculations carried out with the Dalton program and taken from Hedegaard2016. The “full CI” calculations are obtained with the density matrix renormalization group (DMRG).

QM method (potential method)	Potential	PE	vac.	shift
Non. rel. B3LYP (B3LYP/6-31+G*)	127 H2O	7.65	6.69	0.96
Non. rel. full CI (B3LYP/6-31+G*)	3 H2O	7.90	7.46	0.44
Non. rel. full CI (B3LYP/6-31+G*)	127 H2O	8.17	7.46	0.71
Non. rel. B3LYP (B3LYP/aug-cc-pVTZ)	3 H2O	7.42	6.69	0.73

QM method (potential method)	Potential	PE	vac.	shift
Non. rel. B3LYP (B3LYP/aug-cc-pVTZ)	127 H2O	7.89	6.69	1.20
Non. rel. full CI (B3LYP/aug-cc-pVTZ)	3 H2O	7.95	7.46	0.49
Non. rel. full CI (B3LYP/aug-cc-pVTZ)	127 H2O	8.36	7.46	0.90

Electric dipole moments and polarizabilities

It is also possible to calculate simple expectation values or other linear response properties with the PELib module. The work-flow is straightforward, and we only need to add the .PELIB keyword as done for the TDDFT calculation above. The input related to the property is identical to a vacuum calculation. Examples of simple expectation values and linear response properties that are currently implemented with PE is the (electric) dipole moment and static or frequency-dependent polarizabilities. Unlike in a non-relativistic framework, the relativistic framework allow many magnetic properties to be obtained simply as expectation values (e.g. properties related to electron paramagnetic spectroscopy). All PE contributions to these properties will be automatically included from the wave-function optimization and will follow in a future release of an EPR module. Note also that linear response properties that require London orbitals are also not currently possible with PE. An example is given below for a dipole moment calculation input file :

```

**DIRAC
.WAVE FUNCTION
.PROPERTIES
**HAMILTONIAN
.PELIB
**WAVE FUNCTION
.SCF
**PROPERTIES
.DIPOLE
*END OF

```

Things to note

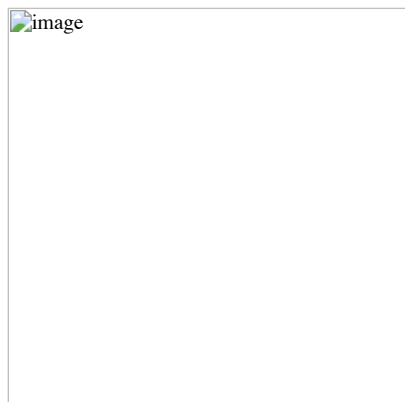
- Calculations with transformed two-component Hamiltonians are generally possible by supplying the program with the requested model under **HAMILTONIAN. In such a calculation, $\hat{v}^{\mathrm{PE}}$ is not transformed.
- Calculations with (linear) complex response or complex polarization propagator (CPP) is possible - see the tutorial “Complex response” (no other thing than activating CPP is needed to PE-CPP calculations).

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/polarizable_embedding/pe_scf.md

orphan

A Hartree-Fock calculation with the polarizable embedding (PE) model

In this tutorial we will cover the set up of a Hartree-Fock calculation using the polarizable embedding (PE) model (Olsen2010, Olsen2011) to include effects from a surrounding environment. Our example concerns a solvent effect, but the PE model is general and can also handle other environments (e.g. proteins or mixed solvents). The implementation of the model in DIRAC is documented in the paper Hedegaard2017, and we refer to this paper for further technical details. Our target system for this tutorial is a water molecule surrounded (or “microsolvated”) by three other water molecules. The system is shown in the figure below. The molecule in ball-and-stick representation is the one that is described with Hartree-Fock, and the three other water molecules are represented by an explicit PE potential. The underlying structure is abstracted from an MD simulation on a larger water cluster Kongsted2002. In real production calculations, PE would be used for the full system (PE is intended for much larger environments than three molecules), but this system is sufficient to show how the setup works.



The current implementation employs the external library [PElib](#) for the PE-related tasks. The module is activated by the HAMILTONIAN_.PELIB keyword under the **HAMILTONIAN section. A few additional (but also very simple) examples can be found in the test directories.

The .mol file (h2o.mol) for the central water molecule is shown below:

```

INTGRL
H2O structure, taken from Kongsted et al. (2002)
-----
C    2      0
      8.0    1
O    0.0000000000  0.0000000000  0.0000000000
LARGE BASIS 6-31++G
      1.0    2

```

```
H -1.4301431032 0.000000000 1.1073930799
H 1.4301431032 0.000000000 1.1073930799
LARGE BASIS 6-31++G
FINISH
```

The more experienced user will notice that the format is exactly the same as for a vacuum calculation. The .inp file (pe-hf.inp) is given below:

```
**DIRAC
.TITLE
H2O with PE
.WAVE FUNCTION
**HAMILTONIAN
.PELIB ! activate polarizable embedding
**INTEGRALS
*TWOINT
.SCREEN
1.00-12
*READIN
.UNCONT
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
10
*END OF
```

PElib is activated by the .PELIB keyword under **HAMILTONIAN. Specification of .PELIB will thus make the program calculate the PE-specific term of the Fock (or Kohn-Sham) matrix and add them to the vacuum part (see Eqs. (19)-(22) in Hedegaard2017). Additional PE-related keywords are given under the *PELIB section. This potential has to be generated but for now we can use the pre-generated potential (3_h2o.pot) given below

3_h2o.pot

The potential contains coordinates for three water molecules surrounding the central water. The three water molecules are represented by multipoles up to 2nd order (charges, dipole, and quadrupoles) and anisotropic dipole-dipole polarizabilities. Now we have all the ingredients needed to start the calculation:

```
pam --inp=pe-hf.inp --mol=h2o.mol --pot=3_h2o.pot
```

The output file will be named pe-hf_h2o_3_h2o.out. When the calculation is converged, the final energy report contains an additional entry; the “Embedding energy”, which is also included in the total energy:

```
TOTAL ENERGY
-----
Electronic energy          :    -85.261494639123399
Other contributions to the total energy
Nuclear repulsion energy   :      9.195432882420004
Embedding energy           :     -0.063104845058299
SS Coulombic correction    :      0.000000028807992

Sum of all contributions to the energy
Total energy               :    -76.108136383446379
```

A PE-DFT calculation can be run in exactly the same manner, but by additionally specifying DFT under the **HAMILTONIAN section (see e.g. the follow-up tutorial <pe_response>).

Things to note

- Currently, the PE model is only implemented for Hartree-Fock and Kohn-Sham DFT.
- X2C Hamiltonian works with the PE model, but the embedding operator is not transformed.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/polprp/general.md

orphan

Computing Electronic Excitations using the Propagator Module POLPRP

Introduction

We want to calculate absorption spectra corresponding to electronic excitations in atoms and molecules. Hereby the absorption energies and the corresponding transition moments are of predominant interest. The POLPRP module allows for the spectra calculation in the valence region of atoms and molecules with an accuracy very similar to CC-2. The numerical solution of the underlying polarization propagator (POLPRP) is based on the ADC-Method (algebraic diagrammatic construction). This terminology is a bit outdated since the emergence of intermediate state representation technique (ISR) where a correlated many-particle basis is constructed by application of perturbation theory. By this, size-consistency of the method can be achieved. At the same time the obtained eigenvectors which represent a particular excited state carry information about the degree of single-particle character of the excitation and the full configuration information in terms of particle-hole and 2p-2h contributions.

Currently, the strict second-order and the extended second-order ADC schemes are available for the solution of the polarization propagator.

Considerations before submitting a POLPRP job

The excitation space is constructed from all active occupied and virtual orbitals given to MOLTRA. From this a very large number of possible excited configurations emerges also spanning a large energy range. For valence excitations only the lowest excitation energies are relevant implying a Davidson diagonalization scheme as the most suitable one.

The calculation requires a SCF/MOLTRA step for calculation of energies only. If one is interested in configuration information one should also do a Mulliken population analysis. Hereby it is strongly recommended to combine similar orbital types into one group label (via the .LABDEF directive, see there) in order not to be flooded with too verbose information not adding to the physical content. For the transition moments the dipole moment integrals in **MO basis** are needed. Our first example of the argon atom will cover all these aspects in detail.

The results obtained have to be interpreted properly depending of the type of Hamiltonian used:

- **NONREL**: 2-component non-relativistic Hamiltonian with point nuclei. You can use contracted basis sets for all atoms, only the large component basis is used. In the various irreps of the final states you will find the singlet and one or more triplet components. In the absence of SOC triplets acquire zero transition moments (implying zero oscillator strengths) but the energies are calculated anyhow. For the classification of the individual irreps the same principles as for RELCCSD apply. A combination of linear symmetry and NONREL is not (yet) possible. The highest available symmetry is D_{2h} .
- **LEVY-LELBOND**: 4-component non-relativistic Hamiltonian. You can use contracted basis sets for all atoms, only the large component basis is used. Same facts for singlet and triplet as for NONREL.
- **SPINFREE**: Same spin manifolds as for NONREL but you include all scalar relativistic effects up to infinite order. Triplet final states get no osc. strength.
- **Four-component Dirac-Coulomb (default)**: The number of available final state irreps is very much reduced and the final states can no longer be classified as pure singlets or pure triplets. Depending on the nuclear charge the former (degenerate) triplet components will now undergo further splitting. One can normally identify the degenerate (SOC-free) parent state without any problems.
- **X2Cmmf**: The results are generally very similar to the four-component case but you harvest all the advantages of the X2Cmmf method (in particular speed). In the valence space no loss of accuracy with respect to 4C was observed.

Creating POLPRP input (here: excitation spectrum of argon)

Our argon molecular input file contains a basis with enough diffuse functions for a reasonably good result. This way of giving the basis is a bit outdated but was necessary in order to exactly compare to another (nonrelativistic) calculation. Any other basis from the basis set directory can be used as well.

argon.mol

The complete input (here with line numbers) reads as

argon.inp

The UNCONTRACT directive (9) is unnecessary here since we plug in an uncontracted set. In a relativistic calculation with SOC contracted basis sets need to be decontracted, however. To activate POLPRP we specify it in the wave function section (14). Furthermore we need Mulliken analysis (18) also for the virtuals (21,22,23). In the MOLTRA section we see that all occupied orbitals and nearly all virtuals are included in the active space (26,27). As mentioned above, for the calculation of transition moments we need the dipole moment integrals in MO basis which is triggered in lines 28-35 of the MOLTRA section. In the POLPRP section we indicate that we want to compute TMs (.DOTRMO) and all irreps where a possible final state can occur are calculated because the .STATE keyword was omitted. Then the program determines all these symmetries by itself. The Print directive in line 38/39 is required to generate the orbital list of active occupied and virtual spinors. You need this orbital list if the postprocessing tools are to be used. The DAVIDSON section controls the iterative diagonalizer that is always called in conjunction with POLPRP. The additionally available LANCZOS diagonalizer used in the one- and two-particle propagators is not applied here and can not be called. The keywords are described in the manual and choosing reasonable values needs a little bit of experience with iterative diagonalizers. The DVMAXSP determines the maximum size of the Hamilton operator projection (the Krylov space) and strongly influences memory demands of the routine.

Working with the output files

Besides the usual Dirac .out file where a lot of technical information of the POLPRP calculation is printed some essential result files are produced that can be used for further analysis and postprocessing. These are the file ADCEVECS.XX, ADCSTATE.XX and ADCTRMOM (the latter only if TMs were activated). XX here stands for the corresponding irrep number that can be 01..32 at most (linear case).

The ADCEVECS.XX files contain the eigenvectors of the large ADC matrix in binary form and can be used for restarts of the module. If copied to the scratch directory, the module reads them in and starts from these already converged eigenvectors.

The ADCSTATE.XX file contains human-readable information about the eigenstates of irrep XX where the most important configurations appear first:

```
Excited state:      1
-----
Exc. energy: 0.461708 au 12.563724 eV PS: 0.960574 Err: 0.000033 Sym: 1
Norm^2 of double exc. contr: 0.039426

128 ( 3, 1Eu) <---- 9 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.355139 0.126124
152 ( 4, 2Eu) <---- 15 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.355139 0.126124
151 ( 4, 2Eu) <---- 16 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.348944 0.121762
127 ( 3, 1Eu) <---- 10 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.348944 0.121762
129 ( 3, 1Eu) <---- 8 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.226284 0.051205
153 ( 4, 2Eu) <---- 14 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.226284 0.051205
154 ( 4, 2Eu) <---- 16 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.184017 0.033862
130 ( 3, 1Eu) <---- 10 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.184017 0.033862
155 ( 4, 2Eu) <---- 15 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.156695 0.024553
131 ( 3, 1Eu) <---- 9 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.156695 0.024553
128 ( 3, 1Eu) <---- 10 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.153918 0.023691
152 ( 4, 2Eu) <---- 16 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.153918 0.023691
127 ( 3, 1Eu) <---- 9 ( 3, 1Eu) 0 ( 0, ) <---- 0 ( 0, ) 0.136675 0.018680
151 ( 4, 2Eu) <---- 15 ( 4, 2Eu) 0 ( 0, ) <---- 0 ( 0, ) -0.136675 0.018680
```

Hereby the excitation energy of the first state in symmetry (irrep) 1 is given in au and eV and PS stands for pole strength which is a bit misleading in this context but describes the percentage of single-excitation character of this particular final state (96.06%). The next entry is the residual error stemming from the Davidson diagonalizer. Obviously, the next line tells us that we have 3.94 % double excitation character.

The following rows now reveal the leading configurations this state is composed of. First let us focus on the two rightmost columns with expansion coefficients (ISR basis!) and their squares. It is observed that these coefficients are pairwise identical apart from a sign reflecting the fact of Kramers pairing. All spinors occur twice in the corresponding Kramers irreps and possess identical orbital energies. Therefore the expansion distributes equally over both possible one-particle configurations. The first

number gives the absolute spinor number with respect to the active orbital list and the number in parenthesis is the relative spinor number within the given irrep. The corresponding first line tells us that our first expansion term consists of a single-particle excitation from occupied spinor 9 into virtual spinor 128 with a coefficient of 0.355139 etc. The other zeros indicate the single-particle character of this excitation and get nonzero for a 2h→2p type. In the framework of ISR and perturbation theory one can approximately think of expansion determinants that are described here. Strictly speaking, these are correlated singly excited configurations. By analyzing all ADCSTATE.XX files you can generate a complete spectrum but without proper utilities (see below) this can become a bit tedious. The utilities can be obtained from the author of this module.

Let's now turn to the ADCTRMOM file that is generated when we asked for transition moments. Actually, there is only one file collecting the information of all symmetries:

```
*****
** Final State Symmetry:      1
*****
```

State Order	1 symm	1 exc. energy TM(x)_r	0.461708 (au) TM(x)_i	12.563724 (eV) TM(y)_r	PS 0.960574 TM(y)_i	@E TM(z)_r	TM(z)_i
0th		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
1st		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2ndA		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2ndB		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2ndC		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,1		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,2		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,3		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,4		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,5		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,6		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,7		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,8		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,9		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2,10		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2p2h A		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
2nd 2p2h B		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
total		0.000000000	0.000000000	0.000000000	0.000000000	0.000000000	0.000000000
Osc. strength:		-->>	0.000000000	a.u.	@0		

The first line is more or less reproducing the entries in ADCSTATE.XX. Next comes a detailed list of individual x,y,z contributions to the transitions moments for each perturbational order and type of contraction. This is a speciality of the ADC method and allows for detailed insights into individual contributions to the TM. TMs can be complex, so both real and imaginary parts are given. The oscillator strength is calculated according to the usual formula using the sum of squares of Tx, Ty and Tz. For easy grepping of energies and oscillator strengths in production the markers "@E" and "@O" were introduced.

In order to collect all the peaks representing an excited final state together with their oscillator strengths from the ADCTRMOM file you can do the following:

```
grep '@E' ADCTRMOM|cut -c1-84 > ex; grep '@O' ADCTRMOM|cut -c30-52 > os; paste ex os > stickspectrum
```

These lines generate the text file 'stickspectrum' with all relevant information of the excited states. Hereby all states within a specific symmetry are listed together. This file can already be used for the generation of a simple gnuplot graph where we draw the peak heights/vs energies.:

```
plot [:][0:1.0] "stickspectrum" u 9:12 w i lw 2
```

Keep in mind that this command plots all peaks irrespective of their experimental observability which is described by the oscillator strength! (shaping up the graph is not discussed here)

 alternate text
alternate text

We see that all peaks exhibit nearly the same height around 0.95. This points to prevailing single excitation character but does not relate to experimental observability. Certainly, the obtained result is not ready for comparison with experiment. In order to arrive at physically meaningful spectra we refer you to the second chapter of the Spectroscopy II tutorial where we compute the excitation spectrum of the transition metal complex $\text{H}_2\text{Os}(\text{CO})_4$ in full scope and with additional utilities necessary to generate it. In the following section we address some aids to interpret the Dirac .out file.

Comments on the POLPRP output (in order of appearance)

- The property handler is called when you ask for transition moments. It prepares the dipole moment integrals represented in MO basis.
- Generating (real/complex) Dav. start vectors (all symm.): The particle/hole matrix that is calculated for all possible symmetries at once is diagonalized and corresponding eigenvectors are written out that serve as start vectors for the Davidson iteration (file DAVSTART.XX). If you obtain a warning that Hermiticity is violated in the p/h blocks something serious went wrong. Check the list of active orbitals then.
- Constructing real satellite part. In this section the coupling and satellite blocks of the ADC matrix are constructed and written out to disk (file: PP_ELE). In parallel runs this matrix is distributed over the compute nodes (files: PP_ELE.XX). Some dimensions and block sizes are reported as well.
- The Davidson algorithm output reports about dimensions, start vectors, orthogonality (important!) and user input.

During the diagonalization you will be prompted the progress as:

root	dim	exc.energy	error	status	time/root (s)
Time for constructing Krylov projection					19.97
Time for constructing residual vectors					6.89
1	40	0.5342178	0.5190262 o	XT	0.16
2	41	0.5453888	0.5187599 o	XT	0.17
3	42	0.5453888	0.5187599 o	XT	0.17
4	43	0.5453888	0.5187599 o	XT	0.17

The 'XT' abbreviation hereby indicates a successful extension of the Krylov space by using the computed preconditioner. The small 'o' denotes a nonconverged root whereas a '*' points to a converged one. Converged roots are no longer touched in the algorithm increasing the efficiency. It can also happen that the preconditioner cannot

determine a vector possessing enough orthogonality to the already converged space. In this case a warning is issued and you should carefully examine the quality of your calculations at the end of the run. A macroiteration denotes a cycle of one complete Krylov space construction followed by collapsing the space (see publication for more details).

The verbose output in case of transition moment calculation can be more or less ignored because it is summarized in the ADCTRMOM file. In the near future we will considerably shrink the output of POLPRP but for the moment it provides a high level of detail useful for the analysis of difficult cases.

The sequence just described is now repeated for all irreps separately. It is a big advantage of the ADC/ISR method that the representation matrices exhibit full block structure. So each symmetry (irrep) can be treated independently and the user can select the desired irreps by the .STATE directive (see manual).

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/polprp/osmium.md

orphan

Full scope application of POLPRP to an osmium carbonyl complex

In this second section we describe a toolchain how to arrive at physically meaningful spectra suitable for comparison to experiment and publication. This toolchain emerged over the years of application of propagator code and has proven to be useful. Shortly, we will combine the individual small helpers to a combined postprocessing module written in Python3 bringing about a lot of simplification.

The excitation spectrum of the osmium complex exhibits sizeable differences between a scalar relativistic and a four-component treatment. By this we extract the effect of pure spin-orbit coupling on the spectrum.

 alternate text
alternate text

Therefore two individual calculations once with the .SPINFREE and the .X2Cmmf Hamiltonian are necessary. Since the postprocessing steps are identical for both pathways we do not need to repeat it. The molecule and X2Cmmf input read as

oscom.mol

oscom.inp

As you can see in the .mol file the complex has C2v symmetry and we use basis sets from the library shipped with DIRAC. In the input file a large number of labdef labels are visible. This is necessary later on if we want to analyze the eigenstate composition in terms of atomic orbital contributions. Hereby the grouping occurs according to the angular momentum of the atomic basis functions. A further distinction is not made and would lead to an excess of information in larger systems. In order to generate the LABDEF labels we use the utility

labdefmaker

that automatically reads the:

GETLAB: SO-labels

```
* Large components: 215
1  L A1 0s s      2  L A1 0s pz      3  L A1 0s dxx      4  L A1 0s dyd      5  L A1 0s dzz      6  L A1 0s fxxz
7  L A1 0s fyyz    8  L A1 0s fzzz    9  L A1 0s g400    10 L A1 0s g220    11 L A1 0s g202    12 L A1 0s g040
. . . . .
```

section. A little disadvantage here is that the ordered LABDEF section should be available already for the Mulliken population analysis for a proper labeling of the atomic contributions. So we recommend to run DIRAC with a preliminary input just producing the GETLAB output. Application of labdefmaker produces the file "labdef" that can be included in the production input afterwards. A manual sorting and grouping of labels is out of scope for larger systems. We perform a strict ADC-2 calculation where the satellite block just consists of the diagonal of the orbital energies (see publications). Before we include the transition moments in our analysis we want to get an overview of the final state composition. Hereby the information of a Mulliken analysis, the active orbital list and the ADCSTATE.XX files are required and their information combined. This is done by the two helper programs

spimaker and excmaker

to be executed in series. spimaker constructs an intermediate list of spinors relating the molecular and atomic basis and excmaker combines this list with the state analysis obtained by the POLPRP calculation. The resulting 'xstates' file then serves as the basis for the gnuplot drawing utility that generates a composition analysis of the hole and the particle space:

 alternate text
alternate text

This graph shows sizeable osmium contributions in the hole and particle space being responsible for SO coupling effects in the excitation spectrum (see below).

In the next step we want to utilize the physical information provided by the transition moments. For this purpose we multiply the pole strength of the impulse peak with the oscillator strength which gives us a modulated spectrum according to the transition probability. This multiplication was done with a simple spreadsheet tool because the number of states is reasonably small. afterwards the new stick spectrum is convoluted with a Lorentzian envelope in order to account for the spectral width. This final step is performed by the

specfolder

tool. You give the number of peaks, the desired number of graphic points and the spectral width in meV. The resulting graph file can again be plotted by gnuplot and results in the following simulated spectrum:

 alternate text
alternate text

Upwards we plot the X2Cmmf spectrum, downwards the SPINFREE spectrum which exhibits the pure effect of spin-orbit coupling on the excitation spectrum. Soon an integrated python tool will come combining all the individual tasks from above.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/properties/complex_response.md

orphan

Complex response

In this tutorial we will look at a simple example of using the complex polarization propagator (CPP) method in DIRAC. For more details on the implementation and the method, see Villaume2010 and Norman2011, respectively.

We consider the LiH molecule:

```
DIRAC
LiH (Exp. geometry from Herzberg R=1.5949 A)

C      2      0      A
      3.      1
Li      0.000000      0.000000      -0.398725
LARGE BASIS aug-cc-pVDZ
      1.      1
H      0.000000      0.000000      1.196175
LARGE BASIS aug-cc-pVDZ
FINISH
```

and generate the following menu file:

```
**DIRAC
.WAVE F
.PROPERTIES
**WAVE F
.SCF
**PROPERTIES
*LINEAR RESPONSE
.OPERATOR
ZDIPLN
.FREQ INTERVAL
0.13 0.16 0.002
.SKIPEP
**END OF
```

Here we ask for the frequency-dependent linear response function $\chi_{zz}(\omega)$ in the frequency window 0.13 - 0.16 a.u. with a step length of 0.002 a.u., calculated at the Hartree-Fock level using the 4-component relativistic Dirac-Coulomb Hamiltonian. The negative of the above response function corresponds to the negative of the zz -component α_{zz} of the dipole-dipole polarisability. Plotting the polarizability we obtain the graph below

RealResponse.png

We observe a singularity around $\omega = 0.147$ au, correspond to a dipole-allowed electronic transition. The presence of the singularity allows us to read off excitation energies from the dispersion (frequency-dependence) of the polarizability. However, it is an unphysical feature and corresponds to an infinitesimal small linewidth in the electronic spectrum.

The unphysical feature can be removed by introducing lifetimes into the response functions. In practice a real frequency ω is replaced by complex $\omega = \omega + i\gamma$ where γ is a user-given parameter related to the inverse of the finite lifetimes of the excited states. Our input now looks like:

```
**DIRAC
.WAVE F
.PROPERTIES
**WAVE F
.SCF
**PROPERTIES
*LINEAR RESPONSE
.OPERATOR
ZDIPLN
.FREQ INTERVAL
0.13 0.16 0.002
.DAMPING
0.005
.SKIPEP
**END OF
```

Here we set $\gamma = 0.005$ au, corresponding to 0.136057 eV. The linear response function now becomes complex. The real and imaginary parts of α_{zz} are plotted below.

ComplexResponse.png

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/properties/dipmom-polariz.md

orphan

Finite-field Coupled Cluster calculations of dipole moment and polarizability

BeH electric properties

This toy molecule in the small STO-2G decontracted basis demonstrates the use of finite-field perturbation theory for calculating the dipole moment (μ_z) and the polarizability (α_{zz}) both at the Hartree-Fock and at the highly correlated Coupled Cluster level. Our molecule is oriented in the z-axis.

Because BeH is open-shell system, we can not employ MP2 and CCSD gradients, which are available only for closed-shell systems. We have to resort to the finite-field perturbation scheme, where we add the z-dipole moment operator as small perturbation to the X2C relativistic Hamiltonian. The symmetry of heterodiatomic molecule must be lowered from linear to C2v because of the z-oriented dipole perturbation operator.

Molecule in electric field

When a molecular system is placed in a homogenous static electric field, $F=[F_x, F_y, F_z]$ (p,q-components), its energy may be expanded in a Taylor serie as follows:

$$E = E_0 - \mu_p F_p - \frac{1}{2} \alpha_{pq} F_p F_q + \dots$$

We keep only z-component of the field, $F=F_z$, and the dipole moment is obtained as the first numerical derivative of the energy according to the electric field at zero field strength:

$$\mu_z = - \lim_{F_z \rightarrow 0} \left(\frac{\partial E(F_z)}{\partial F_z} \right)_{F_z=0}$$

The polarizability (for simplicity, here the zz-component) is calculated via second numerical derivative of the energy according to the electric field at zero field strength:

$$\alpha_{zz} = - \lim_{F_z \rightarrow 0} \left(\frac{\partial^2 E(F_z)}{\partial F_z^2} \right)_{F_z=0}$$

Method of computation

Let us first compute the total energies of few electric field strength. This can be done through replacing the string in the DIRAC input file by the field strength:

```
pam --noarch --replace zff=+0.0005 --mol=BeH.sto-2g.C2v.mol --inp=BeH.x2c_scf_relcc_ffz.inp --put "DFPCM0.BeH.x2c_scf_sto-2g.C2v=DFPCM0"
```

(Input files for download are BeH.x2c_scf_relcc_ffz.inp <../../../../test/ffpt_dipmom_polariz_relcc/BeH.x2c_scf_relcc_ffz.inp>, BeH.sto-2g.C2v.mol <../../../../test/ffpt_dipmom_polariz_relcc/BeH.sto-2g.C2v.mol> and the text MO file, DFPCM0.BeH.x2c_scf_sto-2g.C2v <../../../../test/ffpt_dipmom_polariz_relcc/DFPCM0.BeH.x2c_scf_sto-2g.C2v>.)

The total SCF and CCSD(T) energies varying on field strengths are sorted in the following Table:

Fz	E(SCF,Fz)	E(CCSD(T),Fz)
+0.0010	-14.424976681856371	-14.461324673372751
+0.0005	-14.425860788235255	-14.462193434302819
+0.0000	-14.426746572346620	-14.463063980838802
-0.0005	-14.427634036448282	-14.463936314225093
-0.0010	-14.428523182834038	-14.464810435757153

NOTE: In this quick DIRAC test, only two output files are provided for Coupled Cluster method, BeH.x2c_scf_relcc-0.0005_BeH.sto-2g.C2v.out <../../../../test/ffpt_dipmom_polariz_relcc/result/BeH.x2c_scf_relcc-0.0005_BeH.sto-2g.C2v.out>, BeH.x2c_scf_relcc+0.0005_BeH.sto-2g.C2v.out <../../../../test/ffpt_dipmom_polariz_relcc/result/BeH.x2c_scf_relcc+0.0005_BeH.sto-2g.C2v.out>.

Spreadsheet

In the attached spreadsheet (accessible via Gnumeric, Libre Office Calc or MS Excel), we interpolate five pairs, $[F, E(F)]$, of the Table mytable with the 4th-degree polynomial:

$$E(F) = a_0 + a_1 F + a_2 F^2 + a_3 F^3 + a_4 F^4$$

The negative first derivative of the polynomial expansion at zero field is the dipole moment, based on the equation μ_z :

$$\mu_z = - \lim_{F \rightarrow 0} \left(\frac{\partial E(F)}{\partial F} \right)_{F=0} = -a_1$$

The second derivative (with minus sign) is the polarizability, following prescription in α :

$$\alpha_{zz} = - \lim_{F \rightarrow 0} \left(\frac{\partial^2 E(F)}{\partial F^2} \right)_{F=0} = -2a_2$$

In the spreadsheet environment we employ the function LINEST. For that you have to prepare columns with finite-field strengths powered to 1, 2, 3 and 4.

The binary spreadsheet file available for download is data.ods <../../../../test/ffpt_dipmom_polariz_relcc/data.ods>.

Dipole moment

We calculate here the electronic part of the z-component of the dipole moment as the perturbing field goes in the z-direction.

Expectation value

For the SCF method, we can obtain the dipole moment - both for closed and open-shell systems - via the expectation value, which gives all three components, μ_x , μ_y and μ_z . A good practise is that dipole moment finite-field calculations are verified against the SCF/DFT expectation value.

The SCF expectation value dipole moment - its zz-electronic contribution - is $\mu_z = -1.77324745$ au. (The input file for download, BeH.x2c_scf_dipmom.inp <../../../../test/ffpt_dipmom_polariz_relcc/BeH.x2c_scf_dipmom.inp> and the the corresponding output file, BeH.x2c_scf_dipmom_BeH.sto-2g.C2v.out <../../../../test/ffpt_dipmom_polariz_relcc/result/BeH.x2c_scf_dipmom_BeH.sto-2g.C2v.out>.)

Finite-field values

From the spreadsheet function table, we obtain the value of $a_1 = -1.7732474$ a.u., which is the expansion coefficient of the first order field.

Simple first numerical derivation of the SCF perturbed energy according to the applied electric field, equation mu,

$$\mu_z = - \frac{E(+F) - E(-F)}{2F}$$

gives -1.771568 a.u. for F=0.0005. For the field strength of F=0.001 it is -1.76989 a.u.

The zero field derivative of the fourth order polynomial, equation mu_polyn is more precise when comparing against the above mentioned SCF expectation value, though finite field evaluation is still capable to give value accurate to two decimal places.

Polarizability

For closed-shell systems, the polarizability at the SCF and DFT levels can be calculated via linear response method, which gives all components of the polarizability tensor. For systems with unpaired electrons like this one, however, we have to resort to finite-field perturbative calculations. For simplicity, we focus on the zz-tesnor component of the polarizability, α_{zz}

SCF

For our molecule, the simple numerical second derivate calculation for given F=0.0005 at SCF level, equation alpha

$$\alpha_{zz} = - \frac{E(+F) + E(-F) - 2E(0)}{F^2}$$

gives 6.71996 a.u.. This is close to the second order expansion coefficient, multiplied by 2, giving the value of $\alpha_{zz} = -2a_2 = (-2) * (-3.3599745995852954) = 6.71994919$ a.u.

CCSD(T)

Similar numerical derivation for perturbed CCSD(T) energies and for F=0.0005, according to equation alpha_deriv, gives the value of $\alpha_{zz} = 7.147401232$ a.u. The corresponding expansion coefficient (multiplied by the factor of two) from equation alpha_polyn, produces the value of $\alpha_{zz} = 7.14738420$ a.u.

One can conclude that the F=0.0005 field strength is sufficient to obtain polarizability accurate to 3 decimal places by simple numerical second derivation, equation alpha_deriv.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/properties/mcscf_properties.md

orphan

Properties at the MCSCF level of theory

In this tutorial we will look at a simple example of computing one-electron properties using a Kramers-restricted MCSCF (KRMC) and Hartree-Fock (HF) wave function in DIRAC. For more details on the implementation and the method, see Jensen1996, Thyssen2004, Thyssen2008 and Knecht2009, respectively.

We consider the Be atom using C_{2v} symmetry:

```
INTGRL
Be atom in uncontracted Pierloot basis set, MOLCAS 5, ANO-S
Generated small component via RKB
C 1 2XY Y .10D-15
4. 1
Be 1 .00000000 .00000000 .00000000
LARGE 3 2 1 1
H 7 0 3
2732.3281
410.31981
93.672648
26.587957
8.6295600
3.0562640
1.1324240
H 3 0 3
.18173200
.05917000
.02071000
H 4 0 3
1.1677000
.36500000
.11410000
.03570000
H 3 0 3
.54680000
.14650000
.03930000
FINISH
```

and generate the following menu file:

```
**DIRAC
.TITLE
Testing KRMC and KR-CI for Be in Pierloot basis
```



```
.ANALYZE
.WAVE FUNCTION
.PROPERTIES
**PROPERTIES
.RHONUC
.EFFDEN
.WAVE F
2
DHF
KRMCMC
**HAMILTONIAN
.X2C
**WAVE FUNCTION
.SCF
.KRMCMSCF
*SCF
.CLOSED SHELL
4
*KRMCMSCF
.CI PROGRAM
LUCIAREL
.INACTIVE
1
.GAS SHELL
2
0 2 / 1
2 2 / 3
.SYMMETRY
1
.PRINT
5
*OPTIMI
.ANALYZ
**END OF
```

Here we ask for the contact density $\rho(0)$ (.RHONUC) as well as effective density $\bar{\rho}$ (.EFFDEN) at the Be nucleus calculated at the HF and MCSCF level of theory using the exact two-component Hamiltonian. In the MCSCF step we define a CAS(2,4) space correlating 2 electrons in 4 Kramers pairs (2s2p shell of Be).

The results read as follows:

```
HF:
Rho at nuc Be 01 :      33.94937443146 a.u.
EFFD:Be 01       :      33.94934902177 a.u.
```

```
KRMC:
Rho at nuc Be 01 :      33.89686536890 a.u.
EFFD:Be 01       :      33.89683999776 a.u.
```

From this we can conclude that non-dynamical (and partially dynamical) correlation included in our CAS wave function seems to reduce both, $\rho(0)$ and $\bar{\rho}$ at the Be nucleus compared to the uncorrelated Hartree-Fock values.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/properties/scfexc/tutorial.md

orphan

Excitation energies at the SCF-level

In this tutorial we will look at the theory behind the calculation of excitation energies at the SCF-level, that is, either by time-dependent Hartree-Fock (TD-HF) or TD-DFT. The implementations in DIRAC are presented in Bast2009.

Excitation energies ω_m are found by solving the generalized eigenvalue problem

$$\left(E_0^{[2]} - \hbar\lambda S^{[2]}\right)X_m = 0; \quad (1)$$

where the matrices $E_0^{[2]}$ and $S^{[2]}$ are the electronic Hessian and generalized metric, respectively. From their structure, it can be shown that solution vectors come in pairs

$$\begin{aligned} \lambda_{+n} &= \left| \omega_n \right|, \quad \lambda_{-n} = \left| \omega_n \right| \\ \mathbf{Z}_n &= \mathbf{Z}_n^{\dagger}, \quad \mathbf{Y}_n = \mathbf{Y}_n^{\dagger} \\ \lambda_{-n} &= -\left| \omega_n \right|, \quad \lambda_{+n} = \left| \omega_n \right| \\ \mathbf{Y}_n &= -\mathbf{Z}_n^{\dagger}, \quad \mathbf{Z}_n = \mathbf{Z}_n^{\dagger} \end{aligned}$$

Transition moments associated with operator $\hat{h}_A$ for transitions $n \leftarrow 0$ are generally calculated as

$$\langle n | \hat{h}_A | 0 \rangle = \langle X_{+n} | \dagger | E_{+1} \rangle; \quad (2)$$

where appears solution vector $\mathbf{X}_{+n}^{\dagger}$ and property gradient $\mathbf{E}_{+1}$. The property gradient is given by

$$\mathbf{E}_{+1} = \left[\begin{array}{c} \mathbf{g}_B \\ \Theta_B \mathbf{g}_B \end{array} \right]; \quad \mathbf{g}_{B,ai} = -h_{ai}$$

An important generalization above is that, in addition to Hermitian operators $\hat{h}_B$ ($\Theta_B = +1$), imposed by the tenets of quantum mechanics, we also allow anti-Hermitian ones ($\Theta_B = -1$).

A key to computational efficiency is to consider a decomposition of solution vectors in terms of components of well-defined Hermiticity and time reversal symmetry. Using a pair of solution vectors $\mathbf{X}_{+}$ and $\mathbf{X}_{-}$, we may form Hermitian and anti-Hermitian combinations

$$\begin{aligned} &\begin{aligned} &\begin{array}{l} \mathbf{X}_h = \frac{1}{2}(\mathbf{X}_{+} + \mathbf{X}_{-}) \\ \mathbf{Z}^{\dagger} + \mathbf{Y}^{\dagger} \\ \mathbf{Z}^{\dagger} - \mathbf{Y}^{\dagger} \end{array} \\ &\begin{array}{l} \mathbf{X}_a = \frac{1}{2}(\mathbf{X}_{+} - \mathbf{X}_{-}) \\ \mathbf{Z}^{\dagger} - \mathbf{Y}^{\dagger} \\ \mathbf{Z}^{\dagger} + \mathbf{Y}^{\dagger} \end{array} \end{aligned}$$

The inverse relations therefore provide a separation of solution vectors into Hermitian and anti-Hermitian contributions

$$\mathbf{X}_{+} = \mathbf{X}_h + \mathbf{X}_a; \quad \mathbf{X}_{-} = \mathbf{X}_h - \mathbf{X}_a.$$

Further decomposition of each contribution into time-symmetric and time-antisymmetric parts give vectors that are well-defined with respect to both Hermiticity and time reversal symmetry

$$\mathbf{U}^{\dagger}(\Theta_h, \Theta_t) = \begin{cases} \mathbf{c}^{\dagger} & \text{if } \Theta_t = \Theta_h \\ \mathbf{d}^{\dagger} & \text{if } \Theta_t = -\Theta_h \end{cases}$$

where

$$\mathbf{c}_{ai} = \mathbf{x}_{ai} \quad \text{and} \quad \mathbf{d}_{ai} = \overline{\mathbf{x}_{ai}}$$

and the index overbar refers to a Kramers' partner in a Kramers-restricted orbital set. The scalar product of such vectors is given by Saue2002a

$$\mathbf{U}_1^{\dagger}(\Theta_{h1}, \Theta_{t1}) \mathbf{U}_2(\Theta_{h2}, \Theta_{t2}) = \left(1 + \Theta_{h1}\Theta_{h2}\Theta_{t1}\Theta_{t2}\right) \left[z + \Theta_{h1}\Theta_{h2}z^{\dagger}\right]; \quad z = \left(\mathbf{c}_1^{\dagger} \mathbf{c}_2 + \mathbf{d}_1^{\dagger} \mathbf{d}_2\right),$$

and one may therefore distinguish three cases

$$\begin{aligned} &\mathbf{U}_1^{\dagger}(\Theta_{h1}, \Theta_{t1}) \mathbf{U}_2(\Theta_{h2}, \Theta_{t2}) = \begin{cases} 0 & \text{if } \Theta_{h1}\Theta_{h2} = -\Theta_{t1}\Theta_{t2} \\ 4\text{Re}[z] & \text{if } \Theta_{h1}\Theta_{h2} = \Theta_{t1}\Theta_{t2} = +1 \\ 4\text{iIm}[z] & \text{if } \Theta_{h1}\Theta_{h2} = \Theta_{t1}\Theta_{t2} = -1 \end{cases} \end{aligned}$$

One may show that Hermiticity is conserved when multiplying such a trial vector by the electronic Hessian, whereas it is *reversed* by the generalized metric. However, both the electronic Hessian and the generalized metric conserves time reversal symmetry. The implication is that the time-symmetric and time-antisymmetric components of a solution vector does not mix upon solving the generalized eigenvalue problem, and one can dispense with one of them. From a physical point of view, this can be understood from the observation that excited states can be reached through both time-symmetric and time-antisymmetric operators. In order to employ the quaternion symmetry scheme, we choose to work with the time-symmetric vectors. It follows from the above that their scalar product are either zero or real. In practice, a property gradient is therefore always contracted with the component of the solution vector having the same hermiticity, so that all transition moments are real.

Practical example

We consider the excitation $1s_{1/2} \rightarrow 3p_{3/2}$ of the magnesium atom. We limit attention to boson irrep B_{1u} , corresponding to the symmetry of the z - coordinate (Γ_z). We work within the electric dipole approximation and from a character table of D_{2h} it is clear that we will only get non-zero transition moments from the z -component of the electric dipole operator.

We start from the menu file ed.inp

ed.inp

and the xyz-file Mg.xyz

Mg.xyz

and run our calculation using:

pam --inp=ed --mol=Mg.xyz --get "DFCOEF PAMXVC"

We are keeping some file for later use, see below. It can be seen in the menu file ed.inp that excitations are only allowed from the first *gerade* orbital ($1s_{1/2}$) to the sixth *ungerade* one ($3p_{3/2,1/2}$). The nature of these orbitals can be seen from the Mulliken population analysis in the output.

We have increased the print level significantly in order to be able to look a bit under the hood. Let us first look at the property gradient

$$\mathbf{g}_{ai} \rightarrow \langle 3p_{3/2,1/2} | \hat{\mu}_z | 1s_{1/2} \rangle; \quad \mathbf{g}_{\overline{ai}} \rightarrow \langle 3p_{3/2,1/2} |$$

The matrix element $\mathbf{g}_{ai}$ may be written as

$$\mathbf{g}_{ai} = -\int \Omega_{ai}(\mathbf{r}) \hat{\mu}_z \Phi_i(\mathbf{r}) d^3\mathbf{r}; \quad \Omega_{ai}(\mathbf{r}) = \Phi_a^{\dagger}(\mathbf{r}) \Phi_i(\mathbf{r})$$

and similar for $\overline{g_i}$. For present purposes it suffices to think in terms of 2-component spinors. Since $s_{1/2}$ and $p_{3/2}$ have opposite parity, they will in the conventions of DIRAC have different symmetry structures

```
\begin{aligned}
s_{1/2} &\rightarrow \left[ \begin{array}{c} \Gamma_0, \Gamma_z \end{array} \right] \\
&\left[ \begin{array}{c} \Gamma_y, \Gamma_x \end{array} \right] \\
&\end{array} \right]; \quad p_{3/2} \rightarrow \left[ \begin{array}{c} \Gamma_{xyz}, \Gamma_z \end{array} \right] \\
&\left[ \begin{array}{c} \Gamma_y, \Gamma_x \end{array} \right] \\
&\end{array} \right].
\end{aligned}
```

In addition we have to remember the relation between Kramers partners

```
\begin{aligned}
\phi &= \left[ \begin{array}{c} \phi^\alpha \\ \phi^\beta \end{array} \right] \\
&\quad \overline{\phi} = \left[ \begin{array}{c} -\phi^\beta \\ \phi^\alpha \end{array} \right] \\
&\end{aligned}
```

In terms of symmetry the orbital distributions are

$$\Omega_{\left(p_{3/2}; s_{1/2}\right)} = \left(\Gamma_{xyz}, \Gamma_z\right); \quad \Omega_{\left(p_{3/2}; \overline{s_{1/2}}\right)} = \left(\Gamma_{xyz}, \Gamma_x\right)$$

In the present case the operator symmetry is Γ_z . Since the integrand of the matrix element has to be totally symmetric, we see that only the imaginary part of g_{ai} will be non-zero; the partner element $\overline{g_i}$ is strictly zero. The generally complex gradient elements are combined into a quaternion one

$$g_{AI} = g_{ai} + g_{\overline{ai}} \check{j}$$

and, indeed, when we look in the output we find

propgrad.txt

Here the notation $E_{pol} n l X n Y n Z n$ refers to a Cartesian electric multipole operator $\hat{Q}^{nl} = x^n y^n z^n$; $n_x + n_y + n_z = n$. Since this is a time symmetric, Hermitian operator, it will be contracted with the Hermitian part of the solution vector. In this particular case we have only a single orbital rotation, and between two orbitals of positive energy, so the relevant part of the solution vector is

E+_solvec.txt

Searching in the output, we find the corresponding calculated transition moment

tmom.txt

which is easily seen to be $2 \times 0.35355084 \times 0.00682451$. (With respect to Eq.(3) a factor two seems to be missing; this appears to be connected to the scaling of eigenvectors in subroutine XPPORD.)

We may proceed to visualize the transition moment density

$$T(\mathbf{r}) = -\sum_{ai} x^{\ast}_{+;n;ai} \Omega_{ai} \left(\mathbf{r} \right) \hat{\mu}_z \left(\mathbf{r} \right)$$

as a radial distribution, that is

$$T(r) = \int_0^{2\pi} \int_0^\pi T(\mathbf{r}) r^2 \sin\theta \, d\theta \, d\phi$$

As preparation we first inspect the file PAMXVC of solution vectors

rsread.txt

We see that the Hermitian e-e part of the solution vector E+ is the first vector on the file, which therefore leads us to set up the input file length.inp as

length.inp

We now generate the radial distribution using:

```
pam --inp=length --mol=Mg.xyz --put "DFCOEF PAMXVC" --get "plot.radial.scalar=length.dat"
```

First we may note at the end of the output

radint.txt

and so we see that the radial distribution integrates to the value obtained in our previous calculation.

Next, with a suitable plotting program (I used [xmgrace](#)) we can visualize the radial distribution



To get some idea of the extent of the z-electric dipole moment transition density associated with the $1s_{1/2} \rightarrow 3p_{3/2}$ transition, I have included in the input the calculation of orbital sizes as $\langle r^2 \rangle^{1/2} = \left(\langle x^2 \rangle + \langle y^2 \rangle + \langle z^2 \rangle \right)^{1/2}$. Extracting and using the data from the output I obtain the following table (orbital labels are taken from the Mulliken analysis of the first calculation):

Sym.	Orb.	$\langle x^2 \rangle$	$\langle y^2 \rangle$	$\langle z^2 \rangle$	$\langle r^2 \rangle^{1/2}$
------	------	-----------------------	-----------------------	-----------------------	-----------------------------

E1g	1s 1/2; 1/2	0.008	0.008 0.008 0.151
E1g	2s 1/2; 1/2	0.190	0.190 0.190 0.754
E1g	3s 1/2; 1/2	4.106 4.106 4.106 3.510	
E1u	2p 1/2; 1/2	0.198 0.198 0.198 0.772	
E1u	2p 3/2; -3/2	0.240	0.240 0.120 0.774
E1u	3p 3/2; 1/2	0.160	0.160 0.279 0.774

We see that the radial distribution peaks around 0.25 a_0 which is slightly outside the size of the $1s_{1/2}$ orbital.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/pvcefg/tutorial.md

orphan

Parity-violation contribution to electric field gradient

Introduction

In this tutorial we introduce the calculation of parity violation contribution to the electric field gradient tensor as given in the DIRAC code.

As described in `PROPERTIES` `</manual/properties>`, the parity-violation contribution to the electronic part of the electric field gradient tensor (PVCEFG) of a nucleus K is given by

$$q^{\text{PV}}_{ij}(\mathbf{R}_K) = \langle \mathbf{r} | \hat{q}_{ij}(\mathbf{R}_K) | \mathbf{H}^{\text{PV}}_K \rangle$$

where the contribution from the K nucleus to the nuclear spin-independent parity-violating operator

$$\mathbf{H}^{\text{PV}}_K = \frac{G_F}{2\sqrt{2}} \sum_{i,K} Q_{w,K} \gamma_i^5 \rho_K(\mathbf{r}_i)$$

is expressed in terms of the Fermi coupling constant $G_F=2.222516 \times 10^{-14} \text{E}_{\text{ha}_0^3}$ (this value corresponds to CODATA 2022, but it can change if you use other CODATA set), the weak nuclear charge $Q_{w,K}$ and the normalized nuclear charge density ρ_K (in units of the inverse of cube distances). Results are given in a.u. Further details can be found in `Aucar2025_PVEFG`.

Application to the H2O2 molecule

As an example, we show a calculation of the PV contribution to the EFG tensor of the oxygen and hydrogen nuclei at the Hydrogen peroxide molecule. The input file `efg_dc.inp` is given by

`efg_dc.inp`

whereas the molecular input file `H2O2.mol` is

`H2O2.mol`

The calculation is run using:

`pam --inp=efg_dc --mol=H2O2`

As a result, PVCEFG are obtained at the RPA (i.e., coupled Hartree-Fock) level of approach. The code also works at the DFT level.

Reading the output file

As the `PROPERTIES.PRINT` flag in the input file is set to 1, extra details are printed. The results are given in au. The PVCEFG to the oxygen and hydrogen nuclei will look like:

`PVCEFG-H2O2`

As it can be seen, the total PVC to the EFG tensor elements is further separated in their (e-e) and (e-p) parts.

[!WARNING] The basis set used here is rather small for demonstration purposes; larger basis sets are recommended for production calculations.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/pvcsr/tutorial.md

orphan

Parity-violation contribution to nuclear spin-rotation constants

Introduction

In this tutorial we introduce the calculation of parity violation contribution to nuclear spin-rotation tensors as given in the DIRAC code.

The parity-violation contribution to nuclear spin-rotation (PVCNSR) tensor of a nucleus N is given by

$$\chi_N^{\text{PV}} = \frac{1}{2} \frac{G_F}{\Lambda^2} \frac{1 - 4 \sin^2 \theta_W}{\sqrt{2}} \frac{1}{c} \frac{1}{\Lambda} \langle \rho_N(\mathbf{r}) \rangle \frac{1}{c} \frac{1}{\Lambda} \langle \mathbf{J} \rangle$$

where $\rho_N(\mathbf{r})$ is the normalized nuclear charge density; $\mathbf{I}$ is the inertia tensor of the molecule and $\mathbf{J}_e = \mathbf{L} - \mathbf{R}_{\text{CM}} \times \mathbf{p}_e$ is the electronic total angular momentum.

Application to the H2O2 molecule

As an example, we show a calculation of the PV contribution to the NSR constant of the oxygen nucleus at the Hydrogen peroxide molecule. The input file pvcnr.inp is given by

```
pvcnr.inp
```

whereas the molecular input file H2O2.mol is

```
H2O2.mol
```

The calculation is run using:

```
pam --inp=pvcnr --mol=H2O2
```

As a result, PVCNSR are obtained at the coupled Hartree-Fock level. The code also works at the DFT level.

Reading the output file

As the PROPERTIES_.PRINT flag in the input file is set to 1, the results are fully detailed and given in Hz.

The PVCNSR of the oxygen nucleus will look like:

```
PVCNSR-H2O2
```

As it can be seen, the total SR constant of the oxygen nucleus is given. The linear response function is further separated in their (e-e) and (e-p) parts, as well as in their $\mathbf{L}$ and $\mathbf{S}$ parts.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/qed/effqed.md

orphan

Effective QED potentials

QED effects are high-order relativistic effects that cannot be captured in the framework of the no-pair approximation. A rough explanation of the QED effects that contribute to the Lamb shift of bound-state electrons is as follows:

- Vacuum polarization (VP): A charge in space is surrounded by virtual electron-positron pairs which contributes to its observed charge.
- Self-energy (SE): The electron drags along its electromagnetic field which contributes to its observed mass.

Effective QED potentials are an approximate way to include QED effects in all-electron relativistic Hamiltonians. In the current implementation (see Ref. Sunaga2022) the following potentials are available:

Vacuum polarization

Uehling potential

The Uehling potential is the dominant contribution to the vacuum polarization. It is extracted from the term linear in nuclear charge after regularization and renormalization. For a spherically symmetric nuclear charge distribution it reads

$$\varphi^{\text{VP}}(\mathbf{r}) = \frac{e}{4\pi} \frac{1}{\Lambda} \int_0^\infty d\zeta e^{-\zeta r} \left(\frac{1}{\zeta^3} + \frac{1}{2\zeta^5} \right) \sqrt{\zeta^2 - 1},$$

where $\Lambda = \hbar/mc$ is the reduced Compton length and

$$K_0(x) = \int_1^\infty d\zeta e^{-x\zeta} \left(\frac{1}{\zeta^3} + \frac{1}{2\zeta^5} \right) \sqrt{\zeta^2 - 1}.$$

Electron self-energy

The electron self-energy effect is delocal and hence exchange-like. However, some effective local potentials are available in the literature. Two of them are implemented in DIRAC:

1) Flambaum and Ginges SE potential

The Flambaum-Ginges potential Flambaum2005 is derived from the linear contribution to the self-energy process, but has been further modeled and parametrized to account for the full self-energy process to all orders in $(Z\alpha)$. It splits into two terms, associated with electric and magnetic form factors. The electric contribution is further divided into a high-frequency (HF) and low-frequency (LF) part. The high-frequency is properly derived from the electric form factor, but comes with a cutoff parameter λ to eliminate infrared divergency, whereas the low-frequency is simply modeled as a hydrogen-like function. The final forms are

$$\varphi^{\text{SE}}_{\text{FG}} = \varphi_{\text{mag}} + \varphi_{\text{HF}} + \varphi_{\text{LF}},$$

where

$$\begin{aligned} \varphi_{\text{mag}}(\mathbf{r}) &= \frac{\alpha \hbar}{4 \pi m_e c} i \gamma \cdot \nabla_{\mathbf{r}} \left[\Phi(\mathbf{r}) \left(K_m \left(\varphi_{\text{HF}}(\mathbf{r}) \right) \right) \right], \quad \lambda = \frac{\alpha}{\pi} A(Z) \Phi(\mathbf{r}) K_e \left(\frac{r_x}{\lambda} \right) \\ \varphi_{\text{LF}}(\mathbf{r}) &= -\frac{B(Z)}{e} Z^4 \alpha^5 m_e c^2 e^{-Z r_x / a_0}. \end{aligned}$$

The functions $K_m(x)$ and $K_e(x)$ are as follows:

$$K_m(x) = \int_1^{\infty} d\zeta \frac{e^{-x\zeta}}{\zeta^2 \sqrt{\zeta^2 - 1}};$$

$$K_e(x) = \int_1^{\infty} d\zeta \frac{e^{-x\zeta}}{\sqrt{\zeta^2 - 1}} \left(-\frac{3}{2} + \frac{1}{\zeta^2} + \left(1 - \frac{1}{\zeta^2} \right) \right)$$

A and B are fitting functions adjusted to reproduce the self-energy corrections of hydrogen-like atoms. $\Phi(\mathbf{r})$ is the nuclear electrostatic potential. The above forms are strictly only correct for a point-charge nuclei, that is, for $\Phi(\mathbf{r}) = \frac{Ze}{4\pi\epsilon_0 r_x}$. The corresponding potentials for extended nuclei are properly obtained by convolution with the nuclear charge distribution. However, the above approximation has been reported to work well in the case of the Uehling potential Artemyev1997.

2) Pyykko and Zhao SE potential

This is a Gaussian-type model potential where the parameters are fitted to reproduce the orbital energy and M1 hyperfine splitting within the range $29 \leq Z \leq 83$ Pyykko2003.

$$\varphi^{\text{SE}}_{\text{PZ}}(\mathbf{r}) = B(Z) e^{-(\beta(Z) r_x^2)}$$

B and β are fitting functions.

How to use

Effective QED potentials, which are one-electron operators, should be added to the one-electron Hamiltonian, as follows:

$$\hat{H}_D \rightarrow \hat{H}_D - e \varphi_{\text{effQED}}$$

$$\varphi_{\text{effQED}} = \sum_A^{\text{nuc}} \left(\varphi_{\text{SE}}^A + \varphi_{\text{VP}}^A \right)$$

The DIRAC implementation was obtained by grafting code from the numerical atomic code GRASP GRASP onto the DFT of grid. See *EFFQED for tuning options. The present input gives precise control, but is admittedly somewhat cumbersome.

Example 1: DCG Hamiltonian with Uehling and FG potentials (variational treatment)

ueh_fg.inp

Although the γ operator appears in the expression of the magnetic term above, the reader may find that the matrix part of the magnetic term is ALPHADOT (i.e., this one-electron operator involves the scalar product of α and the scalar operators XSEFGMG, YSEFGMG, and ZSEFGMG). This input is correct because the β part in $\gamma = \beta\alpha$ is already included in the code for a technical reason. In this example, the molecular orbitals are relaxed due to the QED effects. The contributions of effective QED potentials are included here in the total energy.

Example 2: Expectation values of Uehling and PZ potentials (perturbative treatment)

ueh_pz_exp.inp

In this example, the expectation values of the effective QED potentials are obtained using the molecular orbitals calculated based on the Dirac-Coulomb-Gaunt (DCG) Hamiltonian. That is, effQED potentials are treated as perturbations, and the corresponding energies are obtained using first-order perturbation theory. In this case, the individual contributions of effQED potentials to the total energy are printed out in the property section.

Note on the application to core-properties

The PZ potential was fitted to reproduce the QED effects on the hyperfine coupling constant and orbital energies (1s and 2s orbitals), and the fitting functions for the FG potential were adjusted to reproduce the self-energy corrections to high s- and p-states of hydrogenic ions.

However, we have confirmed that these potentials cannot accurately account for QED effects on the orbital energies for core orbitals. In fact, these potential provide different QED effects on the parity-violating energy Sunaga2021.

Therefore, these potential would not accurately provide QED effects on the core properties, such as XPS, Mössbauer, and NMR.

It has been confirmed that these SE potentials yield very similar results in the case of valence properties Sunaga2022.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/reladc/atom.md

orphan

Now we turn to the cadmium atom in D_{2h} with the HAMILTONIAN_, SPINFREE Hamiltonian. The D_{2h} double group has two fermionic IRREPS (g and u) leading to XREPS=1,2. By using the HAMILTONIAN_, SPINFREE option a real calculation in the ordinary D_{2h} point group is performed in DIRAC leading to eight symmetries as known from the character table. Therefore we set XREPS=1,2,3,4,5,6,7,8 if we need all symmetries. The relation of the irrep name to the number can be obtained from the Dirac output and is reproduced in the ADC code as well. After the calculation the A_{1g} final states are the in SSPEC.01, the B_{2g} in SSPEC.02 asf. The complete spectrum is then the merge of SSPEC.01 ... SSPEC.08. Remember that for the plot we only need the IP and the corresponding pole strength. We therefore do a grep '@' on the merged SSPEC.X files (cat SSPEC.* > SPEC.all) and obtain all lines in each symmetry. If we need only a specific range we do "sort -n SPEC.all > SPEC.range" and edit according to our needs. Then we can use gnuplot with "plot 'SPEC.range' u 1:2 w i". For ADC(3) with constant diagrams and 600 Lanczos iterations we would have an input like this:

```

**DIRAC
.TITLE
  input for Cd atom
.WAVE F
.4INDEX
**GENERAL
.DIRECT
  1 1 1
**INTEGRALS
*READIN
*TWOINT
.SOFCK
.SCREEN
1.E-16
**HAMILTONIAN
.SPINFREE
**WAVE FUNCTIONS
.SCF
.RELADC
*SCF
.CLOSED
  30 18
.FCKCNV
5.0E-09
.INTFLG
  1 1 1
**MOLTRA
.INTFLG
  1 1 1
.CORE
1..8
1..6
.ACTIVE
9..25
7..22
**RELADC
.DOSIPS
.ADCLEVEL
  3
.SIPREPS
  8
  1,2,3,4,5,6,7,8
**LANCZOS
.SIPITER
  600
*END OF

```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/reladc/triatomic_molecule.md

orphan

Here we consider a bent triatomic molecule in C_s symmetry. We have only one fermionic IRREP and complex MOs. This is indicated in the ADC output reporting complex ADC matrices. We want to do single ionizations (DOSIPS=T) at the ADC(3) level (ADCLEVEL=3) including constant diagrams (DOCONST=default) and compute all final states in symmetry 1, the only available one (XREPS=1). If the basis set is large, we restrict the iterations for the constant diagrams by releasing convergence a bit (VCONV=1.0E-05) because this step takes longest. We request 1000 Lanczos iterations and consider ionization energies up to 100 eV on screen. The complete spectrum is written to SSPEC.01. You get as many entries as you have Lanczos iterations. A characteristic of the iterative diagonalizer is that the eigenvalues at the edge of the spectrum converge very fast and are reproduced for high iteration numbers. In order to get higher eigenvalues equally tightly converged the number of iterations has to be increased accordingly. In this case the spurious (already converged) eigenvalues are projected out. For the C_s molecule the ADC input would then look like this:

```

**RELADC
.DOSIPS
.SIPREPS
  1 # How many requiered IRREPS are going to follow in the next line
  1
.VCONV
  1.0E-05
**LANCZOS
.SIPITER
  1000
.SIPPRNT
  100.0

```

This part has to be included into an input file, which contains the information about the Hamiltonian, the appropriate number of electrons,

Since we do not activate DIPs nothing else needs to be specified in the Lanczos section.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/relbas.md

orphan

Basis sets for relativistic calculations

Gaussian Type Orbitals (GTOs)

Cartesian Gaussians are defined as

$$G^{\{\alpha\}}_{ijk} = G^{\{\alpha\}}_i(x) G^{\{\alpha\}}_j(y) G^{\{\alpha\}}_k(z)$$

with

$$G^{\{\alpha\}}(x) = \left(\frac{2\alpha}{\pi} \right)^{1/4} \sqrt{\frac{\left(4\alpha \right)^i}{\left(2i-1 \right)!!}} x^i \exp \left[-\alpha x^2 \right]$$

Alternatively one may express a Cartesian Gaussian as

$$G^{\{\alpha\}}_{ijk} = N^{\{\alpha\}}_{ijk} x^i y^j z^k \exp \left[-\alpha r^2 \right]; \quad N^{\{\alpha\}}_{ijk} = \left(\frac{2\alpha}{\pi} \right)^{3/4} \sqrt{\frac{f_{ijk}}{\left(2i-1 \right)!! \left(2j-1 \right)!! \left(2k-1 \right)!!}}$$

where the sum f_{i+j+k} is associated with orbital angular momentum Γ . The factor

$$F_{ijk} = \left(2i-1 \right)!! \left(2j-1 \right)!! \left(2k-1 \right)!!$$

shows that the Cartesian components of a given shell may have different normalization constants. In the HERMIT integral code a single normalization is chosen for each shell by ignoring F_{ijk} meaning that Cartesian Gaussians are normalized to

$$\int G^{\{\alpha\}}_{ijk} G^{\{\alpha\}}_{ijk} d\mathbf{r} = F_{ijk}$$

In practice this means that s- and p-functions are normalized to one. So are d_{110} , d_{101} and d_{011} , whereas d_{200} , d_{020} and d_{002} are normalized to three. f_{111} is normalized to one, f_{210} , f_{201} , f_{120} , f_{102} and f_{012} are normalized to three, and f_{300} , f_{030} and f_{003} are normalized to 15.

Spherical Gaussians are defined by

$$G^{\{\alpha\}}_{lm} = R^{\{\alpha\}}_l \left(r \right) Y_{lm} \left(\theta, \phi \right); \quad \text{quad}$$

where the angular part is given by spherical harmonics Y_{lm} and the radial part by

$$R^{\{\alpha\}}_l = N^{\{\alpha\}}_l r^l \exp \left[-\alpha r^2 \right]; \quad \text{quad} \quad N^{\{\alpha\}}_l = \frac{\left(2\alpha \right)^{3/4} \pi^{1/4}}{\left(2l+1 \right)!! \sqrt{\left(2\alpha \right)}}$$

In passing we note that

$$N^{\{\alpha\}}_{l=2} \sqrt{\frac{\alpha}{2l+1}} N^{\{\alpha\}}_{l-1}$$

For given Γ there are $\frac{1}{2} \left(l+2 \right) \left(l+1 \right)$ spherical Gaussians. The latter basis functions therefore provide more compact basis set expansions. However, in 4-component relativistic calculations the use of spherical Gaussians is somewhat more complicated since the coupling of large and small component basis functions needs to be taken into account.

Kinetic balance

Kinetic balance corresponds to the non-relativistic limit of the exact coupling of large and small component basis functions for positive-energy orbitals. Starting from the radial function of a large component spherical Gaussian basis function one obtains

$$R^S \propto -\left(\frac{\partial}{\partial r} \right) \left(\frac{1}{r} \right) + \frac{\left(1+\kappa^L \right)}{r} R^{\{\alpha\}}_l = \sqrt{\alpha} \left(2l+3 \right) R^{\{\alpha\}}_{l+1} - 2 \sqrt{\alpha} \left(2l+1 \right) R^{\{\alpha\}}_{l-1}$$

We can now distinguish two cases

$$\begin{aligned} R^S &\propto \left(\frac{\partial}{\partial r} \right) \left(\frac{1}{r} \right) + \frac{\left(1+\kappa^L \right)}{r} R^{\{\alpha\}}_l \\ &= \sqrt{\alpha} \left(2l+3 \right) R^{\{\alpha\}}_{l+1} - 2 \sqrt{\alpha} \left(2l+1 \right) R^{\{\alpha\}}_{l-1} \quad \& \quad \text{if } \kappa^L = l; \\ &= \sqrt{\alpha} \left(2l+3 \right) R^{\{\alpha\}}_{l+1} - 2 \sqrt{\alpha} \left(2l+1 \right) R^{\{\alpha\}}_{l-1} \quad \& \quad \text{if } \kappa^L = -(l+1); \end{aligned}$$

The case $\kappa^L < 0$ is straightforward, but a bit more care is needed for the implementation for the case $\kappa^L > 0$. The modified spherical Gaussian to be constructed is

$$G^*_l = N \left(\frac{1}{\sqrt{\left(2l+1 \right) \left(2l-1 \right)}} \right) R^{\{\alpha\}}_{l-2} \left(2l-1 \right) R^{\{\alpha\}}_{l-2} Y_{l-2,m}; \quad \text{quad} \quad N = \frac{1}{\sqrt{\left(2l+1 \right) \left(2l-1 \right)}}$$

Constructing modified spherical harmonics for kinetic balance; the gritty details

We write the spherical harmonic $G^{\{\alpha\}}_{l-2,m}$ as a linear combination of Cartesian Gaussians

$$G^{\{\alpha\}}_{l-2,m} = \sum_{i+j+k=l-2} c^{\{l-2,m\}}_{ijk} G^{\{\alpha\}}_{ijk}$$

In the HERMIT integral code we have selected a transformation such that the solid harmonics are normalized to unity. From this we obtain

$$\begin{aligned} G^*_l &= N \left(\frac{1}{\sqrt{\left(2l+1 \right) \left(2l-1 \right)}} \right) \sum_{i+j+k=l-2} c^{\{l-2,m\}}_{ijk} G^{\{\alpha\}}_{ijk} \\ &= N \sum_{i+j+k=l-2} c^{\{l-2,m\}}_{ijk} \left(\frac{1}{\sqrt{\left(2l+1 \right) \left(2l-1 \right)}} \right) \sum_{i+j+k=l-2} c^{\{l-2,m\}}_{ijk} G^{\{\alpha\}}_{ijk} \end{aligned}$$

$$\begin{aligned} & \left(\frac{N_{\alpha_{ijk}} N_{\alpha_{i+2,j,k}}}{N_{\alpha_{ijk}} N_{\alpha_{i,j+2,k}}} G_{\alpha_{i+2,j,k}} \right. \\ & + \frac{N_{\alpha_{ijk}} N_{\alpha_{i,j,k+2}}}{N_{\alpha_{ijk}} N_{\alpha_{i,j,k+2}}} G_{\alpha_{i,j,k+2}} \\ & \left. - 2 \left(2l-1 \right) G_{\alpha_{ijk}} \right) \end{aligned}$$

The ration of normalization constants in the above relation is given by

$$\frac{N_{\alpha_{ijk}} N_{\alpha_{i+2,j,k}}}{N_{\alpha_{ijk}} N_{\alpha_{i,j+2,k}}} = \frac{1}{4\alpha} \sqrt{\frac{F_{i+2,j,k}}{F_{i,j,k+2}}}$$

However, the expression is much simplified by the fact that factors F_{ijk} are set to one in HERMIT, such that

$$G_{lm} = N \sum_{i+j+k=l-2} c_{l-2,m}_{ijk} \left(\left(G_{\alpha_{i+2,j,k}} + G_{\alpha_{i,j+2,k}} + G_{\alpha_{i,j,k+2}} \right) - 2 \left(2l-1 \right) G_{\alpha_{ijk}} \right)$$

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/relci_q_corrections/q_corrections.md

orphan

Davidson size-consistency corrections for truncated CI wave functions

Background

A serious deficiency of truncated CI expansions, including the most popular ones, namely (multi-reference; MR)CI-SD is the lack of size-consistency; i.e., the energy of the system does not scale properly with the system size. Besides corrections to the CI equations which will improve the energy and the wave function, *a posteriori* corrections (most often referred to as *Davidson-type* corrections or $+Q$) to the energy have become quite popular. A comprehensive overview on this topic has been given recently in a [review paper on multiconfigurational and multireference methods](#). Starting with Dirac14 we have implemented a number of different flavors of the latter *a posteriori* corrections to the energy for the modules KRCI with LUCIAREL or with LUCITA. The nomenclature and detailed equations for each correction is explained in great detail in the above review article (see chapter 2.1.3.1).

Example: Water

Following the example for water in the [review article](#) we need to run two calculations. Although it's possible to do it in one step we will split the task in two.

Step 1: Get reference wave function and configuration(s)

We first start with a multiconfigurational SCF calculation for water with a CAS(4,4) space using a cc-pVDZ basis set. This will yield a suitable reference wave function for the subsequent CI. The MCSCF module will in addition automatically create a file named **refvec.luci** which contains the reference configurations needed later for the $+Q$ corrections. Our CASSCF input `scf-casscf.inp` is

`scf-casscf.inp`

The molecular structure is located in `h2o.xyz`

`h2o.xyz`

Suppose that we have run the CASSCF calculation using:

`pam --inp=scf-casscf.inp --mol=h2o.xyz --outkrmc --get="refvec.luci"`

where we keep the MCSCF coefficients (in KRMCSF) and the reference vectors stored in `refvec.luci` we are now in position to proceed to Step 2.

Step 2: Run the MRCISD calculation and compute the $+Q$ corrections

In the second step we now run a MRCISD calculation using the reference wave function coefficients from the previous step as starting point and compute the $+Q$ corrections to the MRCISD energy. Using the same molecular input structure we run the MRCISD+ Q calculation using the input file `krqi-q.inp`

`krqi-q.inp`

with the following start command:

`pam --inp=krqi-q.inp --mol=h2o.xyz --inkrmc --put="refvec.luci"`

We will find a summary of different $+Q$ corrections printed at the end of the output:

`summary_q.txt`

where a detailed account of the general *performance* of the different $+Q$ corrections (and which other QC programs have implemented which $+Q$ correction by default) is given in chapter 2.1.3.1 of the above given review article. The equations references given in the output match the equation labels given in chapter 2.1.3.1. We strongly recommend the interested user to carefully read this chapter 2.1.3.1.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/reproducing_nr_results.md

orphan

How to reproduce nonrelativistic results

DIRAC nonrelativistic Hamiltonians

To approach the nonrelativistic limit you can either switch to the 4-component Lévy-Leblond Hamiltonian:

```
**DIRAC
[...]
**HAMILTONIAN
.LEVY-LEBLOND
```

or the 2-component nonrelativistic (Schrodinger) Hamiltonian:

```
**DIRAC
[...]
**HAMILTONIAN
.NONREL
```

Changing speed of light

Another option to reproduce nonrelativistic data with the Dirac relativistic Hamiltonian is to increase the speed of light (here to 2000 a.u.):

```
**DIRAC
[...]
**GENERAL
.CVALUE
2000.0
```

Adjusting the nucleus model

Most nonrelativistic programs employ a point nucleus model. Point nucleus is default for .NONREL and Gaussian nucleus is default for .LEVY-LEBLOND. You can always force point nuclei in DIRAC:

```
**DIRAC
[...]
**INTEGRALS
.NUCMOD
1
```

Basis sets

With the above mentioned nonrelativistic Hamiltonians the user can also employ variety of standard nonrelativistic basis sets present in the “basis_dalton” directory.

Altogether, to reproduce results of most nonrelativistic packages the user should set nonrelativistic Hamiltonian, switch to point nucleus if necessary for the compatibility, and choose some nonrelativistic basis set.

Recommended method of choice is the closed shell Hartree-Fock SCF, which can be followed by the CCSD(T) correlation method. In this case you are to check the occupied shells, and active space, respectively.

An example using DALTON

Apart from specifying a nonrelativistic Hamiltonian and a point charge nuclear model in the DIRAC input, the user should be aware of the fact that DIRAC uses Cartesian Gaussian functions, while DALTON uses Spherical Gaussian functions.

One route is to force DALTON to use Cartesian Gaussian functions (refer to the DALTON manual) and the following DIRAC input file:

```
**DIRAC
[...]
**GENERAL
.SPHTRA
0 0
**INTEGRALS
.NUCMOD
1
**HAMILTONIAN
.LEVY-LEBLOND
*END OF
```

A second route is to use DALTON defaults regarding basis functions and the following DIRAC input file:

```
**DIRAC
[...]
**INTEGRALS
.NUCMOD
1
```

```

**HAMILTONIAN
. LEVY-LEBLOND
*END OF

```

The user should make sure that the geometry is specified in atomic units as the conversion factor from Angstrom to AU may be different in the two programs.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/restart/coupled_cluster.md

orphan

How to run a Coupled-Cluster calculations in separate steps

Typically there are no problems in executing the three steps in a coupled-cluster calculation with RELCCSD: 1) the Hartree-Fock calculation, 2) the MOLTRA step, the integral transformation from atomic to molecular orbitals, and 3) the coupled-cluster calculations itself all in one calculation.

However, in some cases it is not the best approach to perform all these in one go. This is especially true for calculations that use a lot of memory in the coupled-cluster part but are more economical in the previous steps (say, if the system has little or no symmetry or, in Fock-Space calculations, if the active space is large).

In such cases, it is possible to do one step, save the results necessary for the next calculation, and then run the next step. This can considerably shorten the total wall time spent on a parallel calculation, since a larger number of MPI processes can be started for the SCF (and/or MOLTRA) step(s) with the same amount of memory per compute node.

Step 1: SCF

From the SCF part, one has to save the DFCOEF file after a successful calculation. If using the 2-component formalism, one can also save the files X2CMAT and AOMOMAT (and possibly AOPROPER), in order to skip the generation of the 4-component to 2-component transformation matrices. If your system supports it, you could keep the scratch directory and then avoid the copying back and forth of these files.

Currently DIRAC's execution script automatically retrieves these files and places them in a gzip-compressed tar archive, so it is probably not necessary to retrieve the files individually if this archive is kept.

Step 2: MOLTRA

MOLTRA will need the DFCOEF file, and also the one-electron Fock matrix (if the 2-component methods are used and also in the case of an frozen density embedding calculation, this will have to be re-generated, so that's why it is a good idea to keep the X2CMAT and AOMOMAT files around.

The result of the MOLTRA step will be a set of files that start with MDC... For SCHEME 4 these files can be considered as 'independent' of the number of processes used in the parallel run (because only one, created by the master, will have information used by other modules like RELCCSD - such as the number of spinors transformed etc.), so it does not matter how the calculation from which they were obtained was performed.

This is not true for SCHEME 6, because there the information is needed by other modules, and then one has to be careful to use the same number of MPI processes in the MOLTRA step and beyond.

Step 3: RELCCSD

RELCCSD will need the MDC* files generated by MOLTRA. One must be careful in the input, however; the keyword .N04INDEX has to be specified, and the **MOLTRA section must not be present in the input. Again, for SCHEME 6 one must be careful to have the same number of MPI processes as in the MOLTRA step.

A note on Fock-Space Calculations

In the case of Fock-Space calculations, an additional trick can be used: if previous/lower sectors are converged, RELCCSD can be restarted and be made skip them. In order to do that one has to do the following apart from the actions outlined above: First, instead of MAXIT=n, use MAXITk=n (where k denotes the sector: 00, 10, 01, 11, 02, 20) and set to zero all the MAXITab for converged sectors to zero. Then restart exactly the same calculation as before (in terms of number of nodes etc.). It is important to have the intermediate files for RELCCSD present in the scratch directory in this case. The easiest way to do so is to keep the scratch directory from the previous run.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/restart/dfcoef_and_dfpcmo.md

orphan

What is the difference between DFCOEF and DFPCMO?

Both files hold the MO coefficients (and some other information) and can be used to restart SCF calculations.

DFCOEF is always generated. To generate DFPCMO, you need the GENERAL_,PCMOOUT keyword.

To see the difference between the two files, run a calculation with:

```

**GENERAL
. PCMOOUT

```

and save both files from the scratch directory:

```
$ pam ... --get "DFCOEF DFPCMO"
```

and then inspect both files. You will see that only DFPCMO is human-readable. Both contain the **same** information. The DFCOEF file is machine-readable and machine-dependent which means that DIRAC cannot restart from a DFCOEF file generated on a different architecture but you can always restart from DFPCMO on any machine. The advantage of the DFCOEF file is that it is smaller in size (but with `gzip` you can compress DFPCMO files to the size of DFCOEF files). The advantage of DFPCMO files is that you can read them yourself (try to understand what the numbers mean) and use them in external scripts and programs.

You do not need any keywords for restarting, simply copy DFPCMO or DFCOEF to the scratch directory and DIRAC will restart automatically.

Try it. It is important that you know how to restart. This will save you many troubles and CPU hours.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/restart/scf.md

orphan

Restarting SCF calculations

A DIRAC calculation can be restarted at various stages depending on the type of calculation. In general it should be enough to give the archive file of a previous calculation as an argument to the `pam` script. DIRAC will then find and use the available data for restart. Alternatively you can copy restart files in using `pam --put` and if DIRAC finds them in the scratch directory, it will automatically restart from them. In general the program can restart from

- DFCOEF Contains orbital coefficients in machine-readable format, and will be used for restarting the SCF procedure.
- DFPCMO Contains orbital coefficients in human-readable format, and will be used for restarting the SCF procedure.
- PAMFCK The Fock matrix. Will be used for restarting the SCF procedure.
- A0PROPER Contains one-electron integrals.
- X2CMAT Contains four- to two-component transformation matrices, and allows you to skip the transformation step of a two component calculation.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/restart/troubleshooting.md

orphan

Troubleshooting

Can I move the DFCOEF file from one machine to another and restart from it?

In general no, unless it is the same architecture.

What you can do instead is to export the DFCOEF in a machine independent format using the `.PCMOUT` keyword. This creates a file called DFPCMO. Copy DFPCMO to the other machine, copy it to the scratch directory using the `pam` script, and DIRAC will correctly restart from it.

I tried to restart but get the error “Incompatible number of basis functions” - what does it mean?

This probably means that you have changed the basis set. Presently DIRAC cannot restart from DFCOEF or DFPCMO if the basis set changes (more correctly if matrix dimensions change).

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/restart/x2c.md

orphan

Restarting X2C calculations

Here we will show how one can run a Hartree–Fock (HF) calculation in 2-component mode with the exact two component (X2C) Hamiltonian and restart a property calculation at the HF level in a second run without the need to re-do the X2C transformation. For the initial SCF step we use the following input file (copy the content below into the `scf_x2c.inp` file):

```
**DIRAC
.WAVE FUNCTION
**MOLECULE
*BASIS
! Use the same basis for all atoms
.DEFAULT
dyall.av3z
**HAMILTONIAN
! Use the exact two-component Hamiltonian
! with atomic mean field spin-orbit
.X2C
**INTEGRALS
*READIN
! A limitation in the AMFI code requires an uncontracted basis
.UNCONTRACT
**WAVE FUNCTION
```

```
.SCF
*END OF
```

together with the following molecule file (copy it into the br2.xyz file):

```
2
Br 0.0 0.0 1.244
Br 0.0 0.0 -1.244
```

This molecule has linear symmetry (Dinf.h), and the program will detect this automatically. In most parts of the program the highest abelian subgroup (D2h) will be used instead of the full linear symmetry. In this example we use Dyall's augmented TZ basis dyall.av3z in uncontracted form.

Prior to the HF calculation, the Hamiltonian transformation from 4- to 2-components will be performed and for restart purposes one should therefore save the files X2CMAT and AOMOMAT containing the Hamiltonian information and the file DFC0EF containing the (converged) MO-coefficients.

We run the initial SCF step using:

```
pam --inp=scf_x2c.inp --mol=br2.xyz --get="X2CMAT AOMOMAT DFC0EF"
```

The output will be written to the file scf_x2c_br2.out, which can be monitored during the calculation.

In order to proceed in a second step with a property calculation of, e.g. the contact density at the Br nucleus at the HF level, we use the following input file (copy and paste it into the property_x2c.inp input file):

```
**DIRAC
!.WAVE FUNCTION      commented out to skip SCF step
.PROPERTIES
**MOLECULE
*BASIS
! Use the same basis for all atoms
.DEFAULT
dyall.av3z
**HAMILTONIAN
! Use the exact two-component Hamiltonian
! with atomic mean field spin-orbit
.X2C
**INTEGRALS
*READIN
! A limitation in the AMFI code requires an uncontracted basis
.UNCONTRACT
**WAVE FUNCTION
.SCF
**PROPERTIES
.RHONUC
*END OF
```

Restarting from the (converged) MO-coefficients and reading the transformed X2C Hamiltonian from file is done automatically once the files are present. We therefore restart the calculation using:

```
pam --inp=property_x2c.inp --mol=br2.xyz --put="X2CMAT AOMOMAT DFC0EF"
```

Restart files

AOMOMAT and X2CMAT never change during the SCF iterations, hence you can use them for an SCF restart at the same geometry. If you however change the geometry and restart from those files, all results will be wrong.

AOMOMAT contains amongst other things the $S^{\pm 1/2}$ overlap matrix which is needed to transform the Fock operator from AO to orthonormal MO basis before diagonalization. X2CMAT contains the transformed two-component one-electron operator with possible two-electron spin-orbit corrects added as well as all property operators (in AO basis) in proper two-component form. All these quantities are independent of the current solution in an SCF iteration.

Shen X2C is restarted with just:

```
pam --put="DFC0EF"
```

(i.e. not putting AOMOMAT and X2CMAT), the X2C code will recompute AOMOMAT and X2CMAT automatically, which takes just a few extra seconds. The subsequent SCF iterations or property evaluations deliver the same results as if AOMOMAT and X2CMAT had been put to the scratch directory.

An X2C calculation currently cannot be restarted by:

```
pam --fullrestart="ARCHIVE.tgz"
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/rotg/tutorial.md

orphan

Molecular rotational g-tensors

Introduction

In this tutorial we introduce the calculation of molecular rotational g-tensors as given in the DIRAC code, based on the theoretical developments by I. Agustín Aucar *et al.* For details, you are welcome to consult Aucar_JCP2014.

The molecular rotational g-tensor elements are given by

$$g_{\alpha\beta} = g_{\alpha\beta}^{\text{nuc}} + g_{\alpha\beta}^{\text{elec}}$$

They have two terms: the first of them is independent of the electronic variables, whereas the second one is given by a linear response function.

In tensorial notation, the rotational g-tensor contributions of a molecule are given by:

$$\begin{aligned} \mathbf{g}^{\text{nuc}} &= m_p^{-1} \sum_M \mathbf{Z}_M \left[\left(\mathbf{R}_{M,G0} \cdot \mathbf{R}_{M,CM} \right) \mathbf{1} - \mathbf{R}_{M,G0} \mathbf{R}_{M,CM} \right] \\ \mathbf{g}^{\text{elec}} &= \mathbf{g}^{\text{LR}} = m_p^{-1} \langle \mathbf{r} | \mathbf{r} | \mathbf{G0} \rangle \cdot \mathbf{c} \cdot \langle \mathbf{r} | \mathbf{r} | \mathbf{CM} \rangle \end{aligned}$$

where $\mathbf{R}_{M,G0}$ is the position of nucleus M with respect to the gauge origin; $\mathbf{R}_{M,CM}$ is the position of nucleus M with respect to the molecular center of mass; $\mathbf{G0}$ is the position of the gauge origin; $\mathbf{I}$ is the inertia tensor of the molecule and $\mathbf{J}_e = \langle \mathbf{r} | \mathbf{r} | \mathbf{CM} \rangle \cdot \langle \mathbf{r} | \mathbf{r} | \mathbf{G0} \rangle$ is the electronic total angular momentum.

Application to the HF molecule

As an example, we show a calculation of the g-tensor of the Hydrogen fluoride molecule. The input file rotg.inp is given by

```
rotg.inp
```

whereas the molecular input file HF_cv3z.mol is

```
HF_cv3z.mol
```

The calculation is run using:

```
pam --inp=rotg --mol=HF_cv3z
```

As a result, g-factors are obtained at the coupled Hartree-Fock level. The code also works at the DFT level.

It is also interesting to note that as HAMILTONIAN_URKBAL is requested in the present calculation, the results will be very close to those obtained using the RKB prescription, because the diamagnetic-like (e-p) contributions to $g_{\alpha\beta}^{\text{LR}}$ are almost zero (see Eq. 41 of Aucar_JCP2014).

Reading the output file

As the PROPERTIES_.PRINT flag in the input file is set to 1, the results are fully detailed.

The g-factor Hydrogen fluoride molecule will look like:

```
rotg-HF
```

One should recall that in the case of linear molecules, as the present one, only one tensor element is printed out (the g-factor) because for these molecules the g-tensor has only two equal and non-zero diagonal elements.

As it can be seen, the total g-tensor of the Hydrogen fluoride molecule is given, and then separated in its two terms (nuc and LR). The latter contribution, the linear response function, is further separated in their (e-e) and (e-p) linear response parts, as well as in their $\mathbf{L}$ and $\mathbf{S}$ parts. It is seen how the (e-p) contribution to the linear response term of the g-factor is almost zero.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/simple_magnetic_balance/tutorial.md

orphan

Calculation of NMR shieldings using simple magnetic balance

In this tutorial we look at 4-component relativistic calculations of NMR shielding constants. For best results we recommend to use the scheme of simple magnetic balance (sMB) in conjunction with London orbitals Olejniczak2012.

Basic theory of magnetic balance

We encourage the reader to consult the original paper Olejniczak2012 for full details; here we just provide some key points.

In the absence of magnetic fields the coupling between the large and small components of the Dirac equation is

$$c \psi^S = \frac{1}{2m} \left[1 + \frac{E-V}{2mc^2} \right]^{-1} (\vec{\sigma} \cdot \vec{p}) \psi^L$$

The exact coupling can not be implemented *per se* at the basis set level due to the energy dependence of the coupling. One therefore employs the non-relativistic limit of this coupling

$$\lim_{c \rightarrow \infty} c \psi^S = \frac{1}{2m} (\vec{\sigma} \cdot \vec{p}) \psi^L,$$

valid for positive-energy solutions (and extended nuclear charge distributions). Construction of the small component basis in this manner assures the correct representation of the kinetic energy operator in the non-relativistic limit, and the coupling is therefore referred to as *kinetic balance* Stanton1984. In a 2-spinor basis kinetic balance can be implemented in such a manner that there is a 1:1 ratio between the sizes of the large and small component basis; this is referred to as *restricted kinetic balance* (RKB).

$$\chi_{\mu}^{2c;S} = (\vec{\sigma} \cdot \vec{p}) \chi_{\mu}^{2c;L}$$

In a scalar basis the straightforward implementation of RKB is not possible and one typically resorts to *unrestricted kinetic balance* (UKB)

$$\chi_{\mu}^{1c;S} = \vec{p} \chi_{\mu}^{1c;L},$$

basically generating the small component basis functions as derivative of the large component ones.

Magnetic fields are introduced through minimal substitution

$$\vec{p} \rightarrow \vec{p} + e \vec{A}$$

which manifestly modifies the coupling between the large and the small components. The non-relativistic limit now reads

$$\lim_{c \rightarrow \infty} c \psi^S = \frac{1}{2m} (\vec{\sigma} \cdot \vec{p}) \psi^L$$

and is referred to as *magnetic balance* Aucar1999. Magnetic balance needs to be taken into account when constructing the first-order correction to the wave function upon inclusion of an external magnetic field and is not assured in a RKB basis Pecul2004. In the cited paper Olejniczak2012 we show that magnetic balance can be obtained in a Gaussian basis combining UKB with London orbitals. However, UKB generally leads to a small component basis larger than the large component basis set from which it was generated and may increase computational cost as well as introduce linear dependencies in the basis set as compared to RKB. We therefore propose calculations of NMR shielding parameters by a simple two-step procedure described in the next section.

A sample calculation

We will consider a 4-component relativistic density functional calculation of the NMR shielding constant of hydrogen fluoride using the KT2 functional Keal2003. The molecular input file `hf.mol` reads:

`hf.mol`

Step 1: SCF calculation

The first step is the generation of the Kohn–Sham wave function in a RKB basis. Although DIRAC employs scalar Gaussian basis functions restricted kinetic balance is by default imposed when transforming to orthonormal basis Visscher2000.

The menu file `scf.inp` reads:

`scf.inp`

and we make sure to recover the coefficient file `DFC0EF` when running the calculation:

`scf.run`

Step 2: Response calculation

In the second step we start from the RKB coefficient obtained in the SCF calculation, but DIRAC will add the UKB complement and calculate the NMR shielding constant using unrestricted kinetic balance combined with London orbitals. The menu file `response.inp` accordingly reads:

`response.inp`

and the `pam` command is:

`response.run`

Comparison with RKB calculations

With the above computational procedure we obtain the following NMR shielding constants:

isotropic shielding

atom	total	dia	para
H	29.9502	21.8737	8.0765
F	415.8316	480.2015	−64.3699

By not adding the UKB complement (deleting `.RKBIMP` and `.URKBAL`), that is, using RKB, gives:

isotropic shielding

atom	total	dia	para
H	28.8028	20.7264	8.0765
F	323.2672	387.6459	−64.3787

If we in addition switch off the use of London-orbitals (deleting `.LONDON`) we obtain:

isotropic shielding

atom	total	dia	para
H	28.2920	20.2563	8.0357
F	322.9065	387.1375	-64.2310

The experimental value for the absolute isotropic shielding available for H in HF is 28.72 ppm Chesnut1994.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/spinrot/tutorial.md

orphan

Nuclear spin-rotation constants

Introduction

In this tutorial we introduce the calculation of nuclear spin-rotation tensors as given in the DIRAC code, based on the theoretical developments by I. Agustín Aucar *et al.* For details, you are welcome to consult Aucar_JCP2012.

The nuclear spin-rotation (SR) tensor elements of a nucleus N are given by

$$M_{N,\alpha\beta} = M_{N,\alpha\beta}^{NU} + M_{N,\alpha\beta}^{EV} + M_{N,\alpha\beta}^{LR}$$

They have three terms: the first of them is independent of the electronic variables, whereas the second and third are given by an expectation value (EV) and a linear response (LR) function, respectively.

In tensorial notation, the nuclear spin-rotation contributions of a nucleus N are given, in SI units, by:

$$\begin{aligned} \mathbf{M}_N^{NU} &= \frac{1}{4\pi\epsilon_0 c^2} \frac{e^2}{h^2} \sum_{M \neq N} \frac{Z_M}{|\mathbf{R}_{MN}|^3} \left[\mathbf{R}_{M,CM} - \left(1 - \frac{Z_N m_p}{m_N g_N} \right) \mathbf{R}_{N,CM} \right] \mathbf{R}_{MN} \\ \mathbf{M}_N^{EV} &= \frac{1}{4\pi\epsilon_0 c^2} \frac{e^2}{h^2} \sum_{M \neq N} \left(1 - \frac{Z_N m_p}{m_N g_N} \right) \left[\mathbf{R}_{M,CM} - \left(1 - \frac{Z_N m_p}{m_N g_N} \right) \mathbf{R}_{N,CM} \right] \mathbf{R}_{MN} \\ \mathbf{M}_N^{LR} &= \frac{1}{4\pi\epsilon_0 c^2} \frac{e^2}{h^2} \sum_{M \neq N} \left[\mathbf{R}_{M,CM} - \left(1 - \frac{Z_N m_p}{m_N g_N} \right) \mathbf{R}_{N,CM} \right] \mathbf{R}_{MN} \end{aligned}$$

where $\mathbf{R}_{MN}$ is the position of nucleus M with respect to the position of nucleus N ; $\mathbf{R}_{N,CM}$ is the position of nucleus N with respect to the molecular center of mass; $\mathbf{I}$ is the inertia tensor of the molecule and $\mathbf{J}_e = (\mathbf{r} - \mathbf{r}_{CM}) \times \mathbf{p} + \mathbf{S}_e$ is the electronic total angular momentum.

For molecules at their equilibrium geometry, it is found that the sum of the first two terms, $\mathbf{M}_N^{NU} + \mathbf{M}_N^{EV}$, is equal to a new tensor which is independent of the electronic variables (see Eq. 60 of Aucar_JCP2012),

$$\mathbf{M}_N^{nuc} = \mathbf{M}_N^{NU} + \mathbf{M}_N^{EV} = \frac{1}{4\pi\epsilon_0 c^2} \frac{e^2}{h^2} \sum_{M \neq N} Z_M \left[\mathbf{R}_{M,CM} - \left(1 - \frac{Z_N m_p}{m_N g_N} \right) \mathbf{R}_{N,CM} \right] \mathbf{R}_{MN} \cdot \mathbf{I}^{-1}$$

Therefore, the electronic dependence of the nuclear spin-rotation tensor of a nucleus N in a molecule in equilibrium is completely given by its linear response term, $\mathbf{M}_N^{LR}$ (see Eq. 59 of Aucar_JCP2012).

[!NOTE] The current implementation gives by default (PROPERTIES_.PRINT values up to 3) results only for molecules in equilibrium.

Application to the HF molecule

As an example, we show a calculation of the SR constant of the fluorine nucleus at the Hydrogen fluoride molecule. The input file spinrot.inp is given by

spinrot.inp

whereas the molecular input file HF_cv3z.mol is

HF_cv3z.mol

The calculation is run using:

pam --inp=spinrot --mol=HF_cv3z

As a result, SR constants are obtained at the coupled Hartree-Fock level. The code also works at the DFT level.

It is also interesting to note that as HAMILTONIAN_.URKBAL is requested in the present calculation, the results will be very close to those obtained using the RKB prescription, because the diamagnetic-like (e-p) contributions to $M_{N,\alpha\beta}^{LR}$ are almost zero (see Eq. 60 of Aucar_JCP2012).

Reading the output file

As the PROPERTIES_.PRINT flag in the input file is set to 4, the results are fully detailed and given in both kHz and ppm units.

The SR constant of the fluorine nucleus, in kHz, will look like:

SR-HF-khz

One should recall that in the case of linear molecules, as the present one, only one tensor element is printed out (the nuclear spin-rotation constant) because for these molecules the SR tensor has only two equal and non-zero diagonal elements.

As it can be seen, the total SR constant of the fluorine nucleus is given, and then separated in its two terms (nuc and LR). The latter contribution, the linear response function, is further separated in their (e-e) and (e-p) parts, as well as in their $\mathbf{L}$ and $\mathbf{S}$ parts. It is seen how the (e-p) contribution to the linear response term of the SR constants is almost zero.

In addition, the results are fully detailed, showing the three terms mentioned above (NU, EV and LR).

Finally, if one is interested in the relationship between the SR constants and PROPERTIES_.SHIELDING (see Aucar_JPCL2016 and AucarChap2019), which is given, in SI units, by

$$\sigma_N = \frac{1}{4\pi\epsilon_0 c^2} \langle \mathbf{e}^2 \rangle \langle \mathbf{r} | \mathbf{r}_N | \mathbf{r} \rangle - \langle \mathbf{r} | \mathbf{r}_N | \mathbf{r} \rangle^3$$

then we print also the SR constants in ppm, by multiplying each SR tensor element $M_{N,\alpha\beta}$ (given in kHz) by $\frac{m_p I_{\beta\gamma}}{g_N \pi \hbar^2}$, where m_p and m_e are the proton and electron masses, respectively, $\hbar$ is the reduced Planck's constant, and g_N is the g-factor of nucleus N.

In this way, one obtains the following output:

SR-HF-ppm

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/srDFT/general.md

orphan

Long-range WFT/short-range DFT

Long-range wave function theory (lrWFT) combined with short-range density functional theory (srDFT) is based on a separation of the two-electron interaction

$$V_{12} = \frac{\text{erf}(\mu r_{12})}{r_{12}} - \frac{1 - \text{erf}(\mu r_{12})}{r_{12}}$$

here using the error function (erf). The first term, the long-range interaction, is assigned to wave function theory, whereas the second term, the short-range interaction, is assigned to DFT. Contrary to conventional DFT the fictitious reference (Kohn-Sham) system is no longer non-interacting and can not be solved exactly, so that the hierarchy of approximate wave function methods must be invoked. MP2-srDFT has been implemented in DIRAC by Ossama Kullie and Trond Saue Kullie2011.

Specification of functionals

srLDA

Short-range LDA is specified as:

```
**HAMILTONIAN
.DFT
CAM p:alpha=0.0 p:beta=1.0 p:mu=0.4 x:slater=1.0 c:svwn5=1.0
```

srGGA

Two flavours of short-range PBE has been implemented in DIRAC. The short-range PBE of Hirao and co-workers is specified as:

```
**HAMILTONIAN
.DFT
CAM p:alpha=0.0 p:beta=1.0 p:mu=1.0 x:PBEX=1.0 c:srPBEC=1.0
```

The short-range PBE of Scuseria and co-workers is specified as:

```
**HAMILTONIAN
.HFXATT
1.0D0
.HFXMU
1.0D0
.DFT
GGAKEY HJSX=1.0 srPBEC=1.0
```

MP2-srDFT

MP2-srDFT is activated by specifying the short-range DFT functional as above and complementing with:

```
**WAVE FUNCTIONS
.SCF
.MP2
```

Further specifications can be given in the *SCF and *MP2CAL subsections.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/start_guess/atomic_huckel.md

orphan

Extended Huckel start

Background

The basis theory is given here: Hoffman1963.

The present development was further motivated by Norman2012.

The particularity of the present approach is that it employs pre-calculated atomic fragments.

A start guess is generated by solving the following general eigenvalue problem

$$H\mathbf{c} = S\mathbf{c}\epsilon$$

where S is the overlap matrix in terms of pre-calculated atomic orbitals. The Hamiltonian matrix is defined as

$$H_{ij} = \left\langle \begin{array}{l} \epsilon_i \\ \text{KS}_{ij} \end{array} \right| \quad \text{for } i \neq j$$

where K is the Wolfsberg-Helmholtz constant. The default value is 1.75, but it can be changed with the keyword SCF_.HUCPAR.

Example

 alternate text
alternate text

We consider a Kohn-Sham calculation of the closed-shell LuF_3 molecule using the PBE functional. We start from the molecular input file LuF3.xyz

LuF3.xyz

and the menu file PBE.inp

PBE.inp

This menu file includes specifications to use uncontracted integrals and specifies a screening threshold for 2-electron integrals. The the method is specified; Here the Kohn-Sham DFT will be used with the PBE functional. The molecule is defined as having 98 closed shell electrons (for the fermion irrep without inversion symmetry). Further more, the Mulliken population analysis is specified to provide labels for individual orbitals and lastly the basis sets used are defined. Note that a special basis set is used for the Lu atom. The basis set must be specified in the menu file when using an xyz file rather than a mol file. Note that the keyword INPTES for testing the inputfile before the run is commented out. Observe that the calculation will take more than an hour. Unfortunately, and curiously, this calculation does not converge

PBE_LuF3.out

We therefore try the extended Hückel method for a better start guess. The input file PBEhuc.inp reads

PBEhuc.inp

Using the keyword SCF_.AD HOC file names for the Lu and F atomic types are given, followed by orbital strings specifying what atomic orbitals to select (see orbital_strings for the syntax). These files can be generated following the procedure outlined in the tutorial Atomic start for “Preparing the atomic start”. Note, that the calculation is specified to be performed without the use of symmetry. The calculation now converges in 20 iterations

PBEhuc_LuF3.out

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/start_guess/atomic_start.md

orphan

Atomic start guess

Initial SCF run

We consider the Hartree-Fock calculation of porphyrin with a mercury center

 alternate text
alternate text

The SCF input file HF.inp reads:

```
**DIRAC
.WAVE FUNCTION
.ANALYZE
**WAVE FUNCTIONS
.SCF
*SCF
.MAXITR
20
.PRINT
2
.CLOSED SHELL
120 120
**ANALYZE
.MULPOP
*END OF
```

for an SCF calculation with a Mulliken population analysis. We specify the maximum number of iterations allowed for the calculation to 20 and as we are choosing a molecule with inversion symmetry we specify the occupation number for both irreducible representations under the keyword CLOSED SHELL. The print level is set to 2 in order to have the eigenvalues printed for each iteration. The molecular input hg-porphyrin.mol reads :

BASIS
6-31 and finite DK3 basis sets, geometry in au, ref ECP: Theor.Chim.Acta 1990,77,123 (from Gaussian)

```
-----
C   4   3   X Y Z
    7.   2
N1   0.000000   4.1196801826   0.000000
N2   4.1196801826   0.000000   0.000000
LARGE BASIS 6-31G
    6.   5
C1   4.642960672   4.642960672   0.000000
C2   2.127242869   5.577562894   0.000000
C3   1.2976452113   8.20576858026   0.000000
C4   8.20576858026   1.2976452113   0.000000
C5   5.577562894   2.127242869   0.000000
LARGE BASIS 6-31G
    1.   3
H1   6.1040918256   6.1040918256   0.000000
H2   2.536971561   9.84908074236   0.000000
H3   9.84908074236   2.536971561   0.000000
LARGE BASIS 6-31G
    80.   1
Hg   0.000000   0.000000   0.000000
LARGE   4   1   1   1   1
F 23   0   0                                     # s-functions
    5.4107818D+07
    1.2757417D+07
    3.8200687D+06
    1.2601464D+06
    4.4355086D+05
    1.6477405D+05
    6.4498870D+04
    2.6536057D+04
    1.1391885D+04
    5.0460108D+03
    2.2877229D+03
    1.0547293D+03
    4.9377065D+02
    2.3032535D+02
    1.1057622D+02
    4.7454363D+01
    2.4814292D+01
    1.0508682D+01
    5.3690964D+00
    1.7593797D+00
    8.2178073D-01
    1.6991016D-01
    5.9613310D-02
F 23   0   0                                     # p-functions
    2.1202540D+07
    4.1579701D+06
    1.0372139D+06
    2.9906582D+05
    9.5123364D+04
    3.3364166D+04
    1.2858845D+04
    5.3680904D+03
    2.3907706D+03
    1.1241754D+03
    5.5303069D+02
    2.8249864D+02
    1.4871000D+02
    7.9474383D+01
    4.3674317D+01
    2.4527063D+01
    1.4058201D+01
    7.9450158D+00
```

```
4.5067591D+00
2.4440572D+00
1.2660103D+00
6.1522560D-01
2.5113161D-01
F 15 0 0 # d-functions
1.4315825D+04
3.7250818D+03
1.3315379D+03
5.5776192D+02
2.5656959D+02
1.2554901D+02
6.3698254D+01
3.3232923D+01
1.7367497D+01
8.9182853D+00
4.5159858D+00
2.1444878D+00
9.7876159D-01
4.1519292D-01
1.5886171D-01
F 10 0 0 # f-functions
1.3065007D+03
4.4185000D+02
1.8456705D+02
8.5884633D+01
4.2090822D+01
2.1327466D+01
1.0817659D+01
5.3881551D+00
2.5585604D+00
1.0775039D+00
FINISH
```

The calculations are carried out in D2h symmetry, so that the Fock matrix is real. From the output file we read:

Atoms and basis sets

Number of atom types: 4
Total number of atoms: 37

label	atoms	charge	prim	cont	basis
N2	4	7	22	9	L - [10s4p 3s2p]
			58	58	S - [4s10p4d 4s10p4d]
C5	20	5	22	9	L - [10s4p 3s2p]
			58	58	S - [4s10p4d 4s10p4d]
H3	12	1	4	2	L - [4s 2s]
			12	12	S - [4p 4p]
Hg	1	80	282	282	L - [23s23p15d10f 23s23p15d10f]
			635	635	S - [23s38p33d15f10g 23s38p33d15f10g]
			858	522	L - large components
			2171	2171	S - small components
total:	37	220	3029	2693	

showing that in AO-basis the Fock-matrix has the dimension 2693 x 2693 and thus requires 7252249 words. The SCF run will require something like three Fock matrices, so in this case 21 Mwords.

Unfortunately, this HF does not converge :

SCF - CYCLE

Energy		ERGVAL	FCKVAL	EVCVAL	Conv.acc	CPU	Integrals	Time stamp
It. 1		-7794.747305914	0.00D+00	0.00D+00	0.00D+00		10.89634323s	Scr. nuclei Fri Oct 19
It. 2		-20119.30505450	1.23D+04	-9.63D+01	1.29D+02		4min 6.910s	LL SL Fri Oct 19
It. 3		-17230.23561991	-2.89D+03	1.95D+02	2.69D+02	DIIS 2	3min55.989s	LL SL Fri Oct 19
It. 4		-20286.13741996	3.06D+03	-8.61D+01	9.85D+01	DIIS 3	3min42.372s	LL SL Fri Oct 19
It. 5		-20126.54429484	-1.60D+02	-9.61D+01	4.27D+01	DIIS 4	3min52.176s	LL SL Fri Oct 19
It. 6		-18673.18452215	-1.45D+03	1.36D+02	1.55D+02	DIIS 5	3min48.236s	LL SL Fri Oct 19
It. 7		-18727.77909394	5.46D+01	-1.82D+00	1.48D+02	DIIS 6	3min53.450s	LL SL Fri Oct 19
It. 8		-18755.21459721	2.74D+01	-8.49D-01	1.45D+02	DIIS 7	4min 4.378s	LL SL Fri Oct 19
It. 9		-18760.27704618	5.06D+00	-1.58D-01	1.45D+02	DIIS 8	3min40.865s	LL SL Fri Oct 19
It. 10		-18773.33929407	1.31D+01	-4.09D-01	1.44D+02	DIIS 9	3min51.322s	LL SL Fri Oct 19
It. 11		-18779.70838778	6.37D+00	-1.28D-01	1.44D+02	DIIS 9	3min42.698s	LL SL Fri Oct 19
It. 12		-18953.17392936	1.73D+02	-4.17D+00	1.33D+02	DIIS 9	3min52.510s	LL SL Fri Oct 19
It. 13		-18938.57698783	-1.46D+01	5.29D-01	1.34D+02	DIIS 9	3min43.147s	LL SL Fri Oct 19
It. 14		-18744.84737430	-1.94D+02	8.55D+00	1.59D+02	DIIS 9	3min56.082s	LL SL Fri Oct 19
It. 15		-18164.22290167	-5.81D+02	-2.01D+02	4.93D+01	DIIS 9	3min48.373s	LL SL Fri Oct 19
It. 16		-16110.21649469	-2.05D+03	2.64D+02	2.90D+02	DIIS 9	3min47.714s	LL SL Fri Oct 19
It. 17		-16272.74479238	1.63D+02	-4.12D+00	2.75D+02	DIIS 9	3min49.219s	LL SL Fri Oct 19
It. 18		-16280.64616752	7.90D+00	-2.09D-01	2.74D+02	DIIS 9	3min40.992s	LL SL Fri Oct 19
It. 19		-16279.46291382	-1.18D+00	5.93D-02	2.75D+02	DIIS 9	3min37.488s	LL SL Fri Oct 19
It. 20		-16289.37827128	9.92D+00	-2.58D-01	2.74D+02	DIIS 9	3min42.223s	LL SL Fri Oct 19

We see that the corrected bare nucleus start is quite far from the energies of subsequent iterations. Although the energy in the second iteration drops considerably it bounces back and gets stuck. We will therefore switch to an atomic start vanLenthe2006 . Observe that this caculation will take more than an hour.

Preparing the atomic start

For each atom we now run a SCF calculation in full (linear) symmetry based on the atomic ground state configuration. For open-shell atoms this amounts to an average-of-configuration (AOC) calculation at the HF level, and a fractional occupation calculation at the DFT level.

Nitrogen atom

The input file `N.inp` reads :

```
**DIRAC
.WAVE FUNCTION
.ANALYZE
**GENERAL
.ACMOUT
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
4 0
.OPEN SHELL
1
3/0,6
**ANALYZE
.MULPOP
*END OF
```

Note the keywords `ACMOUT` asking that the MO coefficients transformed to no symmetry and written to file `DFACMO`. The open shell input specifies 1 open shell with 3 electrons in 0 and 6 spinors respectively for the two irriducible representations. The molecular input `N.mol` is :

```
BASIS
6-31 and finite DK3 basis sets, geometry in au, ref ECP: Theor.Chim.Acta 1990,77,123 (from Gaussian)
-----
C 1
      7. 1
N 0.0 0.0 0.0
LARGE BASIS 6-31G
FINISH
```

The minimal pam command is :

```
pam --mol=N.mol --inp=N.inp --get=DFACMO
mv DFACMO ac.N
```

Other atoms

The input file for the other atoms are prepared in similar manner. For carbon we use the occupation :

```
.CLOSED SHELL
4 0
.OPEN SHELL
1
2/0,6
```

For mercury we use :

```
.CLOSED SHELL
42 38
```

whereas the hydrogen atom is run as a one-electron system, meaning that the keyword for ignoring two-particle interactions `ONESYS` will be invoked :

```
**DIRAC
.WAVE FUNCTION
.ANALYZE
**GENERAL
.ACMOUT
**HAMILTONIAN
.ONESYS
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
2 0
**ANALYZE
.MULPOP
*END OF
```

(The occupation given above is simply to control orbitals printed in the subsequent Mulliken population analysis).

All atoms can be run in a single shot, e.g. using bash :

```
for atom in H C N Hg
do
  pam --mol=${atom}.mol --inp=${atom}.inp --get=DFACMO
  mv DFACMO ac.${atom}
done
```

We are now ready to run the atomic start.

Running the atomic start

The input file `atomstart.inp` reads :

```
**DIRAC
.WAVE FUNCTION
.ANALYZE
**WAVE FUNCTIONS
.SCF
*SCF
.ATOMST
AFNXXX 2
1,2
1.00
3..5
0.5
AFCXXX 2
1,2
1.00
3..5
0.33
AFHXXX 1
1
0.5
AFHGXX 1
1..40
1.00
.CLOSED SHELL
120 120
**ANALYZE
.MULPOP
*END OF
```

The keyword ATOMST is followed by input for each atomic type. The first input AFYYXX specifies the file to be read for an atomtype YY, and next to it a number specifying inversion symmtry (2) or not (1). Next, for each symmetry, the spinors of that symmetry is specified followed by the occupation of the spinors. The occupations chosen corresponds to those of the atomic runs, but the user may modify this at will. Please note that the order of atoms corresponds to the order they appear in the molecule file.

Before running the calculation the user must make provide links to the atomic coefficient files :

```
ln -s ac.N AFNXXX
ln -s ac.C AFCXXX
ln -s ac.H AFHXXX
ln -s ac.Hg AFHGXX
```

and the calculation is then run as :

```
pam --mol=hg-porphyrin.mol --inp=atomstart.inp --copy="AF*" --outcmo
```

This calculation converges smoothly after 17 iterations :

```
SCF - CYCLE
-----

* Convergence on norm of error vector (gradient).
Desired convergence:1.000D-07
Allowed convergence:1.000D-06

* ERGVAL - convergence in total energy
* FCKVAL - convergence in maximum change in total Fock matrix
* EVCVAL - convergence in error vector (gradient)

Energy                ERGVAL      FCKVAL      EVCVAL      Conv.acc      CPU      Integrals      Time stamp
-----
It.   1      -20645.07067749      2.06D+04  0.00D+00  0.00D+00  Atomic s  32.88100123s  LL SL      Mon Mar  4
It.   2      -20633.15458108      -1.19D+01  6.07D-01  8.19D-01      1min11.442s  LL SL      Mon Mar  4
It.   3      -20633.38854493      2.34D-01 -2.13D-01  3.24D-01  DIIS  2      1min 9.861s  LL SL      Mon Mar  4
It.   4      -20633.41415475      2.56D-02  1.52D-01  1.72D-01  DIIS  3      1min 3.733s  LL SL      Mon Mar  4
It.   5      -20633.42153720      7.38D-03 -6.24D-02  2.79D-02  DIIS  4      1min 2.223s  LL SL      Mon Mar  4
It.   6      -20633.42175881      2.22D-04 -4.58D-03  8.00D-03  DIIS  5      1min 3.556s  LL SL      Mon Mar  4
It.   7      -20633.42178693      2.81D-05  2.31D-03  3.88D-03  DIIS  6      1min 1.382s  LL SL      Mon Mar  4
It.   8      -20633.42179098      4.05D-06 -1.68D-03  1.75D-03  DIIS  7      57.84219360s  LL SL      Mon Mar  4
It.   9      -20633.42179253      1.55D-06  3.91D-04  5.46D-04  DIIS  8      57.79122925s  LL SL      Mon Mar  4
It.  10      -20633.42179283      3.01D-07 -5.63D-05  2.16D-04  DIIS  9      55.75250244s  LL SL      Mon Mar  4
It.  11      -20633.42179286      3.30D-08  1.89D-05  7.63D-05  DIIS  9      53.37689209s  LL SL      Mon Mar  4
It.  12      -20633.42179287      4.12D-09 -1.74D-05  2.92D-05  DIIS  9      51.99108887s  LL SL      Mon Mar  4
It.  13      -20633.42179287      7.42D-10  6.49D-06  8.29D-06  DIIS  9      51.30120850s  LL SL      Mon Mar  4
It.  14      -20633.42179287      3.13D-10 -1.22D-06  2.61D-06  DIIS  9      48.06268311s  LL SL      Mon Mar  4
It.  15      -20633.42179287      -2.18D-10  5.07D-07  5.08D-07  DIIS  9      48.58166504s  LL SL      Mon Mar  4
It.  16      -20633.42179287      -2.15D-10 -2.13D-07  2.33D-07  DIIS  9      43.13543701s  LL SL      Mon Mar  4
It.  17      -20633.42179287      5.42D-10  1.06D-07  9.96D-08  DIIS  9      42.37854004s  LL SL      Mon Mar  4

* Convergence after 17 iterations.
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/two_component_hamiltonians/example.md

orphan

Spin-orbit states from the COSCI method

This tutorial demonstrates the importance of the effective mean-field spin-orbit screening on spin-orbit states of open-shell systems. Several two-component Hamiltonians are employed.

Spin-orbit states of the F atom

In the DIRAC test we calculate the energy difference between spin-orbit splitted states of the X^2P state of Fluorine, using the COSCI wavefunction and with several different Hamiltonians. All input files for download (together with output files) are in the corresponding test directory of DIRAC, `test/cosci_energy`.

The following table shows the energy difference between $X^2P_{3/2}$ and $X^2P_{1/2}$ states:

Hamiltonian	Splitting/cm-1
DC	434.511758
BSS+MFSSO	438.792872
BSS_RKB+MFSSO(*)	438.793184
DKH2+MFSSO	438.792782
BSSsfBSO1+MFSSO	438.868634
DKH2sfBSO1+MFSSO	438.868738
BSSsfESO1+MFSSO	438.866098
DKH2sfESO1+MFSSO	438.866201
BSS	583.459766
BSS_RKB(**)	583.459995
DKH2	583.459700
BSSsfESO1	583.533060
DKH2sfESO1	583.533187
BSSsfBSO1	583.535908
DKH2sfBSO1	583.536036
DC2BSS_RKB(DF)	585.906861

() Known as X2C. (*) Known as X2C-NOAMFI.

Calculated values can be divided into two categories: those with the mean-field spin-orbit term (MFSSO) and those without. Results matching the four-component Dirac-Coulomb (DC) Hamiltonian are those containing the MFSSO screening term.

For more information, see Refs. Ilias2001, Ilias2007 .

Spin-orbit states of the Rn^{77+} cation

Let us proceed with the isoelectronic, but heavier system: the Fluorine-like (9 electrons), highly charged Rn^{77+} cation (Z=86). All input files for download (together with output files) are in the corresponding test directory of DIRAC, `test/cosci_energy`. Calculated energy differences between the ground, $X^2P_{3/2}$, and the first excited state, $X^2P_{1/2}$, are in the following table:

Hamiltonian	Splitting/eV
DC	3700.081
BSS+MFSSO	3796.844
DKH2+MFSSO	3777.837
DC2BSS_RKB(DF)	3810.190
BSS	3808.859
BSS_RKB (*)	3810.273
DKH2	3790.044
DKH2sfBSO1+MFSSO	4047.324
DKH2sfBSO1	4056.349

(*) Known as X2C-NOAMFI.

Exercises

1. Why is the MFSSO term more important for the lighter element (F) than for the heavy Rn^{77+} ?
2. The one-electron spin-orbit term, SO1, is sufficient for representing spin-orbital effects in the Fluorine atom, but not of the Rn^{77+} cation. Why ?
3. For the Fluorine atom, increase the speed of light (GENERAL_.CVALUE) in four-component calculations to emulate non-relativistic description. What is the effect on the spin-orbit splitting ? What artificial value of the speed of light generates the DC-SCF energy identical with nonrelativistic SCF energy up to 5 decimal places ?
4. To “increase” relativistic effects in Fluorine, decrease the speed of light in four-component calculations. How does it affect the spin-orbit splitting ?
5. Change the symmetry from D2h to automatic symmetry detection in the F mol file and add molecular spinors analysis to the input file (**ANALYZE). Identify molecular spinors (orbitals) of Fluorine according to the extra quantum number in linear symmetry.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/two_component_hamiltonians/hamiltonians.md

orphan

Selecting a two-component Hamiltonian other than X2C

There are several two-component Hamiltonians besides the X2C Hamiltonian (see HAMILTONIAN_.X2C) implemented in DIRAC arising from the decoupling transformation of the one-electron DIRAC Hamiltonian. Likewise spin-orbit interaction terms can be left out and calculations can be performed in the spin-free mode (i.e. in boson symmetry).

Besides having spin-orbit effects from the decoupling transformation, there is the external AMFI spin-orbit operator which can contribute either with the one-electron spin-orbit operator (accurate to the first order in V , very poor) or with the more valuable mean-field contribution, $u_{\text{so}}(1)$ (also in the first order), which is a reasonable approximation of the ‘Gaunt’ term at the four-component level.

The best two-component Hamiltonian is X2C+MFSSO (similar to BSS+MFSSO or IOTC+MFSSO) where one-electron scalar and spin-orbit effects are up to infinite order, and AMFI MFSSO contributions (mean-field spin-same orbit) provide a ‘screening’ of one-electron spin-orbit terms.

Examples

Both scalar and spin-orbit relativistic effects up to infinite order:

```
.BSS
099
```

Scalar relativistic effects of type “from the beginning” up to the infinite order, no spin-orbit interaction:

```
.SPINFREE
.BSS
109
```

Scalar relativistic effects “from the end” up to the infinite order:

```
.SPINFREE
.BSS
009
```

Traditional scalar relativistic “from the beginning” second-order Douglas-Kroll-Hess Hamiltonian:

```
.SPINFREE
.BSS
102
```

Scalar relativistic “from the end” second-order Douglas-Kroll-Hess Hamiltonian:

```
.SPINFREE
.BSS
002
```

Douglas-Kroll-Hess Hamiltonian with first-order spin-orbit and second-order scalar relativistic effects:

```
.BSS
012
```

Douglas-Kroll-Hess Hamiltonian with second-order spin-orbit and second-order scalar relativistic effects:

```
.BSS
022
```

In the two-component variational scheme is possible to combine external AMFI spin-orbit terms, Ilias2001, together with BSS integrals - see the *AMFI section. Note that AMFI provides only one-center atomic integrals.

Infinite order scalar and spin-orbit terms, (one-electron) mean-field spin-same-orbit (MFSSO2) from AMFI:

```
.BSS
2999
```

This is in fact the ‘maximum’ of two-component relativity, resembling Dirac-Coulomb Hamiltonian.

Second-order Douglas-Kroll-Hess spin-free from ‘the beginning’ with first order spin-orbit (SO1) term plus mean-field spin-same-orbit (MFSSO) from AMFI:

.BSS
2112

SO1 may come either from BSS-transformation or from AMFI. In the latter case it is only for one-center and for point nucleus.

Infinite order scalar and spin-orbit terms, (one-electron) mean-field spin-same and spin-other-orbit (MFSO2) from AMFI:

.BSS
3999

This mimics the Dirac-Coulomb-Gaunt Hamiltonian, as the spin-other-orbit (SOO) term comes from the Gaunt interaction term.

DIRAC allows to switch off AMFI spin-orbit contributions from various centers. See the keyword `AMFI_ .NOAMFC`.

Infinite order scalar terms “from the beginning”, (one-electron) spin orbit (SO1) and the mean-field spin-same-orbit (MFSSO2) terms exclusively from AMFI:

.BSS
4109

Second order (DKH) scalar terms “from the beginning”, (one-electron) spin orbit (SO1) and the mean-field spin-same-orbit (MFSSO2) terms exclusively from AMFI:

.BSS
4102

Infinite order scalar terms “from the end”, (one-electron) spin orbit (SO1) and the mean-field spin-same-orbit and spin-other-orbit (MFSO2) terms from AMFI:

.BSS
5009

Infinite order scalar terms “from the end”, and the (one-electron) spin orbit term (SO1) from AMFI:

.BSS
6009

AMFI one-electron spin-orbit terms - SO1 - are currently for the point nucleus.

For the BSS value ‘axyz’ of y=0, DIRAC employs spin-free picture change transformation of property operators, although the system is not in the boson (spin-free) symmetry for a>1.

Rough first order, DKH1 (not recommended for practical calculations):

.BSS
111

For comparison purposes between BSS-SO1 and AMFI-SO1 atomic one-center integrals (point nucleus only) use:

.BSS
001

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/two_component_hamiltonians/molecular_mean_field.md

orphan

Molecular mean-field X2C

The molecular-mean-field X2C Hamiltonian (X2Cmmf) is **available** for the following methods:

- Coupled Cluster (CC): module RELCCSD
- multi-reference (MR) CC: Fock-space CC (including the intermediate Hamiltonian IHFSCC version): module RELCCSD

The remaining post-SCF modules are **not supported** yet:

- KR-MCSCF
- KR-CI
- TD-DFT

The X2Cmmf Hamiltonian scheme can be used for the Dirac-Coulomb as well as the Dirac-Coulomb-Gaunt Hamiltonian (add `.GAUNT` to `**HAMILTONIAN`).

One-step X2Cmmf

In order to perform a two-component (MR)CC after a four-component SCF calculation add the keyword `.X2Cmmf` to the `**HAMILTONIAN` keyword section:

```
**HAMILTONIAN
.X2Cmmf
```

After the SCF calculation Dirac will carry out the transformation to two-component prior to the 4-index transformation of the two-electron integrals and the CC run.

Two-step X2Cmmf

Besides the one-step X2Cmmf protocol as described above one may also proceed in separate steps. This may be useful if one would like to use less/more MPI processes in the SCF+decoupling step than in the 4-index/CC calculation. The first step (SCF+decoupling) simply requires the `.X2Cmmf` keyword under the `**HAMILTONIAN` input

deck (same as above):

```
**HAMILTONIAN
.X2Cmmf
```

Save the files DFCOEf, AOMOMAT and X2CMAT, for example using the *pam* script:

```
./pam ... --outcmo --get="AOMOMAT X2CMAT"
```

To restart now any further calculation, for example the 4-index transformation (MOLTRA) and/or the (MR)CC run copy the above files to your scratch directory:

```
./pam ... --incmo --put="AOMOMAT X2CMAT"
```

and set the keyword combination as indicated below under the **HAMILTONIAN** input deck:

```
**HAMILTONIAN
.X2C
*X2C
.mmf--restart
```

In order to run a (MR)CC calculation add also the following keyword(s) to the namelist RELCCSSD input:

```
*CCSORT
.USEOE
.NORECMP
END
```

or the new RELCCSD deck input:

```
*CCSORT
.NORECMP
```

Dirac will automatically read the two-component coefficients from the file DFCOEf and proceed in two-component mode for any wave function analysis or post-HF step required for a (MR)CC calculation.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/two_component_hamiltonians/x2c_mol_loc.md

orphan

X2C and local X2C

This tutorial briefly demonstrates the keywords and their usage to invoke the exact two-component (X2C) Hamiltonian and a local X2C variant of it in Dirac. The current available implementation and other features which will be part of the next release Dirac2014 are described in Knecht2014.

A recent and comprehensive review on the topic of relativistic Hamiltonians for chemistry can be found in Saue2011 by Trond Saue. The central idea of any decoupling approach is to generate a two-component Hamiltonian that reproduces the positive-energy spectrum of the parent four-component Hamiltonian in the best possible way. This goal may be achieved by a unitary transformation $\hat{\mathbf{U}}$ that will formally decouple the large and small components by block-diagonalization:

```
\begin{aligned}
\{\hat{h}_{\text{bd}}\} &= \mathbf{U}^{\dagger} \\
\left[\begin{array}{cc} \hat{h}_{\text{LL}} & \hat{h}_{\text{LS}} \\ \hat{h}_{\text{SL}} & \hat{h}_{\text{SS}} \end{array}\right] & \\
\mathbf{U} &= \\
\left[\begin{array}{cc} \hat{h}_{\text{++}} & 0 \\ 0 & \hat{h}_{\text{--}} \end{array}\right], & \\
\end{aligned}
```

Popular choices to obtain a unitary transformation matrix $\mathbf{U}$ are in particular discussed in Section 3 of Saue2011 while a detailed abstract of the actual implementation of the X2C approach in Dirac is given in the last part of Section 3.3 in the same review paper. Property operators are transformed automatically on the fly to two-components within the X2C approach in Dirac in order to avoid the so-called picture-change errors. For a detailed discussion on the latter topic with the example of charge densities at a heavy nucleus see for example Knecht2011 by Knecht and co-workers. Transforming the electronic Hamiltonian based on the above prescription

```
\{\hat{h}^{\text{X2C}}\} = \{\hat{h}_{\text{++}}\} = \\
\left[\mathbf{U}^{\dagger} \right. \\
\left. \hat{h}^{\text{4c}} \mathbf{U} \right]_{\text{++}}
```

it is mandatory to also transform the parent four-component property-operator Ω^{4c} using the same unitary transformation matrix $\mathbf{U}$

```
\{\Omega^{\text{2c}}\} = \\
\left[\mathbf{U}^{\dagger} \right. \\
\left. \Omega^{\text{4c}} \mathbf{U} \right]_{\text{++}}
```

since otherwise we would introduce (significant) errors ('picture-change errors') by simply using the approximate relation

```
\{\Omega^{\text{2c}}\} \approx \left[\Omega^{\text{4c}}\right]_{\text{LL}} .
```

X2C

X2C with spin-same-orbit two-electron corrections

To invoke the X2C Hamiltonian $\hat{H}^{X2C}$ in Dirac which is based on the diagonalization of the 1-electron bare-nucleus Dirac Hamiltonian, use the keyword `.X2C` under the input deck `**HAMILTONIAN`:

```
**HAMILTONIAN
.X2C
```

This includes by default atomic-mean-field two-electron spin-same-orbit corrections from the AMFI module.

X2C with spin-same-orbit and spin-other-orbit two-electron corrections

To add spin-other-orbit corrections (Gaunt term of the Breit interaction) use:

```
**HAMILTONIAN
.X2C
.GAUNT
```

spinfree X2C

In the spinfree case one should use:

```
**HAMILTONIAN
.X2C
.SPINFREE
```

local X2C

For large systems the diagonalisation of the Dirac Hamiltonian — which is an essential step in the derivation of the unitary transformation matrix $\mathbf{U}_{\text{mol}}$ — may become a bottleneck for the full molecule (hence to stress this fact we have introduced the explicit label *mol*). This bottleneck can be avoided by extracting the coupling at the level of individual atoms as proposed in a working protocol by Peng and Reiher Peng2012. In Dirac2013 we have implemented the *diagonal local approximation to the unitary decoupling transformation* (DLU scheme) of Peng2012. The central idea behind this local scheme is to approximate $\mathbf{U}_{\text{mol}}$ by a superposition of $i=1,\dots,N_A$ atomic (or in principle also fragment) matrices $\mathbf{U}_i$ where a decoupling matrix is calculated for each *unique* atom type.

$$\mathbf{U}_{\text{mol}} \approx \mathbf{U}_i \oplus \mathbf{U}_{i+1} \oplus \mathbf{U}_{i+2} \oplus \dots \mathbf{U}_{N_A},$$

In order to benefit from the above DLU approach in Dirac which at present works *only for C1 symmetry* we need to first carry out the atomic calculations and save the atomic $\mathbf{U}_i$ matrices. This can be done as follows. We run an atomic X2C transformation for each *unique* atom in our molecule (here we assume a dyall.v2z basis):

```
**DIRAC
.TITLE
  atomic X2C - U matrix construction
**HAMILTONIAN
.X2C
**WAVE FUNCTION
*SCF
**MOLECULE
*BASIS
.DEFAULT
dyall.v2z
*SYMMETRY
.NOSYM
**INTEGRALS
*READIN
.UNCONTRACT
**END OF
```

Note the use of `.NOSYM` to enforce C1 symmetry in the atomic run even for an atom. A possible corresponding xyz input for an atom (here Cl) would look like:

```
1
chlorine atom XYZ file
Cl 0.0 0.0 0.0
```

We save the file X2CMAT containing the atomic $\mathbf{U}_i$ matrix and rename it according to the atomic number of the respective atom, here X2CMAT.017 for chlorine (Z=17). For mercury (Z=80) the appropriate ending would be X2CMAT.080 and similar for other atoms:

```
$ ./pam --inp=atom.inp --mol=carbon.xyz --get=X2CMAT
$ mv X2CMAT X2CMAT.006
```

After we have created all atomic $\mathbf{U}_i$ matrices for each *unique* atom type in our molecule — let's assume to work on $\text{Hg}(\text{Cl})_2$ where we would need a transformation matrix for Hg and Cl — we are ready to setup the molecular X2C calculation based on the local/atomic approach of the decoupling unitary $\mathbf{U}_i$ matrices. The relevant part of the input would then read:

```
**DIRAC
.TITLE
  local X2C based on atomic U matrices
.WAVE FUNCTION
**HAMILTONIAN
.X2C
*X2C
.fragX2C
**WAVE FUNCTION
.SCF
*SCF
...
...
**MOLECULE
*BASIS
.DEFAULT
```

```
dyall.v2z
*SYMMETRY
.NOSYM
**INTEGRALS
*READIN
.UNCONTRACT
**END OF
```

This input should be used with the following pam line:

```
$ ./pam ... --put=X2CMAT.* --mw=200
```

In this DLU approach proposed by Peng and Reiher Peng²⁰¹² the molecular $\mathbf{U}_{\text{mol}}$ matrix will then be approximated as described above. It works with spinfree as well as all spin-dependent (spin-orbit) Hamiltonian schemes and includes a picture-change transformation of all molecular four-component property operators $\Omega^{\text{4c}}_{\text{mol}}$.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/two_component_hamiltonians/xamfX2C.md

orphan

xamfX2C: Two-electron picture change corrections for the X2C Hamiltonian

This tutorial demonstrates the usage and usefulness of the (extended) atomic mean-field two-electron picture-change corrections for the X2C Hamiltonian.

Short theory

The goal is to reproduce total energy and orbital energies of a 4-component SCF calculation as accurate as possible at the 2-component level. Restricting ourselves first to Hartree-Fock theory, this would require using the correctly transformed set of one- and two-electron integrals

```
\begin{aligned}
\begin{array}{lcl}
\tilde{h}^{\text{2c}}_{\mu\nu} & = & \displaystyle \sum_{XY} \sum_{\alpha\beta} \left[ U^{\dagger} \right]^{\text{LX}}_{\mu\alpha} \left[ h^{\text{4c}} \right]^{\text{XY}}_{\alpha\beta} \\
\tilde{G}^{\text{2c}}_{\mu\nu, \kappa\lambda} & = & \displaystyle \sum_{XYUV} \sum_{\alpha\beta\gamma\delta} \left[ U^{\dagger} \right]^{\text{LX}}_{\mu\alpha} \left[ U^{\dagger} \right]^{\text{LY}}_{\nu\beta} \left[ U^{\dagger} \right]^{\text{LX}}_{\kappa\gamma} \left[ U^{\dagger} \right]^{\text{LY}}_{\lambda\delta}
\end{array}
\end{aligned}
```

where X, Y, U, V in L, S and U is the picture-change transformation matrix that correctly block-diagonalizes the converged 4-component Fock matrix F^{4c} .

For more information on the actual details of the theory and implementation see Ref. Knecht²⁰²².

A gold complex: $[\text{Au}(\text{Cl})_4]^{-}$

We first consider a Au(III)-complex, tetrachloroaurate.

 alternate text
alternate text

Using the structure given in Hargittai_{JACS2001} (Table I: column MP2/aug-cc-pVTZ), we prepare the following xyz-file

AuCl4.xyz

[!WARNING] Notice that by default, for the total energy, DIRAC uses the simple Coulombic correction introduced by Visscher^{1997a}, where the calculation of the expensive (SS)SS-class of integrals is avoided by introducing an energy correction in terms of the repulsion of small component atomic charges. The amfX2C (and eamfX2C) model tries to reproduce 4-component total and orbital energies for a general molecule based on information extracted from *atomic* 4-component relativistic calculations. Since these are relatively cheap, we **strongly** recommend including the (SS)SS-class of integrals is avoided by introducing an energy correction in terms of the repulsion of small component atomic charges. The amfX2C (and eamfX2C) model tries to reproduce 4-component total and orbital energies for a general molecule based on information extracted from *atomic* 4-component relativistic calculations. Since these are relatively cheap, we **strongly** recommend including the (SS)SS-class of integrals in these atomic calculations.

Reference calculation

For reference, we therefore carry out a 4-component relativistic Hartree-Fock calculation with (SS)SS-class of integrals included. We use the dyall.v2z basis set. Our menu file therefore looks like:

```
**DIRAC
.TITLE
AuCl4-
.WAVE F
**HAMILTONIAN
.DOSSSS
**WAVE FUNCTIONS
.SCF
*SCF
.CLOSED SHELL
74 74
**MOLECULE
*BASIS
.DEFAULT
dyall.v2z
*END OF
```

and can be run using:

```
pam --inp=4c --mol=AuCl4.xyz
```

In the output we then find:

TOTAL ENERGY

Charge of molecule : -1

Electronic energy : -22374.645394444109

Other contributions to the total energy
Nuclear repulsion energy : 1495.084959925113

Sum of all contributions to the energy
Total energy : -20879.560434518997

as well as occupied orbital energies:

```
* Fermion symmetry E1g
* Closed shell, f = 1.0000
-2986.172433056 ( 2) -532.228239826 ( 2) -128.126822999 ( 2) -105.062162391 ( 4) -85.994500068 ( 2)
-85.992245877 ( 2) -82.776253554 ( 2) -82.773843683 ( 2) -82.772814925 ( 2) -29.176725071 ( 2)
-13.906134621 ( 2) -13.898588891 ( 2) -13.210720978 ( 2) -13.203947809 ( 2) -13.203621787 ( 2)
-10.449079002 ( 4) -7.890617059 ( 4) -7.827509378 ( 4) -7.825980274 ( 4) -4.715673601 ( 2)
-0.929177412 ( 2) -0.904022207 ( 2) -0.533120234 ( 2) -0.505514832 ( 2) -0.457699080 ( 2)
-0.454463761 ( 2) -0.429307565 ( 2) -0.354242187 ( 2) -0.290147741 ( 2) -0.270966594 ( 2)
-0.265726870 ( 2) -0.252693231 ( 2)

* Fermion symmetry E1u
* Closed shell, f = 1.0000
-509.312470287 ( 2) -441.741820331 ( 2) -441.741607132 ( 2) -117.884404188 ( 2) -105.062163078 ( 4)
-102.771317291 ( 2) -102.767075769 ( 2) -24.775448231 ( 2) -21.109216293 ( 2) -21.099913774 ( 2)
-10.449103312 ( 4) -7.890617462 ( 4) -7.827509914 ( 4) -7.825980151 ( 4) -3.911755156 ( 2)
-3.901812050 ( 2) -3.898146885 ( 2) -3.763318698 ( 2) -3.756380303 ( 2) -3.751940592 ( 2)
-3.750335458 ( 2) -3.216879082 ( 2) -2.570010773 ( 2) -2.569726879 ( 2) -0.901102780 ( 2)
-0.898562174 ( 2) -0.333968015 ( 2) -0.325038247 ( 2) -0.316265455 ( 2) -0.284482135 ( 2)
-0.270870860 ( 2) -0.268370374 ( 2)
```

amfX2C calculation

Initial atomic calculations

As preparation to a amfX2C (or eamfX2C) calculation we need to carry out 4-component relativistic atomic calculations. For gold we use the menu file

Au.inp

as well as the xyz-file

Au.xyz

Notice that we have specified the keyword HAMILTONIAN_.X2Cmmf. This means that DIRAC will run a 4-component relativistic Hartree-Fock calculation and then transform the converged Fock matrix to 2-component form. As recommended, we have included the HAMILTONIAN_.D0SSSS keyword to include the (SSISS)-class of integrals. We have also specified the keyword XAMFI_.AMF such that the amfX2C corrections are prepared and written to file amfPCEC.h5. We run the gold calculation using:

```
pam --inp=Au.inp --mol=Au.xyz --get "amfPCEC.h5=amfPCEC.079.h5"
```

In similar manner, for chlorine we use the menu file

Cl.inp

as well as the xyz-file

Cl.xyz

and run the atomic calculation as:

```
pam --inp=Cl.inp --mol=Cl.xyz --get "amfPCEC.h5=amfPCEC.017.h5"
```

As a final preparatory step we have to merge the generated correction files:

```
$BUILD_DIR/merge_amf.py amfPCEC.079.h5 amfPCEC.017.h5
```

Molecular calculation

We are now set for the molecular amfX2C calculation. We use the menu file

amfX2C.inp

and run the calculation using:

```
pam --inp=amfX2C.inp --mol=AuCl4.xyz --put="amfPCEC.h5"
```

In the output we now find:

TOTAL ENERGY

Charge of molecule : -1

Electronic energy : -22374.646183410361
 ... with amf contributions to 2e-energy : -6.730906332501

Other contributions to the total energy
 Nuclear repulsion energy : 1495.084959925113

Sum of all contributions to the energy
 Total energy : -20879.561223485249

as well as orbital energies:

```
* Fermion symmetry E1g
* Closed shell, f = 1.0000
-2986.172409644 ( 2) -532.228297520 ( 2) -128.126807822 ( 2) -105.062171345 ( 4) -85.994478858 ( 2)
-85.992230143 ( 2) -82.776268135 ( 2) -82.773836896 ( 2) -82.772797520 ( 2) -29.176705603 ( 2)
-13.906104864 ( 2) -13.898568611 ( 2) -13.210714521 ( 2) -13.203931944 ( 2) -13.203597745 ( 2)
-10.449084871 ( 4) -7.890622640 ( 4) -7.827525601 ( 4) -7.825972582 ( 4) -4.715657228 ( 2)
-0.929176110 ( 2) -0.904021904 ( 2) -0.533079011 ( 2) -0.505500573 ( 2) -0.457674033 ( 2)
-0.454442417 ( 2) -0.429318493 ( 2) -0.354227188 ( 2) -0.290151069 ( 2) -0.270967632 ( 2)
-0.265733913 ( 2) -0.252693395 ( 2)

* Fermion symmetry E1u
* Closed shell, f = 1.0000
-509.312585352 ( 2) -441.741860248 ( 2) -441.741615553 ( 2) -117.884391655 ( 2) -105.062172090 ( 4)
-102.771297597 ( 2) -102.767054981 ( 2) -24.775425636 ( 2) -21.109198160 ( 2) -21.099888900 ( 2)
-10.449109460 ( 4) -7.890623044 ( 4) -7.827526119 ( 4) -7.825972449 ( 4) -3.911723029 ( 2)
-3.901789268 ( 2) -3.898128341 ( 2) -3.763312842 ( 2) -3.756362106 ( 2) -3.751916324 ( 2)
-3.750305598 ( 2) -3.216839463 ( 2) -2.570027444 ( 2) -2.569697960 ( 2) -0.901108027 ( 2)
-0.898559198 ( 2) -0.333966781 ( 2) -0.325040872 ( 2) -0.316260263 ( 2) -0.284482236 ( 2)
-0.270861088 ( 2) -0.268373653 ( 2)
```

Comparison with the 4-component reference, shows that the total energy has an error of -0.7 mH, whereas the mean absolute error in the orbital energies is 5.69 μH . If we do *not* use the (SSISS)-integrals, that is, we remove the keyword HAMILTONIAN_.DOSSSS from the preparatory 4-component atomic calculations, as well as the 4-component molecular reference calculation, then the error in the total energy is still -0.7 mH, but the mean absolute error in the orbital energies increases to 51.1 μH . It is also possible to go the other way: If we do add the Gaunt term using the keyword HAMILTONIAN_.GAUNT, then the error in total energy is -1.73 mH, and the mean absolute error in orbital energies is -3.79 μH .

Core-ionization energies

A very nice feature of the amfX2C approach is that the two-electron picture-change corrections adapt to the computational setup. For instance, if we do a Kohn-Sham calculation instead of Hartree-Fock, the corrections are generated for our specific choice of functional as well as basis set. As an example, let us consider core-ionization energies, arising from gold $2p_{1/2}$ (L_2) and $2p_{3/2}$ (L_3) for our gold complex, calculated using the PBE0 functional. The preparatory atomic calculations as well as the amfX2C calculation proceeds as before, except that we have inserted:

.DFT
 PBE0

in the **HAMILTONIAN section. In the amfX2C calculation, we have also asked for a Mulliken population analysis and find for fermion ircip E_{1u} :

```
* Electronic eigenvalue no. 1: -503.04886526180 (Occupation : f = 1.0000)
=====

* Gross populations greater than 0.01000

Gross   Total |   L B3uAu px   L B2uAu py   L B1uAu pz
-----|-----
alpha   0.3333 |   0.0000       0.0000       0.3333
beta    0.6667 |   0.3333       0.3333       0.0000

* Electronic eigenvalue no. 2: -435.57080277128 (Occupation : f = 1.0000)
=====

* Gross populations greater than 0.01000

Gross   Total |   L B3uAu px   L B2uAu py
-----|-----
alpha   0.0000 |   0.0000       0.0000
beta    1.0000 |   0.5000       0.5000

* Electronic eigenvalue no. 3: -435.56940890418 (Occupation : f = 1.0000)
=====

* Gross populations greater than 0.01000

Gross   Total |   L B3uAu px   L B2uAu py   L B1uAu pz
-----|-----
alpha   0.6667 |   0.0000       0.0000       0.6667
beta    0.3333 |   0.1667       0.1667       0.0000
```

Let us now calculate binding energy of these orbitals by ΔSCF . For the L_2 -edge we prepare the menu file L2_amfX2C.inp

L2_amfX2C.inp

This is an open-shell system, at the Kohn-Sham level calculated using fractional occupation. In DIRAC, positive-energy solutions are sorted first with respect to orbital class (closed, open, virtual), then with respect to orbital energies. We therefore use the keyword WAVE_FUNCTION_.REORDER M0 to bring the now singly occupied gold $2p_{1/2}$ orbital just after the occupied orbitals of *ungerade* symmetry. We also use overlap selection to keep the orbital in that position during the SCF iteration. Specifically, we active overlap selection with the keyword SCF_.OVLSEL and in addition use SCF_.NODYNSEL so that overlap is always calculated with respect to the initial set of orbitals. We run the calculation using:

```
pam --inp=L2_amfX2C.inp --mol=AuCl4.xyz --put "amfX2C_AuCl4,h5=CHECKPOINT,h5 amfPCEC,h5" --noh5
```

Note that we added the `--noh5` flag since we do not want to keep the associated checkpoint file. For each of the two $2p_{3/2}$ we proceed in similar manner, simply modifying the reordering, using:

```
.REORDER
1..37
1,3..37,2

and:

.REORDER
1..37
1,2,4..37,3
```

Methane (CH_4) - The ultrarelativistic case

In the following example, we will calculate total energies and spinor energies of CH_4 with several different Hamiltonians within the framework of DFT/PBE/dyall.v2z.

For convenience, in the Table below (see also Table IX in Ref. Knecht2022), we summarise the SCF total energy (E) and spinor energies (ϵ) of the doubly degenerate occupied spinors for CH_4 as obtained from DFT/PBE/v2z calculations within a four-component Dirac–Coulomb (DC^4) as well as a two-component Hamiltonian framework, including the $\text{eamfX2C}_{\text{DC}}$ models. All energies are given in Hartree. The speed of light c was reduced by a factor 10.

	1eX2C_{D}	$\text{AMFIX2C}_{\text{D}}$	$\text{amfX2C}_{\text{DC}}$	$\text{eamfX2C}_{\text{DC}}$	DC^4
E	-42.14220	-42.14039	-42.26469	-42.25774	-42.25850
ϵ_1	-10.22206	-10.22401	-10.36154	-10.36028	-10.35794
ϵ_2	-0.65939	-0.65966	-0.66288	-0.66269	-0.66274
ϵ_3	-0.35705	-0.35288	-0.35649	-0.35291	-0.35290
ϵ_{4-5}	-0.33313	-0.33503	-0.33282	-0.33527	-0.33544

In order to make use of the eamf two-electron picture-change corrections, we will need to run atomic calculations for each element / nuclei type in the molecule. Hence, in the present case, we will run atomic calculations for C and H, respectively. The latter seems odd as H is by definition a one-electron system but as we will see, the MO coefficients will be needed for the $\text{eamfX2C}_{\text{DC}}$ model.

For a detailed discussion of the performance of the various picture-change correction models for the X2C Hamiltonian in comparison with the four-component reference data, we refer the interested user to Ref. Knecht2022. Let us highlight, though, that for the particular case of $\text{amfX2C}_{\text{DC}}$ all picture-change correction terms for DFT (and likewise HF) are derived in molecular basis on the basis of molecular densities which are built from a superposition of atomic input densities. Consequently, in contrast to the simpler $\text{amfX2C}_{\text{DC}}$ model, the former comprises atomic contributions regardless of the actual atom type, that is, both “light” (one-electron) and “heavy” (many-electron) atoms contribute equally. Bearing the latter in mind, the total SCF energies as well as spinor energies compiled in the above Table confirm the unique numerical performance of the $\text{eamfX2C}_{\text{DC}}$ Hamiltonian model in comparison to the DC^4 reference data.

Atomic calculations

[!WARNING] Note that, besides scaling the speed of light by a factor of $c/10$, we also use a point nucleus as well as a different set of constants as default in our calculations. This was done for Ref. Knecht2022 to match calculations with [ReSpect](#). Since we would like to reproduce in this tutorial the results from Ref. Knecht2022, we will keep those details. Needless to say, the eamf model works for finite nuclei just as for point nuclei.

1. H atom

For the atomic run of H, we use the following input (`h.inp`)

```
**DIRAC
.WAVE FUNCTION
**INTEGRAL
.NUCMOD
1
*READIN
.UNCONTRACTED
**GENERAL
.CVALUE
13.703599907400d0
.CODATA
CODATA10
**HAMILTONIAN
.X2Cmmf
.DOSSSS
.DFT
PBE
.ONESYS
*X-AMFI
.amf
.eamf
**WAVE FUNCTION
.SCF
*SCF
**MOLECULE
*BASIS
.DEFAULT
```

```
dya11.v2z
**END OF
```

and the xyz file (h.xyz):

```
1
atom xyz file
H      0.0000000000  0.0000000000  0.0000000000
```

and run the calculation with:

```
$ ./pam --inp=h.inp --mol=h.xyz --get="amfPCEC.h5" && mv amfPCEC.h5 amfPCEC.001.h5
```

In the latter step, we rename the file amfPCEC.h5 for later convenience.

2. C atom

For the atomic run of C, we use the following input (c.inp)

```
**DIRAC
.WAVE FUNCTION
**INTEGRAL
.NUCMOD
1
*READIN
.UNCONTRACTED
**GENERAL
.CVALUE
13.703599907400d0
.CODATA
CODATA10
**HAMILTONIAN
.X2Cmmf
.DOSSSS
.DFT
PBE
*X-AMFI
.amf
.eamf
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
4 0
.OPEN SHELL
1
2/0,6
**MOLECULE
*BASIS
.DEFAULT
dya11.v2z
**END OF
```

and the xyz file (c.xyz):

```
1
atom xyz file
C      0.0000000000  0.0000000000  0.0000000000
```

and run the calculation with:

```
$ ./pam --inp=c.inp --mol=c.xyz --get="amfPCEC.h5" && mv amfPCEC.h5 amfPCEC.006.h5
```

In the latter step, we rename the file amfPCEC.h5 for later convenience.

Molecular calculations

Before embarking on the molecular calculations, we need to make a single preparatory step, that is:

```
$ $BUILD_DIR/merge_amf.py amfPCEC.006.h5 amfPCEC.001.h5
```

which will combine the **atomic** amfPCEC.xxx.h5 files into a single amfPCEC.h5 file ready for use in the molecular $\{\rm (e)amfX2C\}$ calculations below. The python script merge_amf.py is located in the build directory (named in the above code line \$BUILD_DIR) of DIRAC.

The xyz coordinates for $\{\rm CH\}_4$ are given below (ch4.xyz):

```
5
C      0.0000000000  0.0000000000  0.0000000000
H      0.6298891440  0.6298891440 -0.6298891440
H      0.6298891440 -0.6298891440  0.6298891440
H     -0.6298891440  0.6298891440  0.6298891440
H     -0.6298891440 -0.6298891440 -0.6298891440
```

1. $\{\rm 1eX2C\}_{\rm D}$

To run the molecular calculation for $\{\rm CH\}_4$ with the $\{\rm 1eX2C\}_{\rm D}$ Hamiltonian, that is, neglecting any two-electron picture-change corrections, we use the following input (1eX2C.inp):

```
**DIRAC
.WAVE FUNCTION
**GENERAL
```



```
.CVALUE
13.703599907400d0
.CODATA
  CODATA10
**INTEGRAL
.NUCMOD
  1
*READIN
.UNCONTRACTED
**HAMILTONIAN
.X2C
.DFT
.PBE
.NOAMFI
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
  10
**MOLECULE
*BASIS
.DEFAULT
  dyall.v2z
**END OF
```

The calculation can be run with:

```
$ ./pam --inp=1eX2C.inp --mol=ch4.xyz
```

```
2. $`{\rm AMFIX2C}_{\rm D}`$
```

To run the molecular calculation for $\text{\rm CH}_4$ with the $\text{\rm AMFIX2C}_{\rm D}$ Hamiltonian, that is, with two-electron picture-change corrections from the *AMFI module, we use the following input (AMFIX2C.inp):

```
**DIRAC
.WAVE FUNCTION
**GENERAL
.CVALUE
13.703599907400d0
.CODATA
  CODATA10
**INTEGRAL
.NUCMOD
  1
*READIN
.UNCONTRACTED
**HAMILTONIAN
.X2C
.DFT
.PBE
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
  10
**MOLECULE
*BASIS
.DEFAULT
  dyall.v2z
**END OF
```

The calculation can be run with:

```
$ ./pam --inp=AMFIX2C.inp --mol=ch4.xyz
```

```
3. $`{\rm amfX2C}_{\rm DC}`$
```

To run the molecular calculation for $\text{\rm CH}_4$ with the $\text{\rm amfX2C}_{\rm DC}$ Hamiltonian, that is, with atomic-mean-field two-electron picture-change corrections, we use the following input (amfX2C.inp):

```
**DIRAC
.WAVE FUNCTION
**GENERAL
.CVALUE
13.703599907400d0
.CODATA
  CODATA10
**INTEGRAL
.NUCMOD
  1
*READIN
.UNCONTRACTED
**HAMILTONIAN
.X2C
.DFT
.PBE
*X-AMFI
.amf
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
  10
**MOLECULE
*BASIS
```

```
.DEFAULT
dya11.v2z
**END OF
```

The calculation can be run with:

```
$ ./pam --inp=amfX2C.inp --mol=ch4.xyz --put="amfPCEC.h5"
```

4. $\$ \{ \text{rm eamfX2C} \}_ { \text{rm DC} } \$$

To run the molecular calculation for $\$ \{ \text{rm CH} \}_ { 4 } \$$ with the $\$ \{ \text{rm eamfX2C} \}_ { \text{rm DC} } \$$ Hamiltonian, that is, with full extended atomic-mean-field two-electron picture-change corrections, we use the following input (eamfX2C.inp):

```
**DIRAC
.WAVE FUNCTION
**GENERAL
.CVALUE
13.703599907400d0
.CODATA
CODATA10
**INTEGRAL
.NUCMOD
1
*READIN
.UNCONTRACTED
**HAMILTONIAN
.X2C
.DFT
PBE
*X-AMFI
.eamf
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
10
**MOLECULE
*BASIS
.DEFAULT
dya11.v2z
**END OF
```

The calculation can be run with:

```
$ ./pam --inp=eamfX2C.inp --mol=ch4.xyz --put="amfPCEC.h5"
```

5. $\$ ^ { 4 } \{ \text{rm DC} \} \$$

To run the molecular calculation for $\$ \{ \text{rm CH} \}_ { 4 } \$$ with the $\$ ^ { 4 } \{ \text{rm DC} \} \$$ Hamiltonian, that is, in full 4-component mode, we use the following input (4DC.inp):

```
**DIRAC
.WAVE FUNCTION
**GENERAL
.CVALUE
13.703599907400d0
.CODATA
CODATA10
**INTEGRAL
.NUCMOD
1
*READIN
.UNCONTRACTED
**HAMILTONIAN
.DOSSSS
.DFT
PBE
**WAVE FUNCTION
.SCF
*SCF
.CLOSED SHELL
10
**MOLECULE
*BASIS
.DEFAULT
dya11.v2z
**END OF
```

The calculation can be run with:

```
$ ./pam --inp=4DC.inp --mol=ch4.xyz
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/utils/cfread.md

orphan

CFREAD

CFREAD is a utility program for getting information from the file DFCOEf containing MO-coefficients, orbital energies and, if relevant, information about boson or linear symmetries. Usage is straightforward, for instance:

```
dirac 26>cfread.x DFC0EF
```

```
CFREAD; Information read from file : DFC0EF
```

```
- Heading :DIRAC: No title specified !!!
- Total energy: -75.1621606438559979
- Fermion ircops      :      1
```

```
Tue Sep 24 23:05:49 2013
```

```
-----
- Negative-energy orbitals:      0
- Positive-energy orbitals:     21
- A0-basis functions      :     21
```

Note that the electronic energy is calculated before diagonalization, so that coefficients from iteration n refer to electronic energy from iteration $n-1$. At convergence this should not matter.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/utils/labread.md

orphan

LABREAD

LABREAD is a utility program for getting information about integrals stored on the one-electron integral file AOPROPER. Usage is straightforward, for instance:

```
dirac 55>labread.x AOPROPER
```

```
MOLECULE labels found on file :AOPROPER
```

```

1      ***** 25Jul12  SY 1TFFT  OVERLAP
3      ***** 25Jul12  SY 1TFFT  MOLFIELD
5      ***** 25Jul12  SY 1FFFT  BETAMAT
7      ***** 25Jul12  AN 1FTTF  XDIPVEL
9      ***** 25Jul12  AN 1FTTF  YDIPVEL
11     ***** 25Jul12  AN 1FTTF  ZDIPVEL
13     ***** 25Jul12  SY 1TFFT  PVCF1 01
15     ***** 25Jul12  SY 1TFFT  PVCF1 02
17     ***** 25Jul12  SY 1TFFT  FC F1 01
19     ***** 25Jul12  SY 1TFFT  FC F1 02
21     ***** 25Jul12  SY 1FTTF  PVCF1 01
23     ***** 25Jul12  SY 1FTTF  PVCF1 02
25     ***** 25Jul12  SY 1TFFT  ED F1 01
27     ***** 25Jul12  SY 1TFFT  ED F1 02
29     ***** 25Jul12  SY 1TFFT  E0FLABEL
```

```
29 records read before EOF on file.
```

```
STOP
```

The notation *SY* and *AN* indicate whether the integral matrix is symmetric or not. In general only certain combinations of large and small component blocks have been calculated. An even operator has *TFFT*, meaning that only *LL* and *SS* blocks have been generated, whereas an odd operator has *FTTF*, meaning that only *LS* and *SL* blocks were generated. For information about integral labels, see the `one_electron_operators` section.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/utils/twofit.md

orphan

TWOFIT

TWOFIT is a utility program for the extraction of spectroscopic constants ($\hat{r}_e$, $\hat{\omega}_e$, $\hat{x}_e\omega_e$) of diatomic molecules. It can also extract equivalent information from normal modes provided the reduced mass $\hat{\mu}$ is given.

Theory

Consider the following Hamiltonian

$$\hat{H} = \hat{H}_0 + \hat{H}_1 + \hat{H}_2$$

where $\hat{H}_0$ is the Hamiltonian for the harmonic oscillator

$$\hat{H}_0 = \frac{\hat{p}^2}{2\mu} + \frac{1}{2}V_2x^2; \quad E_v^{(0)} = \hbar\omega(v + \frac{1}{2})$$

and $\hat{H}_1$ and $\hat{H}_2$ consists of cubic and quartic contributions, respectively, to a quartic force field

$$\hat{H}_1 = \frac{1}{6}V_3x^3; \quad \hat{H}_2 = \frac{1}{24}V_4x^4$$

From second order perturbation theory we get the following energy

$$E_v = \hbar\omega(v + \frac{1}{2}) + \frac{1}{24}V_4 \langle x^4 \rangle_v - \frac{V_3^2}{36} \langle x^2 \rangle_v^2$$

Further manipulations then give the final expression

```
\begin{aligned}
\begin{array}{lcl}
```

$$E_v = \hbar\omega(v + \frac{1}{2}) + \frac{1}{16}V_4 \left(\frac{\hbar}{\mu\omega} \right)^2 (v + \frac{1}{2}) - \frac{5V_3}{48} \left(\frac{\hbar}{\mu\omega} \right)^3 - \frac{V_4}{16} \left(\frac{\hbar}{\mu\omega} \right)^4$$

to be compared with the general expression

$$E_v = (v + \frac{1}{2})\hbar\omega - (v + \frac{1}{2})^2\hbar\omega x_e$$

The anharmonic constant can accordingly be calculated as

$$\hbar\omega x_e = \left[\frac{5V_3}{48} \left(\frac{\hbar}{\mu\omega} \right)^3 - \frac{V_4}{16} \left(\frac{\hbar}{\mu\omega} \right)^4 \right] / \hbar\omega$$

The average displacement is given as the standard deviation

$$\Delta x = \sigma = \sqrt{\langle x^2 \rangle - \langle x \rangle^2} = \sqrt{\frac{1}{\hbar\omega} \left[\frac{5V_3}{48} \left(\frac{\hbar}{\mu\omega} \right)^3 - \frac{V_4}{16} \left(\frac{\hbar}{\mu\omega} \right)^4 \right]}$$

Usage

The usage of TWOFIT is rather self-explanatory. TWOFIT will read a formatted file of distances and energies prepared by the user. It will then, through a dialogue with the user, perform a polynomial fit to specified degree of the potential curve, find the equilibrium distance r_e by a Newton-Raphson search and from the extracted force constants V_2 , V_3 and V_4 calculate the harmonic frequency ω_e as well as the anharmonic constant $\omega_e x_e$. For a diatomic molecule, the program can look up masses of the most abundant isotope for specified nuclear charge or accept atomic masses from the user in order to calculate the reduced mass μ . For normal modes, the reduced mass must be given directly.

Example1: Spectroscopic constants of HgF

The potential curve of mercury monofluoride HgF has been calculated at the X2C/B3LYP level. We extract the following data into the file B3LYPpot :

```
1.94 -19746.664565273648
1.96 -19746.665970540136
1.98 -19746.667076184542
2.00 -19746.667907574058
2.02 -19746.668483409372
2.04 -19746.668818993279
2.06 -19746.668929911564
```

We fire up TWOFIT:

```
twofit.x
```

```
*****
***** TWOFIT for diatomics : Written by T. Saue *****
*****
```

```
Select one of the following:
1. Spectroscopic constants.
2. Spectroscopic constants + properties.
```

We chose the first option:

```
1
Name of input file with potential curve with " R E(R)" values (A40)
```

We provide the name of the file containing the potential curve:

```
B3LYPpot
Select one of the following:
1. Bond lengths in Angstroms.
2. Bond lengths in atomic units.
(Note that all other quantities only in atomic units !)
```

The bond lengths are in this case in Angstroms:

```
1
* Number of points read: 7
```

```
** SPECTROSCOPIC CONSTANTS:
```

Polynomial fit: Give order of polynomial

With seven points the maximal order of the polynomial is six. We select order five, but the user is encouraged to try different options to see how the calculated spectroscopic constants converge with the number of points, their spacing as well as order of the polynomial:

```
5
* Polynomial fit of order: 5
* Coefficients:
c( 0): -1.943938E+04
c( 1): -4.003912E+02
c( 2): 2.091186E+02
c( 3): -5.469999E+01
c( 4): 7.162816E+00
c( 5): -3.754896E-01
X Predicted Y Relative error
1.940 -1.974666456527E+04 -2.5608E-14
1.960 -1.974666597054E+04 1.3265E-13
1.980 -1.974666707618E+04 -3.4212E-13
2.000 -1.974666790758E+04 4.4879E-13
2.020 -1.974666848340E+04 -3.4194E-13
2.040 -1.974666881900E+04 1.3228E-13
```

```

      2.060      -1.974666892991E+04 -2.6345E-14
* Chi square : .1840E-15
* Chi square per point: .2628E-16
* Number of singularities(SVD): 0
* Local minimum : 2.06044 Angstroms = 3.89366 Bohrs
* Expected energy : -1.9746668930E+04 Hartrees
Select one of the following:
  1. Select masses of the most abundant isotopes.
  2. Employ user-defined atomic masses.
  3. Normal modes: Give reduced mass.

```

We go for the easy solution:

```

1
* Give charge of atom A:

```

The order of atoms does not matter. We select mercury first:

```

80
* Mass      : 201.9706
* Abundance: 29.8600
* Give charge of atom B :

```

and then fluorine:

```

9
* Mass      : 18.9984
* Abundance: 100.0000
* Force constant : 2.2407E+02 N/m
* Frequency    : 1.40297E+13 Hz
                  4.67981E+02 cm-1
* Mean displacement in harmonic ground state: 0.0644 Angstroms
  corresponding to interval: [ 1.9960, 2.1248]
* WARNING : 3 points lie outside interval...
This may reduce the quality of your spectroscopic constants.
* Omega*x_e : 1.64740E+01 cm-1
Do you want to calculate spectroscopic constants with other isotope(s) (y/n)?

```

Note that we get a warning, because the program checks whether all points lie inside the standard deviation of the ground state (harmonic approximation), corresponding to the mean displacement Δx (see above). Such a warning does not disqualify the fit, but the user has to judge whether he has selected his points wisely. Various options follow, but we will simply say no to all of them:

```

n
Do you want to calculate spectroscopic constants
with another polynomial order (y/n)?
n
Do you want to calculate spectroscopic constants at another geometry (y/n)?
n

```

Example 2: Anharmonicity of C-F stretch in CHFCIBr

We start from a set of CCSD(T) energies (using scalar relativistic pseudopotentials) along the normal coordinate Q associated with the C-F stretch of the chiral CHFCIBr:

```

-0.50 -610.12399429
-0.40 -610.62520179
-0.30 -610.86649287
-0.20 -610.97769875
-0.10 -611.02238183
0.00 -611.03293760
0.10 -611.02641387
0.20 -611.01190951
0.50 -610.95789506
0.75 -610.91897869
1.00 -610.89433750

```

The normal coordinates and associated normal coordinate $\mu = 9.7032$ was obtained from vibrational analysis (not using DIRAC). The TWOFIT run proceeds as before:

```
twofit.x
```

```

*****
***** TWOFIT for diatomics : Written by T. Saue *****
*****

```

```

Select one of the following:
  1. Spectroscopic constants.
  2. Spectroscopic constants + properties.
1
Name of input file with potential curve with " R E(R)" values (A40)
chfclbr.pes.dat
Select one of the following:
  1. Bond lengths in Angstroms.
  2. Bond lengths in atomic units.
(Note that all other quantities only in atomic units !)
1
* Number of points read: 11

** SPECTROSCOPIC CONSTANTS:

Polynomial fit: Give order of polynomial
9

```

```

* Polynomial fit of order: 9
* Coefficients:
c( 0): -6.110329E+02
c( 1): 6.084737E-05
c( 2): 2.297324E-01
c( 3): -2.947082E-01
c( 4): 2.477551E-01
c( 5): -1.473893E-01
c( 6): 8.141347E-02
c( 7): -7.485671E-02
c( 8): 4.956602E-02
c( 9): -1.180208E-02
X Predicted Y Relative error
-0.500 -6.101239943742E+02 1.3793E-10
-0.400 -6.106252010277E+02 -1.2483E-09
-0.300 -6.108664959044E+02 4.9673E-09
-0.200 -6.109776918295E+02 -1.1327E-08
-0.100 -6.110223916741E+02 1.6111E-08
0.000 -6.110329287639E+02 -1.4461E-08
0.100 -6.110264185901E+02 7.7249E-09
0.200 -6.110119083146E+02 -1.9564E-09
0.500 -6.109578950951E+02 5.7392E-11
0.750 -6.109189786861E+02 -6.3733E-12
1.000 -6.108943375002E+02 4.0123E-13
* Chi square : .2564E-09
* Chi square per point: .2331E-10
* Number of singularities(SVD): 0
* Local minimum : -0.00007 Angstroms = -0.00013 Bohrs
* Expected energy : -6.1103292877E+02 Hartrees
Select one of the following:
1. Select masses of the most abundant isotopes.
2. Employ user-defined atomic masses.
3. Normal modes: Give reduced mass.

```

but now we select the third option:

```

3
* Give reduced mass in Daltons:
9.7032
* Force constant : 7.1570E+02 N/m
* Frequency : 3.35432E+13 Hz
1.11888E+03 cm-1
* Mean displacement in harmonic ground state: 0.0557 Angstroms
corresponding to interval: [ -0.0558, 0.0557]
* WARNING : 10 points lie outside interval...
This may reduce the quality of your spectroscopic constants.
* Omega*x_e : 9.10592E+00 cm-1

```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/utils/vibcal.md

orphan

VIBCAL

VIBCAL is a utility program allowing the calculation of vibrationally averaged properties.

Theory

Suppose that we have optimized the geometry of our molecule and have carried out a harmonic vibrational analysis. For a selected normal mode we expand the electronic energy E as function of the associated normal coordinate q about the equilibrium position $q=0$.

$$E(q) = E^{[0]} + E^{[1]}q + \frac{1}{2}E^{[2]}q^2 + \frac{1}{6}E^{[3]}q^3 + \dots; \quad E^{[n]} = \left. \frac{\partial^n E}{\partial q^n} \right|_{q=0}$$

We note that $q=0$ implies $E^{[1]}=0$ and that the second derivative $E^{[2]}$ corresponds to the force constant k . We furthermore expand some property P equivalently $P(q)$ in the same manner.

From perturbation theory we now obtain an expression for the expectation value of the property P for vibrational level ν of the selected normal mode

$$P_{\nu} = \left\langle \nu \left| P(q) \right| \nu \right\rangle_q = P^{[0]} + \frac{1}{2}P^{[2]} \left(\frac{\hbar}{\mu\omega_e} \right) \left(\nu + \frac{1}{2} \right) - \frac{1}{6}P^{[3]} E^{[3]} \left(\frac{\hbar}{\mu\omega_e} \right)^2 \frac{1}{2} \left(\nu + \frac{1}{2} \right) \left(\nu + \frac{3}{2} \right) + \dots$$

where appears the reduced mass μ and harmonic frequency $\omega_e = \sqrt{k/\mu}$ of the selected normal mode.

Experimentally one can observe the change in this expectation value between two vibrational levels, given by

$$P_{\nu+1} - P_{\nu} = \frac{1}{2} \frac{\hbar}{\mu\omega_e} \left(P^{[2]} - \frac{1}{2} \mu\omega_e^2 P^{[1]} E^{[3]} \right) \Delta\nu; \quad \Delta\nu =$$

We note that the purely electronic contribution $P^{[0]}$ does not contribute to the vibrational shift. Furthermore, we may note that for a purely harmonic mode ($E^{[3]}=0$) and a property purely linear in the normal coordinate ($P^{[2]}=0$) the vibrational shift is strictly zero.

Usage

VIBCAL will read a formatted file of distances, energies and property values. The distances may be in Angstroms, but all other quantities have to be in atomic units. VIBCAL will next6, through a dialogue, with the user generate the necessary derivatives for the calculation of vibrational shifts of the property.

Example: Parity violation shift associated with the C-F stretch of CHFCIBr

We start from a set of CCSD(T) energies and B3LYP parity violation energies calculated along the normal coordinate Q associated with the C-F stretch of the chiral CHFCIBr:

```
-0.5000000000000000 -0.61012399429E+03 2.633089925380E-17
-0.2000000000000000 -0.61097769875E+03 9.078015651573E-18
-0.1000000000000000 -0.61102238183E+03 4.933686545396E-18
0.0000000000000000 -0.61103293760E+03 1.544047622021E-18
0.1000000000000000 -0.61102641387E+03 -1.026099838285E-18
0.2000000000000000 -0.61101190951E+03 -2.75550508570E-18
0.5000000000000000 -0.61095789506E+03 -1.870247865432E-18
```

The coordinates are in this case given in Angstroms. We fire up VIBCAL:

```
$vibcal.x
```

```
*****
***** VIBCAL for properties *****
*****
```

```
Set print level [default:0]:
0. Basic print level
1. Print additional information
```

Reduced mass (in Daltons)

Here we chose the default print level. For the reduced mass you have to check with the preceeding vibrational analysis. We give:

```
9.7032
Name of the input file (A30) with potential/property curve
```

It is a good idea to have a telling name of the input file :

```
CCpot_B3LYPprp
Select one of the following:
1. Bond lengths in Angstroms.
2. Bond lengths in atomic units.
(Note that all other quantities only in atomic units !)
1
```

Here we choose Angstroms for the normal coordinate. :

```
Select one of the following:
1. Numerical derivatives (assumes fixed step length).
2. Polynomial fits
2
```

In this example we do not have a fixed step length for our distances, so we select the second option which uses polynomials fits. :

```
We have 7 points
Give order of polynomial:
5
Select point for the calculation of derivatives
0.0
```

The distances refer to the normal coordinate associated with the C-F stretch, so we set $Q=0.0$ since this refers to the equilibrium structure. :

Results have been written to CCpot_B3LYPprp.vibcal

Looking into this file we find :

```
*****
**** SOME INFORMATION ****
*****
Reduced mass in Daltons: 9.7032000000000007
2nd potential derivative: 0.42167678827523780
3rd potential derivative: -1.6807622640206681
4th potential derivative: 9.1374786165355175
0th property derivative: 1.55000030970252530E-018
1st property derivative: -1.58073567198751495E-017
2nd property derivative: 2.23189000229251784E-017
```

```
*****
***** RESULTS *****
*****
Difference : -0.24366752E-18 au = -1.6 mHz
PV shift: -3.207 mHz
Harmonic model:
Difference : 0.12921573E-18 au = 0.9 mHz
PV shift: 1.700 mHz
```

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/visual/general/tutorial.md

orphan

How to visualize densities and properties

Tip: this tutorial is also available as a Jupyter notebook in `../././././demo_notebooks/visual_overview.ipynb`. For an interactive basic tutorial, see `../././././demo_notebooks/visual_interactive.ipynb`.

What we will need and cover

DIRAC is able to plot a variety of scalar and vector fields representing unperturbed and perturbed densities. In this tutorial we will walk through a couple of example calculations using the classic molecule benzene.

The visualization module in DIRAC always needs the unperturbed wave function, CHECKPOINT.h5.

For perturbed densities, in addition to CHECKPOINT.h5, we also need the perturbed “wave function” which comes out of a linear response calculation and is typically saved in a file called PAMXVC, here also exported as RESPONSE.h5.

It is useful to perform visualization runs in 3 steps:

- Run the SCF calculation and save CHECKPOINT.h5 or run the response calculation and save CHECKPOINT.h5, PAMXVC (optional) and RESPONSE.h5.
- Calculate densities using CHECKPOINT.h5 (and PAMXVC, RESPONSE.h5) and produce data files.
- Import data files to other programs ([Gnuplot](#), [Mathematica](#), [ParaView](#), [PyNGL](#), ...) to generate plots.

Note that all values (coordinates) are entered in atomic units (bohr), not in ångström.

Getting the unperturbed and perturbed wave function (example molecule: benzene)

We will use the benzene molecule with a very poor basis set for demonstration purposes only. For production calculation this basis set is certainly not sufficient. In real life calculations the basis set needs to be well calibrated. Also the quality of the molecular structural parameters is not important here. We will call the following molecule file benzene.mol:

DIRAC

```
C      2      A
      1.      6
H      0.000000 -2.472878  0.000000
H     -2.141580 -1.236440  0.000000
H      2.141580 -1.236440  0.000000
H     -2.141580  1.236440  0.000000
H      2.141580  1.236440  0.000000
H      0.000000  2.472878  0.000000
LARGE BASIS STO-3G
      6.      6
C      0.000000 -1.391016  0.000000
C     -1.204659 -0.695508  0.000000
C      1.204659 -0.695508  0.000000
C     -1.204659  0.695508  0.000000
C      1.204659  0.695508  0.000000
C      0.000000  1.391016  0.000000
LARGE BASIS STO-3G
FINISH
```

The molecule is oriented in the xy-plane and later we will place the magnetic field vector along the perpendicular z-axis.

And here is a corresponding input file for the response calculation that we will call response.inp:

```
**DIRAC
.WAVE FUNCTION
.PROPERTIES
**HAMILTONIAN
.LVCORR
.URKBAL
**WAVE FUNCTION
.SCF
**PROPERTIES
*LINEAR RESPONSE
.RSPEXP
.A OPERATOR
'B_z oper'
ZAVECTOR
'YDIPLN'
'XDIPLN'
COMFACTOR
-68.517999904721
.B OPERATOR
'B_z oper'
ZAVECTOR
'YDIPLN'
'XDIPLN'
COMFACTOR
-68.517999904721
.ANALYZE
*END OF
```

Later we will want to plot the magnetically induced current density so please observe how we specify the perturbing operator for the magnetic field to be perpendicular to the molecular plane:

```
...
.B OPERATOR
```



```
'B_z oper'
ZAVECTOR
'YDIPLN'
'XDIPLN'
COMFACTOR
-68.51799904721
...
```

This is the operator for the magnetic field along z. If you only want to plot unperturbed densities than you can remove DIRAC_.PROPERTIES and everything under **PROPERTIES. Also note that in production calculation we really should run with uncontracted basis sets:

```
**INTEGRALS
*READIN
.UNCONTRACT
```

But here we will cheat and use a contracted set for speed.

Now we are ready to run the calculation (here done using MPI; finishes in ca. 10 seconds) and save the resulting CHECKPOINT.h5 and PAMXVC:

```
pam [--mpi=8] --scratch=/tmp --mw=10 --inp=response.inp --mol=benzene.mol --get="CHECKPOINT.h5 PAMXVC RESPONSE.h5"
```

Verify that you have the files CHECKPOINT.h5, PAMXVC, and RESPONSE.h5 in your directory, we will need them for the visualizations described further down.

You should also check that you have obtained the following response functions:

```
grep "at freq" response_benzene.out
```

```
<< 1, 1>>: -0.23026724E+02 a.u.,      E--contribution at frequency      0.0000000000 a.u.
<< 1, 1>>:  0.99833086E+02 a.u.,      P--contribution at frequency      0.0000000000 a.u.
       76.80636218049 a.u. at frequency      0.00000000 a.u.      1.33E-06      (converged)
```

These are the paramagnetic (“E–contribution”), diamagnetic (“P–contribution”), and the total zz magnetizability tensor element, respectively. Of course they will have nothing to do with experimental magnetizabilities (because we use a poor basis set) but it does not matter here.

We have given the molecule without symmetry information and let DIRAC find the symmetry.

DIRAC correctly assigns the molecule symmetry but note how the molecule is reoriented:

Original Coordinates

1	0.00000000	-4.67306218	0.00000000	1
1	-4.04699969	-2.33653298	0.00000000	1
1	4.04699969	-2.33653298	0.00000000	1
1	-4.04699969	2.33653298	0.00000000	1
1	4.04699969	2.33653298	0.00000000	1
1	0.00000000	4.67306218	0.00000000	1
6	0.00000000	-2.62863929	0.00000000	1
6	-2.27647559	-1.31431964	0.00000000	1
6	2.27647559	-1.31431964	0.00000000	1
6	-2.27647559	1.31431964	0.00000000	1
6	2.27647559	1.31431964	0.00000000	1
6	0.00000000	2.62863929	0.00000000	1

Symmetry class found: D(6h)

Centered and Rotated

6	-1.31431964	2.27647559	0.00000000	1
6	-1.31431964	-2.27647559	0.00000000	1
6	1.31431964	2.27647559	0.00000000	1
6	1.31431964	-2.27647559	0.00000000	1
6	-2.62863929	0.00000000	0.00000000	1
6	2.62863929	0.00000000	0.00000000	1
1	-2.33653298	4.04699969	0.00000000	1
1	-2.33653298	-4.04699969	0.00000000	1
1	2.33653298	4.04699969	0.00000000	1
1	2.33653298	-4.04699969	0.00000000	1
1	-4.67306218	0.00000000	0.00000000	1
1	4.67306218	0.00000000	0.00000000	1

The following elements were found: X Y Z

It is important to remember that the “Centered and Rotated” geometry is the one that we will probe in the visualization.

Example unperturbed densities

Density in a plane

Let us plot the density in the plane spanned by the points (-8.0 -8.0 0.0), (8.0 -8.0 0.0), and (-8.0 8.0 0.0) with 100 steps along each side:

```
...
**VISUAL
.GRIDS
1
id=1 input=2d
.2D
-8.0 -8.0 0.0
 8.0 -8.0 0.0
100
-8.0 8.0 0.0
100
.GRIDFUNCTIONS
```

```
1
name=ed id_grid=1 purpose=visualization export=tst file_out=ed.tst
*END OF
```

And copy the generated txt files back from the scratch directory:

```
pam --mw=10 --inp=density_2d.inp --mol=benzene.mol --put="CHECKPOINT.h5 RESPONSE.h5" --get="ed.tst"
```

Density in 3D

In this example we plot the density in 3D and 80 points along each direction:

```
...
**VISUAL
.GRIDS
1
id=1 input=create typ=1 npoints=[80,80,80] margin=4.0
.GRIDFUNCTIONS
1
name=ed id_grid=1 purpose=visualization export=cube file_out=ed.cube
*END OF
```

together with the following runscript:

```
pam --mw=10 --inp=density_3d.inp --mol=benzene.mol --put=CHECKPOINT.h5 --get="ed.cube"
```

The generated ed.cube can be used read into [Molekel](#) for instance to produce figure like this one:



Integrating densities in 3D

Densities can be integrated in 3D using the numerical DFT grid. This example will integrate the unperturbed density to the number of electrons:

```
...
**VISUAL
.GRIDS
1
id=1 input=dft
.GRIDFUNCTIONS
1
name=ed id_grid=1 purpose=integration
*END OF
```

Plotting the ELF in 3D

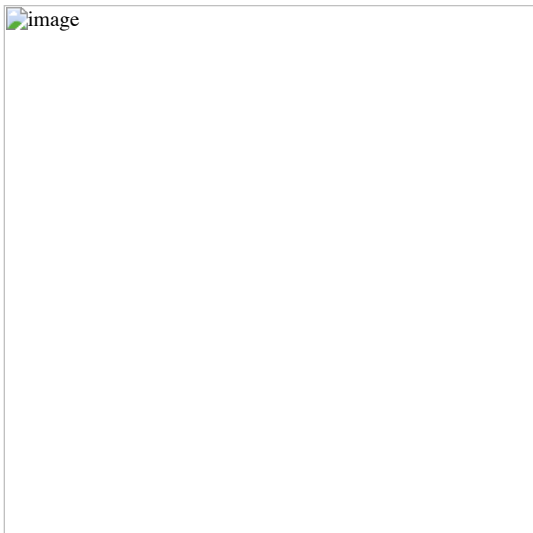
In this plot we visualize the electron localization function in benzene:

```
...
**VISUAL
.GRIDS
1
id=1 input=create typ=1 npoints=[80,80,80] margin=10.0
.GRIDFUNCTIONS
1
name=elf id_grid=1 purpose=visualization export=cube file_out=elf.cube
*END OF
```

Observe how we have increased the cube size (default is 4.0 bohr):

```
pam --mw=10 --inp=elf_3d.inp --mol=benzene.mol --put=CHECKPOINT.h5 --get="elf.cube"
```

Here is the resulting ELF in benzene:



Densities and currents induced by a magnetic field

Current density

We have arrived at the heart of this tutorial: here we will calculate and plot the magnetically induced current density in benzene.

For this we will need CHECKPOINT.h5 and PAMXVC.

Let us first examine the PAMXVC file. For this we will use the `rspread.x` utility to check what is on the PAMXVC file:

```
./rspread.x /path/to/PAMXVC
```

and we get:

Solution vectors found on file : PAMXVC

```
** Solution vector : B_z oper      Irrep:  4 Trev: -1
  1 Type : E- Freq.:    0.000000 Rnorm:   0.13E-05 Length:    157
  2 Type : P- Freq.:    0.000000 Rnorm:   0.13E-05 Length:   2835
```

In general PAMXVC can have many records. In this simple example it only contains two records: first is the “paramagnetic” response (“E-” for “electronic”), second is the “diamagnetic” response (“P-” for “positronic”).

Better names for the two records would be positive-positive energy orbital response, and positive-negative orbital response.

So, if we want to get the paramagnetic current density we will only use record 1, for the diamagnetic current density we will only use record 2, and for the total current density we will use both records.

Let’s start with the total current density. We choose a plane spanned by the points (-6.0 -6.0 0.0), (6.0 -6.0 0.0), and (-6.0 6.0 0.0) with 100 steps along each side (this is the molecular plane):

```
**DIRAC
#.WAVE FUNCTION
**HAMILTONIAN
.LVCORR
.URKBAL
**WAVE FUNCTION
.SCF
**VISUAL
.GRIDS
1
id=1 input=2d
.2D
-6.0 -6.0 0.0
 6.0 -6.0 0.0
 100
-6.0 6.0 0.0
 100
.GRIDFUNCTIONS
2
name=j id_grid=1 purpose=visualization export=tst rspvec_file=RESPONSE.h5 rspvec_id=1 file_out=j_Bz_rspvec1.txt
name=j id_grid=1 purpose=visualization export=tst rspvec_file=RESPONSE.h5 rspvec_id=2 file_out=j_Bz_rspvec2.txt
*END OF
```

If we wanted to see the diamagnetic current density we would only use the record 2:

```
...
**VISUAL
.GRIDFUNCTIONS
1
name=j id_grid=1 purpose=visualization export=tst rspvec_file=RESPONSE.h5 rspvec_id=2 file_out=j_Bz_rspvec2.txt
```

With the following `pam` command we provide CHECKPOINT.h5 and RESPONSE.h5 and copy back the generated 2D plot:

```
pam --mw=10 --inp=j_in_plane.inp --mol=benzene.mol --put="CHECKPOINT.h5 RESPONSE.h5" --get=j_Bz_*.txt
```

How about the current above the molecular plane (say, 1 bohr)? This is how it works:

```
...
.2D
-6.0 -6.0 1.0
 6.0 -6.0 1.0
100
-6.0 6.0 1.0
100
```

with:

```
pam --mw=10 --inp=j_above_plane.inp --mol=benzene.mol --put="CHECKPOINT.h5 RESPONSE.h5" --get=j_Bz_*.txt
```

The generated file plot.2d.vector contains in this case 100*100 lines with 4 numbers each: the first 2 are the position coordinates, the other 2 are projections of the vector components along these coordinates.

Current density with London atomic orbitals

In order to use London atomic orbitals in the calculation and visualization of magnetically induced current density, there are only few modifications needed to be made to the protocol described above.

First, we need to save TBMO file from the preceding response calculation, in addition to CHECKPOINT.h5 and PAMXVC. We should use the rsread.x utility in order to check which record of PAMXVC corresponds contains response to each of components of magnetic field. An exemplary output may be analogous to:

Solution vectors found on file : PAMXVC

```
** Solution vector : LA0-XRM1H1      Irrep:  1 Trev:  -1
 1 Type : E- Freq.:    0.000000 Rnorm:  0.90E-08 Length:   11448
 2 Type : P- Freq.:    0.000000 Rnorm:  0.90E-08 Length:   12744
** Solution vector : LA0-YRM1H1      Irrep:  1 Trev:  -1
 3 Type : E- Freq.:    0.000000 Rnorm:  0.83E-07 Length:   11448
 4 Type : P- Freq.:    0.000000 Rnorm:  0.83E-07 Length:   12744
** Solution vector : LA0-ZRM1H1      Irrep:  1 Trev:  -1
 5 Type : E- Freq.:    0.000000 Rnorm:  0.48E-07 Length:   11448
 6 Type : P- Freq.:    0.000000 Rnorm:  0.48E-07 Length:   12744
```

What identifies records 1 and 2 as containing the response to the “X”-component of an external magnetic field, records 3 and 4 - to the “Y”-component and records 5 and 6 - to the “Z”-component.

Then, we need to change the visualization input slightly:

```
...
.GRIDFUNCTIONS
2
name=j id_grid=1 purpose=visualization export=txt rspvec_file=RESPONSE.h5 rspvec_id=5 lao=1 file_out=j_Bz_rspvec5.txt
name=j id_grid=1 purpose=visualization export=txt rspvec_file=RESPONSE.h5 rspvec_id=6 lao=1 file_out=j_Bz_rspvec6.txt
```

Integrating the current density passing through a plane

This is useful to get a number that can be used to quantify the aromaticity. For this molecule in the present orientation:

...

Centered and Rotated

```
-----
 6      -1.31431964    2.27647559    0.00000000    1
 6      -1.31431964   -2.27647559    0.00000000    1
 6      1.31431964    2.27647559    0.00000000    1
 6      1.31431964   -2.27647559    0.00000000    1
 6      -2.62863929    0.00000000    0.00000000    1
 6      2.62863929    0.00000000    0.00000000    1
 1      -2.33653298    4.04699969    0.00000000    1
 1      -2.33653298   -4.04699969    0.00000000    1
 1      2.33653298    4.04699969    0.00000000    1
 1      2.33653298   -4.04699969    0.00000000    1
 1      -4.67306218    0.00000000    0.00000000    1
 1      4.67306218    0.00000000    0.00000000    1
```

A good choice for an integration plane would be the y-z plane.

This is because in such an integration we do not want to cut through nuclear centers. Close to the nuclei the current density can be very large and non-uniform which leads to a very slow convergence with respect to the numerical grid.

In DIRAC we use the Gauss-Lobatto quadrature. You have to specify a plane, divide this plane into sufficiently many “tiles” (I would start with 5-10 tiles along each side). You also have to specify the order of the Legendre polynomial that is used to generate the roots and weights on each tile.

Let us look at an explicit example:

```
...
.2D_INT
0.0  0.0  0.0
0.0 10.0  0.0
5
0.0  0.0 10.0
5
10
```

This is a plane going from the origin to 10 bohr along y and z. So it is a 10*10 bohr plane. We consider here only the region “above” the molecule. We can do this because of symmetry but have to remember to multiply the result by 2.

The plane is divided into 5 tiles along each side and the order of the Legendre polynomial is 10. To get converged results the plane has to be sufficiently large to cover “all of the current density”, with sufficiently many tiles and sufficiently high polynomial order. You have to experiment a bit.

And this is the interesting output (in atomic units):

```
+-----+
! 2D Gauss-Lobatto numerical integration !
+-----+
```

plane is spanned by 3 points:

```
"origin"    0.0000    0.0000    0.0000
"right"     0.0000   10.0000    0.0000
"top"       0.0000    0.0000   10.0000
```

```
nr of pieces to "right"  5
nr of pieces to "top"   5
```

```
order                10
```

```
scalar            x-component      y-component      z-component
0.0000000000E+00  0.5908630369E-01  0.0000000000E+00  0.0000000000E+00
```

The only significant contribution is along x as it should be (due to symmetry).

If you want, you can plot and integrate at the same time by specifying a plot plane (.2D) and an integration plane (.2D_INT).

What else is there

See the main DIRAC manual for a list of available molecular fields.

Densities for a list of points in space

Let's say that somebody asks you to verify that the density at the nuclear centers of the benzene molecule is same for all 6 centers. This is how it could be done:

```
...
**VISUAL
.GRIDS
1
id=1 input=list
.LIST
6
0.000000 -1.391016 0.000000
-1.204659 -0.695508 0.000000
1.204659 -0.695508 0.000000
-1.204659 0.695508 0.000000
1.204659 0.695508 0.000000
0.000000 1.391016 0.000000
```

We ask for the unperturbed density, 6 points, followed by coordinates of the 6 points.

Densities along a line

For example between two carbon atoms in benzene (100 steps):

```
...
**VISUAL
.GRIDS
1
id=1 input=line
.LINE
-1.31431964 2.27647559 0.0
-1.31431964 -2.27647559 0.0
100
```

Good practices and gotchas

Remember that the molecule can get reoriented if symmetry is detected automatically.

When plotting perturbed densities it is important to use CHECKPOINT.h5 and PAMXVC from the same run. During the visualization step the wave function should not be allowed to (re)optimize.

When plotting orbital densities it is useful to do a 3D integration at the same time. As a sanity check the integrated number should be twice the number of orbitals.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/visual/magnetizability_density/tutorial.md

orphan

Magnetizability density

In this tutorial we will plot the CGO magnetizability density of the water molecule. We will do this in two steps: first we will run the SCF and response calculation and save the CHECKPOINT.h5, PAMXVC, and RESPONSE.h5 files, then we will restart from these and calculate the magnetizability density on a grid of points and write this into a cube file which can be opened by your favorite molecular visualization program (the author of this tutorial likes Avogadro).

Tip: it might be helpful to check the analysis of other magnetic response densities in a Jupyter notebook in `../..../demo_notebooks/visual_overview.ipynb`.

Getting the wave function and the response vectors

Let us use the following molecule input (h2o.mol):

```
h2o.mol
```

Together with a simple job input (magnetizability.inp):

```
magnetizability.inp
```

We can get the wave function with the following run script:

```
magnetizability.run
```

After running the script you should see the files CHECKPOINT.h5 and PAMXVC in your submit directory.

And here is the magnetizability tensor:

Total magnetizability tensor (au)

	Bx	By	Bz
Bx	-1.804460514289	-0.000000000000	-0.000000000001
By	-0.000000000000	-1.617444735515	0.000000000000
Bz	-0.000000000001	-0.000000000000	-1.709814652525

Plotting the magnetizability density

We will use the following run script to generate the cube file:

```
density.run
```

Together with the following job input (density.inp):

```
density.inp
```

Finally you can open cube files with your favorite program and visualize the density. You can sum up all contributions to the total density or plot specific components or paramagnetic and diamagnetic contributions separately.

And this is how the total magnetizability density of water looks (isosurface 0.04 a.u.):



FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/visual/mep/tutorial.md

orphan

Molecular electrostatic potential

In this tutorial we will go through the steps necessary for the visualization of the molecular electrostatic potential (MEP). The MEP at a point $\mathbf{r}$ in space is:

$$\varphi(\mathbf{r}) = \sum_{A=1}^{N_{\text{atoms}}} \frac{Z_A}{|\mathbf{R}_A - \mathbf{r}|} - \int \mathrm{d}\mathbf{r}' \frac{\rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|}$$

As suggested in the main visualization tutorial, we first need to run an SCF calculation and save the CHECKPOINT.h5 file. The cube files with the density and the MEP are then obtained. From these files various plots can be obtained, using your favourite visualization tool.

SCF calculation and density cube file

The tutorial will use the benzaldehyde molecule. The file containing the geometry is called benzaldehyde.mol:

```
DIRAC
Benzaldehyde in 3-21G basis
For the MEP tutorial
C 3 0
1. 6
H_a 2.95439 1.18571 0.44306
H_b 2.91616 2.31795 -1.77044
H_c 0.74709 2.96481 -2.79885
H_d -1.38368 2.48172 -1.61822
H_e -2.61236 1.60741 0.22943
H_f 0.82375 0.69840 1.63196
LARGE BASIS 3-21G
6. 7
C_a 0.80849 1.19783 0.65538
C_b 1.99910 1.47074 -0.01013
C_c 1.97783 2.10389 -1.24794
C_d 0.76341 2.46621 -1.82403
C_e -0.42791 2.19562 -1.16299
C_f -0.41073 1.55949 0.08111
C_g -1.70090 1.28473 0.76121
LARGE BASIS 3-21G
8. 1
O_a -1.79172 0.74000 1.83828
LARGE BASIS 3-21G
FINISH
```

By running:

```
pam --scratch=/tmp --mol=benzaldehyde.mol --inp=dc_dens.inp --get "CHECKPOINT.h5" --get "dc_dens.cube"
```

where the dc-dens.inp file contains:

```
**DIRAC
.WAVE FUNCTION
**WAVE FUNCTION
.SCF
**INTEGRALS
*READIN
.UNCONTRACT
**VISUAL
.GRIDS
2
id=1 input=dft
id=2 input=create typ=1 npoints=[80,80,80] margin=6.0
.GRIDFUNCTIONS
2
name=ed id_grid=1 purpose=visualization export=cube file_out=dc_dens.cube
name=ed id_grid=2 purpose=integration
*END OF
```

we run an SCF calculation using the Dirac-Coulomb Hamiltonian. The CHECKPOINT.h5 file is then used to obtain the cube file with the density. Both files are eventually retrieved by the pam script.

The same is done when using the Levy-Leblond Hamiltonian:

```
...
**HAMILTONIAN
.LEVY-LEBLOND
```

you will just need to change the relevant pam command above.

MEP cube file

Obtaining the MEP cube files requires an input similar to that for the density cube files. By running:

```
pam --scratch=/tmp --mol=benzaldehyde.mol --inp=dc_mep.inp --put="CHECKPOINT.h5" --get="dc_mep.cube"
```

where dc-mep.inp contains:

```
**DIRAC
.WAVE FUNCTION
**WAVE FUNCTION
.SCF
**INTEGRALS
*READIN
.UNCONTRACT
```

```

**VISUAL
.GRIDS
2
id=1 input=dft
id=2 input=create typ=1 npoints=[80,80,80] margin=6.0
.GRIDFUNCTIONS
2
name=esp id_grid=1 purpose=visualization export=cube file_out=dc_mep.cube
name=esp id_grid=2 purpose=integration
*END OF

```

Notice that no .WAVE FUNCTION directive is present, since we are restarting from a CHECKPOINT.h5 file. To run using the Levy-Leblond Hamiltonian, just add the right keyword to the **HAMILTONIAN section:

```

...
**HAMILTONIAN
.LEVY-LEBLOND

```

Obtaining the plots

Once the cube files are ready we can use our favourite program to obtain the plots we need. In the following the density cube file is referred to as dens.cube and the MEP cube file as mep.cube.

[!WARNING] The values in the cube files are in atomic units! That is ea_0^{-3} for the density and $E_{\text{h}} e^{-1}$ for the MEP.

In this tutorial [ParaView](#) was used.

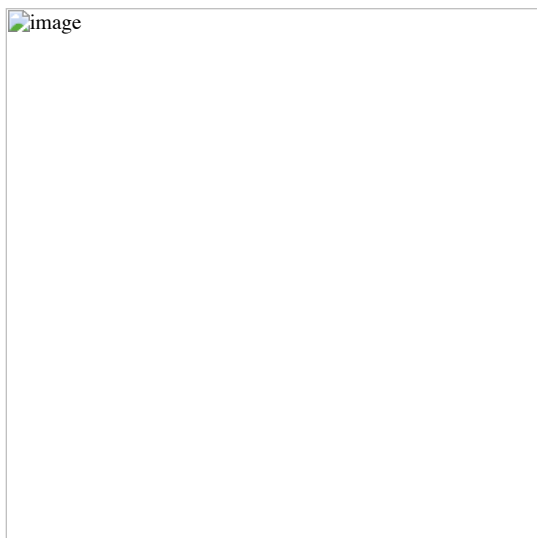
[!WARNING] The following instructions on how to use ParaView **are not** to be intended as an exhaustive guide. They were redacted based on the personal experience of the author of this tutorial. Questions regarding the actual visualization of cube files **must** be addressed to the developers of the visualization program used.

The most common type of plot involving the MEP is maybe the color-mapping of an isodensity surface. In ParaView, you will need to load the dens.cube file as base file and draw an isodensity contour. The mep.cube file is then loaded and its value on that isodensity surface is used as color code:

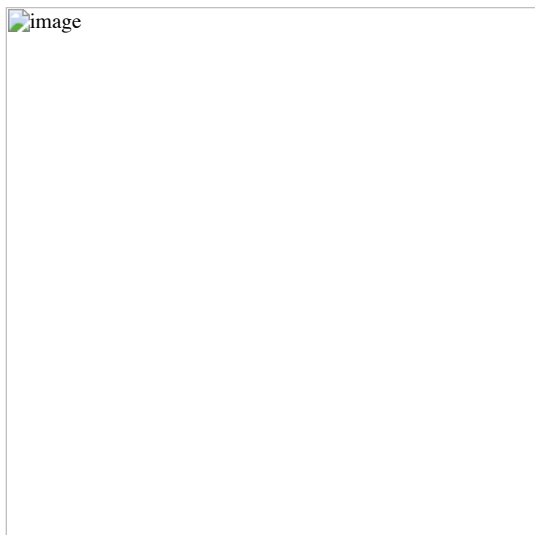
- load the dens.cube file. In the Pipeline Browser you will see that your file has been loaded. Click on the Apply button;
- select Output in the Pipeline Browser;
- go to Filters >> Common >> Glyph;
- in Properties >> Glyph Type select Sphere and apply;
- select Gridded Data in the Pipeline Browser;
- go to Filters >> Common >> Contour;
- in Properties select the isovalue you want and apply. The isovalue 0.2 was here selected;
- load the mep.cube file. Click on the Apply button;
- select Gridded Data for this second file;
- go to Filters >> Alphabetical >> Resample With Dataset;
- in the pop-up window select Input on the left and Gridded Data for the second file on the right;
- now select Source on the left and Contour1 on the right;
- click Apply and you will see the raw result.

To get something better looking: * click the Toggle Color Legend Visibility button to get a legend (you can change its name and position simply); * click the Edit Color Map button; * in the pop-up menu click Choose Preset and pick the color scale you prefer. Here the “Blue to Red Rainbow” has been used; To save a screenshot, go to File >> Save Screenshot.

This results in the following dc_dens-mep.png plot, for the Dirac-Coulomb MEP mapped onto an isodensity contour from the same calculation setup:



The same type of plot is here presented for the Levy-Leblond case:



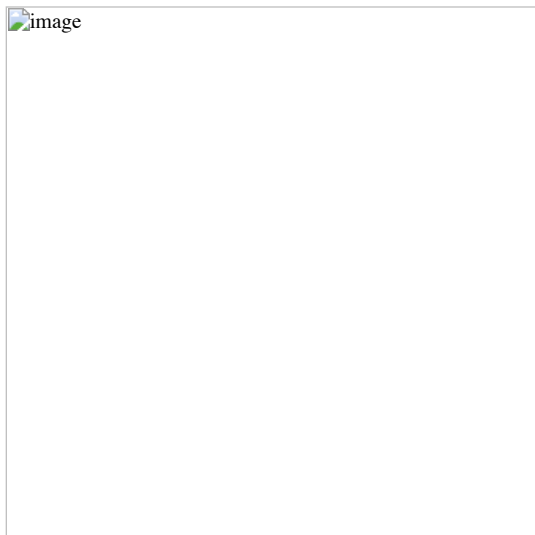
The differences are not striking: the electronic structure of benzaldehyde is not expected to be strongly affected by the inclusion of relativistic effects. As a further example, we can then plot the *difference* between the MEP calculated using the Dirac-Coulomb and the Levy-Leblond Hamiltonians. You will need a way to obtain the difference cube file given the two mep.cube files obtained in the preceding steps of this tutorial. The cubicle.py script provided [here](#) works great for this and other similar types of tasks:

```
cubicle.py --calc="1.0*dc_mep.cube -1.0*ll_mep.cube" > dc-ll_mep.cube
```

Once you have the MEP difference cube file, head over to ParaView and:

- load the dc-ll_mep.cube file. In the Pipeline Browser you will see that your file has been loaded. Click on the Apply button;
- select Output in the Pipeline Browser;
- go to Filters >> Common >> Glyph;
- in Properties >> Glyph Type select Sphere and apply;
- select Gridded Data in the Pipeline Browser;
- go to Filters >> Common >> Slice;
- in Properties select the slicing plane and apply;
- click Apply and you will see the raw result. The color scale can be changed to your liking.

In the case at hand this results in the following plot. As you can see there is practically no difference between the two cases:



[!WARNING] As pointed out in Wheeler2009, it must always be kept in mind that the MEP at a given point is a weighted average of the charge density over all space.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/visual/orbital_densities/tutorial.md

orphan

Plotting orbital densities

In this tutorial we will plot the HOMO density of the water molecule. We will do this in two steps: first we will run the SCF and save the CHECKPOINT.h5 file, then we will restart from this and calculate the orbital density on a grid of points and write this into a cube file which can be opened by your favorite molecular visualization

program (the author of this tutorial likes Avogadro).

Getting the wave function

Let us use the following molecule input (h2o.mol):

```
h2o.mol
```

Together with a simple job input (scf.inp):

```
scf.inp
```

We can get the wave function with the following run script:

```
scf.run
```

After running the script you should see the file CHECKPOINT.h5 in your submit directory.

Plotting the orbital density

Water has 5 orbitals and we will plot the HOMO, orbital nr. 5 (density.inp):

```
density.inp
```

The orbital occupation is controlled by the following keyword:

```
.OCCUPATION
1
1 5 1.0
```

This means that we have one occupation set (first “1”), this occupation set is for Fermion irrep 1 (second “1”), the range is orbital 5 (“5”) and it is fully occupied. To get the total density we could remove this keyword or alternatively:

```
.OCCUPATION
1
1 1-5 1.0
```

We will use the following run script to generate the cube file:

```
density.run
```

Finally you can open plot.3d.cube with your favorite program.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/visual/radial_distributions/tutorial.md

orphan

Plotting radial distributions

In this tutorial we will investigate the valence $3s \rightarrow 3p$ excitation of the magnesium atom and investigate at what radial distances the corresponding electric dipole transition moments pick up contributions.

Tip: this tutorial is also available as a Jupyter notebook in `../demo_notebooks/visual_radial_distributions.ipynb`.

Getting the wave function and transition moments

We use the following atom input (Mg.xyz):

```
Mg.xyz
```

together with the job input (Exc.inp):

```
Exc.inp
```

to be discussed below. We run the job using the command :

```
pam --inp=Exc --mol=Mg.xyz --get "CHECKPOINT.h5 PAMXVC RESPONSE.h5"
```

such that we recover both the coefficient file CHECKPOINT.h5 and the file of solution vector PAMXVC (and RESPONSE.h5).

The magnesium atom is a light atom, so we expect relativistic effects, spin-orbit interaction in particular, to be quite weak, and we can assume more or less pure singlets and triplets. Single excitations $3s \rightarrow 3p$ lead to

```
\begin{aligned}
\left(\begin{array}{c}2\\1\end{array}\right)\left(\begin{array}{c}6\\5\end{array}\right) = 12
\end{aligned}
```

microstates, of which 9 are triplets and 3 are singlets. The atomic calculation will be run in the D_{2h} subgroup and we shall focus on excitations $3s \rightarrow 3p$ in the irrep B_{3u} , which is the symmetry of the x -coordinate. We start from a closed-shell reference, which is therefore totally symmetric. This means that the symmetry of the excitation is also the symmetry of the excited state. One should note that the symmetry refers to the *total* symmetry, including both spatial and spin symmetry. Singlets S are totally symmetric, whereas the components (T_x, T_y, T_z) of the triplet transform as the rotations. From further symmetry analysis we deduce that there will be three $3s \rightarrow 3p$ excitations B_{3u} : i) a $3s \rightarrow 3p_y$ transition of the T_z triplet component, ii) a $3s \rightarrow 3p_z$ transition of the T_y triplet component, and iii) a $3s \rightarrow 3p_x$ transition of singlet character. From Hund's rules we expect the triplet excitations to be the lowest one and will therefore focus on the third, singlet excitation, which is not spin-forbidden. Indeed, when we look in the output we find

```
exc.snippet
```

showing that the first two excitations have essentially no oscillator strength, in contrast to the final one.

Plotting radial distributions

The oscillator strength is given by

$$f_{fi} = \frac{2\omega}{\hbar e^2} \left| \langle f | \hat{T}(\omega_{fi}) | i \rangle \right|^2$$

where the excitation energy is given by $\Delta E = \hbar\omega_{fi}$. Within the electric dipole approximation the reduced interaction operator $\hat{T}$ is given by

$$\hat{T}_L = \boldsymbol{\mu} \cdot \boldsymbol{\epsilon}; \quad \boldsymbol{\mu} = -e\mathbf{r},$$

in the length representation and

$$\hat{T}_V = \frac{e}{\omega} c \boldsymbol{\alpha} \cdot \boldsymbol{\epsilon}$$

in the velocity representation. When the vector $\boldsymbol{\epsilon}$ indicating the (polarization) orientation of the electric field component of the incident light is set to the Cartesian unit vector $\mathbf{e}_x$, the interaction operator spans the B_{3u} irrep. These are the operators specified in the input. However, in the velocity representation the reduced interaction operator $\hat{T}_V$ is frequency-dependent, and since we do not know excitation energies before running our calculation, this part was not specified. The transition moments of the two interaction operators are therefore strikingly different. However, if we take the transition moment of 0.358852917641 a.u. in the velocity representation and divide by the excitation energy 0.14917197 a.u., we get 2.4056323560049524 a.u., which is rather close to the transition moment of 2.403791564532 a.u. obtained in the length representation. It is close, but not identical, because we are using *finite* basis sets. The interaction operator in the two representations may have different basis set requirements and this is what we shall investigate by looking at their radial distribution.

As a reference we will use the number density of the atom, but we will scale all quantities such that they integrate to one, in order to facilitate comparison. We start by getting the radial distribution for the number density. The input (dens.inp) reads

```
dens.inp
```

where we scale the density down by the 12 electrons of magnesium. We run the job using:

```
pam --inp=dens --mol=Mg.xyz --put CHECKPOINT.h5 --get "plot.radial.scalar=dens.dat"
```

Next we turn to the transition moment densities. This requires a little bit more thought. Let us have a look at the file PAMXVC of solution vectors. This is an unformatted file, but each entry comes with a label which can be read by the utility program *rspread.x*. Assuming the program is in your path, you read this information by running:

```
rspread.x PAMXVC
```

The output is

```
rspread.snippet
```

Although there are 3 excitations, there are in fact 12 entries. This is because each solution vector is split into contributions E corresponding to e-e rotations, and contributions P corresponding to e-p rotations. Each component is further split into contributions X^+ that are time symmetric and contributions X^- that are time antisymmetric. The transition moments are obtained by contracting solution vectors with property gradients (see Bast2009 for details). A time-(anti)symmetric operator gives a non-zero contribution only by contraction with the time-(anti)symmetric part of the solution vector. The reduced interaction operator in the length and velocity representations is time-symmetric and time-antisymmetric, respectively. With this in mind we set up the following input (length.inp) for generating the radial distribution of the transition moment in the length representation

```
length.inp
```

and run the job using:

```
pam --inp=length --mol=Mg.xyz --put "CHECKPOINT.h5 RESPONSE.h5" --get "plot.radial.scalar=length.dat"
```

Using the same reasoning, the input (velocity.inp) for generating the radial distribution of the transition moment in the velocity representation reads

```
velocity.inp
```

and we run the job using:

```
pam --inp=velocity --mol=Mg.xyz --put "CHECKPOINT.h5 RESPONSE.h5" --get "plot.radial.scalar=velocity.dat"
```

In both cases the densities are scaled such that the transition moments integrate up to one. Note that we have to also include a sign change. For the length representation this is because the keyword *EDIPX* does not include the negative electron charge. For the velocity representation this is because the keyword *JX* asks for the density of the x -component of the current density, which does include the negative electron charge, whereas the interaction operator in the velocity operator does not. In the output of the first (length) calculation we find:

The radial distribution integrates to: 0.98530240054333240

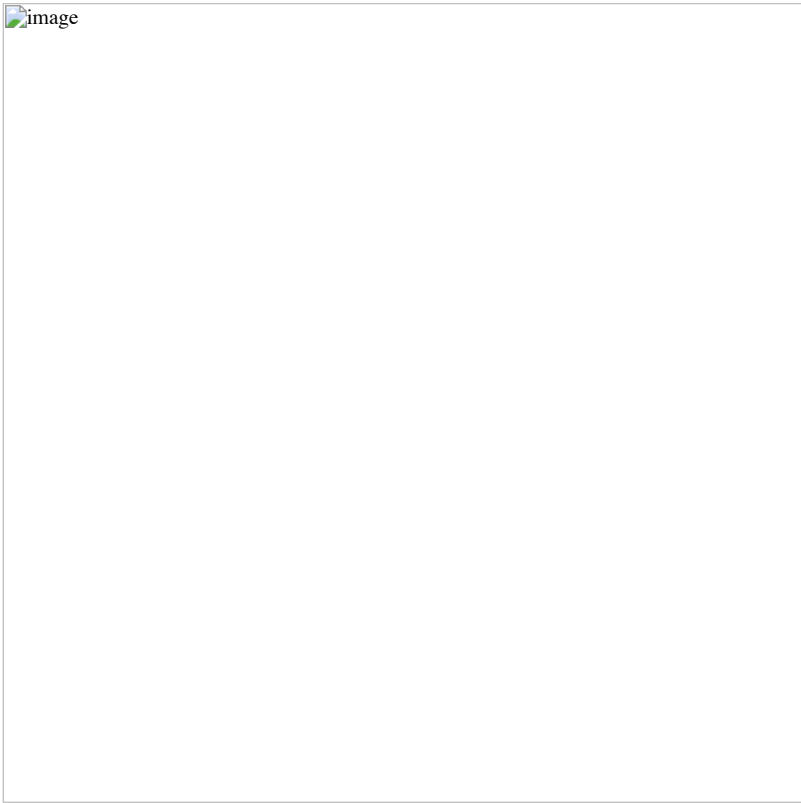
The integration is not very precise since the radial integration uses an even-spaced grid. This is why we have also requested proper 3D integration using the (ultrafine) DFT grid. It shows:

scalar	x-component	y-component	z-component
--------	-------------	-------------	-------------

0.1000000000E+01 0.0000000000E+00 0.0000000000E+00 0.0000000000E+00

showing that the normalization is correct.

We now have three formatted files containing the radial distributions on our specified grid and you may now plot them using your favourite plotting program (I used [xmgrace](#) .)

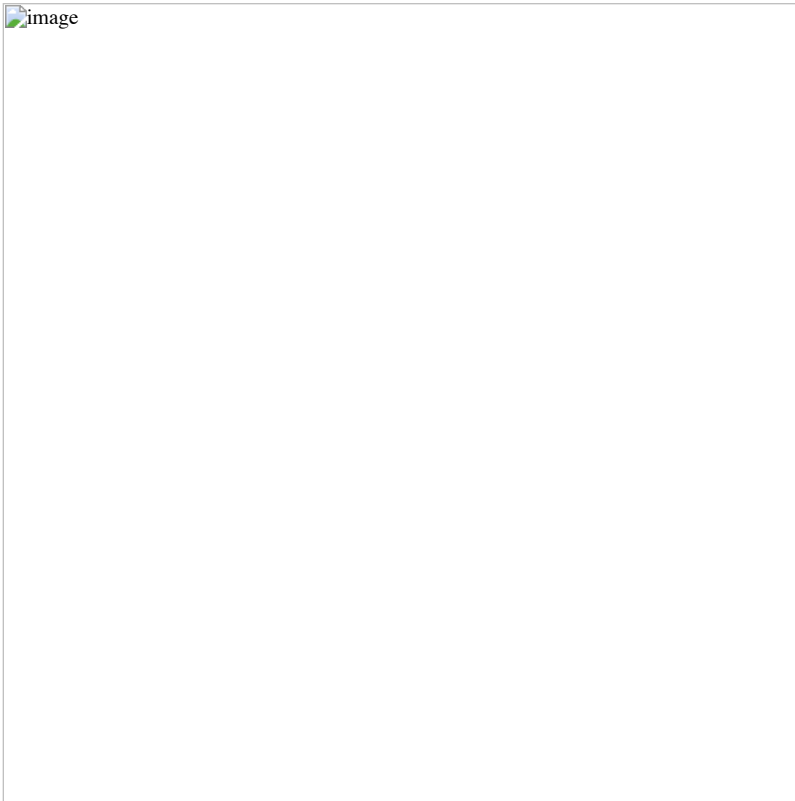


Compared to the number density, the transition moment densities are very diffuse and it is clear that you may need to extend your basis set with diffuse functions in order to properly describe them. We may also note that the transition moment density in the velocity representation is more compact than the one in the length representation; we can in part understand this by the fact that the interaction operator contains a Dirac α matrix which couple large components to the rather compact small components.

Before closing it is interesting to compare the above plot with the one obtained by looking at core excitations $1s \rightarrow 3p$ rather than valence $3s \rightarrow 3p$ ones. We straightforwardly obtain these excitations by specifying what occupied orbitals we may excite from in the excitation energy section, that is:

```
*EXCITATION ENERGIES
.OCCUP
1
[...]
```

This gives the plot below



The transition moment radial distributions are seen to be very much more compact (note the change of axes). We can understand this since the transition moment density now includes a compact core orbital.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/x_ray/BED_Ba/tutorial.md

orphan

X-ray absorption spectroscopy beyond the electric-dipole approximation

Introduction

The validity of the electric dipole approximation comes into question in the X-ray regime, because at these energy scales, the wave length becomes comparable to the spatial extent of the molecule. Therefore, we will assess its validity by comparing the valence and core transitions of the barium atom. In the following, we will use two schemes to simulate effects beyond the electric dipole approximation: truncating the multipole expansion beyond zeroth order and applying the full semi-classical light-matter interaction operator. The expansion of the oscillator strength with respect to the wave vector assumes the form

$$f_{fi} = f_{fi}^{[0]} + f_{fi}^{[2]} + f_{fi}^{[4]} + \cdots,$$

where $f_{fi}^{[0]}$ corresponds to the electric dipole approximation. At arbitrary order, we have

$$f_{fi}^{[2n]} = \frac{2\omega_{fi}}{\hbar} e^2 \sum_{m=0}^n (2 - \delta_{m0}) \epsilon_{p,q} k_{j_1} k_{j_2} \cdots k_{j_{2n}} \text{Re}$$

in which appears the multipole operator

$$\hat{X}_{j_1 \dots j_n; p}^{[n]}(\omega) = \frac{1}{(n+1)!} \hat{Q}_{j_1 \dots j_n; p}^{[n+1]} - \frac{i}{\omega} \frac{1}{n!} \hat{M}_{j_1 \dots j_n}$$

constructed from the electric and magnetic multipole moment. The multipole operator can be expressed in two distinct forms that become equivalent in the complete basis set limit: the length and velocity representation. In the length representation, the multipole moments are represented in their conventional form

$$\begin{aligned} \hat{Q}_{j_1 \dots j_n; p}^{[n+1]} &= -e r_{j_1} \dots r_{j_n} r_p \\ \hat{M}_{j_1 \dots j_n; p}^{[n]} &= \frac{n}{n+1} r_{j_1} \dots r_{j_n} r_p \end{aligned}$$

In the velocity representation, the magnetic multipole remains unchanged, while the electric multipole operator reads

$$\hat{Q}_{j_1 \dots j_n; p}^{[n+1]} = \frac{i}{\omega} r_{j_1} \dots r_{j_n} r_p + \alpha_{p, j_n} r_{j_n} + \alpha_{p, j_{n-1}} r_{j_{n-1}} + \dots + \alpha_{p, j_1} r_{j_1}.$$

A nice feature of this formalism is that the isotropic averaging can be carried out analytically.

For the full interaction, the oscillator strength can be written as

$$f_{fi} = \frac{2\omega_{fi}}{\hbar} e^2 \big| \langle f | \hat{T}(\omega_{fi}) | i \rangle \big|^2; \quad \hat{T}(\omega) = \frac{e}{\omega}$$

This expression contains all corrections from zero to infinity. However, it comes at a somewhat higher computational cost because the integrals depend on the frequency and the isotropic averaging needs to be carried out on a Lebedev grid. For more information, see ref. List_JCP2020.

Valence Transitions

We start of by calculating the valence transitions in the barium atom, using the input valence.inp

valence.inp

the xyz file of the barium atom

Ba.xyz

and the command:

pam --inp=valence.inp --mol=Ba.xyz --mw=700

In this file, the keyword EXCITATION_ENERGIES_.BED activates the calculations done with the full interaction operator. The keyword EXCITATION_ENERGIES_.INTENS is used for the truncated oscillator strenght, where the number below denotes the order. For each boson symmetry we will calculate one excitation. In the electric dipole approximation, this would be rather redundant, because the electric dipole operator only allows excitations that transform as either one of the Cartesian coordinates $\$(B_{\{1u\}}, B_{\{2u\}}, B_{\{3u\}})\$$. The full and truncated oscillator strenghts, allow more transition symmetries. We thus calculate them in this example.

Below you can find the results for the truncated oscillator strenghts:

* Isotropic oscillator strengths (generalized velocity gauge) above threshold :1.00E-08
- [f(0) corresponds to the electric dipole approximation]

Level	Frequency (eV)	Symmetry	f(total)	f(0)	f(2)	f(4)	f(6)	f(8)
6	1.19696	B1u 1u	6.426967E-04	6.426598E-04	3.688481E-08	5.141472E-13	-4.299514E-19	1.801579E-25
				Q-Q	-1.656356E-10	3.127739E-17	-5.019542E-24	7.179734E-31
				M-Q	0.000000E+00	3.705044E-08	-1.988940E-14	5.751024E-21
				M-M	0.000000E+00	0.000000E+00	5.340053E-13	-4.356974E-19
7	1.19696	B2u 0u-	6.426967E-04	6.426598E-04	3.688481E-08	5.141472E-13	-4.299514E-19	1.801579E-25
				Q-Q	-1.656356E-10	3.127739E-17	-5.019542E-24	7.179733E-31
				M-Q	0.000000E+00	3.705044E-08	-1.988940E-14	5.751024E-21
				M-M	0.000000E+00	0.000000E+00	5.340053E-13	-4.356974E-19
8	1.19696	B3u 0u-	6.426967E-04	6.426598E-04	3.688481E-08	5.141472E-13	-4.299514E-19	1.801579E-25
				Q-Q	-1.656356E-10	3.127739E-17	-5.019542E-24	7.179733E-31
				M-Q	0.000000E+00	3.705044E-08	-1.988940E-14	5.751024E-21
				M-M	0.000000E+00	0.000000E+00	5.340053E-13	-4.356974E-19

Sum of oscillator strengths (general length) : 1.92809E-03

* Isotropic oscillator strengths (generalized length gauge) above threshold :1.00E-08
- [f(0) corresponds to the electric dipole approximation]

Level	Frequency (eV)	Symmetry	f(total)	f(0)	f(2)	f(4)	f(6)	f(8)
6	1.19696	B1u 1u	6.330127E-04	6.329761E-04	3.660752E-08	5.143085E-13	-4.300219E-19	1.801837E-25
				Q-Q	-1.627207E-10	2.997773E-17	-4.615767E-24	6.312181E-31
				M-Q	0.000000E+00	3.677024E-08	-1.972681E-14	5.680071E-21
				M-M	0.000000E+00	0.000000E+00	5.340053E-13	-4.356974E-19
7	1.19696	B2u 0u-	6.330127E-04	6.329761E-04	3.660752E-08	5.143085E-13	-4.300219E-19	1.801837E-25
				Q-Q	-1.627207E-10	2.997772E-17	-4.615767E-24	6.312180E-31
				M-Q	0.000000E+00	3.677024E-08	-1.972681E-14	5.680071E-21
				M-M	0.000000E+00	0.000000E+00	5.340053E-13	-4.356974E-19
8	1.19696	B3u 0u-	6.330127E-04	6.329761E-04	3.660752E-08	5.143085E-13	-4.300219E-19	1.801837E-25
				Q-Q	-1.627207E-10	2.997772E-17	-4.615767E-24	6.312180E-31
				M-Q	0.000000E+00	3.677024E-08	-1.972681E-14	5.680071E-21
				M-M	0.000000E+00	0.000000E+00	5.340053E-13	-4.356974E-19

Sum of oscillator strengths (general length) : 1.89904E-03

In the output, you can find in the fourth column the symbols Q-Q, M-Q, and M-M. These refer to the purely electric, mixed electric and magnetic and purely magnetic contribution to the truncated oscillator strenghts. These contributions can be isolated when the multipole operator in the truncated oscillator strength is expanded in terms of its constituent multipole moments. Note that only the dipole allowed transitions are visible. The code did calculate the other transitions, but they fell below the threshold of 10^{-8} and are hence not displayed. Furthermore, the numbers clearly indicate that the multipole expansion is already converged at zeroth order, i.e. the electric dipole approximation. This was to be expected, since it is known that the electric dipole approximation produces good results in the UV-vis regime.

The results from the full interaction confirm the validity of the electric dipole approximation as well: only the dipole allowed transitions were large enough to be printed and their values agree with $f^{\{0\}}$:

* Isotropic oscillator strengths (full light-matter interaction) above threshold :1.00E-08

Level	Frequency (eV)	Symmetry	f(total)
6	1.19696	B1u 1u	6.426967E-04
7	1.19696	B2u 0u-	6.426967E-04
8	1.19696	B3u 0u-	6.426967E-04

Sum of oscillator strengths (full light-matter interaction) : 1.92809E-03

Core Transitions

We will now proceed to calculate the core transitions using the input core.inp

core.inp

the same xyz file and the command:

pam --inp=core.inp --mol=Ba.xyz

Note that this input is nearly identical to the previous one. The main difference is the EXCITATION_ENERGIES_0OCCUP keyword. Once activated, this keyword invokes the restricted-excitation-window approximation. Without this approximation, core excitations can only be calculated by first computing all higher excitations, since the diagonalization routine in DIRAC computes the excitation energies from lowest to highest. The restricted-excitation-window approximation allows us to fix the occupied orbital involved in the excitation. In this example, we set this orbital to the $5s_{1/2}$ orbital, hence calculating the Barium K-edge.

Below you can find the truncated oscillator strengths:

* Isotropic oscillator strengths (generalized velocity gauge) above threshold :1.00E-08
- [f(0) corresponds to the electric dipole approximation]

Level	Frequency (eV)	Symmetry	f(total)	f(0)	f(2)	f(4)	f(6)	f(8)
1	37334.75964	B3g 1g	-3.968737E-06	0.000000E+00	1.745343E-09	-2.506329E-09	1.488143E-08	-3.982858E-06
				Q-Q	1.390509E-21	1.553802E-23	7.343148E-21	-1.065761E-18
				M-Q	0.000000E+00	4.873376E-22	-1.212851E-19	1.136316E-17
				M-M	1.745343E-09	-2.506329E-09	1.488143E-08	-3.982858E-06
2	37334.75964	B2g 1g	-3.968712E-06	0.000000E+00	1.745343E-09	-2.506326E-09	1.488107E-08	-3.982832E-06
				Q-Q	5.887396E-22	2.138116E-22	2.441124E-21	-9.801645E-19
				M-Q	0.000000E+00	1.203034E-21	-2.030584E-19	1.563374E-17
				M-M	1.745343E-09	-2.506326E-09	1.488107E-08	-3.982832E-06
3	37334.75964	B1g 1g	-3.968687E-06	0.000000E+00	1.745343E-09	-2.506324E-09	1.488075E-08	-3.982806E-06
				Q-Q	3.309524E-27	-4.389399E-25	2.141423E-23	1.557558E-22
				M-Q	0.000000E+00	3.374693E-22	2.777935E-20	6.783862E-19
				M-M	1.745343E-09	-2.506324E-09	1.488075E-08	-3.982806E-06
4	37334.76073	Ag 2g	1.262812E-04	0.000000E+00	9.737751E-07	-2.670012E-08	-1.008622E-06	1.263427E-04
				Q-Q	9.737751E-07	-6.744802E-08	-7.083675E-07	3.108814E-05
				M-Q	0.000000E+00	4.074789E-08	-3.006805E-07	9.526082E-05
				M-M	0.000000E+00	0.000000E+00	4.262768E-10	-6.261504E-09
6	37336.26040	B2u 0u-	1.589508E+00	2.163748E-05	1.184342E-06	5.310252E-06	-4.229244E-03	1.593709E+00
				Q-Q	2.163748E-05	-2.459370E-07	8.602505E-07	-2.545372E-02
				M-Q	0.000000E+00	1.430279E-06	4.426366E-06	-4.131899E-03
				M-M	0.000000E+00	0.000000E+00	2.363606E-08	-1.363362E-04
7	37336.26040	B1u 1u	1.589507E+00	2.163748E-05	1.184342E-06	5.310240E-06	-4.229240E-03	1.593708E+00
				Q-Q	2.163748E-05	-2.459370E-07	8.602463E-07	-2.545392E-02
				M-Q	0.000000E+00	1.430279E-06	4.426358E-06	-4.131895E-03
				M-M	0.000000E+00	0.000000E+00	2.363606E-08	-1.363360E-04
8	37336.26040	B3u 0u-	1.589505E+00	2.163748E-05	1.184342E-06	5.310230E-06	-4.229234E-03	1.593706E+00
				Q-Q	2.163748E-05	-2.459370E-07	8.602443E-07	-2.545428E-02
				M-Q	0.000000E+00	1.430280E-06	4.426350E-06	-4.131891E-03
				M-M	0.000000E+00	0.000000E+00	2.363606E-08	-1.363359E-04

Sum of oscillator strengths (general length) : 4.76863E+00

* Isotropic oscillator strengths (generalized length gauge) above threshold :1.00E-08
- [f(0) corresponds to the electric dipole approximation]

Level	Frequency (eV)	Symmetry	f(total)	f(0)	f(2)	f(4)	f(6)	f(8)
1	37334.75964	B3g 1g	-3.968737E-06	0.000000E+00	1.745343E-09	-2.506329E-09	1.488143E-08	-3.982858E-06
				Q-Q	1.310439E-21	-6.202397E-21	1.536998E-18	-1.002294E-16
				M-Q	0.000000E+00	7.330683E-22	-9.185172E-20	9.616797E-18
				M-M	1.745343E-09	-2.506329E-09	1.488143E-08	-3.982858E-06
2	37334.75964	B2g 1g	-3.968712E-06	0.000000E+00	1.745343E-09	-2.506326E-09	1.488107E-08	-3.982832E-06
				Q-Q	5.601652E-22	-2.836271E-21	6.654874E-19	-4.324261E-17
				M-Q	7.722475E-32	1.199779E-21	-1.747560E-19	1.533937E-17
				M-M	1.745343E-09	-2.506326E-09	1.488107E-08	-3.982832E-06
3	37334.75964	B1g 1g	-3.968687E-06	0.000000E+00	1.745343E-09	-2.506324E-09	1.488075E-08	-3.982806E-06
				Q-Q	2.098863E-27	-4.366621E-25	5.233625E-23	-4.489705E-21
				M-Q	0.000000E+00	1.446503E-24	-3.713084E-22	4.904970E-20
				M-M	1.745343E-09	-2.506324E-09	1.488075E-08	-3.982806E-06
4	37334.76073	Ag 2g	-7.052296E-02	0.000000E+00	9.179494E-07	-4.608196E-06	1.103012E-03	-7.162228E-02
				Q-Q	9.179494E-07	-4.647758E-06	1.103403E-03	-7.173915E-02
				M-Q	0.000000E+00	3.956263E-08	-3.907210E-07	1.168808E-04
				M-M	0.000000E+00	0.000000E+00	4.262768E-10	-6.261504E-09
6	37336.26040	B2u 0u-	-5.065597E+02	2.165894E-05	3.399533E-05	-1.473260E-02	3.376557E+00	-5.099215E+02
				Q-Q	2.165894E-05	3.256434E-05	-1.473814E-02	3.381175E+00
				M-Q	0.000000E+00	1.430989E-06	5.512443E-06	-4.617888E-03
				M-M	0.000000E+00	0.000000E+00	2.363606E-08	-1.363362E-04
7	37336.26040	B1u 1u	-5.065597E+02	2.165894E-05	3.399533E-05	-1.473260E-02	3.376557E+00	-5.099216E+02
				Q-Q	2.165894E-05	3.256434E-05	-1.473814E-02	3.381175E+00
				M-Q	0.000000E+00	1.430989E-06	5.512434E-06	-4.617885E-03
				M-M	0.000000E+00	0.000000E+00	2.363606E-08	-1.363360E-04
8	37336.26040	B3u 0u-	-5.065597E+02	2.165894E-05	3.399533E-05	-1.473260E-02	3.376557E+00	-5.099215E+02
				Q-Q	2.165894E-05	3.256434E-05	-1.473814E-02	3.381175E+00
				M-Q	0.000000E+00	1.430989E-06	5.512427E-06	-4.617880E-03
				M-M	0.000000E+00	0.000000E+00	2.363606E-08	-1.363359E-04

Sum of oscillator strengths (general length) : -1.51975E+03

Contrary to previous example, these oscillator strengths do not appear to converge to a certain value. It appears that the magnitude of the oscillator strengths monotonically increases, which implies that the accumulated oscillator strength diverges. In ref. List_JCP2020, it is shown that the oscillator strengths do not diverge, but rather converge extremely slowly.

Due to these artifacts, it is much more preferable to apply the full interaction:

* Isotropic oscillator strengths (full light-matter interaction) above threshold :1.00E-08

Level	Frequency (eV)	Symmetry	f(total)
4	37334.76073	Ag 2g	9.407196E-07
6	37336.26040	B2u 0u-	2.280511E-05
7	37336.26040	B1u 1u	2.280511E-05
8	37336.26040	B3u 0u-	2.280511E-05

Sum of oscillator strengths (full light-matter interaction) : 6.93566E-05

The numbers calculated with this method do not suffer from aforementioned problems. However, if we compare the full oscillator strengths with the lowest non-zero truncated oscillator strength, we find that the two merely differ by 5%. This small difference can be understood by the compactness of the $1s_{1/2}$ orbital. The electric dipole approximation breaks down if the limit $\lambda \ll r$ holds. Even though the wave length is short in this example, the spatial extent of the $1s_{1/2}$ orbital is small, hence reducing non-dipolar effects. Therefore, compared to the electric dipole approximation, the full interaction gives modest corrections to dipole-allowed transitions, but it is essential to include dipole-forbidden transitions. However, it remains an open question whether non-dipolar effects play a more important role in extended systems. I leave it to you to find it out.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/x_ray/CO_N2_IP1s/tutorial.md

orphan

Core ionization in the CO and N2 molecules

Introduction

We would like to study K-edge energies of CO and N2 molecules.

Carbon monoxide

A first approximation to the O1s and C1s ionization energies is provided by Koopmans' theorem. Starting from the molecular input file CO.mol

CO.mol

and the menu file CO.inp

CO.inp

we first run a Hartree-Fock calculation of the ground electronic state of carbon monoxide:

```
pam --inp=CO --mol=CO --get "DFC0EF=cf.CO"
```

We find that $\epsilon_{\text{O1s}} = 20.6838 \text{ eV}$ and $\epsilon_{\text{C1s}} = 11.3672 \text{ eV}$. This is significantly off the values of **542.1 eV** and **295.9 eV**, respectively, reported by Kai Siegbahn and co-workers Siegbahn1969.

Koopmans' theorem work quite well for valence excitations because the two major sources, that is, i) lack of electron correlation and ii) lack of orbital relaxation, typically are of about the same magnitude, but of opposite sign and hence tend to cancel out. This is not the case for core ionization, where orbital relaxation is very much more important than electron correlation. To introduce orbital relaxation of the core-ionized state we carry out a ΔSCF calculation, as pioneered by Paul Bagus and Henry F. Schaefer Bagus_JCP1971.

In order to calculate the O1s ionized system we set up the menu file CO_O1s.inp

CO_O1s.inp

and use the command:

```
pam --inp=CO_O1s --mol=CO --put "cf.CO=DFC0EF"
```

It is seen that we start from the coefficients of the neutral CO molecule. We then converge to the core-ionized system using reordering and overlap selection: In the input we have specified twelve electrons in six inactive (closed) orbitals, followed by a single electron in an active (open) orbital. We want the O1s to be the active orbital, but this is not achieved automatically since DIRAC will normally order orbitals according to their energy. We therefore start by reordering the orbitals such that the O1s orbital from the previous calculation on the neutral system comes out on top of the occupied orbitals. However, this is not enough to converge to the desired state since after the first diagonalization DIRAC will again by default order orbitals according to their energy. This is why we use *overlap selection*, that is, we ask DIRAC to rather order orbitals according to their overlap with some reference orbitals. By default (dynamic overlap selection) this will be the orbitals from the previous iteration. However, in this case we activate *non-dynamic* overlap selection, which means that we order orbitals according to their overlap with the starting orbitals.

The analogous menu file for C1s ionization is CO_C1s.inp

CO_C1s.inp

which puts the C1s on top of the occupied orbitals, and then use the command:

```
pam --inp=CO_C1s --mol=CO --put "cf.CO=DFC0EF"
```

The HF energies of the neutral, the O1s-ionized and the C1s-ionized systems are -112.857912 eV , -92.939954 eV and -101.933919 eV , respectively, from which we calculate $\text{IP}(\text{O1s}) = 19.9180 \text{ eV}$ and $\text{IP}(\text{C1s}) = 10.9240 \text{ eV}$, which agrees remarkably well with the experimental values, in particular for the oxygen K-edge.

[!NOTE] Overlap selection is nowadays marketed hard as MOM (Maximum Orbital Method, see Gilbert_JPCA2008), but this method has been included in DIRAC for at least two decades and goes back to the pioneering work of [Paul Bagus](#). It was used in Bagus_JCP1971, but not reported explicitly. However, it is for instance documented in the [1975 manual of the ALCHEMY program](#) (On pdf page 15 you find a description of keyword MOORDR using a “maximum overlap criterion”).

Nitrogen molecule

We now switch to the isoelectronic nitrogen molecule. Starting from the molecular input file N2.mol

N2.mol

and the menu file N2.inp

N2.inp

we run a Hartree-Fock calculation of the ground electronic state of the nitrogen molecule:

```
pam --inp=N2 --mol=N2 --get "DFC0EF=cf.N2 DFACM0=ac.N2"
```

(note that we are also getting the $C_{1'}$ coefficients; we shall come back to that). We find that $\epsilon(1\sigma_g)=15.6942$ eV and $\epsilon(1\sigma_u)=15.6907$ eV. This is significantly off the value of **409.9 eV** reported by Kai Siegbahn and co-workers Siegbahn1969; the $1\sigma_g$ and $1\sigma_u$ are indistinguishable in the X-ray PES study (see also Wight_JEPRP1972).

We therefore proceed to Δ SCF calculations: for the $1\sigma_g$ ionization we use the menu file N2_Kg.inp

N2_Kg.inp

and the command:

```
pam --inp=N2_Kg --mol=N2 --put "cf.N2=DFC0EF"
```

and equivalent menu file N2_Ku.inp for the $1\sigma_u$ ionization:

N2_Ku.inp

The HF energies of the neutral, the $1\sigma_g$ -ionized and the $1\sigma_u$ -ionized systems are -109.051015 E_h , -93.626378 E_h and -93.630989 E_h , respectively, from which we calculate $IP(1\sigma_g) = 15.4246 E_h = 419.7$ eV and $IP(1\sigma_u) = 15.4200 E_h = 419.6$ eV. However, *this is still far from the experimental value of 409.9 eV!*

Bagus and Schaefer found a similar discrepancy in their study of the O_2^+ molecule Bagus_JCP1972 and found that results improved dramatically by *localization* of the core hole, so let us try this. Localization breaks molecular symmetry, so we go down to $C_{1'}$ using the molecule input file N2_C1.mol

N2_C1.mol

This is the reason why we saved the $C_{1'}$ coefficients in the initial calculation! We next set up the menu file N2loc.inp

N2loc.inp

for Pipek-Mezey localization. Note that we select only the a little trick: We select only the $1\sigma_g$ and $1\sigma_u$ orbitals for localization. We now localize using the command:

```
pam --inp=N2loc --mol=N2_C1 --put "ac.N2=DFC0EF" --get "DFC0EF=ac.N2loc"
```

We calculate the system with a localized core hole using the menu file N2_K.inp

N2_K.inp

and the command:

```
pam --inp=N2_K --mol=N2_C1 --put "ac.N2loc=DFC0EF"
```

In fact, we can do slightly better because we can go up to C_{2v} symmetry using that DIRAC allows the import of coefficients in $C_{1'}$ symmetry from the unformatted file DFACMO to current symmetry. This assumes that the current symmetry is lower than the symmetry used for obtaining the original coefficients, which is the case here. We now calculate the system with a localized core hole using the menu file N2_KC2v.inp

N2_KC2v.inp

the molecule input file N2_C2v.mol

N2_C2v.mol

and the command:

```
pam --inp=N2_KC2v --mol=N2_C2v --put "ac.N2loc=DFACM0"
```

The calculated energy of the core ionized system is -93.971190 E_h and we now obtain an ionization energy of 15.0978 $E_h = 410.3$ eV, in much better agreement with experiment. The price we had to pay was to break symmetry. It is possible to keep symmetry, but now at the price of going to more complex wave functions, as shown by Agren, Bagus and Roos Agren_CPL1981.

FILE: www.diracprogram.org/doc/release-26/_sources/tutorials/x_ray/water_Kedge/tutorial.md

orphan

Core electron excitations and ionization in H2O at the HF and DFT levels

Introduction

We want to study excitation of the oxygen 1s electron of water.

Restricted excitation window TD-HF (REW-TD-HF)

Starting from the molecular input file H2O.mol

H2O.mol

and the menu file H2O.inp

H2O.inp

we first run a Hartree-Fock calculation of the ground electronic state:

```
pam --mol=H2O --inp=H2O --outcmo
```

From Mulliken population analysis we confirm that the oxygen 1s orbital is the first orbital:

```
* Electronic eigenvalue no. 1: -20.581089118212      (Occupation : f = 1.0000)
=====
* Gross populations greater than 0.00010
Gross      Total |      L A1 0  s
-----
alpha      1.0000 |      1.0000
beta       0.0000 |      0.0000
```

We now focus on electric dipole allowed transitions. The molecular symmetry is C_{2v} where the components of the electric dipole operator $-e(x,y,z)$ span irreps (B_1, B_2, A_1) . This leads us to set up the following input

H2O_O1s.inp

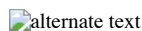
We run our calculation:

```
pam --inp=H2O_O1s --mol=H2O --incmo
```

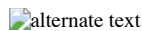
and find the isotropically averaged oscillator strenghts

H2O_1s_spectrum

where we have added by hand the most important virtual orbitals. For reference, orbital densities of the fifteen first canonical HF orbitals of water are given below

 alternate text

We may simulating the spectrum using a Lorentzian lineshape with half-width at half-maximum (HWHM) equal to 0.005 a.u.

 alternate text

Complex response

We can alternatively obtain the above spectrum from complex response. The isotropically averaged ocillator strength associated within the electric dipole approximation is related to the isotropic electric dipole polarizability through the relation

$$f^{\text{iso}}(\omega) = \frac{2m\omega}{\pi e^2} \text{Im}[\alpha^{\text{iso}}(\omega + i\gamma)]$$

where γ is a damping parameter corresponding to the half-width at half-maximum (HWHM) of the Lorentzian lineshape.

We can calculate the real and imaginary parts of the polarizability by complex response. By choosing a frequency window we can directly access the region of the spectrum of interest.

We use the input file

cpp.inp

and the command:

```
pam --incmo --mol=H2O --inp=cpp
```

To get enough data points we do a second run using:

```
.FREQ INTERVAL
20.01 21.0 0.02
```

Extracting the imaginary part of the frequency-dependent electric dipole polarizability and converting to oscillator strength we can directly plot the spectrum as below

 alternate text
alternate text

In the above graph we have also included the results from REW-TDDFT and they completely overlap, except that a keen eyemay note that REW-TDDFT is missing the peak around 570 eV, simply because not enough excitations were specified in the input.

Localizing the K edge

An interesting question is how to localize the K edge in the XANES spectrum. A first approximation to the 1s ionization energy is provided by Koopmans' theorem. We find $IP_{1s} \approx -\epsilon_{1s} = 20.581089 \text{ E}_h = 560.04 \text{ eV}$. This is significantly off the value 539.7 eV reported by Kai Siegbahn and co-workers [*ESCA applied to free molecules* (1969)] Siegbahn1969 (see also [here](#) , but here the work function of the reference metal must be subtracted). reported by Kai Siegbahn and co-workers in 1977(?). We know that Koopman's theorem ignores correlation and orbital relaxation. For valence ionization Koopman's theorem often provides a reasonable approximation since the errors tend to cancel each other. For core excitations orbital relaxation dominates such that Koopman's theorem greatly overestimates ionization energies.

To see this we carry out a average-of-configuration (AOC) calculation of the 1s core-ionized system. Starting from the coefficients from the neutral system and the input H2O_1s.inp

and the command:

```
pam --incmo --mol=H2O --inp=H2O_1s
```

we find a total energy of -56.287847 E_h for the core ionized system compared to -76.115149 E_h for the neutral system. This corresponds to a ΔSCF value of 539.52 eV for the ionization energy, tantalizingly close to experiment.

In passing we note that in the first iteration of this calculation we obtain a total energy of -55.534060 E_h . This is the energy of the core-ionized system obtained using the orbitals of the neutral system. Koopman's theorem is obtained by subtracting this energy from the energy of the neutral system. For the neutral system we obtained -76.115149 E_h , so by taking the difference we obtain 20.581089 E_h , which is exactly the 1s orbital energy in the neutral system.

You should also note that we easily converge to the core-ionized system using reordering and overlap selection: In the input we have specified eight electrons in four inactive (closed) orbitals, followed by a single electron in an active (open) orbital. We want the O1s to be the active orbital, but this is not achieved automatically since DIRAC will normally order orbitals according to their energy. We therefore start by reordering the orbitals such that the O1s orbital from the previous calculation on the neutral system comes out on top of the occupied orbitals. However, this is not enough to converge to the desired state since after the first diagonalization DIRAC will again by default order orbitals according to their energy. This is why we use *overlap selection*, that is, we ask DIRAC to rather order orbitals according to their overlap with some reference orbitals. By default (dynamic overlap selection) this will be the orbitals from the previous iteration. However, in this case we activate *non-dynamic* overlap selection, which means that we order orbitals according to their overlap with the starting orbitals.

[!NOTE] Overlap selection is nowadays marketed hard as MOM (Maximum Orbital Method, see Gilbert_JPCA2008), but this method has been included in DIRAC for at least two decades and goes back to the pioneering work of [Paul Bagus](#). It was used in Bagus_JCP1971, but not reported explicitly. However, it is for instance documented in the 1970 manual of the ALCHEMY program ALCHEMY1970 (in French ! On pdf page 9 you find a description of keyword MOORDR using a "maximum overlap criterion").

The K edge obtained by ΔSCF does not correspond to that of our TD-HF or complex response calculations since they only allow linear response (orbital relaxation).

In the figure below we have plotted oscillator strength per atom for 1s core excitation spectrum for water, taken from the [Gas Phase Core Excitation Database](#) and recorded at 0.7 eV fwhm.

 alternate text
alternate text

The vertical orange line corresponds to the O1s binding energy reported by Siegbahn and co-workers and seems to be too early. We have also plotted the spectrum obtained by REW-TDHF with a Lorentzian width corresponding to the fwhm of the experiment. In green we plot the original spectrum, whereas in red it has been shifted so that our linear response estimate for the ionization energy has been aligned with the experimental O1s binding energy. We can see that the shift improves agreement, but the spacing and relative intensities of peaks do not agree with experiment.

Static Exchange Approximation

In order to incorporate orbital relaxation we carry out a STEx calculation. At the moment symmetry is not implemented, so we turn off symmetry in the molecular input file

H2O_C1.mol

We then first run the ground state:

```
pam --inp=H2O --mol=H2O_C1 --outcmo
```

followed by the 1s core-ionized state:

```
pam --mol=H2O_C1 --inp=H2O_1s --incmo
```

We now run STEx using the input

stex.inp

and the command:

```
pam --inp=stex --mol=H2O_C1 --incmo --put "H2O_1s_H2O_C1,h5=ION.h5"
```

We have plotted the STEx spectrum below together with the experimental one

 alternate text

alternate text

Switching to DFT

Let us now look at what we can do with DFT. The first thing to note is that Koopman's theorem does not hold:

		HF(AOC)	HF(focc)	LDA	PBE	PBE0
Neutral	Energy	-76.115149	-76.115149	-75.962033	-76.438943	-76.437412
	ϵ_{1s}	-20.581089	-20.581089	-18.620721	-18.766629	-19.223080
$1s^{-1}$	Energy	-56.287847	-55.070646	-55.980781	-56.300366	-56.069079
	Energy(0)	-55.534060	-54.347068	-55.231846	-55.540835	-55.319556
ΔE	relax	-19.827303	-21.044503	-19.981252	-20.138577	-20.368333
	norelax	-20.581089	-21.768082	-20.730186	-20.898108	-21.117856
		BP86	BLYP	B3LYP	CAMB3LYP	
Neutral	Energy	-76.525093	-76.508410	-76.487092	-76.496078	
	ϵ_{1s}	-18.786866	-18.791935	-19.145210	-19.219699	
$1s^{-1}$	Energy	-56.366800	-56.340103	-56.148003	-56.115449	
	Energy(0)	-55.606191	-55.582410	-55.398934	-55.366886	
ΔE	relax	-20.158293	-20.168307	-20.339089	-20.380629	
	norelax	-20.918902	-20.926000	-21.088158	-21.129191	

In the above table the entry Energy(0) is the total energy of the core-ionized system calculated using the orbitals of the neutral system. When we calculate the energy difference between the neutral and core-ionized system using the orbitals of the neutral system we reproduce the $1s$ orbital energy to the cited decimals, but this is not the case of any other method, including HF using fractional occupation.

Below we give the Δ SCF numbers in eV. Interestingly AOC-HF at 539.5 eV easily comes closest to the experimental value 539.7 eV. It can furthermore be seen that Δ SCF values obtained with DFT functionals show the trend LDA < GGA < hybrid with LDA closest, but still far from experiment. Switching from average-of-configuration to fractional occupation at the HF level leads to a dramatic deterioration of the agreement with experiment.

	HF(AOC)	HF(focc)	LDA	PBE	PBE0	BP86	BLYP	B3LYP
ϵ_{1s}	560.0	560.0	506.7	510.7	523.1	511.2	511.4	521.0
ΔE (relax)	539.5	572.7	564.1	568.7	574.6	569.2	569.4	573.8
ΔE (no relax)	560.0	592.3	543.7	548.0	554.3	548.5	548.8	553.5

At the DFT level we have employed an energy expression based on fractional occupation, which is the model that leads to Janak's theorem, namely that the derivative of the energy with respect to occupation number n_i gives the energy of the corresponding orbital, that is

$$\frac{dE}{dn_i} = \epsilon_i$$

We may investigate Janak's theorem numerically. We set up a script to do DFT calculations with fractional occupation from 1.0 to 1.9 of the oxygen $1s$ orbital

janak.sh

and also add the energy of the neutral system to our data set. We then carry out polynomial fits to various orders and calculate the derivative of the energy at occupation 2.0 with respect to $1s$ occupation number. We then obtain

	HF(AOC)	HF(focc)	LDA	PBE	PBE0	BP86	BLYP	B3LYP	CA
1	-19.824314	-21.044193	-19.981185	-20.138613	-20.368425	-20.158286	-20.168248	-20.339084	-20.380629
2	-18.196672	-20.579045	-18.618588	-18.765178	-19.222619	-18.785047	-18.789831	-19.143977	-19.218629
3	-18.220409	-20.581507	-18.619168	-18.764949	-19.221929	-18.785159	-18.790354	-19.144055	-19.218529
4	-18.220112	-20.581073	-18.620943	-18.766866	-19.223251	-18.787109	-18.792168	-19.145388	-19.219819
5	-18.220135	-20.581093	-18.620699	-18.766601	-19.223061	-18.786846	-18.791909	-19.145191	-19.219689
6	-18.220131	-20.581090	-18.620724	-18.766633	-19.223084	-18.786872	-18.791940	-19.145213	-19.219709
7	-18.220130	-20.581089	-18.620720	-18.766628	-19.223080	-18.786869	-18.791934	-19.145209	-19.219699
8	-18.220130	-20.581089	-18.620721	-18.766629	-19.223080	-18.786857	-18.791935	-19.145210	-19.219699
9	-18.220130	-20.581089	-18.620721	-18.766629	-19.223080	-18.786879	-18.791935	-19.145210	-19.219699
ϵ_{1s}	-20.581089	-20.581089	-18.620721	-18.766629	-19.223080	-18.786866	-18.791935	-19.145210	-19.219699

We see that a linear fit is clearly insufficient, whereas a quadratic fit is reasonable. However, an 8th order fit is in general needed to converge the energy derivative to the $1s$ orbital energy with the cited number of decimals.

FILE: www.diracprogram.org/doc/release-26/_sources/workshop-slides/first_steps.md

orphan

DIRAC Workshop on Relativistic Quantum Chemistry

Specifying electronic states with single determinant

Hartree-Fock SCF in DIRAC : Hamiltonians

- specify one-component, two-component (X2C) or four-component Hamiltonian
- decide whether neglect spin-orbital effects
- if X2C, you can neglect the AMFI 2-electron correction
- in the case of 4-comp. Hamiltonian specify integral classes

Hartree-Fock SCF in DIRAC : Electronic occupation

- either use the automatic occupation or set the electronic occupation manually
- recommended (but not perfect) approach: start with automatic occupation
- after switch to manual electronic occupation
- always add orbital analysis to check resulting molecular spinors

Getting SCF convergence in DIRAC

- first empty valence shell(s) and see
- atomic start SCF as the best method so far
- for some systems force keeping order of MOs

FILE: www.diracprogram.org/doc/release-26/_sources/workshop-slides/index.md

orphan

Relativistic Quantum Chemistry with DIRAC

Introduction to DIRAC tutorials

Goals of the workshop

- providing some relativistic theory
- main focus on practical examples with DIRAC
- performing several quick tutorial calculations
- analysing and discussing obtained results

Why the DIRAC software

- can demonstrate relativistic effects on manifold of properties
- offering extensive set of explanatory web-tutorials
- continuously developing the code and updating the manual/tutorials
- active discussion group for users

FILE: www.diracprogram.org/doc/release-26/_sources/zreferences.md

orphan

References

List of published papers and other sources (books, theses) documenting the development of various features of DIRAC, some interesting applications and various related references.

Reference entries are sorted alphabetically according to the first author's family name with the year of the publishing. If the first author has several publications in the same year, characters in alphabetical order - "a", "b", "c", etc. - are added after the year.

Please always provide each citation entry with its permanent URL-link. The best web-link is the DOI web identity, used almost for all peer-reviewed papers. For books we recommend to resort to semi-permanent links, like the publisher's web-site, or the popular Google-Books web-space.

References availability

All references are also available for download <references.bib> as one (big) file in the popular BibTex format. Please keep them in the alphabetical order as described above.